

THÈSE DE DOCTORAT

Soutenue à AMU — Aix-Marseille Université

le 8 juillet 2026 par

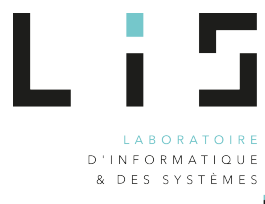
Marius ROLLAND

Des unrolls aux systèmes dynamiques totaux et partiels :
algorithmes de résolution et énumérations des (in)équations polynomiales

Discipline
Informatique

École doctorale
ED 184 Mathématiques et Informatique

Laboratoire/Partenaires de recherche
Laboratoire d'informatique et des systèmes(LIS),
Centre national de la recherche Scientifique (CNRS)



Composition du jury

Thi Ha Duong PHAN Rapporteuse
Associate Professor, Institut
de Mathématiques de Hanoi

Stefan HAAR Rapporteur
Directeur de recherche, INRIA
(Paris-Saclay)

Nadia CREIGNOU Présidente du jury
Professeure des universités,
Univ. Aix-Marseille

Christophe CRESPELLE Examineur
Professeur des universités,
Univ. Côte d'Azur

Kévin PERROT Directeur de thèse
Maître de conférences, Univ.
Aix-Marseille

Antonio Enrico PORRECA Co-Directeur de thèse
Maître de conférences, Univ.
Aix-Marseille

Maximilien GADOULEAU Membre invité
Associate Professor, Durham
University

Affidavit

Je soussigné, Marius ROLLAND, déclare par la présente que le travail présenté dans ce manuscrit est mon propre travail, réalisé sous la direction scientifique de Kévin PERROT et Antonio Enrico PORRECA dans le respect des principes d'honnêteté, d'intégrité et de responsabilité inhérents à la mission de recherche. Les travaux de recherche et la rédaction de ce manuscrit ont été réalisés dans le respect à la fois de la charte nationale de déontologie des métiers de la recherche et de la charte AMU relative à la lutte contre le plagiat.

Ce travail n'a pas été précédemment soumis en France ou à l'étranger dans une version identique ou similaire à un organisme examinateur.

Fait à Santiago du Chili le 27 mars 2026



Cette œuvre est mise à disposition selon les termes de la [Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 4.0 International](#).

Liste de publications et participations aux conférences

Liste des publications réalisés dans le cadre du projet de thèse :

1. Roots in the semiring of finite deterministic dynamical systems, François Doré, Kevin Perrot, Antonio E. Porreca, Sara Riva et Marius Rolland, AUTOMATA2024, 2024
2. Injectivity of polynomials over finite discrete dynamical systems, Antonio E. Porreca et Marius Rolland, CiE2025, 2025
3. Solving “pseudo-injective” polynomial equations over finite dynamical systems, Antonio E. Porreca et Marius Rolland, AUTOMATA2025, 2025
4. Infinite trees for division and roots over finite discrete-time dynamical systems, François Doré, Kevin Perrot, Antonio E. Porreca, Sara Riva et Marius Rolland, Natural Computing, 2026
5. Majority Boolean networks classifying density : structural characterization and complexity, Kévin Perrot, Marius Rolland, LATIN2026, 2026
6. Injective and pseudo-injective polynomial equations : From permutations to dynamical systems, Antonio E. Porreca, Marius Rolland, submitted to Natural Computing
7. Non-trivial automata networks do exist that solve the global majority problem with the local majority rule, Pedro Paulo Balbi, Kévin Perrot, Marius Rolland, Eurico Ruivo, submitted to TCS
8. Solving polynomial equations over partial discrete dynamical systems, Maximilien Gadouleau and Sara Riva and Marius Rolland and Marie-Emile Voge, submitted to journal of graph theory

Participation aux conférences et écoles d'été au cours de la période de thèse :

1. Cellular Automata and Discrete Complex Systems ,2025 ,AUTOMATA, Lille, France
2. Computability in Europe, 2025, CiE, Lisbonne, Portugal

3. Latin American Theoretical Informatics Symposium, 2026, LATIN, Florianopolis, Brésil
4. Thematic Semester "GRAPHS AND BEYOND", 2026, Hanoi, Vietnam

Résumé et mots-clés

Un *système dynamique* fini, déterministe et à temps discret consiste en un couple (A, f_A) où A est un ensemble fini d'états et $f_A : A \rightarrow A$ est une fonction totale de A dans A , appelée *fonction de transition*. Cette dernière définit l'évolution du système au cours du temps. La dynamique correspondante est décrite de manière explicite par son graphe de transition, qui a les états du système pour sommets et, pour chaque état $a \in A$, un arc $(a, f_A(a))$ le reliant à l'état suivant. La forme générale de la dynamique d'un tel système consiste en un ou plusieurs *cycles limites* d'états périodiques (par exemple, $(a, f_A(a), f_A^2(a), \dots, f_A^n(a) = a)$ est un cycle limite de période n), ainsi qu'en un régime transitoire d'états non périodiques en forme d'arbres, dont la racine appartient à l'un des états d'un cycle limite.

Ces systèmes, pris à isomorphisme près, sont appelés FDDS et sont particulièrement adaptés à la démarche scientifique. En effet, ils sont non seulement largement utilisés pour vérifier qu'une spécification possède certaines propriétés attendues ou requises, mais aussi pour modéliser et analyser des phénomènes naturels, notamment en physique et en biologie. Il est également possible de décrire des modèles de calcul déterministes usuels en informatique, tels que les automates cellulaires ou les machines de Turing à ruban borné, sous la forme de systèmes dynamiques finis.

Dans la littérature, il est connu que les FDDS, équipés d'opérations de somme (union disjointe) et de produit (produit direct), forment un semi-anneau commutatif. Nous pouvons donc définir un semi-anneau de polynômes, ainsi que des équations polynomiales dont chaque coefficient et chaque variable sont des FDDS. Ce type d'équation est indécidable dans le cas général, mais appartient à la classe NP lorsque l'un des côtés de l'équation est constant. La résolution des équations de la forme $P(X_1, \dots, X_n) = B$ permet alors de déterminer si B respecte certaines contraintes structurelles, mais aussi d'exhiber une dynamique sous-jacente.

Dans la présente thèse, nous nous concentrons essentiellement sur le cas où P ne contient qu'une unique variable. Nous fournissons un algorithme polynomial permettant de déterminer les comportements transitoires de toutes les solutions possibles de l'équation $P(X) = B$. Est ensuite proposée une méthode pour résoudre en temps polynomial les équations $P(X) = B$ lorsque P satisfait une condition structurelle (injectivité, « pseudo-injectivité »). Nous nous inspirons de cette approche pour mettre en évidence deux paramètres — le nombre de composantes connexes non isomorphes de B et la plus petite longueur de cycle d'un coefficient non constant de P — qui, lorsqu'ils sont fixés, permettent la résolution de $AX = B$ en temps polynomial. Cela nous amène à étudier des sur- et sous-approximations de B . En d'autres termes, nous introduisons des algorithmes polynomiaux pour résoudre les inéquations $P(X) \leq B$ et $P(X) \geq B$. Nous établissons également la possibilité d'énumérer l'ensemble des

solutions de $P(X) \leq B$ avec un délai polynomial entre deux solutions. Enfin, nous généralisons certains de nos résultats au cas où les systèmes ne sont plus décrits par des fonctions totales, mais par des fonctions partielles.

Mots-clés : Systèmes dynamiques, Graphes, Semi-anneaux, Complexité algorithmique

Abstract and keywords

A finite, deterministic, discrete-time *dynamical system* consists of a pair (A, f_A) where A is a finite set of states and $f_A : A \rightarrow A$ is a total function from A to itself, called the *transition function*. The latter defines the evolution of the system over time. These dynamics are described explicitly by its transition graph, whose vertices are the states of the system and which contains, for each state $a \in A$, an arc $(a, f_A(a))$ linking it to the next state. The general form of the dynamics of such a system consists of one or more *limit cycles* of periodic states (for example, $(a, f_A(a), f_A^2(a), \dots, f_A^n(a) = a)$ is a limit cycle of period n), together with a transient regime of non-periodic states arranged in tree, whose root belongs to one of the states of a limit cycle.

These systems, considered up to isomorphism, are called FDDSs and are particularly well suited to the scientific approach. Indeed, they are not only widely used to verify that a specification satisfies certain expected or required properties, but also to model and analyze natural phenomena, notably in physics and biology. It is also possible to describe standard deterministic models of computation in computer science, such as cellular automata or bounded-tape Turing machines, in the form of these systems.

In the literature, it is known that FDDSs, equipped with sum (disjoint union) and product (direct product) operations, form a commutative semiring. We can therefore define a semiring of polynomials, as well as polynomial equations whose coefficients and variables are FDDSs. This type of equation is undecidable in the general case, but belongs to the class NP when one side of the equation is constant. Solving equations of the form $P(X_1, \dots, X_n) = B$ then makes it possible to determine whether B satisfies certain structural constraints, but also to exhibit an underlying dynamics.

In this thesis, we focus essentially on the case where P contains only a single variable. We provide a polynomial-time algorithm for determining the transient behaviors of all possible solutions of the equation $P(X) = B$. We then give a method for solving in polynomial time the equations $P(X) = B$ when P satisfies a structural condition (injectivity, “pseudo-injectivity”). We draw inspiration from this approach to highlight two parameters—the number of non-isomorphic connected components of B and the smallest cycle length of a non-constant coefficient of P —which, when fixed, allow the equation $AX = B$ to be solved in polynomial time. This leads us to study over- and under-approximations of B . In other words, we introduce polynomial-time algorithms for solving the inequalities $P(X) \leq B$ and $P(X) \geq B$. We also show that it is possible to enumerate the set of solutions of $P(X) \leq B$ with polynomial delay between two solutions. Finally, we generalize some of our results to the case where systems are no longer described by total functions, but by partial functions.

Keywords: Dynamical systems, Graphs, Semirings, Computational complexity

Remerciements

Il est des thèses qui se font seules. La mienne n'a, je crois, pas été de celles-là. Elle a été traversée de rencontres, portée par des soutiens, animée par des discussions sans fin, et elle porte en elle l'empreinte de toutes les personnes que je vais tenter de remercier ici, la trace de tous les échanges que nous avons partagés. Je sais que ces quelques mots ne seront jamais à la hauteur de ce que je leur dois, mais je me dois de leur rendre hommage.

Mes premiers remerciements vont à mes deux directeurs de thèse, Kévin et Enrico.

Kévin, travailler avec toi a toujours été un vrai plaisir. Ton calme, surtout dans les moments où le mien me faisait cruellement défaut, ta bienveillance indéfectible et ta capacité à toujours trouver le mot juste ont été un soutien dont je mesure aujourd'hui toute la valeur. Tu as ce don rare de poser la question qui semble anodine en apparence, mais qui demande des jours et des jours de réflexion pour esquisser ne serait-ce qu'une piste de réponse. C'est précisément ce qui rend la recherche si stimulante. Et j'espère un jour pouvoir être un aussi bon encadrant que tu le fus pour moi.

Enrico, tu as été bien plus qu'un directeur pour moi. Tu fus un véritable camarade d'aventure. C'est toi qui es à l'origine de ce sujet, et tu m'as accompagné à chaque étape du chemin avec une générosité et un enthousiasme qui ne se sont jamais démentis. Comme Kévin, tu as su me laisser suffisamment d'autonomie pour explorer librement, tout en me recadrant avec douceur lorsque je m'aventurais dans des directions trop impromptues. Tu m'as aidé à améliorer ma rédaction et mon anglais, en me rappelant patiemment que ce qui me semblait évident ne l'était pas nécessairement pour le lecteur. Nos réunions, officiellement d'une heure, duraient régulièrement une demi-journée, voire une journée entière. Elles étaient toujours un vrai bonheur, même si les échanges pouvaient être animés. Sans toi, cette thèse n'aurait tout simplement pas existé.

Tout cela pour dire, merci à vous deux de m'avoir offert cette chance. Vous avez été les meilleurs encadrants dont on puisse rêver. J'espère pouvoir continuer à collaborer avec vous encore longtemps. Et je souhaite un jour être capable de donner à une doctorante ou un doctorant tout ce que vous m'avez apporté.

Je souhaite également adresser un remerciement tout particulier à Sara, qui a contribué à presque l'ensemble des travaux de ma thèse. *Sara, questa tesi non sarebbe stata la stessa senza di te. Come hai detto più volte, questa tesi è una continuazione, quasi un secondo volume, della tua, e non avrei potuto dirlo meglio. Hai aperto la strada; la tua tesi è servita da base per la mia, e lo hai fatto in modo brillante. Lavorare e scambiare idee con te è stata una vera benedizione e sempre un grande piacere. Hai contribuito a quasi ogni capitolo di questa tesi, individuando errori, suggerendo dimostrazioni alternative e aiutandomi a semplificare, un piccolo, alcune dimostrazioni complesse, il*

che è già un risultato notevole. Come Kevin ed Enrico, mi hai aiutato a migliorare il mio stile e ad ampliare le mie prospettive, facendomi capire che il nostro lavoro può essere di interesse ben oltre la nostra comunità. Abbiamo ancora molto lavoro da fare insieme, e non vedo l'ora.

Je voudrais remercier chaleureusement les membres de mon jury, qui ont accepté de consacrer du temps et de l'attention à ce manuscrit.

Ha Duong, dont la semaine d'échanges à Nice reste pour moi un souvenir particulièrement vivace. Votre vision du problème est souvent orthogonale à la mienne. Et c'est précisément ce que j'apprécie chez vous : vous me faites voir mon propre travail sous un angle entièrement nouveau.

Stefan, votre intérêt pour le sujet a fait de vous une sorte de fil conducteur du domaine. C'est un honneur que de vous avoir comme rapporteur dans la continuité de la thèse de Sara.

Je vous suis sincèrement reconnaissant à vous deux d'avoir rapporté mon manuscrit.

Nadia, vous êtes celle à qui je dois ma découverte des problèmes d'énumération. Vos cours de complexité en M2 ont été déterminants, et une partie entière de ce manuscrit n'existerait pas sans eux. J'espère sincèrement qu'il saura leur rendre un hommage à la hauteur de ce qu'ils m'ont apporté.

Christophe, votre gentillesse et votre curiosité intellectuelle m'ont marqué lors de nos échanges. Vous partagez avec Ha Duong cette approche complémentaire de la mienne, et nos conversations ont toujours été stimulantes.

Maximilien, enfin, vous avez rédigé un article qui constitue le fondement d'une grande partie de mes résultats. J'ai eu la chance de collaborer avec vous, que ce soit à Marseille ou à Durham. Votre façon de prendre de la hauteur sur un problème, de trouver des pistes inattendues, est pour moi une source constante d'inspiration. Je suis heureux des articles que nous avons écrits ensemble et vous remercie pour vos précieuses contributions.

C'est beaucoup pour moi que vous soyez tous dans mon jury et je vous remercie tous d'avoir accepté de lire mon manuscrit. Et plus largement, à quiconque lira ces pages : merci ! Écrire une thèse, c'est avant tout espérer qu'elle sera lue et chaque lecteur compte.

Cette thèse s'est déroulée au sein du LIS, dans l'équipe CANA, sur le site de Luminy. Je ne pouvais pas rêver mieux. L'ambiance y a été, depuis le premier jour, décontractée, chaleureuse et bienveillante. C'est dans ce cadre que j'ai trouvé la confiance pour mener ce travail à son terme.

J'ai eu la chance de partager mon bureau avec Léah pendant trois ans. Ta présence calme et réfléchie, si différente de ma nature plus passionnée dirons-nous, a été pour moi un véritable ancrage. Tu as su m'écouter quand j'en avais besoin et m'ignorer aussi, dans les moments où je devais simplement réfléchir à voix haute. Je te remercie pour cette patience rare et précieuse, dont je mesure aujourd'hui tout ce qu'elle t'a coûté.

Je remercie également Samuel. Le bureau a aussi été partagé avec toi ces derniers mois et même si nous ne nous connaissons pas encore vraiment, ta présence est déjà

un plus important. Mais aussi Victor, tes conseils ont toujours été avisés.

Sylvain mérite une mention toute particulière : nos différents échanges et notre séjour en Italie m'ont beaucoup appris, notamment sur le fonctionnement des choses institutionnelles, que tu as su m'expliquer avec une patience dont je te suis sincèrement reconnaissant.

Je remercie François, avec qui j'ai collaboré sur deux articles. Avec Sara, nous avons trouvé un véritable rythme de croisière et une méthode de travail qui ont rendu cette collaboration aussi agréable qu'efficace. C'est une expérience que je garde en mémoire avec beaucoup de satisfaction. Ton aide fut très importante pour moi. Merci aussi à Enrico Formenti, membre de mon CSI, avec qui j'espère avoir l'occasion de travailler dans les années à venir.

Je voudrais avoir un mot pour Adrien et Florian. Nos échanges et nos travaux en commun m'ont été d'une grande aide et m'ont permis d'élargir mon spectre d'intérêt.

Finalement, je tiens à remercier Patricia, Nelly, Sylvie, Nadine C. et Nadine L. Vous, les membres du service administratif du LIS, qui vous êtes si bien occupées de moi pendant ces années. Sans vous, cette thèse aurait été probablement impossible car c'est vous le réel cerveau du laboratoire.

Vous tous, que vous soyez dans l'équipe CANA ou pas, vous avez été dans mon équipe lors de cette thèse et je vous en suis reconnaissant.

Au sein du labo, je voudrais remercier Benjamin Monmège, membre de mon CSI, ta présence rassurante et tes conseils m'ont été précieux dans les méandres du conseil de département. Alessia Milani, il suffisait parfois d'un de tes sourires croisé dans les couloirs pour que ma journée prenne une meilleure tournure. Jean-Luc Marie, tu m'as offert l'opportunité de rejoindre la commission recherche et le conseil du collège doctoral : une expérience qui compte parmi les plus enrichissantes de ces années de thèse. Noël Novelli, les discussions que j'avais avec toi étaient une rigolade garantie, et sans toi les conseils de département auraient eu une tout autre saveur.

Je remercie plus largement l'ensemble des membres du conseil de département et du conseil du collège doctoral : les réunions auxquelles j'ai participé ont été, à chaque fois, une vraie source d'apprentissage et d'ouverture.

Au-delà du LIS, cette thèse m'a donné l'occasion de travailler avec des collègues venus d'horizons et pays différents. Pedro, Eurico, Lucas, Rocco, Eric et Marco, travailler avec vous a été à la fois un plaisir et une chance. Je remercie plus généralement l'ensemble des co-auteurs des articles que résume cette thèse. L'assistance et l'expertise que vous m'avez apportées ont contribué à la qualité finale de ce travail.

Comment puis-je ne pas remercier mes parents, à qui je dois tout ? Papa, toi qui aimes me rappeler que tu n'as pas le bac, et toi Maman, dont le parcours s'est arrêté à une licence en administration, vous n'étiez peut-être pas préparés à voir votre fils s'engager dans un bac+8 en informatique, surtout en repensant à mes très jeunes années. Ce projet a sûrement semblé un peu fou pour vous, même si vous ne me l'avez jamais dit. Vous m'avez soutenu sans réserve, donné sans compter, et je vous en serai reconnaissant pour le reste de ma vie. Je voudrais te remercier particulièrement, Maman, pour avoir relu ce manuscrit et corrigé mes fautes d'orthographe. Une aide précieuse et concrète, et une preuve de plus, parmi tant d'autres, de ton soutien

indéfectible.

Ma sœur, tu as toujours été là, même dans les moments où nous ne nous entendions pas si bien. Tu as ce talent de me ramener sur terre, de me rappeler que les longues études ne font pas la valeur d'une personne. C'est une leçon d'humilité que je te dois, et que j'espère ne jamais oublier.

Je voudrais dédier une pensée toute particulière à ma marraine, qui ne pourra malheureusement pas lire ces pages. Née en 1938, fille d'une employée des PTT et d'un immigré arménien ayant fui enfant le génocide, tu as accompli un parcours remarquable : des études de droit administratif jusqu'à l'ENA. Ta carrière fut couronnée par ton entrée au Conseil d'État. Un chemin d'autant plus mémorable que tu es d'origine modeste, et d'autant plus admirable que tu es une femme et que tu as gravi les échelons dans le monde d'hommes de ton époque. Ta vie a toujours été pour moi la preuve que le travail et la persévérance finissent par payer, même quand tout semble s'y opposer.

Enfin, mes amis. J'écris ces lignes pour vous dire que je vous aime tous immensément, et que ces années de thèse ont été un temps trop court à passer parmi de si excellents et si admirables compagnons. Je ne connais pas la moitié d'entre vous à moitié autant que je le voudrais; et j'aime moins que la moitié d'entre vous à moitié aussi bien que vous le méritez.

Marc, Amaury et Ioana, vous êtes mes compagnons depuis le M1, et ces années passées à vos côtés comptent parmi les plus heureuses de ma vie. Vous m'avez soutenu dans les hauts et les bas avec une patience que je ne saurais jamais assez vous rendre. Je garderai longtemps ton rire franc, Ioana, même pour les blagues qui ne le méritaient pas, ton calme et ton humanité, Amaury, toujours chaleureux sans jamais être condescendant, et ta franchise, Marc, tu as toujours su me dire ce que tu pensais, même quand ce n'était pas ce que je voulais entendre.

Je te remercie aussi Camille, ta bonne humeur et ta fraîcheur ont rapidement et naturellement illuminé notre petit groupe. Monica, ta profonde gentillesse m'a prouvé que la réputation des Canadiens n'est pas un mythe. Et Samantha, je te connais depuis la L1, et ta compagnie a toujours été un plaisir simple et sincère.

Alice, Marion et Élie, vous faites partie de ce que j'appellerai la clique du midi. Ces repas partagés et ces parties de cartes qui ont ponctué ces années les ont rendues bien plus douces. Ils ont été trop courts.

Finalement, j'aimerais remercier tous les professeurs qui m'ont soutenu. Je n'aurais pas pu aller aussi loin sans vous. J'ai une pensée toute particulière pour mon professeur de mathématiques du collège, c'est vous qui m'avez donné l'amour des mathématiques et ma gratitude envers vous est inépuisable.

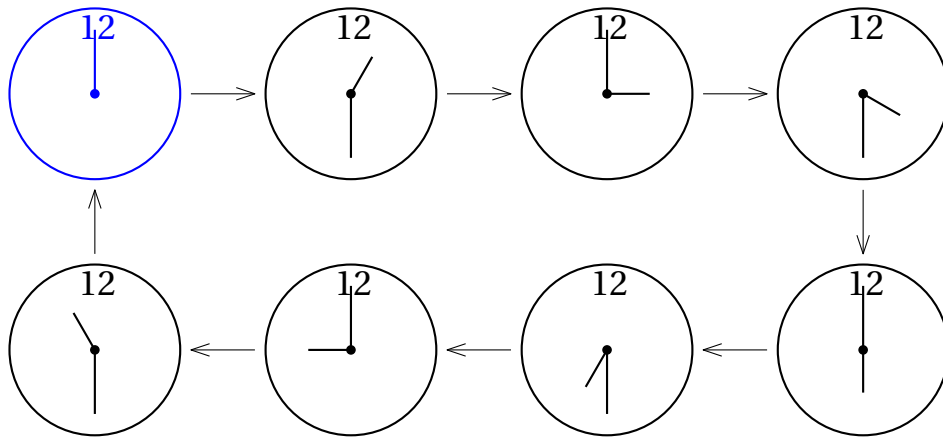
Table des matières

Affidavit	2
Liste de publications et participations aux conférences	3
Résumé et mots-clés	5
Abstract and keywords	7
Remerciements	8
Table des matières	12
Introduction	14
Notation	19
 I. État de l’art et définitions générales	 23
1. État de l’art et définitions	24
1.1. Algèbre des systèmes dynamiques finis déterministes	24
1.2. Résultats de calculabilité et complexité	35
1.3. Gestion des parties transitoires : Unroll	48
 II. Équations polynomiales résolubles en temps polynomial	 65
2. Résolutions d’équations polynomiales sur les unrolls	66
2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe	67
2.2. Complexité de la division d’unroll	75
2.3. Complexité de la résolution des équations monomiales sur les unrolls	84
2.4. Complexité de la résolution des équations polynomiales sur les unrolls	93
2.5. Conclusion et perspectives	100
3. Résolution d’équations polynomiales de FDDS	102
3.1. Le cas où le polynôme est injectif	103
3.1.1. Le cas des racines	103
3.1.2. Le cas des équations $P(X) = B$ où un coefficient non-constant de P contient un dendron	109
3.1.3. Conditions nécessaires à l’injectivité des polynômes de FDDS	111

3.2. Le cas des polynômes “pseudo-injectifs”	114
3.2.1. Résoudre $P(X) = B$	115
3.2.2. Équation sur les permutations encodée par des entiers	127
3.3. Conclusion et perspectives	133
4. Résolution d'équations sur les supports de FDDS	135
4.1. Complexité de l'évaluation des polynômes sur les supports	136
4.2. Équation linéaire sur les supports	142
4.2.1. Résolution des équations linéaires sur les supports	142
4.2.2. Énumération des solutions des équations linéaires sur les supports	148
4.2.3. Approche alternative : décomposition des équations	154
4.3. Des équations linéaires sur les supports à celles sur les FDDS	157
4.4. Conclusion et perspectives	159
III. Extensions des résultats	161
5. Sous- et sur-approximation du résultat	162
5.1. Énumération des solutions de $P(X) \leq B$	164
5.1.1. Les solutions fusion-free de $AX^k \leq B$	166
5.1.2. Fusions de solutions fusion-free	173
5.1.3. Énumération de l'ensemble des solutions de $AX^k \leq B$	181
5.1.4. Énumération de l'ensemble des solutions de $P(X) \leq B$	182
5.1.5. Généralisation au cas multivarié	185
5.2. Résolution des inéquations $P(X_1, \dots, X_n) \geq B$	189
5.2.1. Le cas $AX \geq B$	189
5.2.2. Le cas $P(X) \geq B$	192
5.2.3. Le cas $P(X_1, \dots, X_n) \geq B$	194
5.3. Conclusion et perspectives	195
6. Les systèmes partiels	198
6.1. Propriétés algébriques des systèmes partiels	199
6.1.1. Premier pour les systèmes partiels	200
6.1.2. Injectivité des polynômes sur les systèmes partiels	201
6.2. Résolution de $P(X) = B$ pour les systèmes partiels	210
6.2.1. Résoudre $P(X) = B$ lorsqu'un coefficient non constant A est tel que $\mathcal{T}(A) \neq \emptyset$	211
6.2.2. Résoudre $P(X) = B$ si tous les coefficients sont des forêts finies .	213
6.3. Conclusion et perspectives	227
Conclusion	228
Bibliographie	236

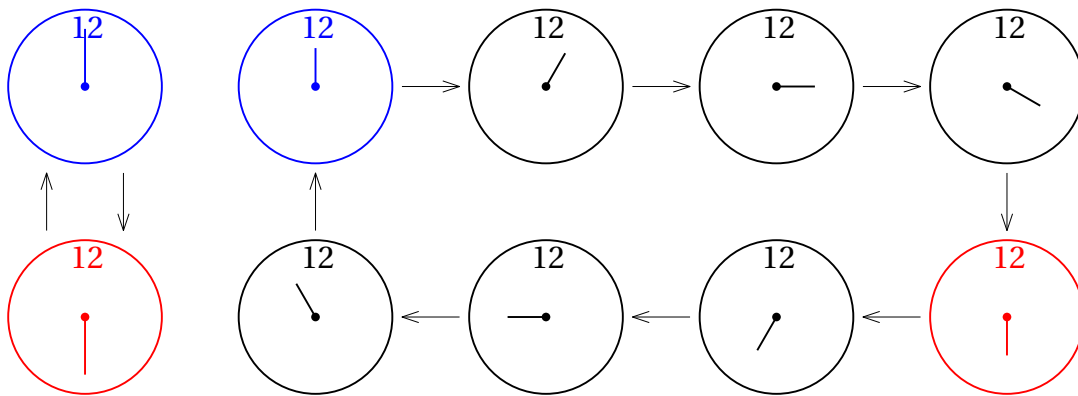
Introduction

Considérons un ingénieur cherchant à fabriquer un nouvel appareil mesurant le temps. Pour cela, il s'inspire d'une horloge à aiguilles et tente de comprendre son fonctionnement. Plutôt que d'analyser directement les rouages internes du mécanisme, il choisit d'en observer le comportement. Pour ce faire, il utilise un appareil photographique prenant un cliché à intervalles réguliers, ici toutes les heures et demie. L'appareil est déclenché à midi, face au cadran de l'horloge. Après plusieurs jours d'observation, l'ingénieur dispose d'un ensemble de données qu'il peut analyser. La synthèse de ces observations peut être représentée sous la forme du graphe suivant.



Chaque sommet du graphe correspond à une photographie, c'est-à-dire à un *état* du système. Un arc relie deux états lorsqu'ils proviennent de photographies successives. L'état initial est en bleu. En mathématiques, un tel objet est appelé un *système dynamique*. Bien qu'obtenu par une méthode empirique, ce système capture la dynamique intrinsèque du phénomène observé.

L'étude attentive de ce graphe révèle que la dynamique observée semble résulter de la combinaison de deux comportements sous-jacents : celui de l'aiguille des heures et celui de l'aiguille des minutes. Ces comportements sont illustrés dans les deux graphes suivants.

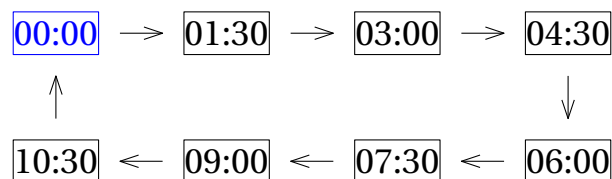


Si nous partons des deux sommets en bleu, puis que nous faisons évoluer les états des deux systèmes de manière synchronisée et que nous superposons les images, nous retrouvons précisément la dynamique globale de l'horloge.

Cette décomposition met en lumière une opération fondamentale sur les systèmes dynamiques, à savoir l'*exécution synchrone*. Deux systèmes dynamiques évoluant de manière synchronisée produisent un nouveau système dont les états sont les couples d'états des systèmes de base, où chaque composante de ce couple suit sa propre loi d'évolution (les arcs du graphe). L'opération ainsi définie jouera un rôle central tout au long de ce manuscrit.

La figure précédente met également en évidence une sensibilité au choix de l'état initial. En effet, en partant d'un état différent (en rouge sur la figure), la dynamique observée diffère. Cette propriété peut être vérifiée expérimentalement en modifiant l'instant de déclenchement de l'appareil photographique. Elle illustre le fait qu'un système dynamique peut présenter plusieurs comportements distincts selon le point de départ, phénomène que nous appelons *exécution alternative*. L'exécution alternative de deux systèmes dynamiques est soit un état du premier système, soit un état du second, chacun évoluant indépendamment de l'autre.

Afin de vérifier que les propriétés observées ne dépendent pas spécifiquement de l'horloge étudiée, l'expérience est reproduite avec une autre horloge. Les résultats obtenus sont les suivants.



Bien que les représentations diffèrent en apparence, leur structure dynamique est identique. Les deux systèmes ne sont que des cycles de même longueur. Cette observation invite à s'abstraire des noms (ou interprétations) des états, pour ne retenir que la structure de la dynamique. Deux systèmes dont les graphes de dynamique sont *isomorphes* seront ainsi considérés comme identiques du point de vue de leur comportement.

Cette perspective nouvelle soulève plusieurs questions naturelles. Un système donné peut-il être décomposé comme l'exécution synchrone de dynamiques plus

simples? Une telle décomposition est-elle calculable efficacement? Plus généralement, étant donnés deux systèmes A et B , existe-t-il un système X dont l'exécution synchrone avec A produit B ?

Contexte mathématique

Afin de fournir un cadre formel aux questions précédentes, nous définissons un système dynamique, noté FDDS, comme une paire (A, f_A) où A est un ensemble fini d'états et $f_A : A \rightarrow A$ est une fonction de transition totale. Pris à isomorphisme près, les FDDS forment un ensemble noté $\mathbb{D}$. De façon générale, un FDDS peut se diviser en composantes connexes, chacune constituée d'un unique cycle dans lequel sont enracinés des arbres orientés vers la racine. Les sommets du cycle constituent la *partie cyclique* du système, et les arbres sa *partie transitoire*. Nous retrouvons ces systèmes dans l'analyse de modèles concrets tels que les réseaux booléens [25, 26] et nous pouvons les appliquer en biologie [50, 49, 3] pour représenter, par exemple, des réseaux de régulation génétique ou des modèles épidémiques. Ils apparaissent également en chimie [19] ou en théorie de l'information [23] où il est souvent utile de décomposer de grands systèmes complexes en sous-systèmes plus petits, plus faciles et moins coûteux à analyser [41]; cela permet de déterminer des propriétés dynamiques globales (*i.e.*, l'existence et le nombre de points fixes ainsi que le temps de convergence vers les comportements limites) en fonction des propriétés correspondantes des sous-systèmes.

Pour ce faire, nous pouvons utiliser les deux opérations introduites par l'exemple précédent, car elles s'étendent à l'ensemble $\mathbb{D}$. L'*exécution alternative* de deux FDDS A et B , notée $A + B$, est le FDDS obtenu en juxtaposant leurs graphes de dynamique par union disjointe. L'*exécution synchrone* de A et B , notée $A \times B$, est le FDDS dont l'ensemble d'états est le produit cartésien $A \times B$, et dont la transition est définie par $f_{A \times B}(u, v) = (f_A(u), f_B(v))$. De plus, le produit se distribue sur la somme, ce qui fait de $(\mathbb{D}, +, \times)$ un *semi-anneau commutatif* [11].

Remarquons que le comportement cyclique du produit de deux FDDS connexes de longueurs de cycle p_A et p_B est entièrement déterminé. En effet, leur exécution synchrone produit exactement $\gcd(p_A, p_B)$ composantes connexes de longueur de cycle $\text{lcm}(p_A, p_B)$. Le comportement transitoire, en revanche, est plus complexe à analyser. Pour ce faire, il a été introduit la notion d'*unroll*. Un *unroll* est une forêt d'arbres infinis périodiques construite en déroulant les cycles d'un FDDS en chemin infini et en y enracinant périodiquement les arbres de la partie transitoire. Cette structure capture la façon dont les états transitoires interagissent lors de l'exécution synchrone, et constituera l'un des outils principaux de ce manuscrit.

À chaque semi-anneau est associé un semi-anneau de polynômes. Un polynôme sur les FDDS est une expression de la forme $P = \sum_{i=0}^m A_i X^i$, où les coefficients A_i sont des FDDS. Ce formalisme permet d'exprimer de nombreuses questions de décomposition. Décider si B est décomposable sous la somme revient à résoudre $X + Y = B$. Déterminer si B est le produit synchrone d'un FDDS A avec un autre système revient à résoudre $AX = B$. Le point central de ce manuscrit est la recherche d'algorithmes

efficaces pour répondre à la question suivante : étant donné un polynôme P sur les FDDS et un FDDS B , existe-t-il un FDDS X tel que $P(X) = B$?

Pour y parvenir, nous commencerons par introduire le cadre mathématique précis ainsi que l'état de l'art (Chapitre 1) lié à ces équations. Après quoi, nous présenterons un algorithme polynomial permettant de déterminer les comportements transitoires de toutes les solutions de $P(X) = B$ (Chapitre 2). Puis, nous proposerons une méthode efficace de résolution de $P(X) = B$ lorsque le polynôme P satisfait certaines conditions structurelles, telles que l'injectivité ou la pseudo-injectivité (Chapitre 3). Nous mettrons ensuite en évidence des paramètres afin de résoudre efficacement $AX = B$ s'ils sont fixés (Chapitre 4). À la suite de quoi, nous introduirons des algorithmes polynomiaux pour les inéquations de la forme $P(X) \leq B$ et $P(X) \geq B$ ainsi qu'une procédure d'énumération avec délai polynomial pour $P(X) \geq B$ où $\geq$ est l'ordre induit par la somme (Chapitre 5). Enfin, nous étendrons ces résultats aux *systèmes partiels* (Chapitre 6). Dans la conclusion, nous synthétiserons l'ensemble des problèmes ouverts que nous aurons présentés dans ce manuscrit et en proposerons de nouveaux, plus généraux.

Annnonce détaillée du plan

Chapitre 1 — Cadre mathématique et état de l'art. Ce chapitre posera les fondations formelles sur lesquelles repose l'ensemble du manuscrit. Nous commencerons par définir formellement ce qu'est un FDDS, puis nous introduirons une relation d'égalité entre FDDS reposant sur les graphes fonctionnels qu'ils décrivent. Ceci nous permettra de mettre en lumière la structure dynamique d'un FDDS. Nous approfondirons ensuite la relation entre FDDS et graphe fonctionnel en démontrant que les graphes fonctionnels munis de l'union disjointe pour addition et du produit direct pour multiplication forment un semi-anneau isomorphe à celui des FDDS avec l'exécution alternative pour somme et l'exécution synchrone pour produit. Nous présenterons par la suite l'état de l'art des résultats de difficulté concernant la résolution d'équations polynomiales sur les FDDS. Enfin, nous introduirons formellement les unrolls et les usages qui en ont déjà été faits.

Chapitre 2 — Résolution des équations polynomiales sur les unrolls. Dans ce chapitre, nous nous concentrerons sur la détermination du comportement transitoire de l'ensemble des solutions de l'équation $P(X) = B$, c'est-à-dire le calcul de l'unroll du FDDS X , noté $\mathcal{U}(X)$, pour toute solution X . Pour y parvenir, nous traiterons successivement les équations de la forme $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X un FDDS connexe, puis nous relâcherons la contrainte de connexité. Après quoi, nous travaillerons sur les équations de la forme $\mathcal{U}(X)^k = \mathcal{U}(B)$, ce qui nous permettra de généraliser aux équations de la forme $\mathcal{U}(A)\mathcal{U}(X)^k = \mathcal{U}(B)$ et de finir par conclure pour le cas des polynômes. La propriété fondamentale sous-tendant ces résultats est que nous pourrions simplement considérer une partie finie des arbres d'unroll pour nos algorithmes.

Chapitre 3 — Résolution pour les polynômes injectifs et pseudo-injectifs.

Ce chapitre s'attaquera à la résolution complète de $P(X) = B$ dans les cas où le polynôme P aura une structure particulière. Nous caractériserons tout d'abord les polynômes sur les FDDS injectifs et proposerons un algorithme en temps polynomial pour résoudre $P(X) = B$ si P est injectif. Nous étendrons ensuite ces résultats à la classe des polynômes pseudo-injectifs, qui est strictement plus générale que celle des polynômes injectifs. Nous montrerons de plus que si P est un polynôme sur les sommes de cycles et B est une somme de cycles, mais où les cycles sont donnés par des entiers codés en binaire au lieu de graphes, alors la résolution de $P(X) = B$ demeure efficace.

Chapitre 4 — Équations sur le support des FDDS. Ce chapitre étudiera quelles sont les composantes connexes pouvant apparaître dans un FDDS X , c'est-à-dire le support de X , tel que $P(X) = B$. Pour cela, nous introduirons le semi-anneau des supports de FDDS, et nous montrerons que décider si une équation polynomiale avec un côté constant admet une solution est un problème appartenant à la classe NP. Nous prouverons que ce problème est même NP-complet si le polynôme est non-linéaire. Nous établirons toutefois que si le polynôme est linéaire, alors nous pouvons résoudre efficacement les équations. Nous démontrerons que dans ce cas, il est même possible d'énumérer l'ensemble des solutions avec un délai polynomial. Puis nous utiliserons ces résultats pour prouver que la résolution de $AX = B$ est réalisable en temps polynomial si nous fixons deux paramètres, à savoir le nombre de composantes connexes non isomorphes de B et la longueur du plus petit cycle de A .

Chapitre 5 — Inéquations et énumération des solutions. Ce chapitre s'intéressera non plus aux équations polynomiales mais aux inéquations de la forme $P(X) \leq B$ et $P(X) \geq B$, où $\leq$ désigne l'ordre partiel induit par la somme sur $\mathbb{D}$. Pour l'inéquation $P(X) \leq B$, nous proposerons un algorithme d'énumération à délai polynomial sur la taille de B . Nous étendrons enfin ces résultats au cas multivarié, en traitant les inéquations de la forme $P(X_1, \dots, X_n) \leq B$ pour une certaine forme de polynôme P . Pour l'inéquation $P(X) \geq B$, nous montrerons qu'une solution peut être calculée en temps polynomial. Nous généraliserons cela aux inéquations de la forme $P(X_1, \dots, X_n) \geq B$ pour n'importe quel polynôme P .

Chapitre 6 — Extension aux systèmes dynamiques partiels. Ce chapitre étudiera un sur-semi-anneau des FDDS : celui des systèmes partiels, c'est-à-dire des graphes orientés dont chaque sommet a un degré sortant d'au plus 1 (contre exactement 1 pour les FDDS). Nous caractériserons les éléments premiers de ce nouveau semi-anneau et nous étendrons ensuite la caractérisation des polynômes injectifs de FDDS. Pour ce faire, nous généraliserons la définition de l'unroll aux systèmes partiels, ce qui nous permettra de montrer que la résolution de $P(X) = B$ sur les systèmes partiels n'est pas plus difficile, au sens des réductions Turing, que la résolution de $P(X) = B$ sur les FDDS.

Notation

Notation	Description
Ensembles de base	
$\mathbb{N}$	Ensemble des entiers naturels (incluant 0).
$\mathbb{D}$	Ensemble des FDDS pris à isomorphisme près.
$\mathbb{G}$	Ensemble des graphes fonctionnels pris à isomorphisme près.
$\mathbb{B}$	Ensemble des permutations, <i>i.e.</i> des FDDS sans partie transitoire.
$\mathbb{B}_1$	Ensemble des collections de points fixes ; sous-semi-anneau isomorphe à $(\mathbb{N}, +, \times)$.
$\mathbb{D}^*$	Clôture par + de l'ensemble des dendrons.
$\mathbb{F}$	Ensemble des arbres d'unroll (forêts infinies périodiques).
$\mathbb{F}_d$	Ensemble des forêts finies de profondeur au plus d .
$\mathbb{F}_f$	Ensemble des forêts finies.
$\mathbb{P}$	Ensemble des partiels, <i>i.e.</i> des FDDS à fonction de transition partielle.
$\mathcal{U}(\mathbb{D})$	Ensemble des unrolls de FDDS.
$\mathcal{S}(\mathbb{D})$	Ensemble des supports de FDDS.
Structures algébriques	
$(\mathbb{D}, +, \times)$	Semi-anneau commutatif des FDDS.
$(\mathbb{G}, +, \times)$	Semi-anneau commutatif des graphes fonctionnels, isomorphe à $(\mathbb{D}, +, \times)$.
$(\mathbb{F}, +, \otimes)$	Semi-anneau des arbres d'unroll.
$(\mathcal{S}(\mathbb{D}), \boxplus, \boxtimes)$	Semi-anneau des supports.

(suite)

Notation	Description
Notations sur les FDDS	
(A, f_A)	FDDS d'ensemble d'états A et de fonction de transition $f_A : A \rightarrow A$.
f_A^n	n -ième composée de f_A avec elle-même; $f_A^0 = \text{id}$.
$f^{-k}(v)$	Ensemble des k -ièmes pré-images de v dans A .
G_A	Graphe de dynamique du FDDS A .
$ A $	Nombre d'états (taille) du FDDS A .
$A + B$	Union disjointe de A et B .
$A \times B$	Produit direct de A et B .
$B - A$	Soustraction de A à B , définie si A est un sous-multi-ensemble de B .
C_i	Cycle de longueur $i > 0$.
$d(u)$	Profondeur d'un sommet u dans un arbre ou un FDDS.
$d(A)$	Profondeur du FDDS A .
$L(X)$	Ensemble des longueurs de cycles des composantes connexes de X .
$\ell(X)$	Plus petite longueur de cycle de X ; $\ell(X) = \min(L(X))$.
$\lambda_p(A)$	Sous-multi-ensemble des composantes connexes de A de longueur de cycle p .
$\Lambda_p(A)$	Sous-multi-ensemble des composantes connexes de A dont la longueur de cycle divise p .
$\mathcal{S}(A)$	Support du FDDS A : ensemble de ses composantes connexes distinctes.
$A \leq B$	A est une sous-approximation de B (A est un sous-multi-ensemble de B).
$\pi_i(A)$	Préfixe de longueur i du FDDS A .
$\mathcal{P}_i(A)$	Super-préfixe de longueur i du FDDS A .

(suite)

Notation	Description
Unrolls et arbres	
$\mathbf{t}$	Arbre appelé t
$\mathbf{F}$	Forêt appelée F
$\mathcal{U}_v(A)$	Arbre d'unroll de A enraciné dans v , pour v sommet du cycle de A .
$\mathcal{U}(A)$	Unroll du FDDS A : somme des $\mathcal{U}_v(A)$ sur les sommets des cycles de A .
$\mathfrak{U}(A)$	Unroll du système partiel A .
$\mathbf{t}_1 \otimes \mathbf{t}_2$	Produit de deux arbres.
$\mathcal{C}_n(\mathbf{t})$	Troncature de l'arbre t à profondeur n .
$\mathcal{C}_n(\mathbf{F})$	Troncature de la forêt F à profondeur n .
$\min(\mathbf{F})$	Plus petit arbre de la forêt $\mathbf{F}$ selon l'ordre $\leq$ sur les arbres.
$\mathcal{D}(\mathbf{t})$	Déracinement de l'arbre $\mathbf{t}$: forêt des sous-arbres enracinés dans la racine de $\mathbf{t}$.
$\mathcal{E}(\mathbf{F})$	Enracinement de la forêt $\mathbf{F}$ dans un nouveau sommet.
$\mathcal{R}_p(\mathbf{t})$	L'enroulement de l'arbre $\mathbf{t}$ à période p : FDDS connexe obtenu en repliant $\mathbf{t}$.
$\sigma_i(\mathbf{t})$	Décalage à droite de i de l'arbre d'unroll $\mathbf{t}$.
$\Sigma(\mathbf{t})$	Ensemble de tous les décalages de $\mathbf{t}$.
$\mathcal{D}^i(A)$	Multi-ensemble des arbres enracinés dans les sommets de profondeur i de A .
$\Gamma_d(\mathbf{F})$	Sous-multi-ensemble des arbres de $\mathbf{F}$ de profondeur au moins d .
Fonctions arithmétiques	
$\gcd(a, b)$	Plus grand commun diviseur de a et b .
$\text{lcm}(a, b)$	Plus petit commun multiple de a et b .
$\text{alcm}_a b$	Anti-lcm de b par a : plus petit x tel que $\text{lcm}(a, x) = b$.

(suite)

Notation	Description
Système partiel	
$\mathcal{T}(A)$	Le multi-ensemble des composantes connexes du système partiel A contenant un cycle.
$\mathcal{F}(A)$	Le multi-ensemble des composantes connexes du système partiel A ne contenant pas de cycle.
$F_k(A)$	Les super-feuilles du système partiel A de profondeur k .
$\chi_d(A)$	Le multi-ensemble des sous-arbres distants de d de A pris à isomorphisme près, c'est-à-dire $\chi_d(A) = \Gamma_d(\mathfrak{U}(A))$.
Ordres sur les FDDS et les composantes connexes	
$\mathbf{s} \leq \mathbf{t}$	Ordre total sur les arbres de [40].
$A \leq_{ct} B$	Ordre sur les composantes connexes : $\ell(A) < \ell(B)$, ou $\ell(A) = \ell(B)$ et $\min(\mathcal{U}(A)) \leq \min(\mathcal{U}(B))$.
$X \leq_P Y$	Ordre sur les composantes connexes induit par le polynôme $P : D_X <_{ct} D_Y$ ou $D_X = D_Y$ et $X \leq_{ct} Y$ avec D_X et D_Y les plus petites composantes connexes de respectivement $P(X)$ et $P(Y)$ selon $\leq_{ct}$.
Classes de complexité	
P	Classe des problèmes de décision résolubles en temps polynomial.
NP	Classe des problèmes de décision vérifiables en temps polynomial.
FP	Classe des problèmes dont la résolution est en temps polynomial.
FPT	<i>Fixed-Parameter Tractable</i> : résoluble en $f(k) \cdot x ^{\mathcal{O}(1)}$ pour un paramètre k fixé.
DelayP	Classe des problèmes d'énumération à délai polynomial entre deux solutions.
EnumP	Classe des problèmes d'énumération des problèmes dont toutes les solutions ont une taille polynomiale et où ces solutions sont vérifiables en temps polynomial.

Première partie

État de l'art et définitions générales

1. État de l'art et définitions

Sommaire

1.1. Algèbre des systèmes dynamiques finis déterministes	24
1.2. Résultats de calculabilité et complexité	35
1.3. Gestion des parties transitoires : Unroll	48

1.1. Algèbre des systèmes dynamiques finis déterministes

Les systèmes dynamiques discrets finis déterministes Les systèmes dynamiques sont des modèles formels utilisés pour représenter l'évolution de phénomènes au cours du temps. Ils servent donc tout aussi bien en physique qu'en biologie pour décrire des phénomènes naturels. Dans ces contextes, le temps et l'espace sont généralement continus et l'évolution du système est régie par des équations différentielles. De ce fait, la simulation de ces systèmes est computationnellement difficile et peut nécessiter des supercalculateurs. Pensez par exemple aux modèles de prévision météorologique.

Ces équations offrent une grande précision qui n'est cependant pas toujours utile. En effet, parfois une simple estimation permet de comprendre un phénomène, d'autant plus si le modèle simplifié peut tourner rapidement sur un simple ordinateur. Une méthode classique que les mathématiques proposent pour obtenir ce genre d'approximation est ce que nous appelons la *discrétisation*.

L'idée derrière cette approche est simplement de fixer une fréquence à laquelle nous réalisons des observations. À partir d'un nombre fini de relevés, nous obtenons un ensemble fini et discret d'états. Intuitivement, nous aurons une transition entre l'état 1 et l'état 2 si nous observons 2 juste après avoir observé 1. Cette méthode produit ce que nous appelons des *systèmes dynamiques finis*.

Dans cette approche, chaque observation décrit un état différent. Or, nous pourrions aussi donner une interprétation à ces échantillons. Par exemple, si nous en avons deux identiques, alors ils décrivent le même état. Via cette analyse, un état peut avoir plusieurs successeurs. Nous obtenons ainsi ce que nous appelons des *systèmes dynamiques finis non-déterministes*. Pour pallier cela, nous pouvons par exemple calculer la probabilité de chaque successeur et ajouter cette information aux transitions. Ceci engendre des *systèmes dynamiques finis probabilistes*. Nous pouvons également tenir compte du voisinage lors de notre analyse pour décrire une fonction. C'est-à-dire que

nous réunissons deux observations identiques en un état si elles ont rigoureusement les mêmes chemins qui les traversent. Ceci décrit ce que nous nommons *système dynamique fini déterministe* et en abrégé FDDS pour *finite deterministic dynamical system* (la Figure 1.2 donne un exemple des constructions évoquées ci-dessus). Ce sont ces derniers que nous allons étudier, c'est pourquoi nous les définissons plus formellement.

Définition 1.1 (FDDS). Un *système dynamique fini déterministe* est une paire (A, f_A) où A est un ensemble fini d'états (possiblement vide) et $f_A : A \rightarrow A$ est une fonction totale, appelée *fonction de transition*, de A dans lui-même qui définit les successeurs de chaque état de A . Quand le contexte est clair, nous assimilerons le FDDS à son ensemble d'états.

Égalité entre FDDS Dans l'optique d'analyser les FDDS, il ne serait pas inintéressant de pouvoir au moins comparer deux FDDS, par exemple savoir décider s'ils sont égaux ou différents. Une solution simple pour vérifier que $A = B$, où A et B sont deux FDDS, serait de tester si A et B ont le même ensemble d'états et si $f_A(i) = f_B(i)$ pour chaque état i de A et B . Cependant, ce test est réducteur. En effet, nous pourrions constituer deux FDDS à partir des mêmes observations mais qui seraient différents, car nous aurions utilisé des nombres pour nommer les états de A et des lettres pour ceux de B . Il nous faudrait donc nous abstraire des noms de chaque sommet pour avoir un test d'égalité qui vérifierait des propriétés plus fondamentales.

Pour cela, nous associons à chaque FDDS une représentation graphique de sa dynamique (le lecteur est invité à consulter [27] pour plus d'informations sur les notions de théorie des graphes que nous utiliserons dans ce manuscrit).

Plus formellement, nous lions chaque FDDS A à un graphe orienté $G_A = (A, E)$ ayant pour ensemble d'arcs les relations induites par f_A , c'est-à-dire les relations $(u, f_A(u))$. Donc $E = \{(u, f_A(u)) \mid u \in A\}$. Ce type de graphe est connu dans la littérature sous le nom de *graphe de la dynamique* de A ou celui de *graphe de la fonction* f_A . Nous pouvons donc définir que deux FDDS A et B sont égaux si et seulement si G_A est égal à G_B sans tenir compte des noms des sommets. Plus formellement, A et B sont égaux si et seulement si G_A est *isomorphe* à G_B .

Définition 1.3 (Isomorphisme de graphes). Soient $G_1 = (V_1, E_1)$ et $G_2 = (V_2, E_2)$ deux graphes. Une fonction $h : V_1 \rightarrow V_2$ est un *homomorphisme* entre G_1 et G_2 si et seulement si pour tout arc (u, v) de E_1 l'arc $(h(u), h(v))$ appartient à E_2 .

Une fonction $g : V_1 \rightarrow V_2$ est un *isomorphisme* entre G_1 et G_2 si et seulement si g est bijective et que g et g^{-1} sont des homomorphismes.

Les graphes G_1 et G_2 sont *isomorphes* si et seulement si il existe un isomorphisme entre G_1 et G_2 .

La notion d'égalité que nous venons d'introduire a bien un sens mathématique, car le fait que deux FDDS aient leur graphe de dynamique isomorphe forme une relation d'équivalence, notée $\simeq$. En effet, $\simeq$ est bien une relation binaire, qui est de plus :

1. État de l'art et définitions – 1.1. Algèbre des systèmes dynamiques finis déterministes

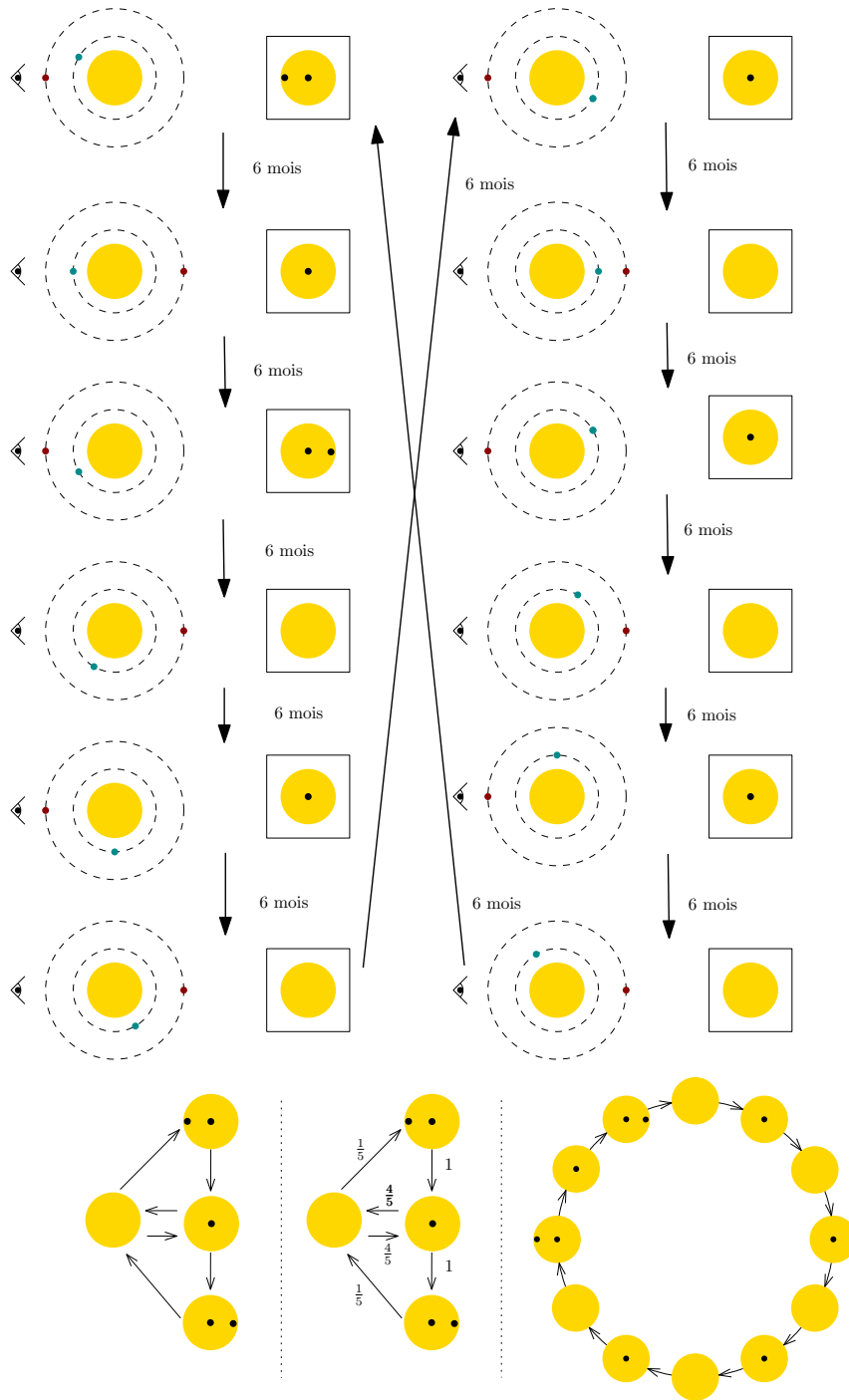


FIGURE 1.2. – Illustration des trois types de systèmes dynamiques finis que nous pouvons construire à partir d'observations d'un phénomène. La partie haute décrit le système de planètes réel (à gauche) et les observations réalisées depuis la Terre (l'œil à gauche). La fréquence entre deux observations est de 6 mois. La partie basse décrit un système non-déterministe (à gauche), probabiliste (au centre) et déterministe (à droite) constructible à partir des observations depuis la Terre.

1. réflexive : l'identité est un isomorphisme d'un graphe vers lui-même,
2. symétrique : si f est un isomorphisme de A vers B , alors f^{-1} est un isomorphisme de B vers A ,
3. transitive : si $A \simeq B$ et que $B \simeq C$, alors $A \simeq C$ car, par composition des isomorphismes, le graphe de la dynamique de A est isomorphe à celui de celle de C .

Ainsi, nous pouvons quotienter l'ensemble des FDDS par la relation $\simeq$ et obtenir l'ensemble $\mathbb{D}$ des FDDS pris à isomorphisme près. Plus formellement, $\mathbb{D}$ est l'ensemble des classes d'équivalence induites par la relation $\simeq$. Par la suite, sauf mention explicite du contraire, un FDDS sera toujours considéré à isomorphisme près et nous utiliserons conséquemment le symbole $=$ au lieu de $\simeq$.

Ainsi, pour décider que deux FDDS sont égaux, il nous faut résoudre l'isomorphisme de graphes orientés, mais la difficulté de ce problème n'est pas encore connue dans le cas général. La communauté soupçonne que ce problème n'est ni dans P ni NP-complet. Mais nos graphes ont une forme particulière rendant leur test d'isomorphisme polynomial. Nous étudions cette forme.

En théorie des graphes, l'ensemble des graphes de dynamique est appelé ensemble des *graphes fonctionnels*. Pris à isomorphisme près, ils forment un ensemble noté $\mathbb{G}$. Nous décidons en espace logarithmique qu'un graphe appartient à $\mathbb{G}$. En effet, il suffit de vérifier que chaque sommet a un degré sortant exactement 1. Cette caractéristique implique qu'un graphe fonctionnel est composé de *composantes faiblement connexes* constituées d'un unique *cycle* simple (ou plus simplement cycle dans la suite) dans lequel sont enracinés des *arbres* orientés des feuilles vers la racine.

Définition 1.4 (Arbres et forêts). Pris à isomorphisme près, un *arbre* orienté des feuilles vers la racine est un graphe connexe dont tous les sommets sauf un, la *racine*, ont degré sortant exactement 1. La racine a, quant à elle, degré sortant 0. Dans la suite, nous noterons en gras et en minuscules les arbres (par exemple **t**) ; un arbre sera toujours orienté vers la racine sauf mention explicite. Un arbre est dit *fini* s'il ne contient qu'un nombre fini de sommets.

Une *forêt* orientée est un graphe dont les composantes connexes sont des arbres. Dans la suite, nous noterons en gras et en majuscules les forêts (par exemple **F**). Nous appelons *forêts finies* les forêts dont tous les arbres sont finis. La Figure 1.5 donne un exemple de forêts.

Par souci de compréhension, nous rappelons qu'un cycle est une séquence de sommets $uv_1 \cdots v_n u$ telle que le graphe contient les arcs (u, v_1) et (v_n, u) et que de plus $v_1 \cdots v_n$ est un *chemin simple*, c'est-à-dire que les v_i sont deux à deux différents et qu'il existe un arc (v_i, v_{i+1}) dans le graphe et ce pour tout i entre 1 et $n - 1$. Enfin, rappelons qu'une composante faiblement connexe est un graphe qui est connexe si nous retirons l'orientation. C'est-à-dire un graphe où, pour toute paire de sommets u, v , il existe un chemin entre u et v . Puisque nous considérerons uniquement des graphes orientés, nous parlerons de composantes connexes au lieu de composantes faiblement connexes. Nous remarquons que tout graphe fonctionnel est *planaire*

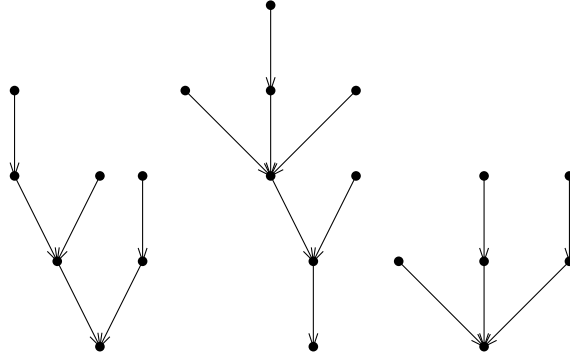


FIGURE 1.5. – Exemple de forêt finie. Chaque composante connexe de ce graphe est un arbre.

extérieur (graphe planaire dont tous les arcs sont en contact avec la face extérieure). En effet, le cycle de chaque composante peut être plongé de façon planaire. Puis nous dessinons les arbres à l'extérieur du cycle sans qu'aucun arc ne se croise. Comme il n'est pas possible de créer une face avec des arbres, tous les arcs du cycle sont toujours en contact avec la face extérieure. À ce point, par [9] et [31], nous déduisons que l'isomorphisme de graphes fonctionnels est complet pour la classe LogSpace. La preuve d'appartenance à LogSpace est introduite dans [9]. Dans celle-ci, nous construisons des arbres étiquetés issus de la décomposition de graphes planaires et nous établissons un ordre total sur ces arbres pouvant être testé en espace logarithmique. Et [31] contient la démonstration que l'isomorphisme de graphes non-orientés planaires et celui de graphes orientés planaires se réduisent, en espace logarithmique, de l'un à l'autre et réciproquement. Si la réduction du cas non-orienté au cas orienté est évidente (la non-orientation équivaut à la double orientation), la preuve que le cas orienté se réduit au cas non-orienté est plus ardue. Pour parvenir à ce résultat, nous introduisons un graphe non-orienté de taille constante encodant sans ambiguïté un arc orienté. Pour nos analyses de complexité, dans lesquelles nous privilégions le temps à l'espace, nous utiliserons le résultat suivant.

Théorème 1.6 ([28]). *L'isomorphisme de graphes planaires est solvable en temps linéaire sur le nombre de sommets des graphes.*

Structure des graphes de dynamique et des FDDS À partir de la structure mise en exergue ci-dessus, nous définissons la *profondeur* d'un sommet u comme suit.

Définition 1.7 (Profondeur). Soit u un sommet d'un arbre $\mathbf{t}$. La *profondeur* de u dans $\mathbf{t}$, notée $d(u)$, est la longueur du chemin entre u et la racine de $\mathbf{t}$. La profondeur de $\mathbf{t}$, notée $d(\mathbf{t})$, est le maximum des profondeurs des sommets qui le composent, en symboles $d(\mathbf{t}) = \max\{d(u) \mid u \in \mathbf{t}\}$. La profondeur d'une forêt $\mathbf{F}$, notée $d(\mathbf{F})$, est la plus grande profondeur des arbres qui la composent, c'est-à-dire $d(\mathbf{F}) = \max\{d(\mathbf{t}) \mid \mathbf{t} \in \mathbf{F}\}$.

Si maintenant u est un sommet d'un graphe G de $\mathbb{G}$, alors la profondeur de u dans G , toujours notée $d(u)$, est la profondeur de u dans l'arbre enraciné dans un cycle de

G qui le contient. La profondeur du graphe G , notée $d(G)$, est égale à la profondeur de la forêt constituée des arbres enracinés dans les cycles de G .

La profondeur d'un FDDS A , notée $d(A)$, est la profondeur de G_A .

Nous traduisons les propriétés évoquées ci-dessus d'un point de vue dynamique. Pour cela, nous définissons que f^0 est l'identité et que f^n , pour un entier $n > 0$, est la n -ème composée de f avec elle-même, où f est une fonction d'un ensemble dans lui-même. Ainsi, comme il existe toujours un chemin entre un sommet du graphe G_A et un cycle de G_A , nous déduisons que nous pouvons construire pour chaque sommet u un chemin infini $u = u_0 u_1 \dots$ tel que $f_A(u_i) = u_{i+1}$ pour tout i . Nous disons alors que u_{i+1} est le *successeur* de u_i . Ce chemin correspond au mot infini $f_A^0(u) f_A^1(u) \dots$ que nous appelons *exécution* de A à partir de u . Nous constatons également que les FDDS dont le graphe de dynamique n'est constitué que d'une unique composante connexe ont des exécutions similaires à partir d'un certain rang, et ce indépendamment de l'état initial. Plus formellement, il existe un entier $k \geq 0$ tel que pour tous les états u, v , il existe un entier $i \geq 0$ tel que $f^{k+i}(u) = f^{k+i}(v)$ pour tout entier $j \geq 0$. Ce type de FDDS est appelé *connexe*.

Remarquons que les états appartenant à un cycle de G_A apparaissent périodiquement dans leurs exécutions. En d'autres mots, si u est un état dans un cycle, alors il existe un entier $k > 0$ tel que $f_A^k(u) = u$. Ces sommets sont qualifiés de *cycliques* et ces k sont appelés *périodes* de u . Les sommets u qui ont 1 comme période sont appelés *points fixes*. Ainsi, chaque état de A est *ultimement périodique*, c'est-à-dire que pour chaque sommet u , il existe un entier $k \geq 0$ tel que $f^k(u)$ est cyclique. Les états dont la profondeur est strictement supérieure à 0, c'est-à-dire les états qui ne sont pas dans des cycles, sont appelés *transitoires* car ils n'apparaissent dans aucune exécution possible après la $d(A)$ -ème position. Ceci met en lumière le fait qu'un FDDS est composé de deux comportements différents, un cyclique et un transitoire.

Somme de FDDS Grâce aux différents outils introduits jusqu'à présent, nous pouvons vérifier efficacement que deux FDDS ont leurs graphes de dynamique isomorphes. Si cela nous permet d'identifier certaines propriétés, les informations que nous inférons demeurent trop globales, entravant les analyses plus fines de dynamiques. Par exemple, nous ne saurions dire si un phénomène observé n'est en réalité la conjonction de dynamiques sous-jacentes. Nous voulons donc définir des opérations rendant possible la décomposition d'un FDDS en plusieurs autres plus réduits, plus simples. Pour illustrer en quoi la décomposition serait intéressante, considérons le nombre 18645. Il peut être vu comme (ou décomposé en) une dizaine de milliers plus huit milliers plus six centaines plus quatre dizaines plus cinq unités. Si cette décomposition paraît évidente, c'est pourtant grâce à elle que nous additionnons rapidement des nombres. En prenant en compte cet exemple, la première opération que nous voulons introduire est la *somme* de deux FDDS. Pour la définir mathématiquement, nous utilisons la structure algébrique appelée *monoïde* (le lecteur est invité à consulter [4] pour une introduction aux notions d'algèbre).

Définition 1.8 (Monoïde). Soit $(M, \diamond)$ un couple où M est un ensemble (fini ou non)

et $\diamond$ est une *loi de composition interne*. Nous disons que $(M, \diamond)$ est un monoïde si et seulement si :

1. M est *clos* par $\diamond$, c'est-à-dire $x \diamond y$ est un élément de M pour tous $x, y \in M$,
2. M contient un élément *neutre*, c'est-à-dire qu'il existe un $e \in M$ vérifiant que pour tout $x \in M$ nous avons $e \diamond x = x \diamond e = x$,
3. $\diamond$ est *associative*, c'est-à-dire que $x \diamond (y \diamond z) = (x \diamond y) \diamond z$ pour tous $x, y, z \in M$.

Si de plus $\diamond$ est *commutative*, c'est-à-dire que $x \diamond y = y \diamond x$ pour tous $x, y \in M$, alors $(M, \diamond)$ est un monoïde commutatif.

Pour définir notre somme de FDDS, nous débutons en introduisant une sur $\mathbb{G}$, puis nous lui donnons une interprétation au sens de la dynamique. Grâce à cette approche, nous nous inspirons de la théorie des graphes utilisant généralement *l'union disjointe* comme addition. L'idée est intuitive, presque naïve. Elle consiste à construire le graphe $G_1 + G_2$ en juxtaposant le graphe G_1 et le graphe G_2 . Cette idée de juxtaposition est mathématiquement définie comme suit.

Définition 1.9 (Union disjointe de graphes). Soient $G_1 = (V_1, E_1)$ et $G_2 = (V_2, E_2)$ deux graphes. L'*union disjointe* de G_1 avec G_2 est le graphe $G_1 + G_2$ ayant pour ensemble de sommets $(V_1 \times \{1\}) \cup (V_2 \times \{2\})$. De plus, le graphe $G_1 + G_2$ contient un arc (u, v) si et seulement s'il existe $i \in \{1, 2\}$ tel que u et v sont des sommets de $(V_i \times \{i\})$ et que l'arc (u, v) appartient à E_i .

Par la définition précédente, un graphe $G = (V, E)$ peut être vu comme une somme de m composantes connexes ou, de façon équivalente, comme un multi-ensemble de composantes connexes. Cette équivalence est induite du fait que l'union disjointe est l'opération de somme usuellement associée aux multi-ensembles.

Ainsi, nous voulons que la somme de deux FDDS A et B ait pour graphe de dynamique celui correspondant à l'union disjointe de G_A et G_B . Ceci est possible par l'*exécution alternative* de FDDS, formellement définie comme suit.

Définition 1.10 (Exécution alternative). Soient A et B deux FDDS. Alors le FDDS $A + B$ est le FDDS dont l'ensemble d'états est l'union disjointe des états de A et de B . Sa fonction de transition f_{A+B} est définie comme suit :

$$f_{A+B}(u) = \begin{cases} f_A(u) & \text{si } u \in A \\ f_B(u) & \text{si } u \in B \end{cases}$$

Nous voyons que $(\mathbb{G}, +)$ est un monoïde avec pour élément neutre (le 0) le graphe vide. De même, $(\mathbb{D}, +)$ est un monoïde dont l'élément neutre est le FDDS vide. Constatons que ces deux monoïdes sont isomorphes.

Définition 1.11 (Isomorphisme de monoïdes). Soient $(M_1, \diamond)$ et $(M_2, \star)$ deux monoïdes. Soit $f : M_1 \rightarrow M_2$ une fonction de M_1 dans M_2 . La fonction f est appelée *morphisme de monoïdes* si et seulement si :

1. f se *distribue* sur $\diamond$, c'est-à-dire que $f(u \diamond v) = f(u) \star f(v)$ pour tous $u, v \in M_1$,

2. $f(e_1) = e_2$ avec e_1 le neutre de M_1 et e_2 celui de M_2 .

Un *isomorphisme* de monoïdes est un morphisme bijectif. Deux monoïdes sont *isomorphes* s'il existe un isomorphisme entre les deux.

Produit d'un FDDS avec un entier De plus, $(\mathbb{D}, +)$ est aussi un $\mathbb{N}$ -semi-module. En effet, en voyant le produit d'un entier n avec un FDDS A comme la somme de n copies de A , alors cette opération vérifie l'ensemble des conditions requises. Ces conditions sont que : $(n+m)(A+B) = nA+mA+nB+mB$ et que $(nm)A = n(mA)$ avec n et m des entiers naturels et A et B des FDDS. De ce fait, nous déduisons que la *forme générale* d'un FDDS A peut être vue comme $\sum_{i=1}^{m_A} a_i A_i$ avec $a_i > 0$ un entier positif, A_i un FDDS connexe non vide pour tout i et où, de plus, chaque A_i est différent de tous les A_j si $i \neq j$.

Produit de FDDS La seconde opération classique permettant de décomposer un entier naturel est la *multiplication*. Par exemple, nous décomposons le nombre 115 en 23 fois 5. Nous souhaitons donc définir un produit de FDDS. Dans cette optique et pour les mêmes raisons que précédemment, nous commençons par le définir pour les graphes fonctionnels.

Considérons $G_1 = (V_1, E_1)$ et $G_2 = (V_2, E_2)$ deux graphes fonctionnels. La théorie des graphes possède trois produits classiques ([27]) pour réaliser la multiplication de G_1 par G_2 . Tout d'abord le *produit cartésien*, noté $G_1 \square G_2$, définissant le graphe $G_3 = (V_3, E_3)$ dont l'ensemble de sommets est le produit cartésien des ensembles V_1 et V_2 , noté $V_1 \times V_2$, et dont l'ensemble des arcs est défini tel que si nous considérons u, u' deux sommets de V_1 et v, v' deux sommets de V_2 alors E_3 contient l'arc $(uv, u'v')$ si et seulement si u et u' sont le même sommet et que l'arc (v, v') appartient à E_2 ou que l'arc (u, u') appartient à E_1 et que v et v' sont le même sommet. Intuitivement, E_3 est le produit cartésien de V_1 avec E_2 union le produit cartésien de V_2 avec E_1 .

La deuxième multiplication est appelée le *produit direct* ou produit tensoriel¹, noté $G_1 \times G_2$. Elle définit un graphe $G_4 = (V_4, E_4)$ dont l'ensemble des sommets est $V_1 \times V_2$ et contenant un arc $(uv, u'v')$ si et seulement si G_1 contient l'arc (u, u') et G_2 contient l'arc (v, v') .

Enfin, la troisième et dernière multiplication possible est le *produit fort*, noté $G_1 \boxtimes G_2$, définissant le graphe $G_5 = (V_5, E_5)$ et ayant pour ensemble de sommets $V_1 \times V_2$ et pour ensemble d'arcs $E_3 \cup E_4$. Autrement dit, le produit fort est la synthèse du produit cartésien et du produit direct.

Comme le montrent les exemples de la Figure 1.12, les graphes fonctionnels ne sont clos ni par le produit cartésien ni par le produit fort. Et puisque nous souhaitons conserver la structure de monoïde, ces deux opérations sont donc disqualifiées. La question est donc : est-ce que $\mathbb{G}$ est clos par le produit direct? La réponse est oui. En effet, pour que le sommet (u', v') soit le successeur du sommet (u, v) , il faut que u' soit le successeur de u et v' le successeur de v . Et comme u et v n'ont qu'un seul successeur, il n'existe qu'un unique sommet (u', v') . Remarquons de plus que nous

1. Ce produit est aussi le produit de Kronecker de leurs matrices d'adjacence.

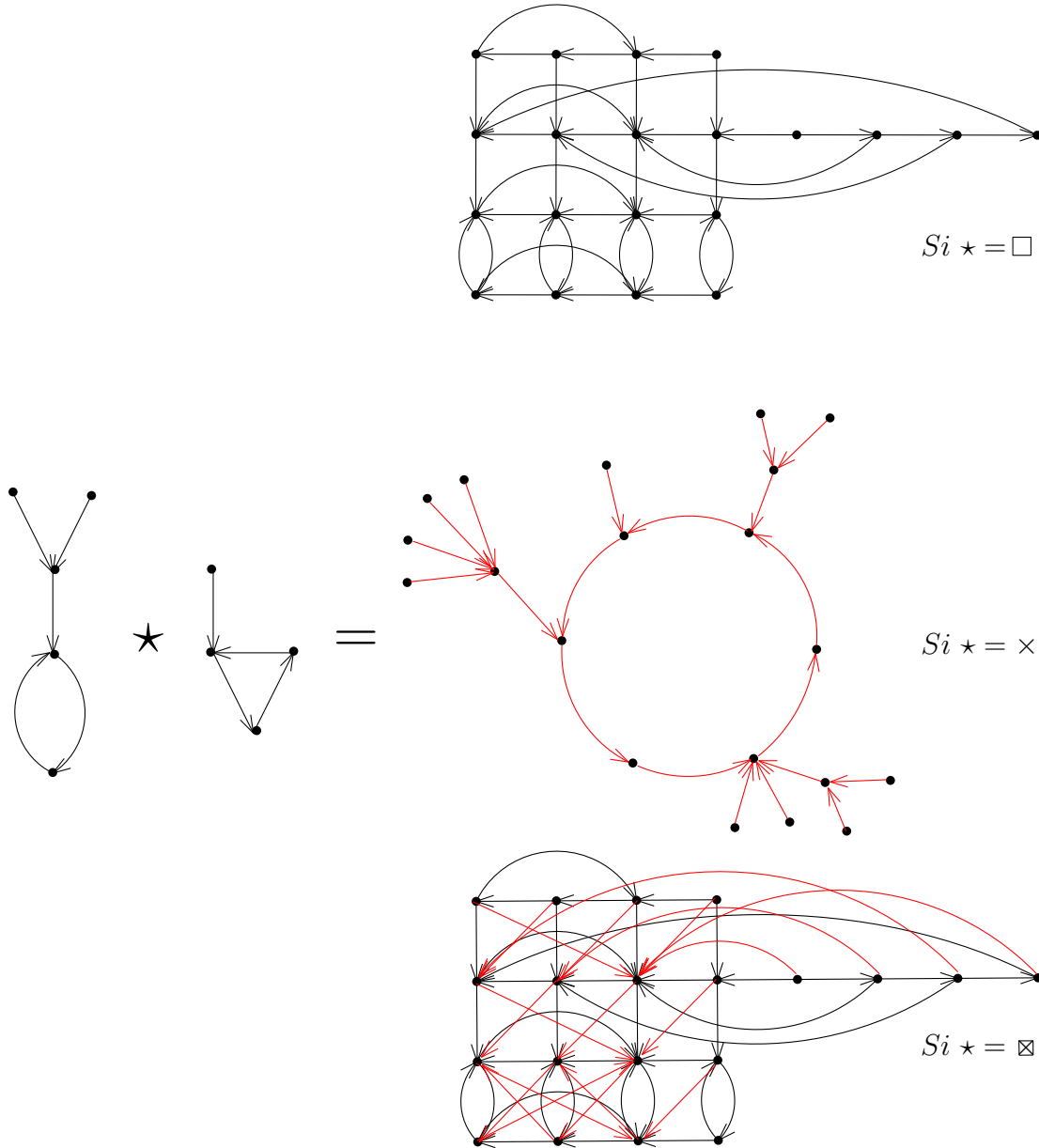


FIGURE 1.12. – Exemple du produit cartésien, direct et fort. À gauche se trouvent les graphes étant multipliés et à droite se trouvent leurs produits cartésiens (en haut), directs (au milieu) et forts (en bas). Les arcs issus du produit cartésien sont noirs tandis que ceux du produit direct sont rouges.

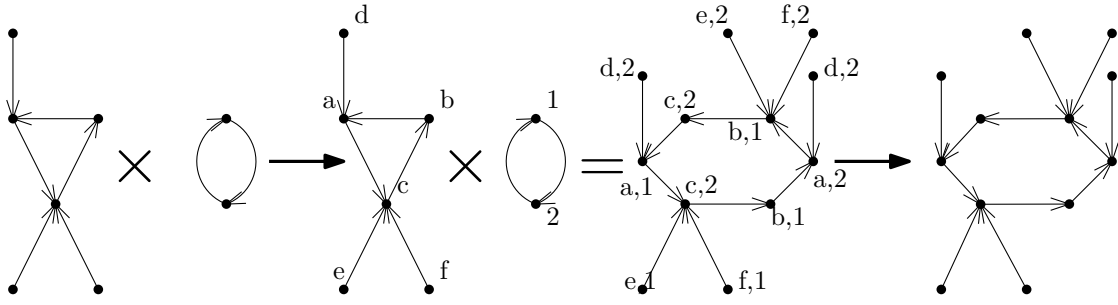


FIGURE 1.13. – Exemple de produit direct. La multiplication de tout à gauche représente les graphes à isomorphisme près. Pour calculer leur produit, nous commençons par donner des noms temporaires à leurs sommets. Ceci est illustré par la multiplication au centre à gauche. Puis nous calculons leur produit direct avec ces noms temporaires, le résultat est le graphe au centre à droite. Enfin, nous prenons ce dernier graphe à isomorphisme près, ce qui est donné par le graphe tout à droite.

pouvons calculer le produit direct de deux éléments G_1, G_2 de $\mathbb{G}$ en temps linéaire sur le nombre de sommets de G_1 fois celui de G_2 . La Figure 1.13 montre un exemple montrant comment calculer le produit direct de graphes à isomorphisme près.

De là, nous constatons que $(\mathbb{G}, \times)$ respecte toutes les conditions pour être un monoïde commutatif dont l'élément neutre (le 1) est le point fixe (le graphe constitué seulement d'un unique cycle de longueur 1). De plus, ce produit peut être interprété en termes de dynamique comme l'*exécution synchrone* de deux FDDS.

Définition 1.14 (Exécution synchrone). Soient A et B deux FDDS. Alors l'exécution synchrone de A et B , notée $A \times B$, est le FDDS ayant pour ensemble d'états $A \times B$ et pour fonction de transition $f_{A \times B}$ définie telle que $f_{A \times B}(uv) = (f_A(u)f_B(v))$.

Nous remarquons que $(\mathbb{D}, \times)$ est un monoïde commutatif isomorphe à $(\mathbb{G}, \times)$. Nous sommes maintenant capables de décomposer un FDDS comme somme ou produit de deux FDDS.

Semi-anneau des FDDS Nous voulons maintenant "mélanger" les deux opérations. Pour illustrer ce propos, reprenons le nombre 115. Nous savons que $115 = 5 \cdot 23$. Mais pour réaliser ce calcul, la méthode enseignée aux écolières et écoliers françaises et français nécessite l'utilisation du signe plus. En effet, elles et ils apprennent que 23 est comme $20 + 3$. De là, ces élèves réalisent $5 \cdot (20 + 3) = (5 \times 20) + (5 \times 3)$. Nous voulons donc distribuer le produit sur la somme. Et à cette fin, nous nous reposons sur la structure algébrique appelée *semi-anneau*.

Définition 1.15 (Semi-anneau). Soit $(M, \oplus, \star)$ un triplet d'éléments où M est un ensemble, $\oplus$ est une addition et $\star$ un produit.

Le triplet $(M, \oplus, \star)$ est un *semi-anneau* si et seulement si :

1. État de l'art et définitions – 1.1. Algèbre des systèmes dynamiques finis déterministes

1. $(M, \oplus)$ est un monoïde commutatif qui a pour élément neutre $\mathbf{0}$,
 2. $(M, \star)$ est un monoïde qui a pour élément neutre $\mathbf{1}$,
 3. le neutre additif est *absorbant* sous $\star$, c'est-à-dire que $\mathbf{0} \star a = a \star \mathbf{0} = \mathbf{0}$ pour tout a de M ,
 4. $\star$ se distribue sur $\oplus$, c'est-à-dire que $(a \oplus b) \star c = (a \star c) \oplus (b \star c)$ pour tous a, b, c de M . Par convention $\star$ est prioritaire sur $\oplus$, donc $(a \star c) \oplus (b \star c) = a \star c \oplus b \star c$.
- Si en plus $(M, \star)$ est commutatif, alors nous disons que $(M, \oplus, \star)$ est un semi-anneau commutatif.

Cette définition et tout ce qui précède prouvent le résultat suivant.

Théorème 1.16 ([13]). $(\mathbb{D}, +, \times)$ est un semi-anneau commutatif.

Nous constatons de plus que $(\mathbb{G}, +, \times)$ est aussi un semi-anneau commutatif.

Définition 1.17 (Isomorphisme de semi-anneaux). Soient $(M_1, \oplus, \star)$ et $(M_2, \boxplus, \star)$ deux semi-anneaux. Soit f une fonction de M_1 dans M_2 . La fonction f est appelée *morphisme de semi-anneaux* si et seulement si f est un morphisme de monoïdes entre $(M_1, \oplus)$ et $(M_2, \boxplus)$ et un morphisme de monoïdes entre $(M_1, \star)$ et $(M_2, \star)$.

La fonction f est appelée *isomorphisme* si et seulement si f est un morphisme bijectif. Deux semi-anneaux sont dits *isomorphes* si et seulement s'il existe un isomorphisme de l'un à l'autre.

Théorème 1.18. $(\mathbb{D}, +, \times)$ est isomorphe à $(\mathbb{G}, +, \times)$.

De ce théorème, nous ne distinguerons plus les FDDS de leur graphe de dynamique et ne parlerons plus que d'union disjointe pour la somme et de produit direct pour la multiplication.

Équation polynomiale de FDDS À chaque semi-anneau est associé un semi-anneau de polynômes. Ainsi, nous voyons la décomposition de FDDS comme des équations polynomiales. Par exemple, étant donné un FDDS B , savoir s'il est décomposable sous la somme revient à se demander si l'équation

$$X + Y = B$$

admet une solution non triviale, c'est-à-dire $X, Y \neq 0$. Tandis que savoir si B est décomposable par le produit revient à se demander s'il existe $X, Y \neq 1$ tels que

$$XY = B.$$

Mais cela nous permet aussi de formuler des questions plus précises que la décomposition. Par exemple, en considérant deux FDDS A et B , la question : A est-il un sous-FDDS de B , c'est-à-dire un sous-multi-ensemble des composantes connexes de B ? est reformulée par l'équation

$$A + X = B$$

admet-elle une solution? La question : B est-elle l'exécution synchrone de A avec un autre FDDS? Se reformule en l'équation

$$AX = B$$

est-elle solvable? Cela nous ouvre également le chemin à des questions plus élaborées permettant des analyses plus fines. Afin de donner un exemple, nous définissons que A^0 est le point fixe et que $A^k = \prod_{i=1}^k A$ pour tout FDDS A et tout entier $k > 0$. De là, nous définissons $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS, c'est-à-dire que X et tous les A_i sont des FDDS. Et nous pouvons nous demander si, pour un B donné, l'équation

$$P(X) = B$$

possède une solution. Cependant, rien n'oblige à se contenter d'une seule variable dans le polynôme ou d'avoir le côté droit de l'équation constant. En d'autres mots, nous sommes en mesure de formuler des problèmes du type : l'équation

$$P(X_1, \dots, X_n) = Q(X_1, \dots, X_n),$$

avec P et Q des polynômes de FDDS sur n variables, a-t-elle une solution? Dans la section suivante, nous présenterons des résultats de calculabilité et de complexité liés aux différentes équations introduites précédemment. Un tableau récapitulatif des résultats est présenté à la fin de ce chapitre.

Notons qu'aucune de ces questions n'est triviale, car ce semi-anneau (et tous ceux que nous étudierons dans ce manuscrit) n'est ni un anneau, c'est-à-dire un semi-anneau avec opposé additif, (un semi-anneau tel que pour tout A il existe B vérifiant $A + B = 0$), ni un corps, c'est-à-dire un anneau avec inverse multiplicatif (un anneau tel que pour tout $A \neq 0$, il existe B vérifiant $A \times B = C_1$). Donc en particulier, il n'existera pas toujours des X tels que $A + X = B$ ou $A \times X = B$, pour A et B donnés.

1.2. Résultats de calculabilité et complexité

Encodage des polynômes Dans cette section, nous étudierons les équations polynomiales présentées dans la section précédente des plus générales aux plus spécifiques. Pour cela, nous devons expliquer comment nous encodons nos entrées. Nous choisissons qu'un FDDS A soit encodé par son graphe donné sous forme explicite. Un FDDS A connexe sera alors représenté par un unique sommet de son cycle et chaque sommet de A aura un pointeur vers son successeur et un tableau de pointeurs vers ses prédécesseurs. Parallèlement, les sommets d'un arbre t auront le même encodage et cet arbre sera représenté par sa racine. Les FDDS généraux et les forêts seront encodés par un tableau de leurs composantes connexes.

Les monômes multivariés $(AX_1^{k_1} \cdots X_n^{k_n})$ seront encodés par un couple (A, t) où t sera un tableau de longueur n , indicé de 1 à n , tel que $t[i] = k_i$ pour tout i . Comme un polynôme P à n variables peut être vu comme une somme de m monômes

1. État de l'art et définitions – 1.2. Résultats de calculabilité et complexité

$M_0, \dots, M_{m-1}$, nous représentons les polynômes par un tableau T tel que $T[i] = M_i$.

Comme nous travaillons principalement avec des polynômes univariés, nous leur définissons un encodage particulier. Dans ces derniers, les monômes univariés (AX^k) seront encodés par un couple (A, k) et les polynômes univariés seront encodés par des tableaux t de monômes dont le coefficient est différent de 0. De plus, ces tableaux seront triés par ordre croissant des puissances des monômes.

Résultat d'indécidabilité Soient P et Q des polynômes de FDDS sur n variables. Commençons donc par les équations de la forme $P(X_1, \dots, X_n) = Q(X_1, \dots, X_n)$.

Théorème 1.19 ([12]). *Soient P et Q des polynômes de FDDS sur n variables. Décider si l'équation $P(X_1, \dots, X_n) = Q(X_1, \dots, X_n)$ admet une solution est un problème semi-décidable et non-décidable.*

Pour montrer ce théorème, constatons en premier lieu qu'une machine de Turing est en mesure d'évaluer les deux polynômes pour n'importe quelles affectations. Donc, s'il existait une solution, nous serions en mesure de construire une machine qui s'arrêterait et renverrait *Vrai*, car $\mathbb{D}$ est un ensemble récursivement énumérable. Pour conclure, cette preuve [12] introduit une réduction depuis la résolution des systèmes d'équations diophantiennes vers le problème de l'arrêt, en d'autres mots, la résolution des équations polynomiales de FDDS est plus générale que celle des équations polynomiales d'entiers naturels, le problème source étant lui-même indécidable du fait d'une réduction depuis la réponse au 10-ème problème de Hilbert [37].

Pour ce faire, la référence débute par la preuve que $(\mathbb{B}_1, +, \times)$ est un sous-semi-anneau de $(\mathbb{D}, +, \times)$, avec $\mathbb{B}_1$ l'ensemble des *collections de points fixes* à isomorphisme près, c'est-à-dire l'ensemble des FDDS dont tous les états sont des points fixes.

Définition 1.20 (Sous-semi-anneau). Soit $(M, \oplus, \star)$ un semi-anneau. Soit N un sous-ensemble de M . Alors $(N, \oplus, \star)$ est un sous-semi-anneau de $(M, \oplus, \star)$ si et seulement si :

1. N est clos par $\oplus$ et $\star$,
2. N contient le neutre additif et multiplicatif.

Notons bien que tout sous-semi-anneau est aussi un semi-anneau.

Les exemples suivants illustrent cette notion de sous-semi-anneau. Tout d'abord, $(\mathbb{B}, +, \times)$ est un sous-semi-anneau de $(\mathbb{D}, +, \times)$ où $\mathbb{B}$ est l'ensemble des sommes de cycles, aussi appelé l'ensemble des *permutations*. Cependant, notons bien que ces objets décrivent les partitions et non les permutations au sens combinatoire. Cette appellation est introduite originellement, car ces objets sont les classes d'équivalence des permutations induites par l'isomorphisme de graphes. Et nous l'utilisons pour des raisons historiques.

Un autre exemple, $(\mathbb{D}^*, +, \times)$ est un sous-semi-anneau de $(\mathbb{D}, +, \times)$, où $\mathbb{D}^*$ est la clôture par $+$ de l'ensemble des *dendrons*.

Définition 1.21 (Dendron). Un *dendron* est un FDDS connexe dont le cycle est le point fixe.

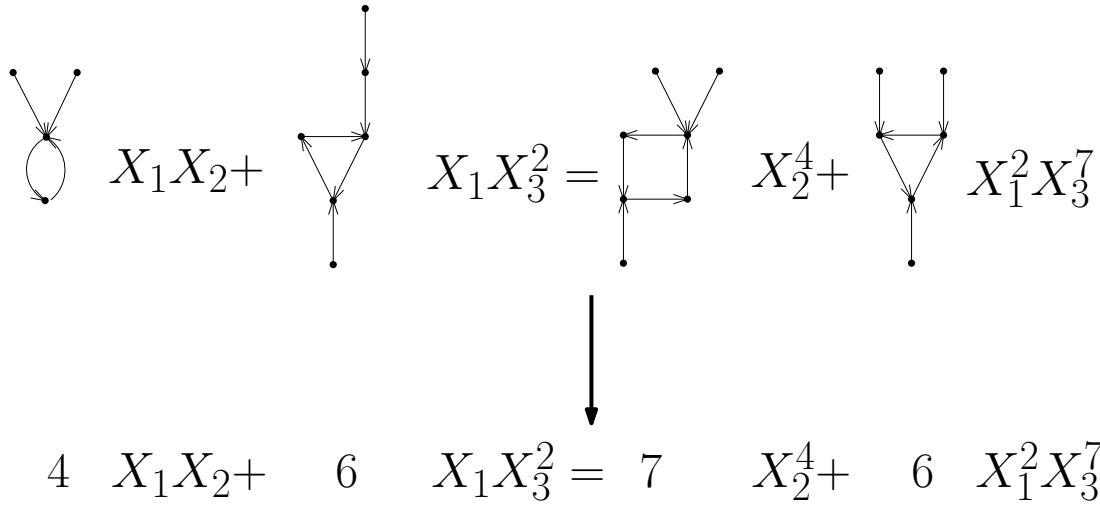


FIGURE 1.22. – Exemple de transformation d’une équation sur les FDDS en équation sur les tailles. Les polynômes P et Q sur les FDDS sont respectivement du côté gauche et droit de l’égalité du haut. Les polynômes P' et Q' sur les entiers naturels sont quant à eux respectivement les côtés gauche et droit de l’équation du bas.

Un dernier exemple est l’ensemble des FDDS ayant une profondeur au plus d muni de $+$ et $\times$ qui est un sous-semi-anneau de $(\mathbb{D}, +, \times)$ et ce pour tout entier $d \geq 0$. Nous constatons que $(\mathbb{B}_1, +, \times)$ est aussi un sous-semi-anneau de tous les sous-semi-anneaux introduits ci-dessus. Notons que $(\mathbb{B}_1, +, \times)$ est un sous-semi-anneau de tous les sous-semi-anneaux de $(\mathbb{D}, +, \times)$, car $\mathbb{B}_1$ est la clôture par somme du point fixe.

Revenons à la preuve du Théorème 1.19. Il est montré dans [12] que $(\mathbb{B}_1, +, \times)$ est égal au semi-anneau des entiers naturels munis des opérations usuelles. En effet, il est montré que la *fonction taille*, notée $|X|$, qui associe à chaque FDDS son nombre d’états, est un morphisme et un isomorphisme depuis respectivement $(\mathbb{D}, +, \times)$ et $(\mathbb{B}_1, +, \times)$ vers le semi-anneau des entiers naturels muni des opérations usuelles. Ainsi, partant de P et Q deux polynômes de FDDS à n variables, nous construisons P' et Q' deux polynômes sur les entiers naturels en appliquant la fonction taille à chaque coefficient. Cette équation est appelée *équation sur les tailles*. Un exemple d’une telle transformation est présenté dans la Figure 1.22.

De ces propriétés, il est observé dans cet article que :

1. si P et Q sont des polynômes sur les FDDS, alors nous avons que l’équation $P(X_1, \dots, X_n) = Q(X_1, \dots, X_n)$ admet une solution si et seulement si c’est aussi le cas de l’équation $P'(X_1, \dots, X_n) = Q'(X_1, \dots, X_n)$.
2. si P et Q sont des polynômes sur $\mathbb{B}_1$, alors l’équation $P(X_1, \dots, X_n) = Q(X_1, \dots, X_n)$ admet une solution si et seulement si l’équation $P'(X_1, \dots, X_n) = Q'(X_1, \dots, X_n)$ en admet aussi une.

NP complétude des systèmes d'équations polynomiales avec côté droit constant Pour atteindre la décidabilité, il est suffisant de définir l'un des deux côtés de l'équation comme constant, par convention le côté droit. En d'autres termes, il s'agit des problèmes de décision de l'existence de solutions aux équations de la forme $P(X_1, \dots, X_n) = B$, avec P un polynôme de FDDS à n variables, cette contrainte induisant que seul un ensemble fini de solutions est possible. En effet, il existe seulement un nombre fini de solutions à l'équation sur les tailles des FDDS et un nombre fini de FDDS de chaque taille. Ce dernier suit la suite A001372 de l'*encyclopédie en ligne des séquences d'entiers* [1]. Nous sommes même en mesure de montrer que ce cas est dans la classe de complexité NP, car il est possible de vérifier que des FDDS $Y_1, \dots, Y_n$ sont bien solutions de $P(Y_1, \dots, Y_n) = B$ en temps polynomial sur m , la taille de B . Pour cela, il suffit d'évaluer P en s'assurant après chaque calcul que la taille du résultat total n'excède pas m puis de vérifier l'isomorphisme en $\mathcal{O}(m)$. Néanmoins, sous l'hypothèse que P est différent de NP, il nous est impossible de résoudre ce problème en temps polynomial. En effet, si nous considérons B un FDDS et P un polynôme de FDDS à n variables, alors décider de l'existence d'une solution à l'équation $P(X_1, \dots, X_n) = B$ est NP-complet par [7].

Pour démontrer cette affirmation, la référence commence par prouver que décider si un système d'équations de FDDS admet au moins une solution est NP-complet. Pour cela, elle réalise une réduction *many-one polynomial* depuis le problème "1 parmi 3-SAT", en abrégé 1,3-SAT, qui est NP-complet ([24, LO4]). Pour rappel, ce problème consiste en la décision de l'existence d'un modèle d'une formule SAT donnée en 3-CNF, nommée φ , telle que chaque clause ne contient qu'un unique littéral à *Vrai*.

Dans cette réduction, elle commence par transformer φ en un système d'équations diophantiennes en associant à chaque variable x_i de φ deux variables y_i et z_i du système. Intuitivement, y_i joue le rôle du littéral x_i et z_i celui du littéral $\neg x_i$. Pour garantir cette propriété, elle ajoute au système l'équation $y_i + z_i = 1$, ceci impliquant que dans une solution du système, une variable a la valeur 1 tandis que l'autre vaut 0. Elle ajoute également l'équation $l_{k,1} + l_{k,2} + l_{k,3} = 1$ pour chacune des clauses $c_k = (l_{k,1} \vee l_{k,2} \vee l_{k,3})$ de φ , avec $l_{k,i}$ la variable associée au littéral (y_j si $l_{k,i} = x_j$ ou z_j si $l_{k,i} = \neg x_j$). Nous voyons directement que ce système d'équations possède une solution si et seulement si la formule φ est satisfaisable avec seulement un littéral vrai par clause. Et en remplaçant alors les 1 par des points fixes, elle déduit que nous avons une réduction many-one polynomiale entre 1,3-SAT et le problème souhaité, ceci concluant la preuve de la proposition suivante.

Proposition 1.23 ([7]). *Décider si un système d'équations polynomiales de FDDS et à côté droit constant possède au moins une solution est NP-complet.*

NP Complétude des équations polynomiales avec côté droit constant La méthode nous permettant de passer des systèmes d'équations aux équations repose sur les éléments *irréductibles*. Pour rappel, un élément u d'un semi-anneau $(M, \diamond, \star)$ est irréductible si et seulement s'il n'est ni inversible, c'est-à-dire que $u \star v \neq 1_M$ pour tout v dans M avec 1_M le neutre multiplicatif du semi-anneau, ni produit de deux

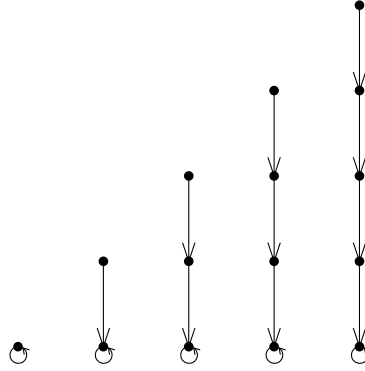


FIGURE 1.24. – Les FDDS D_0, D_1, D_2, D_3, D_4 de gauche à droite.

éléments non-inversibles. Par exemple, dans le semi-anneau des entiers naturels avec la somme et le produit usuels, nous avons que 2 est irréductible, car 2 est seulement égal à 1 fois 2. Cependant, 4 ne l'est pas, car 4 est 2 fois 2 et que 2 est non-inversible. Aussi, 1 n'est pas irréductible, car 1 est inversible.

Pour passer des systèmes d'équations aux équations, l'équation i est multipliée par D_i , le dendron dont la partie transitoire est le chemin simple de longueur i . Un exemple est proposé dans la Figure 1.24. Ces FDDS sont constructibles en temps polynomial sur le nombre d'équations. Puis [7] fait la somme de toutes les équations du système. Remarquons que D_i est irréductible pour tout entier $i > 0$. En effet, s'il existe deux FDDS X, Y tels que $XY = D_i$, alors X ou Y est une permutation. De fait, le produit d'un sommet du comportement transitoire avec n'importe quel sommet produit un sommet transitoire. De plus, X et Y sont tous les deux connexes (car le produit avec un non-connexe est non-connexe) et la longueur des cycles de X et Y est 1, car la longueur du cycle de XY est supérieure ou égale à celle du cycle de X et à celle du cycle de Y . Nous reviendrons ultérieurement plus en détail sur le comportement du produit de deux FDDS. Nous constatons que tous les FDDS issus de la somme de quelques D_i différents sont irréductibles.

Si une solution existe pour les systèmes, alors il en existe aussi une pour l'équation. Et pour l'autre direction, nous constatons que toutes les variables ont une valeur qui est soit 0 soit 1. En effet, pour tout i , multiplier un D_i par un FDDS X qui n'est ni 0 ni 1 produit soit plusieurs copies de D_i soit un FDDS dont au moins une des composantes connexes n'est pas un D_j pour tout j . Or, la partie droite de notre équation est $\sum_{i=1}^m D_i$, avec m le nombre d'équations du système. Elle est donc irréductible ainsi que tous les FDDS composés de sous-ensembles de $\sum_{i=1}^m D_i$. Nous déduisons que la partie de l'équation générant le D_i de la partie droite est celle qui est multipliée par D_i . Donc, la solution de l'équation est aussi une solution du système.

Théorème 1.25 ([7]). *Étant donné un polynôme P à n variables et B un FDDS, décider s'il existe une solution à $P(X_1, \dots, X_n) = B$ est NP-complet, et ce même si P est linéaire.*

Relevons que nous sommes en mesure d'appliquer le même raisonnement en utilisant des cycles connexes de longueur première en lieu et place des D_i . Ainsi, le Théorème 1.25 est aussi vrai si les coefficients de P et B sont des permutations.

1. État de l'art et définitions – 1.2. Résultats de calculabilité et complexité

$\times$							
							
							
							
							
							
							
							

FIGURE 1.26. – Table de multiplication de certains des FDDS d'au plus trois états (provenant de [11])

À ce jour, nous ne savons pas si le problème de décider si une équation polynomiale avec un nombre de variables borné admet une solution est NP-complet ou bien polynomial; et ce même pour l'équation $AX = B$. Il nous arrivera de dire “décider si A divise B ” en lieu et place de “décider s'il existe une solution à l'équation $AX = B$ ” et de substituer “l'équation $AX = B$ admet au moins une solution” par “ A divise B ”. La difficulté de ce problème vient de la richesse du comportement induit par le produit direct. La Figure 1.26 illustre ce propos.

Propriétés du produit de cycles Avant de poursuivre la présentation des différents résultats de difficulté connus, nous introduisons quelques propriétés fondamentales sur le produit direct que les démonstrations de ces derniers utilisent.

Proposition 1.27 ([27]). *Soient G et H deux graphes et soient g, g' deux sommets de G et h, h' deux sommets de H . S'il existe un chemin (non nécessairement simple) de longueur ℓ de g vers g' dans G et de h vers h' dans H , alors il existe un chemin de longueur ℓ de (g, h) vers (g', h') dans $G \times H$. Le plus petit des ℓ possibles est la distance de (g, h) vers (g', h') dans $G \times H$. S'il n'existe pas de chemin de g vers g' ou de h vers h' , alors la distance de (g, h) vers (g', h') est infinie.*

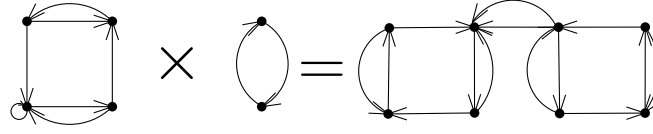


FIGURE 1.30. – Produit de composantes fortement connexes. Nous voyons que le produit des deux graphes engendre une unique composante fortement connexe de 8 sommets. Pour ces graphes, nous avons que $g(G_1)$ est $\gcd(1, 2, 4) = 1$ et $g(G_2) = 2$. Donc $(1 \cdot 2) / \text{lcm}(1, 2) = 1$.

Cette propriété s'étend au produit de n graphes.

Proposition 1.28 ([27]). *Soient $G_1, \dots, G_n$ des graphes, soit $G = G_1 \times \dots \times G_n$ et soient $g = (g_1, \dots, g_n), g' = (g'_1, \dots, g'_n)$ deux sommets de G avec g_i, g'_i des sommets de G_i . La distance dans G entre g et g' est le plus petit ℓ tel que pour tout i , il existe un chemin de g_i vers g'_i dans G_i de longueur ℓ .*

Notons bien que ces propriétés sont définies indépendamment du fait que les graphes soient orientés ou non. Ceci implique que la profondeur du produit de FDDS est le maximum des profondeurs des termes. Rappelons que, puisque le produit se distribue sur la somme, pour comprendre le comportement du produit, il nous suffit d'étudier le cas des FDDS connexes.

Dans le cas non-orienté, Weichsel démontre qu'étant donné deux graphes connexes G et H , le résultat de $G \times H$ est connexe si et seulement si G ou H contient un cycle de longueur impaire [52]. Il est connu que nous pouvons généraliser au cas des produits avec plus de deux termes. Nous aurons alors que si tous les termes possèdent au moins deux sommets, alors le produit aura 2^{n-1} composantes connexes, avec n le nombre de termes ne contenant que des cycles de longueur paire, c'est-à-dire que n est le nombre de graphes bipartis avec un minimum de deux sommets. Dans le cas où au moins un terme ne possède qu'un unique sommet et aucune arête, alors nous aurons $|G_1| \times \dots \times |G_n|$ composantes connexes n'étant que des sommets sans arête. Cependant, cette propriété devient fausse dans le cas des graphes orientés.

Proposition 1.29 ([38]). *Soient $G_1, \dots, G_n$ des graphes orientés composés d'une unique composante fortement connexe. Nous posons $g(G_i)$, le gcd de toutes les longueurs de cycles présentes dans G_i . Alors, $G_1 \times \dots \times G_n$ contient un nombre de composantes égal à :*

$$\frac{\prod_{i=1}^n g(G_i)}{\text{lcm}(g(G_1), \dots, g(G_n))}$$

Un exemple illustrant cette propriété est proposé en Figure 1.30.

Ainsi, dans le cas des FDDS, comme chaque composante connexe ne contient qu'un unique cycle, nous déduisons que le produit $G_1 \times \dots \times G_n$ produira

$$\left(\prod_{i=1}^n p_i \right) / \text{lcm}(p_1, \dots, p_n) = \gcd(p_1, \dots, p_n)$$

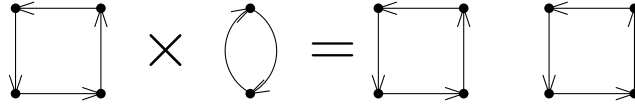


FIGURE 1.32. – Produit de composantes connexes. Nous voyons que le produit des deux graphes induit 2 composantes connexes de 4 sommets.

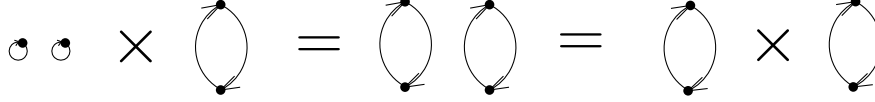


FIGURE 1.33. – Exemple de deux factorisations d'un FDDS en irréductibles.

composantes connexes, avec p_i la longueur du cycle de G_i . Il nous faut maintenant étudier la longueur des cycles de ces composantes connexes. Pour cela, remarquons que si nous considérons A et B deux FDDS connexes et a et b deux sommets de leurs cycles respectifs, alors le sommet (a, b) a une période minimale de $\text{lcm}(p_a, p_b)$ avec p_a et p_b les longueurs de cycle de respectivement A et B . Nous en concluons le théorème suivant.

Proposition 1.31 ([38]). *Soient A et B deux FDDS connexes de longueurs de cycle p_a et p_b respectivement, alors $A \times B$ produit $\text{gcd}(p_a, p_b)$ composantes connexes dont les cycles sont de longueur $\text{lcm}(p_a, p_b)$.*

Un exemple illustrant cette propriété est proposé en Figure 1.32.

La propriété donnée dans ce théorème décrit complètement le comportement cyclique des FDDS. Nous présenterons une technique pour analyser le comportement transitoire dans la Section 1.3.

Primalité dans le semi-anneau des FDDS En s'appuyant sur la propriété précédente, nous constatons que le semi-anneau $(\mathbb{D}, +, \times)$ n'est pas factoriel, c'est-à-dire que la décomposition en irréductibles n'est pas unique. En effet, le produit de deux points fixes multiplié par un cycle de longueur 2 est égal au cycle de longueur 2 multiplié par lui-même. Cette égalité est illustrée dans la Figure 1.33.

Notation 1.34 (Cycle). Soit $i > 0$ un entier. Le cycle de longueur i est noté C_i .

Ainsi, l'exemple donné ci-dessus se réécrit $(C_1 + C_1) \times C_2 = C_2 \times C_2$ ou plus directement, $2C_2 = C_2^2$ où 2 remplace $C_1 + C_1$.

Un exemple de semi-anneau factoriel est celui des entiers naturels munis des opérations de somme et produit usuelles. Et dans ce cas, la décomposition en irréductibles est mieux connue sous le nom de décomposition en facteurs *premiers*. Ceci s'explique, car dans ce semi-anneau, tout élément premier est aussi irréductible. Pour rappel, un élément p d'un semi-anneau $(M, \diamond, \star)$ est dit premier s'il est non nul, non inversible et tel que pour tous a, b de M , s'il existe k tel que $p \star k = a \star b$, alors il existe k' tel que $p \star k' = a$ ou $p \star k' = b$. Notons qu'il existe des semi-anneaux ayant des éléments

premiers non-irréductibles et des semi-anneaux possédant des éléments irréductibles mais non premiers. Un exemple du premier cas est donné dans la Sous-section 6.1.1 tandis que le semi-anneau $(\mathbb{D}, +, \times)$ illustre le second cas. En effet, comme déjà évoqué, $2C_2 = C_2^2$. Cependant, 2 ne divise pas C_2 (car 2 fois n'importe quel FDDS non vide génère au moins deux composantes connexes). En réalité, le semi-anneau $(\mathbb{D}, +, \times)$ est même un exemple d'un cas plus extrême. Par un simple argument de comptage, nous prouvons que le quotient de la division du nombre de FDDS irréductibles de taille n par celui des FDDS de taille n est asymptotiquement 1 [17], c'est-à-dire que la probabilité de tirer aléatoirement un irréductible parmi les FDDS de taille n tend vers 1 lorsque n tend vers l'infini.

Théorème 1.35 ([47, 45]). *Le semi-anneau $(\mathbb{D}, +, \times)$ ne contient aucun premier.*

Lors de la démonstration de ce résultat, nous établissons qu'aucun système dynamique non connexe n'est premier. Puis nous prouvons qu'aucun système dynamique connexe différent d'un dendron n'est premier. Dans ces deux cas, la stratégie utilisée est de considérer un FDDS A , de le multiplier par une certaine permutation P et de construire deux FDDS B et C tels que $AP = BC$ mais A n'est diviseur ni de B ni de C . Enfin, il est démontré dans [45] qu'aucun dendron n'est premier (la stratégie utilisée pour parvenir à ce résultat étant plus complexe, nous invitons le lecteur à consulter cette référence).

Algorithme de résolution et d'énumération des solutions

Approche par énumération de tous les FDDS Une méthode naïve pour résoudre les équations de la forme $P(X) = B$ de façon déterministe est d'énumérer l'ensemble des FDDS de taille au plus $|B|$. Pour parvenir à cela, un *code d'isomorphisme* entre FDDS est introduit dans [10]. Puis cette référence fournit un algorithme énumérant lesdits codes.

Définition 1.36 (Code d'isomorphisme). Soit Σ un alphabet, Σ^* l'ensemble des mots générables par Σ , M un ensemble et $\Phi : M \rightarrow \Sigma^*$ une fonction. Nous définissons que Φ décrit un code d'isomorphisme si et seulement si pour tous u, v de M l'égalité $\Phi(u) = \Phi(v)$ équivaut à ce que u soit isomorphe (ou plus simplement égal) à v . Le résultat de $\Phi(u)$ est appelé code d'isomorphisme de u .

L'article [10] commence par définir un code d'isomorphisme (sur les naturels) pour les arbres au sein d'une composante connexe; puis celui d'une composante connexe comme le vecteur des codes d'isomorphisme de ces arbres en rotation lexicographiquement minimale. Enfin, il caractérise le code des FDDS eux-mêmes comme le vecteur des codes des composantes connexes triées par ordre lexicographique. Afin d'énumérer les FDDS de taille n , il est tout d'abord montré dans [10] comment nous pouvons le faire dans le cas des composantes connexes de taille n via une succession de fusions et de séparations des arbres les composant. Un exemple de ce processus est donné dans la Figure 1.37. Puis la référence explique la manière dont nous pouvons

propager cet algorithme à tous les FDDS en calculant l'ensemble des partitions possibles de n et en appliquant la méthode précédente sur chaque élément de la partition. Nous en déduisons le théorème suivant.

Théorème 1.38 ([10]). *L'énumération de l'ensemble des FDDS de taille n requiert un délai entre la génération de deux éléments successifs en $\mathcal{O}(n^2)$.*

Ce théorème implique que le problème de l'énumération des FDDS appartient à la classe DelayP (le lecteur est invité à consulter [29, 48] pour de plus amples informations sur les classes de complexité liées aux problèmes d'énumération).

Définition 1.39 (Classe DelayP). Ensemble des problèmes d'énumération solvables par un algorithme dont le délai entre deux solutions est polynomial sur la taille des entrées.

L'algorithme de résolution que nous venons d'introduire, c'est-à-dire énumérer l'ensemble des FDDS X d'une certaine taille et vérifier que $P(X) = B$, peut directement être étendu pour résoudre les équations de la forme $P(X_1, \dots, X_n) = B$. En effet, il suffit de l'appliquer récursivement pour chaque variable.

Approche par systèmes d'équations Une approche plus sophistiquée, mais toujours "force brute", consiste à ne pas voir $AX = B$ comme une équation à une variable mais plutôt comme une équation à n variables $A(X_1 + \dots + X_n) = B$. L'idée est que chaque variable X_i est soit 0 soit différente des autres X_j (avec $i \neq j$). Comme il est aussi possible de diviser A en une somme de composantes connexes ($A = \sum_{i=1}^{m_A} A_i$ où tous les A_i sont connexes), l'équation $A(X_1 + \dots + X_n) = B$ peut se réécrire $\sum_{i=1}^{m_A} \sum_{j=1}^n A_i X_j = B$. Dès lors, cette équation peut être vue comme un système de nm_A équations :

$$\left\{ \begin{array}{l} A_1 X_1 = B_1 \\ A_1 X_2 = B_2 \\ \vdots \\ A_2 X_1 = B_n \\ A_2 X_2 = B_{n+1} \\ \vdots \\ A_{m_A} X_n = B_{nm_A} \end{array} \right.$$

où B_i est l'élément de coordonnée i dans $\vec{B}$, une des partitions possibles des composantes connexes de B en nm_A sous-multi-ensembles. Pour compléter cette approche, il faut faire de même avec toutes les partitions possibles des composantes connexes de B en nm_A sous-multi-ensembles.

Un algorithme est introduit dans [13] afin de résoudre les équations de la forme $A_i X_j = B_k$, avec A_i un FDDS connexe et B_k un FDDS dont les cycles sont tous de même

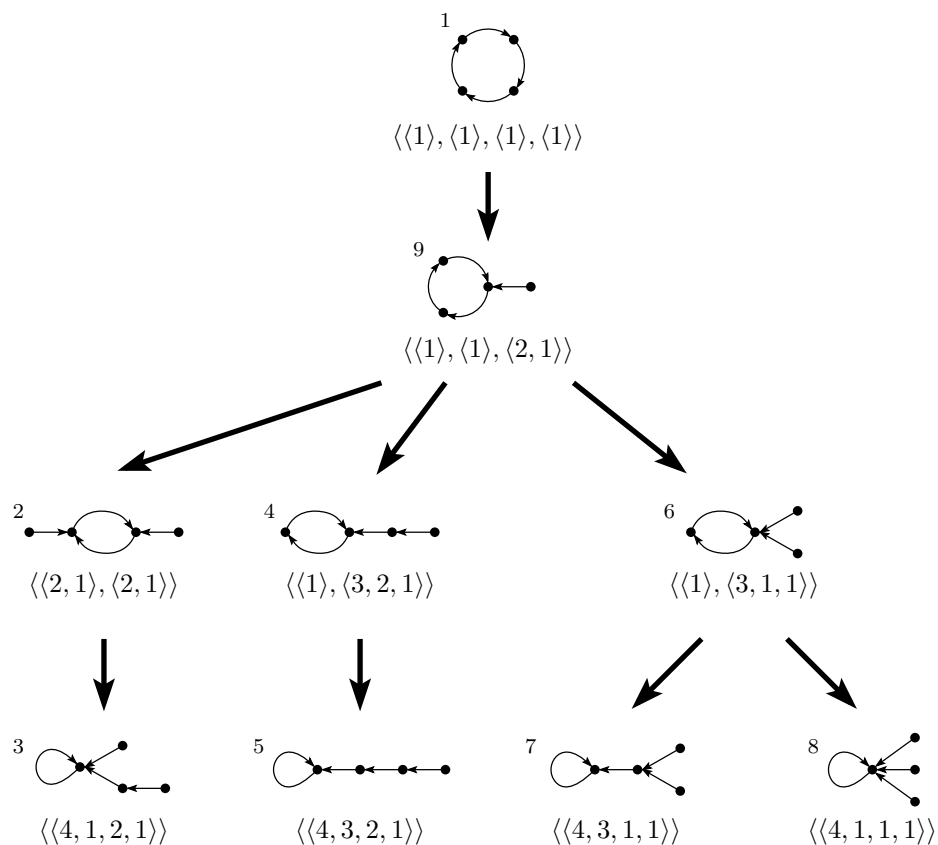


FIGURE 1.37. – Arbre d'énumération des composantes connexes de 4 sommets. Chaque sommet de l'arbre est composé d'un graphe (en haut) et du code d'isomorphisme de ce graphe (en bas). À chaque étape, nous fusionnons deux arbres enracinés dans des sommets voisins dans le cycle en un seul. Les opérations de séparation permettent de séparer un arbre en deux arbres enracinés dans les sommets voisins dans le cycle et ainsi de parcourir l'arbre dans le sens inverse des flèches (figure issue de [10]).

longueur, dans le cas où A_i et B_k sont des permutations, c'est-à-dire les équations de la forme $C_a X = q C_b$. Cette méthode permet également d'énumérer l'ensemble des solutions possibles de ces équations. Cependant, ces travaux ne montrent point que le problème est dans la classe DelayP, car ils ne prouvent aucunement que nous sommes à même de trouver une solution en temps polynomial.

Dans [22, 46], la complexité de l'algorithme est améliorée en utilisant des *diagrammes de décision multivalués*, outils permettant de représenter plus succinctement un ensemble de solutions (le lecteur est invité à consulter [39, 46] pour de plus amples informations). Finalement [15, 46] réussissent à démontrer le théorème suivant :

Théorème 1.40 ([15, 46]). *Décider si $C_a X = q C_b$ admet une solution est réalisable en temps polynomial, même si a et b sont donnés en binaire.*

Néanmoins [15] ne prouve pas que nous sommes en mesure de fournir une solution à l'équation $C_a X = q C_b$ en temps polynomial, et donc *a fortiori*, ne démontre aucunement qu'énumérer toutes les solutions de $C_a X = q C_b$ est dans DelayP, ce que nous prouverons toutefois dans la suite du manuscrit. Plus précisément, la question de la résolution sera traitée dans la Section 3.2 et celle de l'énumération dans la Sous-section 5.1.3.

Finalement, un algorithme de résolution des équations polynomiales multivariées où le polynôme est une somme de monômes univariés est fourni dans [14, 46]. Cette technique utilise une approche similaire à celle décrite ci-dessus, c'est-à-dire en décomposant une équation en un système d'équations de la forme $C_a X = q C_b$. Cet algorithme n'est pas prouvé polynomial et ne devrait pas l'être, sous l'hypothèse que $P \neq NP$. En effet, si par une analyse plus fine nous parvenions à montrer que la complexité de l'algorithme est polynomiale, alors $P = NP$ par le Théorème 1.25. En plus de la résolution, cet algorithme permet également d'énumérer l'ensemble des solutions avec un délai non prouvé polynomial et ne pouvant l'être si $P \neq NP$.

Approche avec le diviseur fixé Toujours dans le but de résoudre $AX = B$ dans le cas spécifique des permutations, il est introduit dans [5] une nouvelle méthode, toujours basée sur une approche “force brute”, qui sera exponentielle mais où l'exposant sera réduit. L'idée est de constituer $k \leq |B|$ tuples $(A_1, B_1, p_1), \dots, (A_k, B_k, p_k)$ à partir de A et de B tels que :

1. $|A_i| = |A|$ pour tout i ,
2. $\sum_{i=1}^k |B_i| \leq |B|$,
3. l'ensemble des solutions de $AX = B$, noté $Sol(A, B)$, est l'union des $Sol(A_i, B_i)$ où nous multiplions la longueur des cycles par p_i ,
4. ℓ_i est un diviseur de ℓ , avec ℓ_i le lcm de l'ensemble des longueurs de cycle de A_i et ℓ celui des longueurs de cycle de A ,
5. p_b est un diviseur de ℓ_i et ce pour chaque longueur de cycle p_b de B_i .

Remarquons que multiplier une longueur de cycle par p_i peut être différent de multiplier un cycle par p_i . Nous n'avons dès lors plus qu'un unique système d'équations à

résoudre. La constitution de ces tuples, si cela est possible, peut être faite en temps polynomial. De là, il est montré dans [5, 6] qu'une approche "force brute" permet de résoudre les équations $A_i X = B_i$ en temps exponentiel sur ℓ_i ; le résultat suivant en est déduit.

Théorème 1.41 ([5, 6]). *Dans le cas des permutations, il est possible de résoudre les équations de la forme $AX = B$ en temps exponentiel sur le lcm des longueurs des cycles de A . De plus, il est possible de compter et d'énumérer toutes les solutions de cette équation avec la même complexité.*

Approche par décomposition des longueurs de cycle en facteurs premiers

Encore dans le but de résoudre les équations de la forme $AX = B$ restreintes aux permutations, sont étudiées dans [42] les équations où tous les cycles de B ont une longueur qui est la puissance d'un unique premier. Pour des raisons de lisibilité, nous appelons *équations à un premier* ce type d'équations. Plus formellement, il existe m entiers positifs $k_1, \dots, k_m$ tels que $k_i < k_{i+1}$ et $B = \sum_{i=1}^m b_{p^{k_i}} C_{p^{k_i}}$. La mise en évidence d'une condition structurelle sur cette classe d'équations permet de prouver le théorème suivant.

Théorème 1.42 ([42]). *Si $AX = B$ est une équation à un premier, alors elle est résoluble en temps polynomial sur la somme des tailles de A et B .*

Il en est déduit un algorithme non prouvé polynomial pour résoudre $AX = B$ dans le cas où A et B sont des permutations générales. Cette nouvelle stratégie repose sur la décomposition en facteurs premiers. L'idée générale est de résoudre l'équation à un premier induite par chaque premier inférieur ou égal à $|B|/|A|$, puis de fusionner les différentes solutions.

Cette stratégie permet également de résoudre les équations de la forme $X^k = B$ en temps polynomial. L'idée est de les solutionner en temps polynomial dans les équations à un premier puis de généraliser en fusionnant les diverses solutions. Dans ce cas, la polynomialité de l'algorithme est une conséquence de [40, 36, 34, 35] qui montrent l'unicité des racines de FDDS. Nous reviendrons plus en détail sur ce résultat dans la Section 1.3. Tout ceci prouve le théorème suivant.

Théorème 1.43 ([42]). *Soit k un entier strictement positif et B une permutation. La résolution de $X^k = B$ est possible en temps polynomial sur $|B|$.*

Nous présenterons dans le Chapitre 3 un algorithme polynomial capable de résoudre $X^k = B$ sans aucune condition sur X et B . L'approche alors utilisée est totalement distincte de celle que nous venons de présenter.

Finalement, [42] utilise encore cette approche de décomposition en équations à un premier pour montrer le théorème suivant.

Théorème 1.44 ([42]). *Soit B une permutation et P un polynôme où tous les coefficients sont des collections de points fixes. Alors $P(X) = B$ est résoluble en temps polynomial sur $|B|$.*

Remarquons que ce théorème implique le Théorème 1.43. En effet, $X^k = B$ peut être vu comme $C_1 \times X^k = B$. Notons que ce théorème est un corollaire du Théorème 3.17.

Définition et algorithme de soustraction Pour rappel, dans cette section, nous avons présenté les résultats de complexité pour la résolution des équations de la forme $P(X_1, \dots, X_n) = B$ puis ayant forme $AX = B$ en insistant sur les cas où A et B sont des permutations. Il nous reste encore à étudier les équations de la forme $A + X = B$. Or ces dernières sont directement solvables en temps polynomial sur la somme des tailles de A et B . En effet, pour trouver le X tel que $A + X = B$, il suffit de prendre une composante connexe de A et de vérifier que B en contient un exemplaire. Si c'est le cas, nous supprimons une copie de cette composante dans A et dans B . Sinon, il n'existe pas de solution. Le X sera constitué des éléments qui resteront dans B quand nous aurons retiré toutes les composantes connexes de A .

Cet algorithme intuitif décrit comment nous pouvons réaliser la soustraction de B par A , notée $B - A$, avec A et B deux FDDS. Ainsi, résoudre $A + X = B$ revient seulement à calculer $B - A$. Tout comme dans le cas des entiers naturels, $b - a$ n'est pas défini si $a > b$, nous notons que $B - A$ ne sera défini que si A est un sous-multi-ensemble de B . Dès lors, nous disons que pour deux multi-ensembles E_1 et E_2 , l'expression $E_2 - E_1$ est définie si et seulement si E_1 est un sous-multi-ensemble de E_2 et représente le multi-ensemble E_3 tel que l'union disjointe de E_1 et E_3 produit E_2 .

1.3. Gestion des parties transitoires : Unroll

Définition des Unrolls et leur algèbre Dans les sections précédentes, nous avons vu qu'un FDDS était la conjonction de deux types de comportements différents, à savoir un comportement cyclique et un comportement transitoire. Nous avons également observé que le produit de deux FDDS connexes A et X produit $\gcd(p_A, p_X)$ composantes connexes de longueur de cycle $\text{lcm}(p_A, p_X)$ avec p_A et p_X les longueurs des cycles de respectivement A et X , ceci nous permettant de conclure que le comportement cyclique du produit de deux FDDS est le produit des comportements cycliques. Ainsi, nous voulons maintenant analyser le comportement transitoire du produit de FDDS vis-à-vis de celui de ses termes. En d'autres mots, nous voudrions énoncer une propriété similaire à : le comportement transitoire du produit de deux FDDS est le produit des comportements transitoires des termes.

Pour cela, nous souhaitons retirer le comportement cyclique du produit de deux FDDS pour ne conserver que sa partie transitoire. Ceci est possible avec la notion d'*unroll* introduite originalement dans [36] puis indépendamment redécouvert dans [40]. Nous présentons dans ce qui suit un chemin de penser conduisant à cette nouvelle notion.

Afin de retirer le comportement cyclique du produit de deux FDDS, une première approche pourrait être de simplement enlever un (ou des) arc(s) au sein des cycles. Ceci nous produirait une forêt finie. Un exemple est donné en Figure 1.45.

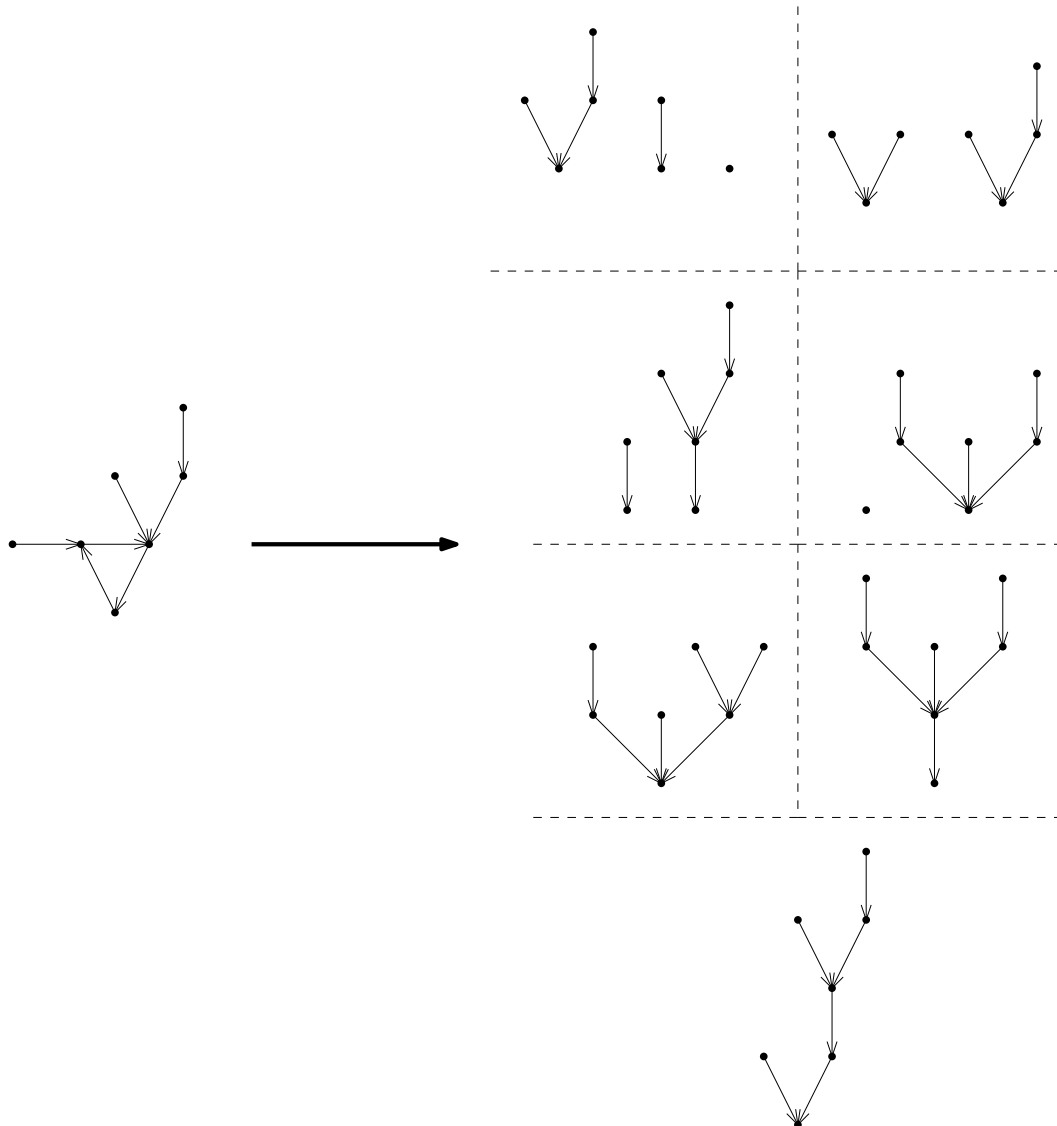


FIGURE 1.45. – À gauche, une composante connexe et à droite l'ensemble des forêts obtenables en retirant des arcs du cycle de la composante de gauche, chacune d'elles étant séparée par une ligne en pointillé. La forêt à trois arbres est issue du retrait des trois arcs du cycle, celle à deux arbres de deux arcs du cycle et celle à un arbre d'un seul arc du cycle.

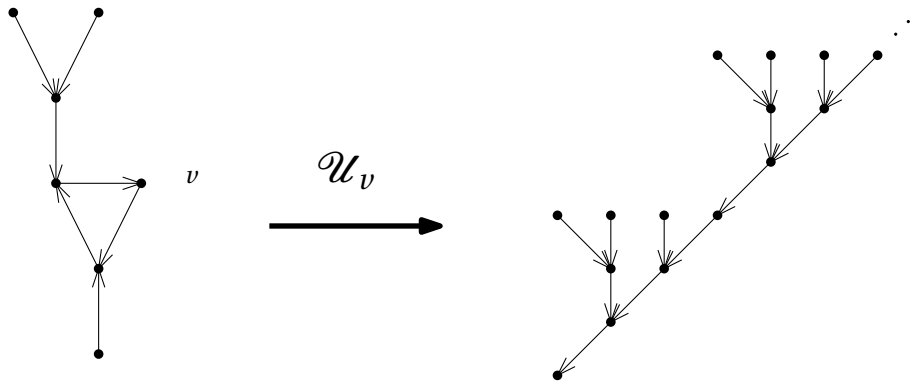


FIGURE 1.47. – Exemple d'arbre d'unroll. À gauche, le FDDS et le sommet v pour lequel nous calculons l'arbre. À droite, l'arbre d'unroll de v .

Pour donner un exemple de cette méthode, nous considérons une composante connexe C dont les sommets cycliques $(1, 2, 3)$ sont les racines des arbres $\mathbf{t}_1, \mathbf{t}_2, \mathbf{t}_3$ respectivement (dans la Figure 1.45, le sommet 1 est le sommet en bas du triangle, le 2 est celui en haut à gauche et le 3 celui en haut à droite). Alors, le résultat de cette opération pourrait être $\mathbf{t}_1, \mathbf{t}_2$ et $\mathbf{t}_3$ (la forêt en haut à gauche de la Figure 1.45), $\mathbf{t}_1$ et un arbre $\mathbf{t}_4$ constitué en enracinant la racine de $\mathbf{t}_2$ dans celle de $\mathbf{t}_3$ (la forêt au milieu à droite de la Figure 1.45), ou encore un arbre $\mathbf{t}_5$ construit en enracinant $\mathbf{t}_4$ dans la racine de $\mathbf{t}_1$ (la forêt en bas à droite de la Figure 1.45).

Hélas, cette méthode entraîne une perte cruciale d'informations vis-à-vis du produit. En effet, si nous reprenons un de nos exemples, la racine de $\mathbf{t}_3$ devrait avoir accès à tous les sommets de $\mathbf{t}_5$.

Pour pallier ce problème, nous pouvons simplement transformer les cycles en chemins infinis et enraciner périodiquement les arbres des composantes connexes le long de ce chemin. Ce type d'arbres peut être vu comme un mot périodique infini à droite dont l'alphabet est l'ensemble des arbres finis, c'est-à-dire un mot $w_a w_a \dots$, où w_a est un mot dont chaque lettre est un arbre fini. Un tel mot w_a est appelé *motif périodique* et sa longueur est son nombre de lettres. Dans notre exemple, nous transformons C en un mot $w_a w_a \dots$ avec w_a le mot $\mathbf{t}_1 \mathbf{t}_2 \mathbf{t}_3$. Nous appelons ces arbres *arbres d'unroll*, formellement définis comme suit (un exemple d'un tel arbre est illustré dans la Figure 1.47).

Définition 1.46 (Arbre d'unroll [40, 16]). Soit $A = (X, f)$ un FDDS et soit v un sommet d'un cycle de A . Pour chaque entier $k \geq 0$, nous notons par $f^{-k}(v)$ l'ensemble des k -èmes préimages de v dans A , c'est-à-dire que $f^{-k}(v) := \{u \in A \mid f^k(u) = v\}$. Nous appelons *arbre d'unroll* de A enraciné en v l'arbre $\mathcal{U}_v(A) = (V, E)$, ayant pour nœuds l'ensemble des couples (u, k) avec $k \geq 0$ un entier et $u \in f^{-k}(v)$. De plus, il y a un arc entre deux sommets (u, k) et (u', k') si et seulement si $f(u) = u'$ et $k' = k - 1$.

Les arbres enracinés dans le cycle de A sont appelés *arbres finis enracinés* dans la branche infinie de $\mathcal{U}_v(A)$.

Maintenant qu'il semble que nous ayons une méthode convenable pour extraire le

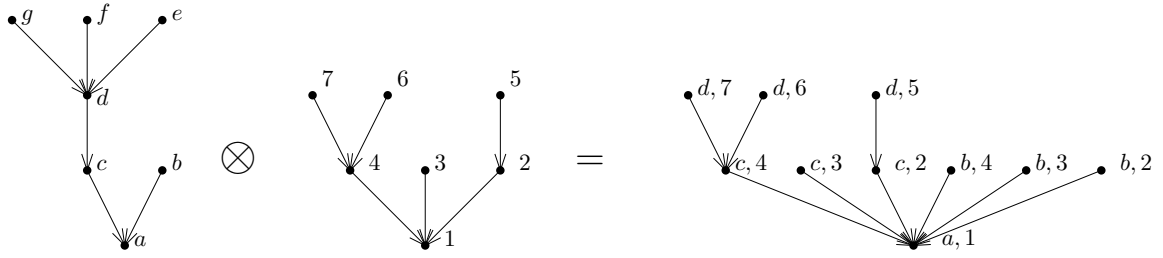


FIGURE 1.49. – Exemple de produit entre deux arbres. À gauche, les deux arbres multipliés et à droite leur produit. Remarquons que la profondeur du résultat est celle du plus petit terme.

comportement transitoire, il nous faut encore définir une opération de produit entre deux de ces arbres qui soit corrélée avec le produit direct des FDDS. Plus précisément, nous voudrions un produit $\otimes$ tel que si $AX = B$ alors :

1. pour tout sommet b d'un cycle de B , il existe a, x des sommets des cycles de A et X tels que $\mathcal{U}_a(A) \otimes \mathcal{U}_x(X) = \mathcal{U}_b(B)$ et,
2. pour tous sommets a, x dans les cycles de A et X , il existe un sommet b dans les cycles de B tel que $\mathcal{U}_a(A) \otimes \mathcal{U}_x(X) = \mathcal{U}_b(B)$.

Cette propriété est bien vérifiée par l'opération définie ci-dessous.

Définition 1.48 (Produit d'arbres [40, 16, 36]). Soient $\mathbf{t}_1 = (V_1, E_1)$ et $\mathbf{t}_2 = (V_2, E_2)$ deux arbres de racines respectives r_1 et r_2 . Le *produit* de $\mathbf{t}_1$ par $\mathbf{t}_2$ est l'arbre $\mathbf{t}_1 \otimes \mathbf{t}_2$ dont l'ensemble de sommets est $V = \{(u, v) \in V_1 \times V_2 \mid d(u) = d(v)\}$, avec $d(u)$ la profondeur de u (distance avec la racine de son arbre) et pour ensemble d'arcs (voir Figure 1.49) :

$$E = \{((u, u'), (v, v')) \mid (u, v) \in E_1, (u', v') \in E_2\} \subseteq V^2.$$

Intuitivement, cette multiplication est semblable au produit direct de graphes, mais appliquée niveau par niveau. Notons bien qu'elle n'est pas définie uniquement sur les arbres d'unroll. Donc, elle s'applique également, par exemple, sur les arbres finis. Il est toutefois important de constater que le produit de deux arbres finis $\mathbf{t}_1$ et $\mathbf{t}_2$ est de profondeur $\min(d(\mathbf{t}_1), d(\mathbf{t}_2))$. Nous pouvons aussi constater que pour réaliser algorithmiquement ce produit, il nous suffirait d'effectuer un parcours en largeur de l'arbre résultat. En effet, si nous considérons deux arbres $\mathbf{s}, \mathbf{t}$ de racines respectives s, t , alors la racine de $\mathbf{s} \otimes \mathbf{t}$ est (s, t) . Et, étant donné un sommet (u, v) de $\mathbf{s} \otimes \mathbf{t}$, pour connaître ses prédécesseurs, il suffit de faire le produit cartésien des prédécesseurs de u avec ceux de v . En conclusion, le temps de calcul pour effectuer ce produit est linéaire sur la taille de son résultat.

Cependant, comme les arbres d'unroll sont intrinsèquement liés à leurs racines, le produit de deux de ces arbres ne capture que localement (au niveau d'une composante connexe) le comportement du produit de deux FDDS. De plus, si nous considérons trois FDDS non nuls A, X, B et tels que $AX = B$, alors $AX \neq 2B$ mais les propriétés sur le produit sont bien respectées. En effet, intuitivement le produit associe chaque b

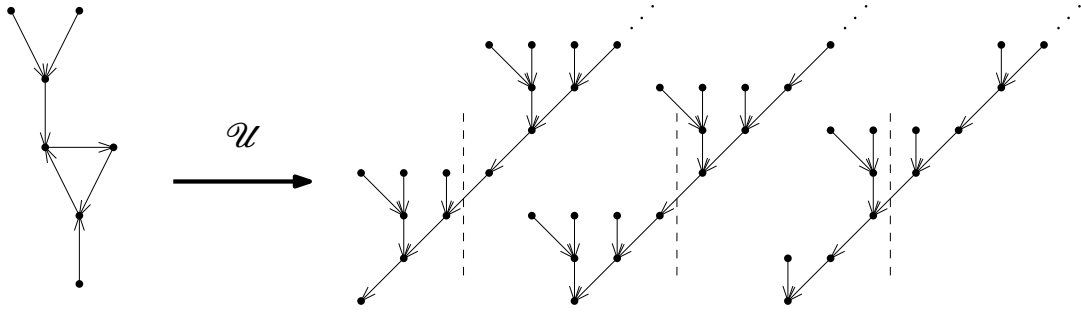


FIGURE 1.51. – Exemple d'unroll. À gauche, le FDDS sur lequel nous appliquons l'unroll et à droite la forêt résultante. De gauche à droite, les arbres d'unroll sont enracinés dans le sommet du triangle en haut à droite, en haut à gauche et en bas. Les traits en pointillé sont la délimitation d'un motif périodique de chaque arbre. Notons que les arbres sont, de gauche à droite, obtenus en supprimant la racine et l'arbre fini qui lui est associé. Ceci, en termes de mots, représente un décalage d'un vers la droite.

à un couple (a, x) et chaque couple (a, x) à un b ; la première de ces associations est injective mais pas la seconde (deux couples peuvent être associés au même b).

Pour pallier cela, il nous suffit de considérer non pas un arbre d'unroll par composante connexe mais tous ceux que nous pouvons produire à partir de cette dernière. Nous appelons cela l'*unroll* de la composante connexe. Pour fournir une définition plus formelle, nous rappelons que, puisque les forêts sont des graphes, nous pouvons définir la somme de deux forêts F_1 et F_2 , notée $F_1 + F_2$, comme l'union disjointe des arbres de F_1 et de F_2 . Un exemple est donné dans la Figure 1.51.

Définition 1.50 (Unroll [40, 36]). Étant donné un FDDS connexe A , nous disons que l'unroll de A , noté $\mathcal{U}(A)$, est la somme des arbres $\mathcal{U}_v(A)$ pour tout sommet v dans le cycle de A . Sauf mention contraire, les unrolls sont toujours considérés à isomorphisme près. Et plus généralement, l'unroll d'un FDDS $A = \sum_{i=0}^{m_A} a_i A_i$ est $\mathcal{U}(A) = \sum_{i=0}^{m_A} a_i \mathcal{U}(A_i)$.

Notons par $\mathcal{U}(\mathbb{D})$ l'ensemble des unrolls de FDDS, *i.e.*, $\mathcal{U}(\mathbb{D}) = \{\mathcal{U}(A) \mid A \in \mathbb{D}\}$. Alors, à partir des définitions de la somme et du produit, nous constatons que $(\mathcal{U}(\mathbb{D}), +)$ est un monoïde commutatif avec comme élément neutre $\mathbf{0}$ la forêt sans arbre. En outre, $(\mathcal{U}(\mathbb{D}), \otimes)$ est aussi un monoïde commutatif avec comme élément neutre $\mathbf{1}$ le chemin simple infini et où le neutre additif est absorbant. De surcroît, la multiplication est distributive sur la somme, c'est-à-dire que pour toutes forêts F_1, F_2, F_3 , l'égalité $F_1 \otimes (F_2 + F_3) = F_1 \otimes F_2 + F_1 \otimes F_3$ est vérifiée. Nous pouvons généraliser ce raisonnement à tous les éléments de $\mathbb{F}$, l'ensemble des forêts, mais aussi aux éléments de $\mathbb{F}_d$, l'ensemble des forêts de profondeur au plus d . Nous en concluons ainsi la proposition suivante.

Proposition 1.52. $(\mathcal{U}(\mathbb{D}), +, \otimes)$ est un semi-anneau commutatif et $\mathcal{U}(X)$ est un morphisme de semi-anneaux entre $(\mathbb{D}, +, \times)$ et $(\mathcal{U}(\mathbb{D}), +, \otimes)$. De plus, $(\mathbb{F}, +, \otimes)$ et $(\mathbb{F}_d, +, \otimes)$ sont aussi des semi-anneaux commutatifs, et ce pour tout entier naturel d .

Remarque 1.53. L'ensemble des forêts finies, noté $\mathbb{F}_f$, muni de l'union disjointe pour addition et de $\otimes$ pour produit, n'est pas un semi-anneau, car il ne possède pas de neutre multiplicatif.

Annulabilité des unrolls : approche par comptages d'homomorphismes

Nous pouvons étudier plusieurs problèmes liés à ces semi-anneaux et aux semi-anneaux de polynômes qui leur sont associés. Par exemple, quels éléments sont *annulables*. Cette notion est utile pour, entre autres, simplifier des divisions.

Définition 1.54 (Annulabilité). Soit A un élément d'un semi-anneau $(M, \oplus, \star)$. Nous définissons que A est *annulable* si $AX = AY$ équivaut à ce que $X = Y$ pour tous éléments X, Y de M . De façon équivalente, un élément A est dit annulable s'il est le coefficient d'un monôme linéaire injectif.

Et nous nous interrogeons plus généralement sur quels sont les polynômes injectifs, c'est-à-dire pour quelle fonction polynomiale P de forêt, nous avons $P(\mathbf{X}) = P(\mathbf{Y})$ si et seulement si $\mathbf{X} = \mathbf{Y}$. Au-delà de leurs aspects purement théoriques, ces deux notions ont une importance pratique. En effet, la contrainte d'unicité peut induire des conditions structurelles fortes pouvant simplifier la résolution des équations de la forme $P(X) = B$ ou même $P(X) = Q(X)$.

Pour répondre à ces questions, nous introduisons la notion de *troncature* d'une forêt à une certaine profondeur. L'idée de cette opération est de retirer l'ensemble des sommets d'une forêt qui ont une profondeur strictement supérieure à un entier donné (un exemple est donné dans la Figure 1.56). Plus formellement :

Définition 1.55 (Troncature). Soient $\mathbf{t}$ un arbre et $d \geq 0$ un entier, la *troncature* de $\mathbf{t}$ à profondeur d , notée $\mathcal{C}_d(\mathbf{t})$, est le sous-graphe induit par les sommets de $\mathbf{t}$ dont la profondeur est inférieure ou égale à d .

La troncature d'une forêt $\mathbf{F} = \sum_{i=1}^m \mathbf{t}_i$ à profondeur d , notée $\mathcal{C}_d(\mathbf{F})$, est la somme des troncatures des arbres $\mathbf{t}_i$ à profondeur d , i.e. $\mathcal{C}_d(\mathbf{F}) = \sum_{i=1}^m \mathcal{C}_d(\mathbf{t}_i)$.

Nous pouvons constater que la fonction de troncature à profondeur d est un morphisme entre les semi-anneaux $(\mathbb{F}, +, \otimes)$ et $(\mathbb{F}_d, +, \otimes)$. De la définition du produit, nous avons qu'aucune forêt de profondeur finie (une forêt qui ne contient que des arbres de profondeur finie) n'est annulable. En effet, si nous considérons $\mathbf{A}$ une forêt de profondeur d et deux forêts différentes $\mathbf{X}$ et $\mathbf{Y}$ de profondeur au moins d telles que $\mathcal{C}_d(\mathbf{X}) = \mathcal{C}_d(\mathbf{Y})$ alors $\mathbf{A}\mathbf{X} = \mathbf{A}\mathbf{Y}$ puisque le produit d'arbres tronqué à profondeur d . Même si aucune forêt finie ne peut être annulable, elle respecte cependant une "approximation" de cette propriété, c'est-à-dire que même si pour toute forêt $\mathbf{A}$ il existe deux forêts $\mathbf{X}$ et $\mathbf{Y}$ telles que $\mathbf{A}\mathbf{X} = \mathbf{A}\mathbf{Y}$ et $\mathbf{X} \neq \mathbf{Y}$, le fait que $\mathbf{A}\mathbf{X} = \mathbf{A}\mathbf{Y}$ induit que $\mathbf{X}$ et $\mathbf{Y}$ ont une structure commune. Ceci est expliqué plus en détail dans le lemme suivant.

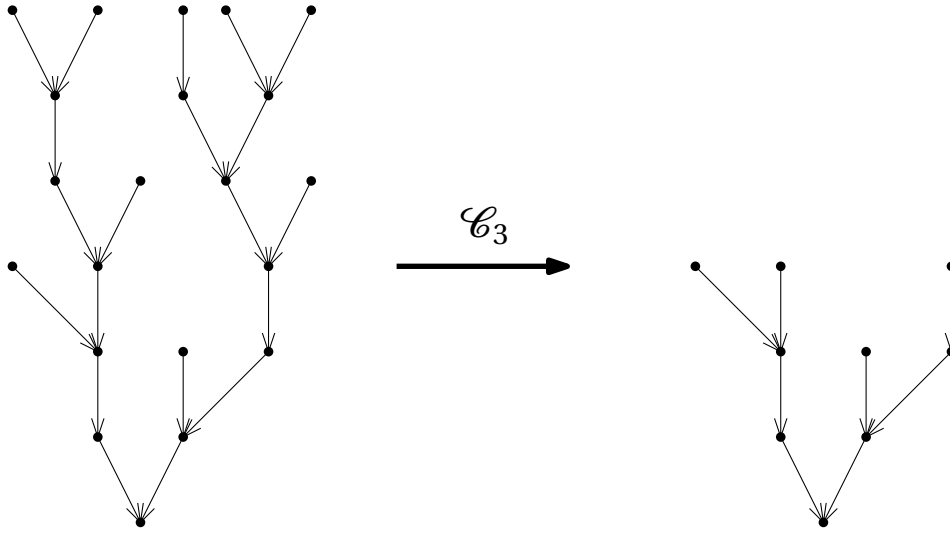


FIGURE 1.56. – Exemple de troncature d'un arbre à profondeur 3. À gauche, l'arbre que nous tronquons, et, à droite, le résultat.

Lemme 1.57 ([16, 40]). *Soient $\mathbf{a}, \mathbf{x}, \mathbf{y}$ trois arbres finis tels que $\mathbf{a}$ est de profondeur d . Alors, $\mathbf{ax} = \mathbf{ay}$ si et seulement si $\mathcal{C}_d(\mathbf{x}) = \mathcal{C}_d(\mathbf{y})$.*

Corollaire 1.58. *Les arbres de profondeur infinie sont annulables dans $(\mathbb{F}, +, \otimes)$ et ceux de profondeur d sont annulables dans $(\mathbb{F}_d, +, \otimes)$.*

Ces résultats ont été obtenus de deux manières différentes. Premièrement, dans [16] par le comptage de morphismes. Pour expliquer cette approche, nous définissons que $\text{hom}(H, G)$ est le nombre de morphismes de H dans G . Cette technique repose sur le théorème suivant :

Théorème 1.59 ([34, 35]). *Deux graphes orientés G et G' sont égaux si et seulement si $\text{hom}(H, G) = \text{hom}(H, G')$ pour tout graphe orienté connexe H .*

Pour arriver au résultat voulu, il fut démontré que pour tout graphe orienté H et toutes forêts finies $\mathbf{F}_1, \mathbf{F}_2$, nous avons $\text{hom}(H, \mathbf{F}_1 \otimes \mathbf{F}_2) = \text{hom}(H, \mathbf{F}_1) \cdot \text{hom}(H, \mathbf{F}_2)$. En effet, $\text{hom}(H, \mathbf{F}_1) \neq 0$ si et seulement si $\text{hom}(H, \mathcal{C}_n(\mathbf{F}_2)) \neq 0$ avec n la profondeur de $\mathbf{F}_1$. Ce résultat est dû au fait que $\text{hom}(H, \mathbf{t}) \neq 0$ avec $\mathbf{t}$ un arbre de profondeur n si et seulement si $\text{hom}(H, P_n) \neq 0$, avec P_n le chemin simple de longueur n . Cela vient du fait qu'il existe un homomorphisme de $\mathbf{t}$ vers P_n et un autre homomorphisme de P_n vers $\mathbf{t}$.

Annulabilité des unrolls : approche par ordre total compatible avec le produit Une preuve différente pour caractériser les éléments annulables est introduite dans [40, 36] et repose sur la définition d'un ordre total sur les arbres, compatible avec le produit, dans le cas des arbres de profondeur infinie.

Définition 1.60 (Ordre compatible avec une opération). Soit $\leq$ une relation d'ordre sur les éléments du monoïde $(M, \diamond)$. Nous disons que $\leq$ est compatible avec $\diamond$ si et seulement si $a \diamond b \leq a \diamond c$ équivaut à $b \leq c$ pour tout triplet d'éléments a, b, c de M où a n'est pas absorbant.

Il existe plusieurs paradigmes pour définir des ordres sur les arbres. Il y a par exemple les ordres de génération, ou encore ceux tenant compte des informations comme la taille ou la profondeur des sous-arbres. Parmi ceux-ci, nous en choisissons un qui est similaire à celui utilisé dans [33] qui permet de définir un ordre total sur les arbres calculable en espace logarithmique. Ce paradigme est formellement défini comme suit.

Définition 1.61 (Paradigme de l'ordre). Soient s, t deux arbres de racines respectives s, t . Alors nous disons que $s \leq t$ si et seulement si :

1. le nombre de prédécesseurs de s est strictement inférieur à celui de t ou,
2. les sommets s et t ont n prédécesseurs et $(s_1, \dots, s_n) \leq_{lex} (t_1, \dots, t_n)$, où $s_1, \dots, s_n$ et $t_1, \dots, t_n$ sont les sous-arbres enracinés dans s et t respectivement tels que $s_1 \leq \dots \leq s_n$ et $t_1 \leq \dots \leq t_n$ où $\leq_{lex}$ est induit par l'ordre $\leq$ récursivement défini.

Pour concrètement calculer cet ordre sur les arbres finis, nous introduisons un code d'isomorphisme. Pour produire ce code, nous construisons récursivement une *forme canonique* des arbres. Dans cette forme, les arbres enracinés dans un sommet u sont tous triés de gauche à droite par ordre croissant, et ce pour tout nœud u de l'arbre. Après quoi, nous réalisons un simple parcours en largeur au cours duquel nous écrivons successivement le nombre de prédécesseurs de chaque sommet. Le code de l'arbre est le mot ainsi construit. La Figure 1.62 explicite les étapes de construction pour cet ordre. Pour généraliser cet ordre aux arbres infinis, nous associons à chaque arbre infini t un vecteur infini $\vec{v}$ tel que la i -ème coordonnée de $\vec{v}$ contient le code de $\mathcal{C}_i(t)$. Puis pour comparer deux arbres infinis, nous comparons lexicographiquement leurs vecteurs.

Notation 1.63 (Minimum de forêts). Soit F une forêt. La notation $\min(F)$ désigne l'arbre minimal de F selon l'ordre défini ci-dessus. Remarquons que l'unroll d'un FDDS A est une forêt. Nous utilisons également la notation $\min(\mathcal{U}(A))$ avec la même signification que ci-dessus.

En utilisant le Théorème 3.2 de [2, page 83], nous déduisons que le temps de calcul des formes canoniques des arbres enracinés dans les sommets de profondeur i est en $\mathcal{O}(n_i + d)$ où n_i est la somme des tailles de ces arbres et d est le degré de l'arbre en entrée, c'est-à-dire le nombre de prédécesseurs du sommet qui en a le plus. Donc, nous pouvons le faire en $\mathcal{O}(n)$ avec n la taille de l'arbre en entrée.

Lemme 1.64. *La forme canonique d'un arbre fini et son code d'isomorphisme sont calculables avec une complexité en $\mathcal{O}(n^2)$, où n est la taille de l'entrée.*

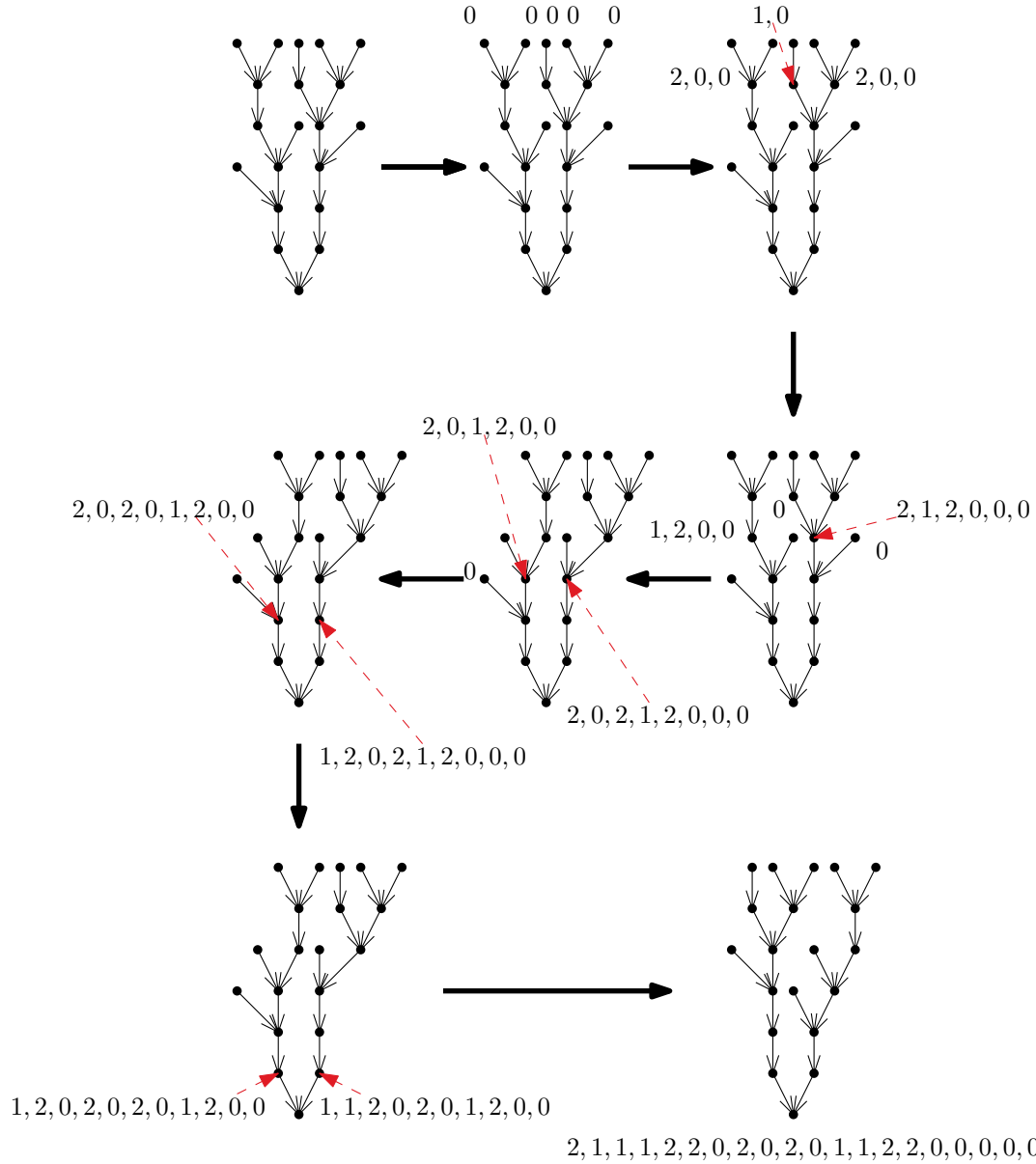


FIGURE 1.62. – Calcul du code et de la forme canonique de l'arbre en haut à gauche. À chaque étape, nous calculons le code des sommets d'une profondeur donnée en partant des sommets de profondeur maximale pour atteindre la racine. Notons de plus qu'une fois les codes calculés, nous trions par ordre croissant chaque ensemble de prédécesseurs en fonction de leur code.

1. État de l'art et définitions – 1.3. Gestion des parties transitoires : Unroll

Le résultat d'annulabilité des arbres de profondeur infinie est généralisé aux unrolls dans [36]. En d'autres termes, cette méthode permet également de démontrer que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(A)\mathcal{U}(Y)$ si et seulement si $\mathcal{U}(X) = \mathcal{U}(Y)$ et ce pour tous FDDS A, X, Y .

Théorème 1.65 ([36]). *Tout unroll est annulable.*

Nous redémontrerons ce théorème dans la Section 2.1. Cependant, nous utiliserons une approche différente, car elle sera basée sur des aspects algorithmiques permettant de résoudre les équations $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ pour les X connexes.

Injectivité des racines des FDDS : approche par unroll L'existence d'un ordre compatible avec le produit implique également le lemme suivant.

Lemme 1.66. *Pour tous arbres $\mathbf{s}$ et $\mathbf{t}$ et pour tout entier $k > 0$, nous avons $\mathbf{s}^k = \mathbf{t}^k$ si et seulement si $\mathbf{s} = \mathbf{t}$.*

Pour généraliser cela aux forêts, il nous suffit d'introduire les deux opérations suivantes. Tout d'abord l'opération de *déracinement* d'un arbre $\mathbf{t}$, notée $\mathcal{D}(\mathbf{t})$, qui produit une forêt constituée des arbres enracinés dans la racine de $\mathbf{t}$. Nous pouvons généraliser cela aux forêts $\mathbf{F} = \sum_{i=1}^m \mathbf{t}_i$ en définissant simplement que $\mathcal{D}(\mathbf{F}) = \sum_{i=1}^m \mathcal{D}(\mathbf{t}_i)$. Ensuite, l'opération d'*enracinement* d'une forêt $\mathbf{F}$, notée $\mathcal{E}(\mathbf{F})$, enracine tous les arbres de $\mathbf{F}$ dans un nouveau sommet r ; $\mathcal{E}(\mathbf{F})$ est donc un arbre. Notons que $\mathcal{D}(\mathcal{E}(\mathbf{F})) = \mathbf{F}$ et que $\mathcal{E}(\mathcal{D}(\mathbf{t})) = \mathbf{t}$.

Nous montrons sans difficulté que $\mathcal{D}(\mathbf{X})$ et $\mathcal{E}(\mathbf{X})$ sont des fonctions distributives sur le produit qui, de plus, conservent le chemin infini comme neutre de la multiplication. En outre, $\mathcal{D}(\mathbf{X})$ est distributive sur la somme et conserve le neutre additif. Cependant, ce n'est pas le cas de $\mathcal{E}(\mathbf{X})$. En effet, $\mathcal{E}(\mathbf{ab})$ est un seul arbre tandis que $\mathcal{E}(\mathbf{a}) + \mathcal{E}(\mathbf{b})$ en sont deux, si $\mathbf{a}$ et $\mathbf{b}$ sont des arbres non vides. Ainsi,

Lemme 1.67 ([40]). *La fonction $\mathcal{D}(\mathbf{X})$ est un endomorphisme du semi-anneau $(\mathbb{F}, +, \otimes)$. Tandis que $\mathcal{E}(\mathbf{X})$ est un endomorphisme du monoïde $(\mathbb{F}, \otimes)$.*

Grâce à ces propriétés, nous déduisons que si $\mathbf{F}^k = \mathbf{S}^k$ alors $\mathcal{E}(\mathbf{F}^k) = \mathcal{E}(\mathbf{S}^k)$; impliquant que $\mathcal{E}(\mathbf{F})^k = \mathcal{E}(\mathbf{S})^k$. Et comme $\mathcal{E}(\mathbf{F}), \mathcal{E}(\mathbf{S})$ sont des arbres, le Lemme 1.66 induit que $\mathcal{E}(\mathbf{F}) = \mathcal{E}(\mathbf{S})$. Maintenant, par une simple application de $\mathcal{D}(\mathbf{X})$ des deux côtés de l'égalité, nous concluons que $\mathbf{F} = \mathbf{S}$. Ceci termine la preuve de la proposition suivante.

Proposition 1.68 ([40, 36]). *Soient $\mathbf{X}, \mathbf{Y}$ deux forêts et $k > 0$ un entier. Alors $\mathbf{X}^k = \mathbf{Y}^k$ si et seulement si $\mathbf{X} = \mathbf{Y}$. Et en particulier, $\mathcal{U}(X)^k = \mathcal{U}(Y)^k$ si et seulement si $\mathcal{U}(X) = \mathcal{U}(Y)$ et ce pour tous FDDS X et Y .*

Il est de plus montré, dans [40] et [36], que dans le cas des permutations, $X^k = Y^k$ si et seulement si $X = Y$. Ces preuves reposent sur la construction d'équations permettant de décrire les longueurs des cycles de X^k ainsi que leurs multiplicités (ces données jouant le rôle de constantes dans l'équation) à partir de ces mêmes informations, cette fois considérées comme inconnues, présentes dans X . Pour conclure,

les articles établissent l'unicité des solutions de ces équations. De là, grâce à la Proposition 1.52 et via une identification des composantes connexes avec leur unroll, fondée sur la longueur des cycles desdites composantes, les deux articles démontrent la proposition suivante.

Proposition 1.69 ([40, 36]). *Soient A, B deux FDDS et $k > 0$ un entier. Alors, $A^k = B^k$ si et seulement si $A = B$.*

Injectivité des racines des FDDS : approche par comptage de morphismes

Une autre preuve d'une des directions de la Proposition 1.69 est induite par [34, 35]. En reprenant la technique de comptage de morphismes, ils purent prouver le théorème suivant.

Théorème 1.70 ([34, 35]). *Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme de graphe orienté. S'il existe un A_i , avec $i > 0$, qui contient un point fixe, alors P est injectif.*

Pour arriver à ce résultat, ils montrèrent que pour tout graphe orienté connexe G , la fonction $\text{hom}(G, X)$ est un morphisme entre $(\mathcal{G}, +, \times)$ et le semi-anneau des entiers naturels avec les opérations usuelles, où $(\mathcal{G}, +, \times)$ désigne le semi-anneau des graphes orientés, muni de l'union disjointe pour somme, et du produit direct, comme multiplication. Ceci signifie qu'étant donné un polynôme univarié de graphe orienté $P = \sum_{i=0}^m A_i X^i$, s'il existe deux graphes orientés X, Y tels que $P(X) = P(Y)$ alors $\sum_{i=0}^m \text{hom}(G, A_i) \text{hom}(G, X)^i = \sum_{i=0}^m \text{hom}(G, A_i) \text{hom}(G, Y)^i$ pour tout graphe orienté connexe G . Maintenant, en partant de cette équation et en s'appuyant sur le Théorème 1.59, il ne reste plus qu'à démontrer que $\text{hom}(G, X) = \text{hom}(G, Y)$ pour montrer que $X = Y$.

De l'équation précédente et comme $\text{hom}(G, P(X)) \geq \text{hom}(G, A_0)$ étant donné que $\text{hom}(G, P(X)) = \text{hom}(G, P(X) - A_0) + \text{hom}(G, A_0)$ deux cas sont possibles. Le premier est que $\text{hom}(G, P(X)) - \text{hom}(G, A_0) \neq 0$ et donc, par l'injectivité des polynômes univariés dans les entiers naturels, nous déduisons que $\text{hom}(G, X) = \text{hom}(G, Y)$. Le second est que $\text{hom}(G, P(X)) - \text{hom}(G, A_0) = 0$. Et de nouveau deux cas sont possibles. Soit $\text{hom}(G, A_i) = 0$ pour tout $i > 0$, signifiant que $\text{hom}(G, X)$ peut être différent de $\text{hom}(G, Y)$. Soit ce n'est pas le cas et donc $\text{hom}(G, X) = \text{hom}(G, Y) = 0$. Or, si au moins l'un des A_i , avec $i > 0$, contient un point fixe, alors $\text{hom}(G, A_i) \geq 1$. En effet, la fonction qui envoie tous les sommets de G vers un point fixe de A_i est un homomorphisme.

Annulabilité des FDDS En corollaire, nous déduisons qu'un FDDS A est annulable s'il possède un point fixe. Cependant [40] fournit une caractérisation complète dont la preuve est basée sur les unrolls et une analyse des longueurs des cycles.

Théorème 1.71 ([40, 34, 35]). *Un FDDS A est annulable si et seulement si A contient un dendron.*

En résumé, si $AX = AY$, alors, comme A possède un point fixe, nous pouvons retrouver la plus petite longueur de cycle d'une composante connexe C de X et C' de

Y . De là, en passant par les unrolls, nous avons $\min(\mathcal{U}(C)) = \min(\mathcal{U}(C'))$. Ceci signifie que $C = C'$ et donc que $A(X - C) = A(Y - C')$. Et récursivement, nous déduisons que $X = Y$.

L'une des principales contributions de cette thèse est la généralisation de cette technique de preuve afin de résoudre algorithmiquement, pour certains polynômes de FDDS P , les équations de la forme $P(X) = B$ en temps polynomial en la somme des tailles des coefficients de P et de celle de B . Elle nous permet également d'énumérer l'ensemble des solutions de ces équations avec un délai polynomial.

Et pour l'autre direction, il est montré dans [40] qu'il nous est possible d'associer deux FDDS X et Y différents à chaque A qui ne contient pas de point fixe et tels que $AX = AY$. Ces FDDS seront des permutations et l'idée de la construction est de définir un Y à partir de X en substituant des cycles par des points fixes. Pour plus de détails, le lecteur est invité à consulter la preuve de [40, Lemma 33] ou la Sous-section 3.1.3. Cette méthode est semblable à celle introduite dans [43] pour montrer que les graphes orientés sans point fixe ne sont pas annulables.

Division d'un dendron par un dendron Finalement, nous pouvons nous intéresser au cas où $\mathcal{U}(X)$ n'est pas seulement un morphisme mais un isomorphisme. Pour cela, reprenons le sous-semi-anneau $(\mathbb{D}^*, +, \times)$, où $\mathbb{D}^*$ est la clôture par la somme de l'ensemble des dendrons. Par une simple analyse, nous constatons que $(\mathcal{U}(\mathbb{D}^*), +, \otimes)$ est un sous-semi-anneau de $(\mathcal{U}(\mathbb{D}), +, \otimes)$, avec $\mathcal{U}(\mathbb{D}^*)$ l'ensemble des unrolls des éléments de $\mathbb{D}^*$. De plus, $\mathcal{U}(X)$ est un morphisme (par définition) surjectif entre ces deux semi-anneaux. Pour montrer qu'il est, en outre, injectif, il suffit de constater que si $\mathcal{U}(A) = \mathcal{U}(B)$, A et B des dendrons, alors les arbres finis enracinés dans les racines de $\mathcal{U}(A)$ et $\mathcal{U}(B)$ sont identiques. Et cet arbre est la partie transitoire de A et de B . Ceci induit que $A = B$ et, via une récurrence, nous pouvons conclure que la propriété est vérifiée pour tout élément de $\mathbb{D}^*$.

Ainsi, si nous avons un algorithme polynomial qui, étant donné $A, B \in \mathbb{D}^*$ permet de décider s'il existe une solution aux équations de la forme $\mathcal{U}(A) \otimes \mathcal{U}(X) = \mathcal{U}(B)$, alors nous en aurions aussi un qui permettrait de le faire pour les équations de la forme $A \times X = B$. L'existence d'un tel algorithme dans ce cas est montrée dans [40]. Pour cela, ils commencent par montrer que nous ne pouvons que considérer la troncature des unrolls au lieu des unrolls eux-mêmes. La propriété exacte est décrite dans la proposition suivante.

Proposition 1.72 ([40]). *Soient A, X, B des dendrons et $n \geq d(B)$. Alors, $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ si et seulement si $\mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(B))$.*

Intuitivement, tronquer à profondeur $d(B)$ est suffisant car les arbres ainsi formés contiennent l'intégralité du comportement transitoire du FDDS. Notons que la taille de la troncature d'un arbre d'unroll de A ou B à cette profondeur n est quadratique sur la taille des FDDS originaux. En effet, nous pouvons borner la taille d'un arbre fini enraciné dans l'une des branches infinies de $\mathcal{U}(A)$ et $\mathcal{U}(B)$ par $|A + B|$. Et le nombre d'arbres finis enracinés dans la branche infinie conservée dans la troncature est au plus $|A + B|$.

Algorithme 1.74 $\text{DivisionOriginal}(\mathbf{a}, \mathbf{b})$

```

1:  $\mathbf{B} \leftarrow \mathcal{D}(\mathbf{b})$ ,
2:  $\mathbf{X} \leftarrow \emptyset$ ,
3: Tant que  $\mathbf{B} \neq \emptyset$  faire
4:    $d \leftarrow d(\mathbf{B})$ ,
5:    $\mathbf{B}_d \leftarrow$  le multi-ensemble des arbres de  $\mathbf{B}$  ayant profondeur au moins  $d$ ,
6:    $\mathbf{A}_d \leftarrow$  le multi-ensemble des arbres de  $\mathcal{D}(\mathbf{a})$  ayant profondeur au moins  $d$ ,
7:    $\mathbf{m}_{\mathbf{B}_d} \leftarrow \min(\mathbf{B}_d)$ ,
8:    $\mathbf{m}_{\mathbf{A}_d} \leftarrow \min(\mathbf{A}_d)$ ,
9:    $\mathbf{x} \leftarrow \text{DivisionOriginal}(\mathbf{m}_{\mathbf{B}_d}, \mathbf{m}_{\mathbf{A}_d})$ ,
10:  Si  $\mathbf{x} = \perp$  ou  $\mathbf{x} \otimes \mathcal{D}(\mathbf{a}) \not\subseteq \mathbf{B}$  alors
11:    renvoyer  $\perp$ ,
12:   $\mathbf{B} \leftarrow \mathbf{B} - \mathbf{x} \otimes \mathcal{D}(\mathbf{a})$ ,
13:   $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{x}$ ,
14: renvoyer  $\mathcal{E}(\mathbf{X})$ .

```

Maintenant, comme $\mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(B))$ sont connexes, il nous suffit de trouver un algorithme polynomial permettant de “diviser” deux arbres finis. Pour réaliser cela, l'article [40] utilise l'ordre total défini plus tôt pour récursivement calculer une branche $\mathbf{x}$ de $\mathcal{C}_n(\mathcal{U}(X))$ à partir de l'une de $\mathcal{C}_n(\mathcal{U}(A))$ et d'une autre de $\mathcal{C}_n(\mathcal{U}(B))$. Puis réaliser $\mathbf{x} \otimes \mathcal{D}(\mathcal{C}_n(\mathcal{U}(A)))$ avant de retirer le résultat de $\mathcal{D}(\mathcal{C}_n(\mathcal{U}(B)))$ et de répéter l'opération jusqu'à ce qu'il n'y ait plus d'éléments dans $\mathcal{D}(\mathcal{C}_n(\mathcal{U}(B)))$. Notons que le raisonnement utilisé tient pour n'importe quel ordre total sur les arbres s'il est compatible avec le produit.

Comme cette opération de division sera utilisée dans différents algorithmes du Chapitre 2, nous allons ici reprendre l'algorithme de [40] en le modifiant légèrement pour diminuer son temps d'exécution. Cet algorithme prend en entrée deux arbres $\mathbf{a}$ et $\mathbf{b}$. Intuitivement, il commence par calculer la profondeur d de $\mathbf{b}$ puis cherche le plus petit arbre de $\mathcal{D}(\mathbf{a})$ et de $\mathcal{D}(\mathbf{b})$ qui a une profondeur d'au moins d . Il réalise récursivement la division de ces deux arbres pour obtenir $\mathbf{x}$. Puis il calcule $\mathcal{D}(\mathbf{a}) \otimes \mathbf{x}$ et le retire si possible de $\mathcal{D}(\mathbf{b})$. Ensuite, il ajoute $\mathbf{x}$ comme sous-arbre du résultat et recommence tant que $\mathcal{D}(\mathbf{b})$ n'est pas vide. Un exemple d'exécution est donné dans la Figure 1.73. La version originale de l'algorithme est l'Algorithme 1.74. Il prend en entrée deux arbres $\mathbf{a}$ et $\mathbf{b}$ et renvoie l'arbre $\mathbf{x}$ tel que $\mathbf{ax} = \mathbf{b}$, s'il en existe un.

Nous constatons tout d'abord qu'il n'est pas nécessaire, dans l'Algorithme 1.74, de réaliser les instructions 5 et 6 dans la boucle. En effet, pour remplacer 5, nous pourrions construire un tableau T_b de listes chaînées d'arbres triés en ordre croissant tel que la case d du tableau contient l'ensemble des arbres de profondeur d dans $\mathbf{B}$. Plus généralement, le tableau T_b est appelé la *table ordonnée* de la forêt $\mathbf{B}$. Nous utilisons cette représentation car elle nous permet de vérifier qu'une forêt d'arbres de profondeur d triée en ordre croissant, est un sous-multi-ensemble de $T_b[d]$, en temps linéaire sur la taille de $T_b[d]$. Le cas échéant, nous pouvons également retirer cette forêt de $T_b[d]$ en temps linéaire. Ceci s'explique car comparer $\mathbf{t}_1$ et $\mathbf{t}_2$ est linéaire

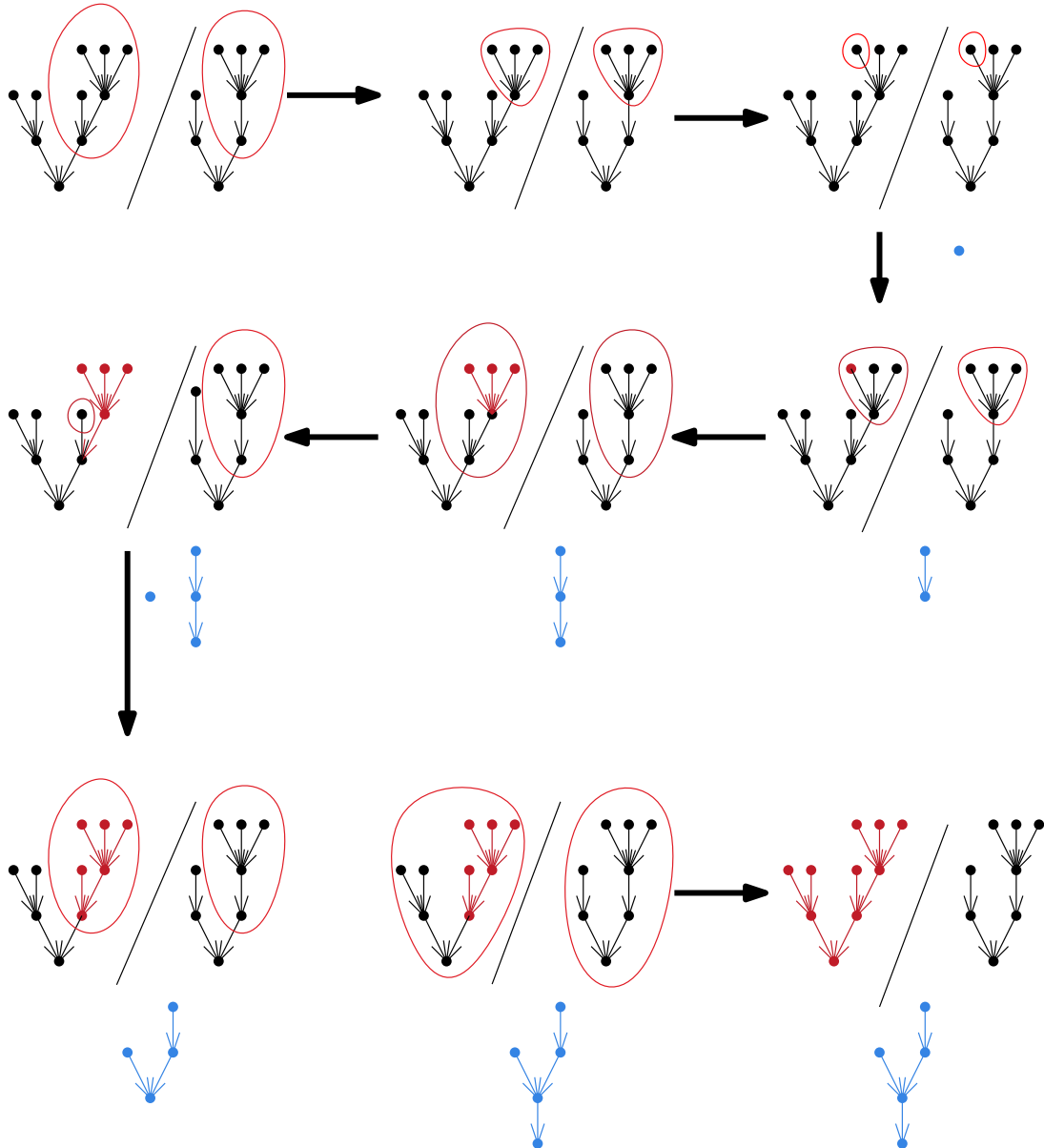


FIGURE 1.73. – Exemple d'exécution de l'algorithme de division. Nous divisons l'arbre à gauche du trait par celui situé à droite. Les arbres entourés en rouge sont ceux considérés récursivement. Ceux en bleu correspondent aux quotients construits récursivement. Les parties des arbres de gauche en rouge sont celles ayant déjà été traitées par l'algorithme. Le dernier arbre bleu représente le quotient.

Algorithme 1.75 DivisionRec($T_b, \mathbf{a}, T_a, \mathbf{X}$)

```

1: Si  $d(\mathbf{a}) = 0$  alors
2:   renvoyer  $\mathbf{a}$ ,
3:  $d$  le plus grand entier tel que  $T_b[d]$  est non vide,
4:  $\mathbf{m}_b \leftarrow \min(T_b[d])$ ,
5:  $\mathbf{m}_a \leftarrow T_a[d]$ ,
6: construire  $T'_b$  la table ordonnée de  $\mathbf{m}_b$ ,
7: construire  $T'_a$  la table des arbres minimaux associée à  $\mathbf{m}_a$ ,
8:  $\mathbf{x} \leftarrow \text{Division}(T'_b, \mathbf{m}_a, T'_a, \emptyset)$ ,
9: Si  $\mathbf{x} = \perp$  alors
10:  renvoyer  $\perp$ ,
11: calculer  $\mathbf{x} \otimes \mathcal{D}(\mathbf{a})$  en s'assurant que sa taille soit au plus  $|\mathbf{b}|$ ,
12: construire  $T_x$  la table ordonnée associée à  $\mathbf{x} \otimes \mathcal{D}(\mathbf{a})$ ,
13: Si  $T_x$  n'est pas un sous-multi-ensemble de  $T_b$  alors
14:  renvoyer  $\perp$ ,
15: retirer  $T_x$  de  $T_b$ ,
16: Si  $T_b$  contient un arbre alors
17:  renvoyer DivisionRec( $T_b, \mathbf{a}, T_a, \mathbf{X} + \mathbf{x}$ ),
18: Sinon
19:  renvoyer  $\mathcal{E}(\mathbf{X})$ .

```

Algorithme 1.76 Division($\mathbf{a}, \mathbf{b}$)

```

1: soit  $T_b$  la table ordonnée de  $\mathbf{b}$ ,
2: soit  $T_a$  la table des arbres minimaux associée à  $\mathbf{a}$ ,
3: renvoyer DivisionRec( $T_b, \mathbf{a}, T_a, \emptyset$ ).

```

sur $\min(|\mathbf{t}_1|, |\mathbf{t}_2|)$ si le code des arbres est connu. Et nous ne faisons qu'au plus une comparaison par arbre de $T_b[d]$.

Pour remplacer $\mathbf{A}_d$, nous utilisons un simple tableau T_a tel que dans sa case d'indice d se trouve le plus petit arbre de profondeur au moins d . Comme $\mathbf{a}$ est en forme canonique, il nous suffit d'un simple parcours des prédécesseurs de sa racine pour faire cela. Nous appelons cette structure la *table des arbres minimaux* associée à $\mathbf{a}$.

Ainsi, grâce à nos choix de structures de données, nous pouvons réaliser les instructions 7 et 8 en grand $\mathcal{O}$ du nombre de prédécesseurs de la racine de $\mathbf{b}$ pour la première et $\mathcal{O}(1)$ pour la seconde. Nous déduisons aussi que les instructions 10 et 12 sont en $\mathcal{O}(|\mathbf{b}| + |\mathbf{x} \otimes \mathcal{D}(\mathbf{a})|^2)$. En effet, il nous faut un temps en $\mathcal{O}(|\mathbf{x} \otimes \mathcal{D}(\mathbf{a})|^2)$ pour calculer les codes des arbres de $\mathbf{x} \otimes \mathcal{D}(\mathbf{a})$. Puis nous faisons les comparaisons/retraits avec un temps en $\mathcal{O}(|\mathbf{b}|)$. Enfin, pour simplifier l'analyse de la complexité, nous transformons la boucle Tant que en des appels récursifs. Avec toutes ces modifications, nous obtenons l'algorithme 1.76. Il prend en entrée T_b , la table ordonnée d'un arbre $\mathbf{b}$ où tous les arbres sont en forme canonique, $\mathbf{a}$, un arbre en forme canonique, et T_a , sa table des arbres minimaux, ainsi que $\mathbf{X}$, le résultat récursivement calculé.

1. État de l'art et définitions – 1.3. Gestion des parties transitoires : Unroll

De là, nous pouvons constater que le temps de calcul entre deux actions consécutives sur la pile des appels récurifs est dans le pire des cas en $\mathcal{O}(|\mathbf{b}|^2)$. En effet, le pire cas correspond à retirer tous les arbres dans T_b . De plus, toutes les instructions possibles entre les appels récurifs sont en $\mathcal{O}(|\mathbf{b}|)$ sauf celles où nous construisons T_x , qui est en $\mathcal{O}(|\mathbf{b}|^2)$.

Ainsi, comme nous réalisons au plus un appel récurif par sommet de $\mathbf{b}$, nous pouvons borner le nombre de ces appels par $|\mathbf{b}|$, nous déduisons que la complexité de l'algorithme est $\mathcal{O}(|\mathbf{b}|^3)$. En ajoutant le temps d'exécution du prétraitement, nous concluons le théorème suivant.

Théorème 1.77. *La complexité de la division de $\mathbf{b}$ par $\mathbf{a}$ est $\mathcal{O}(m^3)$ avec m la taille de $\mathbf{b}$.*

Pour conclure cet état de l'art, nous proposons un tableau récapitulatif de tous les résultats de complexité que nous avons présentés jusqu'à ce point.

Problème	Conditions	complexité	réf
$P(X_1, \dots, X_n) = Q(X_1, \dots, X_n)$	P, Q polynômes quelconques	Indécidable	[11]
$P(X_1, \dots, X_n) = B$	P multivarié sur les FDDS, B FDDS	D NPC	[7]
	P multivarié sur les permutations, B permutations	E EnumP	[46]
$P(X) = B$	P univarié sur les FDDS, B FDDS	D NP	
	P univarié sur les permutations, B permutations		
	P univarié sur $\mathbb{B}_1$, B un élément de $\mathbb{B}_1$	R FP	[42]
$AX = B$	A, B FDDS quelconques	D NP	
	A, B permutation, équation à un premier	R FP	[42]
	A, B permutations générales	D NP	
	$C_a X = q \cdot C_b$, a, b donnés en binaire	D P	[15]
	A, B des dendrons	R FP	[40]
$X^k = B$	B permutation, $k \geq 1$	R FP	[42]
	B FDDS quelconque, $k \geq 1$	D NP	

TABLEAU 1.78. – Tableau récapitulatif de l'état de l'art. Dans les résultats de complexité, nous avons préfixé par D les résultats concernant les problèmes de décision, par R ceux pour la résolution et par E ceux pour l'énumération. Rappelons que si nous pouvons résoudre un problème en temps polynomial, alors le problème de décision qui lui est associé est aussi soluble en temps polynomial.

Deuxième partie

Équations polynomiales résolubles en temps polynomial

2. Résolutions d'équations polynomiales sur les unrolls

Sommaire

2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe	67
2.2. Complexité de la division d'unroll	75
2.3. Complexité de la résolution des équations monomiales sur les unrolls	84
2.4. Complexité de la résolution des équations polynomiales sur les unrolls	93
2.5. Conclusion et perspectives	100

Dans la suite de ce manuscrit, un polynôme sera toujours considéré comme univarié sauf mention explicite du contraire. Soient $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS et B un FDDS. Une méthode qui pourrait permettre de trouver un FDDS X tel que $P(X) = B$ en temps polynomial, sur la somme des tailles des coefficients de P et de B , serait de déterminer $\mathcal{U}(X)$ puis de trouver les longueurs de cycle des composantes connexes de X associées à chacun des arbres. Pour arriver à faire cela, il nous faut développer une solution algorithmique capable de construire une représentation finie de $\mathcal{U}(X)$ en temps polynomial.

Dans ce chapitre, nous commencerons par montrer que nous sommes à même de construire un FDDS X connexe tel que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ en temps polynomial sur la taille de B avec A et B deux FDDS donnés en entrée (Section 2.1). Pour cela, nous établirons que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ admet une solution si et seulement si $\mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(B))$ admet une solution avec n suffisamment grand mais polynomial sur la taille de B (Proposition 2.1). Afin de démontrer cette dernière propriété, nous prouverons qu'il est possible de comparer deux arbres d'unroll à partir de leur troncature à profondeur polynomiale (Lemme 2.4). Nous attirons l'attention du lecteur sur le fait que ce résultat sera utilisé dans les chapitres suivants.

Après quoi, nous démontrerons que nous sommes en mesure de résoudre les équations de la forme $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ en temps polynomial sur la taille de B et ce sans conditions sur X (Section 2.2). Pour cela, nous prouverons que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ admet une solution si et seulement si l'équation sur les forêts fini $\mathcal{C}_n(\mathcal{U}(A))\mathbf{X} = \mathcal{C}_n(\mathcal{U}(B))$, avec n plus grand que le précédent mais toujours polynomial sur la taille de B , admet aussi une solution (Théorème 2.23).

Troisièmement, nous montrerons qu'il est possible de trouver un FDDS X tel que $\mathcal{U}(X)^k = \mathcal{U}(B)$ avec $k \geq 0$ un entier (Section 2.3). Cette démonstration reposera sur le fait que $\mathcal{U}(X)^k = \mathcal{U}(B)$ admet une solution si et seulement s'il existe une forêt $\mathbf{X}$ telle que $\mathbf{X}^k = \mathcal{C}_n(\mathcal{U}(B))$ avec le même n que précédemment (Théorème 2.40).

2. Résolutions d'équations polynomiales sur les unrolls – 2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe

Finalement, nous étendrons ces résultats au polynôme d'unroll (Section 2.4). Nous démontrerons que nous sommes en mesure de calculer efficacement un FDDS X tel que $\sum_{i=0}^m \mathcal{U}(A_i)\mathcal{U}(X)^i = \mathcal{U}(B)$ (Théorème 2.58). Nous montrerons par la même occasion que tous les polynômes d'unroll sont injectifs (Théorème 2.52).

2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe

Le but premier de cette section est d'étendre l'algorithme de décision pour la division d'éléments de $\mathbb{D}^*$ en un algorithme de résolution pour les équations de la forme $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X un FDDS connexe. Pour cela, nous commencerons par généraliser la Proposition 1.72 des dendrons à tous les FDDS. Ceci est résumé dans la proposition suivante.

Proposition 2.1. *Soient A, X et B trois FDDS avec α le nombre d'arbres d'unroll de $\mathcal{U}(B)$. Et soit $n \geq 2\alpha + d(B)$. Alors :*

$$\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B) \iff \mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(B)).$$

Pour parvenir à un tel résultat, nous démontrons le Théorème 2.2. En effet, grâce à lui, nous avons :

$$\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(AX) = \mathcal{U}(B) \iff \mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(AX)) = \mathcal{C}_n(\mathcal{U}(B)),$$

avec $n \geq 2\alpha + d(B)$ où α est le nombre d'arbres de $\mathcal{U}(B)$.

Théorème 2.2. *Soient A et B deux FDDS avec α le nombre d'arbres d'unroll dans $\mathcal{U}(B)$ et soit $n \geq 2\alpha + d(B)$. Alors :*

$$\mathcal{U}(A) = \mathcal{U}(B) \iff \mathcal{C}_n(\mathcal{U}(A)) = \mathcal{C}_n(\mathcal{U}(B)).$$

Afin de démontrer ce théorème, nous utiliserons le fait que nous pouvons considérer les arbres d'unroll comme des mots infinis. En effet, cette représentation nous donne accès aux outils de la combinatoire des mots. Mais pour utiliser ces derniers, il nous faut expliquer comment nous pouvons retrouver les “lettres” de ces mots, c'est-à-dire les arbres finis enracinés dans la branche infinie, à partir de la troncature d'un arbre d'unroll.

Lemme 2.3. *Soit A un FDDS connexe et soit p la longueur du cycle de A . Soit $\mathbf{t}$ un de ses arbres d'unroll et soit $n \geq p + d(A)$ un entier naturel et soit $r = n - d(A)$. Nous considérons $v_n \cdots v_0$ un chemin orienté d'une feuille de $\mathcal{C}_n(\mathbf{t})$ vers sa racine et dont la longueur est n . Alors les sommets $v_r, \dots, v_0$ font nécessairement partie de la branche infinie de $\mathbf{t}$.*

Démonstration. Tout d'abord, remarquons que l'ensemble des arbres finis enracinés dans la branche de $\mathbf{t}$ a une profondeur au plus $d(A)$. Maintenant, pour prouver

2. Résolutions d'équations polynomiales sur les unrolls – 2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe

l'assertion, nous raisonnons par l'absurde. Plus précisément, nous supposons qu'au moins un des sommets $v_r, \dots, v_0$ n'appartient pas à la branche infinie de $\mathbf{t}$ et prouvons que $n < r + d(A)$. Soit v_a le sommet de $v_r, \dots, v_0$ appartenant à la branche infinie de $\mathbf{t}$ et dont la profondeur est maximale; un tel sommet existe toujours car v_0 appartient, par définition, à cette branche. Remarquons que $a < r$ sinon $v_r, \dots, v_0$ ne contiennent que des sommets de la branche infinie. Relevons également que la profondeur de l'arbre enraciné dans v_a est inférieure ou égale à $d(A)$. Ce qui implique que $d(v_n) = n \leq a + d(A)$. Or, comme $a < r$, nous déduisons que $n < r + d(A)$. Ce qui est la contradiction attendue. $\square$

Le premier argument de la preuve du théorème sera que la profondeur où nous tronquons est suffisante pour distinguer chaque arbre de $\mathcal{U}(A)$ et $\mathcal{U}(B)$ à partir de leurs arbres coupés. Pour justifier cet argument, nous donnons pour chaque paire d'arbres d'unroll une borne à partir de laquelle la propriété sera valide.

Lemme 2.4. *Soient A, B deux FDDS. Soient $\mathbf{a} \in \mathcal{U}(A)$ et $\mathbf{b} \in \mathcal{U}(B)$ et soient a et b des périodes quelconques de respectivement $\mathbf{a}$ et $\mathbf{b}$. Soit $n \geq a + b - \gcd(a, b) + \max(d(A), d(B))$ un entier naturel. Alors,*

1. $\mathbf{a} < \mathbf{b}$ si et seulement si $\mathcal{C}_n(\mathbf{a}) < \mathcal{C}_n(\mathbf{b})$
2. $\mathbf{a} = \mathbf{b}$ si et seulement si $\mathcal{C}_n(\mathbf{a}) = \mathcal{C}_n(\mathbf{b})$.

Démonstration. Supposons que $\mathbf{a} = \mathbf{b}$. Alors, par définition $\mathcal{C}_i(\mathbf{a}) = \mathcal{C}_i(\mathbf{b})$ pour tout entier naturel i . Le sens direct du point 2 suit.

Supposons maintenant que $\mathcal{C}_n(\mathbf{a}) < \mathcal{C}_n(\mathbf{b})$. Pour le raisonnement suivant, nous notons par $\langle \mathbf{x} \rangle$ le code de l'arbre $\mathbf{x}$, et par $\langle \mathbf{x} \rangle_i$ le i -ème entier dans ce code (voir Figure 1.62). De l'hypothèse, il existe une différence entre les codes de $\langle \mathcal{C}_n(\mathbf{a}) \rangle$ et $\langle \mathcal{C}_n(\mathbf{b}) \rangle$. Posons $\min$ l'indice de la différence la plus à gauche. Considérons $u_{\mathbf{a}}$ et $u_{\mathbf{b}}$ les sommets correspondant à $\min$ dans respectivement $\mathcal{C}_n(\mathbf{a})$ et $\mathcal{C}_n(\mathbf{b})$. Pour montrer que $\mathbf{a} < \mathbf{b}$, il suffit de prouver que les profondeurs de $u_{\mathbf{a}}$ et de $u_{\mathbf{b}}$ sont au plus $n - 1$. En effet, comme le code est basé sur un parcours en largeur, les mots formés par les codes de $\mathcal{C}_n(\mathbf{a})$ et $\mathcal{C}_n(\mathbf{b})$ auxquels nous retirons les entiers associés aux sommets de profondeur n sont des préfixes de respectivement $\mathbf{a}$ et $\mathbf{b}$. Nous remarquons que $u_{\mathbf{a}}$ et $u_{\mathbf{b}}$ sont de même profondeur. Donc comme le code d'une feuille est 0, il suit que $u_{\mathbf{b}}$ ne peut pas être une feuille. En effet, $\langle \mathcal{C}_n(\mathbf{b}) \rangle_{\min} > \langle \mathcal{C}_n(\mathbf{a}) \rangle_{\min} \geq 0$. Ainsi, $d(u_{\mathbf{b}}) = d(u_{\mathbf{a}}) < n$ puisque tous les sommets de profondeur n dans $\langle \mathcal{C}_n(\mathbf{b}) \rangle_{\min}$ sont des feuilles. Nous concluons que la différence la plus à gauche entre $\mathcal{C}_n(\mathbf{a})$ et $\mathcal{C}_n(\mathbf{b})$ est aussi la différence la plus à gauche entre $\mathbf{a}$ et $\mathbf{b}$. La réciproque du point 1 suit.

Dans cette preuve, nous ajoutons que a et b sont minimales. Mais le résultat suit directement sans cette restriction. Pour montrer que $\mathbf{a} < \mathbf{b}$ implique que $\mathcal{C}_n(\mathbf{a}) < \mathcal{C}_n(\mathbf{b})$ (sens direct du point 1), nous raisonnons par contraposée, c'est-à-dire que nous allons prouver que $\mathcal{C}_n(\mathbf{a}) \geq \mathcal{C}_n(\mathbf{b})$ implique que $\mathbf{a} \geq \mathbf{b}$. Deux cas sont donc possibles. Le premier est que $\mathcal{C}_n(\mathbf{a}) > \mathcal{C}_n(\mathbf{b})$. Et par le point précédent, $\mathbf{a} > \mathbf{b}$.

La preuve du second cas revient à prouver la réciproque du point 2. Nous supposons donc que $\mathcal{C}_n(\mathbf{a}) = \mathcal{C}_n(\mathbf{b})$. Puisque $\mathbf{a}$ et $\mathbf{b}$ sont deux arbres d'unroll, nous pouvons les

2. Résolutions d'équations polynomiales sur les unrolls – 2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe

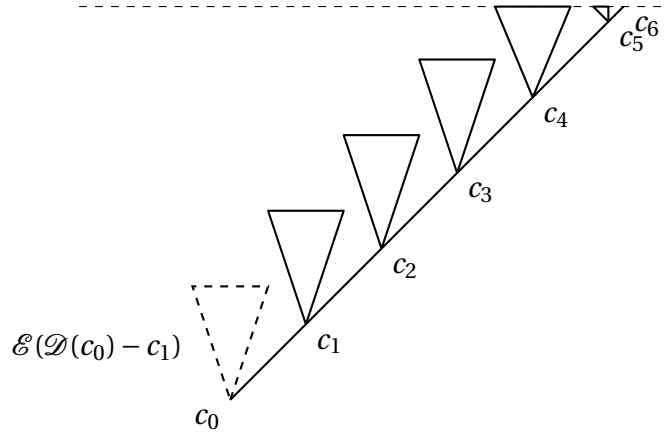


FIGURE 2.5. – Exemple d'un arbre tronqué à la profondeur 6. Les triangles en pointillé illustrent $\mathcal{E}(\mathcal{D}(\mathbf{c}_0) - \mathbf{c}_1)$. La figure est issue de [18].

$\mathbf{a}_0 = \mathbf{b}_0 = \mathbf{b}_3$	$\mathbf{a}_0 = \mathbf{b}_0 = \mathbf{b}_3 = \mathbf{a}_2$
$\mathbf{a}_1 = \mathbf{b}_1$	$\mathbf{a}_1 = \mathbf{b}_1 = \mathbf{b}_0 = \mathbf{a}_0$
$\mathbf{a}_2 = \mathbf{b}_2$	$\mathbf{a}_2 = \mathbf{b}_2 = \mathbf{b}_1 = \mathbf{a}_1$

FIGURE 2.6. – Sur la gauche, le cas où nous avons considéré $n = \max(a, b) + \max(d(a), d(b))$ et sur la droite le cas où nous avons considéré $n = a + b - \gcd(a, b) + \max(d(a), d(b))$. Nous pouvons voir que dans le cas de gauche, il n'est pas possible de continuer la preuve par identification des $\mathbf{a}_i$ avec des $\mathbf{a}_j$. La figure est issue de [18].

considérer comme deux mots périodiques, $w_{\mathbf{a}}w_{\mathbf{a}}\dots$ et $w_{\mathbf{b}}w_{\mathbf{b}}\dots$ avec $w_{\mathbf{a}} = \mathbf{a}_0 \cdots \mathbf{a}_{a-1}$ et $w_{\mathbf{b}} = \mathbf{b}_0 \cdots \mathbf{b}_{b-1}$. De surcroît, puisque n est suffisamment grand, par le Lemme 2.3, nous pouvons identifier les $a + b - \gcd(a, b)$ premiers sommets issus des branches infinies dans $\mathcal{C}_n(\mathbf{a})$ et $\mathcal{C}_n(\mathbf{b})$. Nous pouvons ainsi reconstruire $\mathbf{a}_i$ et $\mathbf{b}_i$ pour chaque entier naturel $0 \leq i < a + b - \gcd(a, b)$.

Pour ce faire, nous considérons $\mathbf{c}_i$ (resp. $\mathbf{d}_i$) l'arbre enraciné dans le i -ème nœud de la troncature de la branche infinie de $\mathcal{C}_n(\mathbf{a})$ (resp. $\mathcal{C}_n(\mathbf{b})$). Alors, $\mathcal{D}(\mathbf{c}_i)$ et $\mathcal{D}(\mathbf{d}_i)$ contiennent un et seulement un arbre avec profondeur $n - i - 1$, à savoir $\mathbf{c}_{i+1}$ et $\mathbf{d}_{i+1}$ respectivement. Ceci implique deux choses :

1. $\mathcal{E}(\mathcal{D}(\mathbf{c}_i) - \mathbf{c}_{i+1}) = \mathcal{E}(\mathcal{D}(\mathbf{d}_i) - \mathbf{d}_{i+1})$,
2. $\mathbf{a}_i = \mathcal{E}(\mathcal{D}(\mathbf{c}_i) - \mathbf{c}_{i+1})$ et $\mathbf{b}_i = \mathcal{E}(\mathcal{D}(\mathbf{d}_i) - \mathbf{d}_{i+1})$ (voir la Figure 2.5 pour un exemple).

Nous concluons donc que $\mathbf{a}_i = \mathbf{b}_i$ pour tout entier naturel i entre 0 et $a + b - \gcd(a, b) - 1$ inclus. Ainsi, les mots périodiques associés à $\mathbf{a}$ et $\mathbf{b}$ sont égaux sur leurs $a + b - \gcd(a, b)$ premières lettres, donc par le deuxième théorème de la périodicité de [20], il résulte que $w_{\mathbf{a}} \cdots = w_{\mathbf{b}} \cdots$. En d'autres mots, $\mathbf{a} = \mathbf{b}$. (Pour avoir une intuition de pourquoi le facteur $a + b - \gcd(a, b)$ est important ici, voir la Figure 2.6). $\square$

Afin d'utiliser le Lemme 2.4 dans la preuve du Théorème 2.2, il nous faut montrer

2. Résolutions d'équations polynomiales sur les unrolls – 2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe

que $2\alpha + d(B)$, avec α le nombre d'arbres de $\mathcal{U}(B)$, est bien supérieur à tous les $a + b - \gcd(a, b) + \max(d(A), d(B))$ avec a et b des périodes minimales d'arbres d'unroll de $\mathcal{U}(A)$ et $\mathcal{U}(B)$ respectivement. Or, par définition de l'unroll, α borne chaque plus petite période d'arbres d'unroll de $\mathcal{U}(B)$. En effet, si une composante connexe C de longueur de cycle c admet comme arbre d'unroll $\mathbf{t}$ de plus petite période p , alors $c \geq p$. Cela est dû au fait que chaque arbre dans $\mathcal{U}(C)$ est par définition de période c . Donc α borne toutes les plus petites périodes d'arbres d'unroll de $\mathcal{U}(B)$. Dans le même temps, pour que deux forêts soient égales, elles doivent avoir le même nombre d'arbres. Ainsi, α borne aussi toutes les plus petites périodes d'arbres d'unroll dans $\mathcal{U}(A)$. Ceci implique que $2\alpha \geq a + b \geq a + b - \gcd(a, b)$ pour n'importe quels a et b définis ci-dessus. Il nous reste donc à montrer que $d(B) \geq \max(d(A), d(B))$. Ceci est le propos du lemme suivant.

Lemme 2.7. Soient A et B deux FDDS avec α le nombre d'arbres d'unroll de $\mathcal{U}(B)$. Soit $n \geq 2\alpha + d(B)$. Si $\mathcal{C}_n(\mathcal{U}(A)) = \mathcal{C}_n(\mathcal{U}(B))$ alors $d(A) = d(B)$.

Démonstration. Nous allons montrer la contraposée. Supposons que $d(A) \neq d(B)$. Sans perte de généralité, nous choisissons que $d(B) < d(A)$. Puisque n est suffisamment grand, nous déduisons que, pour tout arbre $\mathbf{b}$ de $\mathcal{C}_n(\mathcal{U}(B))$, dans la forêt $\mathcal{D}(\mathbf{b})$ nous avons exactement un arbre de profondeur $n - 1$ et tous les autres arbres ont une profondeur au plus $d(B) - 1$.

Cependant, il existe un arbre $\mathbf{a} \in \mathcal{C}_n(\mathcal{U}(A))$ tel que $\mathcal{D}(\mathbf{a})$ contient un arbre de profondeur $n - 1$ et au moins un autre arbre de profondeur $\min(n - 1, d(A) - 1)$. Donc, comme $d(B) - 1 < d(A) - 1$, il s'ensuit que $\mathcal{D}(\mathbf{b}) \neq \mathcal{D}(\mathbf{a})$ pour tout arbre $\mathbf{b}$ de $\mathcal{C}_n(\mathcal{U}(B))$. Ceci induit que $\mathbf{a}$ n'est pas un arbre de $\mathcal{C}_n(\mathcal{U}(B))$, nous en concluons que $\mathcal{C}_n(\mathcal{U}(A)) \neq \mathcal{C}_n(\mathcal{U}(B))$. $\square$

De ce lemme, nous avons que $2\alpha + d(B)$ borne $a + b - \gcd(a, b) + \max(d(A), d(B))$ pour tous a et b des périodes minimales d'arbres d'unroll de respectivement $\mathcal{U}(A)$ et $\mathcal{U}(B)$, avec A et B des FDDS. Nous avons donc tout ce qui est nécessaire pour notre preuve.

Preuve du Théorème 2.2. ($\Rightarrow$) Si $\mathcal{U}(A) = \mathcal{U}(B)$ alors $\mathcal{C}_i(\mathcal{U}(A)) = \mathcal{C}_i(\mathcal{U}(B))$ pour tout entier naturel i .

($\Leftarrow$) Nous supposons que $\mathcal{C}_n(\mathcal{U}(A)) = \mathcal{C}_n(\mathcal{U}(B))$. Tout d'abord, nous expliquons comment nous reconstruisons $\mathcal{U}(A)$ et $\mathcal{U}(B)$ à partir de $\mathcal{C}_n(\mathcal{U}(A))$ et $\mathcal{C}_n(\mathcal{U}(B))$ respectivement.

Comme la troncature ne change pas le nombre d'arbres, nous supposons que $\mathcal{U}(A) = \sum_{i=1}^{\alpha} \mathbf{a}_i$ et $\mathcal{U}(B) = \sum_{i=1}^{\alpha} \mathbf{b}_i$. Nous posons a_i (resp. b_i) la plus petite période de $\mathbf{a}_i$ (resp. $\mathbf{b}_i$). De plus, comme nous l'avons expliqué dans le paragraphe précédant cette démonstration, nous avons que $n \geq a_i + b_j - \gcd(a_i, b_j) + \max(d(A), d(B))$ pour tous les a_i et tous les b_j .

Maintenant, puisque $\mathcal{C}_n(\mathcal{U}(A)) = \mathcal{C}_n(\mathcal{U}(B))$, nous obtenons qu'il existe une fonction bijective $f : \{1, \dots, \alpha\} \rightarrow \{1, \dots, \alpha\}$ telle que $\mathcal{C}_n(\mathbf{a}_i) = \mathcal{C}_n(\mathbf{b}_{f(i)})$. Et donc comme par le point précédent n est suffisamment grand, par le Lemme 2.4, nous inférons que $\mathbf{a}_i = \mathbf{b}_{f(i)}$, et ce pour tout i . Nous concluons ainsi que $\mathcal{U}(A) = \mathcal{U}(B)$. $\square$

2. Résolutions d'équations polynomiales sur les unrolls – 2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe

Remarque 2.8. Notons que $\mathcal{C}_n(\mathcal{U}(A))$ contient au plus m arbres avec m la taille de A . De plus, la taille des arbres finis enracinés dans les sommets des chemins infinis de $\mathcal{U}(A)$ est bornée par m . Donc la taille d'un arbre de $\mathcal{C}_n(\mathcal{U}(A))$ est au plus nm . Ainsi, puisque n est linéaire sur m , il résulte que la taille d'un arbre de $\mathcal{C}_n(\mathcal{U}(A))$ est en $\mathcal{O}(m^2)$ et, conséquemment, $\mathcal{C}_n(\mathcal{U}(A))$ a une taille en $\mathcal{O}(m^3)$.

Ainsi, pour résoudre $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe, il nous suffit de diviser $\mathcal{C}_n(\mathcal{U}(B))$ par $\mathcal{C}_n(\mathcal{U}(A))$, avec un n suffisamment grand, puis de vérifier que le quotient est la troncature de l'unroll d'un FDDS connexe. Pour la première étape, nous allons expliquer comment utiliser l'Algorithme 1.76 pour diviser les forêts.

Lemme 2.9. Soient $\mathbf{A}$, $\mathbf{X}$, et $\mathbf{B}$ trois forêts finies. Alors, nous avons que $\mathbf{AX} = \mathbf{B}$ si et seulement si $\mathcal{E}(\mathbf{A})\mathcal{E}(\mathbf{X}) = \mathcal{E}(\mathbf{B})$.

Démonstration. Remarquons que deux forêts $\mathbf{F}_1, \mathbf{F}_2$ sont égales si et seulement si $\mathcal{E}(\mathbf{F}_1) = \mathcal{E}(\mathbf{F}_2)$. En effet, si $\mathbf{F}_1 = \mathbf{F}_2$ alors $\mathcal{E}(\mathbf{F}_1) = \mathcal{E}(\mathbf{F}_2)$. Pour la direction opposée, si $\mathcal{E}(\mathbf{F}_1) = \mathcal{E}(\mathbf{F}_2)$ alors :

$$\mathbf{F}_1 = \mathcal{D}(\mathcal{E}(\mathbf{F}_1)) = \mathcal{D}(\mathcal{E}(\mathbf{F}_2)) = \mathbf{F}_2$$

De là, $\mathbf{AX} = \mathbf{B}$ si et seulement si $\mathcal{E}(\mathbf{AX}) = \mathcal{E}(\mathbf{B})$. Et comme $\mathcal{E}(\mathbf{X})$ se distribue sur le produit d'arbres, $\mathcal{E}(\mathbf{AX}) = \mathcal{E}(\mathbf{B})$ si et seulement si $\mathcal{E}(\mathbf{A})\mathcal{E}(\mathbf{X}) = \mathcal{E}(\mathbf{B})$. $\square$

Avant d'expliquer comment il est possible de vérifier qu'une forêt est la troncature de l'unroll d'un FDDS connexe, nous remarquons que grâce à ce résultat, nous pouvons étendre le Lemme 1.57 des arbres aux forêts.

Lemme 2.10. Soient $\mathbf{A}$, $\mathbf{X}$, et $\mathbf{Y}$ trois forêts finies. Alors, nous avons que $\mathbf{AX} = \mathbf{AY}$ si et seulement si $\mathcal{C}_{d(\mathbf{A})}(\mathbf{X}) = \mathcal{C}_{d(\mathbf{A})}(\mathbf{Y})$.

Démonstration. ($\Leftarrow$) Par la définition du produit d'arbres, tous les sommets de $\mathbf{X}$ (resp., $\mathbf{Y}$) qui ont une profondeur strictement supérieure à $d(\mathbf{A})$ n'ont pas d'impact sur le produit $\mathbf{AX}$ (resp., $\mathbf{AY}$). Ainsi, il résulte que $\mathbf{AX} = \mathbf{A}\mathcal{C}_{d(\mathbf{A})}(\mathbf{X})$ et aussi $\mathbf{AY} = \mathbf{A}\mathcal{C}_{d(\mathbf{A})}(\mathbf{Y})$. Et puisque $\mathcal{C}_{d(\mathbf{A})}(\mathbf{X}) = \mathcal{C}_{d(\mathbf{A})}(\mathbf{Y})$, nous concluons que $\mathbf{AX} = \mathbf{AY}$.

($\Rightarrow$) Pour la réciproque, nous supposons que $\mathbf{AX} = \mathbf{AY}$. Par le Lemme 2.9, nous déduisons que $\mathcal{E}(\mathbf{A})\mathcal{E}(\mathbf{X}) = \mathcal{E}(\mathbf{A})\mathcal{E}(\mathbf{Y})$. De là, comme $\mathcal{E}(\mathbf{A})$, $\mathcal{E}(\mathbf{X})$, et $\mathcal{E}(\mathbf{Y})$ sont des arbres finis, par le Lemme 1.57, nous obtenons l'équation suivante :

$$\mathcal{C}_{d(\mathcal{E}(\mathbf{A}))}(\mathcal{E}(\mathbf{X})) = \mathcal{C}_{d(\mathcal{E}(\mathbf{A}))}(\mathcal{E}(\mathbf{Y})) \quad (1)$$

Relevons que $\mathcal{E}(\mathcal{C}_i(\mathbf{F})) = \mathcal{C}_{i+1}(\mathcal{E}(\mathbf{F}))$ et que $\mathcal{D}(\mathcal{C}_i(\mathbf{F})) = \mathcal{C}_{i-1}(\mathcal{D}(\mathbf{F}))$ et ce pour toute forêt $\mathbf{F}$ et tout entier $i > 0$. Et en particulier, si $\mathbf{F} = \mathcal{E}(\mathbf{X})$ et $i = d(\mathbf{A}) + 1 = d(\mathcal{E}(\mathbf{A}))$, alors

$$\mathcal{D}(\mathcal{C}_{d(\mathcal{E}(\mathbf{A}))}(\mathcal{E}(\mathbf{X}))) = \mathcal{C}_{d(\mathbf{A})}(\mathbf{X}).$$

De même, $\mathcal{D}(\mathcal{C}_{d(\mathcal{E}(\mathbf{A}))}(\mathcal{E}(\mathbf{Y}))) = \mathcal{C}_{d(\mathbf{A})}(\mathbf{Y})$. Donc, en appliquant $\mathcal{D}(\mathbf{X})$ des deux côtés dans l'équation (1), nous concluons que $\mathcal{C}_{d(\mathbf{A})}(\mathbf{X}) = \mathcal{C}_{d(\mathbf{A})}(\mathbf{Y})$. $\square$

2. Résolutions d'équations polynomiales sur les unrolls – 2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe

La propriété décrite dans le lemme ci-dessus implique que les forêts de profondeur d sont annulables dans le semi-anneau $(\mathbb{F}_d, +, \otimes)$. Nous pouvons généraliser ce résultat aux unrolls. En effet, considérons A, X et Y trois FDDS. Du Théorème 2.2, il résulte que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(A)\mathcal{U}(Y)$ si et seulement si $\mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(Y))$, pour un entier n suffisamment grand. Or, comme $\mathcal{C}_n(\mathcal{U}(A))$ est annulable dans le semi-anneau $(\mathbb{F}_n, +, \otimes)$, nous déduisons que $\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(Y))$. Et de nouveau par le Théorème 2.2, car n est suffisamment grand, $\mathcal{U}(X) = \mathcal{U}(Y)$. Cette réflexion prouve le théorème suivant.

Théorème 2.11. *Soient A, X, Y trois FDDS. Alors, $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(A)\mathcal{U}(Y)$ si et seulement si $\mathcal{U}(X) = \mathcal{U}(Y)$.*

Expliquons maintenant comment vérifier qu'une forêt finie est bien la troncature de l'unroll d'un FDDS connexe. Pour cela, nous allons introduire la notion d'*enroulement* d'un arbre $\mathbf{t}$ à période p . Cette fonction ne sera définie que sur les arbres de profondeur $d \geq p$ et telle que parmi les arbres enracinés dans les sommets de profondeur $p - 1$, il n'existe qu'un unique arbre $\mathbf{s}$ de profondeur $d - (p - 1)$. Cette condition nous servira à rendre l'opération d'enroulement déterministe et peut être réécrite comme suit : $\mathcal{D}^p(\mathbf{t})$ ne contient qu'un unique arbre de profondeur maximale, avec $\mathcal{D}^p$ signifiant l'application successive de $\mathcal{D}$ p fois.

Définition 2.12 (Enroulement). Soit p un entier naturel et soit $\mathbf{t}$ un arbre de racine t tels que $\mathcal{D}^p(\mathbf{t})$ ne contient qu'un unique arbre $\mathbf{s}$ de profondeur maximale. Soit u le successeur de la racine de $\mathbf{s}$ dans $\mathbf{t}$. L'*enroulement* de $\mathbf{t}$ à période p , noté $\mathcal{R}_p(\mathbf{t})$, est le FDDS connexe construit en :

1. retirant l'arbre $\mathbf{s}$ de $\mathbf{t}$,
2. ajoutant un arc de t vers u .

Un exemple d'enroulement est présenté dans la Figure 2.13.

Intuitivement, si $\mathbf{t}$ est la troncature d'un arbre d'unroll $\mathbf{t}'$ de période p à profondeur suffisante alors $\mathbf{t}'$ est un arbre de l'unroll de $\mathcal{R}_p(\mathbf{t})$. De plus, les sommets entre u et t dans $\mathbf{t}$ forment le cycle de $\mathcal{R}_p(\mathbf{t})$ et font partie de la branche infinie de $\mathbf{t}'$.

Dans la suite, nous effectuerons toujours l'enroulement d'arbres vérifiant les conditions nécessaires, comme nous le justifions dans le lemme suivant.

Lemme 2.14. *Soit $k > 0$ un entier. Soient A et B deux FDDS. Soient $\mathbf{a}$ et $\mathbf{b}$ deux arbres de respectivement $\mathcal{U}(A)$ et $\mathcal{U}(B)$. Soient a et b les périodes minimales de respectivement $\mathbf{a}$ et $\mathbf{b}$. Soit $n \geq a + b - \gcd(a, b) + d(B)$ un entier. S'il existe $\mathbf{x}$ tel que $\mathcal{C}_n(\mathbf{a})\mathbf{x}^k = \mathcal{C}_n(\mathbf{b})$ alors $\mathcal{D}^i(\mathcal{C}_n(\mathbf{x}))$ ne contient qu'un unique arbre de profondeur maximale et ce pour tout entier $0 \leq i < n - d(B)$.*

Démonstration. Supposons qu'il existe $\mathbf{x}$ tel que $\mathcal{C}_n(\mathbf{a})\mathbf{x}^k = \mathcal{C}_n(\mathbf{b})$. De plus, par l'absurde, supposons qu'il existe un entier $0 \leq i < n - d(B)$ tel que $\mathcal{D}^i(\mathcal{C}_n(\mathbf{x}))$ contient au moins deux arbres de profondeur maximale.

2. Résolutions d'équations polynomiales sur les unrolls – 2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe

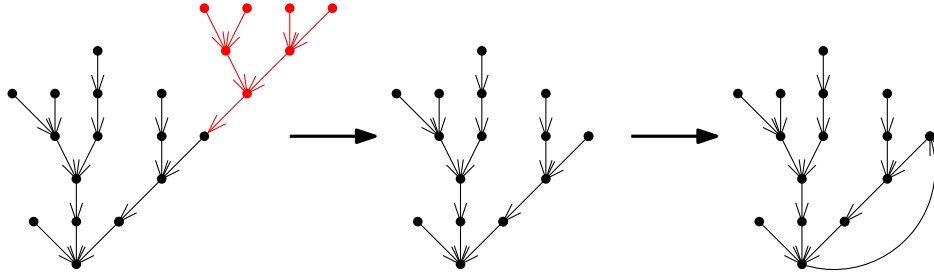


FIGURE 2.13. – Enroulement à période $p = 4$ de l'arbre $\mathbf{t}$ à gauche. Remarquons que $\mathcal{D}^4$ de cet arbre ne contient qu'un seul arbre $\mathbf{s}$ de profondeur maximale étant dans cette figure constitué des sommets rouges, nous pouvons donc bien effectuer l'enroulement. L'arbre du milieu représente l'arbre après que nous ayons réalisé l'opération 1, c'est-à-dire le retrait de $\mathbf{s}$. Le système de droite est le résultat de l'opération 2, c'est-à-dire l'ajout d'un arc de la racine t de $\mathbf{t}$ vers le successeur u de la racine de $\mathbf{s}$. Relevons que nous n'aurions pas pu réaliser l'enroulement à période 5 de l'arbre de gauche car $\mathcal{D}^5$ contient deux arbres de profondeur maximale.

Comme $\mathcal{C}_n(\mathbf{a})\mathbf{x}^k = \mathcal{C}_n(\mathbf{b})$ il suit que $d(\mathbf{x}) \geq n$ puisque $\mathcal{C}_n(\mathbf{b})$ a profondeur n . Donc, $\mathcal{C}_n(\mathbf{x})$ est de profondeur n et par conséquent $\mathcal{D}^i(\mathcal{C}_n(\mathbf{x}))$ contient un arbre de profondeur $n - i - 1$. Ainsi, comme $n - i - 1$ est la plus grande profondeur possible pour les arbres de $\mathcal{D}^i(\mathcal{C}_n(\mathbf{x}))$, nous déduisons que $\mathcal{D}^i(\mathcal{C}_n(\mathbf{x}))$ contient au moins deux arbres de profondeur $n - i - 1$ et donc $\mathcal{D}^i(\mathcal{C}_n(\mathbf{x}))^k$ contient au moins 2^k arbres de profondeur $n - i - 1$.

Observons que puisque n est suffisamment grand, par le Lemme 2.3, nous pouvons identifier dans $\mathcal{C}_n(\mathbf{b})$ le sommet b_{i+1} de la branche infinie de $\mathbf{b}$ ayant profondeur $i + 1$. Remarquons de plus que dans $\mathcal{D}^i(\mathcal{C}_n(\mathbf{b}))$, seul l'arbre enraciné dans b_{i+1} a profondeur maximale. En effet, cet arbre a profondeur $n - i - 1 > d(B) - 1$ tandis que tous les autres arbres de $\mathcal{D}^i(\mathcal{C}_n(\mathbf{b}))$ ont profondeur au plus $d(B) - 1$ puisqu'ils sont issus d'arbres finis enracinés sur la branche infinie de $\mathbf{b}$.

Maintenant comme $\mathcal{C}_n(\mathbf{a})$ est de profondeur n , car $\mathbf{a}$ est de profondeur infinie, il résulte que $\mathcal{D}^i(\mathcal{C}_n(\mathbf{a}))$ contient au moins un arbre de profondeur $n - i - 1$. De là, $\mathcal{D}^i(\mathcal{C}_n(\mathbf{a}))\mathcal{D}^i(\mathcal{C}_n(\mathbf{x}))^k$ contient au moins 2^k arbres de profondeur $n - i - 1$. Nous en concluons que $\mathcal{D}^i(\mathcal{C}_n(\mathbf{a}))\mathcal{D}^i(\mathcal{C}_n(\mathbf{x}))^k \neq \mathcal{D}^i(\mathcal{C}_n(\mathbf{b}))$ induisant que $\mathcal{C}_n(\mathbf{a})\mathcal{C}_n(\mathbf{x})^k \neq \mathcal{C}_n(\mathbf{b})$, contradiction. $\square$

De cette définition, nous déduisons que l'enroulement peut être réalisé via un simple parcours de l'arbre $\mathbf{t}$. Donc le temps requis pour créer $\mathcal{R}_p(\mathbf{t})$ est linéaire sur la taille de $\mathbf{t}$. De plus, une conséquence immédiate de la définition est exposée dans le lemme suivant. Ce dernier justifie pourquoi nous nous sommes intéressés au cas où X est connexe.

Lemme 2.15. Soit $\mathbf{X}$ une forêt de profondeur d composée de p arbres. S'il existe un FDDS X tel que $\mathcal{C}_d(\mathcal{U}(X)) = \mathbf{X}$ alors $X = \mathcal{R}_p(\mathbf{t})$ avec $\mathbf{t}$ un arbre de $\mathbf{X}$.

2. Résolutions d'équations polynomiales sur les unrolls – 2.1. Complexité de la résolution $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X connexe

Algorithme 2.16 UnrollDivisionConnexe(A, B)

```

1: Si  $|A| > |B|$  alors
2:   renvoyer  $\perp$ ,
3: soit  $\alpha$  le nombre d'arbres de  $\mathcal{U}(B)$  et soit  $n \leftarrow 2\alpha + d(B)$ ,
4:  $\mathbf{A} \leftarrow \mathcal{C}_n(\mathcal{U}(A)), \mathbf{B} \leftarrow \mathcal{C}_n(\mathcal{U}(B))$ ,
5:  $\mathbf{a} \leftarrow \min(\mathbf{A}), \mathbf{b} \leftarrow \min(\mathbf{B})$ ,
6:  $\mathbf{x} \leftarrow \text{Division}(\mathbf{a}, \mathbf{b})$  (Algorithme 1.76),
7: construire le FDDS connexe  $X$  issu de l'enroulement de l'arbre  $\mathbf{x}$  à période  $p$  en
   s'assurant que la taille de  $X$  soit inférieure à celle de  $B$ , avec  $p = \alpha / \alpha_A$  où  $\alpha_A$  est le
   nombre d'arbres de  $\mathcal{U}(A)$ ,
8:  $\mathbf{X} \leftarrow \mathcal{C}_n(\mathcal{U}(X))$ ,
9: Si  $\mathbf{AX} \neq \mathbf{B}$  alors
10:  renvoyer  $\perp$ ,
11: renvoyer  $X$ .
```

Soient A et B deux FDDS. Nous pouvons maintenant formuler l'Algorithme 2.16 trouvant, s'il existe, un FDDS connexe X tel que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ (cet algorithme prenant A et B en entrée). Le temps de calcul de cet algorithme est de plus polynomial sur la taille de A et B . Constatons bien que cette méthode ne permet pas de résoudre en temps polynomial les équations de la forme $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$, car il existe des cas où $\mathcal{U}(X)$ n'est pas l'unroll d'un FDDS connexe comme le montre la Figure 2.18.

Proposition 2.17. *Soient A et B des FDDS. La résolution de $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec X un FDDS connexe (Algorithme 2.16) a une complexité en $\mathcal{O}(m^6)$ où m est la taille de B .*

Démonstration. Montrons tout d'abord la correction de l'Algorithme 2.16. L'instruction 9 et la Proposition 2.1 impliquent que si l'algorithme renvoie un FDDS X alors il est correct. Il reste donc à prouver que s'il existe un FDDS connexe X tel que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ alors X est bien renvoyé.

À cette fin, supposons qu'un tel X existe. Donc $\mathcal{C}_n(\mathcal{U}(AX)) = \mathcal{C}_n(\mathcal{U}(B))$ et la longueur du cycle de X est p . De ce fait, nous déduisons tout d'abord que l'enroulement de n'importe quel arbre de $\mathcal{C}_n(\mathcal{U}(X))$ à période p est X . Donc par le Lemme 2.15, il reste à montrer que l'arbre $\mathbf{x}$ de l'instruction 7 est un arbre de $\mathcal{C}_n(\mathcal{U}(X))$. Pour ce faire, remarquons que puisque l'ordre total sur les arbres est compatible avec le produit, il suit que $\min(\mathcal{U}(A))\min(\mathcal{U}(X)) = \min(\mathcal{U}(B))$. Et donc, par la Proposition 2.1, nous avons aussi $\mathcal{C}_n(\min(\mathcal{U}(A)))\mathcal{C}_n(\min(\mathcal{U}(X))) = \mathcal{C}_n(\min(\mathcal{U}(B)))$. Maintenant, par le Lemme 2.4, nous déduisons que $\mathcal{C}_n(\min(\mathcal{U}(A))) = \min(\mathcal{C}_n(\mathcal{U}(A)))$, de même $\mathcal{C}_n(\min(\mathcal{U}(X))) = \min(\mathcal{C}_n(\mathcal{U}(X)))$ et $\mathcal{C}_n(\min(\mathcal{U}(B))) = \min(\mathcal{C}_n(\mathcal{U}(B)))$. Ceci implique que $\mathbf{b}$ est bien divisible par $\mathbf{a}$ avec $\mathbf{b}$ et $\mathbf{a}$ les arbres de l'étape 5. De là, par la correction de l'algorithme de division et par le Lemme 1.57, il suit que $\mathbf{x}$ est bien l'unique quotient de $\mathbf{b}$ par $\mathbf{a}$ de profondeur n . En d'autres mots, $\mathbf{x} = \min(\mathcal{C}_n(\mathcal{U}(X)))$ et la correction suit.

Soit m la taille de B . Pour l'étude de la complexité, nous avons que dans le pire des cas, la taille de A est inférieure ou égale à m . Donc par la Remarque 2.8, la taille de

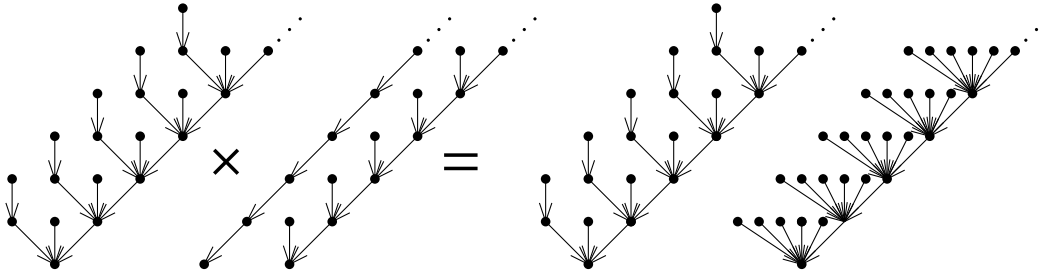


FIGURE 2.18. – Exemple d'une équation de la forme $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ admettant une solution n'étant pas l'unroll d'un système connexe.

$\mathcal{C}_n(\mathcal{U}(A))$ et $\mathcal{C}_n(\mathcal{U}(B))$ est en $\mathcal{O}(m^3)$. De plus, nous constatons que nous pouvons calculer ces troncatures via des parcours de A et de B correspondant aux parcours de $\mathcal{C}_n(\mathcal{U}(A))$ et de $\mathcal{C}_n(\mathcal{U}(B))$. Donc le temps de calcul de $\mathcal{C}_n(\mathcal{U}(A))$ et de $\mathcal{C}_n(\mathcal{U}(B))$ est en $\mathcal{O}(m^3)$.

Afin d'effectuer la recherche des éléments minimaux de $\mathcal{C}_n(\mathcal{U}(A))$ et $\mathcal{C}_n(\mathcal{U}(B))$, il nous faut calculer leurs codes, ce qui est réalisable en temps quadratique sur leurs tailles (Lemme 1.64). Ceci est donc faisable avec un temps en $\mathcal{O}(m^5)$ (le calcul du code d'un arbre est en $\mathcal{O}((m^2)^2)$ car la taille d'un arbre est en $\mathcal{O}(m^2)$ par la Remarque 2.8 et nous en calculons au plus m pour $\mathcal{C}_n(\mathcal{U}(A))$ et m pour $\mathcal{C}_n(\mathcal{U}(B))$). Et de ce fait, la recherche de **a** et **b** sera en $\mathcal{O}(m^3)$ (la comparaison entre deux arbres est en $\mathcal{O}(m^2)$ et nous en réalisons au plus $\mathcal{O}(m)$ pour trouver **a** et m pour trouver **b**).

Le calcul de la division de **b** par **a** est réalisable en $\mathcal{O}(m^6)$ (Théorème 1.77) car **a** et **b** ont une taille en $\mathcal{O}(m^2)$. De plus, la taille du quotient, s'il existe, est en $\mathcal{O}(m^2)$. Ceci implique que le calcul de X via l'enroulement est en $\mathcal{O}(m^2)$. Dans le pire des cas, ce dernier aura une taille bornée par m . Donc sa troncature a une taille en $\mathcal{O}(m^3)$ et est effectuable en $\mathcal{O}(m^3)$. Ainsi, dans le pire des cas $\mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(X))$ a une taille en $\mathcal{O}(m^6)$ et le temps de ce calcul est linéaire sur cette dernière. Comme le test d'isomorphisme de forêt est linéaire [28], il suit que vérifier que $\mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(B))$ est en temps $\mathcal{O}(m^6)$. Nous concluons donc que le temps de calcul de l'algorithme est en $\mathcal{O}(m^6)$ et la proposition suit. $\square$

Relevons encore une fois que la non-existence d'un FDDS X connexe tel que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ n'implique pas l'inexistence d'une solution, comme le montre la Figure 2.18. Nous étendrons cet algorithme aux unrolls généraux dans la Section 2.2.

2.2. Complexité de la division d'unroll

Bien que nous puissions trouver en temps polynomial le quotient de la division de deux forêts, nous ne pouvons en déduire un algorithme ni de décision ni de résolution dans le cas général de la division de deux unrolls. En effet, si la Proposition 2.1 est un premier pas, elle ne montre pas que l'existence d'une solution sur les troncatures implique celle d'une solution sur les unrolls. Autrement dit, elle n'établit pas que s'il existe une forêt **X** telle que $\mathcal{C}_n(\mathcal{U}(A))\mathbf{X} = \mathcal{C}_n(\mathcal{U}(B))$ alors il existe un FDDS X tel que

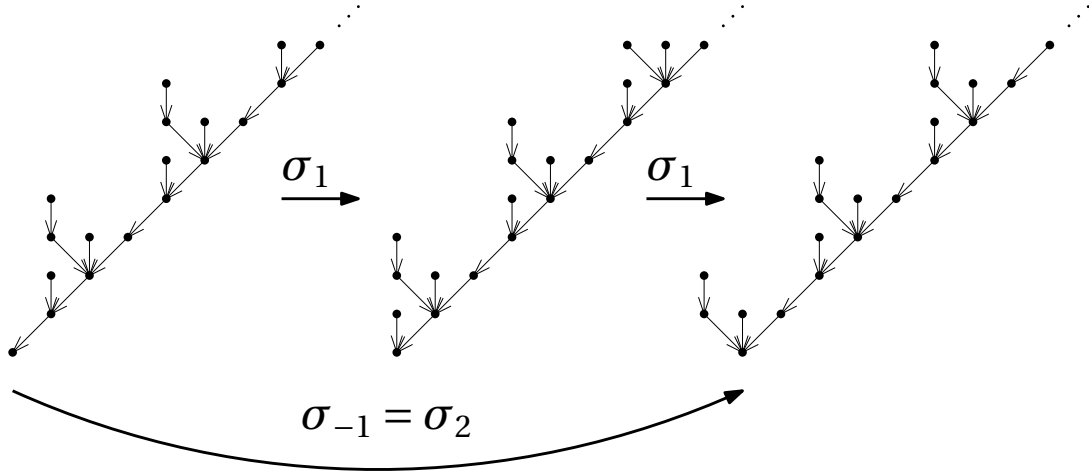


FIGURE 2.20. – Exemple de décalage. Les trois arbres représentent l'ensemble des décalages de l'arbre de gauche puisqu'il a une période minimale de 3.

$\mathbf{X} = \mathcal{C}_n(\mathcal{U}(X))$, avec n un entier naturel suffisamment grand. Notre but est donc de démontrer qu'une telle propriété est vérifiée. Et pour cela, nous commençons par prouver que le quotient de deux unrolls est, s'il existe, un unroll.

Proposition 2.19. *Soient A, B deux FDDS. S'il existe une forêt $\mathbf{X}$ telle que $\mathcal{U}(A)\mathbf{X} = \mathcal{U}(B)$ alors il existe X un FDDS tel que $\mathbf{X} = \mathcal{U}(X)$.*

Pour cela, nous introduisons deux nouvelles notations. Étant donné $\mathbf{t}$ un arbre d'unroll, nous notons par $\sigma_i(\mathbf{t})$ son *décalage* obtenu par le retrait des nœuds de la branche infinie de $\mathbf{t}$ (et les arbres finis qui leur sont enracinés) dont la profondeur est strictement inférieure à i avec $i \geq 0$ un entier naturel. Par commodité, nous définissons que $\sigma_{-i}(\mathbf{t})$ est égal à $\sigma_{kp-i}(\mathbf{t})$ avec i un entier naturel, p la plus petite période de $\mathbf{t}$ et k le plus petit entier naturel tel que $kp - i \geq 0$. Un exemple de décalages positifs et négatifs est présenté dans la Figure 2.20. Du point de vue des mots périodiques, cela correspond simplement à retirer les i lettres les plus à gauche du mot. Notons que $\sigma_i(\mathbf{t}_1\mathbf{t}_2) = \sigma_i(\mathbf{t}_1)\sigma_i(\mathbf{t}_2)$ pour tous arbres d'unroll $\mathbf{t}_1, \mathbf{t}_2$ et tout entier naturel i .

Nous définissons les *décalages* de $\mathbf{t}$, notés $\Sigma(\mathbf{t})$, comme l'ensemble des arbres d'unroll générés par un décalage de $\mathbf{t}$, plus formellement $\Sigma(\mathbf{t}) = \{\sigma_i(\mathbf{t}) \mid i \in \mathbb{N}\}$. Du point de vue des mots, $\Sigma(\mathbf{t})$ est l'ensemble des mots différents que nous pouvons produire en retirant un préfixe fini aux mots associés à $\mathbf{t}$. Lorsque le contexte est clair, nous utiliserons également $\Sigma(\mathbf{t})$ pour décrire la *forêt* composée des arbres de $\Sigma(\mathbf{t})$. Remarquons que la cardinalité de cet ensemble est aussi la période minimale du mot $\mathbf{t}$. Ainsi, $\Sigma(\mathbf{t})$ est aussi l'unroll du plus petit FDDS D , en termes de taille, qui admet $\mathbf{t}$ comme arbre d'unroll. En effet, si un FDDS D' admet $\mathbf{t}$ comme arbre d'unroll, alors $\Sigma(\mathbf{t}) = \mathcal{U}(D)$ est un sous-multi-ensemble de $\mathcal{U}(D')$. Donc la taille de D est inférieure ou égale à celle de D' . Nous pouvons ainsi reconstruire $\mathcal{U}(D)$ à partir de seulement un de ces arbres d'unroll. L'idée derrière notre démonstration de la Proposition 2.19 est de montrer que $\mathbf{X}$ est une somme de $\Sigma(\mathbf{t})$ pour certains arbres d'unroll $\mathbf{t}$. Nous le ferons via une

2. Résolutions d'équations polynomiales sur les unrolls – 2.2. Complexité de la division d'unroll

réurrence. Mais avant cela, nous allons prouver que chaque arbre de $\mathbf{X}$ est un arbre d'unroll. Cette propriété est une conséquence du lemme suivant.

Lemme 2.21. *Soient A, B deux FDDS et $\mathbf{a}$ un arbre d'unroll de $\mathcal{U}(A)$. S'il existe un arbre $\mathbf{x}$ tel que la forêt $\Sigma(\mathbf{a})\mathbf{x}$ est un sous-multi-ensemble de $\mathcal{U}(B)$ alors $\mathbf{x}$ est un arbre d'unroll et la forêt $\mathbf{a}'\Sigma(\mathbf{x})$ est un sous-multi-ensemble de $\mathcal{U}(B)$ pour tout $\mathbf{a}'$ de $\Sigma(\mathbf{a})$.*

Démonstration. Supposons qu'il existe $\mathbf{x}$ tel que $\Sigma(\mathbf{a})\mathbf{x}$ est un sous-multi-ensemble de $\mathcal{U}(B)$. Ceci induit que $\mathbf{x}$ contient exactement une branche infinie. Soit $\mathbf{b} = \mathbf{a}\mathbf{x}$ et soient a et b des périodes de respectivement $\mathbf{a}$ et $\mathbf{b}$. Comme le décalage d'un produit et le produit des décalage, nous obtenons que $\sigma_{\text{lcm}(a,b)}(\mathbf{a})\sigma_{\text{lcm}(a,b)}(\mathbf{x}) = \sigma_{\text{lcm}(a,b)}(\mathbf{b})$. Or, puisque $\text{lcm}(a, b)$ est une période de $\mathbf{a}$ et $\mathbf{b}$, il suit que $\sigma_{\text{lcm}(a,b)}(\mathbf{a}) = \mathbf{a}$ et $\sigma_{\text{lcm}(a,b)}(\mathbf{b}) = \mathbf{b}$. Ainsi, par le Corollaire 1.58, nous obtenons $\sigma_{\text{lcm}(a,b)}(\mathbf{x}) = \mathbf{x}$. Nous en concluons que $\mathbf{x}$ est un arbre d'unroll de période $\text{lcm}(a, b)$.

Soit $\mathbf{a}'$ un arbre de $\Sigma(\mathbf{a})$. Par définition, $\Sigma(\mathbf{a}) = \Sigma(\mathbf{a}')$. Montrons maintenant que $\mathbf{a}'\Sigma(\mathbf{x})$ est un sous-multi-ensemble de $\mathcal{U}(B)$. Pour ce faire, commençons par prouver que $\mathcal{U}(B)$ contient au moins une copie de chaque arbre de $\mathbf{a}'\Sigma(\mathbf{x})$. Soient i, j deux entiers. Comme $\Sigma(\mathbf{a}')\mathbf{x}$ est un sous-multi-ensemble de $\mathcal{U}(B)$, nous déduisons que $\sigma_i(\mathbf{a}')\mathbf{x}$ est un arbre de $\mathcal{U}(B)$. De là par définition des unrolls, $\sigma_{i+j}(\mathbf{a}')\sigma_j(\mathbf{x})$ est également un élément de $\mathcal{U}(B)$. Et c'est donc en particulier aussi le cas de $\mathbf{a}'\sigma_j(\mathbf{x})$. Car si $i = -j$ alors $\sigma_{j+i}(\mathbf{a}') = \mathbf{a}'$.

Maintenant, pour conclure la preuve du lemme, il reste donc à montrer que $\mathbf{a}'\Sigma(\mathbf{x})$ ne contient pas deux copies du même arbre. Or ceci est une conséquence directe du Corollaire 1.58. En effet, s'il existait deux arbres $\mathbf{t}_1, \mathbf{t}_2$ dans $\Sigma(\mathbf{x})$ tels que $\mathbf{t}_1\mathbf{a} = \mathbf{t}_2\mathbf{a}$, alors par ce dernier $\mathbf{t}_1 = \mathbf{t}_2$. $\square$

Comme dit précédemment, nous allons montrer la Proposition 2.19 par récurrence. L'idée de cette dernière sera d'identifier $\mathbf{x}$ le plus petit arbre de $\mathbf{X}$, de montrer que $\Sigma(\mathbf{x})$ est un sous-multi-ensemble de $\mathbf{X}$ et donc que nous pouvons retirer $\Sigma(\mathbf{x})$ de $\mathbf{X}$ et réappliquer le même raisonnement. Nous extrayons la partie de la démonstration qui vise à prouver que $\Sigma(\mathbf{x})$ est un sous-multi-ensemble de $\mathbf{X}$ dans le lemme suivant.

Lemme 2.22. *Soient A, B deux FDDS. S'il existe une forêt $\mathbf{X}$ telle que $\mathcal{U}(A)\mathbf{X} = \mathcal{U}(B)$ alors $\min(\mathbf{X})$ est un arbre d'unroll et $\Sigma(\min(\mathbf{X}))$ est un sous-multi-ensemble de $\mathbf{X}$.*

Démonstration. Nous supposons qu'il existe $\mathbf{X}$ telle que $\mathcal{U}(A)\mathbf{X} = \mathcal{U}(B)$. Par souci de lisibilité, nous posons $\mathbf{x} = \min(\mathbf{X})$, $\mathbf{a} = \min(\mathcal{U}(A))$ et $\mathbf{b} = \min(\mathcal{U}(B))$. Par conséquent, la minimalité de $\mathbf{b}$ implique $\mathbf{a}\mathbf{x} = \mathbf{b}$. Et par définition des unrolls, $\sigma_i(\mathbf{b}) = \sigma_i(\mathbf{a})\sigma_i(\mathbf{x})$ est un arbre de $\mathcal{U}(B)$ pour tout i . Afin de montrer que $\Sigma(\mathbf{x})$ est un sous-multi-ensemble de $\mathbf{X}$, nous allons prouver que la seule manière de générer $\Sigma(\mathbf{b})$ à partir de $\mathcal{U}(A)$ et $\mathbf{X}$ est de multiplier $\sigma_i(\mathbf{a})$ et $\sigma_i(\mathbf{x})$ pour tout i . Ce qui implique l'assertion. Plus formellement, nous démontrerons que s'il existe deux arbres $\mathbf{a}', \mathbf{x}'$ de respectivement $\mathcal{U}(A)$ et $\mathbf{X}$ tels que $\mathbf{a}'\mathbf{x}' = \sigma_i(\mathbf{a})\sigma_i(\mathbf{x})$ pour un entier naturel i , alors $\mathbf{a}'$ est un arbre de $\Sigma(\mathbf{a})$ et $\mathbf{x}'$ est un arbre de $\Sigma(\mathbf{x})$.

Pour ce faire, il nous faut d'abord montrer que $\mathbf{x}$ est bien un arbre d'unroll. Or, comme $\mathcal{U}(A)\mathbf{x}$ est un sous-multi-ensemble de $\mathcal{U}(B)$, nous déduisons du Lemme 2.21,

que $\mathbf{x}$ est un arbre d'unroll et que $\mathbf{a}_j \Sigma(\mathbf{x})$ est un sous-multi-ensemble de $\mathcal{U}(B)$, et ce pour tout arbre $\mathbf{a}_j$ de $\mathcal{U}(A)$.

Soit i un entier naturel. Considérons maintenant $\mathbf{a}', \mathbf{x}'$ deux arbres de respectivement $\mathcal{U}(A)$ et $\mathbf{X}$ tels que $\mathbf{a}'\mathbf{x}' = \sigma_i(\mathbf{a})\sigma_i(\mathbf{x})$. De là pour les mêmes raisons que précédemment, il s'ensuit que $\mathbf{x}'$ est un arbre d'unroll et que $\mathbf{a}\Sigma(\mathbf{x}')$ est un sous-multi-ensemble de $\mathcal{U}(B)$. Ainsi, nous déduisons deux propriétés. Premièrement, $\sigma_{-i}(\mathbf{a}')\sigma_{-i}(\mathbf{x}') = \mathbf{a}\mathbf{x}$. Deuxièmement, $\mathbf{a}\sigma_{-i}(\mathbf{x}')$ est un arbre de $\mathcal{U}(B)$. Maintenant, la minimalité de $\mathbf{a}$ implique que $\sigma_{-i}(\mathbf{a}') \geq \mathbf{a}$. Et puisque l'ordre est compatible avec le produit, de la première propriété il résulte que $\sigma_{-i}(\mathbf{x}') \leq \mathbf{x}$; entraînant que $\mathbf{a}\sigma_{-i}(\mathbf{x}') \leq \mathbf{a}\mathbf{x}$. Mais, la seconde propriété implique $\mathbf{a}\sigma_{-i}(\mathbf{x}') \geq \mathbf{a}\mathbf{x}$. Nous concluons donc que $\mathbf{a}\sigma_{-i}(\mathbf{x}') = \mathbf{a}\mathbf{x}$ et $\sigma_{-i}(\mathbf{x}') = \mathbf{x}$ par le Corollaire 1.58. Ceci signifie que $\mathbf{x}'$ est un arbre de $\Sigma(\mathbf{x})$. $\square$

Nous pouvons maintenant réaliser la récurrence en elle-même et conclure la preuve.

Preuve de la Proposition 2.19. Nous supposons qu'il existe $\mathbf{X}$ telle que $\mathcal{U}(A)\mathbf{X} = \mathcal{U}(B)$. Soient $\mathbf{a}, \mathbf{x}, \mathbf{b}$ les arbres minimaux de $\mathcal{U}(A), \mathbf{X}$ et $\mathcal{U}(B)$ respectivement. Ainsi, par le Lemme 2.22, il résulte que $\Sigma(\mathbf{x})$ est un sous-multi-ensemble de $\mathbf{X}$. Donc, $\mathcal{U}(A)\Sigma(\mathbf{x})$ est un sous-multi-ensemble de $\mathcal{U}(B)$. De là, comme nous pouvons associer à $\Sigma(\mathbf{x})$ un FDDS D tel que $\mathcal{U}(D) = \Sigma(\mathbf{x})$, nous déduisons que $\mathcal{U}(A)\Sigma(\mathbf{x})$ est l'unroll de AD . Et donc il existe B' un FDDS dont l'unroll est $\mathcal{U}(B) - \mathcal{U}(A)\Sigma(\mathbf{x})$ (cette opération est définie à la fin dans le dernier paragraphe de la Section 1.2). De plus, $\mathcal{U}(A)(\mathbf{X} - \Sigma(\mathbf{x})) = \mathcal{U}(B')$. Nous pouvons donc appliquer le même raisonnement. De proche en proche, nous concluons que $\mathbf{X}$ est une somme d'unrolls et donc aussi un unroll. $\square$

À ce point, pour obtenir un algorithme permettant de décider s'il existe une solution à $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$, il nous faut encore montrer que l'existence d'une solution à l'équation $\mathcal{C}_n(\mathcal{U}(A))\mathbf{X} = \mathcal{C}_n(\mathcal{U}(B))$, avec un n suffisamment grand, entraîne celle d'une solution sur les unrolls. Pour ce faire, nous expliquerons comment nous pouvons reconstruire un arbre d'unroll à partir de chaque arbre de la solution de l'équation sur les troncatures. Cela sera le propos du Théorème 2.23. Notons bien que la profondeur où nous tronquons est différente entre ce théorème et la Proposition 2.1. Ceci vient du fait que pour notre preuve nous aurons besoin que la troncature de chaque paire d'arbres d'unroll $\mathbf{a}, \mathbf{b}$, de $\mathcal{U}(A)$ et $\mathcal{U}(B)$ respectivement, contient au moins une profondeur $i > 0$ telle que $\sigma_i(\mathbf{a}) = \mathbf{a}$ et $\sigma_i(\mathbf{b}) = \mathbf{b}$. Pour garantir le respect de cette condition, nous élevons α au carré pour dépasser le plus petit multiple commun de deux périodes, elles-mêmes plus petites que α .

Théorème 2.23. *Soient A, B deux FDDS et $n \geq 2\alpha^2 + d(B)$ avec α le nombre d'arbres d'unroll dans $\mathcal{U}(B)$. Il existe un FDDS X tel que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ si et seulement s'il existe une forêt finie $\mathbf{X}$ telle que $\mathcal{C}_n(\mathcal{U}(A))\mathbf{X} = \mathcal{C}_n(\mathcal{U}(B))$. De plus, le cas échéant, $\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathbf{X})$.*

Démonstration. Un sens est donné par la Proposition 2.1. Pour la réciproque, nous supposons qu'il existe une forêt $\mathbf{X}$ telle que $\mathcal{C}_n(\mathcal{U}(A))\mathbf{X} = \mathcal{C}_n(\mathcal{U}(B))$. Comme tous les arbres de $\mathcal{C}_n(\mathcal{U}(A))$ et de $\mathcal{C}_n(\mathcal{U}(B))$ ont profondeur n , de la définition du produit d'arbre, nous pouvons supposer sans perte de généralité que $\mathbf{X}$ est une forêt dont tous

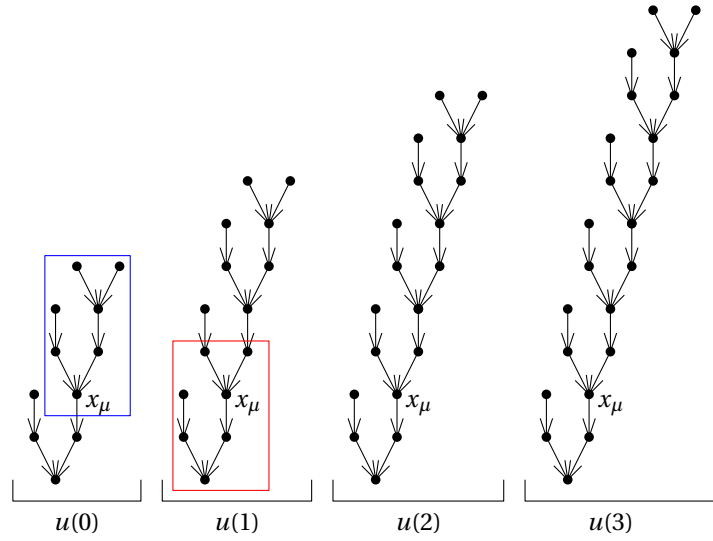


FIGURE 2.24. – Exemple de la suite u définie dans la preuve du Théorème 2.23 avec $\mu = 2$. L'argument final de la démonstration de l'Affirmation 2.25 est que l'arbre encadré en bleu est égal à celui encadré en rouge.

les arbres ont profondeur n . De plus, puisque n est suffisamment grand, le Lemme 2.4 implique que pour tous arbres $\mathbf{a}_1, \mathbf{a}_2$ de $\mathcal{U}(A)$ si $\mathcal{C}_n(\mathbf{a}_1) = \mathcal{C}_n(\mathbf{a}_2)$ alors $\mathbf{a}_1 = \mathbf{a}_2$. C'est aussi le cas entre $\mathcal{U}(B)$ et $\mathcal{C}_n(\mathcal{U}(B))$.

Soit $\mathbf{x}$ un arbre de $\mathbf{X}$. Nous allons expliquer comment nous pouvons construire un arbre d'unroll $\mathbf{y}$ tel que $\mathcal{C}_n(\mathbf{y}) = \mathbf{x}$. Pour cela, la première chose qu'il faut observer est que $\alpha^2 \geq \text{lcm}(|\Sigma(\mathbf{a}_i)|, |\Sigma(\mathbf{b}_j)|)$ pour tous arbres $\mathbf{a}_i, \mathbf{b}_j$ de respectivement $\mathcal{U}(A)$ et $\mathcal{U}(B)$. En effet, par définition des unrolls, nous avons que $|\Sigma(\mathbf{a}_i)|$ et $|\Sigma(\mathbf{b}_j)|$ sont respectivement inférieurs ou égaux aux nombres d'arbres dans $\mathcal{U}(A)$ et $\mathcal{U}(B)$. L'assertion suit du fait que le nombre d'arbres dans $\mathcal{U}(A)$ est au plus α .

Soient $\mathbf{a}$ et $\mathbf{b}$ deux arbres de respectivement $\mathcal{U}(A)$ et $\mathcal{U}(B)$ tels que $\mathcal{C}_n(\mathbf{a})\mathbf{x} = \mathcal{C}_n(\mathbf{b})$. Notons par μ le plus petit multiple commun de leur plus petite période respective. Par ce qui précède, $\mu \leq \alpha^2$. Donc par le Lemme 2.3, tous les arbres de profondeur maximale de $\mathcal{C}_n(\mathbf{a})$ et $\mathcal{C}_n(\mathbf{b})$ sont enracinés dans un unique sommet de profondeur μ , notés a_μ et b_μ respectivement. Donc, de la définition du produit d'arbres, c'est aussi le cas pour $\mathbf{x}$ et nous notons par x_μ ce sommet. Soit u la suite définie par $u(0) = \mathbf{x}$ et $u(i+1)$ est l'arbre $\mathbf{x}$ où nous remplaçons l'arbre enraciné dans x_μ par $u(i)$. La Figure 2.24 donne un exemple de cette suite.

Affirmation 2.25. Soit $i \geq 0$ un entier. Alors, $\mathcal{C}_{n+i\cdot\mu}(\mathbf{a})u(i) = \mathcal{C}_{n+i\cdot\mu}(\mathbf{b})$.

Preuve. Nous prouvons cette propriété par récurrence sur la profondeur. Plus précisément, si nous considérons $a_{j\cdot\mu}, x_{j\cdot\mu}$ et $b_{j\cdot\mu}$ les copies de respectivement a_μ, x_μ et b_μ dans $\mathcal{C}_{n+i\cdot\mu}(\mathbf{a}), u(i)$ et $\mathbf{b}$ de profondeur $j \cdot \mu$ alors nous prouvons que les arbres enracinés dans $a_{j\cdot\mu}, x_{j\cdot\mu}$ et $b_{j\cdot\mu}$ sont égaux pour tout $0 \leq j \leq i$. Prenons $i \cdot \mu$ comme cas de base. Par définition, nous avons que les arbres enracinés dans $a_{i\cdot\mu}, x_{i\cdot\mu}$ et $b_{i\cdot\mu}$ sont respectivement $\mathcal{C}_n(\mathbf{a}), \mathbf{x}$ et $\mathcal{C}_n(\mathbf{b})$. Donc la propriété est vérifiée par hypothèse.

Soit j un entier entre 1 et i . Supposons que la propriété est vraie pour j . Montrons alors qu'elle est vraie pour $j - 1$. Soient $\mathbf{a}_j, \mathbf{x}_j$ et $\mathbf{b}_j$ les arbres enracinés dans respectivement $a_{j \cdot \mu}, x_{j \cdot \mu}$ et $b_{j \cdot \mu}$. De même, nous posons $\mathbf{a}_{j-1}, \mathbf{x}_{j-1}$ et $\mathbf{b}_{j-1}$ les arbres enracinés dans respectivement $a_{(j-1) \cdot \mu}, x_{(j-1) \cdot \mu}$ et $b_{(j-1) \cdot \mu}$. Ainsi, nous devons prouver que $\mathbf{a}_{j-1} \mathbf{x}_{j-1} = \mathbf{b}_{j-1}$. Remarquons que puisque n est suffisamment grand, l'ensemble des sommets de $\mathbf{a}_{j-1}, \mathbf{x}_{j-1}$ et $\mathbf{b}_{j-1}$ ayant une profondeur strictement supérieure à n sont des sommets de $\mathbf{a}_j, \mathbf{x}_j$ et $\mathbf{b}_j$ respectivement. Donc, comme par hypothèse de récurrence nous avons $\mathbf{a}_j \mathbf{x}_j = \mathbf{b}_j$ pour montrer que $\mathbf{a}_{j-1} \mathbf{x}_{j-1} = \mathbf{b}_{j-1}$, il suffit de démontrer que $\mathcal{C}_n(\mathbf{a}_{j-1}) \mathcal{C}_n(\mathbf{x}_{j-1}) = \mathcal{C}_n(\mathbf{b}_{j-1})$. Ce qui revient à démontrer que $\mathcal{C}_n(\mathbf{x}_{j-1}) = \mathbf{x}$ puisque $\mathbf{a}$ et $\mathbf{b}$ sont des arbres périodiques. En outre comme par construction, $\mathbf{x}_{j-1}$ correspond à l'arbre $\mathbf{x}$ où nous remplaçons l'arbre enraciné dans $x_{(j-1) \cdot \mu}$ par $\mathbf{x}_j$, nous n'avons plus qu'à prouver que l'arbre $\mathbf{x}_1$ enraciné dans x_μ dans $\mathbf{x}$ est égal à $\mathcal{C}_{n-\mu}(\mathbf{x}_j)$. Ce qui par construction revient à démontrer que $\mathbf{x}_1 = \mathcal{C}_{n-\mu}(\mathbf{x})$ (dans la Figure 2.24 cela revient à prouver que l'arbre encadré de rouge est égal à celui encadré de bleu).

Soient $\mathbf{a}_1$ et $\mathbf{b}_1$ les arbres enracinés dans respectivement a_μ dans $\mathcal{C}_n(\mathbf{a})$ et b_μ dans $\mathcal{C}_n(\mathbf{b})$. Comme expliqué précédemment, nous avons que $\mathbf{a}_1 \mathbf{x}_1 = \mathbf{b}_1$. Puisque $\mathbf{a}$ et $\mathbf{b}$ sont périodiques, nous avons que $\mathbf{a}_1 = \mathcal{C}_{n-\mu}(\mathbf{a})$ et $\mathbf{b}_1 = \mathcal{C}_{n-\mu}(\mathbf{b})$. De là, étant donné que par hypothèse $\mathcal{C}_n(\mathbf{a}) \mathbf{x} = \mathcal{C}_n(\mathbf{b})$, il suit que $\mathcal{C}_{n-\mu}(\mathbf{a}) \mathcal{C}_{n-\mu}(\mathbf{x}) = \mathcal{C}_{n-\mu}(\mathbf{b})$ impliquant que $\mathbf{a}_1 \mathcal{C}_{n-\mu}(\mathbf{x}) = \mathbf{b}_1$. Nous déduisons alors que $\mathbf{a}_1 \mathcal{C}_{n-\mu}(\mathbf{x}) = \mathbf{a}_1 \mathbf{x}_1$. Ainsi, comme $\mathbf{a}_1, \mathcal{C}_{n-\mu}(\mathbf{x})$ et $\mathbf{x}_1$ ont la même profondeur, le Lemme 1.57 induit que $\mathbf{x}_1 = \mathcal{C}_{n-\mu}(\mathbf{x})$ et l'affirmation suit. $\diamond$

Ainsi, nous sommes en mesure de construire un arbre d'unroll $\mathbf{y}$ de période μ tel que $\mathcal{C}_r(\mathbf{y}) = \mathcal{C}_r(u(i))$ et ce pour tout entier $i \geq 0$ et tout entier r entre 0 et $n + i \cdot \mu$. Intuitivement, $\mathbf{y}$ est la limite de u quand i tend vers l'infini.

Pour conclure que cette méthode permet bien de construire une forêt d'arbres d'unroll $\mathbf{X}$, il nous faut encore prouver que nous ne pouvons pas construire deux arbres différents à partir de $\mathbf{y}$. Pour cela, considérons $\mathbf{a}', \mathbf{b}'$ deux arbres de $\mathcal{U}(A)$ et $\mathcal{U}(B)$ respectivement tels que $\mathcal{C}_n(\mathbf{a}') \mathbf{y} = \mathcal{C}_n(\mathbf{b}')$. Nous notons $\mathbf{x}'$ l'arbre construit par la méthode précédente en utilisant $\mathcal{C}_n(\mathbf{a}')$ et $\mathcal{C}_n(\mathbf{b}')$. Donc $\mathbf{x}'$ est de période μ . Ainsi comme $\mathcal{C}_n(\mathbf{x}) = \mathcal{C}_n(\mathbf{x}')$ et que n est suffisamment grand, le Lemme 2.4 induit que $\mathbf{x} = \mathbf{x}'$.

Soit $\mathbf{Y}$ la forêt d'arbres d'unroll reconstruite à partir de $\mathbf{X}$ comme expliqué précédemment. Nous avons donc que $\mathcal{U}(A) \mathbf{Y} = \mathcal{U}(B)$. Ainsi de la Proposition 2.19, nous concluons qu'il existe un FDDS X tel que $\mathcal{U}(X) = \mathbf{Y}$. Et le théorème suit. $\square$

Ce théorème montre que nous pouvons décider de l'existence d'une solution à l'équation $\mathcal{U}(A) \mathcal{U}(X) = \mathcal{U}(B)$ avec A et B des FDDS en temps polynomial sur m la taille de B . En effet, la profondeur de troncature de l'énoncé est bien polynomiale sur m , impliquant que la troncature de $\mathcal{U}(A)$ et $\mathcal{U}(B)$ est aussi de taille polynomiale sur m . Maintenant, il ne reste qu'à appliquer l'algorithme de division des forêts finies, qui est polynomial sur m , pour conclure. Cependant, la preuve décrit une méthode constructive permettant de trouver l'unroll. Nous la traduisons dans l'Algorithme 2.26. Il prend en entrée deux FDDS A et B et renvoie un FDDS X tel que $\mathcal{U}(A) \mathcal{U}(X) = \mathcal{U}(B)$, s'il en existe un.

Algorithme 2.26 UnrollDivision(A, B)

```

1: Si  $|A| > |B|$  alors
2:   renvoyer  $\perp$ ,
3:  $X \leftarrow \emptyset$ ,
4: soit  $\alpha$  le nombre d'arbres de  $\mathcal{U}(B)$  et soit  $n \leftarrow 2\alpha^2 + d(B)$ ,
5:  $\mathbf{A} \leftarrow \mathcal{C}_n(\mathcal{U}(A))$ ,  $\mathbf{B} \leftarrow \mathcal{C}_n(\mathcal{U}(B))$  où nous avons associé à chaque arbre de  $\mathbf{A}$  et de  $\mathbf{B}$ 
   leur longueur de cycle d'origine,
6: Tant que  $\mathbf{B}$  n'est pas vide faire
7:    $\mathbf{a} \leftarrow \min(\mathbf{A})$ ,  $\mathbf{b} \leftarrow \min(\mathbf{B})$ ,
8:    $\mathbf{x} \leftarrow \text{Division}(\mathbf{a}, \mathbf{b})$  (Algorithme 1.76),
9:   construire  $w_{\mathbf{x}}$  un motif périodique de  $\mathbf{x}$  de longueur  $\text{lcm}(a, b)$  avec  $a$  et  $b$  les
   longueurs associées à respectivement  $\mathbf{a}$  et  $\mathbf{b}$ ,
10:  trouver  $p$  la plus petite période du mot  $w_{\mathbf{x}}$ ,
11:  construire le FDDS connexe  $D$  issu de l'enroulement de l'arbre  $\mathbf{x}$  à période  $p$ 
   en s'assurant que la taille de  $X$  soit inférieure à celle de  $B$ ,
12:  Si  $\mathbf{F}$  n'est pas un sous-multi-ensemble de  $\mathbf{B}$  avec  $\mathbf{F} \leftarrow \mathbf{A}\mathcal{C}_n(\mathcal{U}(D))$  alors
13:    renvoyer  $\perp$ ,
14:     $X \leftarrow X + D$ ,
15:     $\mathbf{B} \leftarrow \mathbf{B} - \mathbf{F}$ ,
16:  $\mathbf{X} \leftarrow \mathcal{C}_n(\mathcal{U}(X))$ ,
17: Si  $\mathbf{A}\mathbf{X} \neq \mathbf{B}$  alors
18:   renvoyer  $\perp$ ,
19: renvoyer  $X$ .
```

Avant la preuve de correction de cet algorithme ainsi que de l'étude de sa complexité, nous introduisons le lemme technique suivant qui nous permet de borner la plus petite période des mots $w_{\mathbf{x}}$ de l'algorithme par b et non par $\text{lcm}(a, b)$ où les a et b sont ceux de l'instruction 9 de l'Algorithme 2.26.

Lemme 2.27. Soient A et X deux FDDS. Soient $\mathbf{a}$ et $\mathbf{x}$ des arbres d'unroll de respectivement $\mathcal{U}(A)$ et $\mathcal{U}(X)$ tels que $\mathbf{a} = \min(\Sigma(\mathbf{a}))$ et $\mathbf{x} = \min(\Sigma(\mathbf{x}))$ et soient a et x leur période minimale respective. Alors l'arbre $\mathbf{ax}$ a comme période minimale $\text{lcm}(a, x)$.

Démonstration. Soit l'arbre $\mathbf{b} = \mathbf{ax}$ et b sa période minimale. Il suit que $\sigma_b(\mathbf{a})\sigma_b(\mathbf{x}) = \mathbf{b}$. De là comme l'ordre est compatible avec le produit et puisque $\mathbf{a}$ est minimal, nous déduisons que $\sigma_b(\mathbf{a}) \geq \mathbf{a}$. Ainsi, deux cas sont possibles.

1. Soit $\sigma_b(\mathbf{a}) = \mathbf{a}$. Par le Corollaire 1.58, il en résulte que $\sigma_b(\mathbf{x}) = \mathbf{x}$. Ce qui signifie que $\mathbf{a}$ et $\mathbf{x}$ sont de période b .
2. Soit $\sigma_b(\mathbf{a}) > \mathbf{a}$. Donc, comme l'ordre est compatible avec le produit, nous avons que $\sigma_b(\mathbf{x}) < \mathbf{x}$. Mais ceci est en contradiction avec la minimalité de $\mathbf{x}$.

Maintenant comme $\text{lcm}(a, x)$ est une période de $\mathbf{b}$, il suit que $b \leq \text{lcm}(a, x)$. De plus, comme b est une période de $\mathbf{a}$ et $\mathbf{x}$, nous déduisons que b est un multiple de a et x . Ceci induit que $b \geq \text{lcm}(a, x)$ et le lemme suit. $\square$

Corollaire 2.28. Soient $\mathbf{a}$ et $\mathbf{b}$ deux arbres d'unroll tels que $\mathbf{a} = \min(\Sigma(\mathbf{a}))$ et $\mathbf{b} = \min(\Sigma(\mathbf{b}))$. Si $\mathbf{a}$ divise $\mathbf{b}$ alors leur quotient $\mathbf{x}$ est de période p et ce pour tout entier p qui est une période de $\mathbf{b}$.

Théorème 2.29. Soient A et B deux FDDS. La division de $\mathcal{U}(B)$ par $\mathcal{U}(A)$ (Algorithme 2.26) est calculable en temps $\mathcal{O}(m^{10})$ avec m la taille de B .

Démonstration. Pour la preuve de correction, nous constatons tout d'abord que si l'algorithme renvoie X un résultat alors $\mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(B))$. De là, par le Théorème 2.23, nous avons bien que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$.

Maintenant, supposons qu'il existe X tel que $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$. Nous allons montrer que l'algorithme trouve un FDDS Y tel que $\mathcal{U}(X) = \mathcal{U}(Y)$. Constatons d'abord que puisque l'ordre est compatible avec le produit, $\min(\mathcal{U}(A))\min(\mathcal{U}(X)) = \min(\mathcal{U}(B))$. Et de plus, comme n est suffisamment grand, du Lemme 2.4, nous déduisons que $\min(\mathcal{C}_n(\mathcal{U}(A)))\min(\mathcal{C}_n(\mathcal{U}(X))) = \min(\mathcal{C}_n(\mathcal{U}(B)))$. Donc comme l'algorithme de division est correct, nous parvenons à trouver $\mathbf{x} = \min(\mathcal{C}_n(\mathcal{U}(X)))$ (instruction 8). Par le Théorème 2.23, nous avons de plus que $\min(\mathcal{U}(X))$ est de période $\text{lcm}(a, b)$ avec a et b deux périodes de $\mathbf{a}$ et $\mathbf{b}$ (instruction 7) respectivement. Donc $w_{\mathbf{x}}$ est un motif périodique de $\min(\mathcal{U}(X))$. Et puisque $\Sigma(\min(\mathcal{U}(X)))$ est un sous-multi-ensemble de $\mathcal{U}(X)$, nous concluons que $\mathcal{U}(D)$ (instruction 11) est un sous-multi-ensemble de $\mathcal{U}(X)$. En outre, $\mathcal{U}(A)\mathcal{U}(D)$ est aussi un sous-multi-ensemble de $\mathcal{U}(B)$ et donc il existe bien un FDDS B' tel que $\mathcal{U}(B') = \mathcal{U}(B) - \mathcal{U}(A)\mathcal{U}(D)$. Par récurrence, nous parvenons à construire un FDDS Y tel que $\mathcal{U}(X) = \mathcal{U}(Y)$.

Passons maintenant à l'étude de complexité. Pour vérifier que la taille de A est au plus m , il ne faut qu'un temps en $\mathcal{O}(m)$. Comme n est en $\mathcal{O}(m^2)$, nous déduisons que chaque arbre de $\mathcal{C}_n(\mathcal{U}(A))$ et $\mathcal{C}_n(\mathcal{U}(B))$ a une taille en $\mathcal{O}(m^3)$ et donc que ses forêts ont une taille en $\mathcal{O}(m^4)$ par un raisonnement similaire à la Remarque 2.8. De là, le calcul de $\mathcal{C}_n(\mathcal{U}(A))$ et $\mathcal{C}_n(\mathcal{U}(B))$ est en $\mathcal{O}(m^4)$.

Pour trouver $\mathbf{a}$ et $\mathbf{b}$ (instruction 7), nous trions $\mathcal{C}_n(\mathcal{U}(A))$ et $\mathcal{C}_n(\mathcal{U}(B))$. Pour cela, nous pouvons construire $\mathcal{E}(\mathcal{C}_n(\mathcal{U}(A)))$ et $\mathcal{E}(\mathcal{C}_n(\mathcal{U}(B)))$ prenant un temps linéaire en m . Nous obtenons ainsi deux arbres dont la taille est en $\mathcal{O}(m^4)$. De là, nous calculons le code de ces arbres, requérant un temps en $\mathcal{O}(m^8)$ (Lemme 1.64). En effet, lors du calcul du code nous trions par ordre croissant les arbres de $\mathcal{D}(\mathcal{E}(\mathcal{C}_n(\mathcal{U}(A)))) = \mathcal{C}_n(\mathcal{U}(A))$ et $\mathcal{D}(\mathcal{E}(\mathcal{C}_n(\mathcal{U}(B)))) = \mathcal{C}_n(\mathcal{U}(B))$. Il ne reste plus qu'à calculer $\mathcal{D}(\mathcal{E}(\mathcal{C}_n(\mathcal{U}(A))))$ et $\mathcal{D}(\mathcal{E}(\mathcal{C}_n(\mathcal{U}(B))))$ prenant un temps linéaire. Après quoi, nous trouvons $\mathbf{a}$ et $\mathbf{b}$ avec un temps en $\mathcal{O}(1)$.

Comme $\mathbf{a}$ et $\mathbf{b}$ ont une taille en $\mathcal{O}(m^3)$, nous concluons que nous pouvons calculer leur quotient $\mathbf{x}$ (instruction 9) en $\mathcal{O}(m^9)$. De surcroît, la taille de $\mathbf{x}$ est en $\mathcal{O}(m^3)$. Construire $w_{\mathbf{x}}$ peut être en $\mathcal{O}(m^3)$ via un parcours.

Le Lemme 2.27 induit que la valeur du p de l'instruction 10 est comprise entre 1 et m . Donc pour le trouver nous cherchons le plus petit i entre 1 et m tel que $\sigma_i(w_{\mathbf{x}}) = w_{\mathbf{x}}$. Dans ce contexte, $\sigma_i(w_{\mathbf{x}})$ correspond au mot construit à partir de $w_{\mathbf{x}}$ en retirant le préfixe de longueur i et en l'ajoutant comme suffixe. Remarquons que le calcul des différents décalages peut se faire en temps linéaire sur le nombre de lettres dans $w_{\mathbf{x}}$, c'est-à-dire en $\mathcal{O}(m^2)$. La vérification que les deux mots $\sigma_i(w_{\mathbf{x}})$ et $w_{\mathbf{x}}$ sont égaux est

quant à elle en $\mathcal{O}(m^3)$. Ceci est dû à [28] car nous voyons les mots $\sigma_i(w_x)$ comme des arbres qui ont une taille en $\mathcal{O}(m^3)$ pour tout i . Nous concluons ainsi que nous réalisons au plus m fois des opérations en $\mathcal{O}(m^3)$. Donc la recherche de p est en $\mathcal{O}(m^4)$.

Comme la taille du D de l'instruction 11 est au plus m , nous déduisons que le construire prend un temps en $\mathcal{O}(m)$. Donc $\mathcal{C}_n(\mathcal{U}(D))$ a une taille en $\mathcal{O}(m^4)$ ce qui implique que $\mathbf{F} = \mathcal{C}_n(\mathcal{U}(A))\mathcal{C}_n(\mathcal{U}(D))$ a une taille en $\mathcal{O}(m^8)$ et le temps pour le calculer est linéaire sur sa taille.

Maintenant, pour vérifier que la forêt $\mathbf{F}$ calculée dans l'instruction 15 est un sous-multi-ensemble de $\mathcal{C}_n(\mathcal{U}(B))$, nous supposons que sa taille est bornée par celle de $\mathcal{C}_n(\mathcal{U}(B))$. Donc le tri de $\mathbf{F}$ est en $\mathcal{O}(m^8)$. De là, la vérification que $\mathbf{F}$ est un sous-multi-ensemble de $\mathcal{C}_n(\mathcal{U}(B))$ est en $\mathcal{O}(m^4)$ car $\mathbf{F}$ et $\mathcal{C}_n(\mathcal{U}(B))$ sont triés par ordre croissant. Ceci signifie que le temps d'une itération de la boucle est en $\mathcal{O}(m^9)$. Donc comme la boucle est exécutée au plus m fois, nous concluons que son temps de calcul est en $\mathcal{O}(m^{10})$. Ceci induit que celui de l'algorithme est en $\mathcal{O}(m^{10})$. $\square$

Pour conclure cette section, présentons deux formes canoniques de FDDS permettant de représenter un unroll.

Nous définissons que A est *composant-minimal* si le nombre de composantes connexes de A est supérieur ou égal au nombre de composantes de tout FDDS A' tel que $\mathcal{U}(A) = \mathcal{U}(A')$ (A aura les plus petites composantes connexes). Symétriquement A est *composant-maximal* si le nombre de composantes connexes de A est inférieur ou égal au nombre de composantes de tout FDDS A' tel que $\mathcal{U}(A) = \mathcal{U}(A')$ (A aura les plus grandes composantes connexes). L'Exemple 2.30 montre un FDDS étant composant-maximal et un autre étant composant-minimal.

Exemple 2.30. Considérons $A = C_5 + C_2$. Alors, A n'est ni composant-maximal ni composant-minimal. Cependant, $A' = 7C_1$ est composant-minimal tandis que $A'' = C_7$ est composant-maximal. Remarquons que $\mathcal{U}(A) = \mathcal{U}(A') = \mathcal{U}(A'')$.

Lemme 2.31. Soit A un FDDS et soit $\mathcal{U}(A) = a_1\Sigma(\mathbf{a}_1) + \dots + a_n\Sigma(\mathbf{a}_n)$ tels que $\mathbf{a}_i$ n'appartient à aucun $\Sigma(\mathbf{a}_j)$ si $i \neq j$. Alors, A est composant-minimal (resp. composant-maximal) si $A = a_1A_1 + \dots + a_nA_n$ (resp. $A_1 + \dots + A_n$) tel que $\mathcal{U}(A_i) = \Sigma(\mathbf{a}_i)$ (resp. $\mathcal{U}(A_i) = a_1\Sigma(\mathbf{a}_1)$).

Démonstration. Soit A' un FDDS tel que $\mathcal{U}(A') = \mathcal{U}(A)$. Posons $A' = \sum_{i=1}^{m'} a'_i A'_i$ tel que chaque A'_i est connexe et A'_i est différent de tous les A'_j avec $i \neq j$.

Soit $1 \leq i \leq m'_A$ un entier. Puisque $\mathcal{U}(A) = \mathcal{U}(A')$, il existe j tel que $\mathbf{a}_j$ est un arbre de $\mathcal{U}(A'_i)$. Trois cas sont possibles pour $a'_i A'_i$.

1. $\mathcal{U}(A'_i) \neq \Sigma(\mathbf{a}_j)$. Comme A'_i est connexe, il résulte que $\mathcal{U}(A'_i)$ contient $c > 1$ copies de $\Sigma(\mathbf{a}_j)$. Donc le FDDS $A'' = A' - A'_i + cA_j$ a plus de composantes connexes que A' .
2. $\mathcal{U}(A'_i) = \Sigma(\mathbf{a}_j)$ et $a'_i \neq a_j$. Donc, il existe un entier k tel que $\mathbf{a}_j$ est un arbre de $\mathcal{U}(A'_k)$ et $\mathcal{U}(A'_k) \neq \Sigma(\mathbf{a}_j)$.

2. Résolutions d'équations polynomiales sur les unrolls – 2.3. Complexité de la résolution des équations monomiales sur les unrolls

3. $\mathcal{U}(A'_i) = \Sigma(\mathbf{a}_j)$ et $a'_i = a_j$.

De cette disjonction de cas, nous déduisons que A est composant-minimal et de plus tous les FDDS A' ayant le même unroll auront strictement moins de composantes connexes.

Une réflexion symétrique prouve le cas composant-maximal. □

Du Théorème 2.29, nous déduisons que nous pouvons résoudre en temps polynomial les équations de la forme $AX = B$, si l'équation admet une solution composant-minimale ou composant-maximale.

Algorithme 2.32. Soient deux FDDS A et B , nous pouvons calculer X tel que X soit composant-minimal ou composant-maximal et que $AX = B$ (s'il existe) en réalisant les opérations suivantes :

1. fixer X à $\emptyset$,
2. calculer $\mathcal{U}(X) \leftarrow \frac{\mathcal{U}(B)}{\mathcal{U}(A)}$ avec l'Algorithme 1.76, s'il existe, sinon renvoyant $\perp$,
3. Considérer $\mathbf{x}$ un arbre de $\mathcal{U}(X)$,
4. trouver x le nombre de copies de $\mathbf{x}$ dans $\mathcal{U}(X)$ et p la plus petite période de $\mathbf{x}$,
5. construire $C = qD$ où D est l'enroulement de $\mathbf{x}$ à période p' avec :
 - dans le cas composant-maximal : $q = 1$ et $p' = x \cdot p$,
 - dans le cas composant-minimal : $q = x$ et $p' = p$,
6. ajouter D à X ,
7. retirer $\mathcal{U}(C)$ de $\mathcal{U}(X)$ en bouclant à la ligne 3 si $\mathcal{U}(X)$ est non vide,
8. vérifier que $AX = B$ et renvoyer X si c'est le cas et $\perp$ sinon.

2.3. Complexité de la résolution des équations monomiales sur les unrolls

La résolution d'équations de la forme $\mathcal{U}(A)\mathcal{U}(X)^k = \mathcal{U}(B)$ peut être réalisée en deux temps. Tout d'abord, trouver un FDDS Y tel que $\mathcal{U}(A)\mathcal{U}(Y) = \mathcal{U}(B)$. Puis un nouveau FDDS Z tel que $\mathcal{U}(Z)^k = \mathcal{U}(Y)$. Ceci est une conséquence de l'annulabilité des unrolls puisque par définition $\mathcal{U}(Y)$ sera unique s'il existe. Ainsi, dans cette section, nous allons prouver qu'étant donné un FDDS B et un entier $k > 0$, nous pouvons trouver un FDDS X tel que $\mathcal{U}(X)^k = \mathcal{U}(B)$ en temps polynomial s'il en existe un. Pour ceci, nous commençons par étudier le cas des racines de forêts.

Avant toute chose, nous définissons qu'une forêt élevée à la puissance 0-ème est le neutre multiplicatif, à savoir le chemin infini. Comme une forêt est l'union disjointe des arbres qui la composent, si $\mathbf{X} = \mathbf{x}_1 + \dots + \mathbf{x}_m$ alors :

$$\mathbf{X}^k = \sum_{k_1 + \dots + k_m = k} \binom{k}{k_1, \dots, k_m} \prod_{i=1}^m \mathbf{x}_i^{k_i}$$

2. Résolutions d'équations polynomiales sur les unrolls – 2.3. Complexité de la résolution des équations monomiales sur les unrolls

avec $k > 0$ un entier. Rappelons que, lorsque $k_1 + \dots + k_m = k$:

$$\binom{k}{k_1, \dots, k_m} = \frac{k!}{k_1! \dots k_m!}.$$

De surcroît, il n'existe qu'une unique manière (à isomorphisme près) de décomposer chaque forêt comme somme d'arbres.

De cette forme, nous déduisons que pour résoudre $\mathbf{X}^k = \mathbf{B}$, nous pouvons construire un algorithme qui tantôt fait un appel récursif pour calculer une racine et tantôt des divisions. Cependant, nous voudrions caractériser précisément à quel moment nous faisons l'un et à quel moment nous faisons l'autre. Pour ceci, nous aimerions être capables d'identifier, à partir de $\mathbf{X} = \mathbf{x}_1 + \dots + \mathbf{x}_n$ avec $\mathbf{x}_i \leq \mathbf{x}_{i+1}$, le plus petit arbre de $\mathbf{X}^k$ ayant profondeur $d(\mathbf{x}_i)$ et étant issu de $\mathbf{x}_i \times \mathbf{X}^{k-1}$. L'idée étant qu'à partir d'un arbre $\mathbf{t}$ de $\mathbf{X}^k$ comme décrit ci-dessus, nous retrouverons l'arbre $\mathbf{x}_i$ qui lui est associé soit via un appel récursif si $\mathbf{t} = \mathbf{x}_i^k$, soit en divisant $\mathbf{t}$ par un certain arbre de $\mathbf{X}^{k-1}$ ayant déjà été calculé.

Lemme 2.33. *Soient $\mathbf{X}$ une forêt de la forme $\mathbf{X} = \mathbf{x}_1 + \dots + \mathbf{x}_n$ (avec $\mathbf{x}_i \leq \mathbf{x}_{i+1}$) et $k > 0$ un entier. Pour tout arbre $\mathbf{x}_i$ de profondeur d_i de $\mathbf{X}$, le plus petit arbre $\mathbf{t}_s$ de $\mathbf{X}^k$ de profondeur d_i étant issu de $\mathbf{x}_i \mathbf{X}^{k-1}$ est égal à $\mathbf{x}_i \mathbf{t}_m^{k-1}$, où $\mathbf{t}_m$ est le plus petit arbre de $\mathbf{X}$ dont la profondeur est au moins d_i .*

Démonstration. Tout d'abord, comme $\mathbf{t}_s$ est de profondeur d_i , le produit d'arbres induit que tous les facteurs de $\mathbf{t}_s$ ont une profondeur au moins d_i et au moins un de ces facteurs a profondeur exactement d_i . De plus, comme l'ordre est compatible avec le produit, la minimalité de $\mathbf{t}_s$ implique que $\mathbf{t}_s$ est isomorphe à $\mathbf{t}_i \mathbf{y}$ avec $\mathbf{y}$ le plus petit arbre de $\mathbf{X}^{k-1}$ dont la profondeur est au moins d_i . Il ne reste plus qu'à montrer que $\mathbf{y} = \mathbf{t}_m^{k-1}$. Pour cela, remarquons que $\mathbf{t}_m^2 < \mathbf{t}_m \mathbf{t}_a$ avec $\mathbf{t}_a$ un arbre de $\mathbf{X}$ de profondeur au moins d_i . Donc, par une simple récurrence, nous déduisons que $\mathbf{y} = \mathbf{t}_m^{k-1}$. $\square$

Rappelons que puisque $\mathcal{D}(\mathbf{X})$ est un morphisme du semi-anneau $(\mathbb{F}, +, \otimes)$ dans lui-même, il respecte la propriété suivante. Cette dernière est l'argument important pour justifier que nous pouvons reconstruire des arbres récursivement.

Lemme 2.34. *Soit $k > 0$ un entier et soit $\mathbf{a}$ un arbre. Alors, $\mathcal{D}(\mathbf{a}^k) = \mathcal{D}(\mathbf{a})^k$.*

Nous pouvons maintenant introduire une procédure algorithmique (Algorithme 2.36) pour calculer la racine d'une forêt. Dans cette dernière, nous reconstruirons la solution niveau par niveau en partant de la profondeur maximale. Pour cela, nous rechercherons le plus petit arbre de la solution qui a au moins une profondeur donnée afin de trouver tous les arbres de cette profondeur (en accord avec les Lemmes 2.33 et 2.34).

Dans l'Algorithme 2.36, nous commençons par tester si la forêt $\mathbf{B}$ est un chemin. En effet, si c'est le cas alors elle est elle-même sa propre racine pour tout entier $k \geq 0$.

Puis si ce n'est pas le cas, nous vérifions que $k \leq \lfloor \log_2 m \rfloor$, avec m la taille de $\mathbf{B}$. En effet, comme $\mathbf{B}$ n'est pas un chemin, il existe un niveau i où elle contient $y \geq 2$ nœuds. Et dans ce cas, s'il existe $\mathbf{X}$ telle que $\mathbf{X}^k = \mathbf{B}$ alors le niveau i de $\mathbf{X}$ contient $\sqrt[k]{y}$ nœuds.

2. Résolutions d'équations polynomiales sur les unrolls – 2.3. Complexité de la résolution des équations monomiales sur les unrolls

Or, le plus petit entier strictement supérieur à 1 ayant une racine k -ème entière est 2^k , ainsi, $y \geq 2^k$. Mais puisque $y \leq m$, nous avons que $k \leq \log_2 m$.

Après cela, nous arrivons dans le cœur de la procédure. Ici, l'idée principale est d'extraire le plus petit arbre parmi les arbres les plus hauts (avec la plus grande profondeur), *i.e.* l'arbre $\mathbf{t}_s$ du Lemme 2.33. Nous utiliserons cet arbre pour reconstruire un des arbres de $\mathbf{X}$, *i.e.* l'arbre $\mathbf{x}_i$ du Lemme 2.33. Cela peut être réalisé selon deux scénarios.

Le premier scénario est que l'arbre $\mathbf{t}_s$ est strictement inférieur au plus petit arbre que nous avons reconstruit, *i.e.* l'arbre $\mathbf{t}_m$ du Lemme 2.33, élevé à la puissance k . Alors, nous calculons un nouvel arbre $\mathbf{x}_i$ de $\mathbf{X}$ par un appel récursif. Nous pouvons voir cela comme notre hypothèse de récurrence.

Le second scénario est que $\mathbf{t}_s$ est supérieur ou égal à $\mathbf{t}_m^k$. Donc, par le Lemme 2.33, il est le produit de $\mathbf{t}_m^{k-1}$ avec un nouvel arbre $\mathbf{x}_i$ de $\mathbf{X}$. Et nous pouvons donc trouver cet arbre $\mathbf{x}_i$ par un appel à l'algorithme de division.

Dans les deux cas, après la reconstruction du nouvel arbre $\mathbf{x}_i$ de $\mathbf{X}$, nous calculons une forêt $\mathbf{F}$ composée de tous les produits possibles entre l'arbre $\mathbf{x}_i$ et $k-1$ arbres de la partie de $\mathbf{X}$ déjà calculée. En d'autres mots, si nous appelons $\mathbf{Y}$ la forêt que nous sommes en train de construire ($\mathbf{Y}$ contenant $\mathbf{x}_i$) alors $\mathbf{F} = \mathbf{x}_i \mathbf{Y}^{k-1}$. Puis nous retirons $\mathbf{F}$ de $\mathbf{B}$. Cela nous permet de prélever progressivement des arbres $\mathbf{t}_s$ de $\mathbf{B}$ étant de plus en plus grands (vis-à-vis de l'ordre sur les arbres) et ayant profondeur $d(\mathbf{t}_s)$ s'il en existe. Lorsque nous retirons tous les arbres de $\mathbf{B}$ obtenables depuis l'ensemble des arbres de $\mathbf{X}$ avec une profondeur au moins $d(\mathbf{t}_s)$, il ne reste que des arbres dont la profondeur est au plus $d(\mathbf{t}_s) - 1$ et nous répétons alors la procédure pour ces arbres. À la fin, nous déduisons que $\mathcal{E}(\mathbf{X})^k$ est égal à $\mathcal{E}(\mathbf{B})$. Ce qui prouve l'hérédité.

De ce fait, en considérant comme cas de base celui où $\mathbf{B}$ est un chemin simple, nous déduisons l'Algorithme 2.36 prenant en entrées une forêt $\mathbf{B}$ étant le côté droit, un entier $k > 0$ étant la puissance et une forêt $\mathbf{X}$ étant la solution récursivement calculée. Nous concluons la preuve du théorème suivant.

Théorème 2.35. *Étant donné une forêt $\mathbf{B}$ et $k > 0$ un entier, nous pouvons résoudre $\mathbf{X}^k = \mathbf{B}$ avec l'Algorithme 2.36.*

La Figure 2.37 donne un exemple d'exécution de l'Algorithme 2.36. Dans cette figure, les sommets rouges représentent les sommets de l'arbre de départ ayant été supprimés au cours de l'exécution. Les arbres en bleu correspondent à la valeur de X (la solution que nous construisons) à l'appel récursif courant. Les parties du grand arbre étant entourées représentent les différentes valeurs de $\mathbf{t}_s$. Tandis que les arbres bleus étant entourés sont les valeurs de $\mathbf{t}_m$. Une flèche simple verte correspond au début, si elle est étiquetée par empile, et à la fin, si elle est étiquetée par dépile, de l'appel récursif de la ligne 11. De même, une double flèche creuse grise correspond au début et à la fin de l'appel récursif de la ligne 25. La flèche pleine rouge représente une itération de la boucle.

Théorème 2.38. *Le calcul de la racine k -ème d'une forêt finie $\mathbf{B}$ (Algorithme 2.36) est réalisable en temps $\mathcal{O}(m^4)$ avec m la taille de $\mathbf{B}$.*

2. Résolutions d'équations polynomiales sur les unrolls – 2.3. Complexité de la résolution des équations monomiales sur les unrolls

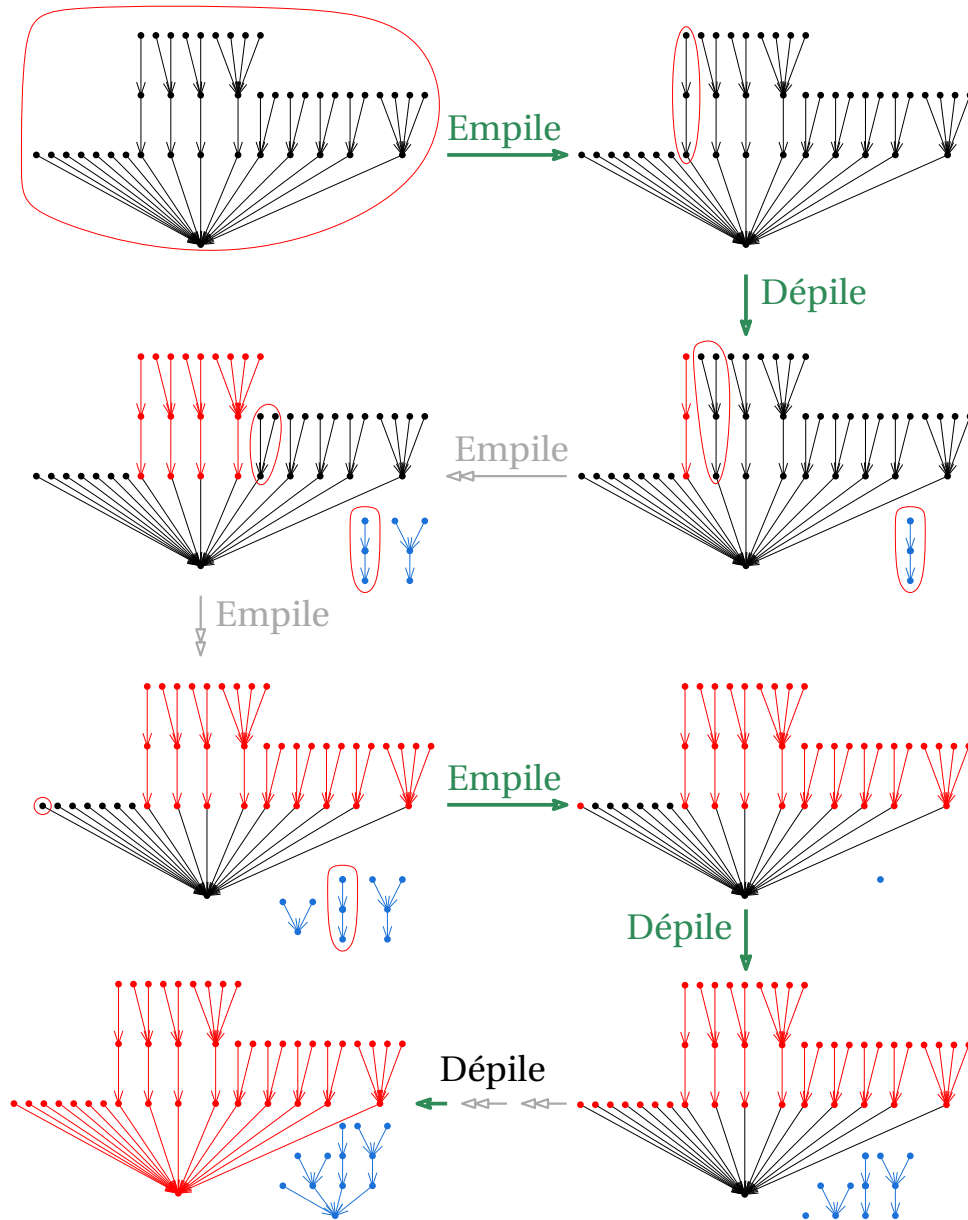


FIGURE 2.37. – Exemple d'exécution de l'Algorithme 2.36 prenant en entrées l'arbre en haut à gauche et $k = 2$.

Algorithme 2.36 Racine($\mathbf{B}, k, \mathbf{X}$)

```

1: Si  $\mathbf{B}$  est un chemin simple alors
2:   renvoyer  $\mathbf{B}$ ,
3: Si  $k > \lfloor \log_2 |\mathbf{B}| \rfloor$  alors
4:   renvoyer  $\perp$ ,
5:  $\mathbf{t}_m \leftarrow \min(\mathbf{X}^k)$ ,
6: soit  $d$  la profondeur de  $\mathbf{B}$ ,
7: soit  $\mathbf{B}_d$  la forêt contenant les arbres de  $\mathbf{B}$  ayant une profondeur au moins  $d$ ,
8:  $\mathbf{B} \leftarrow \mathbf{B} - \mathbf{B}_d$ ,
9:  $\mathbf{t}_s \leftarrow \min(\mathbf{B}_d)$ ,
10: Si  $\mathbf{X} = \emptyset$  ou  $\mathbf{t}_m^k > \mathbf{t}_s$  alors
11:    $\mathbf{x}_i \leftarrow \mathcal{E}(\text{Racine}(\mathcal{D}(\mathbf{t}_s), k, \emptyset))$ ,
12:    $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{x}_i$ ,
13:   Si  $\mathbf{x}_i = \perp$  ou  $\mathbf{x}_i \mathbf{X}^{k-1} \not\subseteq \mathbf{B}_d$  alors
14:     renvoyer  $\perp$ ,
15:    $\mathbf{t}_m \leftarrow \mathbf{x}_i$ ,
16:    $\mathbf{B}_d \leftarrow \mathbf{B}_d - \mathbf{x}_i \mathbf{X}^{k-1}$ ,
17: Tant que  $\mathbf{B}_d \neq \emptyset$  faire
18:    $\mathbf{t}_s \leftarrow \min(\mathbf{B}_d)$ ,
19:    $\mathbf{x}_i \leftarrow \text{Division}(\mathbf{t}_s, \mathbf{t}_m^{k-1})$  (Algorithme 1.76),
20:    $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{x}_i$ ,
21:   Si  $\mathbf{x}_i = \perp$  ou  $\mathbf{x}_i (\mathbf{X})^{k-1} \not\subseteq \mathbf{B}_d$  alors
22:     renvoyer  $\perp$ ,
23:    $\mathbf{B}_d \leftarrow \mathbf{B}_d - \mathbf{x}_i \mathbf{X}^{k-1}$ ,
24: Si  $\mathbf{B}$  n'est pas vide alors
25:   renvoyer Racine( $\mathbf{B}, k, \mathbf{X}$ ),
26: renvoyer  $\mathbf{X}$ .
```

Démonstration. Nous supposons que la forêt $\mathbf{B}$ donnée en entrée est déjà triée en ordre croissant et que chacun des sommets a accès à son code. Ceci peut être obtenu via un pré-traitement réalisable en $\mathcal{O}(m^2)$.

Le temps de calcul lié aux deux premiers tests (vérifier que $\mathbf{B}$ est un chemin et que $k \leq \lfloor \log_2 m \rfloor$) est linéaire. Nous allons montrer que le temps de calcul des opérations qui séparent deux actions consécutives sur la pile des appels récursifs est en $\mathcal{O}(m^3)$, c'est-à-dire la complexité entre : le début de deux appels récursifs, la fin d'un appel et le début d'un nouveau, la fin de deux appels et le début d'un appel et la fin d'un appel. Pour ce faire, il nous faut étudier la complexité de la boucle tant que des lignes 17–23. Nous commençons par borner celle d'une seule itération.

Supposons que nous avons construit $\mathbf{B}_d$ telle que ses arbres soient triés en ordre croissant. Alors, trouver $\mathbf{t}_s$ (instruction 18) prend un temps en $\mathcal{O}(1)$ (c'est le premier arbre de la liste). Puis nous réalisons une division (instruction 19), dont le temps de calcul sera $\mathcal{O}(m_s^3)$ (Théorème 1.77), avec m_s la taille de $\mathbf{t}_s$. Pour vérifier que $\mathbf{x}_i \mathbf{X}^{k-1}$

2. Résolutions d'équations polynomiales sur les unrolls – 2.3. Complexité de la résolution des équations monomiales sur les unrolls

soit un sous-multi-ensemble de $\mathbf{B}_d$ (instruction 21), nous commençons par calculer $\mathbf{x}_i \mathbf{X}^{k-1}$ en nous assurant que la taille de ce produit soit bornée par m . Puis trions les arbres de ce résultat en ordre croissant, ce qui est possible en temps $\mathcal{O}(m^2)$ par le Lemme 1.64. Enfin, la vérification que chaque arbre de $\mathbf{x}_i \mathbf{X}^{k-1}$ appartient à $\mathbf{B}_d$ peut être faite via un unique parcours des arbres de $\mathbf{B}_d$ durant lequel nous réaliserons un nombre de tests d'égalité en $\mathcal{O}(m)$ prenant chacun un temps en $\mathcal{O}(m)$. Donc la complexité de la ligne 21 est en $\mathcal{O}(m^2)$. Ceci prouve également que nous réalisons les lignes 13, 16 et 23 en $\mathcal{O}(m^2)$. Nous concluons finalement que le temps de calcul d'une itération de la boucle est en $\mathcal{O}(m^2 + m_s^3)$.

Relevons que le nombre d'itérations de la boucle est au plus m . De plus, la somme des différents m_s est inférieure ou égale à m car les $\mathbf{t}_s$ sont des arbres de $\mathbf{B}$. Nous en déduisons que la complexité en temps de la boucle est en $\mathcal{O}(m^3)$. Nous constatons que toutes les autres opérations (hors appel récursif) ont une complexité dominée par $\mathcal{O}(m^3)$.

De là, comme nous pouvons borner le nombre d'appels récursifs par m , nous concluons que le temps de calcul de l'algorithme est en $\mathcal{O}(m^4)$ et le théorème suit. $\square$

Corollaire 2.39. *Soit $\mathbf{B}$ une forêt finie de taille m . Alors il est possible de décider, en $\mathcal{O}(m^4 \log m)$, s'il existe une forêt $\mathbf{X}$ et un entier $k > 0$ tels que $\mathbf{X}^k = \mathbf{B}$.*

Démonstration. Par le Théorème 2.38, nous pouvons calculer la racine k -ème de $\mathbf{B}$ en $\mathcal{O}(m^4)$ et ce pour tout entier $k \geq 0$. Et puisque le nombre de k possibles est borné par le logarithme de la taille de $\mathbf{B}$, essayer toutes les valeurs possibles de k est réalisable en temps logarithmique. $\square$

Du Corollaire 2.39, nous pouvons aisément conclure que le problème correspondant d'énumération, i.e. trouver tous les couples $\mathbf{X}, k$ tels que $\mathbf{X}^k = \mathbf{B}$, appartient à une classe de complexité incluse dans la classe DelayP.

Nous sommes en mesure d'adapter le Théorème 2.23 de la division au calcul des racines.

Théorème 2.40. *Soient B un FDDS et k et n deux entiers tels que $k \geq 0$ et $n = 2\alpha^2 + d(B)$ avec α le nombre d'arbres d'unroll dans $\mathcal{U}(B)$. Alors, il existe un FDDS X tel que $\mathcal{U}(X)^k = \mathcal{U}(B)$ si et seulement s'il existe une forêt finie $\mathbf{X}$ telle que $\mathbf{X}^k = \mathcal{C}_n(\mathcal{U}(B))$. De plus, le cas échéant, $\mathcal{C}_n(\mathcal{U}(X)) = \mathbf{X}$.*

Démonstration. ($\Rightarrow$) Supposons qu'il existe un FDDS X tel que $\mathcal{U}(X)^k = \mathcal{U}(B)$. Donc $\mathcal{C}_i(\mathcal{U}(X))^k = \mathcal{C}_i(\mathcal{U}(B))$ pour tout entier $i \geq 0$. En particulier, $\mathcal{C}_n(\mathcal{U}(X))^k = \mathcal{C}_n(\mathcal{U}(B))$ et la direction suit.

($\Leftarrow$) Si $k = 0$ alors $\mathcal{U}(B)$ est un chemin infini. Ainsi en particulier si $\mathbf{X}$ est un chemin de longueur n alors $\mathbf{X}^k = \mathcal{C}_n(\mathcal{U}(B))$. Donc en posant $X = B$ l'assertion suit.

Supposons maintenant que $k > 0$. Remarquons que de l'hypothèse, nous avons que tous les arbres de $\mathbf{X}^k$ ont profondeur n . Ceci induit que tous les arbres de $\mathbf{X}$ ont aussi cette profondeur. Pour ce cas, nous raisonnons en deux temps. Tout d'abord, nous prouvons qu'il existe une forêt d'arbres d'unroll $\mathbf{Y}$ telle que $\mathbf{X} = \mathcal{C}_n(\mathbf{Y})$. Puis nous montrons que $\mathbf{Y}$ est une somme de décalages, donc un unroll.

2. Résolutions d'équations polynomiales sur les unrolls – 2.3. Complexité de la résolution des équations monomiales sur les unrolls

Soit $\mathbf{x}$ un arbre de $\mathbf{X}$. Alors, il existe un arbre $\mathbf{b}$ de $\mathcal{U}(B)$ de plus petite période p tel que $\mathbf{x}^k = \mathcal{C}_n(\mathbf{b})$. Par une preuve similaire à celle du Théorème 2.23, nous sommes capables de construire un arbre d'unroll $\mathbf{y}$ de période p tel que $\mathcal{C}_n(\mathbf{y}) = \mathbf{x}$. Puisque n est suffisamment grand, il nous est permis d'utiliser le Lemme 2.4. Ce dernier implique que $\mathbf{y}^k = \mathbf{b}$. En appliquant cette méthode à tous les arbres de $\mathbf{X}$, nous obtenons une forêt d'arbres d'unroll $\mathbf{Y}$ dont chaque arbre a une période au plus α .

Passons à la preuve que $\mathbf{Y}$ est une somme de décalages. Nous montrons cela par une récurrence. Plus précisément, nous démontrons qu'il existe $\mathbf{y}_1, \dots, \mathbf{y}_{m_Y}$ tels que $\sum_{i=1}^{m_Y} \Sigma(\mathbf{y}_i) = \mathbf{Y}$ avec $\mathbf{y}_1 = \min(\mathbf{Y})$ et $\mathbf{y}_i = \min(\mathbf{Y} - \sum_{j=1}^{i-1} \Sigma(\mathbf{y}_j))$. Le cas de base et l'hérédité sont donnés dans les deux prochaines affirmations.

Affirmation 2.41. *La forêt $\mathbf{Y}$ admet $\Sigma(\mathbf{y}_1)$ comme sous-multi-ensemble.*

Preuve. Par définition de $\mathbf{y}_1$ et puisque l'ordre est compatible avec le produit, il suit que $\mathbf{y}_1^k$ est le plus petit arbre de $\mathbf{Y}^k$ et a période au plus α , car $\mathbf{y}_1$ a une période p bornée par α et que $\mathbf{y}_1^k$ a une période égale à $\text{lcm}(p, \dots, p) = p$. De plus, comme n est suffisamment grand, nous déduisons du Lemme 2.4 que $\mathcal{C}_n(\mathbf{y}_1)^k$ est le plus petit arbre de $\mathbf{X}^k = \mathcal{C}_n(\mathcal{U}(B))$. Donc, toujours par ce lemme, nous obtenons $\mathbf{y}_1^k = \mathbf{b}$ avec $\mathbf{b} = \min(\mathcal{U}(B))$.

Soit i un entier. Montrons que $\mathbf{Y}$ contient $\sigma_i(\mathbf{y}_1)$. Par définition des unrolls, nous avons que $\sigma_i(\mathbf{y}_1)^k = \sigma_i(\mathbf{b})$ est un arbre de $\mathcal{U}(B)$. Comme $\mathbf{X}^k = \mathcal{C}_n(\mathcal{U}(B))$, il existe $\mathbf{x}_1, \dots, \mathbf{x}_k$ des arbres de $\mathbf{X}$ tels que $\prod_{j=1}^k \mathbf{x}_j = \mathcal{C}_n(\sigma_i(\mathbf{b}))$. Et par construction, nous inférons qu'il existe $\mathbf{t}_1, \dots, \mathbf{t}_k$ dans $\mathbf{Y}$ tels que $\mathcal{C}_n(\mathbf{t}_j) = \mathbf{x}_j$ pour tout j .

Observons que puisque n est suffisamment grand, la preuve du Lemme 2.3 induit que seul un arbre de $\mathcal{D}(\mathcal{C}_n(\sigma_i(\mathbf{b})))$ est de profondeur maximale, à savoir $n - 1$. Donc $\mathcal{D}(\mathbf{x}_j)$ contient également un seul arbre de profondeur maximale et ce pour tout j . En effet, tous les $\mathcal{D}(\mathbf{x}_j)$ contiennent un arbre de profondeur $n - 1$, car les $\mathbf{x}_j$ sont des arbres de profondeur n . Et comme $\mathcal{D}(\mathcal{C}_n(\sigma_i(\mathbf{b}))) = \mathcal{D}(\prod_{j=1}^k \mathbf{x}_j)$, s'il existait un $\mathbf{x}_j$ tel que $\mathcal{D}(\mathbf{x}_j)$ contient deux arbres de profondeur $n - 1$ alors c'est aussi le cas dans $\mathcal{D}(\prod_{j=1}^k \mathbf{x}_j)$, contradiction.

Il suit alors que l'arbre de profondeur maximale de $\mathcal{D}(\mathcal{C}_n(\sigma_i(\mathbf{b})))$ est $\mathcal{C}_{n-1}(\sigma_{i+1}(\mathbf{b}))$ et celui de $\mathcal{D}(\mathbf{x}_j)$ est $\mathcal{C}_{n-1}(\sigma_1(\mathbf{t}_j))$ pour tout j . Nous pouvons appliquer ce même raisonnement pour tout entier inférieur ou égal à α ce qui nous permet de conclure que $\mathcal{C}_{n-i'}(\sigma_{i'}(\mathbf{t}_j)) = \mathcal{C}_{n-i'}(\sigma_{i+i'}(\mathbf{b})) = \mathcal{C}_{n-i'}(\mathbf{b})$ avec $i' = -i \pmod p$. Il suit aussi que $\prod_{j=1}^k \mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_j)) = \mathcal{C}_{n-i'}(\mathbf{y}_1)^k$.

Ce qui implique qu'il existe un $\mathbf{t}_j$ tel que $\mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_j)) \leq \mathcal{C}_{n-i'}(\mathbf{y}_1)$. De là, puisque $i \leq \alpha$ et comme $\mathbf{t}_j$ et $\mathbf{y}_1$ ont une période bornée par α , il suit que $n - i'$ est toujours suffisamment grand pour appliquer le Lemme 2.4. Et donc, $\sigma_{-i}(\mathbf{t}_j) \leq \mathbf{y}_1$.

En outre, par construction nous avons que $\mathbf{t}_j^k$ est un arbre de $\mathcal{U}(B)$; par définition des unrolls, il suit que $\sigma_{-i}(\mathbf{t}_j)^k$ est un arbre de $\mathcal{U}(B)$. Ainsi, par la minimalité de $\mathbf{b}$, il résulte que $\sigma_{-i}(\mathbf{t}_j)^k \geq \mathbf{y}_1^k$. Étant donné que l'ordre est compatible avec le produit, nous concluons que $\sigma_{-i}(\mathbf{t}_j) \geq \mathbf{y}_1$ et donc $\sigma_{-i}(\mathbf{t}_j) = \mathbf{y}_1$. De là, comme par le Lemme 1.57, nous pouvons simplifier $\prod_{r=1}^k \mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_j)) = \mathcal{C}_{n-i'}(\mathbf{y}_1)^k$ par $\mathcal{C}_{n-i'}(\mathbf{y}_1)$, nous sommes

2. Résolutions d'équations polynomiales sur les unrolls – 2.3. Complexité de la résolution des équations monomiales sur les unrolls

en mesure d'appliquer récursivement ce raisonnement et déduisons que $\sigma_{-i}(\mathbf{t}_r) = \mathbf{y}_1$ pour tout entier r . $\diamond$

Affirmation 2.42. Soit $1 \leq j < m_Y$ un entier. Si $\mathbf{F} = \sum_{i=1}^j \Sigma(\mathbf{y}_i)$ est un sous-multi-ensemble de $\mathbf{Y}$ alors c'est aussi le cas de $\mathbf{F} + \Sigma(\mathbf{y}_{j+1}) = \sum_{i=1}^{j+1} \Sigma(\mathbf{y}_i)$.

Preuve. Comme $\mathbf{F}$ est un sous-multi-ensemble de $\mathbf{Y}$, il suit que $\mathbf{F}^k$ est un sous-multi-ensemble de $\mathbf{Y}^k$. De là, puisque $\mathbf{Y}^k$ contient α arbres, nous avons que $\mathbf{F}^k$ contient au plus α arbres. En outre, comme $\mathbf{F}$ est un unroll, nous avons que $\mathbf{F}^k$ en est aussi un et donc tous ces arbres ont une période bornée par α . De plus, comme $\mathcal{C}_n(\mathbf{Y}) = \mathbf{X}$, il suit qu'il existe un sous-multi-ensemble $\mathbf{X}'$ de $\mathbf{X}$ et un sous-multi-ensemble $\mathbf{B}$ de $\mathcal{U}(B)$ tels que $\mathcal{C}_n(\mathbf{F})^k = (\mathbf{X}')^k = \mathcal{C}_n(\mathbf{B})$. Donc, comme n est suffisamment grand, le Lemme 2.4 induit que $\mathbf{F}^k = \mathbf{B}$. Ceci signifie que $\mathcal{U}(B) - \mathbf{F}^k$ est bien défini et est un unroll.

Soit $\mathbf{y}_{j+1}$ le plus petit arbre de $\mathbf{Y} - \mathbf{F}$. Par le Lemme 2.33, nous avons que $\mathbf{y}_1^{k-1} \mathbf{y}_{j+1}$ est le plus petit arbre de $\mathbf{Y}^k - \mathbf{F}^k$. Nous en concluons que $\mathcal{C}_n(\mathbf{y}_1^{k-1} \mathbf{y}_{j+1})$ est le plus petit arbre de $\mathbf{X}^k - \mathcal{C}_n(\mathbf{F})^k$. De surcroît, puisque la plus petite période de $\mathbf{y}_1^{k-1}$ est bornée par la plus petite période de $\mathbf{y}_1$, elle-même bornée par α , il suit que l'arbre $\mathbf{y}_1^{k-1} \mathbf{y}_{j+1}$ est un arbre de période au plus α^2 . Donc, comme n est suffisamment grand, le Lemme 2.4 implique que $\mathbf{y}_1^{k-1} \mathbf{y}_{j+1} = \mathbf{b}$ avec $\mathbf{b}$ le plus petit arbre de $\mathcal{U}(B) - \mathbf{F}^k$ avec période p .

Soit i un entier. Montrons que $\mathbf{Y}$ contient $\sigma_i(\mathbf{y}_{j+1})$. Par définition des unrolls, nous avons que $\sigma_i(\mathbf{y}_1^{k-1} \mathbf{y}_{j+1}) = \sigma_i(\mathbf{b})$ est un arbre de $\mathcal{U}(B) - \mathbf{F}^k$. Conséquemment, comme $\mathbf{X}^k - \mathcal{C}_n(\mathbf{F})^k = \mathcal{C}_n(\mathcal{U}(B)) - \mathcal{C}_n(\mathbf{F})^k$, il résulte qu'il existe $\mathbf{x}_1, \dots, \mathbf{x}_k$ des arbres de $\mathbf{X}$ avec au moins un arbre dans $\mathbf{X} - \mathcal{C}_n(\mathbf{F})$ tels que $\prod_{r=1}^k \mathbf{x}_r = \mathcal{C}_n(\sigma_i(\mathbf{b}))$. Et par construction, nous inférons qu'il existe $\mathbf{t}_1, \dots, \mathbf{t}_k$ dans $\mathbf{Y}$ avec au moins un arbre dans $\mathbf{Y} - \mathbf{F}$ tels que $\mathcal{C}_n(\mathbf{t}_r) = \mathbf{x}_r$ pour tout r . Soit $i' = -i \bmod p$. Alors, nous avons que $\prod_{r=1}^k \mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) = \mathcal{C}_{n-i'}(\mathbf{y}_1^{k-1} \mathbf{y}_{j+1})$. Soit r un entier entre 1 et k . Montrons maintenant que $\mathcal{C}_{n-i'}(\mathbf{y}_1) \leq \mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) \leq \mathcal{C}_{n-i'}(\mathbf{y}_{j+1})$. Puisque par construction $\mathbf{t}_r^k$ est un arbre de $\mathcal{U}(B)$, nous déduisons de la définition des unrolls que $\sigma_{-i}(\mathbf{t}_r)^k$ est un arbre de $\mathcal{U}(B)$. Par conséquent, la minimalité de $\mathbf{y}_1^k$ induit que $\sigma_{-i}(\mathbf{t}_r)^k \geq \mathbf{y}_1^k$ et aussi $\sigma_{-i}(\mathbf{t}_r) \geq \mathbf{y}_1$. Et comme $i \leq \alpha$, nous avons que $n - i'$ est toujours suffisamment grand pour appliquer le Lemme 2.4, et ainsi $\mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) \geq \mathcal{C}_{n-i'}(\mathbf{y}_1)$. Nous en concluons que $\mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) \leq \mathcal{C}_{n-i'}(\mathbf{y}_{j+1})$ pour tout r .

Montrons que $\sigma_{-i}(\mathbf{t}_k) \geq \mathbf{y}_{j+1}$. Comme par hypothèse sur $\mathbf{t}_1, \dots, \mathbf{t}_k$, il existe un entier, disons k sans perte de généralité, tel que $\mathbf{t}_k$ appartient à $\mathbf{Y} - \mathbf{F}$. Comme $\mathbf{t}_k$ est un arbre de $\mathbf{Y} - \mathbf{F}$, l'hypothèse de récurrence induit que $\sigma_i(\mathbf{y}_1)^{k-1} \mathbf{t}_k$ est un arbre de $\mathbf{Y}^k - \mathbf{F}^k$. Il suit que $\mathcal{C}_n(\sigma_i(\mathbf{y}_1)^{k-1} \mathbf{t}_k)$ est un arbre de $\mathbf{X}^k - \mathcal{C}_n(\mathbf{F})^k$. De ce fait, comme n est toujours suffisamment grand, l'arbre $\sigma_i(\mathbf{y}_1)^{k-1} \mathbf{t}_k$ est un arbre de $\mathcal{U}(B) - \mathbf{F}^k$. Du Lemme 2.21, il en résulte que $\mathbf{y}_1^{k-1} \sigma_{-i}(\mathbf{t}_k)$ est aussi un arbre de $\mathcal{U}(B) - \mathbf{F}^k$. Par conséquent, $\mathbf{y}_1^{k-1} \sigma_{-i}(\mathbf{t}_k) \geq \mathbf{y}_1^{k-1} \mathbf{y}_{j+1}$. La compatibilité de l'ordre avec le produit permet de conclure que $\sigma_{-i}(\mathbf{t}_k) \geq \mathbf{y}_{j+1}$. De ce fait, la réflexion précédente induit que $\sigma_{-i}(\mathbf{t}_k) = \mathbf{y}_{j+1}$.

À ce point, nous avons démontré que $\mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_k)) = \mathcal{C}_{n-i'}(\mathbf{y}_{j+1})$. Ainsi, étant donné que $\prod_{r=1}^k \mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) = \mathcal{C}_{n-i'}(\mathbf{y}_1^{k-1} \mathbf{y}_{j+1})$, nous déduisons du Lemme 1.57 que $\prod_{r=1}^{k-1} \mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) = \mathcal{C}_{n-i'}(\mathbf{y}_1)^{k-1}$. Donc, si l'un des arbres $\mathbf{t}_r$, avec r entre 1 et

2. Résolutions d'équations polynomiales sur les unrolls – 2.3. Complexité de la résolution des équations monomiales sur les unrolls

$k - 1$, est tel que $\mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) > \mathcal{C}_{n-i'}(\mathbf{y}_1)$ alors $\prod_{r=1}^{k-1} \mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) > \mathcal{C}_{n-i'}(\mathbf{y}_1)^{k-1}$ car $\mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) \geq \mathcal{C}_{n-i'}(\mathbf{y}_1)$ pour tout r . Par voie de conséquence, nous déduisons que $\mathcal{C}_{n-i'}(\sigma_{-i}(\mathbf{t}_r)) = \mathcal{C}_{n-i'}(\mathbf{y}_1)$ et comme $n - i'$ est toujours suffisamment grand pour appliquer le Lemme 2.4, nous concluons que $\sigma_{-i}(\mathbf{t}_r) = \mathbf{y}_1$ pour tout r et l'affirmation suit. $\diamond$

Nous concluons que $\mathbf{Y}$ est un unroll tel que $\mathcal{C}_n(\mathbf{Y}) = \mathbf{X}$. De ce fait, $\mathcal{C}_n(\mathbf{F})^k = \mathbf{X}^k$ et donc tous les arbres de $\mathcal{C}_n(\mathbf{F})^k$ ont une période bornée par α . Donc, comme n est suffisamment grand, le Lemme 2.4 implique que $\mathbf{Y}^k = \mathcal{U}(B)$. $\square$

Corollaire 2.43. Soient A, B deux FDDS et k et n deux entiers tels que $k \geq 0$ et $n = 2\alpha^2 + d(B)$ avec α le nombre d'arbres d'unroll dans $\mathcal{U}(B)$. Il existe un FDDS X tel que $\mathcal{U}(A)\mathcal{U}(X)^k = \mathcal{U}(B)$ si et seulement s'il existe une forêt finie $\mathbf{X}$ telle que $\mathcal{C}_n(\mathcal{U}(A))\mathbf{X}^k = \mathcal{C}_n(\mathcal{U}(B))$. De plus, le cas échéant, $\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathbf{Y})$.

Proposition 2.44. Soient $k > 0$ un entier et B un FDDS. S'il existe une forêt $\mathbf{X}$ telle que $\mathbf{X}^k = \mathcal{U}(B)$ alors il existe un FDDS X tel que $\mathcal{U}(X) = \mathbf{X}$.

Démonstration. Supposons qu'il existe une forêt $\mathbf{X}$ telle que $\mathbf{X}^k = \mathcal{U}(B)$. Alors, nous avons $\mathcal{C}_n(\mathbf{X})^k = \mathcal{C}_n(\mathcal{U}(B))$ pour tout entier $n \geq 0$. De ce fait, par le Théorème 2.40, il suit qu'il existe un FDDS X tel que $\mathcal{U}(X)^k = \mathcal{U}(B)$. La Proposition 1.68 induit que $\mathbf{X} = \mathcal{U}(X)$. $\square$

Corollaire 2.45. Soient A, B deux FDDS et $k \geq 0$ un entier. S'il existe $\mathbf{X}$ telle que $\mathcal{U}(A)\mathbf{X}^k = \mathcal{U}(B)$ alors il existe X un FDDS tel que $\mathcal{U}(X) = \mathbf{X}$.

Du Théorème 2.40, nous déduisons l'Algorithme 2.46 permettant de calculer la racine d'un unroll. Il prend en entrée un FDDS B et un entier $k \geq 0$.

La correction de l'Algorithme 2.46 est une conséquence directe du Théorème 2.40. De plus, une analyse de complexité similaire à celle du Théorème 2.29 nous permet de déduire que cet algorithme s'exécute en $\mathcal{O}(m^{16})$ où m est la taille de B . Ce qui conclut la preuve du théorème suivant.

Théorème 2.47. Le calcul des racines d'unroll (Algorithme 2.46) est réalisable en $\mathcal{O}(m^{16})$ où m est la taille de B .

Étant donné que le degré du polynôme (16) est important et que nous utiliserons régulièrement cet algorithme, nous ne réaliserons plus d'étude détaillée de complexité (sauf dans un cas particulier) et nous nous contenterons de montrer que l'algorithme est polynomial.

Corollaire 2.48. Soient A et B deux FDDS et $k \geq 0$ un entier. Nous trouvons en temps polynomial un FDDS X tel que $\mathcal{U}(A)\mathcal{U}(X)^k = \mathcal{U}(B)$ s'il en existe un.

Corollaire 2.49. Soient A et B deux FDDS. Il est possible de décider, en temps polynomial, s'il existe un FDDS X et un entier $k \geq 0$ tels que $\mathcal{U}(A)\mathcal{U}(X)^k = \mathcal{U}(B)$.

Finalement, nous pouvons adapter l'Algorithme 2.32 pour résoudre en temps polynomial les équations de la forme $AX^k = B$, avec A et B deux FDDS, si X est un FDDS composant-maximal ou composant-minimal.

Algorithme 2.46 $\text{UnrollRacine}(B, k)$

```

1:  $X \leftarrow \emptyset$ ,
2: soit  $\alpha$  le nombre d'arbres de  $\mathcal{U}(B)$  et soit  $n \leftarrow 2\alpha^2 + d(B)$ ,
3:  $\mathbf{B} \leftarrow \mathcal{C}_n(\mathcal{U}(B))$  où nous avons associé à chaque arbre de  $\mathbf{B}$  leur longueur de cycle d'origine,
4:  $\mathbf{X} \leftarrow \text{Racine}(\mathbf{B}, k, \emptyset)$  (Algorithme 2.36),
5: Si  $\mathbf{X}$  n'est pas défini alors
6:   renvoyer  $\perp$ ,
7: Tant que  $\mathbf{X}$  n'est pas vide faire
8:   construire  $w_{\mathbf{x}}$  un motif périodique d'un arbre  $\mathbf{x}$  de  $\mathbf{X}$  de longueur  $b$  où  $b$  est la longueur associée à  $\mathbf{x}^k$ ,
9:   trouver  $p$  la plus petite période du mot  $w_{\mathbf{x}}$ ,
10:  réaliser  $D$  l'enroulement de  $\mathbf{x}$  à période  $p$ ,
11:   $X \leftarrow X + D$ ,
12:   $\mathbf{X} \leftarrow \mathbf{X} - \mathcal{C}_n(\mathcal{U}(D))$ ,
13: renvoyer  $X$ .
```

2.4. Complexité de la résolution des équations polynomiales sur les unrolls

Cette section est l'aboutissement de ce chapitre. Elle vise à étudier les équations polynomiales sur les unrolls avec côté droit constant. Les unrolls seront encodés par des FDDS ayant lesdits unrolls. C'est-à-dire que nous allons étudier les équations de la forme $P(\mathbf{X}) = \mathcal{U}(B)$ avec $P = \sum_{i=0}^m \mathcal{U}(A_i)\mathbf{X}^i$ où $A_0, \dots, A_m$ et B sont des FDDS. Nous montrerons que nous pouvons résoudre de telles équations en temps polynomial sur la taille des FDDS. Nous prouverons également que s'il existe une solution $\mathbf{X}$ à une telle équation alors il existe un FDDS X tel que $\mathcal{U}(X) = \mathbf{X}$. De plus, ces démonstrations nous permettront d'établir que tous les polynômes sur les unrolls sont injectifs. Dans la suite de cette section, nous ferons de multiples preuves par récurrence sur la profondeur d d'une forêt $\mathbf{F}$. Dans ces preuves, nous ne considérons pas forcément $\mathbf{F}$ elle-même mais plutôt un sous-multi-ensemble de $\mathbf{F}$, noté $\Gamma_d(\mathbf{F})$, contenant l'ensemble des arbres de $\mathbf{F}$ dont la profondeur est au moins d . Relevons que les définitions de la somme et du produit induisent que $\Gamma_d(\mathbf{A} + \mathbf{B}) = \Gamma_d(\mathbf{A}) + \Gamma_d(\mathbf{B})$ et $\Gamma_d(\mathbf{A} \otimes \mathbf{B}) = \Gamma_d(\mathbf{A}) \otimes \Gamma_d(\mathbf{B})$ et ce pour toutes forêts $\mathbf{A}$ et $\mathbf{B}$ et tout entier $d \geq 0$.

Pour permettre tout cela, en accord avec le Théorème 2.29, nous allons étudier le cas des forêts finies. Et une fois de plus, l'ensemble de notre raisonnement reposera principalement sur l'ordre total compatible avec le produit de [40] et introduit dans la Section 1.3. Considérons donc $P = \sum_{i=0}^m \mathbf{A}_i \mathbf{X}^i$ un polynôme de forêts. À l'instar de la Section 2.3, nous voudrions être capables d'identifier, à partir de $\mathbf{X} = \mathbf{t}_1 + \dots + \mathbf{t}_n$ avec $\mathbf{t}_i \leq \mathbf{t}_{i+1}$, le plus petit arbre de $P(\mathbf{X})$ qui est issu de $\min(\mathbf{A}_j)\mathbf{t}_i \mathbf{X}^{j-1}$, pour un certain j , et qui a la même profondeur que $\mathbf{t}_i$ et ce pour tout arbre $\mathbf{t}_i$. L'idée étant que pour connaître chaque $\mathbf{x}_i$ de $\mathbf{X}$ nous ferions une division par $\min(\mathbf{A}_j)$ pour obtenir un arbre

2. Résolutions d'équations polynomiales sur les unrolls – 2.4. Complexité de la résolution des équations polynomiales sur les unrolls

$\mathbf{t}$ puis soit nous calculerions sa racine si $\mathbf{t} = \mathbf{t}_i^j$ soit nous diviserions $\mathbf{t}$ par le plus petit arbre issu de la partie de $\mathbf{X}$ déjà calculée élevée à la puissance $k - 1$.

Lemme 2.50. Soit $d \geq 0$ un entier et soit $P = \sum_{i=1}^m \mathbf{A}_i \mathbf{X}^i$ un polynôme de forêts finies sans terme constant (i.e. sans $\mathbf{A}_0$) tel que chaque arbre des $\mathbf{A}_i$ a une profondeur au moins d . Soit $\mathbf{X}$ une forêt de profondeur d .

Alors, il existe un entier $1 \leq r \leq m$ tel que pour tout arbre $\mathbf{x}_j$ de $\mathbf{X}$ ayant profondeur h , nous avons que

$$\min \{ \mathbf{a} \mathbf{x} \mathbf{x}_j \mid 1 \leq i \leq m, \mathbf{a} \in \mathbf{A}_i, \mathbf{x} \in \Gamma_{d(h)}(\mathbf{X})^{i-1} \} = \min(\mathbf{A}_r) \min(\Gamma_h(\mathbf{X}))^{r-1} \mathbf{x}_j.$$

Démonstration. Soit $\mathbf{x}_j$ un arbre de $\mathbf{X}$ ayant profondeur h . Soit $\mathbf{x}_1$ le plus petit arbre de $\mathbf{X}$ ayant profondeur au moins h , en symbole $\mathbf{x}_1 = \min(\Gamma_h(\mathbf{X}))$. Commençons par montrer le cas où $\mathbf{x}_j = \mathbf{x}_1$.

Soit $\mathbf{t}_{min}$ le plus petit arbre de $P(\mathbf{X})$ ayant profondeur au moins h , en symbole $\mathbf{t}_{min} = \min(\Gamma_h(P(\mathbf{X})))$. Donc il existe un entier $1 \leq r \leq m$ tel que la forêt $\Gamma_h(\mathbf{A}_r) \Gamma_h(\mathbf{X})^r$ contient l'arbre $\mathbf{t}_{min}$. Soit $\mathbf{a}_1$ le plus petit arbre de $\mathbf{A}_r$. Notons que par définition, $\mathbf{a}_1$ a une profondeur supérieure ou égale à h .

Comme l'ordre est compatible avec le produit, il résulte que $\mathbf{x}_1^2 \leq \mathbf{x}_1 \mathbf{x}_k$ pour tout arbre $\mathbf{x}_k$ de $\Gamma_h(\mathbf{X})$. Par récurrence, nous déduisons que $\mathbf{t}_{min}$ est un arbre de $\Gamma_h(\mathbf{A}_r) \mathbf{x}_1^r$. De façon analogue, nous avons aussi $\mathbf{a}_1 \mathbf{x}_1^r \leq \mathbf{a}_k \mathbf{x}_1^r$ pour tout arbre $\mathbf{a}_k$ de $\Gamma_h(\mathbf{A}_r)$, et nous concluons que $\mathbf{t}_{min} = \mathbf{a}_1 \mathbf{x}_1^r$. La propriété est donc vraie pour $\mathbf{x}_j = \mathbf{x}_1$.

Nous supposons maintenant que $\mathbf{x}_j \neq \mathbf{x}_1$ induisant que $\mathbf{x}_j > \mathbf{x}_1$. En reprenant le résultat du point précédent, nous déduisons du Lemme 1.57 que

$$\mathcal{C}_{d(\mathbf{x}_1)}(\mathbf{a}_1 \mathbf{x}_1^{r-1}) \leq \mathcal{C}_{d(\mathbf{x}_1)}(\mathbf{a} \mathbf{x}_1^{i-1})$$

et ce pour tout entier $1 \leq i \leq m$ et tout arbre $\mathbf{a}$ de $\mathbf{A}_i$. Observons que par le choix de $\mathbf{x}_1$ nous avons que $d(\mathbf{x}_1) \geq h$, ce qui implique que

$$\mathcal{C}_h(\mathbf{a}_1 \mathbf{x}_1^{r-1}) \leq \mathcal{C}_h(\mathbf{a} \mathbf{x}_1^{i-1}).$$

En effet, comme l'ordre est issu d'un parcours en largeur, nous sommes à même de prouver que $\mathbf{a} \leq \mathbf{b}$ si et seulement si $\mathcal{C}_n(\mathbf{a}) \leq \mathcal{C}_n(\mathbf{b})$ pour tout entier $n \geq 0$.

Maintenant, comme $\mathbf{x}_j$ est de profondeur h , il résulte que $\mathcal{C}_h(\mathbf{a}_1 \mathbf{x}_1^{r-1}) \mathbf{x}_j = \mathbf{a}_1 \mathbf{x}_1^{r-1} \mathbf{x}_j$ et $\mathcal{C}_h(\mathbf{a} \mathbf{x}_1^{i-1}) \mathbf{x}_j = \mathbf{a} \mathbf{x}_1^{i-1} \mathbf{x}_j$. Nous concluons que $\mathbf{a}_1 \mathbf{x}_1^{r-1} \mathbf{x}_j \leq \mathbf{a} \mathbf{x}_1^{i-1} \mathbf{x}_j$. Et puisque $\mathbf{x}_1^{k-1} \leq \mathbf{x}$ pour tout entier $k > 0$ et avec $\mathbf{x} \in \Gamma_h(\mathbf{X})^{k-1}$, le lemme suit. $\square$

Grâce au Lemme 2.50, nous pouvons montrer le premier résultat important de cette section. Rappelons que l'ensemble des forêts finies de profondeur au plus d_{max} avec $+$ et $\otimes$ forme un semi-anneau (Proposition 1.52).

Proposition 2.51. Soit $P = \sum_{i=0}^m \mathbf{A}_i \mathbf{X}^i$ avec $d_{max} = \max_{i>0} (d(\mathbf{A}_i))$ et $d(\mathbf{A}_0) \leq d_{max}$. Alors, P est injectif dans le semi-anneau des forêts finies de profondeur au plus d_{max} .

Démonstration. Nous supposons, tout d'abord, que P n'a pas de terme constant. Soient $\mathbf{X}$ et $\mathbf{Y}$ deux forêts finies ayant profondeur au plus d_{max} . Nous supposons que

2. Résolutions d'équations polynomiales sur les unrolls – 2.4. Complexité de la résolution des équations polynomiales sur les unrolls

$P(\mathbf{X}) = P(\mathbf{Y})$. Soit h la profondeur de $P(\mathbf{X})$. Des hypothèses, il suit que $h \leq d_{\max}$. En effet, aucun arbre de $\mathbf{X}, \mathbf{Y}$ et des coefficients de P n'est plus profond que $d_{\max}$. Et puisque l'un des coefficients de P contient un arbre de profondeur $d_{\max}$, il suit que $h = d(\mathbf{X}) = d(\mathbf{Y})$. Ainsi, $\mathbf{X}$ et $\mathbf{Y}$ respectent la condition de profondeur du Lemme 2.50.

Par le Lemme 2.50, il existe $i, j \in \{1, \dots, m\}$, avec $i \leq j$ tels que, le plus petit arbre avec facteur $\mathbf{x}_k$ (resp., $\mathbf{y}_k$) dans $\Gamma_d(\sum_{i=1}^m \mathbf{A}_i \mathbf{X}^i)$ est égal à $\mathbf{a}_{i,1} \mathbf{x}_1^{i-1} \mathbf{x}_k$ (resp., $\mathbf{a}_{j,1} \mathbf{y}_1^{j-1} \mathbf{y}_k$), avec $\mathbf{a}_{i,1} = \min(\Gamma_h(\mathbf{A}_i))$ et $\mathbf{a}_{j,1} = \min(\Gamma_h(\mathbf{A}_j))$. Des hypothèses, nous déduisons que $\mathbf{a}_{i,1} \mathbf{x}_1^i = \min(\Gamma_h(P(\mathbf{X}))) = \min(\Gamma_h(P(\mathbf{Y}))) = \mathbf{a}_{j,1} \mathbf{y}_1^j$.

Nous montrons maintenant que $\mathbf{x}_1 = \mathbf{y}_1$. Par l'absurde, et sans perte de généralité, nous supposons que $\mathbf{x}_1 < \mathbf{y}_1$. Alors, il suit que $\mathbf{x}_1^j < \mathbf{y}_1^j$. Comme l'ordre est compatible avec le produit, nous déduisons que $\mathbf{a}_{j,1} \mathbf{x}_1^j < \mathbf{a}_{j,1} \mathbf{y}_1^j = \mathbf{a}_{i,1} \mathbf{x}_1^i$. Néanmoins, $\mathbf{a}_{j,1} \mathbf{x}_1^j$ est un arbre de $\Gamma_h(P(\mathbf{X}))$, ce qui est en contradiction avec la minimalité de $\mathbf{a}_{i,1} \mathbf{x}_1^i$.

Finalement, comme $\mathbf{x}_1 = \mathbf{y}_1$, nous déduisons que $P(\mathbf{X}) - P(\mathbf{x}_1) = P(\mathbf{Y}) - P(\mathbf{y}_1)$. Si $P(\mathbf{X}) - P(\mathbf{x}_1)$ contient encore un arbre de profondeur h , il suit que $\mathbf{X}$ et $\mathbf{Y}$ contiennent au moins deux arbres de profondeur h . En effet, si ce n'est pas le cas, nous avons que tous les arbres de $\mathbf{X} - \mathbf{x}_1$ et de $\mathbf{Y} - \mathbf{y}_1$ ont une profondeur strictement inférieure à h . Et donc, comme au moins un de ces arbres apparaît dans chaque produit de $P(\mathbf{X}) - P(\mathbf{x}_1)$ et de $P(\mathbf{Y}) - P(\mathbf{y}_1)$, il suit que ces deux forêts ne contiennent que des arbres de profondeur strictement inférieure à h .

Supposons donc que $P(\mathbf{X}) - P(\mathbf{x}_1)$ contient un arbre de profondeur h . Alors, par le Lemme 2.50 et puisque l'ordre est compatible avec le produit, il suit que le plus petit arbre de $\Gamma_h(P(\mathbf{X}) - P(\mathbf{x}_1))$ est $\mathbf{a}_{i,1} \mathbf{x}_1^{i-1} \mathbf{x}_2$ avec $\mathbf{x}_2$ le plus petit arbre de $\Gamma_h(\mathbf{X} - \mathbf{x}_1)$. De même, le plus petit arbre de $\Gamma_h(P(\mathbf{Y}) - P(\mathbf{y}_1))$ est $\mathbf{a}_{j,1} \mathbf{y}_1^{j-1} \mathbf{y}_2$ avec $\mathbf{y}_2$ le plus petit arbre de $\Gamma_h(\mathbf{Y} - \mathbf{y}_1)$. Et donc $\mathbf{a}_{i,1} \mathbf{x}_1^{i-1} \mathbf{x}_2 = \mathbf{a}_{j,1} \mathbf{y}_1^{j-1} \mathbf{y}_2$.

Maintenant, comme $\mathbf{x}_1 = \mathbf{y}_1$, il suit que $\mathbf{a}_{i,1} \mathbf{x}_1^i = \mathbf{a}_{j,1} \mathbf{x}_1^j$ et le Lemme 1.57 induit que

$$\mathcal{C}_h(\mathbf{a}_{i,1} \mathbf{x}_1^{i-1}) = \mathcal{C}_h(\mathbf{a}_{j,1} \mathbf{x}_1^{j-1}).$$

Et donc puisque l'ordre est compatible avec le produit, nous inférons que

$$\mathcal{C}_h(\mathbf{a}_{i,1} \mathbf{x}_1^{i-1}) \mathbf{x}_2 = \mathbf{a}_{i,1} \mathbf{x}_1^{i-1} \mathbf{x}_2 = \mathbf{a}_{j,1} \mathbf{x}_1^{j-1} \mathbf{x}_2 = \mathcal{C}_h(\mathbf{a}_{j,1} \mathbf{x}_1^{j-1}) \mathbf{x}_2.$$

Par conséquent, $\mathbf{a}_{j,1} \mathbf{x}_1^{j-1} \mathbf{x}_2 = \mathbf{a}_{j,1} \mathbf{y}_1^{j-1} \mathbf{y}_2$ et comme $\mathbf{a}_{j,1} \mathbf{x}_1^{j-1}$ et $\mathbf{a}_{j,1} \mathbf{y}_1^{j-1}$ ont une profondeur au moins h , nous déduisons du Lemme 1.57 que $\mathcal{C}_h(\mathbf{x}_2) = \mathcal{C}_h(\mathbf{y}_2)$. Et de ce fait, puisque $\mathbf{x}_2$ et $\mathbf{y}_2$ ont une profondeur d'au plus h , nous concluons que $\mathbf{x}_2 = \mathbf{y}_2$.

Par récurrence, nous concluons que $\Gamma_h(\mathbf{X}) = \Gamma_h(\mathbf{Y})$. Nous pouvons répéter récursivement ce raisonnement sur $\mathbf{X} - \Gamma_h(\mathbf{X})$ et $\mathbf{Y} - \Gamma_h(\mathbf{Y})$, ce qui nous permet de conclure que $\mathbf{X} = \mathbf{Y}$.

Cette preuve s'étend aux polynômes avec termes constants $\mathbf{A}_0$. En effet, si maintenant $P = Q + \mathbf{A}_0$ avec Q un polynôme de forêts sans terme constant, alors $P(\mathbf{X}) = P(\mathbf{Y})$ induit que $Q(\mathbf{X}) = Q(\mathbf{Y})$. Mais comme $Q(\mathbf{X}) = Q(\mathbf{Y})$, cela implique que $\mathbf{X} = \mathbf{Y}$. Et de ce fait, P est aussi injectif. $\square$

2. Résolutions d'équations polynomiales sur les unrolls – 2.4. Complexité de la résolution des équations polynomiales sur les unrolls

Ceci nous permet de montrer le théorème suivant directement à partir de la Proposition 2.51.

Théorème 2.52. *Tous les polynômes sur les unrolls sont injectifs.*

Démonstration. Supposons que $P(\mathcal{U}(X)) = P(\mathcal{U}(Y))$. Alors, pour tout entier $n \geq 0$, il suit que $\mathcal{C}_n(P(\mathcal{U}(X))) = \mathcal{C}_n(P(\mathcal{U}(Y)))$. Si $\mathcal{U}(X)$ et $\mathcal{U}(Y)$ sont différents, il devrait exister un entier n tel que $\mathcal{C}_n(\mathcal{U}(X)) \neq \mathcal{C}_n(\mathcal{U}(Y))$. Cependant, par la Proposition 2.51 et puisque $\mathcal{C}_n(X)$ sont des morphismes, nous concluons que $\mathcal{C}_n(\mathcal{U}(X)) = \mathcal{C}_n(\mathcal{U}(Y))$ pour tout entier $n \geq 0$. $\square$

La preuve de la Proposition 2.51 suggère aussi un algorithme (inspiré de l'algorithme présenté dans la Section 2.3 pour la résolution des équations $\mathcal{U}(A)\mathcal{U}(X)^k = \mathcal{U}(B)$) capable de résoudre les équations polynomiales de forêts finies. En commençant avec la profondeur maximale $d_{\max}$, nous pouvons résoudre l'équation $\Gamma_{d_{\max}}(P(\mathbf{X})) = \Gamma_{d_{\max}}(\mathbf{B})$. Dans les faits, s'il existe une solution $\mathbf{X}$, nous la trouverons en construisant d'abord $\Gamma_{d_{\max}}(\mathbf{X})$. Pour ce faire, nous résolvons l'équation $\min(\Gamma_{d_{\max}}(\mathbf{A}_i))\mathbf{x}_1^i = \min(\Gamma_{d_{\max}}(\mathbf{B}))$ pour un certain $i \in \{1, \dots, m\}$. Cela peut être réalisé en divisant $\min(\Gamma_{d_{\max}}(\mathbf{B}))$ par $\min(\Gamma_{d_{\max}}(\mathbf{A}_i))$ via l'Algorithme 1.76 puis en calculant la racine i -ème dudit quotient via l'Algorithme 2.36.

Nous obtenons ainsi l'arbre $\mathbf{x}_1$ qui est le plus petit arbre de $\Gamma_{d_{\max}}(\mathbf{X})$. Autrement dit, le plus petit arbre de la solution étant de profondeur maximale. Ainsi, par le Lemme 2.50, nous pouvons récursivement reconstruire l'ensemble des arbres de $\Gamma_{d_{\max}}(\mathbf{X})$. En effet, le j -ème arbre $\mathbf{x}_j$ de cette forêt est égal à :

$$\frac{\min(\Gamma_{d_{\max}}(\mathbf{B}) - \Gamma_{d_{\max}}(P(\mathbf{x}_1 + \dots + \mathbf{x}_{j-1})))}{\min(\Gamma_{d_{\max}}(\mathbf{A}_i))\mathbf{x}_1^{i-1}}.$$

De là, il nous faut encore trouver l'ensemble des arbres de $\mathbf{X}$ dont la profondeur est inférieure à $d_{\max}$. Pour cela, nous bâtirons récursivement $\Gamma_d(\mathbf{X})$ depuis $\Gamma_{d'}(\mathbf{X})$ où $d < d'$ est la profondeur d'un arbre de $\mathbf{B}$ et $d' \leq d_{\max}$ est la plus petite profondeur d'un arbre de $\mathbf{B}$ strictement supérieure à d . Ceci peut être réalisé en trouvant $\mathbf{x} = \min(\Gamma_d(\mathbf{X}))$. Et pour faire cela, nous considérons trois arbres :

1. $\mathbf{x}_m = \min(\Gamma_{d'}(\mathbf{X}))$ le plus petit arbre de $\mathbf{X}$ dont la profondeur est au moins d' ,
2. $\mathbf{t} = \min(\Gamma_d(P(\mathbf{x}_m)))$ le plus petit arbre de $P(\mathbf{x}_m)$ dont la profondeur est au moins d , et
3. $\mathbf{b} = \min(\Gamma_d(\mathbf{B}))$ le plus petit arbre de $\mathbf{B}$ dont la profondeur est au moins d .

Deux cas sont alors possibles. Soit $\mathbf{t} = \mathbf{b}$ et donc $\mathbf{x}_m = \mathbf{x}$. Sinon, il existe un coefficient $\mathbf{A}_j$ tel que $\mathbf{a}\mathbf{x}^j = \mathbf{b}$ avec $\mathbf{a} = \min(\Gamma_d(\mathbf{A}_j))$. Donc pour trouver $\mathbf{x}$, il suffit de calculer $\sqrt[j]{\mathbf{b}/\mathbf{a}}$.

À ce point, nous pouvons retrouver les arbres de $\mathbf{X}$ qui ont profondeur d de la même manière que pour $d_{\max}$, c'est-à-dire en calculant :

$$\frac{\min(\Gamma_d(\mathbf{B}) - \Gamma_d(P(\mathbf{X})))}{\mathbf{a}\mathbf{x}^{j-1}}.$$

2. Résolutions d'équations polynomiales sur les unrolls – 2.4. Complexité de la résolution des équations polynomiales sur les unrolls

Nous voyons directement que toutes les opérations effectuées dans cet algorithme peuvent être réalisées en temps polynomial. De plus, comme le nombre d'itérations est borné par la profondeur de $\mathbf{B}$, nous déduisons que cette procédure est réalisable en temps polynomial si nous arrivons à trouver un coefficient correct en temps polynomial. En effet, la Proposition 2.51 garantit l'existence mais ne précise pas quel coefficient doit être utilisé à chaque profondeur. Pour pallier ce problème, nous introduisons le lemme suivant.

Lemme 2.53. *Soit $P = \sum_{i=1}^m \mathbf{A}_i \mathbf{X}^i$ un polynôme de forêts finies sans terme constant, soit $\mathbf{B}$ une forêt finie et soit d la profondeur d'un arbre de $\mathbf{B}$. S'il existe $\mathbf{X}$ de profondeur au plus $d(\mathbf{B})$ telle que $P(\mathbf{X}) = \mathbf{B}$ alors le plus petit arbre $\mathbf{x}$ de $\Gamma_d(\mathbf{X})$ est égal à $\sqrt[k]{\mathbf{b}/\mathbf{a}}$ avec $\mathbf{b} = \min(\Gamma_{d'}(\mathbf{B}))$, $\mathbf{a} = \min(\Gamma_{d'}(\mathbf{A}_k))$, d' la profondeur de $\mathbf{x}$ et $k \geq 1$ le plus grand entier possible vérifiant les deux conditions suivantes :*

1. *le résultat de $\sqrt[k]{\mathbf{b}/\mathbf{a}}$ est bien défini,*
2. *$P(\mathbf{x})$ est un sous-multi-ensemble de $\mathbf{B}$.*

Démonstration. Supposons qu'il existe une forêt $\mathbf{X}$ de profondeur au plus $d(\mathbf{B})$ telle que $P(\mathbf{X}) = \mathbf{B}$. Soit $\mathbf{x} = \min(\Gamma_d(\mathbf{X}))$, soit d' sa profondeur, soient $\mathbf{a}, \mathbf{b}$ comme en énoncé et soit $\mathbf{t}$ le résultat de $\sqrt[k]{\mathbf{b}/\mathbf{a}}$. Montrons que $\mathbf{x} = \mathbf{t}$.

Par la minimalité de $\mathbf{b}$ et par le Lemme 2.50, il existe un entier r tel que $\mathbf{x} = \sqrt[r]{\mathbf{b}/\mathbf{a}'}$ avec $\mathbf{a}' = \min(\Gamma_{d'}(\mathbf{A}_r))$. Or, nous savons alors que $\mathbf{a}'\mathbf{x}^r = \mathbf{b}$ et que $\mathbf{a}\mathbf{t}^k = \mathbf{b}$. Ceci induit que la profondeur de $\mathbf{t}$ est au moins d' .

Supposons, par l'absurde, que $\mathbf{t} \neq \mathbf{x}$. Donc $k \leq r$ par hypothèse sur k . Seulement, si $k = r$ alors $\mathbf{a} = \mathbf{a}'$ et puisque $d(\mathbf{t}) = d(\mathbf{a}') = d(\mathbf{x})$, le Lemme 1.57 et l'injectivité des racines impliquent que $\mathbf{t} = \mathbf{x}$. Donc $k < r$. Cela implique que $\mathbf{A}_k$ et $\mathbf{A}_r$ contiennent chacun au moins un arbre de profondeur au moins d' . Donc $\mathbf{B}$ possède au moins quatre arbres de profondeur au moins d . En effet, comme d' est la profondeur de $\mathbf{x}$ qui est un arbre de $\Gamma_d(\mathbf{X})$, il suit que $d \leq d'$.

De là, par minimalité de $\mathbf{x}_r = \mathbf{a}'\mathbf{x}^r$ et puisque $\mathbf{t}_r = \mathbf{a}'\mathbf{t}^r$ est un arbre de $\Gamma_d(\mathbf{B})$, dû à la condition 2, il suit que $\mathbf{t}_r \geq \mathbf{x}_r$. Et comme $\mathbf{t} \neq \mathbf{x}$, nous avons en particulier que $\mathbf{t}_r > \mathbf{x}_r$. Ainsi, comme l'ordre est compatible avec le produit, nous déduisons que $\mathbf{t}^r > \mathbf{x}^r$ et cela induit que $\mathbf{t} > \mathbf{x}$.

Or, $\mathbf{t}^k > \mathbf{x}^k$. Et étant donné que l'ordre est compatible avec le produit, $\mathbf{b} = \mathbf{a}\mathbf{t}^k > \mathbf{a}\mathbf{x}^k$. Ce qui entre en contradiction avec la minimalité de $\mathbf{b}$. $\square$

Ainsi, comme trouver un coefficient qui respecte les deux conditions de l'énoncé du Lemme 2.53 peut se faire en temps polynomial (via une simple boucle sur les coefficients du polynôme), nous concluons que notre algorithme est bien polynomial. Ce dernier est résumé dans l'Algorithme 2.55. Il prendra en entrées le polynôme P sur les forêts ainsi que B le côté droit de l'équation. Un exemple d'exécution de cet algorithme est présenté dans la Figure 2.54.

Pour appliquer cette approche aux unrolls, nous étendons le Corollaire 2.43 pour montrer que nous pouvons résoudre les équations sur les unrolls via celles sur les forêts finies.

$$\begin{aligned}
 P &= \text{diagram}_1 X + \text{diagram}_2 X^2 \\
 \mathbf{B} &= 2 \text{diagram}_3 + 4 \text{diagram}_4 + 2 \text{diagram}_5 + 2 \text{diagram}_6 + 3 \text{diagram}_7 + 2 \text{diagram}_8 + \text{diagram}_9 \\
 \hline
 \Gamma_2(P) &= \text{diagram}_1 X + \text{diagram}_2 X^2 \\
 \Gamma_2(\mathbf{B}) &= 2 \text{diagram}_3 + 2 \text{diagram}_4 \\
 \mathbf{X} &= \sqrt{\frac{\mathbf{b}}{\mathbf{a}}} = \text{diagram}_5 \\
 \mathbf{B} - P(\mathbf{X}) &= \text{diagram}_3 + 3 \text{diagram}_4 + \text{diagram}_5 + \text{diagram}_6 + 3 \text{diagram}_7 + 2 \text{diagram}_8 + \text{diagram}_9 \\
 \hline
 \Gamma_2(P) &= \text{diagram}_1 X + \text{diagram}_2 X^2 \\
 \Gamma_2(\mathbf{B}) &= \text{diagram}_3 + \text{diagram}_4 \\
 \mathbf{X} &= \mathbf{X} + \frac{\mathbf{b}}{\mathbf{a} \min(\mathbf{X})} = 2 \text{diagram}_5 \\
 \mathbf{B} - P(\mathbf{X}) &= 2 \text{diagram}_3 + 3 \text{diagram}_4 + 2 \text{diagram}_5 + \text{diagram}_9 \\
 \hline
 \Gamma_1(P) &= \text{diagram}_1 X + \text{diagram}_2 X^2 \\
 \Gamma_1(\mathbf{B} - P(\mathbf{X})) &= 3 \text{diagram}_3 + 3 \text{diagram}_4 + 2 \text{diagram}_5 + \text{diagram}_9 \\
 \mathbf{X} &= \mathbf{X} + \sqrt{\frac{\mathbf{b}}{\mathbf{a}}} = 2 \text{diagram}_5 + \text{diagram}_6 \\
 \mathbf{B} - P(\mathbf{X}) &= \emptyset
 \end{aligned}$$

FIGURE 2.54. – Exemple d'exécution de l'Algorithme 2.55 prenant en entrées le P et le $\mathbf{B}$ du haut. Chaque partie séparée par une ligne en pointillé représente une itération de la boucle.

2. Résolutions d'équations polynomiales sur les unrolls – 2.4. Complexité de la résolution des équations polynomiales sur les unrolls

Algorithme 2.55 SolveurPolyForêt($P, \mathbf{B}$)

```

1:  $\mathbf{x}_1 \leftarrow \emptyset, \mathbf{a}_1 \leftarrow \emptyset, \mathbf{X} \leftarrow \emptyset, r \leftarrow m$  avec  $m$  le degré de  $P$ ,
2: soit  $d$  la profondeur de  $B$ ,
3: Tant que  $B - P(\mathbf{X})$  est non vide faire
4:    $\mathbf{b} \leftarrow \min(\Gamma_d(\mathbf{B} - P(\mathbf{X})))$ ,
5:   Si  $\mathbf{x}_1 = \emptyset$  ou  $\mathbf{a}_1 \mathbf{x}_1^r > \mathbf{b}$  alors
6:     Soit  $i > 0$  le plus petit exposant dans  $P$ ,
7:      $\mathbf{t} \leftarrow \emptyset, \mathbf{x}_1 \leftarrow 1$ ,
8:     Tant que  $i \leq m$  et  $\mathbf{t} \neq \mathbf{x}_1$  faire
9:        $\mathbf{a} \leftarrow \min(\Gamma_d(\mathbf{A}_i))$ ,
10:       $\mathbf{t} \leftarrow \sqrt[i]{\mathbf{b}/\mathbf{a}}$ ,
11:      Si  $\mathbf{t}$  est défini et que  $P(\mathbf{t})$  est un sous-multi-ensemble de  $\mathbf{B}$  alors
12:         $\mathbf{x}_1 \leftarrow \mathbf{t}, \mathbf{a}_1 \leftarrow \mathbf{a}, r \leftarrow i$ ,
13:         $i \leftarrow$  le plus petit exposant de  $P$  strictement supérieur à  $i$ ,
14:      Si  $\mathbf{t} = \emptyset$  alors
15:        renvoyer  $\perp$ ,
16:       $\mathbf{x} \leftarrow \mathbf{x}_1$ ,
17:    Sinon
18:       $\mathbf{x} \leftarrow \frac{\mathbf{b}}{\mathbf{a}_1 \mathbf{x}_1^{r-1}}$ ,
19:      Si  $\mathbf{x}$  n'est pas défini alors
20:        renvoyer  $\perp$ ,
21:       $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{x}$ ,
22:      Si  $P(\mathbf{X})$  n'est pas un sous-multi-ensemble de  $\mathbf{B}$  alors
23:        renvoyer  $\perp$ ,
24:    renvoyer  $\mathbf{X}$ .

```

Algorithme 2.56 SolveurPolyUnroll(P, B)

```

1:  $n \leftarrow 2\alpha^2 + d(B)$  avec  $\alpha$  le nombre d'arbres de  $\mathcal{U}(B)$ ,
2:  $\mathbf{B} \leftarrow \mathcal{C}_n(\mathcal{U}(B))$  en associant à chaque arbre de  $\mathbf{B}$  la longueur de son cycle d'origine,
3: construire  $P' \leftarrow \sum_{i=0}^m \mathcal{C}_n(\mathcal{U}(A_i)) X^i$  où les  $A_i$  sont les coefficients de  $P$ ,
4:  $\mathbf{a} \leftarrow \min(\sum_{i=1}^m \mathcal{C}_n(\mathcal{U}(A_i)))$ ,
5:  $\mathbf{X} \leftarrow \text{SolveurPolyForêt}(P', \mathbf{B})$  (Algorithme 2.55),
6: Tant que  $\mathbf{X}$  n'est pas vide faire
7:   construire  $w_{\mathbf{x}}$  un motif périodique de  $\mathbf{x} = \min(\mathbf{X})$  de longueur  $b$  où  $b$  est la
   longueur associée à  $\mathbf{a}\mathbf{x}^k$ ,
8:   trouver  $p$  la plus petite période du mot  $w_{\mathbf{x}}$ ,
9:   construire  $D$  l'enroulement de  $\mathbf{x}$  à période  $p$ ,
10:   $X \leftarrow X + D$ ,
11:   $\mathbf{X} \leftarrow \mathbf{X} - \mathcal{C}_n(\mathcal{U}(D))$ ,
12: renvoyer  $X$ .

```

Proposition 2.57. Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS, soit B un FDDS et α le nombre d'arbres dans $\mathcal{U}(B)$. Soit $n \geq 2 \cdot \alpha^2 + d(B)$ un entier. Alors, il existe une forêt finie $\mathbf{X}$ de profondeur n , telle que $\sum_{i=0}^m \mathcal{C}_n(\mathcal{U}(A_i)) \mathbf{X}^i = \mathcal{C}_n(\mathcal{U}(B))$ si et seulement s'il existe un FDDS X tel que $\mathcal{U}(P(X)) = \mathcal{U}(B)$. De plus, le cas échéant $\mathcal{C}_n(\mathcal{U}(X)) = \mathbf{X}$.

Démonstration. Nous supposons que $\sum_{i=0}^m \mathcal{C}_n(\mathcal{U}(A_i)) \mathbf{X}^i = \mathcal{C}_n(\mathcal{U}(B))$. Tout d'abord, notons que nous pouvons appliquer un raisonnement similaire à celui de la preuve du Lemme 2.22 afin de montrer que s'il existe une forêt $\mathbf{X}$ telle que $\sum_{i=0}^m \mathcal{U}(A_i) \mathbf{X}^i = \mathcal{U}(B)$ alors il existe un FDDS X vérifiant que $\mathcal{U}(X) = \mathbf{X}$.

Puisque α borne la plus grande longueur de cycle de $A_0, \dots, A_m, B$ et comme, d'après les hypothèses, $d(B)$ borne $d(A_i)$ pour chaque i , nous disposons de tous les outils nécessaires pour appliquer un raisonnement similaire à celui du Théorème 2.23. Et ainsi, il existe un FDDS X tel que $\mathcal{U}(P(X)) = \mathcal{U}(B)$. La réciproque est directe. $\square$

Cette proposition a deux conséquences. La première est que la solution de l'équation $\sum_{i=0}^m \mathcal{U}(A_i) \mathbf{X}^i = \mathcal{U}(B)$ est toujours un unroll (si elle existe). En outre, à partir de la solution $\mathbf{X}$ renvoyée par l'algorithme sur l'entrée $\sum_{i=0}^m \mathcal{C}_n(\mathcal{U}(A_i)) \mathbf{X}^i = \mathcal{C}_n(\mathcal{U}(B))$, avec un n suffisamment grand, nous pouvons reconstruire $\mathcal{U}(X)$ en employant la méthode du Théorème 2.29. Nous en déduisons bien une procédure polynomiale (Algorithme 2.56) pour la résolution d'équations sur les unrolls, ces derniers étant codés par des FDDS ayant ces unrolls.

Théorème 2.58. Soient P un polynôme d'unroll (encodés par des FDDS) et B un FDDS. Alors, nous pouvons résoudre $P(\mathcal{U}(X)) = \mathcal{U}(B)$ en temps polynomial.

2.5. Conclusion et perspectives

Dans ce chapitre, nous avons présenté un algorithme qui, étant donné $P = \sum_{i=0}^m A_i X^i$ sur les FDDS et B un FDDS, produit en temps polynomial un FDDS X tel que $P(X) = B$, si un tel X existe. Pour cela, nous sommes passés par plusieurs étapes, à savoir la résolution des équations de la forme $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ avec un X connexe puis $\mathcal{U}(A)\mathcal{U}(X) = \mathcal{U}(B)$ sans condition sur X et enfin $\mathcal{U}(X)^k = \mathcal{U}(B)$ avec $k \geq 0$.

Pour montrer tous ces résultats, nous avons développé les outils permettant de résoudre des équations sur les unrolls à partir d'équations sur leur troncature. Ceci nous a permis d'utiliser des procédures de résolution sur les arbres finis.

Nous pouvons concevoir diverses extensions de ces résultats nous permettant de formuler plusieurs questions. La première et la plus intuitive est de passer des équations de la forme $P(X) = \mathcal{U}(B)$ à celles de la forme $Q(X_1, \dots, X_n) = \mathcal{U}(B)$ avec Q un polynôme de $n > 1$ variables (le cas $n = 1$ ayant été l'objet de ce chapitre). Si les outils développés dans ce chapitre devraient permettre de montrer que ce problème est dans la classe NP (il n'y est pas directement car les unrolls sont infinis), nous ne saurions en déduire ni la NP-complétude ni l'appartenance à la classe P.

Une question corrélée serait de déterminer le temps de calcul suffisant à résoudre les systèmes d'équations sur les unrolls avec membre droit constant, c'est-à-dire les

systèmes de la forme :

$$\left\{ \begin{array}{l} P_1(X) = \mathcal{U}(B_1) \\ P_2(X) = \mathcal{U}(B_2) \\ \vdots \\ P_n(X) = \mathcal{U}(B_n) \end{array} \right.$$

Une autre question est de savoir si décider si $P(X) = \mathcal{U}(B)$ admet une solution est complet pour la classe P. En effet, si de prime abord, il paraît difficile, ne serait-ce que de diviser deux unrolls sans un espace polynomial, nous pourrions faire la même réflexion pour comparer deux arbres. Cependant, Lindell a démontré dans [33] qu'il est possible de comparer deux arbres (avec un ordre différent du nôtre) en espace logarithmique. Il est donc envisageable que nous puissions adapter son algorithme pour comparer deux arbres avec notre ordre en espace logarithmique. Puis d'utiliser ce nouvel algorithme pour diviser deux arbres en espace logarithmique.

Enfin, nous pouvons nous demander si au lieu de ne produire en temps polynomial qu'un seul FDDS solution à $P(\mathcal{U}(X)) = \mathcal{U}(B)$, nous ne pourrions énumérer l'ensemble des FDDS X' possibles tels que $\mathcal{U}(X') = \mathcal{U}(X)$ soit en temps polynomial sur la taille de B , si le nombre de tels X' est polynomial sur cette même taille, soit avec un délai polynomial sur la taille de B si le nombre de tels X' est super-polynomial sur cette même taille.

3. Résolution d'équations polynomiales de FDDS

Sommaire

3.1. Le cas où le polynôme est injectif	103
3.1.1. Le cas des racines	103
3.1.2. Le cas des équations $P(X) = B$ où un coefficient non-constant de P contient un dendron	109
3.1.3. Conditions nécessaires à l'injectivité des polynômes de FDDS	111
3.2. Le cas des polynômes "pseudo-injectifs"	114
3.2.1. Résoudre $P(X) = B$	115
3.2.2. Équation sur les permutations encodée par des entiers	127
3.3. Conclusion et perspectives	133

Afin de résoudre les équations polynomiales sur les FDDS avec côté droit constant, *i.e.* les équations de la forme $P(X) = B$ avec P un polynôme sur les FDDS et B un FDDS, nous serions tentés de recourir à une stratégie similaire à celle utilisée dans le Chapitre 2. C'est-à-dire que nous souhaiterions définir un ordre total sur les FDDS étant compatible avec le produit et utiliser ce dernier pour reconstruire X composante par composante. Cependant, cette approche est impossible.

Lemme 3.1. *Il n'existe pas d'ordre total sur les FDDS étant compatible avec le produit direct.*

Démonstration. Supposons par l'absurde qu'il existe $\leq$ un ordre total sur les FDDS compatible avec le produit. De là, comme $C_2 \times C_2 = 2 \times C_2$ avec $2 = C_1 + C_1$, nous avons que $C_2 \times C_2 \leq 2 \times C_2$ et $C_2 \times C_2 \geq 2 \times C_2$. Ainsi, comme $\leq$ est compatible avec le produit, il suit que $C_2 \leq 2$ et de même $C_2 \geq 2$. Ceci induit que $C_2 = 2$. Contradiction. $\square$

Nous devons donc trouver une approche alternative. Dans ce chapitre, nous expliquerons comment exploiter les résultats du Chapitre 2 afin de résoudre en temps polynomial les équations de la forme $P(X) = B$ avec B un FDDS et P un polynôme injectif (Section 3.1). Nous étendrons par la suite ce résultat aux équations $P(X) = B$ avec P un polynôme "pseudo-injectif" (Section 3.2).

Pour cela, nous cherchons pour chaque arbre de $\mathcal{U}(X)$ une période correcte pour l'enroulement. À cette fin, nous prouverons que nous sommes en mesure de trouver le plus petit arbre de chaque composante de X si elles sont triées selon un certain ordre

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

(Lemme 3.8 et 3.14). De ce fait, si nous sommes en mesure de développer un algorithme polynomial produisant les longueurs de cycle des composantes connexes de X une par une et triées selon un certain ordre, alors il nous est possible de reconstruire les composantes connexes de X en temps polynomial.

3.1. Le cas où le polynôme est injectif

Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS, c'est-à-dire que X et tous les A_i sont des FDDS. Dans cette section, nous fournirons un algorithme polynomial capable de résoudre les équations de la forme $X^k = B$ avec B un FDDS et $k \geq 0$ un entier. Puis nous généraliserons ce résultat aux équations de la forme $P(X) = B$ avec P un polynôme injectif, c'est-à-dire qu'au moins l'un des coefficients (un A_i) non-constants ($i \neq 0$) de P contient un dendron. Nous apporterons par la même occasion une preuve alternative du Théorème 1.70 dans le cas restreint des éléments de $\mathbb{D}$. Comme la propriété précédente n'est qu'une condition suffisante à l'injectivité, nous concluons cette section par la preuve qu'elle est aussi nécessaire pour fournir une caractérisation complète des polynômes injectifs.

3.1.1. Le cas des racines

Afin de produire un algorithme efficace pour résoudre les équations de la forme $X^k = B$ avec $k > 1$ un entier, nous allons expliquer comment retrouver les composantes connexes de X à partir de celles de B . Nous nous restreignons au cas où $k > 1$ car pour le cas où $k = 0$, il suffit de vérifier que B est le point fixe et alors n'importe quel X est solution. Et si maintenant $k = 1$, la question est triviale : B est toujours la solution.

Par le Théorème 2.58, nous pouvons trouver $\mathcal{U}(X)$ en temps polynomial à partir de B et $k > 1$. Il nous faut donc encore chercher à quelle période enrouler les arbres de $\mathcal{U}(X)$. Pour ce faire, nous devons trouver les longueurs des cycles des composantes connexes de la solution, si elle existe. C'est pourquoi nous introduisons deux notations qui nous permettent de manipuler simplement les différentes longueurs de cycles dans X . Tout d'abord, nous noterons $L(X)$ l'ensemble des longueurs de cycle des composantes connexes de X et $\ell(X)$ la plus petite de ces longueurs. En symbole, $\ell(X) = \min(L(X))$. Remarquons que si X est connexe, alors $\ell(X)$ est la taille du cycle de X . Afin de construire les composantes connexes de X à partir de B , nous débutons notre étude par la structure de X^k et de B . Notre but est de trouver, à partir de certaines composantes connexes de B , quelles composantes connexes de X l'ont générée, *i.e.* la contiennent comme composante connexe de leur produit. Pour les identifier, nous introduisons les deux notations suivantes.

Soit A un FDDS et $p > 0$ un entier. Nous notons par $\lambda_p(A)$ le sous-multi-ensemble des composantes connexes de A ayant un cycle de taille p et par $\Lambda_p(A)$ le sous-multi-ensemble de A dont les cycles ont une longueur divisant p . De cette définition, nous déduisons le résultat suivant.

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

Lemme 3.2. Soit $p > 0$ un entier. Alors, $\Lambda_p(X)$ est un endomorphisme de $(\mathbb{D}, +, \times)$.

Démonstration. Tout d'abord, si nous considérons que $\mathbf{1}$ est le point fixe (le neutre de $(\mathbb{D}, \times)$), nous avons directement que $\Lambda_p(\mathbf{1}) = \mathbf{1}$.

Maintenant, considérons deux FDDS A et B . Directement, $\Lambda_p(A+B) = \Lambda_p(A) + \Lambda_p(B)$ car la somme est l'union disjointe. Il nous faut encore établir que la fonction $\Lambda_p(X)$ se distribue sur le produit, en symbole $\Lambda_p(AB) = \Lambda_p(A)\Lambda_p(B)$. Par la distributivité du produit sur la somme, il est suffisant de démontrer cette propriété dans le cas où A et B sont connexes.

Rappelons que du produit de deux FDDS connexes U et V résultent $\gcd(\ell(U), \ell(V))$ composantes connexes de longueur de cycle $\text{lcm}(\ell(U), \ell(V))$. Ainsi, tous les cycles de AB sont de même longueur. Donc soit $\Lambda_p(AB) = AB$ si $\text{lcm}(\ell(A), \ell(B))$ divise p ou bien $\Lambda_p(AB) = \emptyset$ dans le cas contraire. Rappelons aussi que $\text{lcm}(\ell(A), \ell(B))$ divise p si et seulement si $\ell(A)$ et $\ell(B)$ divisent p . Pour des raisons pédagogiques, nous redémontrerons cette propriété dans les paragraphes suivants. Cette affirmation permet de conclure le lemme car nous en déduisons que $\Lambda_p(AB) = AB$ si et seulement si $\Lambda_p(A) = A$ et $\Lambda_p(B) = B$.

Supposons dans un premier temps que $\text{lcm}(\ell(A), \ell(B))$ divise p . Comme a et b divisent $\text{lcm}(a, b)$ pour tous entiers $a, b > 0$, nous déduisons que $\ell(A)$ et $\ell(B)$ divisent aussi p par transitivité.

Supposons à l'inverse que $\ell(A)$ et $\ell(B)$ divisent p . Ainsi, $\Lambda_p(A) = A$ et $\Lambda_p(B) = B$. Considérons $\prod_{i=1}^{\infty} q_i^{a_i}$, $\prod_{i=1}^{\infty} q_i^{b_i}$ et $\prod_{i=1}^{\infty} q_i^{c_i}$ les décompositions en facteurs premiers de respectivement $\ell(A)$, $\ell(B)$ et p . Par définition, $u = \prod_{i=1}^{\infty} q_i^{u_i}$ divise $v = \prod_{i=1}^{\infty} q_i^{v_i}$ si et seulement si $u_i \leq v_i$ pour tout entier $i \geq 1$ et ce pour tous entiers $u, v > 0$. Donc nous avons que $a_i, b_i \leq c_i$ pour tout entier $i \geq 1$. Ceci induit que $\max(a_i, b_i) \leq c_i$ pour tout entier $i \geq 1$. Ainsi, le nombre $r = \prod_{i=1}^{\infty} q_i^{\max(a_i, b_i)}$ divise p . Or cela implique que $r = \text{lcm}(\ell(A), \ell(B))$. $\square$

Le Lemme 3.2 est un premier pas vers l'objectif d'acquérir une méthode pour reconstruire X à partir de B . En effet, il nous fournit une silhouette des longueurs de cycles présents dans X . Cependant, nous n'avons pas la possibilité d'en déduire précisément $L(X)$. Et conséquemment, nous ne connaissons pas les périodes auxquelles enrouler les arbres de $\mathcal{U}(X)$. Afin de recueillir des informations plus précises, nous introduisons l'ordre total suivant sur les composantes connexes.

Définition 3.3 (ordre $\leq_{ct}$). Soient A et B deux FDDS connexes et soient $\mathbf{a} = \min(\mathcal{U}(A))$ et $\mathbf{b} = \min(\mathcal{U}(B))$. Nous disons que $A \leq_{ct} B$ si et seulement si :

1. $\ell(A) < \ell(B)$,
2. $\ell(A) = \ell(B)$ et $\mathbf{a} \leq \mathbf{b}$.

Par convention, $\emptyset <_{ct} A$ et ce pour tous les FDDS connexe A .

Pour justifier que cette relation est antisymétrique, notons bien que si deux composantes connexes ont la même longueur de cycle et le même arbre d'unroll minimal, alors elles sont égales.

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

Pour l'utiliser dans un algorithme efficace, il nous faut montrer que la comparaison est réalisable en temps polynomial. Pour cela, remarquons que les plus petites périodes de $\mathbf{a}$ et $\mathbf{b}$ sont respectivement au plus $\ell(A)$ et $\ell(B)$. Donc, par le Lemme 2.4, il résulte que $\mathbf{a} \leq \mathbf{b}$ si et seulement si $\mathcal{C}_n(\mathbf{a}) \leq \mathcal{C}_n(\mathbf{b})$ avec $n = \ell(A) + \ell(B) + \max(d(A), d(B))$. En effet, dans l'énoncé du Lemme 2.4, il faut que $n \geq a + b - \gcd(a, b) + \max(d(A), d(B))$ avec a et b les périodes minimales des arbres $\mathbf{a}$ et $\mathbf{b}$. Ainsi comme $\ell(A) \geq a$ et $\ell(B) \geq b$, la valeur choisie pour n est suffisante. Ceci prouve le lemme suivant.

Lemme 3.4. *Soient A et B deux FDDS connexes. Nous sommes en mesure de vérifier que $A \leq_{ct} B$ en temps polynomial de la taille de A plus la taille de B .*

Dans ce qui suit, nous utiliserons l'ordre $\leq_{ct}$ afin de construire X composante connexe par composante connexe en respectant cet ordre. C'est-à-dire que si nous considérons X_i et X_{i+1} deux composantes connexes générées successivement, alors $X_i \leq_{ct} X_{i+1}$. Nous disons alors que nous construisons X trié selon $\leq_{ct}$. Pour ce faire, nous réaliserons des preuves où nous retirerons à X certains de ses sous-multi-ensembles de composantes connexes que nous appelons *préfixe*.

Définition 3.5 (préfixe et super-préfixe). Soit $n \geq 0$ un entier. Supposons maintenant que $A = A_1 + \dots + A_{m_A}$. Alors le *préfixe* de longueur i de A , noté $\pi_i(A)$ est soit $\emptyset$ si $i = 0$ ou soit A si $i \geq m_A$ ou soit le FDDS $A_1 + \dots + A_i$.

Supposons maintenant que $A = \sum_{i=1}^{m_A} a_i A_i$ tel que chaque A_i est connexe et si $i \neq j$, alors $A_i \neq A_j$ pour tous i et j . Alors le *super-préfixe* de longueur i de A , noté $\mathcal{P}_i(A)$ est soit $\emptyset$ si $i = 0$ ou soit A si $i \geq m_A$ ou soit le FDDS $a_1 A_1 + \dots + a_i A_i$ si i est compris entre 1 et $m_A - 1$. Notons que ces deux notions sont fortement liées à l'ordre des composantes connexes dans A , c'est-à-dire que deux décompositions différentes de A en composantes connexes peuvent engendrer des préfixes et super-préfixes différents.

Lemme 3.6. *Soient $X = X_1 + \dots + X_{m_X}$ un FDDS trié selon $\leq_{ct}$ et soit $k > 0$ et soit $0 \leq i < m_X$ des entiers. Alors $\ell(X^k - \pi_i(X)^k) = \ell(X_{i+1})$.*

Démonstration. Rappelons que X^k est un multinôme des composantes connexes de X . C'est-à-dire que :

$$X^k = \sum_{k_1 + \dots + k_{m_X} = k} \binom{k}{k_1, \dots, k_{m_X}} \prod_{i=1}^{m_X} X_i^{k_i}$$

Par conséquent, pour toute composante connexe Y de $X^k - \pi_i(X)^k$, il existe des composantes connexes $X_{y_1}, \dots, X_{y_{k-1}}$ de X et X_{y_k} une composante de $X - \pi_i(X)$ telle que $\ell(Y) = \text{lcm}(\ell(X_{y_1}), \dots, \ell(X_{y_k}))$. Et comme $\text{lcm}(a_1, \dots, a_k) \geq \min(a_1, \dots, a_k)$ pour tous entiers $a_1, \dots, a_k > 0$, nous déduisons que $\ell(Y) \geq \ell(X_{y_k})$. Donc par la minimalité de $\ell(X_{i+1})$ dans $X - \pi_i(X)$, toutes les composantes connexes de $X^k - \pi_i(X)^k$ ont un cycle dont la taille est au moins $\ell(X_{i+1})$. En symbole, $\ell(X^k - \pi_i(X)^k) \geq \ell(X_{i+1})$.

Pour montrer l'inégalité inverse, il nous suffit de prouver que $X^k - \pi_i(X)^k$ contient une composante connexe dont le cycle est de taille $\ell(X_{i+1})$. À cette fin, remarquons

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

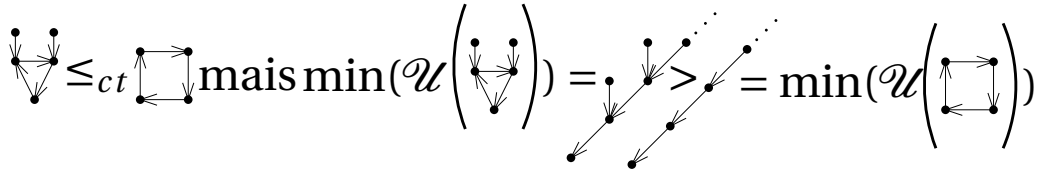


FIGURE 3.9. – Le FDDS ci-dessus a ses composantes connexes triées selon $\leq_{ct}$. Notons alors que sa plus petite composante connexe a un arbre d'unroll minimal supérieur à celui de sa plus grande composante connexe.

que X_{i+1}^k est un sous-multi-ensemble de $X^k - \pi_i(X)^k$. Et comme toutes les composantes connexes de X_{i+1}^k ont un cycle de la taille $\text{lcm}(\ell(X_{i+1}), \dots, \ell(X_{i+1})) = \ell(X_{i+1})$, le lemme est vérifié. $\square$

Corollaire 3.7. *Si X est une permutation, alors X est un sous-multi-ensemble de X^k .*

Notons que nous pouvons aussi montrer le corollaire précédent en utilisant la décomposition en multinôme de X^k et en exploitant le fait que la puissance $k > 0$ d'un cycle est constituée de quelques copies de ce cycle.

À présent, nous connaissons tout aussi bien $\mathcal{U}(X)$ que les longueurs de cycles de chaque composante connexe de X . Cependant, nous ne savons dire quel arbre de $\mathcal{U}(X)$ doit être enroulé à quelle période x de $L(X)$. En d'autres termes, si nous reprenons l'énoncé du Lemme 3.6, nous ne saurions dire ce qu'est $\mathcal{U}(X_{i+1})$. Ceci est le propos du lemme suivant.

Lemme 3.8. *Soit $X = X_1 + \dots + X_{m_X}$ un FDDS trié selon l'ordre $\leq_{ct}$. Soit $i \geq 0$ un entier. Alors, $\min(\mathcal{U}(X_{i+1}))$ est égal à l'arbre d'unroll minimal de $\Lambda_{\ell(X_{i+1})}(X - \pi_i(X))$.*

La figure 3.9 donne un exemple de pourquoi il est important de prendre le plus petit arbre de l'unroll de $\Lambda_{\ell(X_{i+1})}(X - \pi_i(X))$ et non juste le plus petit arbre de l'unroll de $X - \pi_i(X)$.

Démonstration. Soit $\mathbf{t}$ l'arbre d'unroll minimal de $\mathcal{U}(\Lambda_{\ell(X_{i+1})}(X - \pi_i(X)))$. Soit Y une composante connexe de $\Lambda_{\ell(X_{i+1})}(X - \pi_i(X))$ qui admet $\mathbf{t}$ comme arbre d'unroll. Montrons que $\mathbf{t} = \min(\mathcal{U}(X_{i+1}))$.

Notons que X_{i+1} est la plus petite composante connexe de $\Lambda_{\ell(X_{i+1})}(X - \pi_i(X))$ selon $\leq_{ct}$. Ainsi, $X_{i+1} \leq_{ct} Y$ et soit $\ell(X_{i+1}) < \ell(Y)$ ou bien $\ell(X_{i+1}) = \ell(Y)$ et alors $\min(\mathcal{U}(X_{i+1})) \leq \mathbf{t}$.

Or, $\Lambda_{\ell(X_{i+1})}(X - \pi_i(X))$ ne contient que des composantes connexes dont le cycle a longueur $\ell(X_{i+1})$ car $\pi_i(X)$ contient l'ensemble des composantes connexes de X ayant un cycle de longueur strictement inférieure à $\ell(X_{i+1})$. Donc $\ell(X_{i+1}) = \ell(Y)$. Et comme X_{i+1} est une composante connexe de $\Lambda_{\ell(X_{i+1})}(X - \pi_i(X))$, il résulte que $\mathbf{t} \leq \min(\mathcal{U}(X_{i+1}))$. Donc $\mathbf{t} = \min(\mathcal{U}(X_{i+1}))$. Et aussi $Y = X_{i+1}$. $\square$

Les Lemmes 3.6 et 3.8 permettent de conclure la proposition suivante.

Proposition 3.10. *Soient $X = X_1 + \dots + X_{m_X}$ un FDDS trié selon $\leq_{ct}$ et $k > 1, i \geq 0$ deux entiers. Soit D la plus petite composante connexe de $X^k - \pi_i(X)^k$ selon $\leq_{ct}$. Alors $X_{i+1} :$*

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

Algorithme 3.12 RacineFDDS(B, k)

```

1: Si  $k > \log_2 |B|$  alors
2:   renvoyer  $\perp$ ,
3:  $X \leftarrow \emptyset, n \leftarrow 2\alpha + d(B)$  avec  $\alpha$  le nombre d'arbres dans  $\mathcal{U}(B)$ ,
4: Tant que  $X^k \neq B$  faire
5:    $p \leftarrow \ell(B - X^k)$ ,
6:    $R \leftarrow \text{UnrollRacine}(\Lambda_p(B), k)$  (Algorithme 2.46),
7:   Si  $R$  n'est pas défini ou  $\mathcal{C}_n(\mathcal{U}(\Lambda_p(X)))$  n'est pas un sous-multi-ensemble de
      $\mathcal{C}_n(\mathcal{U}(R))$  alors
8:     renvoyer  $\perp$ ,
9:    $\mathbf{x} \leftarrow \min(\mathcal{C}_n(\mathcal{U}(R)) - \mathcal{C}_n(\mathcal{U}(\Lambda_p(X))))$ ,
10:  construire  $D$  l'enroulement de  $\mathbf{x}$  à période  $p$ ,
11:  Si  $\min(\mathcal{C}_n(\mathcal{U}(D))) \neq \mathbf{x}$  ou  $\mathcal{C}_n(\mathcal{U}(D))$  n'est pas un sous-multi-ensemble de
      $\mathcal{C}_n(\mathcal{U}(R)) - \mathcal{C}_n(\mathcal{U}(\Lambda_p(X)))$  alors
12:    renvoyer  $\perp$ ,
13:  ajouter  $D$  à  $X$ ,
14:  Si  $|X| > \sqrt[k]{|B|}$  ou  $X^k$  n'est pas un sous-multi-ensemble de  $B$  alors
15:    renvoyer  $\perp$ ,
16: renvoyer  $X$ .
```

1. a une longueur de cycle égale à $\ell(D)$,
2. son arbre d'unroll minimal est celui de $\Lambda_{\ell(D)}(X - \pi_i(X))$.

De cette proposition, nous déduisons l'Algorithme 3.12 capable de résoudre les équations de la forme $X^k = B$ avec $k > 1$ et B un FDDS. Ce dernier prend en entrée un FDDS B et un entier $k > 1$ et renvoie un FDDS X tel que $X^k = B$ s'il en existe un. Un exemple d'exécution de l'Algorithme 3.12 est donné en Figure 3.11.

Comme nous vérifions que $k \leq \log_2 |B|$ et nous ne réalisons X^k que si la taille de X est inférieure ou égale à $\sqrt[k]{|B|}$, nous déduisons que l'exponentiation naïve permet de calculer X^k en temps polynomial sur $|B|$, ce qui induit que cet algorithme est bien polynomial sur $|B|$ car toutes ces instructions sont polynomiales et que le nombre d'itérations de la boucle est au plus le nombre de composantes connexes dans B . Il nous faut encore montrer qu'il est correct.

Théorème 3.13. *Soit B un FDDS et $k \geq 0$ un entier. Le calcul de la racine k -ème de B (Algorithme 3.12) est polynomial sur la taille de B .*

Démonstration. Si l'algorithme renvoie X , alors directement $X^k = B$. Il ne nous reste donc qu'à démontrer que s'il existe une solution alors l'algorithme la trouve. Pour cela, supposons qu'il existe un FDDS Y tel que $Y^k = B$. De plus, supposons que $Y = Y_1 + \dots + Y_{m_Y}$ est trié selon $\leq_{ct}$. Nous allons montrer par récurrence sur les composantes connexes de Y que $X = Y$. Plus précisément que $\pi_i(Y)$ est la valeur de X au début de la $(i + 1)$ -ème itération avec $i \geq 0$ le nombre d'itérations de la boucle.

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

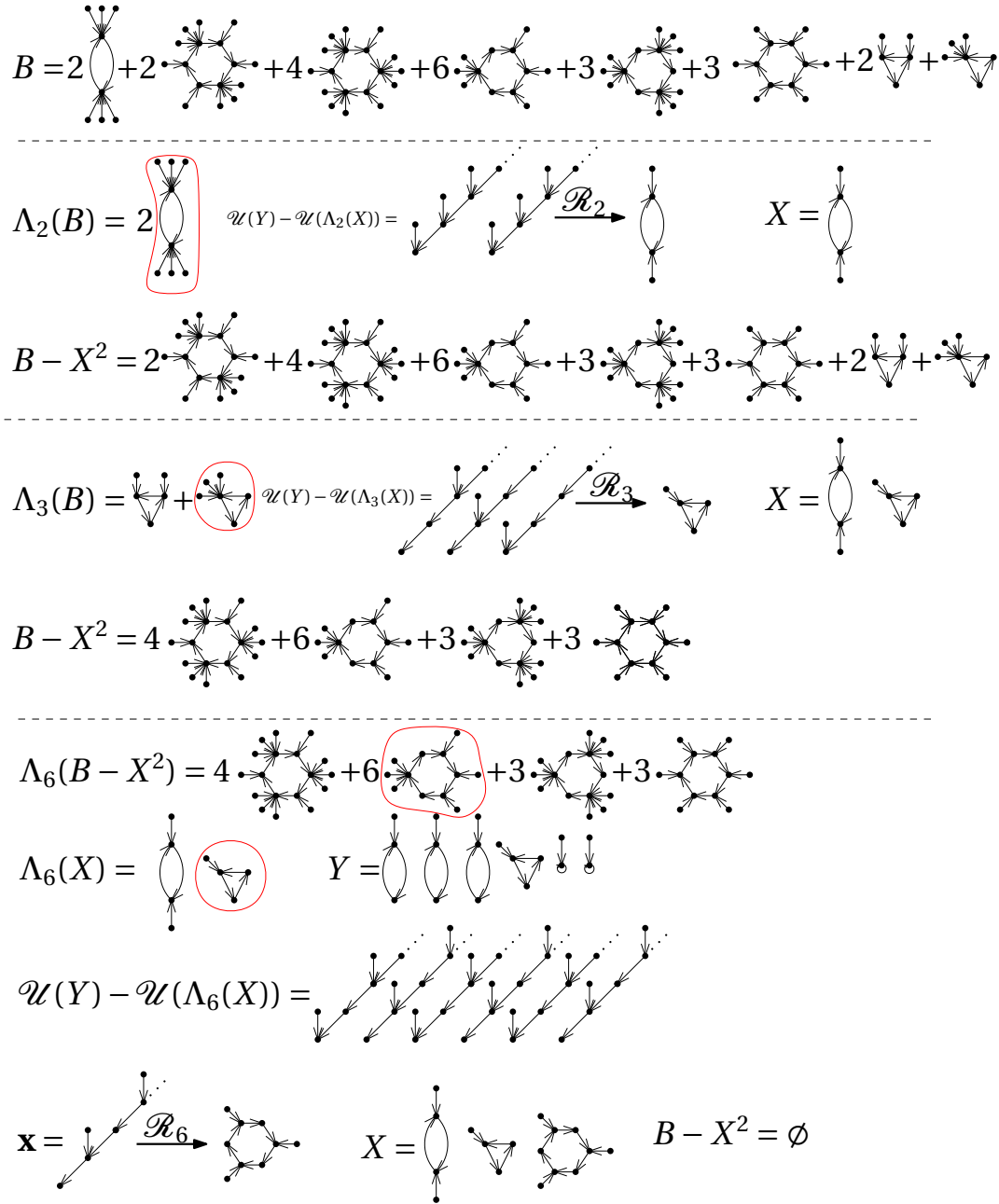


FIGURE 3.11. – Exemple d'exécution de l'Algorithme 3.12 prenant en entrées le FDDS B du haut et l'entier 2. Chaque partie séparée par une ligne en pointillé représente une itération de la boucle. Les composantes connexes de B entourées sont celles minimales par $\leq_{ct}$. La composante connexe de $\Lambda_6(X)$ est celle ayant le plus petit arbre d'unroll.

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

Avant la première itération de la boucle (instruction 3), nous avons que $X = \emptyset$, ce qui est bien égal au préfixe de Y de longueur 0 par définition.

Supposons que la propriété soit vraie pour i . Montrons alors qu'elle est vraie pour $i + 1$. Comme par hypothèse de récurrence $\pi_i(Y) = X'$ avec X' la valeur de X au début de la $(i + 1)$ -ème itération, nous déduisons que X'^k est un sous-multi-ensemble de B . Donc $B - X'^k$ est bien défini. De plus, de la Proposition 3.10, nous déduisons qu'à l'instruction 5, la valeur de p est la longueur du cycle de la composante Y_{i+1} .

Comme Y est une solution, nous déduisons que $\mathcal{U}(\Lambda_p(B))$ admet une racine k -ème. Donc par le Théorème 2.47, il suit que le FDDS R , de la ligne 6, est bien défini. Et de plus $\mathcal{U}(R) = \mathcal{U}(\Lambda_p(Y))$ par le Théorème 2.52. De là, $\mathcal{C}_n(\mathcal{U}(\Lambda_p(X')))$ est un sous-multi-ensemble de $\mathcal{C}_n(\mathcal{U}(R))$ de nouveau par hypothèse de récurrence. De surcroît, comme n est suffisamment grand, le Lemme 2.4 et l'hypothèse de récurrence induisent que $\mathbf{x}$ (ligne 9) est la troncature de $\mathbf{t} = \mathcal{U}(\Lambda_p(Y)) - \Lambda_p(\pi_i(Y))$. Donc, de la Proposition 3.10, nous obtenons que $\mathbf{t}$ est l'arbre d'unroll minimal de Y_{i+1} . Nous en déduisons donc que le FDDS construit en 10 est Y_{i+1} et donc qu'à la fin de l'itération $X = Y_1 + \dots + Y_i + Y_{i+1} = \pi_{i+1}(Y)$. Ceci conclut l'hérédité et la preuve du théorème. $\square$

3.1.2. Le cas des équations $P(X) = B$ où un coefficient non-constant de P contient un dendron

Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS où l'un des coefficients non-constants contient un dendron. Dans cette partie, nous expliquerons comment généraliser le Lemme 3.6 de X^k à $P(X)$ grâce à cette condition sur P . Notons que le Lemme 3.8 lui demeure utilisable, ce qui nous permet d'obtenir un algorithme polynomial pour les équations de la forme $P(X) = B$. Nous adapterons aussi ces deux lemmes (3.6 et 3.8) afin de montrer l'injectivité de P .

Lemme 3.14. *Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme dont au moins un coefficient non-constant contient un dendron. Soit $X = X_1 + \dots + X_{m_X}$ un FDDS trié selon $\leq_{ct}$. Finalement soit $0 \leq i < m_X$ un entier. Alors $\ell(P(X) - P(\pi_i(X))) = \ell(X_{i+1})$.*

Démonstration. Remarquons que $\ell(A_j(X^j - \pi_i(X))^j)$ est supérieur ou égal à $\ell(A_j)$ et à $\ell((X^j - \pi_i(X))^j)$ pour tout entier $j > 0$. Par le Lemme 3.6, $\ell(A_j(X^j - \pi_i(X))^j) \geq \ell(X_{i+1})$. Maintenant comme $L(P(X) - P(\pi_i(X))) = \bigcup_{j=1}^m L(A_j(X^j - \pi_i(X))^j)$, nous déduisons que $\ell(P(X) - P(\pi_i(X))) \geq \ell(X_{i+1})$.

Pour l'inégalité inverse, comme il existe un $j > 0$ tel que A_j contient D un dendron, il résulte que $A_j(X^j - \pi_i(X))^j$ contient une composante connexe dont le cycle a une longueur égale à $\text{lcm}(1, \ell(X_{i+1})) = \ell(X_{i+1})$, cette dernière étant issue du produit $D X_{i+1}^j$. Nous concluons que $\ell(X_{i+1}) \geq \ell(P(X) - P(\pi_i(X)))$ et donc le lemme suit. $\square$

Des Lemmes 3.14 et 3.8, nous déduisons la proposition suivante.

Proposition 3.15. *Soit P un polynôme dont au moins un coefficient non-constant contient un dendron. Soient $X = X_1 + \dots + X_{m_X}$ un FDDS trié selon $\leq_{ct}$. Soit D la plus petite composante connexe de $P(X) - P(\pi_i(X))$ selon $\leq_{ct}$. Alors X_{i+1} :*

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

1. a a une longueur de cycle égale à $\ell(D)$, et
2. son arbre d'unroll minimal est celui de $\Lambda_{\ell(D)}(X - \pi_i(X))$.

Corollaire 3.16. Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS. Si au moins un des coefficients non-constants de P contient un dendron, alors P est injectif.

Démonstration. Soient $X = X_1 + \dots + X_{m_X}$ et $Y = Y_1 + \dots + Y_{m_Y}$ deux FDDS triés selon $\leq_{ct}$ tels que $P(X) = P(Y)$. Montrons que $X = Y$. Pour cela, nous allons montrer par récurrence que $\pi_i(X) = \pi_i(Y)$ pour tout entier $i \geq 0$.

Supposons que $i = 0$, alors par définition $\pi_i(X) = \pi_i(Y) = \emptyset$. La propriété est donc vraie dans le cas de base.

Soit $i \geq 0$ un entier. Supposons que $\pi_i(X) = \pi_i(Y)$. Montrons que $\pi_{i+1}(X) = \pi_{i+1}(Y)$. Remarquons que pour cette démonstration, il nous suffit de prouver que $X_{i+1} = Y_{i+1}$. Commençons par établir que $\ell(X_{i+1}) = \ell(Y_{i+1})$. Comme $\pi_i(X) = \pi_i(Y)$ par hypothèse de récurrence, il suit que $P(X) - P(\pi_i(X)) = P(Y) - P(\pi_i(Y))$. Donc l'affirmation suit par la Proposition 3.15.

Il nous faut maintenant prouver que $\min(\mathcal{U}(X_{i+1})) = \min(\mathcal{U}(Y_{i+1}))$. Pour cela, remarquons que puisque $P(X) = P(Y)$, alors $\Lambda_p(P(X)) = \Lambda_p(P(Y))$ pour tout entier $p > 0$. Et en particulier $\Lambda_{\ell(X_{i+1})}(P(X)) = \Lambda_{\ell(X_{i+1})}(P(Y))$. De ce fait, le Lemme 3.2 induit que $\sum_{i=0}^m \Lambda_p(A_i) \Lambda_p(X)^i = \sum_{i=0}^m \Lambda_p(A_i) \Lambda_p(Y)^i$ car $\Lambda_p(X)$ est un morphisme de $(\mathbb{D}, +, \times)$ dans lui-même.

Comme la propriété que nous souhaitons démontrer porte sur des arbres d'unroll, nous allons maintenant les faire apparaître. Pour cela, nous appliquons la fonction $\mathcal{U}(X)$ à gauche et à droite de l'égalité et la distribuons sur les sommes et produits, nous obtenons que $\sum_{i=0}^m \mathcal{U}(\Lambda_p(A_i)) \mathcal{U}(\Lambda_p(X))^i = \sum_{i=0}^m \mathcal{U}(\Lambda_p(A_i)) \mathcal{U}(\Lambda_p(Y))^i$. Maintenant, comme par le Théorème 2.52 tous les polynômes d'unroll sont injectifs, nous déduisons que $\mathcal{U}(\Lambda_p(X)) = \mathcal{U}(\Lambda_p(Y))$. Et l'hypothèse de récurrence induit que $\mathcal{U}(\Lambda_p(X)) - \mathcal{U}(\Lambda_p(\pi_i(X))) = \mathcal{U}(\Lambda_p(Y)) - \mathcal{U}(\Lambda_p(\pi_i(Y)))$. En appliquant les factorisations nécessaires (possibles par la Proposition 1.52 et le Lemme 3.2), nous déduisons que $\mathcal{U}(\Lambda_p(X - \pi_i(X))) = \mathcal{U}(\Lambda_p(Y - \pi_i(Y)))$. Donc, l'affirmation suit par la Proposition 3.15. $\square$

De la Proposition 3.15, nous déduisons que de simples modifications de l'Algorithme 3.12 permettent de résoudre les équations de la forme $P(X) = B$. En effet, si en entrée nous donnons P au lieu de k et que dans l'algorithme nous remplaçons les lignes 5,6 et 14 par :

- 5 $p \leftarrow \ell(B - P(X))$,
- 6 $R \leftarrow \text{SolveurPolyUnroll}(\sum_{i=0}^m \mathcal{U}(\Lambda_p(A_i))X, \mathcal{U}(B))$ (Algorithme 2.56) vérifiant que R est bien défini,
- 14 vérifier que $|X| \leq \sqrt[m]{|B|}$ et que $P(X)$ est un sous-multi-ensemble de B ,

Alors nous obtenons un algorithme dont la correction et la polynomialité peuvent être montrées par un raisonnement similaire à celui utilisé pour la preuve du Théorème 3.13. Nous en concluons ainsi l'énoncé suivant.

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

I	α_I	δ_I	$\prod_{a \in N-I}$	β_I
$\emptyset$	216	1	36	252
2	216	1	18	198
3	216	1	12	204
6	216	1	6	210
2,3	216	1	6	222
2,6	216	2	3	222
3,6	216	3	2	222
2,3,6	216	6	1	210

TABLEAU 3.18. – Tableau représentant les valeurs des suites α, β et δ pour tous les sous-ensembles de $\{2, 3, 6\}$. La colonne d'en-tête I donne les sous-ensembles, celles d'en-tête α_I, δ_I et β_I les valeurs des suites évaluées pour I .

Théorème 3.17. Soient P un polynôme de FDDS et B un FDDS. Si au moins l'un des coefficients non-constants de P contient un dendron, alors nous résolvons $P(X) = B$ en temps polynomial.

3.1.3. Conditions nécessaires à l'injectivité des polynômes de FDDS

Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS. Nous montrons maintenant que si aucun coefficient non-constant de P ne contient de dendron, alors P n'est pas injectif. Ceci signifie que l'algorithme de la section précédente permet déjà de résoudre toutes les équations de la forme $P(X) = B$ avec P un polynôme sur les FDDS injectif. Pour cela, nous allons construire à partir de P deux FDDS différents X et Y tels que $P(X) = P(Y)$, et nous prendrons la construction introduite dans la preuve du Lemme 33 de [40] (également dans [43]). Notre but premier est de montrer que si A ne contient pas de dendron, alors il existe deux FDDS différents X et Y tels que $AX^k = AY^k$ et ce pour tout entier $k > 0$.

Commençons par rappeler la construction. Soit δ_J une suite définie récursivement par $\delta_\emptyset = 1$ et $\delta_{J \cup \{a\}} = \gcd(a, \text{lcm}(J)) \delta_J$ pour tout entier $a \geq 0$ n'étant pas dans J , notons que si $J = \emptyset$, alors $\text{lcm}(J) = 1$. Soit N un ensemble d'entiers strictement supérieurs à 1. Pour chaque sous-ensemble I de N , nous définissons les deux suites $\alpha_I = \delta_N \prod_{a \in N} a$ et $\beta_I = \alpha_I + (-1)^{|I|} \delta_I \prod_{a \in N-I} a$. Un exemple de ces suites est donné dans le Tableau 3.18. Nous utiliserons α_I et β_I pour construire le X et le Y énoncés plus tôt. Nous commençons par le cas où A est un cycle.

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

Lemme 3.19. Soit $k \geq 1$ un entier. Soit a un élément de $N \subseteq \mathbb{N}$. Soient I un sous-ensemble de $N - \{a\}$ et $J = I \cup \{a\}$. Alors :

$$C_a(\beta_I C_{\text{lcm}(I)} + \beta_J C_{\text{lcm}(J)})^k = C_a(\alpha_I C_{\text{lcm}(I)} + \alpha_J C_{\text{lcm}(J)})^k.$$

Démonstration. De la preuve du Lemme 33 de [40], nous avons :

$$C_a(\beta_I C_{\text{lcm}(I)} + \beta_J C_{\text{lcm}(J)}) = C_a(\alpha_I C_{\text{lcm}(I)} + \alpha_J C_{\text{lcm}(J)}).$$

Ainsi, nous pouvons remplacer une instance du terme $C_a(\beta_I C_{\text{lcm}(I)} + \beta_J C_{\text{lcm}(J)})$ dans l'équation de l'énoncé ci-dessus et ainsi obtenir :

$$C_a(\beta_I C_{\text{lcm}(I)} + \beta_J C_{\text{lcm}(J)})^k = (\alpha_I C_{\text{lcm}(I)} + \alpha_J C_{\text{lcm}(J)}) C_a(\beta_I C_{\text{lcm}(I)} + \beta_J C_{\text{lcm}(J)})^{k-1}.$$

Le lemme suit en appliquant récursivement la même substitution. $\square$

Nous allons maintenant étendre cette construction au cas où A est une permutation qui ne contient pas de cycle de longueur 1.

Lemme 3.20. Soit $A = A_1 + \dots + A_{m_A}$ une permutation. Si $1 \notin L(A)$, alors il existe deux FDDS X et Y différents tels que $AX^k = AY^k$ pour tout entier $k > 0$.

Démonstration. Soient $X = \sum_{I \subseteq L(A)} \alpha_I C_{\text{lcm}(I)}$ et $Y = \sum_{I \subseteq L(A)} \beta_I C_{\text{lcm}(I)}$. Par exemple, si $A = C_2 + C_3 + C_6$, alors en reprenant les valeurs de la Table 3.18, nous obtenons $X = 216(C_1 + C_2 + C_3 + 5C_6)$ tandis que $Y = 252C_1 + 198C_2 + 204C_3 + 1086C_6$. Puisque pour tout sous-ensemble I de $L(A)$ l'entier β_I est égal à α_I plus un terme non-nul, il résulte que $\alpha_I \neq \beta_I$. Donc en particulier, $\alpha_\emptyset \neq \beta_\emptyset$. Et comme ces deux entiers décrivent le nombre de points fixes dans X et Y , nous déduisons que $X \neq Y$.

Soit a un élément de $L(A)$. Soient $I_1, \dots, I_n$ tous les sous-ensembles possibles de $L(A) - \{a\}$. Nous définissons $J_1, \dots, J_n$ sous-ensembles de $L(A)$ tels que $J_i = I_i \cup \{a\}$ pour tout entier i entre 1 et n . Alors $Y = \sum_{i=1}^n (\beta_{I_i} C_{\text{lcm}(I_i)} + \beta_{J_i} C_{\text{lcm}(J_i)})$. En distribuant la puissance et le produit sur la somme, nous obtenons :

$$C_a Y^k = \sum_{k_1 + \dots + k_n = k} \binom{k}{k_1, \dots, k_n} C_a(\beta_{I_1} C_{\text{lcm}(I_1)} + \beta_{J_1} C_{\text{lcm}(J_1)})^{k_1} \prod_{i=2}^n (\beta_{I_i} C_{\text{lcm}(I_i)} + \beta_{J_i} C_{\text{lcm}(J_i)})^{k_i}.$$

Or comme le Lemme 3.19 induit que :

$$C_a(\beta_{I_1} C_{\text{lcm}(I_1)} + \beta_{J_1} C_{\text{lcm}(J_1)})^{k_1} = C_a(\alpha_{I_1} C_{\text{lcm}(I_1)} + \alpha_{J_1} C_{\text{lcm}(J_1)})^{k_1}.$$

nous pouvons remplacer ces termes dans l'équation précédente, et obtenir que :

$$C_a Y^k = \sum_{k_1 + \dots + k_n = k} \binom{k}{k_1, \dots, k_n} C_a(\alpha_{I_1} C_{\text{lcm}(I_1)} + \alpha_{J_1} C_{\text{lcm}(J_1)})^{k_1} \prod_{i=2}^n (\beta_{I_i} C_{\text{lcm}(I_i)} + \beta_{J_i} C_{\text{lcm}(J_i)})^{k_i}.$$

3. Résolution d'équations polynomiales de FDDS – 3.1. Le cas où le polynôme est injectif

En répétant cette substitution pour tout i entre 2 et n , il suit que

$$C_a Y^k = C_a \sum_{k_1 + \dots + k_n = k} \binom{k}{k_1, \dots, k_n} \prod_{i=1}^n (\alpha_{I_i} C_{\text{lcm}(I_i)} + \alpha_{J_i} C_{\text{lcm}(J_i)})^{k_i}.$$

Et puisque $X^k = \sum_{k_1 + \dots + k_n = k} \binom{k}{k_1, \dots, k_n} \prod_{i=1}^n (\alpha_{I_i} C_{\text{lcm}(I_i)} + \alpha_{J_i} C_{\text{lcm}(J_i)})^{k_i}$, nous concluons que $C_a Y^k = C_a X^k$.

Cette propriété est vraie pour chaque composante connexe C_a de A , donc en additionnant tous les termes et en faisant les factorisations nécessaires, il suit que $AX^k = AY^k$. $\square$

Notons que le X et le Y que nous avons construits dans le Lemme 3.20 sont des permutations. C'est donc aussi le cas de X^k et Y^k car les permutations forment un semi-anneau commutatif. Remarquons que le produit d'un FDDS générique avec une permutation a une forme particulière. En effet, si nous considérons $A = A_1 + \dots + A_{m_A}$ un FDDS et $X = X_1 + \dots + X_{m_X}$ une permutation tels que chaque A_i et chaque X_j sont connexes, alors pour chaque i et j , le FDDS $A_i X_j$ sera constitué de $\gcd(\ell(A_i), \ell(X_j))$ composantes connexes de longueur de cycle $\text{lcm}(\ell(A_i), \ell(X_j))$ et dont l'arbre d'unroll minimal est égal à celui de A_i .

Lemme 3.21 ([40, 47, 45]). *Soient A un FDDS connexe et X une permutation. Alors toutes les composantes connexes de AX ont leur arbre d'unroll minimal qui est égal à celui de A .*

Corollaire 3.22 ([40, 47, 45]). *Soient A un FDDS connexe et X et Y deux permutations. Si $C_{\ell(A)} X = C_{\ell(A)} Y$, alors $AX = AY$.*

Démonstration. Supposons que $C_{\ell(A)} X = C_{\ell(A)} Y$. Donc il existe une fonction bijective f des composantes connexes de $C_{\ell(A)} X$ vers celles de $C_{\ell(A)} Y$ telle que $\ell(f(D)) = \ell(D)$ pour toute composante connexe D de $C_{\ell(A)} X$.

Maintenant, considérons AX et AY . Nous avons qu'il existe deux fonctions bijectives g et h de respectivement l'ensemble des composantes connexes de AX vers les composantes connexes de $C_{\ell(A)} X$ et de l'ensemble des composantes connexes AY vers $C_{\ell(A)} Y$ telles que $\ell(g(D)) = \ell(D)$ et $\ell(h(D)) = \ell(D)$.

Par le Lemme 3.21, nous avons que chaque composante connexe de AX et AY a comme arbre d'unroll minimal $\min(\mathcal{U}(A))$. Donc D_X et $D_Y = h^{-1}(f(g(D_X)))$ ont le même arbre d'unroll minimal pour toute composante connexe D_X de AX . Et de plus, par définition, D_X et D_Y ont la même taille de cycles. Donc $D_X = D_Y$. $\square$

Grâce à cette propriété, nous sommes à même de généraliser le Lemme 3.20 au cas où A est un FDDS sans dendron. Ceci permet de caractériser l'injectivité des monômes de FDDS.

Proposition 3.23. *Un monôme de FDDS non-constant, c'est-à-dire de la forme AX^k avec $k > 0$, est injectif si et seulement si A contient un dendron.*

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

Démonstration. Sans perte de généralité, nous posons $A = A_1 + \dots + A_{m_A}$ tel que chaque A_i est une composante connexe.

($\Leftarrow$) Si A contient un dendron, alors le sens est donné par le Corollaire 3.16.

($\Rightarrow$) Pour l'autre direction, supposons que A ne contient pas de dendron. Donc $L(A)$ ne contient pas 1. Soit $A' = A'_1 + \dots + A'_{m_A}$ la permutation telle que $\ell(A'_i) = \ell(A_i)$ pour tout i . Par la preuve du Lemme 3.20, il existe deux permutations différentes X et Y telles que $A'_i X^k = A'_i Y^k$. Donc par le Corollaire 3.22, nous avons que $A_i X^k = A_i Y^k$. Donc par somme $AX^k = AY^k$. $\square$

Corollaire 3.24. Soit A un FDDS. Alors le monôme AX^k , avec $k > 0$, est injectif si et seulement si AX^i est injectif pour tout entier $i > 0$.

Pour terminer la caractérisation de l'injectivité, nous prouvons qu'un polynôme est injectif si et seulement si un monôme dérivé de P l'est également. Le coefficient de ce monôme est la somme des coefficients non-constants du polynôme, tandis que son degré est seulement strictement positif. C'est-à-dire que nous allons démontrer que $P = \sum_{i=0}^m A_i X^i$ est injectif si et seulement si $(A_1 + \dots + A_m)X^k$ est injectif pour n'importe quel entier $k > 0$. L'idée derrière ceci est que la somme des coefficients non-constants d'un polynôme contient un dendron si et seulement si au moins l'un des coefficients non-constants en contient un.

Théorème 3.25. Soient $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS et $M = (\sum_{i=1}^m A_i)X$ un monôme de FDDS. Alors P est injectif si et seulement si M est injectif.

Démonstration. Supposons que M est injectif. Donc par la Proposition 3.23, $\sum_{i=1}^m A_i$ contient un dendron. Donc au moins l'un des coefficients non-constants de P contient un dendron et la direction suit par le Corollaire 3.16.

Pour l'autre direction, supposons que M ne soit pas injectif. Soient X et Y les deux FDDS construits dans la preuve de la Proposition 3.23. Comme précédemment, $X \neq Y$ mais $A_i X^i = A_i Y^i$ pour tout i entre 0 et m (si $i = 0$, alors X^i et Y^i sont des points fixes). Donc par somme, $P(X) = P(Y)$. $\square$

3.2. Le cas des polynômes “pseudo-injectifs”

Soit $P = \sum_{i=0}^m A_i X^i$. En prenant en compte la définition des polynômes de FDDS injectifs issue de la section précédente, nous constatons que la plus petite longueur de cycles présente dans les coefficients non-constants, en symbole $\ell(A_1 + \dots + A_m)$, est un diviseur de l'ensemble des longueurs de cycles des coefficients non-constants car cette plus petite longueur est 1. Autrement dit, $\ell(A_1 + \dots + A_m)$ divise tous les éléments de $L(A_1 + \dots + A_m)$. Nous appelons polynômes “pseudo-injectifs” les polynômes respectant cette condition sans pour autant que $\ell(A_1 + \dots + A_m)$ soit égale à 1. De plus, il résulte que $\ell(A_1 + \dots + A_m)$ sera appelée la *graine* de P . Si le polynôme est un monôme (donc $P = AX^k$), nous préférons dire que le coefficient est *pseudo-annulable*, car il s'agit d'une généralisation des éléments annulables, à dire que le monôme est pseudo-injectif. Il est toutefois important de noter que cette appellation n'est issue

que d'une condition sur la forme et non sur le “niveau d'injectivité” du polynôme. C'est-à-dire que nous ne bornons pas le nombre de FDDS $X_1, \dots, X_r$ différents tels que $P(X_1) = \dots = P(X_r)$.

Dans cette section, nous montrerons que la résolution des équations de la forme $P(X) = B$ dans le cas où P est pseudo-injectif est polynomiale. Ceci nous permettra de résoudre en temps polynomial les équations de la forme $AX^k = B$ dans le cas où A est un FDDS connexe et $k > 0$ un entier. Ce type d'équation est connu dans la littérature sur les équations de FDDS sous le nom de “basic equations” [14, 39, 13, 46]. Et elles ont un rôle majeur dans le pipeline, introduit dans [14, 46], permettant de résoudre et d'énumérer les solutions de certaines équations polynomiales multivariées. Ces équations de base auront aussi leur importance dans les chapitres suivants.

3.2.1. Résoudre $P(X) = B$

Avant de résoudre $P(X) = B$ avec P un polynôme pseudo-injectif, nous proposons d'étudier le cas plus élémentaire des équations de la forme $C_a C_x = b C_b$. Une méthode simple pour résoudre ces équations est d'énumérer tous les entiers j entre 1 et b et de vérifier pour chacun si $C_a C_j = b C_b$. Ce qui est polynomial sur b . Étudions quelles seraient les valeurs possibles pour j . Pour cela, rappelons que si $C_a C_x = b C_b$ alors $\text{lcm}(a, x) = b$. Donc par définition du lcm, si nous considérons $\prod_{i=1}^{\infty} p_i^{a_i}$ et $\prod_{i=1}^{\infty} p_i^{x_i}$ la décomposition en facteurs premiers de a et x respectivement alors $\text{lcm}(a, x) = s_x s_a e$ avec

$$s_x = \prod_{\substack{i=1 \\ a_i < x_i}}^{\infty} p_i^{x_i}, s_a = \prod_{\substack{i=1 \\ x_i < a_i}}^{\infty} p_i^{a_i} \text{ et } e = \prod_{\substack{i=1 \\ a_i = x_i}}^{\infty} p_i^{a_i}.$$

Nous en déduisons donc que s_x est le plus petit entier tel que $b = \text{lcm}(a, s_x)$. Dans la suite, si a divise b , ce nombre s_x sera appelé l'*anti-lcm* de b par a , noté $\text{alcm}_a b$.

Définition 3.26 (anti-lcm). Soient $a, b > 0$ deux entiers dont la décomposition en facteurs premiers est respectivement $\prod_{i=1}^{\infty} p_i^{a_i}$ et $\prod_{i=1}^{\infty} p_i^{b_i}$ et tels que a divise b . Alors, l'*anti-lcm* de b par a est le nombre

$$\text{alcm}_a b = \prod_{\substack{i=1 \\ b_i > a_i}}^{\infty} p_i^{b_i}.$$

Par abus de notation, nous nous autorisons à définir l'*anti-lcm* de B par A avec A et B deux FDDS comme $\text{alcm}_A B = \text{alcm}_{\ell(A)} \ell(B)$ ainsi que l'*anti-lcm* de B par g avec B un FDDS et $g \geq 1$ un entier comme $\text{alcm}_g B = \text{alcm}_g \ell(B)$.

Notons que nous pouvons calculer $\text{alcm}_a b$ en temps polynomial sur b . En effet, une approche naïve mais efficace consiste à trouver le plus petit entier x tel que $\text{lcm}(a, x) = b$ via le test de tous les entiers entre 1 et b . Une approche plus sophistiquée pour le calcul du $\text{alcm}_a b$ consiste simplement à calculer $\gcd((b/a)^{\log b}, b)$, pouvant donc être réalisé en temps polynomial sur $\log_2 b$. Ceci est une conséquence du lemme suivant.

Algorithme 3.28 DivisionPermutation(A, B)

```

1: Tant que  $B \neq \emptyset$  faire
2:   Si  $|A| > |B|$  alors
3:     renvoyer  $\perp$ ,
4:   construire  $D$  le cycle de longueur  $\text{alcm}_A B$ ,
5:   trouver  $d_1$  et  $d_2$  le nombre de copies de  $C_{\ell(B)}$  dans respectivement  $AD$  et  $B$ ,
6:   Si  $eAD$  n'est pas un sous-multi-ensemble de  $B$  avec  $e \leftarrow d_2/d_1$  alors
7:     renvoyer  $\perp$ ,
8:    $X \leftarrow X + eD$ ,
9:    $B \leftarrow B - (eAD)$ ,
10: renvoyer  $X$ .
```

Lemme 3.27. Soient a et b deux entiers strictement positifs de décomposition en facteurs premiers respective $\prod_{i=1}^{\infty} p_i^{a_i}$ et $\prod_{i=1}^{\infty} p_i^{b_i}$ et tels que a divise b . Alors, nous avons $\text{alcm}_a b = \gcd((b/a)^k, b)$ pour tout entier $k \geq b_i$ et pour tout entier $i \geq 1$.

Démonstration. Remarquons que la décomposition en facteurs premiers de b/a est $\prod_{i=1}^{\infty} p_i^{b_i - a_i}$. Soit p_i un nombre premier de cette décomposition. Notons que celle de $(b/a)^k$ est $\prod_{i=1}^{\infty} p_i^{k(b_i - a_i)}$. Deux cas sont alors possibles. Soit $b_i - a_i = 0$ alors $p_i^{k(b_i - a_i)} = 1$ et donc l'exposant de p_i dans la décomposition en facteurs premiers de $\gcd((b/a)^k, b)$ est 0. Soit $b_i - a_i \geq 1$ induisant que $b_i > a_i$. Nous avons alors que $k(b_i - a_i) \geq b_i$ par la condition sur k . Donc, l'exposant de p_i dans la décomposition en facteurs premiers de $\gcd((b/a)^k, b)$ est b_i . $\square$

De plus, comme le nombre e défini plus tôt est co-premier avec $\text{alcm}_a b$, nous avons que $\gcd(e \cdot \text{alcm}_a b, a) = \gcd(e, a) \cdot \gcd(\text{alcm}_a b, a) = e \cdot \gcd(\text{alcm}_a b, a)$. Donc pour vérifier qu'il existe un x tel que $C_a C_x = b C_b$, il nous suffit de calculer $\text{alcm}_a b$ puis de diviser b par $\gcd(\text{alcm}_a b, a)$ et de vérifier que ce quotient (qui joue le rôle du e) est co-premier avec $\text{alcm}_a b$, si cela est possible. Notons que $C_a e C_{\text{alcm}_a b} = b C_b$.

Nous pourrions intuitivement penser qu'il est possible d'étendre ce raisonnement au cas où A et/ou B seraient des permutations quelconques. Nous obtiendrions ainsi une procédure semblable à l'Algorithme 3.28. Ce dernier prend en entrées A et B deux permutations et doit renvoyer une permutation X , s'il en existe une, telle que $AX = B$.

Remarquons que cet algorithme est réalisable en temps polynomial car chacune de ces opérations est exécutable en temps polynomial sur $|B|$ et que le nombre d'itérations de la boucle est polynomial sur $|B|$. Cependant, cette intuition est incorrecte.

Exemple 3.29. Considérons l'équation $(C_2 + C_3)X = 5C_6$ alors notre algorithme ne trouve pas de solution car $\text{alcm}_2 6 = 3$ et $3C_3$ n'est pas un sous-multi-ensemble de $5C_6$. Mais $(C_2 + C_3)C_6 = 5C_6$.

Nous allons toutefois prouver que cette approche fonctionne pour résoudre $AX = B$ si A est pseudo-annulable.

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

Commençons par montrer que si l'équation admet une solution, alors $L(X)$ doit contenir un multiple de $\text{alcm}_A B$. Notons que cette propriété est incorrecte si A n'est pas pseudo-annulable.

Exemple 3.30. Considérons $A = C_2 + C_3$ et $B = 3C_3 + C_6$ alors $\text{alcm}_A B$ n'est pas défini car 3 n'est pas un multiple de 2. Cependant $AX = B$ admet bien une solution, C_3 .

Lemme 3.31. Soient A, X et B des permutations telles que A est pseudo-annulable et B est non-vide. Si $AX = B$ alors il existe un diviseur q de $\ell(A)$ tel que $q \cdot \text{alcm}_A B$ appartient à $L(X)$ et $\gcd(q, \text{alcm}_A B) = 1$.

Démonstration. Supposons que $AX = B$. Alors, il existe un entier $a \in L(A)$ et un entier $x \in L(X)$ tels que $\ell(B) = \text{lcm}(a, x)$. Nous montrons qu'il existe q un diviseur de $\ell(A)$ tel que $x = q \cdot \text{alcm}_A B$ et $\gcd(q, \text{alcm}_A B) = 1$. Pour cela, nous commençons par établir que x est un multiple de $\text{alcm}_A B$. Autrement dit, nous prouvons que $\text{lcm}(\ell(A), x) = \ell(B)$.

Remarquons que $\text{lcm}(\ell(A), x) \geq \ell(B)$. En effet, comme $AX = B$, il s'ensuit que $\text{lcm}(\ell(A), x)$ est un élément de $L(B)$. Pour l'inégalité inverse, rappelons que $\ell(A)$ divise tous les éléments de $L(A)$ puisque A est pseudo-annulable. Donc en particulier $\ell(A)$ divise a . Ainsi, $\text{lcm}(\ell(A), x)$ divise $\text{lcm}(a, x)$ et $\text{lcm}(\ell(A), x) \leq \text{lcm}(a, x) = \ell(B)$. Nous en concluons que $\text{lcm}(\ell(A), x) = \ell(B)$. Ceci implique que x est un multiple de $\text{alcm}_A B$ par définition de alcm . Autrement dit, il existe un entier $q > 0$ tel que $x = q \text{alcm}_A B$. Il ne reste plus qu'à montrer que q est un diviseur de $\ell(A)$ co-premier avec $\text{alcm}_A B$.

Puisque x divise $\ell(B)$, aucun facteur premier de q n'apparaît dans la décomposition en facteurs premiers de $\text{alcm}_A B$ avec un exposant strictement supérieur à 0. Sinon son exposant dans x serait plus grand que dans la décomposition en facteurs premiers de $\ell(B)$. Donc q est co-premier avec $\text{alcm}_A B$. En outre, q divise $\ell(B) / \text{alcm}_A B$.

Finalement, si nous considérons $\prod_{i=1}^{\infty} p_i^{a_i}$ et $\prod_{i=1}^{\infty} p_i^{b_i}$ la décomposition en facteurs premiers de $\ell(A)$ et $\ell(B)$, alors

$$\frac{\ell(B)}{\text{alcm}_A B} = \prod_{\substack{i=1 \\ a_i=b_i}}^{\infty} p_i^{a_i}.$$

En constatant que ce nombre est un diviseur de $\ell(A)$, nous concluons que q est aussi un diviseur de $\ell(A)$. □

Corollaire 3.32. Si $AX = B$, avec A et B deux permutations telles que A est pseudo-annulable, alors pour chaque cycle $C_x \in X$ qui vérifie $C_{\ell(B)} \in AC_x$, il existe un diviseur q de $\ell(A)$ satisfaisant $q \cdot \text{alcm}_A B = x$ et $\gcd(q, \text{alcm}_A B) = 1$.

Bien que le Lemme 3.31 et le Corollaire 3.32 décrivent partiellement un cycle de X , une inconnue demeure toujours : quelle valeur donner à q . Dans le lemme suivant, nous montrerons que nous sommes en mesure de lever cette inconnue en fixant la valeur de q à 1. Là encore, ceci n'est possible que puisque A est pseudo-annulable. Nous avons déjà illustré cela avec l'exemple $(C_2 + C_3)C_6 = 5C_6$ (Exemple 3.29).

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes
“pseudo-injectifs”

Lemme 3.33. Soit $A = \sum_{i \in L(A)} a_i C_i$ une permutation pseudo-annulable et B, X des permutations. Soit q un diviseur de $\ell(A)$ tel que $\gcd(q, \text{alcm}_A B) = 1$. Alors, nous avons $A(X + C_{q \cdot \text{alcm}_A B}) = B$ si et seulement si $A(X + dC_{(q/d) \cdot \text{alcm}_A B}) = B$ pour tout d diviseur de q .

Démonstration. Soient $x = q \cdot \text{alcm}_A B$, d un diviseur de q et $a \in L(A)$. Dans cette preuve, nous calculerons $C_a C_x$. Raison pour laquelle nous commençons par étudier $\gcd(a, x)$ et $\text{lcm}(a, x)$.

Tout d'abord, comme q est co-premier avec $\text{alcm}_A B$, nous avons que :

$$\gcd(x, a) = \gcd(\text{alcm}_A B, a) \gcd(q, a).$$

Maintenant, comme A est pseudo-annulable, il s'ensuit que $\ell(A)$ est un diviseur de a . Et donc par transitivité q , d et q/d sont aussi des diviseurs de a . De ce fait, nous déduisons que $\gcd(q, a) = q$ et $q = d \cdot (q/d) = d \cdot \gcd(q/d, a)$. Donc l'équation précédente se réécrit :

$$\begin{aligned} \gcd(x, a) &= q \cdot \gcd(\text{alcm}_A B, a) \\ &= d \cdot \gcd(q/d, a) \gcd(\text{alcm}_A B, a) \\ &= d \cdot \gcd((q/d) \cdot \text{alcm}_A B, a) \end{aligned}$$

En outre, pour tout diviseur d' de a co-premier avec $\text{alcm}_A B$, il s'ensuit que :

$$\begin{aligned} \text{lcm}(d' \cdot \text{alcm}_A B, a) &= \text{lcm}(\text{lcm}(d', a), \text{lcm}(\text{alcm}_A B, a)) \\ &= \text{lcm}(\text{lcm}(\text{alcm}_A B, a), a) \\ &= \text{lcm}(\text{alcm}_A B, a) \end{aligned}$$

De là, nous obtenons que :

$$\begin{aligned} C_a C_x &= \gcd(x, a) C_{\text{lcm}(x, a)} \\ &= d \cdot \gcd((q/d) \cdot \text{alcm}_A B, a) \cdot C_{\text{lcm}((q/d) \cdot \text{alcm}_A B, a)} \\ &= d C_a C_{(q/d) \cdot \text{alcm}_A B} \end{aligned}$$

Ce qui implique que :

$$\begin{aligned} AC_x &= \sum_{i \in L(A)} a_i C_i C_x \\ &= \sum_{i \in L(A)} a_i d C_i C_{(q/d) \cdot \text{alcm}_A B} \\ &= A d C_{(q/d) \cdot \text{alcm}_A B} \end{aligned}$$

D'où le lemme. □

Du Corollaire 3.32 et du Lemme 3.33, nous pourrions démontrer que l'Algorithme 3.28 est correct si A et B sont des permutations avec A pseudo-annulable.

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

Lemme 3.34. Soient A et B des permutations avec A pseudo-annulable. Si l'équation $AX = B$ admet au moins une solution, alors l'Algorithme 3.28 produit celle qui maximise le nombre de composantes connexes.

Démonstration. Cette démonstration est basée sur les super-préfixes (Définition 3.5). Supposons que l'Algorithme 3.28 renvoie X . Montrons que $AX = B$. Par construction $X = e_1 X_1 + \dots + e_{m_X} X_{m_X}$ tel que $e_i AX_i$ est un sous-multi-ensemble de B_{i-1} avec $B_i = B - A\mathcal{P}_i(X)$ et de plus $B_{m_X} = \emptyset$. Nous déduisons donc que B_i est aussi égal à $B_{i-1} - (e_i AX_i)$. En effet,

$$B_i = B - A\mathcal{P}_i(X) = B - A\mathcal{P}_{i-1}(X) - (e_i AX_i) = B_{i-1} - (e_i AX_i).$$

Cela induit que $B_{i-1} = B_i + (e_i AX_i)$. Donc en particulier

$$B_{m_X-1} = B_{m_X} + (e_{m_X} AX_{m_X}) = e_{m_X} AX_{m_X}$$

et récursivement $B = B_0 = AX$.

Supposons qu'il existe une permutation Y de m_Y composantes telle que $AY = B$. Montrons maintenant que l'algorithme trouve une solution. Pour cela, commençons par définir quelle solution l'algorithme construit. Par le Lemme 3.31, il existe un entier $q_1 > 0$ et une permutation Y_1 tels que $Y = Y_1 + C_{q_1 \cdot \text{alcm}_A B}$. Nous pouvons donc définir la permutation $B_1 = AY_1$. En appliquant ce raisonnement récursivement sur A et B_1 , nous déduisons qu'il existe m_Y entiers $q_1, \dots, q_{m_Y}$ tels que :

$$Y = C_{q_1 \cdot \text{alcm}_A B_0} + \dots + C_{q_{m_Y} \cdot \text{alcm}_A (B_{m_Y-1})}$$

avec B_i la permutation récursivement définie par $B_0 = B$ et $B_i = AY_i$ où :

$$Y_i = Y - \left(\sum_{j=1}^i C_{q_j \cdot \text{alcm}_A B_j} \right)$$

Du Lemme 3.33, nous inférons que $A(Y_1 + C_{q_1 \cdot \text{alcm}_A B_0}) = A(Y_1 + q_1 C_{\text{alcm}_A B_0})$. Donc, en appliquant ce raisonnement récursivement, nous déduisons que $AX = B$ avec :

$$X = q_1 C_{\text{alcm}_A B_0} + \dots + q_{m_Y} C_{\text{alcm}_A (B_{m_Y-1})}$$

Observons que X a le plus de composantes connexes possible car il a les cycles les plus petits possible. En réalisant les factorisations nécessaires afin de ne conserver que des composantes connexes différentes, nous déduisons que $X = e_1 X_1 + \dots + e_{m_X} X_{m_X}$ tel que $\ell(X_i) = \text{alcm}_A(B - \mathcal{P}_{i-1}(X))$. Ceci implique que $\ell(AX_i) < \ell(AX_j)$ si et seulement si $i < j$ et ce pour tous entiers i, j entre 1 et m_X . De même $\ell(AX_i) = \ell(AX_j)$ si et seulement si $i = j$.

Montrons que cette valeur de X est renvoyée par l'algorithme. Pour cela, nous allons montrer par récurrence sur la longueur du super-préfixe de X qu'à la fin de la i -ème itération de la boucle, la solution partielle est $\mathcal{P}_i(X)$. Comme avant la première itération de la boucle, la solution partielle est $\emptyset$, nous déduisons que la propriété est

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

vraie pour $i = 0$.

Soit $i \geq 0$ un entier. Supposons que la propriété est vraie pour i . Montrons alors qu'elle est vraie pour $i + 1$. Comme, par hypothèse d'induction, au commencement de la $(i + 1)$ -ème itération de la boucle, la solution partielle contient $\mathcal{P}_i(X)$, il nous suffit de démontrer que nous ajoutons exactement $e_{i+1}X_{i+1}$ au résultat au cours de la $(i + 1)$ -ème itération.

Tout d'abord, montrons que $D = X_{i+1}$ avec D la composante construite à la ligne 4. Dans cette optique notons que, par construction, la valeur de B au commencement de la $(i + 1)$ -ème itération est la valeur originale de B moins $\mathcal{P}_i(X)$. Donc D est un cycle de longueur $\text{alcm}_A(B - \mathcal{P}_i(X))$. Ce qui implique que $D = X_{i+1}$. Nous n'avons plus qu'à montrer que $e_{i+1} = d_2/d_1$ avec les d_1 et d_2 de l'instruction 6.

Pour démontrer cette affirmation, il nous suffit de prouver que $\ell(AX_{i+1})$ n'est pas un élément de $L(A(X - \mathcal{P}_{i+1}(X)))$. En effet, comme $AX = B$, nous déduisons alors que $A(X - \mathcal{P}_i(X)) = B - A\mathcal{P}_i(X)$. Donc $e_{i+1}AX_{i+1}$ est un sous-multi-ensemble de $B - A\mathcal{P}_i(X)$. Ce qui signifie que $e_{i+1} \cdot d_2 + c = d_1$ avec c le nombre de copies de $C_{\ell(AX_{i+1})}$ dans $B - A\mathcal{P}_{i+1}(X)$. Et si $\ell(AX_{i+1})$ n'est pas un élément de $L(B - A\mathcal{P}_{i+1}(X))$, alors $c = 0$ et $e_{i+1} \cdot d_2 = d_1$. Relevons que $d_2 > 0$ car il y a au moins une copie du plus petit cycle de $B - A\mathcal{P}_i(X)$ dans $B - A\mathcal{P}_i(X)$. Soit X_j avec $j \geq i + 1$ tel que $\ell(AX_{i+1})$ est un élément de $L(AX_j)$. Donc $\ell(AX_j) \leq \ell(AX_{i+1})$. De surcroît, le fait que $j \geq i + 1$ implique que $\ell(AX_j) \geq \ell(AX_{i+1})$. Donc $\ell(AX_j) = \ell(AX_{i+1})$ et $j = i + 1$. $\square$

Du Lemme 3.34, nous obtenons que s'il existe une solution à l'équation $AX = B$ avec A une permutation pseudo-annulable, alors l'Algorithme 3.28 produit une solution. Ainsi, cet algorithme ne renvoie $\perp$ que s'il n'existe pas de solution à l'équation $AX = B$. Nous en concluons la proposition suivante.

Proposition 3.35. *Soient A et B deux permutations avec A pseudo-annulable. Alors nous sommes en mesure de résoudre $AX = B$ en temps polynomial sur la taille de B en utilisant l'Algorithme 3.28.*

Nous pouvons généraliser nos résultats au cas des équations de la forme $P(X) = B$ où P est un polynôme pseudo-injectif sur les permutations et B est une permutation. En premier lieu, nous étendons le Lemme 3.31 comme suit.

Lemme 3.36. *Soient B et X deux permutations et $P = \sum_{i=0}^m A_i X^i$ un polynôme pseudo-injectif dont tous les coefficients sont des permutations. Soit g la graine de P . Alors, $P(X) = B$ implique qu'il existe un diviseur entier $q > 0$ de g tel que $q \cdot \text{alcm}_g(B - A_0)$ est un élément de $L(X)$ et $\text{gcd}(q, \text{alcm}_g(B - A_0)) = 1$.*

Démonstration. Supposons que $P(X) = B$. Donc, $P(X) - A_0 = B'$ avec $B' = B - A_0$. Ainsi, il existe un entier i entre 1 et m , et deux entiers a et y de respectivement $L(A_i)$ et $L(Y^i)$ tels que $\ell(B') = \text{lcm}(a, y)$.

Commençons par montrer qu'il existe un y' dans $L(Y)$ tel que $\text{lcm}(a, y) = \text{lcm}(a, y')$. Comme y est un élément de $L(Y^i)$, il existe $y_1, \dots, y_i$ des éléments de $L(Y)$ tels que $y = \text{lcm}(y_1, \dots, y_i)$. Donc $\text{lcm}(a, y_j) \leq \text{lcm}(a, y)$ pour tous les y_j . Or comme $L(Y)$ est un sous-ensemble de $L(Y^i)$, le Corollaire 3.7 induit que $\text{lcm}(a, y_j) \geq \text{lcm}(a, y)$.

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

Prouvons maintenant que $\text{lcm}(g, y') = \text{lcm}(a, y)$. Remarquons que $\text{lcm}(g, y')$ est un élément de $L(B - A_0)$ car y' est un élément de $L(Y^j)$ pour tout entier $j > 0$. Ceci induit que $\text{lcm}(g, y') \geq \text{lcm}(a, y)$. Cependant, comme g est un diviseur de a par définition, $\text{lcm}(g, y') \leq \text{lcm}(a, y') = \text{lcm}(a, y)$. D'où l'affirmation. De là, nous déduisons que $y' = q \cdot \text{alcm}_g(B - A_0)$. Par un raisonnement similaire à celui utilisé dans la preuve du Lemme 3.31, nous concluons que q est un diviseur de g co-premier avec $\text{alcm}_g(B - A_0)$. $\square$

Ainsi, nous sommes en mesure de généraliser le Lemme 3.33 comme suit.

Lemme 3.37. *Soient B et X deux permutations et $P = \sum_{i=0}^m A_i X^i$ un polynôme pseudo-injectif dont chaque coefficient est une permutation. Soit g la graine de P et soit $q > 0$ un diviseur entier de g tel que $\gcd(q, \text{alcm}_g(B - A_0)) = 1$. Alors $P(X + C_{q \cdot \text{alcm}_g B}) = B$ si et seulement si $P(X + dC_{(q/d) \cdot \text{alcm}_g B}) = B$ pour tout diviseur entier $d > 0$ de q .*

Démonstration. Soit $y = q \cdot \text{alcm}_g B$ et soit $d > 0$ un diviseur entier de q . Montrons d'abord que la propriété est vraie dans le cas des différents monômes de P . Soient j un entier entre 1 et m et a un élément de $L(A_j)$. Remarquons que la définition de q et d induit qu'ils divisent tous deux g et a . Par conséquent, d'après la preuve du Lemme 3.33, il s'ensuit que $C_a C_y^j = C_y^{j-1} C_a (dC_{(q/d) \cdot \text{alcm}_g(B - A_0)})$. En appliquant la même substitution récursivement, nous déduisons que $C_a C_y^j = C_a (dC_{(q/d) \cdot \text{alcm}_g B})^j$. En sommant les composantes connexes de A_j , nous obtenons que $A_j C_y^j = A_j (dC_{(q/d) \cdot \text{alcm}_g B})^j$. Maintenant, en sommant les différents monômes de P , ceci nous permet de conclure que $P(X + C_y) = P(X + dC_{(q/d) \cdot \text{alcm}_g B})$. $\square$

Les deux lemmes précédents nous permettent de déduire que nous pouvons adapter l'Algorithme 3.28 pour résoudre en temps polynomial les équations ayant la forme $P(X) = B$ si P est un polynôme pseudo-injectif (le Tableau 3.39 en montre une exécution). Ceci est résumé dans l'Algorithme 3.38 qui prend en entrée un polynôme pseudo-injectif P sur les permutations et une permutation B . Cette procédure renvoie une permutation X , s'il en existe une, telle que $P(X) = B$.

Notons que le test de l'étape 6 est nécessaire, puisque m peut en principe être plus grand que $|B|$ et alors $|X + D| > 1$ impliquerait que $|P(X + D)| \geq 2^m \geq 2^{|B|}$, et donc que le calcul de $P(X + D)$ requerrait un temps et un espace exponentiel. Le test lui-même peut être réalisé en utilisant l'exponentiation rapide et en s'arrêtant aussitôt qu'un résultat intermédiaire excède $|B|$ afin de garantir le temps polynomial.

Il s'ensuit que les étapes 6, 8 et 10 demandent un temps polynomial sur la taille des entrées. Nous concluons que cet algorithme est bien polynomial. En outre, comme pour l'Algorithme 3.28, si l'algorithme renvoie X , alors $P(X) = B$. Pour démontrer la direction inverse, il nous est possible d'adapter le raisonnement du Lemme 3.34 en exploitant les Lemmes 3.36 et 3.37 (en lieu et place des Lemmes 3.31 et 3.33) en remplaçant le produit par A par l'évaluation de P et en ajoutant les copies d'une composante connexe une par une au lieu de toutes simultanément.

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

Algorithme 3.38 $\text{SolveurPseudoPolyPermutation}(P, B)$

```

1: Si  $A_0$  n'est pas un sous-multi-ensemble de  $B$  alors
2:   renvoyer  $\perp$ ,
3:  $B \leftarrow B - A_0$ ,  $P \leftarrow P - A_0$  et  $X \leftarrow \emptyset$ ,
4: Tant que  $B \neq \emptyset$  faire
5:   créer  $D$  un cycle de longueur  $\text{alcm}_g B$ ,
6:   Si  $|P(X + D)| = \sum_{i=1}^m |A_i| |X + D|^i > |B|$  ou  $P(X + D) - P(X)$  n'est pas un sous-
     multi-ensemble de  $B$  alors
7:     renvoyer  $\perp$ ,
8:   Si  $P(X + D) - P(X)$  n'est pas un sous-multi-ensemble de  $B$  alors
9:     renvoyer  $\perp$ ,
10:   $B \leftarrow B - (P(X + D) - P(X))$  et  $X \leftarrow X + D$ ,
11: renvoyer  $X$ .
```

Proposition 3.40. *La résolution des équations de la forme $P(X) = B$, où B est une permutation, et P un polynôme pseudo-injectif dont tous les coefficients sont des permutations, est polynomiale sur la somme des tailles des coefficients de P et de celle de B .*

Nous considérons maintenant le cas où $A_0, \dots, A_m$ et B ne sont pas seulement des permutations. Comme pour le cas précédent, notre idée principale est de reconstruire itérativement la solution. À cette fin, nous exploiterons le Lemme 3.8 qui nous décrira quels arbres d'unroll appartiennent à chaque composante connexe. Il est toutefois important de noter que contrairement à la Section 3.1, nous ne générerons pas nécessairement les composantes connexes suivant $\leq_{ct}$. Ceci est dû au fait que même pour g fixé, la fonction $\text{alcm}_g X$ n'est pas monotone croissante.

Exemple 3.41. *Fixons g à 2, nous avons que $4 < 6$ mais pourtant :*

$$\text{alcm}_2 4 = 4 > \text{alcm}_2 6 = 3.$$

Pour pallier ce problème, nous introduisons un nouvel ordre total sur les composantes connexes. Ce dernier sera lié à un polynôme contrairement à l'ordre $\leq_{ct}$. De cette manière, nous comparerons différentes composantes connexes vis-à-vis d'un contexte.

Définition 3.42 (ordre $\leq_P$). Soit P un polynôme de FDDS sans terme constant et soient X, Y deux composantes connexes. Soient D_X et D_Y les plus petites composantes connexes de respectivement $P(X)$ et $P(Y)$ selon $\leq_{ct}$. Nous définissons que $X \leq_P Y$ si et seulement si :

1. $D_X <_{ct} D_Y$, ou
2. $D_X = D_Y$ et $X \leq_{ct} Y$.

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

B	Y	D	$P(Y + D)$	$P(Y)$	$P(Y + D) - P(Y)$
$16C_2 + 4C_4 + 18C_6 + C_{12}$	$\emptyset$	C_1	$C_2 + C_4 + C_6$	$\emptyset$	$C_2 + C_4 + C_6$
$15C_2 + 3C_4 + 17C_6 + C_{12}$	C_1	C_1	$4C_2 + 2C_4 + 2C_6$	$C_2 + C_4 + C_6$	$3C_2 + C_4 + C_6$
$12C_2 + 2C_4 + 16C_6 + C_{12}$	$2C_1$	C_1	$9C_2 + 3C_4 + 3C_6$	$4C_2 + 2C_4 + 2C_6$	$5C_2 + C_4 + C_6$
$7C_2 + C_4 + 15C_6 + C_{12}$	$3C_1$	C_1	$16C_2 + 4C_4 + 4C_6$	$9C_2 + 3C_4 + 3C_6$	$7C_2 + C_4 + C_6$
$14C_6 + C_{12}$	$4C_1$	C_3	$16C_2 + 4C_4 + 18C_6 + C_{12}$	$16C_2 + 4C_4 + 4C_6$	$14C_6 + C_{12}$
$\emptyset$	$4C_1 + C_3$				

TABLEAU 3.39. – Une exécution de l’Algorithme 3.38 sur l’équation $C_2X^2 + (C_4 + C_6)X = 16C_2 + 4C_4 + 18C_6 + C_{12}$. Chaque ligne de la table correspond à une itération des lignes 5–10. Cette exécution renvoie la solution $X = 4C_1 + C_3$. Remarquons que cette équation admet aussi comme solution $2C_2 + C_3$ et $2C_1 + C_2 + C_3$, mais l’Algorithme 3.38 ne trouve que la solution qui maximise le nombre de composantes connexes. Ce tableau est issu de [44].

Si le contexte est clair, P désignera toujours le polynôme que nous considérons. Remarquons que deux composantes connexes A et B peuvent être telles que $A <_P B$ mais pourtant $B <_{ct} A$ comme l’illustre l’exemple suivant.

Exemple 3.43. Considérons $P = C_2X$ un polynôme sur les FDDS, $A = C_4$ et $B = C_3$. Alors $B <_{ct} A$. En effet, $3 < 4$. Cependant, $A <_P B$ car $\ell(C_2C_4) = 4 < 6 = \ell(C_2C_3)$.

Afin d’utiliser le Lemme 3.8, nous l’adaptions en changeant l’hypothèse sur X . C’est-à-dire qu’au lieu de supposer que les composantes connexes de X sont triées selon $\leq_{ct}$, nous supposons qu’elles le seront selon $\leq_P$. Mais avant cela, nous introduisons un résultat technique permettant de relier la longueur du cycle de la plus petite composante connexe de $P(X_{i+1})$ selon $\leq_{ct}$ avec celle de $P(X) - P(\pi_i(X))$, avec $X = X_1 + \dots + X_{m_X}$ où tous les X_j sont connexes et triés selon $\leq_P$.

Lemme 3.44. Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme de FDDS sans terme constant. Soit $X = X_1 + \dots + X_{m_X}$ un FDDS trié selon l’ordre $\leq_P$. Soit $i \geq 0$ un entier. Soit D_X la plus petite composante connexe selon $\leq_{ct}$ de $P(X_{i+1})$. Alors, $\ell(D_X) = \ell(P(X) - P(\pi_i(X)))$.

Démonstration. Soit A_j un coefficient de P tel que $A_j X^j$ contient D_X comme composante connexe. Soit A une composante connexe de A_j telle que $A X^j$ contient D_X comme composante connexe. Nous déduisons donc que $\ell(D_X) = \text{lcm}(\ell(A), \ell(X_{i+1}))$

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

via un raisonnement similaire à celui utilisé dans la preuve du Lemme 3.36. Plus généralement, pour chaque composante X_k entre X_{i+1} et X_{m_X} , il existe un entier a_k de $L(A_1 + \dots + A_m)$ tel que $\ell(P(X_k)) = \text{lcm}(a_k, \ell(X_k))$. Et la minimalité de X_{i+1} induit que $\text{lcm}(\ell(A), \ell(X_{i+1})) \leq \text{lcm}(a_k, \ell(X_k))$. Maintenant, comme par définition $\text{lcm}(a_k, \ell(X_k)) \leq \text{lcm}(a, \ell(X_k))$ pour tout entier a de $L(A_1 + \dots + A_m)$, il en résulte que $\text{lcm}(\ell(A), \ell(X_{i+1})) \leq \text{lcm}(a, \ell(X_k))$. Enfin, comme $\text{lcm}(a, \ell(X_k)) \leq \text{lcm}(a, \ell(X_k), x)$ pour tout entier $x > 0$ l'affirmation est vérifiée. $\square$

Le lemme suivant explique comment nous pouvons retrouver le plus petit arbre d'unroll d'une composante connexe de X à partir de l'évaluation d'un polynôme P .

Lemme 3.45. *Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme de FDDS sans terme constant. Soit $X = X_1 + \dots + X_{m_X}$ un FDDS trié selon l'ordre $\leq_P$. Soit $i \geq 0$ un entier. Alors, $\min(\mathcal{U}(X_{i+1}))$ est égal à l'arbre d'unroll minimal de $\Lambda_{\ell(P(X)-P(\pi_i(X))}(X - \pi_i(X))$.*

Démonstration. Soit $\mathbf{t}$ l'arbre d'unroll minimal de $\Lambda_{\ell(X_{i+1})}(X - \pi_i(X))$. Soit Y une composante connexe de $\Lambda_{\ell(X_{i+1})}(X - \pi_i(X))$ qui admet $\mathbf{t}$ comme arbre d'unroll. Montrons que $Y = X_{i+1}$. Soient D_X et D_Y les plus petites composantes connexes, respectivement, de $P(X_{i+1})$ et $P(Y)$ selon $\leq_{ct}$. Par définition X_{i+1} est la plus petite composante connexe de $\Lambda_{\ell(X_{i+1})}(X - \pi_i(X))$ selon $\leq_P$. Notons que $\ell(D_X) = \ell(P(X) - P(\pi_i(X)))$ par le Lemme 3.44. Ainsi, $X_{i+1} \leq_P Y$ et donc $D_X \leq_{ct} D_Y$. Montrons que $D_X = D_Y$.

Soit A_j un coefficient de P tel que $A_j X^j$ contient D_X comme composante connexe. Soit A une composante connexe de A_j telle que $A X^j$ contient D_X comme composante connexe. Prouvons que $\ell(D_X) = \ell(D_Y)$. Nous avons donc que $\ell(A)$ est un diviseur de $\ell(D_X)$. Et comme $\ell(Y)$ est un diviseur de $\ell(X_{i+1})$ qui est aussi un diviseur de $\ell(D_X)$, nous déduisons que $\text{lcm}(\ell(Y), \ell(A)) \leq \ell(D_X)$. De plus, la minimalité de D_Y selon $\leq_{ct}$ induit que $\ell(D_Y) \leq \text{lcm}(\ell(Y), \ell(A))$. Donc de la définition de $\leq_{ct}$, l'affirmation s'ensuit. De plus, chaque composante connexe de $A Y^i$ a pour longueur de cycles $\ell(D_Y)$.

Démontrons maintenant que $\min(\mathcal{U}(D_X)) = \min(\mathcal{U}(D_Y))$. Observons que, dû au Lemme 2.33, le plus petit arbre d'unroll de D_X est $\min(\mathcal{U}(A)) \min(\mathcal{U}(X_{i+1}))^j$. Donc étant donné que $\mathbf{t} \leq \min(\mathcal{U}(X_{i+1}))$ et puisque l'ordre sur les arbres est compatible avec le produit, nous déduisons que $\min(\mathcal{U}(A)) \mathbf{t}^j \leq \min(\mathcal{U}(A)) \min(\mathcal{U}(X_{i+1}))^j$. Et étant donné que $A Y^j$ contient une composante connexe qui admet $\min(\mathcal{U}(A)) \mathbf{t}^j$ comme arbre d'unroll, il en résulte que $\min(\mathcal{U}(D_Y)) \leq \min(\mathcal{U}(D_X))$. Ainsi, la définition de $\leq_{ct}$ implique que $\min(\mathcal{U}(D_Y)) = \min(\mathcal{U}(D_X))$ et donc $D_X = D_Y$.

De là, comme $X_{i+1} \leq_P Y$ et puisque $D_X = D_Y$, nous inférons que $X_{i+1} \leq_{ct} Y$. Or comme $\ell(Y)$ est un diviseur de $\ell(X_{i+1})$, il s'ensuit que X_{i+1} et Y ont la même longueur de cycles. De fait, $\min(\mathcal{U}(X_{i+1})) \leq \mathbf{t}$. Et alors la minimalité de $\mathbf{t}$ induit que $\min(\mathcal{U}(X_{i+1})) = \mathbf{t}$ d'où le lemme. $\square$

Ainsi, en associant les Lemmes 3.36 et 3.45, nous obtenons le résultat suivant.

Lemme 3.46. *Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme pseudo-injectif de graine g et soit B un FDDS. S'il existe $X = X_1 + \dots + X_{m_X}$ (où tous les X_i sont connexes) une solution de $P(X) = B$ trié selon $\leq_P$ alors la composante $i + 1$:*

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

1. *a son arbre d'unroll minimal, de plus petite période p , qui est égal à celui de $\Lambda_{\ell(B_I)}(X - \pi_i(X))$ où $B_i = B - P(\pi_i(X))$, et*
2. *une longueur de cycles égale à $q \cdot \text{lcm}(\text{alcm}_g B_i, p)$ avec $q > 0$ un diviseur de $\ell(B_i) / \text{lcm}(\text{alcm}_g B_i, p)$ tel que $\gcd(q, \text{alcm}_g B_i) = 1$.*

Démonstration. Supposons qu'il existe un FDDS X tel que $P(X) = B$. Sans perte de généralité, nous supposons de plus que $X = X_1 + \dots + X_{m_X}$, où chaque X_i est connexe, et qu'il est trié selon $\leq_P$. Nous allons montrer le lemme par récurrence forte sur la longueur du préfixe.

Soit $I \geq 0$ un entier. Supposons que le lemme soit vrai pour tout entier $0 \leq i \leq I$. Montrons alors qu'il est vrai pour tout $0 \leq i \leq I + 1$. Pour cela, nous n'avons besoin que de montrer qu'il est vrai pour $I + 1$.

Par le Lemme 3.45, nous avons que $\min(\mathcal{U}(X_{I+1}))$ est l'arbre d'unroll minimal de $\Lambda_{\ell(B_I)}(X - \pi_I(X))$. Donc X_{I+1} doit avoir une longueur de cycles qui est un multiple de p la période de cet arbre. En outre, par le Lemme 3.44, nous avons que $\ell(P(X_{I+1})) = \ell(B_I)$. De ce fait, par un raisonnement similaire à celui utilisé dans le Lemme 3.36, nous inférons que $\ell(B_I) = \text{lcm}(g, \ell(X_{I+1}))$. Ceci induit que $\ell(X_{I+1})$ est un multiple de $\text{alcm}_g B_I$. Nous déduisons donc que $\ell(X_{I+1}) = q \cdot \text{lcm}(p, \text{alcm}_g B_I)$. Nous en concluons que q est un diviseur de $\ell(B_I) / \text{lcm}(p, \text{alcm}_g B_I)$ et donc, de la définition de alcm , il en résulte que q est co-premier avec $\text{alcm}_g B_I$. Car, dans le cas contraire, $q \cdot \text{lcm}(p, \text{alcm}_g B_I)$ n'est pas un diviseur de g . $\square$

Grâce à ce lemme, nous sommes en mesure d'étendre le Lemme 3.37 comme suit :

Lemme 3.47. *Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme pseudo-injectif de graine g et B un FDDS. Soit $X = X_1 + \dots + X_{m_X}$, où tous les X_i sont connexes, trié selon $\leq_P$. Alors, $P(X) = B$ si et seulement si $P(X - X_i + dD) = B$ pour tout diviseur d de q , où q est un diviseur de $\ell(B') / \text{lcm}(\text{alcm}_g B', p)$, avec $B' = B - P(\pi_{i-1}(X))$, et X_i et D sont deux composantes connexes telles que :*

1. *ces deux composantes connexes ont le même arbre d'unroll minimal de plus petite période p ;*
2. *la longueur du cycle de X_i est $q \cdot \text{lcm}(\text{alcm}_g B', p)$;*
3. *la longueur du cycle de D est $(q/d) \cdot \text{lcm}(\text{alcm}_g B', p)$.*

Démonstration. Soit A une composante connexe appartenant à l'un des coefficients non-constants de P et soit a sa longueur de cycles. Considérons X_i et D comme dans l'énoncé. Pour montrer le lemme, il suffit de prouver que $AX_i^k = A(dD)^k$ pour tout entier $k > 0$. En effet, si cette affirmation est vraie, alors $AX_i^k Y = A(dD)^k Y$ pour tout entier $k > 0$ et tous les FDDS Y . Et donc tous les termes du polynôme qui dépendent de X_i seront égaux à ceux qui dépendent de dD .

Comme X_i et D ont le même arbre d'unroll minimal (condition 1), que X_i et D sont connexes (conditions 2 et 3) et puisque $\mathcal{U}(X)$ est un morphisme, il s'ensuit que $\mathcal{U}(X_i^k) = \mathcal{U}(X_i)^k = (d\mathcal{U}(D))^k = \mathcal{U}((dD)^k)$. Donc $\mathcal{U}(AX_i^k) = \mathcal{U}(A \times (dD)^k)$. Notons aussi que la connexité de X_i et D induit que toutes les composantes connexes de X_i^k

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

et $(dD)^k$ ont la même longueur de cycle, à savoir respectivement $\ell(X_i)$ et $\ell(D)$. De surcroît, comme A est également connexe, nous inférons que toutes les composantes connexes de AX_i^k ont un cycle de longueur $\text{lcm}(a, \ell(X_i)) = \ell(AX_i)$ tandis que celles de $A(dD)^k$ ont un cycle de longueur $\text{lcm}(a, \ell(D)) = \ell(A(dD))$. Il ne reste plus qu'à démontrer que $\ell(AX_i) = \ell(A \times (dX_c))$ pour conclure. Autrement dit, il nous faut montrer que $\text{lcm}(a, q \cdot \text{lcm}(p, \text{alcm}_g B')) = \text{lcm}(a, (q/d) \cdot \text{lcm}(p, \text{alcm}_g B'))$.

Observons que par construction, $q \cdot \gcd(p, a)$ et tous ses diviseurs sont co-premiers avec $\text{lcm}(p, \text{alcm}_g B') / \gcd(p, a)$ et de plus $q \cdot \gcd(p, a)$ est un diviseur de g donc de a . Nous en déduisons que :

$$\begin{aligned} \text{lcm}\left(a, \frac{q}{d} \text{lcm}(p, \text{alcm}_g B')\right) &= \text{lcm}\left(a, \frac{q}{d} \cdot \gcd(p, a) \cdot \frac{\text{lcm}(p, \text{alcm}_g B')}{\gcd(p, a)}\right) \\ &= \text{lcm}\left(\text{lcm}\left(a, \frac{q}{d} \cdot \gcd(p, a)\right), \text{lcm}\left(a, \frac{\text{lcm}(p, \text{alcm}_g B')}{\gcd(p, a)}\right)\right) \\ &= \text{lcm}\left(\text{lcm}(a, q \cdot \gcd(p, a)), \text{lcm}\left(a, \frac{\text{lcm}(p, \text{alcm}_g B')}{\gcd(p, a)}\right)\right) \\ &= \text{lcm}\left(a, q \cdot \gcd(p, a) \cdot \frac{\text{lcm}(p, \text{alcm}_g B')}{\gcd(p, a)}\right) \\ &= \text{lcm}(a, q \cdot \text{lcm}(p, \text{alcm}_g B')) \quad \square \end{aligned}$$

En utilisant les Lemmes 3.46 et 3.47, nous déduisons qu'en remplaçant la ligne 5 de l'Algorithme 3.38 par :

5.1 $\mathcal{U}(Y) \leftarrow \text{SolveurPolyUnroll}(\sum_{i=1}^m \Lambda_{\ell(B)}(A_i) X^i, \Lambda_{\ell(B)}(B))$ (Algorithme 2.56),

5.2 $\mathbf{x} \leftarrow \min(\mathcal{U}(Y) - \mathcal{U}(\Lambda_{\ell(B)}(X)))$,

5.3 trouver p la période minimale de $\mathbf{x}$,

5.4 créer D l'enroulement de $\mathbf{x}$ à période $\text{lcm}(p, \text{alcm}_g B)$,

nous obtenons un algorithme polynomial puisque les nouvelles lignes peuvent être exécutées efficacement. De surcroît, les Lemmes 3.46 et 3.47 permettent de généraliser la Proposition 3.40 et ainsi obtenir le théorème suivant.

Théorème 3.48. *Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme pseudo-injectif et soit B un FDDS. Alors nous pouvons résoudre $P(X) = B$ en temps polynomial sur la somme des tailles des coefficients de P et de celle de B .*

Notons qu'une conséquence directe des Lemmes 3.46 et 3.47 est que les équations considérées dans cette section ont une unique solution qui maximise le nombre de composantes connexes (celle qui est renvoyée par l'algorithme décrit ci-dessus). Puisque cette solution minimise la longueur des cycles qu'elle contient, nous l'appelons *solution canonique relaxée*. Cette solution est le pendant de la solution composante-maximale que nous avons définies pour les unrolls. Ces deux solutions peuvent être différentes, puisque nous ne faisons pas nécessairement les enroulements des arbres aux mêmes périodes.

3.2.2. Équation sur les permutations encodée par des entiers

Dans cette partie, nous réexaminons les équations sur les permutations mais en changeant l'encodage. En effet, jusqu'à maintenant les permutations étaient représentées explicitement via leur graphe (ce qui est équivalent à encoder le nombre d'états en unaire) afin de conserver le point de vue dynamique. Cependant, ces objets sont codables de façon plus réduite. Par exemple, nous faisons le choix ici de représenter une permutation A comme un tableau de couples (n, l) où l est la longueur d'un cycle et n est la multiplicité de C_l dans A . Ainsi, $A[i].l$ décrira la longueur du i -ème cycle de A tandis que $A[i].n$ sera son nombre d'occurrences. L'Exemple 3.49 illustre ce nouvel encodage.

Exemple 3.49. *Considérons la permutation $4C_2 + 7C_{12} + 8C_6$. Alors, nous créons un tableau T de trois cases tel que $T[0] = (4, 2)$, $T[1] = (7, 12)$ et $T[2] = (8, 6)$.*

Avec cet encodage, nous pouvons réaliser le produit de deux permutations A et B , encodées par des tableaux de longueur respective c_A et c_B , en créant un tableau de longueur $c_A \cdot c_B$ où la case (i, j) contient le produit de $A[i]$ avec $B[j]$. Ce dernier est égal à (n, l) avec $n = A[i].n \cdot B[j].n \cdot \gcd(A[i].l, B[j].l)$ et $l = \text{lcm}(A[i].l, B[j].l)$. Une fois ceci fait, nous trions ce tableau en fonction des longueurs de cycle et créons un nouveau tableau, possiblement plus petit, dans lequel nous regroupons chaque case contenant la même composante l en sommant leurs coefficients n .

Dans la suite de cette section, nous supposons que toutes les permutations sont encodées comme décrit ci-dessus; et nous prouverons que les équations de la forme $P(X) = B$, avec P un polynôme pseudo-injectif sur les permutations, sont solvables efficacement avec ce nouvel encodage. Notons que nous nous autorisons à pouvoir avoir $B[i].n = 0$ pour certains i au cours de nos calculs. Pour cela, nous introduisons en premier lieu une adaptation de l'Algorithme 3.28 afin de le rendre compatible avec ce nouvel encodage. Cette version sera très proche de l'original mais nous choisissons de la rendre aussi explicite que possible pour en analyser précisément la complexité. Elle est donnée dans l'Algorithme 3.50. Ce dernier prend en entrées deux permutations A et B avec A pseudo-annulable, dans le nouvel encodage, et renvoie une permutation X dans cet encodage, si elle existe, telle que $AX = B$.

Constatons que la preuve du Lemme 3.34 permet aussi de montrer la correction de l'Algorithme 3.50. Il nous reste donc à étudier sa complexité. Cependant, pour réaliser cette analyse, nous devons expliquer comment nous effectuons l'instruction 6, à savoir le calcul du $\text{alcm}_a b$. Nous avons déjà démontré que nous sommes en mesure de calculer $\text{alcm}_a b$ en temps polynomial sur $\log_2 b$ (et donc de $\log_2 a$ car a doit diviser b). Nous proposons toutefois une méthode plus efficace que le calcul de $\gcd((b/a)^{\log_2 b}, b)$. Cette procédure (Algorithme 3.51) est similaire à celle de l'Algorithme 8 de [46] et de l'Algorithme 2 de [13]. Le tableau 3.52 présente un exemple d'exécution de l'Algorithme 3.51.

Lemme 3.53. *Le calcul de $\text{alcm}_a b$ est réalisable avec un temps en $\mathcal{O}(\log b(\log \log b))$ si nous admettons que les quatre opérations arithmétiques de base (addition, soustraction, multiplication, division) sur les entiers sont réalisables en temps constant.*

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes
“pseudo-injectifs”

Algorithme 3.50 $\text{SolveurPseudoPolyPermutation}(A, B)$

```

1: trier  $A$  et  $B$  en ordre croissant des longueurs de cycle,
2: Pour  $min$  de 0 au nombre de cases de  $B$  moins 1 faire
3:   Si  $|A| > |B|$  alors
4:     renvoyer  $\perp$ ,
5:   Si  $B[min].n > 0$  alors
6:      $p \leftarrow \text{alcm}_{A[0].l}(B[min].l)$ ,
7:     trouver  $d$  la multiplicité du cycle de longueur  $B[min].l$  dans le produit  $AC_p$ ,
8:     calculer  $e = B[min].n/d$  puis construire  $D$  la permutation  $(e, p)$ ,
9:     Si  $AD$  n'est pas un sous-multi-ensemble de  $B$  alors
10:      renvoyer  $\perp$ ,
11:       $X \leftarrow X + D$ ,
12:      retirer  $AD$  de  $B$ , c'est-à-dire soustraire les  $(AD)[i].n$  à  $B[j].n$  si  $(AD)[i].l = B[j].l$ ,
13: renvoyer  $X$ .
```

Algorithme 3.51 $\text{alcm}_b a$

```

1:  $res_1 \leftarrow b/a$ ,
2:  $res_2 \leftarrow 0$ ,
3: Tant que  $res_1 \neq res_2$  faire
4:    $res_2 \leftarrow res_1$ ,
5:    $pow \leftarrow res_1^2$ ,
6:    $res_1 \leftarrow \text{gcd}(b, pow)$ ,
7: renvoyer  $res_1$ .
```

res_1	res_2	pow
12	0	144
48	12	2304
78	48	589824
6144	768	37748736
6144	6144	–

TABLEAU 3.52. – Table des valeurs des trois variables de l'Algorithme 3.51 au cours du calcul de $\text{alcm}_a b$ avec $b = 43008$ et $a = 3584$. Chaque ligne correspond à une itération de la boucle.

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

Démonstration. Soient $\prod_{i=1}^{\infty} p_i^{a_i}$ et $\prod_{i=1}^{\infty} p_i^{b_i}$ les décompositions en facteurs premiers de respectivement a et b . Comme a divise b , nous avons que $a_i \leq b_i$ pour tout entier $i \geq 1$. Montrons que res_1 (Algorithme 3.51) contient bien le $\text{alcm}_a b$ à la fin de l'algorithme. En symbole :

$$res_1 = \prod_{\substack{i=1 \\ 0 \leq a_i < b_i}}^{\infty} p_i^{b_i}$$

Nous réalisons cette preuve par récurrence sur le nombre d'itérations de la boucle. Plus précisément, nous démontrons qu'à la fin de la n -ème itération de la boucle, res_1 est égale à :

$$\prod_{\substack{i=1 \\ b_i \leq 2^n(b_i - a_i)}}^{\infty} p_i^{b_i} \prod_{\substack{i=1 \\ 0 \leq 2^n(b_i - a_i) < b_i}}^{\infty} p_i^{2^n(b_i - a_i)}$$

Comme avant la première itération de la boucle, res_1 contient b/a , la propriété est vérifiée. Soit $n \geq 0$ un entier. Supposons que la propriété est vraie pour n . Démontrons alors qu'elle est vraie pour $n + 1$. Par hypothèse de récurrence, au commencement de la $(n + 1)$ -ème itération, res_1 est égale à :

$$\prod_{\substack{i=1 \\ b_i \leq 2^n(b_i - a_i)}}^{\infty} p_i^{b_i} \prod_{\substack{i=1 \\ 0 \leq 2^n(b_i - a_i) < b_i}}^{\infty} p_i^{2^n(b_i - a_i)}.$$

Donc la valeur de pow est :

$$\prod_{\substack{i=1 \\ b_i \leq 2^n(b_i - a_i)}}^{\infty} p_i^{2b_i} \prod_{\substack{i=1 \\ 0 \leq 2^n(b_i - a_i) < b_i}}^{\infty} p_i^{2^{n+1}(b_i - a_i)}.$$

Relevons que nous pouvons décomposer cette dernière comme suit :

$$\prod_{\substack{i=1 \\ b_i \leq 2^n(b_i - a_i)}}^{\infty} p_i^{2b_i} \prod_{\substack{i=1 \\ 2^n(b_i - a_i) < b_i \leq 2^{n+1}(b_i - a_i)}}^{\infty} p_i^{2^{n+1}(b_i - a_i)} \prod_{\substack{i=1 \\ 0 \leq 2^{n+1}(b_i - a_i) < b_i}}^{\infty} p_i^{2^{n+1}(b_i - a_i)}.$$

Cela nous permet de conclure que $\text{gcd}(b, pow)$ est égal à :

$$\prod_{\substack{i=1 \\ b_i \leq 2^{n+1}(b_i - a_i)}}^{\infty} p_i^{b_i} \prod_{\substack{i=1 \\ 0 < 2^{n+1}(b_i - a_i) < b_i}}^{\infty} p_i^{2^{n+1}(b_i - a_i)}$$

et l'hérédité s'ensuit.

Soit j l'entier tel que $b_j \geq b_i$ pour tout entier $i \geq 1$. En reprenant la preuve du Lemme 3.27, nous déduisons qu'après $n = \log b_j$ nous avons que $2^n \log b_j \geq 2^n(b_i - a_i) \geq b_j$. Ce qui induit que la valeur de res_1 est celle souhaitée. Notons qu'à la fin de la $(n + 1)$ -ème itération de la boucle, la valeur de res_1 est la même que celle à la fin de la n -ème itération. Ce qui induit que la boucle s'arrête et que res_1 contient la bonne

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

valeur.

Pour la complexité, nous relevons de la preuve par récurrence précédente que res_1 est toujours inférieur ou égal à b au cours des itérations de la boucle. Donc le temps de calcul du corps de la boucle est en $\mathcal{O}(\log b)$. En outre, comme dit précédemment, le nombre d'itérations peut être borné par $\log_2 b_j + 1$. Or $b \geq p_j^{b_j} \geq 2^{b_j}$ impliquant que $b_j + 1$ est borné par $\log_2 b + 1$. Nous concluons alors que la complexité de la boucle est $\mathcal{O}((\log_2 b)(\log \log b))$. D'où le lemme. $\square$

Lemme 3.54. *Le temps de calcul de l'Algorithme 3.50 est :*

$$\mathcal{O}(c^2 \cdot \log(cd) + c \cdot (\log d)(\log \log d))$$

où c est la longueur du tableau encodant A plus celle du tableau encodant B , et d est la longueur du plus grand cycle de A et B .

Démonstration. Tout d'abord, les opérations hors de la boucle de l'Algorithme 3.50 sont toutes réalisables avec un temps en $\mathcal{O}(c \log c)$. Il nous reste donc à analyser la complexité de la boucle. Pour cela, remarquons que le nombre d'itérations de cette dernière est borné par la longueur du tableau codant B , d'où par c . Ceci induit qu'il nous suffit de multiplier le temps de calcul du corps de la boucle par c .

Nous l'analysons instruction par instruction. La ligne 3 a une complexité en $\mathcal{O}(c)$ car ce n'est que la somme des $n \cdot l$ présents dans A et B . L'instruction 6 a un temps d'exécution en $\mathcal{O}((\log d)(\log \log d))$ par le Lemme 3.53. Le calcul de AC_p prend un temps en $\mathcal{O}(c \log d)$. En effet, nous réaliserons au plus c calculs de lcm, de gcd (ainsi que quelques sommes et produits) de nombres inférieurs ou égaux à d . De là, la complexité du tri de AC_p est en $\mathcal{O}(c \log c)$ via un tri fusion. Finalement, nous regroupons tous les cycles qui ont la même longueur dans AC_p via un simple parcours du tableau trié, donc en $\mathcal{O}(c)$. Ainsi, pour trouver d , nous réalisons une recherche dichotomique du tableau précédent afin de localiser $B[\min].l$, induisant que la ligne 7 est réalisable avec un temps en $\mathcal{O}(c \log(cd))$.

L'étape 8 est réalisable en temps constant. Tandis que l'instruction 9 s'exécute en un temps en $\mathcal{O}(c \log(cd))$. En effet, comme déjà expliqué, le calcul puis le tri de AD est en $\mathcal{O}(c \log(cd))$. Après quoi la vérification que AD est un sous-multi-ensemble de B est en $\mathcal{O}(c)$. La ligne 11 est en temps en $\mathcal{O}(c)$ tandis que l'étape 12 est en $\mathcal{O}(c \log(cd))$ pour les mêmes raisons que pour 9. Finalement, l'instruction 13 est en $\mathcal{O}(c)$ (c'est un simple parcours de B). Nous en concluons que le résultat s'ensuit. $\square$

Une conséquence du Lemme 3.54 est que nous sommes en mesure d'améliorer le temps de calcul de l'Algorithme 3.28 si A est pseudo-annulable quand les permutations sont données explicitement en convertissant les entrées avec l'encodage proposé ci-dessus.

Corollaire 3.55. *Soient A et B des permutations telles que A est pseudo-annulable. Si A et B sont données en entrée par leur graphe explicite alors le calcul d'un X tel que $AX = B$ peut être réalisé avec une complexité en $\mathcal{O}(m \log m)$ avec m la somme des tailles de A et de B .*

3. Résolution d'équations polynomiales de FDDS – 3.2. Le cas des polynômes “pseudo-injectifs”

Démonstration. Transformer des permutations encodées par des graphes en des permutations encodées par des tableaux triés est réalisable en parcourant les graphes pour trouver les longueurs de cycles ($\mathcal{O}(m)$) puis en triant la liste ainsi obtenue ($\mathcal{O}(m \log m)$) puis en regroupant les cases du tableau qui contiennent la même longueur de cycle ($\mathcal{O}(m)$). Donc le temps de calcul de la traduction entre les deux encodages est en $\mathcal{O}(m \log m)$.

Notons que le nombre d'éléments c du tableau est en $\mathcal{O}(\sqrt{m})$. En effet, la plus petite somme de cycles avec c cycles différents a $\sum_{i=1}^c i = c(c+1)/2$ états. De là, par le Lemme 3.54, nous concluons que nous trouvons la solution X avec une complexité en $\mathcal{O}((\sqrt{m})^2 \log(\sqrt{m}m) + \sqrt{m}(\log m)(\log \log m)) = \mathcal{O}(m \log m)$. $\square$

Pour propager l'Algorithme 3.50 aux équations $P(X) = B$ où $P = \sum_{i=0}^m A_i X^i$ est un polynôme pseudo-injectif sur les permutations et B une permutation avec notre encodage compact, nous devons expliciter comment représenter ces équations. Une première approche serait de voir P comme un tableau de $m+1$ cases (m est le degré de P). La case i de ce tableau contiendrait alors la permutation A_i encodée par un tableau. Avec cet encodage, le tableau pourrait avoir une taille inutilement longue. Par exemple, l'équation $AX^k = B$ requerrait k cases qui contiendraient la permutation vide.

Dans l'encodage que nous utilisons, le polynôme P sera toujours exprimé sous la forme d'un tableau mais chaque case contiendra un couple (i, A_i) avec i le degré associé à X et A_i une permutation non-vide. Nous voyons bien que cette stratégie résout le problème évoqué ci-dessus. Nous formulons donc l'Algorithme 3.56 qui prendra en entrée un polynôme pseudo-injectif P sur les permutations avec le nouvel encodage ainsi qu'une permutation B . Cette procédure renvoie une permutation X telle que $P(X) = B$ s'il en existe une.

Théorème 3.57. *Soient P un polynôme pseudo-injectif sur les permutations et B une permutation. Alors, l'équation $P(X) = B$ peut être résolue avec un temps de calcul en $\mathcal{O}(c^2 \cdot \log(cd) \cdot \log q + c(\log d)(\log \log d))$ où c est la somme des longueurs des tableaux représentant les permutations dans les coefficients de P et B , d la longueur du plus grand cycle de B et q la plus grande multiplicité dans B .*

Démonstration. Grâce à un raisonnement similaire à la Proposition 3.40, il s'ensuit que si l'algorithme renvoie X , alors $P(X) = B$. Supposons maintenant qu'il existe une solution à $P(X) = B$. Montrons alors que l'algorithme renvoie bien un X . De notre hypothèse, l'Algorithme 3.38 renvoie une solution $Y = \sum_{i=1}^{m_Y} y_i Y_i$ telle que Y_i est une composante connexe et que $P(Y_i) <_{ct} P(Y_{i+1})$ pour tout entier i entre 1 et $m_Y - 1$, autrement dit $Y_i <_P Y_{i+1}$ pour tous ces entiers i . Nous allons prouver par récurrence sur le nombre d'itérations de la boucle externe qu'à la fin de la i -ème itération de cette dernière, X contient $\mathcal{P}_i(Y)$. Notons que la propriété est bien vérifiée avant la première itération de la boucle.

Soit $i \geq 0$ un entier. Supposons que la propriété est vraie pour i . Montrons alors qu'elle est vraie pour $i+1$. Remarquons que de l'hypothèse de récurrence nous avons que $\ell(B - P(X)) = \ell(B - P(\mathcal{P}_i(Y)))$. Donc la valeur de C_p calculée à la $(i+1)$ -ème

Algorithme 3.56 SolveurPseudoPolyPermutation2(P, B)

```

1: Si  $A_0$  n'est pas un sous-multi-ensemble de  $B$  alors
2:   renvoyer  $\perp$ ,
3:  $min \leftarrow 0, B \leftarrow B - A_0, P \leftarrow P - A_0, A \leftarrow \sum_{i=1}^m A_i$  et  $X \leftarrow \emptyset$ ,
4: trier  $A$  et  $B$  en ordre croissant des longueurs de cycle,
5: Tant que  $min$  est inférieur au nombre de cases de  $B$  faire
6:   Si  $|A| > |B|$  alors
7:     renvoyer  $\perp$ ,
8:   Si  $B[min].n > 0$  alors
9:      $p \leftarrow \text{alcm}_{A[0].l}(B[min].l)$ ,
10:    réaliser une recherche dichotomique du nombre  $n$  de copies de  $C_p$  en
    prenant comme test d'infériorité  $B[min].n - r \neq 0$  avec  $r$  le nombre de copies du
    plus petit cycle de  $P(X + nC_p) - P(X)$  et comme test de supériorité de vérifier que
     $P(X + nC_p)$  n'est pas un sous-multi-ensemble de  $B$ ,
11:    Si  $n$  n'existe pas alors
12:      renvoyer  $\perp$ ,
13:       $X \leftarrow X + (n, p)$ ,
14:      retirer  $P(X + nC_p) - P(X)$  à  $B$ , c'est-à-dire soustraire les  $(AD)[i].n$  à  $B[j].n$ 
      si  $(AD)[i].l = B[j].l$ ,
15:       $min \leftarrow min + 1$ ,
16: renvoyer  $X$ .
```

itération est bien égale à Y_{i+1} . Il reste à démontrer que $n = y_{i+1}$ avec le n calculé lors de l'instruction 10. Ceci est induit du fait que cette instruction correspond à une recherche dichotomique de y_{i+1} . En effet, par hypothèse de récurrence :

$$P(X + nC_p) - P(X) = P(\mathcal{P}_i(Y) + nC_p) - P(\mathcal{P}_i(Y)).$$

En outre, par définition de Y , les cycles de longueur minimale de $B - P(\mathcal{P}_i(Y))$ sont issus d'un produit avec Y_{i+1} . Par conséquent, si $n > y_{i+1}$ alors nous avons que $P(\mathcal{P}_i(Y) + nC_p) - P(\mathcal{P}_i(Y))$ n'est pas un sous-multi-ensemble de $B - P(\mathcal{P}_{i+1}(Y))$ et si $n < y_{i+1}$ alors $B - (P(\mathcal{P}_i(Y) + nC_p) - P(\mathcal{P}_i(Y)))$ contient des cycles de longueur $\ell(B - P(\mathcal{P}_i(Y)))$ alors que ce n'est pas le cas de $B - P(\mathcal{P}_{i+1}(Y))$. L'hérédité et la correction de l'algorithme suivent.

Pour l'analyse de la complexité, notons que toutes les instructions en dehors de la boucle ont un temps d'exécution en $\mathcal{O}(c \log c)$. Le nombre d'itérations de la boucle est au plus c . Dans cette boucle, nous réaliserons une recherche dichotomique dont le nombre d'itérations est borné par $\log q$. De là, par un raisonnement similaire à celui de la preuve du Lemme 3.54, il en résulte que les opérations à l'intérieur de cette recherche ont un temps de calcul en $c \log(cd)$ et que les autres opérations de la boucle sont exécutables en temps $\mathcal{O}((\log d)(\log \log d))$. Donc le temps de calcul de l'algorithme est en $\mathcal{O}(c^2 \cdot \log(cd) \cdot \log q + c(\log d)(\log \log d))$. $\square$

3. Résolution d'équations polynomiales de FDDS – 3.3. Conclusion et perspectives

De nouveau, nous pouvons utiliser cet algorithme pour produire une solution à $P(X) = B$ où les graphes sont donnés en forme explicite, avec P un polynôme pseudo-injectif de FDDS et B un FDDS. En effet, si nous notons m la somme des tailles des coefficients de P et de celle de B , nous avons que le degré du polynôme requiert d'être au plus $\log m$ et ce test peut être réalisé en temps linéaire sur m . De là, nous avons que la valeur de c de l'énoncé du Théorème 3.57 est bornée par :

$$(1 + \log m) \sqrt{m / (1 + \log m)} = \sqrt{m(1 + \log m)}.$$

Donc, en bornant la valeur de d de ce même énoncé par m , il s'ensuit que :

$$c^2 \log(cd) \leq m(\log m)(\log \sqrt{m^3(1 + \log m)}) \leq m(\log m)(\log m^4) = m(\log m)^2.$$

De plus, nous avons que :

$$c(\log d)(\log \log d) \leq \sqrt{m(1 + \log m)}(\log m)(\log \log m) \leq m(\log m)^2.$$

Ainsi, nous déduisons le corollaire suivant.

Corollaire 3.58. *Soient P un polynôme pseudo-injectif sur les permutations et B une permutation. Si P et B sont donnés en entrée par leur graphe explicite alors le calcul d'un X tel que $P(X) = B$ peut être réalisé avec une complexité en $\mathcal{O}(m(\log m)^2)$ avec m la somme des tailles des coefficients de P et de celle de B .*

3.3. Conclusion et perspectives

Dans ce chapitre, nous avons caractérisé les polynômes sur les FDDS qui sont injectifs. Nous avons également développé un algorithme polynomial permettant de résoudre les équations de la forme $P(X) = B$ si P est injectif. Par la suite, nous avons étendu ce résultat au cas où P est pseudo-injectif.

Ces résultats ouvrent la voie à plusieurs pistes de recherche différentes. La première serait de passer d'une équation à un système d'équations où tous les polynômes seraient pseudo-injectifs. C'est-à-dire les systèmes de la forme

$$\left\{ \begin{array}{l} P_1(X) = B_1 \\ P_2(X) = B_2 \\ \vdots \\ P_n(X) = B_n \end{array} \right.$$

avec $P_1, \dots, P_n$ des polynômes pseudo-injectifs. Ceci nous permettrait d'améliorer le pipeline introduit dans [14, 46] et rappelé dans la Section 1.2. Cela pourrait nous permettre de prouver que résoudre $P(X) = B$ est réalisable en temps polynomial si nous fixons le nombre de composantes connexes de B et, par la même occasion, nous

3. Résolution d'équations polynomiales de FDDS – 3.3. Conclusion et perspectives

prouverions que résoudre $Q(X_1, \dots, X_n) = B$ est faisable en temps polynomial, pour un certain polynôme multivarié Q , si le nombre de composantes connexes de B et n sont fixés.

Une autre piste serait d'étudier les équations de la forme $P(X) = Q(X)$ avec P et Q des polynômes pseudo-injectifs. Nous pourrions aussi plus simplement étudier les équations de la forme $P(X) = Q(X)$ avec P et Q des polynômes de FDDS quelconques. En effet, si le problème est décidable, car l'équation sur les tailles associée admet un nombre fini de solutions, aucun résultat de difficulté n'a encore été donné pour ce cas.

Nous pouvons également nous demander si nous sommes à même d'énumérer l'ensemble des solutions de l'équation $P(X) = B$ avec P un polynôme pseudo-injectif avec un délai polynomial. Nous répondrons à cette question par l'affirmative dans le Chapitre 5. Cette question a un attrait particulier à l'aune de la conjecture suivante.

Conjecture 3.59. *Les polynômes admettant le plus d'antécédents ayant la même image sont parmi les polynômes pseudo-injectifs.*

Un autre axe possible est d'étendre les travaux de la Section 3.2.2. C'est-à-dire d'étudier la résolution des équations de la forme $P(X) = B$ où les FDDS sont codés de manière succincte, par exemple par un circuit booléen. Nous pensons pouvoir prouver que, dans ce cas, décider de l'existence d'une solution à $P(X) = B$ est PSPACE-difficile car nous croyons être en mesure de démontrer que l'isomorphisme de graphe fonctionnel (le cas $A = B$) encodé par des circuits booléens est complet pour la classe PSPACE. Cependant, la question de l'appartenance à PSPACE reste encore pleinement ouverte.

Finalement, une possible voie de recherche est d'étudier si le problème de résoudre $P(X) = B$ peut se réduire à la résolution de $Q(X) = B'$ avec Q le polynôme construit en ne conservant que les cycles des composantes connexes des coefficients de P et B' la permutation induite par B de la même manière. Plus précisément, nous voudrions savoir s'il est possible de construire une solution de $P(X) = B$ à partir d'une solution de $Q(X) = B'$ en temps polynomial.

4. Résolution d'équations sur les supports de FDDS

Sommaire

4.1. Complexité de l'évaluation des polynômes sur les supports	136
4.2. Équation linéaire sur les supports	142
4.2.1. Résolution des équations linéaires sur les supports	142
4.2.2. Énumération des solutions des équations linéaires sur les supports	148
4.2.3. Approche alternative : décomposition des équations	154
4.3. Des équations linéaires sur les supports à celles sur les FDDS	157
4.4. Conclusion et perspectives	159

Dans le chapitre précédent, nous avons vu comment résoudre les équations de la forme $P(X) = B$, avec certaines contraintes sur P . Cependant, les techniques que nous avons utilisées ne semblent pas pouvoir être étendues au cas où P serait général. Dans l'optique de développer une procédure plus systématique, qui fonctionnerait sans contrainte sur P , nous proposons de séparer ce problème en deux sous-problèmes distincts. En effet, la résolution de $P(X) = B$ peut être vue comme la recherche d'un X tel que l'ensemble des composantes connexes (sans les multiplicités) de $P(X)$ est égal à celui de B , puis la recherche des multiplicités des composantes connexes de X pour que $P(X) = B$. Pour analyser le premier de ces deux problèmes, nous introduisons la définition suivante.

Définition 4.1 (Support d'un FDDS). Soit $A = \sum_{i=1}^{m_A} a_i A_i$ un FDDS tel que chaque A_i est connexe et si $i \neq j$ alors $A_i \neq A_j$ pour tous i, j . Nous définissons le *support* de A , noté $\mathcal{S}(A)$, comme l'ensemble $\{A_1, \dots, A_{m_A}\}$.

Soit $\mathcal{S}(\mathbb{D})$ l'ensemble des supports des FDDS, en symboles $\mathcal{S}(\mathbb{D}) = \{\mathcal{S}(A) \mid A \in \mathbb{D}\}$. Si la fonction $\mathcal{S}(X)$ est une fonction de $\mathbb{D}$ dans lui-même, ce n'est toutefois ni un morphisme du monoïde $(\mathbb{D}, +)$ dans lui-même, ni un morphisme du monoïde $(\mathbb{D}, \times)$ dans lui-même (et de fait non plus un morphisme de $(\mathbb{D}, +, \times)$ dans lui-même). En effet, $\mathcal{S}(C_2 + C_2) = \mathcal{S}(C_2 \times C_2) = C_2$ mais $\mathcal{S}(C_2) + \mathcal{S}(C_2) = \mathcal{S}(C_2) \times \mathcal{S}(C_2) = 2C_2$. Cependant, nous voyons qu'il est possible d'adapter la somme et le produit de $\mathbb{D}$ pour définir un semi-anneau à partir de $\mathcal{S}(\mathbb{D})$. Pour cela, il suffit de prendre l'union, notée $\boxplus$ (et non l'union disjointe) comme somme et de définir que le résultat du produit, noté $\boxtimes$, n'est pas l'union disjointe des composantes qu'il engendre mais seulement l'union.

4. Résolution d'équations sur les supports de FDDS – 4.1. Complexité de l'évaluation des polynômes sur les supports

Définition 4.2. Soient A et B des FDDS. Alors la somme de $\mathcal{S}(A)$ et $\mathcal{S}(B)$, notée $\mathcal{S}(A) \boxplus \mathcal{S}(B)$, est le support tel que chaque composante connexe qu'il contient appartient soit à $\mathcal{S}(A)$ soit à $\mathcal{S}(B)$. En symboles, $\mathcal{S}(A) \boxplus \mathcal{S}(B) = \{C \mid C \in \mathcal{S}(A) \vee C \in \mathcal{S}(B)\}$.

Le produit de $\mathcal{S}(A)$ par $\mathcal{S}(B)$, noté $\mathcal{S}(A) \boxtimes \mathcal{S}(B)$, est donné par la formule $\mathcal{S}(\mathcal{S}(A) \times \mathcal{S}(B))$.

Proposition 4.3. L'ensemble des supports de FDDS muni de $\boxplus$ pour addition et $\boxtimes$ pour multiplication forme un semi-anneau commutatif et $\mathcal{S}(X)$ est un morphisme entre les semi-anneaux $(\mathbb{D}, +, \times)$ et $(\mathcal{S}(\mathbb{D}), \boxplus, \boxtimes)$.

Pour justifier que $\mathcal{S}(X)$ est un morphisme entre les semi-anneaux $(\mathbb{D}, +, \times)$ et $(\mathcal{S}(\mathbb{D}), \boxplus, \boxtimes)$, remarquons directement qu'une composante connexe appartient à $A + B$ si et seulement si elle appartient à A ou à B , donc $\mathcal{S}(X)$ se distribue sur $+$, et que $\mathcal{S}(A) \times \mathcal{S}(B)$ produit les mêmes composantes connexes que $A \times B$ mais avec des multiplicités différentes. De ce fait, savoir s'il existe X tel que $P(X)$ et B ont le même ensemble de composantes connexes revient à se demander s'il existe une solution à l'équation sur les supports $\boxplus_{i=0}^m \mathcal{S}(A_i) \mathcal{S}(X)^i = \mathcal{S}(B)$.

Dans ce chapitre, nous prouverons que décider de l'existence d'une solution de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$, avec P un polynôme multivarié sur les supports, est dans la classe NP. Et nous montrerons en outre que ce problème est NP-complet si P est non-linéaire (Section 4.1).

Nous prouverons par la suite que si P est linéaire alors nous sommes à même de résoudre $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ en temps polynomial (Section 4.2). Nous montrerons par la même occasion qu'énumérer l'ensemble des supports solutions possibles de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ peut être réalisé avec un délai polynomial (Théorème 4.33). Par ailleurs, nous bornerons la taille de l'ensemble des supports solutions (Théorème 4.39).

Finalement, nous expliquerons comment utiliser nos résultats pour démontrer que la résolution de $AX = B$ est faisable en temps polynomial si nous fixons le nombre de composantes connexes différentes de B et la taille du plus petit cycle de A (Section 4.3).

4.1. Complexité de l'évaluation des polynômes sur les supports

Soit $n > 0$ un entier. Soit P un polynôme sur les supports à n variables. Dans cette section, nous étudions le temps de calcul nécessaire pour réaliser $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$. Nous faisons le choix d'exprimer ce temps de calcul en fonction de m la taille du résultat et non sur celle des entrées comme à l'accoutumée. Pour justifier ce choix, remarquons que si X est un dendron alors le calcul de $\mathcal{S}(X^k)$ est nécessairement exponentiel en k car $|\mathcal{S}(X)^k| = |X^k| = |X|^k$ car nos graphes sont donnés en unaire.

Dans ce qui suit, nous introduirons un algorithme qui étant donné $\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)$ calculera $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ en temps polynomial sur mn fois le nombre de monômes constituant P . Ce résultat n'est pas aussi direct que dans le cas des FDDS. En effet, il existe une différence fondamentale entre les polynômes de FDDS et ceux sur

4. Résolution d'équations sur les supports de FDDS – 4.1. Complexité de l'évaluation des polynômes sur les supports

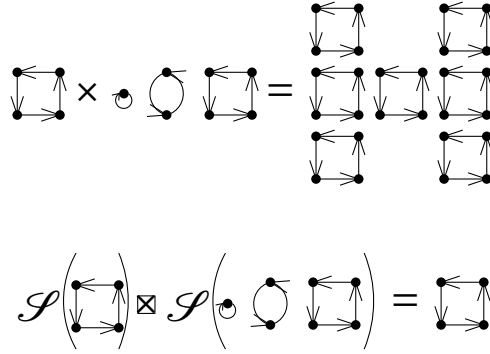


FIGURE 4.4. – Exemple où le produit de support a une taille inférieure à celle de ses termes.

les supports : la taille d'un polynôme de FDDS évalué en $X_1, \dots, X_n$ est supérieure ou égale à celle des $X_1, \dots, X_n$ mais cette propriété est fausse pour les polynômes de support. Un exemple de cette différence entre FDDS et support est donné dans la Figure 4.4.

Afin d'accomplir l'objectif de cette section, nous décomposons P comme une union de r monômes $M_1, \dots, M_r$ sur n variables, et nous démontrerons que la taille de chaque X_i est polynomiale sur la taille de $M_j(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ pour un certain j .

Lemme 4.5. *Soit X un FDDS et $k > 0$ un entier. Alors la taille de $\mathcal{S}(X^k)$ est inférieure ou égale à celle de $\mathcal{S}(X^{k+1})$.*

Démonstration. Pour cette démonstration, nous construisons une fonction injective $f : \mathcal{S}(X^k) \rightarrow \mathcal{S}(X^{k+1})$ telle que la taille de C est inférieure ou égale à celle de $f(C)$ pour toute composante connexe C de $\mathcal{S}(X^k)$. Ceci étant suffisant pour conclure la preuve car la taille de $\mathcal{S}(X^{k+1})$ est supérieure ou égale à celle de la somme des tailles des $f(C)$. La Figure 4.6 illustre la fonction que nous définissons.

Soit C une composante connexe de $\mathcal{S}(X^k)$, soit p sa longueur de cycle et soit D une composante connexe de $\Lambda_p(X)$ dont l'arbre d'unroll minimal est $\min(\mathcal{U}(\Lambda_p(X)))$. Notons que D est bien définie puisque p est le lcm de k éléments de $L(X)$. Soit Y la plus petite composante connexe de CD suivant l'ordre $\leq_{ct}$. Directement Y est une composante connexe de X^{k+1} .

Nous pouvons donc définir $f : \mathcal{S}(X^k) \rightarrow \mathcal{S}(X^{k+1})$ la fonction qui associe à chaque C de $\mathcal{S}(X^k)$ la composante Y décrite ci-dessus. Notons que $\ell(C) = \ell(f(C))$. Montrons que f est injective.

Soient Z_1, Z_2 deux composantes connexes de $\mathcal{S}(X^k)$ telles que $f(Z_1) = f(Z_2)$. Montrons que $Z_1 = Z_2$. Par le choix de f , les plus petites composantes connexes de $Z_1 D_1$ et $Z_2 D_2$, selon l'ordre $\leq_{ct}$, sont égales, avec D_1 et D_2 des composantes connexes ayant leur arbre d'unroll minimal égal à $\min(\mathcal{U}(\Lambda_{\ell(Z_1)}(X)))$ et $\min(\mathcal{U}(\Lambda_{\ell(Z_2)}(X)))$ respectivement. Cependant, comme $\ell(Z_1) = \ell(f(Z_1)) = \ell(f(Z_2)) = \ell(Z_2)$, nous déduisons que D_1 et D_2 ont le même arbre d'unroll minimal.

Maintenant, comme l'ordre sur les arbres est compatible avec le produit, il s'ensuit que $\min(\mathcal{U}(D_1)) \min(\mathcal{U}(Z_1)) = \min(\mathcal{U}(D_2)) \min(\mathcal{U}(Z_2))$. Et donc du Corollaire 1.58,

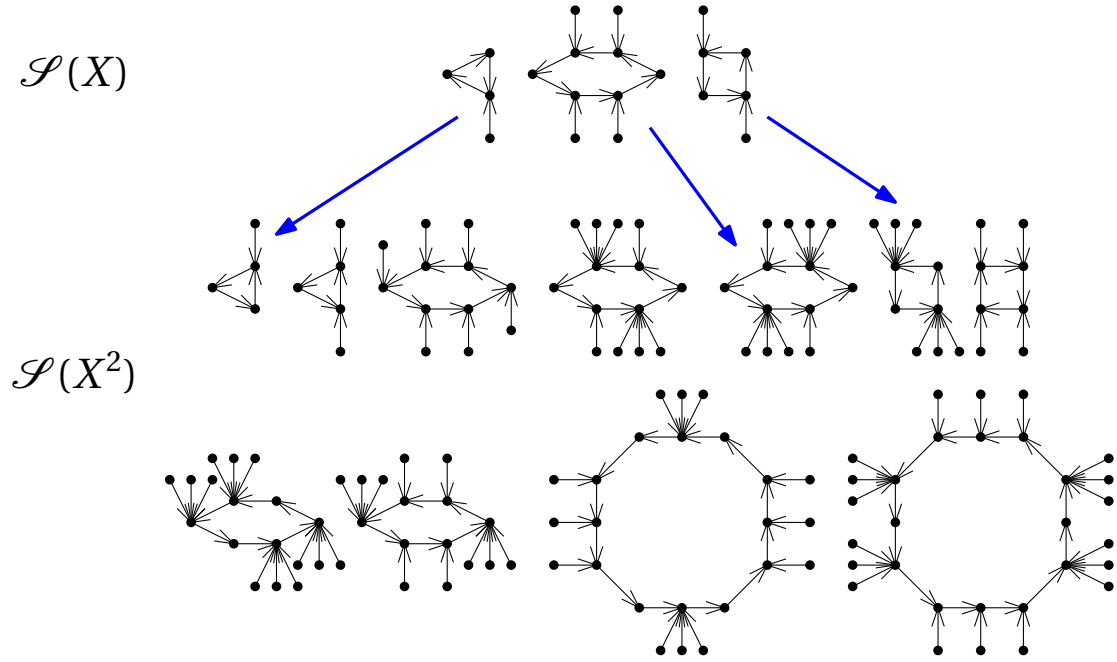


FIGURE 4.6. – Exemple de la fonction f définie dans la preuve du Lemme 4.5. La valeur de D pour les composantes connexes de $\mathcal{S}(X)$ de longueur de cycle 3 et celle de longueur de cycle 6 est la composante connexe de $\mathcal{S}(X)$ de longueur de cycle 3. Tandis que la valeur de D pour la composante connexe de $\mathcal{S}(X)$ de longueur de cycle 4 est la composante de $\mathcal{S}(X)$. Les flèches bleues représentent la fonction f .

il suit que $\min(\mathcal{U}(Z_1)) = \min(\mathcal{U}(Z_2))$ et donc Z_1 et Z_2 ont la même longueur de cycle et le même arbre d'unroll minimal. L'affirmation suit.

Il reste à démontrer que la taille de chaque C est inférieure ou égale à celle de $f(C)$. Pour ce faire, nous utilisons une propriété de [8] : Pour tous FDDS A et B nous avons que A et B sont des mineurs de chaque composante connexe de $A \times B$. Ici, nous considérons la définition des mineurs de graphe orienté introduite dans [30]. C'est-à-dire qu'un graphe orienté H est un mineur d'un autre graphe orienté G s'il peut être construit à partir de G en :

- supprimant des arcs et des sommets,
- contractant un arc e si sa source (resp. sa destination) a degré sortant (resp. entrant) 1.

De cette propriété, nous déduisons que la taille de A et celle de B sont inférieures ou égales à celle de chaque composante connexe de $A \times B$. Induisant que la taille de chaque C est inférieure ou égale à celle de $f(C)$. $\square$

Grâce à ce lemme, nous savons que la taille de chaque $\mathcal{S}(X)$ est inférieure ou égale à la taille de $\mathcal{S}(X^k)$ pour tout entier $k > 0$. Ainsi, étant donné k en binaire et $\mathcal{S}(X)$, le calcul de $\mathcal{S}(X^k)$ est réalisable en temps polynomial sur la taille de $\mathcal{S}(X^k)$ fois $\log k$ via l'exponentiation rapide. En effet, cette méthode permet de réaliser au plus $\lceil \log_2 k \rceil$

4. Résolution d'équations sur les supports de FDDS – 4.1. Complexité de l'évaluation des polynômes sur les supports

produits entre des supports dont la taille est bornée par celle de $\mathcal{S}(X^k)$, notée m . De plus pour réaliser ces produits entre supports, nous pouvons simplement effectuer le produit direct des supports, ce qui prend un temps en $\mathcal{O}(m^2)$, puis les simplifications nécessaires pour n'avoir qu'une seule copie de chaque composante connexe, ce qui prend également un temps en $\mathcal{O}(m^2)$, ces opérations produisent un support dont la taille est bornée par m .

À ce point, pour déduire que $M(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ avec M un monôme sur les supports, peut être calculé en temps polynomial sur sa taille, il nous faut encore prouver que le calcul de $\mathcal{S}(X_1) \boxtimes \dots \boxtimes \mathcal{S}(X_n)$, c'est-à-dire le produit de n supports, est réalisable en temps polynomial sur la taille du résultat fois n . Ceci est le propos du lemme suivant.

Lemme 4.7. *Soit $n > 0$ un entier. Soient $X_1, \dots, X_n$ des FDDS. Soit m la taille du produit des supports des FDDS $X_1, \dots, X_n$. Alors, ce calcul est réalisable en temps polynomial sur $2nm\sqrt{m}$ et chaque sous-produit possible à une taille au plus $2m\sqrt{m}$.*

Démonstration. Soit N l'ensemble des entiers entre 1 et n . Pour montrer ce lemme, nous prouvons dans un premier temps que la taille des $\boxtimes_{i \in I} \mathcal{S}(X_i)$ est bornée par $2m\sqrt{m}$ et ce pour tout sous-ensemble I de N .

Soit I un sous-ensemble de N . Soit D une composante connexe de $\mathcal{S}(\prod_{i \in N-I} X_i)$ ayant le plus petit arbre d'unroll possible. Soit C une composante connexe de $\prod_{i \in I} X_i$ et soit Z la plus petite composante connexe de $C \times D$ selon $\leq_{ct}$. Nous montrons que le nombre de composantes Y dans $\mathcal{S}(\prod_{i \in I} X_i)$ telles que YD admet Z comme composante connexe minimale selon $\leq_{ct}$ est au plus $2m\sqrt{m}$. Notons que par construction Z est une composante connexe de $\mathcal{S}(X_1) \boxtimes \dots \boxtimes \mathcal{S}(X_n)$.

Soit Y une composante connexe de $\mathcal{S}(\prod_{i \in I} X_i)$. Trois choix sont possibles pour Y .

1. Soit $\text{lcm}(\ell(D), \ell(C)) \neq \text{lcm}(\ell(D), \ell(Y))$. Par conséquent, Z n'est pas une composante connexe de YD .
2. Soit $\text{lcm}(\ell(D), \ell(C)) = \text{lcm}(\ell(D), \ell(Y))$ mais $\min(\mathcal{U}(C)) \neq \min(\mathcal{U}(Y))$. Donc, comme le plus petit arbre d'unroll de Z est $\min(\mathcal{U}(C)) \min(\mathcal{U}(D))$ et puisque l'ordre est compatible avec le produit, il suit que Z n'est pas la plus petite composante connexe de YD .
3. Soit $\text{lcm}(\ell(D), \ell(C)) = \text{lcm}(\ell(D), \ell(Y))$ et $\min(\mathcal{U}(C)) = \min(\mathcal{U}(Y))$. Auquel cas, la plus petite composante connexe de YD selon $\leq_{ct}$ est Z .

De cette analyse, nous inférons que le nombre de composantes connexes Y telles que la plus petite composante connexe de YD , selon $\leq_{ct}$ est Z , est borné par le nombre de diviseurs de $q = \text{lcm}(\ell(D), \ell(C))$, donc par $2\sqrt{q}$. Conséquemment, comme Z est une composante connexe de $\mathcal{S}(X_1) \boxtimes \dots \boxtimes \mathcal{S}(X_n)$ il suit que $q \leq m$ et donc $2\sqrt{q} \leq 2\sqrt{m}$. Pour finir, notons que le nombre de Z possibles est borné par le nombre de composantes connexes dans $\mathcal{S}(X_1) \boxtimes \dots \boxtimes \mathcal{S}(X_n)$ et donc aussi par m .

Du point précédent, nous déduisons que le produit de $\mathcal{S}(X_1) \boxtimes \dots \boxtimes \mathcal{S}(X_i)$ avec $\mathcal{S}(X_{i+1})$ est réalisable en temps polynomial sur $2m\sqrt{m}$. En effet, il suffit d'effectuer le produit direct (en temps $\mathcal{O}((2m\sqrt{m})^2)$) puis les simplifications requises effectuaibles en temps polynomial sur la taille du FDDS produit. Ceci induit que le calcul de $\mathcal{S}(X_1) \boxtimes \dots \boxtimes \mathcal{S}(X_n)$ est réalisable en temps polynomial sur n fois $2m\sqrt{m}$. $\square$

4. Résolution d'équations sur les supports de FDDS – 4.1. Complexité de l'évaluation des polynômes sur les supports

Des deux résultats précédents, nous concluons le lemme suivant.

Lemme 4.8. *Soit $n > 0$ un entier. Soit $M = \mathcal{S}(A) \boxtimes \mathcal{S}(X_1)^{k_1} \boxtimes \dots \boxtimes \mathcal{S}(X_n)^{k_n}$ un monôme de n variables sur les supports. Alors, étant donné $\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)$, nous calculons $M(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ en temps polynomial sur $nm \log k_j$ avec j l'entier tel que $k_j \geq k_i$ pour tous les i et m la taille de $M(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$.*

Nous déduisons du Lemme 4.8 qu'il est possible d'évaluer $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ en temps polynomial sur $nm \log d$ avec d le degré de P et m la taille de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$. En effet, comme la somme de supports est l'union, il suit que $M_i(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ est un sous-ensemble de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ pour tous les i . Donc pour évaluer P , il suffit de faire r évaluations de monômes puis de calculer l'union des supports. De ce qui précède, le calcul de chaque $M_i(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ est polynomial sur le logarithme du degré de M_i fois nm . Donc l'évaluation des r monômes est polynomiale sur $rnm \log d$. Et comme le nombre d'éléments dans $M_i(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ est polynomial sur celui de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$, nous en concluons que l'union des supports des monômes est polynomiale sur rm . Ceci termine la preuve de la proposition suivante.

Proposition 4.9. *Soient $n > 0$ un entier, P un polynôme de degré d constitué de $r > 0$ monômes sur n variables et $X_1, \dots, X_n$ des FDDS. Soit m la taille de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$. Alors le calcul de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ est polynomial sur $nmr \log d$.*

Corollaire 4.10. *Soient P un polynôme de support de n variables et B un FDDS. Alors décider s'il existe une solution à l'équation $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ est dans la classe NP.*

Démonstration. Tout d'abord, notons que les Lemmes 4.5 et 4.7 induisent que s'il existe une solution à $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ alors il en existe une où les $X_1, \dots, X_n$ ont une taille au plus $2b\sqrt{b}$ avec b la taille de $\mathcal{S}(B)$.

Il reste à démontrer qu'étant donné $X_1, \dots, X_n$ nous pouvons vérifier efficacement que $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$. Supposons que $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) \subseteq \mathcal{S}(B)$. Alors, par la Proposition 4.9, nous pouvons calculer $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ en temps polynomial et vérifier que deux supports sont égaux est réalisable en temps polynomial par la polynomialité du test d'isomorphisme des FDDS.

Supposons maintenant que $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) \not\subseteq \mathcal{S}(B)$. Donc il existe un monôme M dans P tel que $M(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ n'est pas un sous-ensemble de $\mathcal{S}(B)$. Prouvons que nous vérifions cela en temps polynomial. Considérons que ce monôme M soit égal à $\mathcal{S}(A) \boxtimes \mathcal{S}(Y_1)^{k_1} \boxtimes \dots \boxtimes \mathcal{S}(Y_n)^{k_n}$. Nous expliquons étape par étape comment et à quel moment nous pouvons montrer que $M(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ n'est pas un sous-ensemble de $\mathcal{S}(B)$.

Commençons par l'évaluation de $\mathcal{S}(X_i)^{k_i}$ pour tous les i . Par les Lemmes 4.5 et 4.7, deux cas sont possibles pour la taille des valeurs calculées au cours de l'exponentiation rapide. Soit une valeur a une taille qui dépasse $2b\sqrt{b}$. Donc, la contraposée du Lemme 4.7 induit que $M(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ n'est pas un sous-ensemble de $\mathcal{S}(B)$. Ce test peut bien être fait en temps polynomial car la première fois où il échoue la taille

4. Résolution d'équations sur les supports de FDDS – 4.1. Complexité de l'évaluation des polynômes sur les supports

de la valeur calculée est au plus $(2b\sqrt{b})^2 = 4b^3$. Soit toutes les valeurs ont une taille au plus $2b\sqrt{b}$ et nous passons au cas suivant.

Poursuivons avec le produit $\mathcal{S}(X_1)^{k_1} \boxtimes \dots \boxtimes \mathcal{S}(X_n)^{k_n}$. Par un raisonnement similaire à celui du point précédent, nous pouvons vérifier en temps polynomial si la taille de $\mathcal{S}(X_1)^{k_1} \boxtimes \dots \boxtimes \mathcal{S}(X_n)^{k_n}$ est bornée par $2b\sqrt{b}$. Et si ce n'est pas le cas, la contraposée du Lemme 4.7 induit que $M(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ n'est pas un sous-ensemble de $\mathcal{S}(B)$.

Arrivé à ce point, le calcul de $M(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ est réalisable en temps polynomial et la vérification que $M(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n))$ n'est pas un sous-ensemble de $\mathcal{S}(B)$ est polynomiale. $\square$

Pour conclure cette section, nous montrons qu'étant donné un polynôme P de $n > 0$ variables et B un FDDS alors décider s'il existe une solution à l'équation $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ est NP-complet si P est non-linéaire. Pour cela, il nous suffit de montrer que ce problème est NP-difficile puisque l'appartenance à NP est donnée par le Corollaire 4.10. Pour ce faire, nous allons reprendre la réduction présentée dans la Section 1.2 qui a permis de prouver la Proposition 1.23 et le Théorème 1.25. Pour rappel, la réduction ayant établi que décider si un système d'équations de FDDS à côté droit constant admet une solution est NP-complet (Proposition 1.23) part du problème 1, 3 – SAT et commence par créer un système d'équations sur les entiers naturels. Pour cela, elle associe à chaque variable x_i de la formule en entrée deux variables y_i et z_i qui correspondent à la variable et à sa négation. Pour garantir cette correspondance, le système d'équations qui est créé contient l'équation $y_i + z_i = 1$ impliquant que y_i ou bien z_i est égal à 1 mais pas les deux. Puis, pour chaque clause $(l_1 \vee l_2 \vee l_3)$ nous ajoutons une équation de la forme $l_1 + l_2 + l_3 = 1$ en remplaçant l_j par y_i si $l_j = x_i$ ou z_i si $l_j = \neg x_i$. Ainsi, en remplaçant les 1 par des points fixes C_1 , nous prouvons que le problème sur les systèmes d'équations est NP-difficile.

Notons que nous ne pouvons pas appliquer ce même raisonnement aux supports. En effet, remarquons que l'équation $\mathcal{S}(y_i + z_i) = \mathcal{S}(C_1)$ n'empêche pas la possibilité que $\mathcal{S}(y_i) = \mathcal{S}(z_i) = C_1$. En d'autres mots, appliquer la fonction $\mathcal{S}(X)$ à toutes les équations ajoute la solution où toutes les variables du système sont à 1 qui n'encode pas une solution cohérente à 1, 3 – SAT.

Pour contourner ce problème, nous ajoutons à notre système l'équation $y_i z_i = 0$. Ainsi, l'équation $\mathcal{S}(y_i + z_i) = C_1$ induit que $\mathcal{S}(y_i)$ et/ou $\mathcal{S}(z_i)$ est égal à C_1 tandis que $\mathcal{S}(y_i z_i) = \emptyset$ impose que $\mathcal{S}(y_i)$ et/ou $\mathcal{S}(z_i)$ vaut $\emptyset$. Par conséquent, le support de l'une des deux variables est égal à C_1 tandis que l'autre est égal à $\emptyset$. Ainsi, la réduction que nous venons de décrire, partant de 3 – SAT et non plus 1, 3 – SAT, nous permet de conclure la proposition suivante.

Proposition 4.11. *Décider si un système d'équations polynomiales non linéaires de support et à côté droit constant possède au moins une solution est NP-complet.*

De là, en appliquant le même raisonnement que pour la preuve du Théorème 1.25, c'est-à-dire en multipliant par des dendrons, nous concluons la démonstration du résultat suivant.

Théorème 4.12. *Décider si une équation polynomiale non linéaire de support et à côté droit constant possède au moins une solution est NP-complet.*

Il nous reste à couvrir le cas des équations et systèmes d'équations de support linéaire et à côté droit constant. Ceci sera le propos des sections suivantes.

4.2. Équation linéaire sur les supports

4.2.1. Résolution des équations linéaires sur les supports

Dans cette partie, nous montrons que nous pouvons résoudre les équations linéaires sur les supports en temps polynomial. Plus formellement, nous prouvons qu'étant donné $n > 0$ un entier, $A_1, \dots, A_n$ et B des FDDS non vides alors l'équation $\mathcal{S}(A_1)\mathcal{S}(X_1) + \dots + \mathcal{S}(A_n)\mathcal{S}(X_n) = \mathcal{S}(B)$ est soluble en temps polynomial sur la taille du support de chaque coefficient et sur la taille du support de B . En analysant ce résultat à l'aune du Théorème 1.25, nous déduisons qu'il est simple (polynomial) de trouver les possibles composantes connexes d'une solution à l'équation $A_1 X_1 + \dots + A_n X_n = B$ mais qu'il est NP-difficile de trouver le nombre de copies de chaque composante connexe.

Notons que le cas linéaire a une différence majeure avec le cas général (degré du polynôme non borné) qui le rend plus simple à analyser : deux composantes connexes dans l'affectation de variables n'ont pas "d'impact" l'une sur l'autre. En effet, si deux composantes connexes D_1 et D_2 sont affectées à deux variables différentes, disons X_1 et X_2 alors si $\mathcal{S}(A_1 D_1)$ et $\mathcal{S}(A_2 D_2)$ sont des sous-ensembles de $\mathcal{S}(B)$, c'est aussi le cas de $\mathcal{S}(A_1 D_1 + A_2 D_2)$ car ce support est l'union de deux sous-ensembles de $\mathcal{S}(B)$. De même, si maintenant D_1 et D_2 sont affectées à la même variable, disons X , et que les supports de $A D_1$ et de $A D_2$ sont des sous-ensembles du support de B alors c'est aussi le cas de $\mathcal{S}(A(D_1 + D_2))$ car ce dernier est égal à l'union des supports de $A D_1$ et de $A D_2$. Cette propriété n'est pas garantie à partir des équations de degré 2 puisque $\mathcal{S}(A(D_1 + D_2)^2)$ n'est pas nécessairement l'union des supports de $A D_1^2$ et de $A D_2^2$.

Exemple 4.13. *Considérons $A = C_1, B = C_2 + C_3, D_1 = C_2$ et $D_2 = C_3$. Alors les supports de $A D_1^2$ et $A D_2^2$ sont bien des sous-ensembles de celui de B mais comme $\mathcal{S}(A D_1 D_2)$ est un sous-ensemble de $\mathcal{S}(A(D_1 + D_2)^2)$ et qu'il est égal à C_6 , il suit que $\mathcal{S}(A(D_1 + D_2)^2)$ n'est pas un sous-ensemble de $\mathcal{S}(B) = C_2 \boxplus C_3$.*

L'intuition précédente est formalisée dans le lemme suivant et la propriété présentée dans ce dernier est illustrée par l'Exemple 4.15.

Lemme 4.14. *Soient A et B deux FDDS. S'il existe une solution à $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ alors le support $\mathcal{S}(X) = \boxplus_{\mathcal{S}(X') \in I} \mathcal{S}(X')$ en est une solution, avec I l'ensemble de tous les supports solutions. En symbole, $I = \{\mathcal{S}(X') \mid \mathcal{S}(A)\mathcal{S}(X') = \mathcal{S}(B)\}$.*

Démonstration. Soit m la taille de $\mathcal{S}(B)$. Supposons qu'il existe une solution à l'équation $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$. Soit I l'ensemble de tous les supports solutions de l'équation

4. Résolution d'équations sur les supports de FDDS – 4.2. Équation linéaire sur les supports

précédente. Notons que I est fini car le Lemme 4.7 induit que chaque élément de I a une taille au plus $2m\sqrt{m}$ et qu'il existe un nombre fini de supports d'une certaine taille. Soit $\mathcal{S}(X)$ l'union de tous les supports de I . L'intuition derrière cette preuve est que chaque sommet support solution ne fera qu'augmenter les multiplicités des composantes connexes de $\mathcal{S}(B)$ dans $\mathcal{S}(A)\mathcal{S}(X)$. Or ces multiplicités sont ignorées par le support. Montrons que $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$. De la définition de $\mathcal{S}(X)$ nous avons que :

$$\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(A) \left(\bigsqcup_{\mathcal{S}(X') \in I} \mathcal{S}(X') \right) = \bigsqcup_{\mathcal{S}(X') \in I} (\mathcal{S}(A)\mathcal{S}(X')) = \bigsqcup_{\mathcal{S}(X') \in I} \mathcal{S}(B) = \mathcal{S}(B).$$

□

Exemple 4.15. *Considérons $A = C_8$, $B = C_8 + C_{24}$ et l'équation $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$. Alors l'ensemble des solutions de $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ peut être décrit comme la somme $\mathcal{S}(Y) \bigsqcup \mathcal{S}(Z)$ avec $\mathcal{S}(Y)$ un sous-ensemble de $\{C_1, C_2, C_4, C_8\}$ ($\mathcal{S}(Y)$ permet de générer C_8 dans $\mathcal{S}(B)$) et $\mathcal{S}(Z)$ un sous-ensemble de $\{C_3, C_6, C_{12}, C_{24}\}$ ($\mathcal{S}(Z)$ permet de générer C_{24} dans $\mathcal{S}(B)$). Donc en particulier $\mathcal{S}(Y) \bigsqcup \mathcal{S}(Z) = \{C_1, C_2, C_4, C_8, C_3, C_6, C_{12}, C_{24}\}$ est aussi une solution.*

Soient A, B des FDDS. Rechercher la solution de $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ étant l'union de toutes les solutions a plusieurs avantages que nous exploiterons dans le reste de ce chapitre. Tout d'abord, comme nous le prouverons dans le reste de cette section, elle est trouvable efficacement. De plus, si nous parvenons à en borner la taille, nous bornerons par la même occasion la taille de tous les supports solutions. Elle permet également d'énumérer l'ensemble des supports solutions en délai polynomial (Section 4.2.2). Finalement, cette solution sera la plus adaptée pour nous ramener des équations sur les supports à celle sur les FDDS (Section 4.3).

Nous fournirons dans la suite de cette section un algorithme de résolution qui se reposera sur la propriété énoncée ci-dessus. Mais pour parvenir à cela, nous devons étudier la forme des composantes connexes que nous devons mettre dans les solutions. À cette fin, nous caractérisons ces formes dans le cas univarié et en dérivons un algorithme. Puis nous généraliserons ce dernier au cas multivarié dans un second temps.

Soit $M = \mathcal{S}(A)\mathcal{S}(X)$ un monôme linéaire univarié sur les supports et soit $\mathcal{S}(X)$ et $\mathcal{S}(B)$ tels que $M(\mathcal{S}(X)) = \mathcal{S}(B)$ et $\mathcal{S}(X)$ est une solution maximisant le nombre de composantes connexes. Nous savons que $\mathcal{S}(X)$ est unique car s'il existait un $\mathcal{S}(Y) \not\subseteq \mathcal{S}(X)$ tel que $M(\mathcal{S}(Y)) = \mathcal{S}(B)$ alors $\mathcal{S}(X) \bigsqcup \mathcal{S}(Y)$ serait aussi une solution et aurait plus de composantes connexes que $\mathcal{S}(X)$ et $\mathcal{S}(Y)$.

En outre, pour chaque composante connexe B_i de $\mathcal{S}(B)$, il existe une composante connexe D de $\mathcal{S}(X)$ telle que B_i est comprise dans $M(D)$. Et comme $M(\mathcal{S}(X)) = \mathcal{S}(B)$, dans certains cas nous aurons que les composantes B_i sont de surcroît les plus petites composantes connexes de $M(D)$, selon $\leq_{ct}$. Nous introduisons le lemme suivant qui permet de caractériser ces cas.

4. Résolution d'équations sur les supports de FDDS – 4.2. Équation linéaire sur les supports

Lemme 4.16. *Soit $M = \mathcal{S}(A)\mathcal{S}(X)$ un monôme linéaire sur les supports et soit $\mathcal{S}(X)$ et $\mathcal{S}(B)$ tels que $M(\mathcal{S}(X)) = \mathcal{S}(B)$ et $\mathcal{S}(X)$ est une solution maximisant le nombre de composantes connexes. Alors D est une composante connexe de $\mathcal{S}(X)$ si et seulement si elle vérifie les conditions suivantes :*

1. *il existe B_i une composante connexe de B telle que $E = \Lambda_{\ell(B_i)}(\mathcal{S}(A))$ n'est pas vide,*
2. *D a pour arbre d'unroll minimal $\min(\mathcal{U}(B_i)) / \min(\mathcal{U}(A_i))$ de plus petite période p avec A_i la plus petite composante connexe de E selon $\leq_{ct}$ ayant comme arbre d'unroll minimal $\min(\mathcal{U}(D))$,*
3. *D a une longueur de cycle égale à $q \text{lcm}(p, \text{alcm}_{A_i} B_i)$ avec q un diviseur de $\ell(B_i) / \text{lcm}(p, \text{alcm}_{A_i} B_i)$ et donc diviseur de $\ell(A_i)$,*
4. *$M(D)$ est un sous-ensemble de $\mathcal{S}(B)$.*

Démonstration. ($\Leftarrow$) Si la condition 4 de l'énoncé est vérifiée, alors $M(X + D) = \mathcal{S}(B)$. Donc $\mathcal{S}(A)(\mathcal{S}(X) \boxplus \mathcal{S}(D)) = \mathcal{S}(B)$. Et comme $\mathcal{S}(X)$ est maximale, nous déduisons que $\mathcal{S}(X) \boxplus \mathcal{S}(D) = \mathcal{S}(X)$ et conséquemment D est une composante connexe de X .

($\Rightarrow$) Supposons que D est une composante connexe de X . Donc la condition 4 est vérifiée. Montrons, tout d'abord, que la condition 1 est vérifiée. D'après l'hypothèse, il existe une composante connexe B_i de B qui est égale à la plus petite composante connexe de AX_c selon $\leq_{ct}$. Et en particulier, il existe une composante connexe A_c de A telle que B_i est une composante connexe de $A_c D$. De là, par définition du produit, $\ell(B_i) = \text{lcm}(\ell(A_c), \ell(D))$. Donc D est non vide.

Démontrons maintenant la condition 2. Considérons la composante connexe A_i décrite dans l'énoncé. Prouvons que B_i est une composante connexe de $A_i D$. Remarquons que l'ensemble des composantes connexes de $A_i D$ a une longueur de cycle qui est $\text{lcm}(\ell(A_i), \ell(D)) \leq \ell(B_i)$. Et comme $\mathcal{S}(A_i D)$ est un sous-ensemble de $\mathcal{S}(AX_c)$, la minimalité de B_i induit que $\text{lcm}(\ell(A_i), \ell(D)) = \ell(B_i)$. Nous en déduisons donc que $\min(\mathcal{U}(B_i)) \leq \min(\mathcal{U}(A_i)) \min(\mathcal{U}(D))$. Or, comme l'ordre est compatible avec le produit, la minimalité de B_i induit que $\min(\mathcal{U}(B_i)) = \min(\mathcal{U}(A_c)) \min(\mathcal{U}(D))$. De là, nous avons $\min(\mathcal{U}(A_c)) \min(\mathcal{U}(D)) \leq \min(\mathcal{U}(A_i)) \min(\mathcal{U}(D))$ et nous avons aussi $\min(\mathcal{U}(A_c)) \leq \min(\mathcal{U}(A_i))$. Mais $\min(\mathcal{U}(A_i)) \leq \min(\mathcal{U}(A_c))$ par définition de A_i . Ceci induit que $\min(\mathcal{U}(A_i)) \min(\mathcal{U}(D)) = \min(\mathcal{U}(B_i))$ et la condition suit.

Soit p la plus petite période de l'arbre d'unroll minimal de D . Montrons que la condition 3 est vérifiée. Rappelons que par le point précédent $\text{lcm}(\ell(A_i), \ell(D)) = \ell(B_i)$. Donc $\ell(D)$ est un multiple de $\text{alcm}_{A_i} B_i$. Et par définition, c'est aussi un multiple de p . Donc $\ell(D) = q \text{lcm}(p, \text{alcm}_{A_i} B_i)$ avec q un diviseur de $\ell(B_i) / \text{lcm}(p, \text{alcm}_{A_i} B_i)$. En effet, si ce n'est pas le cas, alors $q \text{lcm}(p, \text{alcm}_{A_i} B_i)$ n'est pas un diviseur de $\ell(B_i)$. Nous en concluons que toutes les conditions et la direction suivent. $\square$

Corollaire 4.17. *Soit $M = \mathcal{S}(A)\mathcal{S}(X)$ un monôme linéaire univarié sur les supports et soient X et B tels que $M(\mathcal{S}(X)) = \mathcal{S}(B)$. Alors, X est une solution avec le support maximisant le nombre de composantes connexes si et seulement s'il contient toutes les composantes connexes vérifiant les conditions du Lemme 4.16.*

Algorithme 4.19 $\text{DivisionSupport}(A, B)$

```

1:  $X \leftarrow \emptyset$ ,
2:  $S \leftarrow \mathcal{S}(B)$ ,
3: Tant que  $S$  n'est pas vide faire
4:   soit  $B_i$  une composante de  $S$ ,
5:    $S \leftarrow S - B_i$ ,  $E \leftarrow \Lambda_{\ell(B_i)}(\mathcal{S}(A))$ ,
6:   Si  $E$  n'est pas vide alors
7:     chercher  $A_i$  la plus petite composante connexe de  $E$  selon  $\leq_{ct}$  ayant comme
       arbre d'unroll minimal  $\min(\mathcal{U}(D))$ ,
8:      $\mathbf{x} \leftarrow \min(\mathcal{U}(B_i)) / \min(\mathcal{U}(A_i))$ ,
9:     Si  $\mathbf{x}$  est défini et que  $\mathbf{x} = \min(\Sigma(\mathbf{x}))$  alors
10:      calculer  $p$  la plus petite période de  $\mathbf{x}$  et  $x = \text{lcm}(p, \text{alcm}_{A_i} B_i)$ ,
11:      Pour chaque diviseur  $q$  de  $\ell(B_i)/x$  faire
12:        construire  $D$  l'enroulement de  $\mathbf{x}$  à période  $qx$ ,
13:        Si  $\mathcal{S}(A)\mathcal{S}(D)$  est un sous-ensemble de  $\mathcal{S}(B)$  alors
14:           $X \leftarrow X \boxplus D$ ,
15: Si  $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$  alors
16:   renvoyer  $X$ ,
17: renvoyer  $\perp$ .
```

Corollaire 4.18. Soit $M = \mathcal{S}(A)\mathcal{S}(X)$ un monôme linéaire sur les supports et soient X et B tels que $M(\mathcal{S}(X)) = \mathcal{S}(B)$ avec $\mathcal{S}(X)$ la solution qui admet le plus de composantes connexes. Alors, $\mathcal{S}(X)$ contient au plus $2m_B\sqrt{p_A}$ composantes connexes, avec m_B le nombre de composantes connexes de $\mathcal{S}(B)$ et p_A la plus grande longueur de cycle $\mathcal{S}(A)$.

Démonstration. La condition 3 du Lemme 4.16 induit que pour chaque B_i il y a au plus $2\sqrt{\ell(A_i)}$ valeurs possibles pour D . En effet, le nombre de D possibles est borné par le nombre de diviseurs de $\ell(A_i)$ qui est au plus $2\sqrt{\ell(A_i)}$. Donc le nombre de D possibles est au plus $2\sqrt{p_A}$. Le résultat suit en multipliant cette borne par le nombre de composantes connexes dans $\mathcal{S}(B)$. $\square$

Nous introduisons maintenant l'Algorithme 4.19 permettant de résoudre les équations de la forme $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$. Ce dernier est juste une application itérative du Lemme 4.16. Il prend en entrée deux FDDS A et B tels que $A = \mathcal{S}(A)$ et $B = \mathcal{S}(B)$. Relevons que nous effectuons le test de la ligne 15 de cet algorithme car au cours des itérations de la boucle commençant à la ligne 3, nous ne faisons que construire un X tel que $\mathcal{S}(A)\mathcal{S}(X)$ est un sous-ensemble de $\mathcal{S}(B)$. Ainsi, ce test nous permet de vérifier que nous avons l'égalité au lieu de la simple inclusion.

La correction de l'Algorithme 4.19 est une conséquence directe du Lemme 4.16. En effet, dans l'algorithme nous cherchons une composante connexe B_i de S qui vérifie la condition 1 du Lemme 4.16 (lignes 4–6). Puis pour chaque B_i qui vérifie cette condition, il teste si le couple (A_i, B_i) vérifie la condition 2 (lignes 8 et 9). Après quoi, si c'est le cas, il calcule l'ensemble des rolls possibles respectant la condition 3 (boucle de la ligne 11). Et finalement, pour chacun des rolls construits, il teste la condition 4

(ligne 13). Induisant qu'il réalise une recherche exhaustive de toutes les possibles composantes connexes. Nous en concluons, par le Corollaire 4.17, que si l'équation $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ admet une solution alors l'algorithme trouve celle qui contient le plus de composantes connexes dans son support. Ainsi, pour démontrer le lemme suivant, il suffit de prouver que l'algorithme est polynomial.

Lemme 4.20. *Soient A et B des FDDS. La division de $\mathcal{S}(B)$ par $\mathcal{S}(A)$ (Algorithme 4.19) est réalisable en temps polynomial sur la taille de $\mathcal{S}(B)$.*

Démonstration. Remarquons que l'ensemble des instructions de l'algorithme autres que celles des lignes 8 et 10 sont directement polynomiales. En effet, par le Lemme 4.7 la taille de $\mathcal{S}(A)$ est polynomiale sur celle de $\mathcal{S}(B)$. Donc la constitution de E (instruction 5) est réalisable en temps polynomial. De plus, par un raisonnement similaire à celui utilisé pour montrer le Lemme 3.4, nous parvenons à trouver A_i (instruction 7) en temps polynomial sur la taille de $\mathcal{S}(E)$ qui est elle-même bornée par celle de $\mathcal{S}(A)$.

De plus, la création d'un D (instruction 12) se fait en temps polynomial sur la taille de D et tester que $\mathcal{S}(A)\mathcal{S}(D)$ est un sous-ensemble de $\mathcal{S}(B)$ est réalisable en temps polynomial sur la taille de $\mathcal{S}(B)$ par une stratégie similaire à celle utilisée pour vérifier que $\mathcal{S}(A)\mathcal{S}(D) = \mathcal{S}(B)$. Ainsi, comme le nombre de q (boucles 11) possibles est borné par la taille du cycle de B_i , l'instruction 11 est bien exécutée en temps polynomial sur la taille du support de B .

De là, si les lignes 8 et 10 ont un temps de calcul polynomial sur la taille du support de B , alors toutes les opérations du corps de la boucle de l'algorithme seront en temps polynomial sur cette taille et puisque le nombre d'itérations est borné par le nombre de composantes connexes dans $\mathcal{S}(B)$, il suit que le temps de calcul total de la boucle est polynomial sur cette taille.

Montrons alors que les lignes 8 et 10 ont une complexité polynomiale sur la taille du support de B . Pour cela, nous réexpliquons comment diviser des arbres d'unroll en prouvant qu'il est possible de le faire en temps polynomial sur la taille du support de B . Considérons $n = 2\ell(B_i) + d(B_i)$. Comme la taille de A_i et de B_i est polynomiale sur la taille de $\mathcal{S}(B)$, nous déduisons que c'est aussi le cas du calcul de $\mathcal{C}_n(\mathcal{U}(A_i))$ et $\mathcal{C}_n(\mathcal{U}(B_i))$.

Maintenant, grâce au Lemme 2.4, nous sommes en mesure de trouver le plus petit arbre d'unroll de $\mathcal{U}(B_i)$ en temps polynomial sur la taille de B_i et ce grâce à $\mathcal{C}_n(\mathcal{U}(B_i))$. De plus, comme la longueur de cycle de A_i est un diviseur de celle de B_i , nous déduisons que n est suffisamment grand pour pouvoir appliquer le Lemme 2.4 sur $\mathcal{C}_n(\mathcal{U}(A_i))$. Et donc, nous trouvons le plus petit arbre d'unroll de A_i de la même façon que pour B_i .

À ce point, nous calculons $\mathbf{t}_0 = \min(\mathcal{C}_n(\mathcal{U}(B_i))) / \min(\mathcal{C}_n(\mathcal{U}(A_i)))$ en temps polynomial sur la taille du support de B . Donc nous réalisons Y l'enroulement de $\mathbf{t}_0$ à période $\ell(B_i)$ en temps polynomial et, par la même technique que décrite ci-dessus, nous calculons $\mathcal{C}_n(\mathbf{t})$ en temps polynomial avec $\mathbf{t}$ le plus petit arbre d'unroll de Y . Et ainsi, nous vérifions que $\mathcal{C}_n(\mathbf{t}) = \mathbf{t}_0$ en temps polynomial. De là, si $\mathcal{C}_n(\mathbf{t}) = \mathbf{t}_0$, nous déduisons que $\min(\mathcal{U}(A_i))\mathbf{t} = \min(\mathcal{U}(B_i))$. Dans le cas contraire, $\min(\mathcal{U}(A_i))$ ne divise pas $\min(\mathcal{U}(B_i))$. En effet, s'il existait $\mathbf{t}'$ tel que $\min(\mathcal{U}(A_i))\mathbf{t}' = \min(\mathcal{U}(B_i))$ alors $\mathbf{t}_0$

4. Résolution d'équations sur les supports de FDDS – 4.2. Équation linéaire sur les supports

en serait la troncature à profondeur n . Nous pourrions donc retrouver $\mathbf{t}'$ en réalisant l'enroulement de $\mathbf{t}_0$ à période $\ell(B_i)$ car le Corollaire 2.28 implique que $\mathbf{t}'$ est de période $\ell(B_i)$. Donc $\mathbf{t}'$ est un arbre de $\mathcal{U}(Y)$ qui est strictement supérieur à $\mathbf{t}$.

Étant donné que $\mathbf{t}$ a pour période $\ell(B_i)$ nous retrouvons sa période minimale en testant ces différents préfixes. Ceci est bien réalisable en temps polynomial sur la taille du support de B . $\square$

Corollaire 4.21. *Soient A et B deux FDDS. Alors nous pouvons trouver la solution à $\mathcal{S}(A)\mathcal{S}(X) \subseteq \mathcal{S}(B)$ maximisant le nombre de composantes connexes en temps polynomial sur la taille de $\mathcal{S}(A)$ et $\mathcal{S}(B)$.*

Démonstration. Si nous supprimons le test de la ligne 15 de l'Algorithme 4.19, alors il renvoie un X tel que $\mathcal{S}(A)\mathcal{S}(X) \subseteq \mathcal{S}(B)$. Nous prouvons alors le corollaire en utilisant le même raisonnement qu'au Lemme 4.20 concernant l'analyse de complexité. $\square$

Dans le but d'étendre cet algorithme au cas linéaire multivarié, nous notons que les affectations des variables peuvent être traitées indépendamment, elles n'ont pas d'"impact" les unes sur les autres. Par exemple, si X_1 et X_2 sont des solutions à $\mathcal{S}(A_1)\mathcal{S}(X_1) \boxplus \mathcal{S}(A_2)\mathcal{S}(X_2) = \mathcal{S}(B)$ alors c'est aussi le cas de X'_1 tel que $\mathcal{S}(X_1)$ est un sous-ensemble de $\mathcal{S}(X'_1)$ et $\mathcal{S}(A_1)\mathcal{S}(X'_1)$ est un sous-ensemble de $\mathcal{S}(B)$. Cette intuition est formalisée dans le lemme suivant.

Lemme 4.22. *Soit $n > 0$ un entier, soient $A_1, \dots, A_n$ et B des FDDS. Si l'équation $\boxplus_{i=1}^n \mathcal{S}(A_i)\mathcal{S}(X_i) = \mathcal{S}(B)$ admet solution alors $\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)$ est aussi solution avec $\mathcal{S}(X_i)$ l'union de tous les supports $\mathcal{S}(X')$ tels que $\mathcal{S}(A_i)\mathcal{S}(X') \subseteq \mathcal{S}(B)$ pour tous les i .*

Démonstration. Supposons que l'équation $\boxplus_{i=1}^n \mathcal{S}(A_i)\mathcal{S}(Y_i) = \mathcal{S}(B)$ admet solution. Par définition nous avons que $\mathcal{S}(X_i)$ est l'union de tous les supports $\mathcal{S}(X'_i)$ tels que $\mathcal{S}(A_i)\mathcal{S}(X'_i) \subseteq \mathcal{S}(B)$. Montrons que $\boxplus_{i=1}^n \mathcal{S}(A_i)\mathcal{S}(X_i) = \mathcal{S}(B)$. De la définition des $\mathcal{S}(X_i)$ nous avons que

$$\boxplus_{i=1}^n \mathcal{S}(A_i)\mathcal{S}(X_i) = \boxplus_{i=1}^n \mathcal{S}(A_i) \left(\boxplus_{\mathcal{S}(X'_i) \subseteq \mathcal{S}(X)} \mathcal{S}(X'_i) \right) = \boxplus_{i=1}^n \boxplus_{\mathcal{S}(X'_i) \subseteq \mathcal{S}(X)} (\mathcal{S}(A_i)\mathcal{S}(X'_i)) \subseteq \mathcal{S}(B).$$

Mais puisque $\boxplus_{\mathcal{S}(X'_i) \subseteq \mathcal{S}(X)} \mathcal{S}(X'_i)$ contient $\mathcal{S}(Y_i)$ pour tous les i , nous déduisons que

$$\boxplus_{i=1}^n \boxplus_{\mathcal{S}(X'_i) \subseteq \mathcal{S}(X)} (\mathcal{S}(A_i)\mathcal{S}(X'_i)) = \boxplus_{i=1}^n \boxplus_{\mathcal{S}(X'_i) \neq \mathcal{S}(Y_i)} (\mathcal{S}(A_i)\mathcal{S}(X'_i)) \boxplus \boxplus_{i=1}^n \mathcal{S}(A_i)\mathcal{S}(Y_i)$$

Donc $\boxplus_{i=1}^n \mathcal{S}(A_i)\mathcal{S}(X_i)$ est l'union d'un sous-ensemble de $\mathcal{S}(B)$ avec $\mathcal{S}(B)$ donc est égale à $\mathcal{S}(B)$. La direction suit. $\square$

De ce lemme, nous déduisons un algorithme étant capable de résoudre efficacement $\boxplus_{i=1}^n \mathcal{S}(A_i)\mathcal{S}(X_i) = \mathcal{S}(B)$. Ce dernier calcule pour chaque variable le support qui maximise le nombre de composantes connexes qui est tel que $\mathcal{S}(A_i)\mathcal{S}(X_i)$ est un

4. Résolution d'équations sur les supports de FDDS – 4.2. Équation linéaire sur les supports

sous-ensemble de $\mathcal{S}(B)$. Puis, cet algorithme évalue le polynôme avec l'affectation décrite ci-dessus et teste si le résultat est bien égal à $\mathcal{S}(B)$.

La correction de cette procédure est immédiate et nous remarquons que si l'étape de construction de l'affectation a un temps de réalisation polynomial sur la taille de $\mathcal{S}(B)$ alors c'est aussi le cas de l'algorithme.

Nous voulons donc obtenir un algorithme polynomial sur la taille du support de B , qui, pour chaque variable, calculera le support X_i avec le plus de composantes connexes tel que $\mathcal{S}(A_i)\mathcal{S}(X_i)$ est un sous-ensemble de $\mathcal{S}(B)$. Pour réaliser ceci, observons que nous sommes en mesure d'adapter le Lemme 4.16 en relâchant la contrainte $M(\mathcal{S}(X)) = \mathcal{S}(B)$ en $M(\mathcal{S}(X)) \subseteq \mathcal{S}(B)$. Induisant que l'Algorithme 4.19 où nous retirons l'instruction 15 permet de réaliser l'opération voulue.

Remarquons que nous pouvons généraliser cela au cas où le polynôme admet un terme constant. En effet, il suffit d'appliquer l'algorithme précédent et d'ajouter le support de ce terme au résultat. Ceci concluant la preuve du théorème suivant.

Théorème 4.23. *Soit $n > 0$ un entier, soient $A_0, \dots, A_n$ et B des FDDS. Résoudre*

$$\mathcal{S}(A_0) \boxplus \boxplus_{i=1}^n \mathcal{S}(A_i)\mathcal{S}(X_i) = \mathcal{S}(B)$$

est réalisable en temps polynomial sur n fois la taille de $\mathcal{S}(B)$.

4.2.2. Énumération des solutions des équations linéaires sur les supports

Dans cette partie, nous montrons qu'il est possible d'énumérer toutes les solutions aux équations de la forme

$$\mathcal{S}(A_0) \boxplus \mathcal{S}(A_1)\mathcal{S}(X_1) \boxplus \dots \boxplus \mathcal{S}(A_n)\mathcal{S}(X_n) = \mathcal{S}(B)$$

avec un délai polynomial. Pour cela, nous commençons une fois de plus par le cas $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ et nous montrons comment étendre le support solution avec le plus de composantes pour générer l'ensemble des supports solutions.

L'idée sur laquelle repose notre méthode est simple. Elle consiste à prendre un support solution, et pour chacune de ses composantes connexes, générer un nouveau support à partir du support solution en lui retirant cette dernière. Puis tester si le nouveau support est bien solution. Si c'est le cas alors nous réappliquons le même algorithme sur le support construit. Ceci décrit une procédure récursive qui permet d'énumérer l'ensemble des supports solutions obtenables en retirant une composante connexe. Constatons toutefois qu'une solution peut être générée plusieurs fois au cours des appels récursifs.

Exemple 4.24. *Soient $A = C_4$ et $B = C_{12}$. Alors le support solution maximisant le nombre de composantes connexes est $\{C_3, C_6, C_{12}\}$ et chacun de ses sous-ensembles est solution. Si nous appliquons la procédure décrite précédemment, nous générons $\{C_6, C_{12}\}$ et $\{C_3, C_{12}\}$ et $\{C_3, C_6\}$ à partir de la première solution. Puis à partir de :*

Algorithme 4.25 EnumSupportSol1Var($A, B, \mathcal{S}(X), D$)

- 1: **Énumérer** $\mathcal{S}(X)$,
 - 2: **Pour chaque** composante connexe D' de $\mathcal{S}(X)$ **faire**
 - 3: **Si** $D <_{ct} D'$ et $\mathcal{S}(A)(\mathcal{S}(X) \boxminus D') = \mathcal{S}(B)$ **alors**
 - 4: EnumSupportSol1Var($A, B, \mathcal{S}(X) \boxminus D', D'$).
-

- $\{C_6, C_{12}\}$ nous générons C_{12} et C_6 ,
- $\{C_3, C_{12}\}$ nous générons C_3 et C_{12} ,
- $\{C_3, C_6\}$ nous générons C_3 et C_6 .

Donc certaines solutions sont générées plusieurs fois.

Pour résoudre ce problème, nous définissons l'Algorithme 4.25. Ce dernier prend en entrée deux FDDS A et B , un support solutions $\mathcal{S}(X)$ à $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ et une composante connexe D de $\mathcal{S}(X)$. Il se repose sur l'idée décrite ci-dessus mais ajoute comme condition que les composantes connexes retirées doivent être strictement supérieures à D par l'ordre $\leq_{ct}$. Nous démontrerons ultérieurement que cette condition additionnelle permet d'éviter de produire plusieurs fois la même solution, en d'autres mots d'éviter la production de *doublons*.

Soient A et B des FDDS et soit X la solution renvoyée par l'Algorithme 4.19 sur A et B triés selon $\leq_{ct}$. Nous démontrons que EnumSupportSol1Var($A, B, X, \emptyset$) (Algorithme 4.25) permet d'énumérer l'ensemble des supports solutions de l'équation $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ avec un délai polynomial sur la somme des tailles de $\mathcal{S}(A)$, $\mathcal{S}(X)$ et $\mathcal{S}(B)$. Pour ce faire, nous nous reposerons sur le théorème suivant.

Théorème 4.26 (Folklore). *Soit $\mathcal{A}$ un algorithme et $\mathcal{I}$ une instance d'un problème. Alors, $\mathcal{A}$ est un algorithme d'énumération des solutions de $\mathcal{I}$ avec un délai polynomial si les conditions suivantes sont vérifiées :*

1. *tous les produits de $\mathcal{A}(\mathcal{I})$ sont des solutions de $\mathcal{I}$,*
2. *toutes les solutions de $\mathcal{I}$ sont produites par $\mathcal{A}(\mathcal{I})$,*
3. *$\mathcal{A}(\mathcal{I})$ ne produit pas de doublons,*
4. *le délai entre la génération de deux solutions de $\mathcal{A}(\mathcal{I})$ est polynomial sur la taille de $\mathcal{I}$.*

Constatons que EnumSupportSol1Var($A, B, X, \emptyset$) (Algorithme 4.25) respecte la condition 1 du Théorème 4.26 par construction. Montrons que la condition 2 est bien vérifiée.

Lemme 4.27. *Soient A et B des FDDS et soit X la solution renvoyée par l'Algorithme 4.19 sur A et B triés selon $\leq_{ct}$. Soit X' un FDDS tel que $\mathcal{S}(A)\mathcal{S}(X') = \mathcal{S}(B)$. Alors, $\mathcal{S}(X')$ est produit lors de l'exécution de EnumSupportSol1Var($A, B, X, \emptyset$).*

Démonstration. Supposons que $\mathcal{S}(X) = \{X_1, \dots, X_{m_X}\}$ tel que $X_i <_{ct} X_{i+1}$ pour tous les i . Alors, par définition de X , il existe I un sous-ensemble de $\{1, \dots, m_X\}$ tel que $\mathcal{S}(X') = \boxplus_{i \in I} X_i$. Montrons que $\mathcal{S}(X')$ est produit lors de l'exécution de l'appel

4. Résolution d'équations sur les supports de FDDS – 4.2. Équation linéaire sur les supports

$\text{EnumSupportSol1Var}(A, B, X, \emptyset)$. Pour cela, nous prouvons par récurrence sur les composantes connexes de $\mathcal{S}(X)$ qu'il existe un chemin dans l'arbre des appels récursifs tel que la valeur de $\mathcal{S}(X)$ est $\bigsqcup_{i \in I_j} X_i \bigsqcup \bigsqcup_{i=j+1}^{m_X} X_i$ avec I_j le plus grand sous-ensemble de I qui ne contient que des entiers inférieurs ou égaux à j .

Nous notons que la propriété est vérifiée pour $\text{EnumSupportSol1Var}(A, B, X, \emptyset)$. Soit $0 \leq n < m_X$ un entier. Supposons que la propriété soit vraie pour n . Donc, il existe un chemin dans l'arbre des appels récursifs aboutissant à un appel tel que $\mathcal{S}(X)_n = \bigsqcup_{i \in I_n} X_i \bigsqcup \bigsqcup_{i=n+1}^{m_X} X_i$ pour support solutions, avec I_n le plus grand sous-ensemble de I qui ne contient que des entiers inférieurs ou égaux à n . Deux cas sont possibles pour $\mathcal{S}(X)_n$. Soit il est égal à $\mathcal{S}(X')$ et la propriété suit. Soit $\mathcal{S}(X)_n$ contient une composante connexe qui n'est pas dans $\mathcal{S}(X')$. Posons X_j la plus petite composante connexe de $\mathcal{S}(X)_n$ telle que j n'est pas dans I .

Par construction, l'appel énumérant $\mathcal{S}(X)_n$ engendre un nouvel appel tel que $\mathcal{S}(X)_j = \mathcal{S}(X)_n \bigsqcup X_j$. Induisant que,

$$\mathcal{S}(X)_j = \bigsqcup_{i \in I_n} X_i \bigsqcup \bigsqcup_{i=n+1}^{j-1} X_i \bigsqcup \bigsqcup_{i=j}^{m_X} X_i \bigsqcup X_j = \bigsqcup_{i \in I_n} X_i \bigsqcup \bigsqcup_{i=n+1}^{j-1} X_i \bigsqcup \bigsqcup_{i=j+1}^{m_X} X_i.$$

Par construction $\bigsqcup_{i=n+1}^{j-1} X_i$ est l'ensemble des composantes connexes qui ont un indice supérieur ou égal à $n+1$ et inférieur ou égal à $j-1$. Donc si $j = n+1$ cet ensemble est vide. Sinon, dû au choix de j , il contient uniquement des composantes connexes qui ont un indice dans I . Nous inférons que $\mathcal{S}(X)_j = \bigsqcup_{i \in I_j} X_i \bigsqcup \bigsqcup_{i=j+1}^{m_X} X_i$ avec I_j le plus grand sous-ensemble de I ne contenant que des entiers inférieurs ou égaux à j . Nous en concluons que le cas et le lemme suivent. $\square$

Démontrons maintenant la condition 3 du Théorème 4.26. Pour cela, nous justifions que la condition sur D' de la ligne 3 empêche la production de doublons.

Lemme 4.28. Soient A et B des FDDS et soit X la solution renvoyée par l'Algorithme 4.19 sur A et B triés selon $\leq_{ct}$. Alors, l'exécution de $\text{EnumSupportSol1Var}(A, B, X, \emptyset)$ ne produit aucun doublon.

Démonstration. De l'hypothèse, nous supposons sans perte de généralité que X est égal à $\{X_1, \dots, X_{m_X}\}$ tel que $X_i <_{ct} X_{i+1}$ pour tout entier i entre 1 et m_X . Soit X' une solution produite par $\text{EnumSupportSol1Var}(A, B, X, \emptyset)$. Donc, il existe I un sous-ensemble de $\{1, \dots, m_X\}$ tel que $X' = \bigsqcup_{i \in I} X_i$. Pour montrer le lemme, nous prouvons par récurrence sur les valeurs prises par D qu'il n'existe qu'un unique chemin dans l'arbre des appels récursifs tel que le n -ème appel a $\mathcal{S}(X)_n = \bigsqcup_{i \in I_n} X_i \bigsqcup \{X_{n+1}, \dots, X_{m_X}\}$ pour valeur de support de X , avec I_n le plus grand sous-ensemble de I qui ne contient que des entiers inférieurs ou égaux à n .

Notons que la propriété est vérifiée quand $D = \emptyset$. En effet, il n'y a que l'appel $\text{EnumSupportSol1Var}(A, B, X, \emptyset)$ qui admet cette valeur pour D . Soit $0 \leq n < m_X$ un entier. Supposons que la propriété soit vraie pour n . Constatons que si $\mathcal{S}(X)_n = \mathcal{S}(X')$ alors la propriété est vérifiée. Donc nous supposons de plus que $\mathcal{S}(X)_n \neq \mathcal{S}(X')$. Par hypothèse de récurrence, il n'existe qu'une unique suite d'appels récursifs telle que

4. Résolution d'équations sur les supports de FDDS – 4.2. Équation linéaire sur les supports

$\mathcal{S}(X)_n = \boxplus_{i \in I_n} X_i \boxplus \{X_{n+1}, \dots, X_{m_X}\}$ avec $\mathcal{S}(X)_n$ la valeur du support de la solution produite et X_n la composante connexe en entrée de cet appel. Pour montrer l'hérédité, il nous faut prouver deux propriétés.

1. Ce n -ème appel engendre uniquement un ayant pour valeur de support $\mathcal{S}(X)_j = \boxplus_{i \in I_j} X_i \boxplus \{X_{j+1}, \dots, X_{m_X}\}$, avec X_j la valeur prise par D dans cet appel.
2. Aucun appel autre que celui qui a pour valeur de support $\mathcal{S}(X)_n$ n'engendre un appel qui a pour valeur de support $\mathcal{S}(X)_j$.

Dans l'arbre d'appel récursif, dont les nœuds ont pour nom les valeurs données en entrée de $\mathcal{S}(X)$ et D , ces deux propriétés se traduisent en :

1. Il n'y a pas de multi-arcs entre $\mathcal{S}(X)_n, X_n$ et $\mathcal{S}(X)_j, X_j$.
2. Les seuls arcs entrants dans $\mathcal{S}(X)_j, X_j$ sortent de $\mathcal{S}(X)_n, X_n$.

Démontrons pour commencer que la propriété 1 est vérifiée. Soit X_j une composante connexe de $\mathcal{S}(X)_n$ telle que $j > n$ est le plus petit entier possible vérifiant que $j \notin I$. En d'autres termes, X_j est la plus petite composante connexe de $\mathcal{S}(X)_n$ telle que $X_j \notin \mathcal{S}(X')$ et $X_n <_{ct} X_j$. Par un raisonnement similaire à celui utilisé pour la preuve du Lemme 4.27 nous déduisons que $\mathcal{S}(X)_j = \mathcal{S}(X)_n \boxminus X_j$. Ainsi, retirer n'importe quelle autre composante connexe de $\mathcal{S}(X)_n$ ne permet pas d'obtenir $\mathcal{S}(X)_j$. Et donc comme dans l'Algorithme 4.25, chaque composante connexe n'est retirée au plus qu'une seule fois par appel récursif, l'affirmation suit.

Prouvons maintenant que la propriété 2 est vérifiée. Remarquons que dans l'Algorithme 4.25, un appel récursif n'engendre que des appels où nous retirons une composante connexe du support en entrée. Donc pour montrer cette propriété, nous pouvons ne considérer que les appels prenant en entrée Y et X_y tels qu'il existe une composante connexe $X'_y > X_y$ de $\mathcal{S}(Y)$ telle que $\mathcal{S}(X)_j = \mathcal{S}(Y) - X'_y$. Ainsi, nous avons

$$\mathcal{S}(X)_n = \mathcal{S}(X)_j \boxplus X_j = (\mathcal{S}(Y) \boxminus X'_y) \boxplus X_j.$$

Nous montrons que $\mathcal{S}(X)_n = \mathcal{S}(Y)$ et $X_j = X'_y$. Ainsi, la preuve de la propriété suivra par l'hypothèse de récurrence. Remarquons qu'il est suffisant de montrer que $X_j = X'_y$. En effet, si c'est le cas, nous avons

$$\mathcal{S}(X)_n = (\mathcal{S}(Y) \boxminus X'_y) \boxplus X_j = \mathcal{S}(Y).$$

Supposons par l'absurde que $X'_y \neq X_j$. Cela induit que nous pouvons commuter l'union et le retrait. En symbole, $(\mathcal{S}(Y) \boxminus X'_y) \boxplus X_j = (\mathcal{S}(Y) \boxplus X_j) \boxminus X'_y$. Ceci impliquant que $\mathcal{S}(X)_n \boxplus X'_y = \mathcal{S}(Y) \boxplus X_j$. Comme $X'_y \neq X_j$ deux cas sont possibles. Soit $X_j <_{ct} X'_{c'_y}$. Donc comme $X_{c_n} <_{ct} X_j$, nous déduisons de la définition de $\mathcal{S}(X)_n$ que ce dernier contient X_j et X'_y . De ce fait, $\mathcal{S}(X)_j = \mathcal{S}(X)_n \boxminus X_j$ contient aussi X'_y . Il suit que $\mathcal{S}(X)_j$ est différent de $\mathcal{S}(Y) \boxminus X'_{c'_y}$, contradiction. La preuve pour l'inégalité inverse est symétrique. Nous concluons que $X'_y = X_j$ et l'hérédité et le lemme suivent. $\square$

Terminons avec la preuve de la condition 3 du Théorème 4.26. Notre réflexion repose sur le théorème suivant.

Théorème 4.29 (Folklore). *Soit $\mathcal{A}$ un algorithme récursif et $\mathcal{J}$ une instance d'un problème tels que $\mathcal{A}(\mathcal{J})$ énumère toutes les solutions de $\mathcal{J}$ sans doublons. Alors, le délai de $\mathcal{A}$ est polynomial sur la taille de $\mathcal{J}$ si les conditions suivantes sont vérifiées :*

1. *la profondeur de l'arbre des appels récursifs de $\mathcal{A}(\mathcal{J})$ est polynomiale sur la taille de $\mathcal{J}$*
2. *tous les appels récursifs conduisent à une solution et le temps de calcul de $\mathcal{A}$ hors appels récursifs est polynomial sur la taille de $\mathcal{J}$.*

Proposition 4.30. *Soient A et B des FDDS et soit X la solution renvoyée par l'Algorithme 4.19 sur A et B triés selon $\leq_{ct}$. Alors $\text{EnumSupportSol1Var}(A, B, X, \emptyset)$ énumère l'ensemble des solutions de $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ avec un délai polynomial sur la taille du support de B .*

Démonstration. Comme l'Algorithme $\text{EnumSupportSol1Var}$ (4.25) est récursif et que par les Lemmes 4.27 et 4.28 $\text{EnumSupportSol1Var}(A, B, X, \emptyset)$ énumère l'ensemble des solutions de $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ si nous montrons que cet appel respecte les conditions du Théorème 4.29 alors le lemme suivra.

Or, nous constatons que l'arbre de récursion de $\text{EnumSupportSol1Var}(A, B, X, \emptyset)$ a une profondeur qui est bornée par le nombre de composantes connexes dans $\mathcal{S}(X)$ qui est polynomial sur la taille de $\mathcal{S}(B)$ par le Corollaire 4.18. Donc la condition 1 du Théorème 4.29 est respectée.

En outre, nous remarquons que le nombre d'itérations de la boucle de l'Algorithme 4.25 est aussi borné par le nombre de composantes connexes dans $\mathcal{S}(X)$, donc polynomialement borné par la taille de $\mathcal{S}(B)$. De là, comme seule cette boucle démarre des appels récursifs et puisque toutes les opérations hors appels récursifs sont exécutables en temps polynomial sur la taille de $\mathcal{S}(B)$ et que chaque appel récursif produit une solution, nous déduisons que la condition 2 du Théorème 4.29 est vérifiée. $\square$

Soit $n > 0$ un entier. Pour généraliser ce résultat au cas où le polynôme P est sur n variables, nous introduisons un algorithme qui en partant d'une affectation des variables $X_1, \dots, X_n$ énumère l'ensemble des valeurs possibles X'_1 telles que $P(X'_1, \dots, X_n)$ est toujours solution. Puis pour chacun de ces X'_1 , nous effectuons la même opération sur X_2 et ainsi de suite. Cette procédure (Algorithme 4.31) peut être construite à partir d'une adaptation de l'Algorithme 4.25 comme suit. Elle prend en entrée un polynôme de n variables sur les supports (chaque coefficient A_i est encodé par un FDDS tel que $A_i = \mathcal{S}(A_i)$), ainsi qu'un FDDS B et des FDDS $X_1, \dots, X_n$ encodant une solution à $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$. Elle prend aussi en entrée un entier i étant le numéro de la variable à considérer et une composante connexe D . L'appel permettant d'énumérer l'ensemble des solutions de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ prendra 1 pour valeur de i et $\emptyset$ pour celle de D .

Observons que chaque solution produite par l'Algorithme 4.31 est, par construction, telle que $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$.

En considérant $\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)$ la solution de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ calculée grâce à la technique introduite dans la partie 4.2.1, c'est-à-dire celle maximisant le

Algorithme 4.31 EnumSupportSolVars($P, X_1, \dots, X_n, B, i, D$)

```

1: Si  $i > n$  alors
2:   renvoyer ,
3: Si  $i = 1$  ou  $D \neq \emptyset$  alors
4:   Énumérer  $X_1, \dots, X_n$ ,
5: Pour chaque composante connexe  $D'$  de  $X_i$  telle que  $D <_{ct} D'$  faire
6:   Soit  $X'_1, \dots, X'_n$  l'affectation construite à partir de  $X_1, \dots, X_n$  en retirant  $D'$  à  $X_i$ ,
7:   Si  $P(\mathcal{S}(X'_1), \dots, \mathcal{S}(X'_n)) = \mathcal{S}(B)$  alors
8:     EnumSupportSolVars( $P, X'_1, \dots, X'_n, B, i, D'$ ),
9: EnumSupportSolVars( $P, X_1, \dots, X_n, B, i + 1, \emptyset$ ).

```

nombre de composantes connexes et où chaque variable est triée selon $\leq_{ct}$, nous déduisons que EnumSupportSolVars($P, X_1, \dots, X_n, B, 1, \emptyset$) ne produit pas de doublons. Ceci est une conséquence directe du Lemme 4.28 et de la condition additionnelle positionnée avant l'énumération de la solution (sans cette condition nous aurions des doublons car lors du changement de variables nous ne changeons pas la solution).

Ainsi, pour montrer que cet algorithme permet d'énumérer toutes les solutions de l'équation $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$, il reste à prouver que le premier appel, à savoir EnumSupportSolVars($P, X_1, \dots, X_n, B, 1, \emptyset$), génère toutes les solutions. Si nous y parvenons, un raisonnement similaire à celui utilisé pour démontrer la Proposition 4.30 conclura que le délai entre deux solutions est polynomial sur la taille des entrées et donc le théorème suivant suivra.

Proposition 4.32. *Soit $n > 0$ un entier. Soit P un polynôme de support sur n variables et B un FDDS. Soient $\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)$ la solution de $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ calculée grâce à la technique utilisée pour prouver le Théorème 4.23. Alors l'appel récursif EnumSupportSolVars($P, X_1, \dots, X_n, B, 1, \emptyset$) énumère toutes les solutions de l'équation $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ avec un délai polynomial sur la taille des supports des entrées.*

Démonstration. Comme dit précédemment, il suffit de démontrer que toutes les solutions de l'équation $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ sont générées par le premier appel, à savoir EnumSupportSolVars($P, X_1, \dots, X_n, B, 1, \emptyset$). Pour cela, nous considérons $\mathcal{S}(Y'_1), \dots, \mathcal{S}(Y'_n)$ une solution de l'équation $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$.

Nous montrons cette assertion par récurrence sur les indices des variables. C'est-à-dire que nous montrons qu'il existe une suite d'appels récursifs telle que lorsque la valeur de i change alors $\mathcal{S}(X_j) = \mathcal{S}(Y_j)$ pour tout entier $1 \leq j \leq i - 1$. Nous prenons $i = 1$ comme cas de base. Cet appel est EnumSupportSolVars($P, X_1, \dots, X_n, B, 1, \emptyset$) et donc la propriété est vérifiée.

Soit $1 \leq i \leq n$. Supposons que la propriété est vraie pour i . Montrons alors qu'elle est vraie pour $i + 1$. Par hypothèse de récurrence, il existe une suite d'appels récursifs menant de EnumSupportSolVars($P, X_1, \dots, X_n, B, 1, \emptyset$) à celui où toutes les valeurs des X_j où $j < i$ sont fixées, i.e., EnumSupportSolVars($P, \mathcal{S}(Y_1), \dots, \mathcal{S}(Y_{i-1}), X_i, \dots, X_n, B, i, \emptyset$).

De plus, comme nous considérons l'ensemble des retraits de composante connexe D' possible dans la boucle 5 de l'Algorithme 4.31, un raisonnement similaire à celui utilisé dans la preuve du Lemme 4.27, nous permet de déduire qu'il existe une suite d'appels récursifs partant de $\text{EnumSupportSolVars}(P, \mathcal{S}(Y_1), \dots, \mathcal{S}(Y_{i-1}), X_i, \dots, X_n, B, i, \emptyset)$ et arrivant à $\text{EnumSupportSolVars}(P, \mathcal{S}(Y_1), \dots, \mathcal{S}(Y_i), X_{i+1}, \dots, X_n, B, i, \emptyset)$. Donc l'hérédité et la proposition suivent. $\square$

Ceci permet de conclure le théorème suivant.

Théorème 4.33. *Le problème de l'énumération de l'ensemble des solutions aux équations de la forme $P(\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)) = \mathcal{S}(B)$ avec P un polynôme linéaire sur les supports est dans la classe de complexité DelayP.*

4.2.3. Approche alternative : décomposition des équations

Soit A un FDDS. Alors $\mathcal{S}(A)$ peut être décomposé par l'union de plusieurs manières. Par exemple, le support $\mathcal{S}(A)$ peut être vu comme l'union du support de chaque composante connexe dans A . Et ainsi, si $A = A_1 + \dots + A_{m_A} = \mathcal{S}(A)$, avec toutes les A_i connexes, alors $\mathcal{S}(A) \boxtimes \mathcal{S}(X) = \boxplus_{i=1}^{m_A} \mathcal{S}(A_i) \mathcal{S}(X)$. De ce fait, résoudre l'équation $\mathcal{S}(A) \boxtimes \mathcal{S}(X) = \mathcal{S}(B)$ revient à chercher une solution au système d'équations :

$$\begin{cases} \mathcal{S}(A_1) \boxtimes \mathcal{S}(X) = \mathcal{S}(B_1) \\ \vdots \\ \mathcal{S}(A_{m_A}) \boxtimes \mathcal{S}(X) = \mathcal{S}(B_{m_A}) \end{cases}$$

avec les B_i des sous-ensembles (non nécessairement disjoints) de B tels que $\boxplus_{i=1}^{m_A} \mathcal{S}(B_i) = \mathcal{S}(B)$. Cette décomposition est l'adaptation aux supports de la technique utilisée dans [46, 14] et que nous avons rappelée dans la section 1.2.

S'il est assez clair, grâce aux résultats de la partie 4.2.1, que chaque équation de ce système est solvable en temps polynomial sur la taille de $\mathcal{S}(A_i)$ plus celle de $\mathcal{S}(B_i)$; nous ne savons pas s'il est possible de construire ce système en temps polynomial avant d'avoir trouvé la solution. Le problème vient des valeurs de $\mathcal{S}(B_i)$ qu'il associe à chaque $\mathcal{S}(A_i)$. En outre, il nous faut encore expliquer comment trouver, s'il en existe, une solution au système à partir de chaque solution aux équations. En effet, comme expliqué dans la partie 4.2.2, il peut exister de multiples solutions à chaque équation.

Pour trouver la valeur de chaque B_i en temps polynomial sans avoir la solution au départ, nous proposons de chercher la valeur de $\mathcal{S}(Y)$ maximisant le nombre de composantes connexes et telle que $\mathcal{S}(A_i) \mathcal{S}(Y)$ est un sous-ensemble de $\mathcal{S}(B)$. La valeur de B_i étant alors simplement $\mathcal{S}(A_i) \mathcal{S}(Y)$.

Lemme 4.34. *Soient $A = A_1 + \dots + A_{m_A}$ et B deux FDDS tels que chaque A_i est connexe. Soient $Y_1, \dots, Y_{m_A}$ des FDDS tels que $\mathcal{S}(Y_i)$ est la solution de $\mathcal{S}(A_i) \mathcal{S}(X) \subseteq \mathcal{S}(B)$ maximisant le nombre de composantes connexes. S'il existe $\mathcal{S}(X)$ tel que $\mathcal{S}(A) \mathcal{S}(X) = \mathcal{S}(B)$ alors $\mathcal{S}(X)$ est sous-ensemble de $\mathcal{S}(Y_i)$ pour tout entier i entre 1 et m_A .*

4. Résolution d'équations sur les supports de FDDS – 4.2. Équation linéaire sur les supports

Démonstration. Soit $1 \leq i \leq m_A$ un entier. Supposons que $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$. Il suit que $\mathcal{S}(A_i)\mathcal{S}(X)$ est un sous-ensemble de $\mathcal{S}(B)$. Comme, par définition, $\mathcal{S}(A_i)\mathcal{S}(Y_i)$ est aussi un sous-ensemble de $\mathcal{S}(B)$, nous déduisons que c'est encore le cas de $\mathcal{S}(A_i)\mathcal{S}(X) \boxplus \mathcal{S}(A_i)\mathcal{S}(Y_i) = \mathcal{S}(A_i)(\mathcal{S}(X) \boxplus \mathcal{S}(Y_i))$. Montrons que $\mathcal{S}(X)$ est un sous-ensemble de $\mathcal{S}(Y_i)$.

Supposons, par l'absurde, que ce n'est pas le cas. Donc $\mathcal{S}(Y_i)$ est un sous-ensemble strict de $\mathcal{S}(Y_i + X)$. Ainsi, comme la fonction support est un morphisme entre $(\mathbb{D}, +, \times)$ et $(\mathcal{S}(\mathbb{D}), \boxplus, \boxtimes)$, il résulte que $\mathcal{S}(Y_i) \boxplus \mathcal{S}(X) = \mathcal{S}(Y_i + X)$. Nous en concluons que $\mathcal{S}(Y_i)$ n'est pas maximal, contradiction. $\square$

Du lemme, nous inférons que s'il existe un FDDS X tel que $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ alors $\mathcal{S}(X)$ est un sous-ensemble de l'intersection des supports de chaque Y_i . En symbole, $\mathcal{S}(X) \subseteq \bigcap_{i=1}^{m_A} \mathcal{S}(Y_i)$. Cependant, nous ne savons pas encore de quel sous-ensemble il s'agit et leur test exhaustif serait exponentiel. C'est pourquoi, nous introduisons le lemme suivant qui montre que nous pouvons fixer la valeur de $\mathcal{S}(X)$ à $\bigcap_{i=1}^{m_A} \mathcal{S}(Y_i)$.

Lemme 4.35. *Soient $A = A_1 + \dots + A_{m_A}$ et B deux FDDS tels que chaque A_i est connexe. Soient $Y_1, \dots, Y_{m_A}$ des FDDS tels que $\mathcal{S}(Y_i)$ est la solution de $\mathcal{S}(A_i)\mathcal{S}(X) \subseteq \mathcal{S}(B)$ avec le plus de composantes connexes possibles. S'il existe une solution à $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ alors $\mathcal{S}(A)(\mathcal{S}(Y)) = \mathcal{S}(B)$ avec $\mathcal{S}(Y) = \bigcap_{i=1}^{m_A} \mathcal{S}(Y_i)$. De plus, $\mathcal{S}(Y)$ est la solution qui maximise le nombre de composantes connexes.*

Démonstration. Supposons qu'il existe un FDDS X tel que $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$. De plus, supposons que $\mathcal{S}(X)$ est la solution avec le plus de composantes connexes. Pour montrer que $\mathcal{S}(A)\mathcal{S}(Y) = \mathcal{S}(B)$, nous démontrerons l'inclusion dans les deux sens. Pour prouver que $\mathcal{S}(B)$ est un sous-ensemble de $\mathcal{S}(A)\mathcal{S}(Y)$, remarquons que le support de X est un sous-ensemble de $\mathcal{S}(Y)$ par le Lemme 4.34. Donc, nous avons que $\mathcal{S}(B) = \mathcal{S}(A)\mathcal{S}(X)$ est un sous-ensemble de $\mathcal{S}(A)\mathcal{S}(Y)$.

Pour l'autre inclusion, rappelons que la définition de X induit que $\mathcal{S}(A_i)\mathcal{S}(Y)$ est un sous-ensemble de $\mathcal{S}(B)$ pour tous les i . Donc, $\boxplus_{i=1}^{m_A} \mathcal{S}(A_i)\mathcal{S}(Y)$ est aussi un sous-ensemble de $\mathcal{S}(B)$. $\square$

Ainsi, si nous considérons $A = A_1 + \dots + A_{m_A}$ et B deux FDDS tels que tous les A_i sont connexes, alors nous sommes en mesure de résoudre $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$, en trouvant des FDDS $Y_1, \dots, Y_{m_A}$ tels que $\mathcal{S}(A_i)\mathcal{S}(Y_i)$ est le plus grand sous-ensemble de $\mathcal{S}(B)$ possible pour chaque i , ce qui est réalisable en temps polynomial par le Corollaire 4.17. Puis de calculer X l'intersection des supports de tous les Y_i avant de vérifier que $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ ce qui est aussi réalisable en temps polynomial.

Exemple 4.36. *Soit $A = C_4 + C_9 + C_{36}$ et $B = C_{36}$. Nous calculons les solutions maximales de :*

- $\mathcal{S}(C_4)\mathcal{S}(X) \subseteq \mathcal{S}(C_{36})$ qui est $C_9 + C_{18} + C_{36}$.
- $\mathcal{S}(C_9)\mathcal{S}(X) \subseteq \mathcal{S}(C_{36})$ valant $C_4 + C_{12} + C_{36}$.
- $\mathcal{S}(C_{36})\mathcal{S}(X) \subseteq \mathcal{S}(C_{36})$ étant $C_1 + C_2 + C_3 + C_4 + C_6 + C_{12} + C_{18} + C_{36}$.

Enfin nous réalisons l'intersection de ces solutions et nous obtenons C_{36} qui est bien solution de $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$.

4. Résolution d'équations sur les supports de FDDS – 4.2. Équation linéaire sur les supports

Si cette approche est plus sophistiquée et potentiellement plus lourde computationnellement que celle introduite dans la partie 4.2.1, elle permet cependant d'améliorer le Corollaire 4.18 comme suit.

Lemme 4.37. *Soit $M = \mathcal{S}(A)\mathcal{S}(X)$ un monôme linéaire univarié sur les supports et soient X et B tels que $M(\mathcal{S}(X)) = \mathcal{S}(B)$ avec $\mathcal{S}(X)$ la solution qui admet le plus de composantes connexes. Alors, $\mathcal{S}(X)$ contient au plus $2m_B\sqrt{\ell(A)}$ composantes connexes, avec m_B le nombre de composantes connexes de $\mathcal{S}(B)$.*

Démonstration. Soit A_i une composante connexe de A avec longueur de cycle minimale. Soit Y_i un FDDS dont le support est la solution de $\mathcal{S}(A_i)\mathcal{S}(X) \subseteq \mathcal{S}(B)$ avec le plus de composantes connexes possibles. Alors par le Corollaire 4.18, le support de Y_i a au plus $2m_B\sqrt{\ell(A_i)}$ composantes connexes. Par le Lemme 4.34, s'il existe X tel que $\mathcal{S}(A)\mathcal{S}(X) = \mathcal{S}(B)$ alors le support solutions qui maximise le nombre de composantes connexes est un sous-ensemble de celui de Y_i . Donc le lemme suit. $\square$

Relevons que nous sommes en mesure de retirer la dépendance entre le nombre de composantes connexes de $\mathcal{S}(A)$ et celui des équations du système que nous créons. Pour cela, nous reprenons la notion de *graine* introduite pour les polynômes pseudo-injectifs. Pour rappel, la graine d'un polynôme pseudo-injectif est la plus petite longueur des cycles de ses coefficients non-constants. Ainsi, par définition, cette valeur divise la longueur de tous les cycles des coefficients non-constants. En nous reposant sur cette propriété, nous définissons que l'ensemble des graines de A est l'ensemble des entiers de $L(A)$ minimaux au sens de la division, c'est-à-dire qu'un entier g est une graine de A si et seulement si g appartient à $L(A_1 + \dots + A_m)$ et u ne divise pas g pour tout entier u de $L(A) - \{g\}$.

Exemple 4.38. *Si nous considérons $A = C_2 + C_3 + C_6 + C_{14} + C_{35} + C_{2267} + C_{2268}$ alors les graines de A sont 2, 3, 35 et 2267.*

Il est clair que cette notion de graine peut être définie de la même manière pour $\mathcal{S}(A)$. De ce fait, il nous est possible de décomposer $\mathcal{S}(A)$ en :

$$\bigsqcup_{g \in G(A)} \mathcal{S}(\mathcal{M}_g(A))$$

avec $G(A)$ l'ensemble des graines de $\mathcal{S}(A)$ et $\mathcal{M}_g(A)$ le multi-ensemble des composantes connexes de A qui ont un cycle dont la longueur est un multiple de g . Ainsi, si $\mathcal{S}(A)$ a r graines $g_1, \dots, g_r$, alors au lieu de résoudre le système du Lemme 4.34, nous résolvons :

$$\left\{ \begin{array}{l} \mathcal{M}_{g_1}(A)(\mathcal{S}(X)) \subseteq \mathcal{S}(\mathcal{M}_{g_1}(B)) \\ \vdots \\ \mathcal{M}_{g_r}(A)(\mathcal{S}(X)) \subseteq \mathcal{S}(\mathcal{M}_{g_r}(B)) \end{array} \right.$$

Nous pouvons également utiliser cette approche dans le cas multivarié. En effet, il suffit de traiter indépendamment chaque variable, c'est-à-dire créer les sys-

4. Résolution d'équations sur les supports de FDDS – 4.3. Des équations linéaires sur les supports à celles sur les FDDS

tèmes et faire l'intersection des Y_i pour chacune d'elles et vérifier que l'affectation ainsi construite est bien solution. Ceci est confirmé par le Lemme 4.22. De là, nous concluons que le nombre de composantes connexes dans l'ensemble des supports solutions est au plus $2nm_B\sqrt{g}$ avec n le nombre de variables, m_B le nombre de composantes connexes dans $\mathcal{S}(B)$ et $g = \max(\ell(A_1), \dots, \ell(A_n))$ si l'équation est $\mathcal{S}(A_1)\mathcal{S}(X_1) \boxplus \dots \boxplus \mathcal{S}(A_n)\mathcal{S}(X_n) = \mathcal{S}(B)$. Ceci prouvant le théorème suivant.

Théorème 4.39. *Soient $\mathcal{S}(A_1), \dots, \mathcal{S}(A_n)$ et $\mathcal{S}(B)$ des supports. S'il existe des supports $\mathcal{S}(X_1), \dots, \mathcal{S}(X_n)$ tels que $\mathcal{S}(A_1)\mathcal{S}(X_1) \boxplus \dots \boxplus \mathcal{S}(A_n)\mathcal{S}(X_n) = \mathcal{S}(B)$ alors le nombre de composantes connexes dans $\mathcal{S}(X_1) \boxplus \dots \boxplus \mathcal{S}(X_n)$ est borné par $2nm_B\sqrt{g}$ avec $g = \max(\ell(A_1), \dots, \ell(A_n))$, n le nombre de variables et m_B le nombre de composantes connexes dans $\mathcal{S}(B)$.*

4.3. Des équations linéaires sur les supports à celles sur les FDDS

Soient A et B deux FDDS. Supposons sans perte de généralité que $A = \sum_{i=1}^{m_A} a_i A_i$ et $B = \sum_{i=1}^{m_B} b_i B_i$ avec A_i et B_j étant triés selon $\leq_{ct}$. Soit $\mathcal{S}(Y)$ le support qui admet le plus de composantes connexes et tel que $\mathcal{S}(A)\mathcal{S}(Y) = \mathcal{S}(B)$. Dans cette section, nous présenterons une technique permettant de construire, si cela est possible, un FDDS X à partir de $\mathcal{S}(Y)$ tel que $AX = B$.

Remarquons que comme $AX = B$, nous avons que $\mathcal{S}(X)$ est un sous-ensemble de $\mathcal{S}(Y)$ par le Lemme 4.14. Ainsi, nous pouvons voir X comme la somme de tous les éléments de $\mathcal{S}(Y)$ précédés d'un coefficient étant un entier positif ou nul. En symboles, si nous supposons que $\mathcal{S}(Y) = X_1 \boxplus \dots \boxplus X_{m_Y}$ tel que $X_i <_{ct} X_{i+1}$, alors $X = x_1 X_1 + \dots + x_{m_Y} X_{m_Y}$ avec $x_i \geq 0$ un entier pour tous les i . Notre but maintenant est de trouver quelle valeur donner à ces coefficients.

Pour cela, rappelons que $\mathcal{S}(a_i A_i)\mathcal{S}(x_j X_j)$ est un sous-ensemble de $\mathcal{S}(B)$ pour tous les i, j . Donc le résultat de $A_i \times X_j$ est une somme d'éléments de $\mathcal{S}(B)$ précédée d'un coefficient étant un entier positif ou nul. En symboles, $A_i \times X_j = \sum_{k=1}^{m_B} c_{k,i,j} B_k$ où les $c_{k,i,j}$ sont des entiers positifs ou nuls. De ce fait, $(a_i A_i) \times (x_j X_j) = \sum_{k=1}^{m_B} (a_i \cdot x_j \cdot c_{k,i,j}) B_k$. Nous en déduisons le système d'équations linéaires suivant.

$$\begin{cases} \sum_{j=1}^{m_Y} (x_j \cdot \sum_{i=1}^{m_A} (a_i \cdot c_{1,i,j})) = b_1 \\ \vdots \\ \sum_{j=1}^{m_Y} (x_j \cdot \sum_{i=1}^{m_A} (a_i \cdot c_{m_B,i,j})) = b_{m_B} \end{cases} \quad (2)$$

Proposition 4.40. *Soient A et B deux FDDS. Supposons sans perte de généralité que $A = \sum_{i=1}^{m_A} a_i A_i$ et $B = \sum_{i=1}^{m_B} b_i B_i$ avec A_i, B_j des composantes connexes telles que $A_i <_{ct} A_{i+1}$ et $B_j <_{ct} B_{j+1}$ pour tous i, j . Soit $\mathcal{S}(Y)$ le support qui admet le plus de composantes connexes et tel que $\mathcal{S}(A)\mathcal{S}(Y) = \mathcal{S}(B)$. S'il existe un FDDS X tel que $AX = B$ alors le système (2) admet une solution sur les entiers naturels.*

4. Résolution d'équations sur les supports de FDDS – 4.3. Des équations linéaires sur les supports à celles sur les FDDS

Démonstration. Supposons qu'il existe X tel que $AX = B$. Donc comme explicité précédemment, nous pouvons réécrire X comme une somme d'éléments de $\mathcal{S}(Y)$ avec des coefficients, c'est-à-dire que $X = x_1 X_1 + \dots + x_{m_Y} X_{m_Y}$. Ainsi,

$$A \times X = \sum_{k=1}^{m_B} \left(\sum_{j=1}^{m_Y} (x_j \cdot \sum_{i=1}^{m_A} (a_i \cdot c_{k,i,j})) \right) B_k.$$

Nous en déduisons donc que $\sum_{j=1}^{m_Y} (x_j \cdot \sum_{i=1}^{m_A} (a_i \cdot c_{k,i,j})) = b_k$ pour tous les k . La direction suit. $\square$

Notons que cette transformation est réalisable en temps polynomial. En effet, elle consiste principalement en des calculs de produits $a_i A_i$ avec des X_j . Et comme le calcul de $\mathcal{S}(Y)$ est polynomial sur la taille de B par le Théorème 4.23, nous concluons que nous sommes en mesure de transformer une équation $AX = B$ sur les FDDS en un système d'équations linéaires sur les entiers naturels en temps polynomial.

La procédure que nous venons d'introduire ressemble fort à une réduction. Nous serions donc tentés de montrer l'implication inverse de la Proposition 4.40. Cependant, comme le problème que nous considérons n'est pas un problème de décision mais un problème fonctionnel, nous devons montrer qu'il est possible de construire une solution à l'équation $AX = B$ à partir des solutions du système (2) en temps polynomial.

Proposition 4.41. *Soient A et B deux FDDS. Supposons aussi que $A = \sum_{i=1}^{m_A} a_i A_i$ et $B = \sum_{i=1}^{m_B} b_i B_i$ avec A_i, B_j des composantes connexes avec $A_i <_{ct} A_{i+1}$ et $B_j <_{ct} B_{j+1}$ pour tous les i, j . Soit $\mathcal{S}(Y)$ le support qui admet le plus de composantes connexes et tel que $\mathcal{S}(A) \cdot \mathcal{S}(Y) = \mathcal{S}(B)$. Alors, nous pouvons transformer toutes les solutions du système (2) sur les entiers naturels en une solution à $AX = B$ en temps polynomial.*

Démonstration. Soient $Y_1, \dots, Y_{m_Y}$ les composantes connexes de $\mathcal{S}(Y)$. Supposons que le système (2) admet une solution $x_1, \dots, x_{m_Y}$. Ainsi, nous construisons, en temps polynomial, un système d'équations sur les FDDS en multipliant la ligne i de (2) par B_i . Notons que le système résultant admet aussi une solution. Maintenant, en sommant toutes les lignes, nous obtenons

$$\sum_{i=1}^{m_A} \sum_{j=1}^{m_Y} \sum_{k=1}^{m_B} (a_i \cdot x_j \cdot c_{k,i,j}) B_k = \sum_{k=1}^{m_B} b_k B_k.$$

Cette opération est bien réalisable en temps polynomial.

En substituant $\sum_{k=1}^{m_B} (a_i \cdot x_j \cdot c_{k,i,j}) B_k$ par $(a_i A_i) \times (x_j X_j)$, il résulte que

$$\sum_{i=1}^{m_A} \sum_{j=1}^{m_Y} (a_i A_i) \times (x_j X_j) = B.$$

Ces substitutions peuvent bien être réalisées en temps polynomial.

4. Résolution d'équations sur les supports de FDDS – 4.4. Conclusion et perspectives

Ainsi, en factorisant par $\sum_{i=1}^{m_A} (a_i A_i)$ nous déduisons que

$$\left(\sum_{i=1}^{m_A} (a_i A_i) \right) \times \left(\sum_{j=1}^{m_Y} (x_j Y_j) \right) = B$$

induisant que $AX = B$ avec $X = \sum_{i=1}^{m_Y} x_i Y_i$. □

Ceci conclut la preuve du théorème suivant.

Théorème 4.42. *Soient A et B des FDDS. Alors, résoudre $AX = B$ se réduit à la recherche d'une solution entière naturelle au système (2).*

Un raisonnement similaire permet de démontrer que la recherche d'une solution avec le moins (resp. le plus) de composantes connexes se réduit à la programmation linéaire en nombres entiers. En effet, il suffit de minimiser (resp. maximiser) la somme des x_j sous les contraintes :

$$\left\{ \begin{array}{l} \sum_{j=1}^{m_Y} x_j \cdot (\sum_{i=1}^{m_A} (a_i \cdot c_{1,i,j})) = b_1 \\ \vdots \\ \sum_{j=1}^{m_Y} x_j \cdot (\sum_{i=1}^{m_A} (a_i \cdot c_{m_B,i,j})) = b_{m_B} \\ x_1, \dots, x_{m_Y} \geq 0 \end{array} \right.$$

Ainsi, comme il est prouvé dans [32] que cette famille de programmes peut être résolue en temps polynomial si le nombre de variables est fixé, il suit que la recherche des solutions à l'équation $AX = B$ minimisant ou maximisant le nombre de composantes connexes est polynomiale si nous fixons le nombre de composantes connexes différentes de B et la taille du plus petit cycle de A . En effet, le nombre de variables du système créé est égal au nombre de composantes connexes dans $\mathcal{S}(Y)$ qui est lui-même borné par ces facteurs (Lemme 4.37). Notons qu'en nous reposant sur le Théorème 4.39, nous pouvons utiliser une approche similaire pour le cas multivarié. Ceci prouve le théorème suivant.

Théorème 4.43. *La résolution de $AX = B$ est réalisable en temps polynomial si nous fixons le nombre de composantes connexes différentes de B et la longueur du plus petit cycle de A . Elle est donc FPT avec les paramètres donnés ci-dessus.*

Pour rappel, un problème paramétré est dans la classe FPT s'il existe une fonction calculable f telle que ce problème est résoluble en temps $f(k)|x|^{\mathcal{O}(1)}$ avec x l'entrée du problème et k la valeur du paramètre pour l'entrée x . Le lecteur est invité à consulter [21] pour plus d'informations sur la complexité paramétrée.

4.4. Conclusion et perspectives

Dans ce chapitre, nous avons montré que décider s'il existe une solution à une équation polynomiale sur les supports avec côté droit constant est NP-complet pour

4. Résolution d'équations sur les supports de FDDS – 4.4. Conclusion et perspectives

les polynômes multivariés généraux mais est polynomial si le polynôme est linéaire. Pour montrer ce second point, nous avons introduit un type de solution maximale calculable en temps polynomial. Nous avons par la suite établi qu'à partir de cette solution maximale, nous pouvions énumérer toutes les autres solutions avec un délai polynomial. Après quoi, nous avons introduit une méthode permettant de borner le nombre de composantes connexes présentes dans une solution en fonction du nombre de variables dans le polynôme, du nombre de composantes connexes dans la partie droite et d'une des plus petites longueurs de cycle parmi les coefficients. Cette borne nous a permis de montrer que la résolution de $AX = B$ est FPT avec comme paramètre le nombre de composantes connexes différentes de B et $\ell(A)$. Pour cela, nous avons utilisé le fait que la programmation linéaire en nombres entiers était elle-même FPT avec pour paramètre le nombre de variables.

Avec cette vision, nous pouvons aussi utiliser la programmation linéaire sur les nombres réels. Dans ce cas, la résolution des programmes linéaires est réalisable en temps polynomial [51]. De là, nous pouvons soit utiliser des approches, efficaces en pratique, pour obtenir une solution sur les entiers naturels, soit faire les arrondis nécessaires et obtenir une “bonne” approximation de la solution entière naturelle optimale. Dans ce dernier cas, nous pourrions traduire cette solution approximée des entiers naturels au FDDS. Cependant, la résolution de $AX = B$ n'étant pas un problème d'optimisation, nous obtiendrions un FDDS tel que $AX \neq B$.

De plus, la programmation linéaire en nombres entiers est souvent utilisée pour montrer que des problèmes sont approximables à facteur constant. Il serait donc intéressant d'étudier cette piste, dans des travaux futurs, pour essayer de montrer qu'il est possible d'obtenir des *sur-approximations* ou des *sous-approximations*, avec un écart constant entre le nombre de composantes connexes de AX et celui de B .

Nous pourrions aussi essayer d'adapter la preuve de NP-difficulté des équations polynomiales avec un nombre non-borné de variables au problème où le nombre de variables est borné, par exemple à 1. En d'autres mots, nous voudrions savoir si décider s'il existe une solution à $P(\mathcal{S}(X)) = \mathcal{S}(B)$ peut être résolu efficacement ou bien est NP-complet.

Troisième partie

Extensions des résultats

5. Sous- et sur-approximation du résultat

Sommaire

5.1. Énumération des solutions de $P(X) \leq B$	164
5.1.1. Les solutions fusion-free de $AX^k \leq B$	166
5.1.2. Fusions de solutions fusion-free	173
5.1.3. Énumération de l'ensemble des solutions de $AX^k \leq B$	181
5.1.4. Énumération de l'ensemble des solutions de $P(X) \leq B$	182
5.1.5. Généralisation au cas multivarié	185
5.2. Résolution des inéquations $P(X_1, \dots, X_n) \geq B$	189
5.2.1. Le cas $AX \geq B$	189
5.2.2. Le cas $P(X) \geq B$	192
5.2.3. Le cas $P(X_1, \dots, X_n) \geq B$	194
5.3. Conclusion et perspectives	195

Comme expliqué dans les chapitres précédents, nous ne connaissons à ce jour aucun algorithme polynomial capable de diviser deux FDDS. Et *a fortiori* nous n'en connaissons aucun capable de résoudre des équations de la forme $P(X) = B$ avec P un polynôme de FDDS. Cependant, si nous considérons les résultats du chapitre précédent, il semblerait que la difficulté pour formuler un algorithme pour la division ne vient pas tant de choisir quelle composante connexe nous pouvons mettre dans X (trouver le support de X) que de la multiplicité associée à chacune d'elles. Ainsi, il serait intéressant d'étudier le problème où nous relâcherions la condition d'égalité sur les multiplicités et nous la remplacerions par une relation d'ordre. Nous obtenons ainsi une *sur-* ou *sous-approximation* en fonction du sens de l'inégalité.

Définition 5.1 (sur-approximation et sous-approximation). Soient A et B deux FDDS. Nous disons que A est une *sous-approximation* de B , notée $A \leq B$, si A est un sous-multi-ensemble de B . Inversement, nous disons que A est une *sur-approximation* de B , notée $A \geq B$, si B est un sous-multi-ensemble de A . Ou de manière équivalente, si A est un *sur-multi-ensemble* de B .

Si, comme nous le montrerons dans ce chapitre, le passage de l'égalité à l'inégalité a un intérêt théorique (impact sur le temps de calcul), il a aussi une utilité pratique. En effet, nous pouvons utiliser les sous-approximations afin de décrire un algorithme “branch and bound” pour résoudre $P(X) = B$. Aussi, comme expliqué au début de la

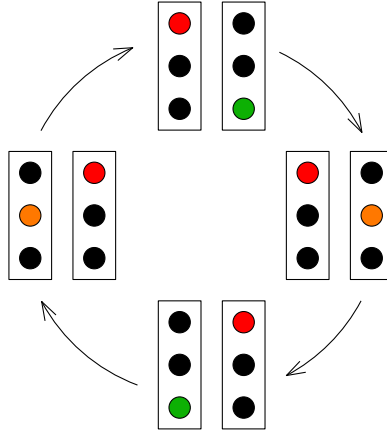


FIGURE 5.2. – Exemple de deux feux tricolore évoluant en parallèle.

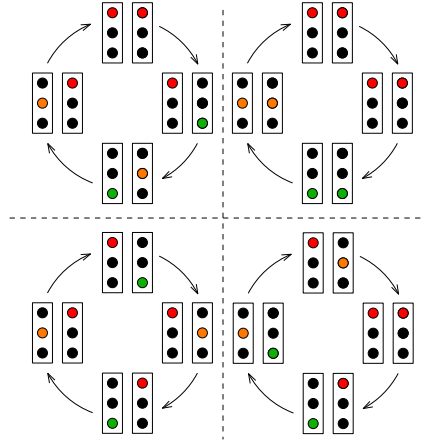


FIGURE 5.3. – Exécution synchrone de deux feux tricolore.

Section 1.1 les FDDS peuvent être issues de la synthèse d'exécution de phénomènes naturels. Or, au cours de cette synthèse, nous pourrions commettre une erreur, par exemple en réduisant deux copies d'une composante connexe en une seule ou bien en n'observant qu'une partie du phénomène. Prenons comme exemple le système induit par les feux tricolores de la Figure 5.2. Il est clair que ce système peut être décomposé comme le produit de deux feux tricolores évoluant en parallèle. Cependant, le produit de ces deux dynamiques ne produit pas celle décrite dans la Figure 5.2 mais celle de la Figure 5.3. Ainsi, en pratique, au lieu de résoudre $AX = B$, nous pouvons simplement chercher une sur- ou sous-approximation de B et filtrer les composantes connexes de AX non désirées grâce à la somme, c'est-à-dire en choisissant certains états initiaux.

Dans ce chapitre, nous présenterons (Section 5.1) un algorithme d'énumération (Algorithme 5.34) avec un délai polynomial de l'ensemble des FDDS X tels que $P(X)$ est une sous-approximation de B (Théorème 5.36). Il nous permettra aussi d'énumérer avec délai polynomial l'ensemble des FDDS X tels que $P(X + C) \not\leq B$ pour toute composante connexe C non vide. Il sera basé sur la *fusion* de plusieurs copies de composantes connexes. Nous montrerons aussi que nous pouvons le généraliser à

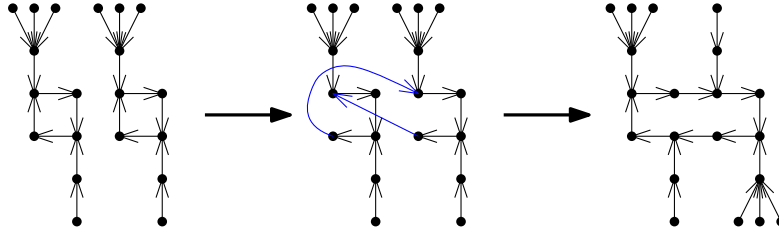


FIGURE 5.6. – Exemple de fusion de deux composantes connexes. Les deux composantes connexes de longueur de cycle 4 à gauche de la flèche centrale étant identiques, nous pouvons les fusionner en une seule composante connexe présentée à droite de la flèche centrale, ayant longueur de cycle 8 et le même unroll que les deux composantes de départ.

certaines familles de polynômes multivariés.

Puis nous traiterons le cas où $P(X)$ doit être une sur-approximation de B (Section 5.2). Dans cette partie, nous introduirons une forme standard des solutions aux inéquations $P(X) \geq B$. Remarquons que ces inéquations n'admettent pas toujours de solution.

Exemple 5.4. *Considérons $A = C_2$ et $B = C_3$, alors $AX \geq B$ n'admet pas de solution, car pour tous les FDDS X , nous avons que chacune des composantes connexes de AX a un cycle de longueur paire.*

Nous généraliserons de nouveau cet algorithme aux cas multivariés.

5.1. Énumération des solutions de $P(X) \leq B$

Dans cette section, nous introduisons un algorithme qui énumère l'ensemble des solutions de $P(X) \leq B$ avec un délai polynomial sur la taille de B où $P = \sum_{i=0}^m A_i X^i$ est un polynôme de FDDS. Comme dit précédemment, cet algorithme repose sur la notion de *fusion* de plusieurs copies d'une composante connexe.

Définition 5.5 (Fusion). Soit un FDDS contenant au moins $n > 0$ copies d'une composante connexe A de longueur de cycle p . Alors, nous pouvons *fusionner* n copies de A pour former une nouvelle composante connexe A' issue de l'enroulement de l'arbre d'unroll minimal de A à période np . Notons que $\mathcal{U}(A') = n\mathcal{U}(A)$.

En pratique, pour calculer la fusion de composantes connexes, nous nous contenterons de créer un cycle de longueur np et nous y enracinerons périodiquement les arbres enracinés dans le cycle de A . Cette opération est réalisable en temps polynomial. L'idée principale derrière cette notion de fusion est qu'en fonction du contexte, n copies d'une composante connexe ont le même comportement vis-à-vis du produit que la fusion de ces n composantes connexes, comme illustré dans le Lemme 3.47 ou dans la Figure 5.6.

5. Sous- et sur-approximation du résultat – 5.1. Énumération des solutions de $P(X) \leq B$

Pour simplifier l'analyse de notre procédure, nous supposons sans perte de généralité que le polynôme P n'a pas de terme constant (pas de A_0). Nous pouvons faire cette supposition car $P(X) \leq B$ si et seulement si $P(X) - A_0 \leq B - A_0$ est bien définie. Donc l'ensemble des solutions de $P(X) \leq B$ est le même que celui de $P(X) - A_0 \leq B - A_0$.

Afin de démontrer le résultat souhaité, nous nous reposons sur le lemme intuitif suivant.

Lemme 5.7. Soient $P = \sum_{i=1}^m A_i X^i$ et $P' = \sum_{i=1}^m A'_i X^i$ deux polynômes de FDDS tels que $A'_i \leq A_i$ pour tout entier i entre 1 et m . Soit B un FDDS. S'il existe X tel que $P(X) \leq B$, alors $P'(X) \leq B$.

La preuve de ce lemme est directe puisque $\leq$ est un ordre partiel compatible avec la somme et le produit. Donc, nous pouvons nous contenter de $P' = AX$ avec A une unique composante connexe d'un unique coefficient A_i de P . De ce fait, nous sommes en mesure de reformuler la propriété précédente en :

Lemme 5.8. Soit $P = \sum_{i=1}^m A_i X^i$. Soit B un FDDS. S'il existe X tel que $P(X) \leq B$, alors $AX^i \leq B$ avec A une composante connexe d'un A_i .

Ainsi, pour énumérer l'ensemble des solutions de $P(X) \leq B$, il suffit d'énumérer celles de $AX^i \leq B$ puis de les filtrer. Commençons donc par montrer que nous énumérons l'ensemble des solutions de $AX^k \leq B$ avec délai polynomial sur la taille de B si A est connexe et $k \geq 1$ est un entier.

Puisque A est connexe, nous avons que AX^k est un polynôme pseudo-injectif. Donc, étant donné un FDDS X , il n'existe que deux types de solutions Y telles que $AY^k = AX^k$:

1. celui dont nous pouvons "scinder" (opération inverse de fusionner) une composante connexe en au moins 2 plus petites et en restant égale à AX^k ,
2. celui où cette scission est impossible.

Les première sont appelées solutions *fusion-free* tandis que les autres sont simplement appelées solutions *fusionnées*. Notons que le Lemme 3.47 induit que s'il existe une solution non-vide alors il en existe une fusion-free.

Exemple 5.9. Considérons $A = C_3$ et $B = 6C_6$. Alors l'équation $AX = B$ admet $X = 6C_2$ comme solution fusion-free et $X = 3C_2 + C_6$ et $X = 2C_6$ comme solutions fusionnées. En effet, C_6 est la fusion de $3C_2$. Ces trois FDDS sont les seules solutions de l'équation $AX = B$.

Si nous remarquons directement que pour un B donné, il n'existe qu'une seule solution fusion-free à $AX^k = B$, ce n'est pas le cas pour les inéquations $AX^k \leq B$.

Exemple 5.10. Considérons $A = C_2$ et $B = 5C_4$, alors $AX = B$ n'admet pas de solution. Pour autant, $AX \leq B$ admet 3 solutions fusion-free : $\emptyset, C_4, 2C_4$.

En effet, l'ensemble des solutions de $AX^k \leq B$ peut être vu comme l'union de tous les ensembles de solutions des équations $AX^k = B'$ avec B' un sous-multi-ensemble de B . De cette remarque, nous déduisons qu'une approche naïve pour énumérer l'ensemble des solutions de $AX^k \leq B$ serait d'énumérer l'ensemble des sous-multi-ensembles de B et pour chacun d'eux, d'énumérer toutes les solutions de l'équation qui lui est associée.

Bien que correcte, cette méthode n'est pas efficace. En effet, le FDDS B peut admettre un nombre exponentiel de sous-multi-ensembles dont les équations associées n'admettent pas de solutions.

Exemple 5.11. *Considérons $A = C_2$ et $B = C_1 + C_2 + C_3 + C_4 + C_5$. Alors il existe 32 sous-ensembles de B et donc 32 équations, mais seulement 2 admettent une solution, à savoir $AX = \emptyset$ et $AX = C_2$.*

Donc, pour garantir le délai polynomial, nous devons d'abord introduire un algorithme qui trouve chaque solution fusion-free avec un délai polynomial sur la taille de B . Puis dans un second temps, il nous faudra introduire un algorithme capable d'énumérer l'entière des solutions fusionnées générables depuis une solution fusion-free avec un délai polynomial sur la taille de B . Et finalement, nous aurons à expliquer comment nous composons ces deux algorithmes pour énumérer l'ensemble des solutions de $AX^k \leq B$ toujours avec un délai polynomial sur la taille de B .

5.1.1. Les solutions fusion-free de $AX^k \leq B$

Pour trouver la structure de l'algorithme que nous recherchons et la principale propriété que nous devons montrer, nous proposons de réaliser une analyse exploratoire de la manière de décider si, étant donné une solution de $AX^k \leq B$, nous pouvons ajouter (ou non) une composante connexe C non-scindable dans X pour que $A(X + C)^k \leq B$.

Soit $k > 0$ un entier. Soient A un FDDS connexe et X et B deux FDDS tels que $AX^k \leq B$. Considérons B_1 la plus petite composante connexe de $B - AX^k$ selon $\leq_{ct}$. Alors trois cas sont possibles.

1. $AX^k = B$, ou en d'autres mots, $B_1 = \emptyset$. Directement, pour toute composante connexe C nous avons $A(X + C)^k \not\leq B$. Dans ce cas, nous disons que X est *maximal par inclusion*.
2. $B_1 \neq \emptyset$ mais il n'existe pas de C tel que $B_1 \leq A(X + C)^k \leq B$. Cela n'implique pas que X est maximal par inclusion. En effet, peut-être qu'il n'existe pas de C tel que $B_1 \leq A(X + C)^k$, mais que $B_2 \leq A(X + C)^k \leq B$ avec B_2 la seconde plus petite composante connexe de B selon $\leq_{ct}$. De ce fait, nous devons appliquer le même raisonnement sur $B - B_1$. Et nous pouvons bien le faire car $AX^k \leq B - B_1$, par définition de B_1 .
3. $B_1 \neq \emptyset$ et il existe C tel que $B_1 \leq A(X + C)^k \leq B$. Ainsi, nous pouvons ajouter C à X . Cependant, il est aussi possible que $A(X + 2C)^k \leq B$. Ou même qu'il existe une composante connexe $C' \neq C$ non-scindable telle que $A(X + C + C')^k \leq B$. Et

5. Sous- et sur-approximation du résultat – 5.1. Énumération des solutions de $P(X) \leq B$

dans ce cas, $A(X + C')^k \leq B$ induit que $X + C'$ est aussi une solution fusion-free si X l'était. Donc, notre algorithme doit aussi trouver ces solutions. Nous en déduisons que nous aurons à générer tous les FDDS $X + nC$ pour tous les entiers $n \geq 0$ tels que $A(X + nC)^k \leq B$. Et pour cela, il est suffisant de trouver le plus grand entier $n_{\max}$ tel que $A(X + (n_{\max} + 1)C)^k \not\leq B + AX^k$. Après quoi, il nous faudra appliquer le même raisonnement pour $X = X + nC$ et $B = B - uB_1$ avec n entre 0 et $n_{\max}$ et u le nombre de copies de B_1 dans $B - A(X + nC)^k$.

Cette analyse illustre la structure de l'algorithme que nous utiliserons pour énumérer les solutions fusion-free de $AX^k \leq B$ (Algorithme 5.17). De plus, elle met en lumière deux questions importantes auxquelles nous devons répondre : pourquoi si $X + C'$ est une solution de $AX^k \leq B$ alors il n'existe qu'un unique n et un unique C tels que $X + nC$ est fusion-free et $A(X + C')^k = A(X + nC)^k$, et comment nous pouvons calculer ce n et ce C en temps polynomial sur la taille de B . Les réponses à ces questions sont le propos des deux lemmes suivants.

Lemme 5.12. *Soit $k > 0$ un entier et soient A, X et B trois FDDS tels que A est connexe et X est une solution fusion-free de $AX^k \leq B$. Soit B_1 la plus petite composante connexe de $B - AX^k$ selon $\leq_{ct}$. Supposons qu'il existe une composante connexe C' vérifiant les deux conditions suivantes :*

1. B_1 est une composante connexe de $A(X + C')^k - AX^k$,
2. $A(X + C')^k$ est une sous-approximation de B .

Alors, il existe un unique couple n, C avec $n > 0$ un entier et C une composante connexe telle que $A(X + C)^k = A(X + nC)^k$ et $X + nC$ est une solution fusion-free de $AX^k \leq B$.

Démonstration. Soit p la plus petite période de $\min(\mathcal{U}(C'))$. Du Lemme 3.46, nous déduisons que $\ell(C') = q \cdot \text{lcm}(\text{alcm}_A B_1, p)$ avec q un diviseur de $\ell(B_1) / \text{lcm}(\text{alcm}_A B_1, p)$. Considérons le couple n, C avec $n = q$ et C qui est la composante connexe issue de l'enroulement de $\min(\mathcal{U}(C'))$ à période $\text{lcm}(\text{alcm}_A B_1, p)$. Par le Lemme 3.47, nous avons que $A(X + C')^k = A(X + nC)^k$. De plus, $X + nC$ est une solution fusion-free. En effet, X est fusion-free par hypothèse et C est la plus petite composante connexe possible, selon $\leq_{ct}$, telle qu'il existe un entier e vérifiant $A(X + C')^k = A(X + eC)^k$. Montrons que n, C est l'unique couple tel que $X + nC$ est une solution fusion-free et $A(X + C')^k = A(X + nC)^k$.

Soient d, D un couple entier strictement positif, composante connexe telle que $A(X + C')^k = A(X + dD)^k$ et tel que $X + dD$ est une solution fusion-free. Notons que si $\min(\mathcal{U}(C)) = \min(\mathcal{U}(D))$ alors $C = D$. En effet, le fait que ces deux FDDS ont le même arbre d'unroll minimal et les hypothèses induisent que D a une longueur de cycle qui est un multiple de $\text{lcm}(\text{alcm}_A B_1, p)$ et donc un multiple de $\ell(C)$. Donc pour que $X + dD$ soit fusion-free, il faudra que $\ell(D) = \ell(C)$.

Remarquons aussi que n est le seul entier tel que $A(X + C')^k = A(X + nC)^k$. Ainsi, si $D = C$ alors $d = n$. Donc pour que le lemme suive, il suffit de prouver que si $A(X + C')^k = A(X + dD)^k$ alors C' et D ont le même arbre d'unroll minimal. Pour cela, nous démontrons que $A(X + C')^k - AX^k$ et $A(X + dD)^k - AX^k$ ont la même plus petite composante connexe, selon $\leq_{ct}$, à savoir B_1 (Affirmation 5.13). Puis, nous nous

5. Sous- et sur-approximation du résultat – 5.1. Énumération des solutions de
 $P(X) \leq B$

servirons de ce résultat pour montrer que $\mathbf{a} \min(\mathcal{U}(C))\mathbf{x}^{k-1} = \mathbf{b} = \mathbf{a} \min(\mathcal{U}(C'))\mathbf{x}'^{k-1}$ avec $\mathbf{x}' = \min(\mathcal{U}(\Lambda_{\ell(B_1)}(X) + C'))$, $\mathbf{x} = \min(\mathcal{U}(\Lambda_{\ell(B_1)}(X) + nC))$, $\mathbf{b} = \min(\mathcal{U}(B_1))$ et $\mathbf{a} = \min(\mathcal{U}(A))$ (Affirmation 5.14). Grâce à ce résultat, nous pourrions conclure l'assertion.

Affirmation 5.13. *La composante B_1 est la plus petite composante connexe D' de $A(X + C')^k - AX^k$ selon $\leq_{ct}$.*

Preuve. Constatons d'abord que $B_1 \leq_{ct} D'$. En effet, comme $A(X + C')^k \leq B$, il suit que $A(X + C')^k - AX^k \leq B - AX^k$. Signifiant que D' est une composante connexe de $B - AX^k$ qui, par définition, est au moins aussi grande que B_1 par $\leq_{ct}$. Pour montrer l'inégalité inverse, observons que puisque B_1 est une composante connexe de $A(X + C')^k - AX^k$, la minimalité de D' dans ce FDDS induit que $D' \leq_{ct} B_1$ et donc $D' = B_1$. $\diamond$

Notons que pour les mêmes raisons que précédemment, nous avons que B_1 est la plus petite composante connexe de $A(X + dD)^k - AX^k$ selon $\leq_{ct}$.

Affirmation 5.14. $\mathbf{a} \min(\mathcal{U}(C))\mathbf{x}^{k-1} = \mathbf{b} = \mathbf{a} \min(\mathcal{U}(C'))\mathbf{x}'^{k-1}$.

Preuve. Montrons $\mathbf{b} = \mathbf{a} \min(\mathcal{U}(C'))\mathbf{x}'^{k-1}$. Notons qu'une réflexion similaire prouve que $\mathbf{a} \min(\mathcal{U}(C))\mathbf{x}^{k-1} = \mathbf{b}$.

Observons que dans $(\Lambda_{\ell(B_1)}(X + C'))^{k-1}$ il existe une composante connexe Y qui a pour arbre d'unroll minimal $\mathbf{x}'^{k-1}$. Soit D' la plus petite composante connexe de $AC'Y$ selon $\leq_{ct}$. Remarquons tout d'abord que par l'Affirmation 5.13, nous avons que $B_1 \leq_{ct} D'$ et qu'il existe une composante connexe Y' de $(\Lambda_{\ell(B_1)}(X + C'))^{k-1}$ telle que $AC'Y'$ admet B_1 comme composante connexe. Montrons que Y et Y' ont le même arbre d'unroll minimal.

Pour cela, constatons que le choix de A, C' et Y induit que le lcm de leurs longueurs de cycle est au plus $\ell(B_1)$. Et comme $B_1 \leq_{ct} D'$, la définition de $\leq_{ct}$ implique que $\ell(D') = \ell(B_1)$ et que $\mathbf{b} \leq \mathbf{a} \min(\mathcal{U}(C')) \min(\mathcal{U}(Y'))$.

De surcroît, comme l'arbre minimal d'unroll de Y est le plus petit arbre d'unroll de $(\Lambda_{\ell(B_1)}(X + C'))^{k-1}$, il résulte que

$$\mathbf{a} \min(\mathcal{U}(C'))\mathbf{x}'^{k-1} \leq \mathbf{a} \min(\mathcal{U}(C')) \min(\mathcal{U}(Y')).$$

Nous en déduisons que $\mathbf{b} = \mathbf{a} \min(\mathcal{U}(C')) \min(\mathcal{U}(Y'))$, nous inférons aussi que

$$\mathbf{a} \min(\mathcal{U}(C'))\mathbf{x}'^{k-1} \geq \mathbf{a} \min(\mathcal{U}(C')) \min(\mathcal{U}(Y'))$$

et donc

$$\mathbf{a} \min(\mathcal{U}(C'))\mathbf{x}'^{k-1} = \mathbf{a} \min(\mathcal{U}(C')) \min(\mathcal{U}(Y')).$$

Donc par le Corollaire 1.58, il suit que $\min(\mathcal{U}(Y)) = \min(\mathcal{U}(Y'))$. $\diamond$

À ce point, nous avons que $\mathbf{a} \min(\mathcal{U}(C))\mathbf{x}^{k-1} = \mathbf{a} \min(\mathcal{U}(C'))\mathbf{x}'^{k-1}$ par l'Affirmation 5.14. Donc, par le Corollaire 1.58, il suit que $\min(\mathcal{U}(C))\mathbf{x}^{k-1} = \min(\mathcal{U}(C'))\mathbf{x}'^{k-1}$. Pour conclure que $\min(\mathcal{U}(C)) = \min(\mathcal{U}(C'))$, nous démontrons que $\mathbf{x} = \mathbf{x}'$.

Algorithme 5.15 TrouverCompCoUnf(A, B, X, k)

- 1: calculer B_1 la plus petite composante connexe de $B - AX^k$ selon $\leq_{ct}$,
 - 2: **Si** $\ell(A)$ n'est pas un diviseur de $\ell(B_1)$ **alors**
 - 3: **renvoyer** $\perp$,
 - 4: calculer $\mathbf{a}$ le plus petit arbre d'unroll de A , $\mathbf{b}$ celui de B_1 et $\mathbf{m}$ celui de $\Lambda_{\ell(B_1)}(X)$,
 - 5: **Si** $\mathbf{a}\mathbf{m}^k < \mathbf{b}$ **alors**
 - 6: $\mathbf{x} \leftarrow \mathbf{b}/(\mathbf{a}\mathbf{m}^{k-1})$,
 - 7: **Sinon**
 - 8: $\mathbf{x} \leftarrow \sqrt[k]{\mathbf{b}/\mathbf{a}}$,
 - 9: **Si** $\mathbf{x}$ est indéfini **alors**
 - 10: **renvoyer** $\perp$,
 - 11: calculer p la plus petite période de $\mathbf{x}$,
 - 12: **renvoyer** l'enroulement de $\mathbf{x}$ à la période $\text{lcm}(\text{alcm}_A B_1, p)$.
-

Supposons par l'absurde que $\mathbf{x} \neq \mathbf{x}'$. Deux cas sont possibles mais ils sont symétriques l'un de l'autre. Donc nous nous contentons de traiter le cas où $\mathbf{x} < \mathbf{x}'$. De leur définition, il suit que soit $\mathbf{x}$ est un arbre de l'unroll de $\Lambda_{\ell(B_1)}(X)$, soit ce n'est pas le cas et donc $\mathbf{x} = \min(\mathcal{U}(C))$. Cependant, le premier cas entre en contradiction avec la définition de $\mathbf{x}'$. En effet, comme $\mathbf{x}$ est un arbre de l'unroll de $\Lambda_{\ell(B_1)}(X)$, il est aussi un arbre de l'unroll de $\Lambda_{\ell(B_1)}((X + C'))$, induisant que $\mathbf{x}'$ n'est pas minimal. Ainsi, il ne reste que le cas où $\mathbf{x} = \min(\mathcal{U}(C))$. Or, cette égalité et l'hypothèse impliquent que $\min(\mathcal{U}(C))\mathbf{x}^{k-1} = \mathbf{x}^k < \mathbf{x}'^k \leq \min(\mathcal{U}(X_i))\mathbf{x}'^{k-1}$. Donc nous avons $\min(\mathcal{U}(C))\mathbf{x}^{k-1} \neq \min(\mathcal{U}(C'))\mathbf{x}'^{k-1}$, contradiction. $\square$

De cette preuve nous déduisons l'Algorithme 5.15 pour calculer les valeurs correctes de C . Il prend 3 FDDS en entrée, à savoir A le coefficient, B la partie droite ainsi que X la solution en cours de construction. Il prend également un entier k qui est la puissance du monôme.

Observons que, par le Lemme 3.4, la comparaison de deux composantes connexes selon $\leq_{ct}$ est polynomiale sur la taille desdites composantes. Donc trouver la plus petite composante connexe de B selon $\leq_{ct}$ est polynomial. De là, comme nous testons que $\ell(A)$ divise $\ell(B_1)$, nous déduisons que si ce test est vérifié, alors $\mathbf{a}$ est de période $\ell(B_1)$. Donc le Lemme 2.4 induit que le plus petit arbre de $\mathcal{C}_n(\mathcal{U}(A))$ est égale à $\mathcal{C}_n(\mathbf{a})$ avec $n = 2\alpha + d(B)$ où α est le nombre d'arbres de $\mathcal{U}(B)$. De même, le plus petit arbre de $\mathcal{C}_n(\mathcal{U}(B_1))$ est $\mathcal{C}_n(\mathbf{b})$ tandis que celui de $\mathcal{C}_n(\mathcal{U}(\Lambda_{\ell(B)}(X)))$ est $\mathcal{C}_n(\mathbf{m})$. Notons que comme le produit de deux arbres de période $\ell(B_1)$ est aussi de période $\ell(B_1)$, il suit que $\mathbf{a}\mathbf{m}^j$ est de période $\ell(B_1)$ et ce pour tout entier $j > 0$. Donc par le Lemme 2.4, il s'avère que $\mathbf{a}\mathbf{m}^k < \mathbf{b}$ si et seulement si $\mathcal{C}_n(\mathbf{a})\mathcal{C}_n(\mathbf{m}) < \mathcal{C}_n(\mathbf{b})$. Donc, nous sommes en mesure de comparer $\mathbf{a}\mathbf{m}^k$ à $\mathbf{b}$ en temps polynomial sur la taille de B . Finalement, pour le calcul de $\mathbf{x}$, il est suffisant de calculer $\mathbf{y}$ qui est $\mathcal{C}_n(\mathbf{b})/(\mathcal{C}_n(\mathbf{a})\mathcal{C}_n(\mathbf{m})^{k-1})$, soit $\sqrt[k]{\mathcal{C}_n(\mathbf{b})/\mathcal{C}_n(\mathbf{a})}$. Puis de construire Y l'enroulement de $\mathbf{y}$ à période $\ell(B_1)$ et enfin vérifier que la troncature du plus petit arbre d'unroll de Y à profondeur n est égale à $\mathbf{y}$. Cette approche est suffisante car si $\mathbf{x}$ est bien défini, alors il est de période $\ell(B_1)$

Algorithme 5.17 EnumInfCo(A, B, B_o, X, k)

```

1: Tant que  $B \neq \emptyset$  faire
2:   soit  $B_1$  la plus petite composante connexe de  $B$  selon  $\leq_{ct}$  et soit  $b_1$  le nombre
   de copies de cette composante dans  $B$ ,
3:    $C \leftarrow \text{TrouverCompCoUnf}(A, B, X, k)$ ,
4:    $q \leftarrow 1$ ,
5:   Tant que  $A(X + qC)^k \leq B_o$  faire
6:     soit  $B'$  le FDDS  $B_o - A(X + qC)^k$  sans composante connexe inférieure ou
     égale à  $B_1$  selon  $\leq_{ct}$ ,
7:     EnumInfCo( $A, B', B_o, X + qC, k$ )
8:      $q \leftarrow q + 1$ ,
9:    $B \leftarrow B - b_1 B_1$ ,
10: Énumérer  $X$ .

```

par le Corollaire 2.28. Ceci induit que $\mathcal{C}_n(\mathbf{x}) = \mathbf{y}$ s'ils sont définis. Comme toutes les opérations décrites ci-dessus sont polynomiales sur la taille de B et puisque toutes les autres opérations de l'algorithme sont directement polynomiales, nous concluons le lemme suivant.

Lemme 5.16. *L'Algorithme 5.15 est polynomial sur la taille de B .*

Nous sommes maintenant capables d'exploiter l'analyse réalisée au début de cette partie afin de formuler une procédure récursive (Algorithme 5.17) qui énumère l'ensemble des solutions fusion-free de $AX^j \leq B$ lorsque A est connexe. L'algorithme prend comme entrées un entier k et quatre FDDS A, B, B_o et X . Elles joueront respectivement les rôles de puissance, de coefficient, du terme de droite modifié au cours des itérations, du terme de droite jamais altéré et de solution calculée. Ainsi, pour énumérer l'ensemble des solutions de $AX^k \leq B$, le premier appel de notre procédure prendra comme entrées $(A, B, B, \emptyset, k)$. La Figure 5.18 donne une exécution de cet algorithme.

Pour montrer que l'Algorithme 5.17 permet d'énumérer l'ensemble des solutions fusion-free de $AX^k \leq B$ avec un délai polynomial sur la taille de B , nous nous reposons sur les Théorèmes 4.26 et 4.29. Afin d'établir que la condition 1 du Théorème 4.26 est vérifiée (toutes les solutions de l'algorithme sont bien des solutions fusion-free de $AX^k \leq B$), il nous suffit de remarquer que l'Algorithme 5.17 n'est rien de plus que l'implémentation de la preuve du Lemme 5.12. Le lemme suivant permet de montrer les conditions 2 et 3 du Théorème 4.26.

Lemme 5.19. *Soit k un entier positif et soient A, X et B trois FDDS avec A connexe. Alors, toutes les solutions fusion-free de $AX^k \leq B$ sont énumérées par EnumInfCo($A, B, B, \emptyset, k$) une et une seule fois.*

Démonstration. Soit $X = \sum_{j=1}^{m_X} x_j X_j$ une solution fusion-free de $AX^k \leq B$ telle que chaque X_i est connexe et $X_i <_P X_{i+1}$ (Définition 3.42) et ce pour tout entier i entre 1

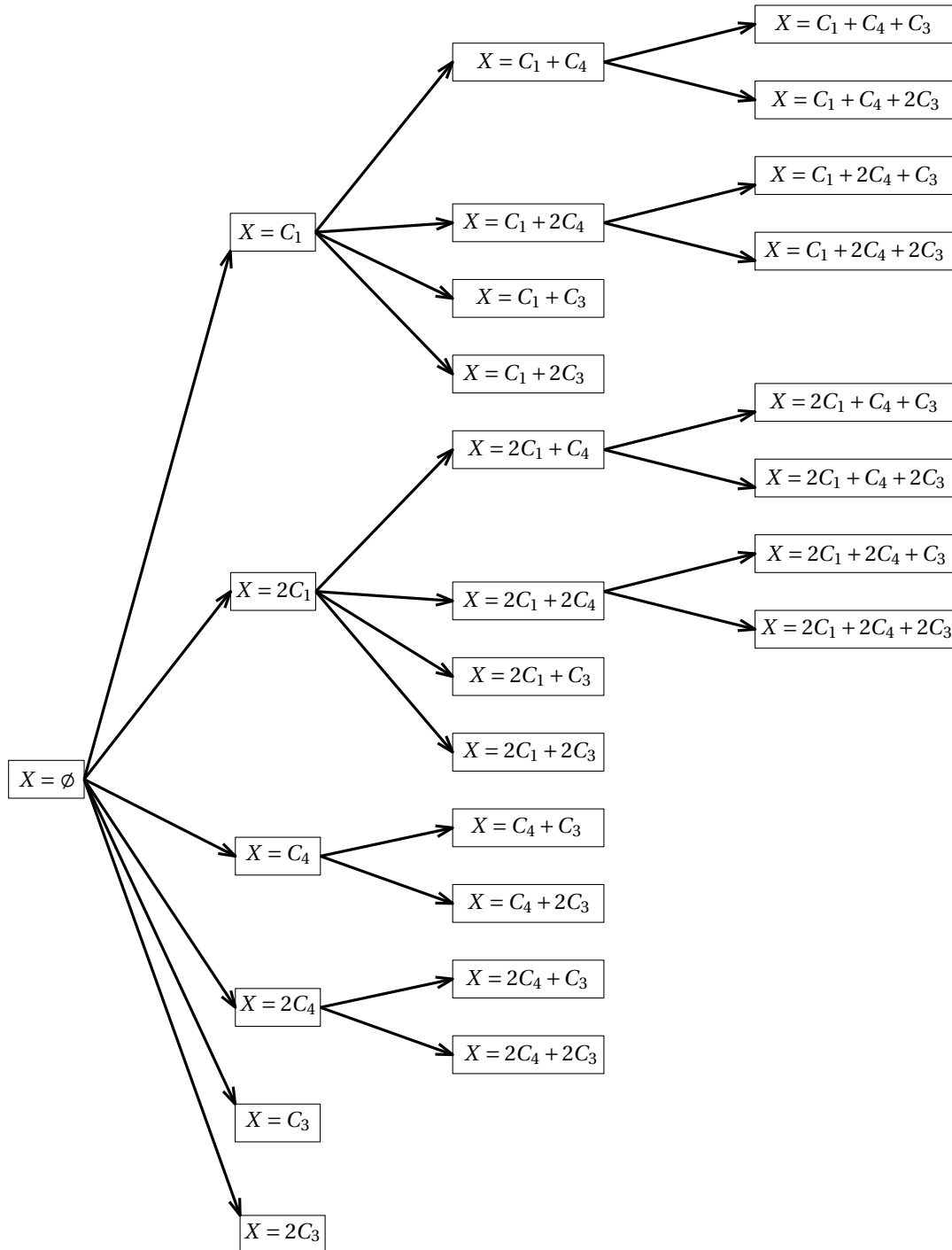


FIGURE 5.18. – L'arbre des appels récurrents de l'Algorithme 5.17 pour l'équation $C_2X \leq 2C_2 + 4C_4 + 2C_6$. Les composantes connexes sont toujours triées selon l'ordre $\leq_p$.

et $m_X - 1$. Nous montrons le lemme par récurrence sur la longueur du super-préfixe. Plus précisément, nous démontrons qu'étant donné un super-préfixe (Définition 3.5) de Y de longueur i , il n'existe qu'un unique appel récursif dont la solution est $\mathcal{P}_i(X)$. De plus, dans l'arbre des appels récursifs, cet appel aura une profondeur de i .

Prenons $i = 0$ comme cas de base. Alors, $\mathcal{P}_i(X) = \emptyset$. Remarquons qu'il existe un appel récursif dont la solution est $\emptyset$, à savoir le premier. De plus, comme la taille des solutions d'une branche de l'arbre des appels récursifs augmente strictement, il suit que le seul appel dont la solution a une taille de 0 est le premier. Le cas est vérifié.

Soit $0 \leq i < m_X$ un entier. Supposons que les propriétés soient vraies pour i et montrons que cela implique qu'elles sont vraies pour $i + 1$. Soit d l'index du B donné en entrée tel que B_d est la plus petite composante connexe de $A(\mathcal{P}_{i+1}(X))^k - A(\mathcal{P}_i(X))^k$ s'il est défini. Trois cas sont possibles.

1. $d = 1$. Prouvons d'abord l'existence de l'appel récursif. Pour cela, constatons que l'hypothèse d'induction et le Lemme 5.12 impliquent que le résultat de l'Algorithme 5.15 à la ligne 3 est égal à X_{i+1} . De plus, comme nous avons que $(X + nC)^k \leq_{ct} (X + (n+1)C)^k$, il résulte que si $A(X + nC)^k \not\leq B_0$ alors nous obtenons que $A(X + (n + j)C)^k \not\leq B_0$ pour tout entier $j \geq 0$. Conséquemment, tous les possibles et corrects nombres de copies q de C sont couverts par la seconde boucle de l'Algorithme 5.17 (lignes 5–8). Et donc en particulier, nous réalisons un appel récursif (à la ligne 7) où la valeur de la solution est augmentée de $x_{i+1}C = x_{i+1}X_{i+1}$. Ainsi, l'hypothèse de récurrence induit que la solution calculée est $\mathcal{P}_i(X) + x_{i+1}X_{i+1} = \mathcal{P}_{i+1}(X)$. De plus, cet appel récursif a profondeur $i + 1$ (i vient de l'hypothèse de récurrence et $+1$ a pour origine le nouvel appel).

Il reste à démontrer l'unicité de cet appel. Notons que durant un appel récursif mais aussi à la fin de la boucle externe, nous retirons de B toutes les copies de la composante B_1 . Ainsi, la définition de l'ordre $\leq_P$ induit que X_j ne peut être généré après X_{j+1} , et ce pour tous les j . Par voie de conséquence, un appel dont la solution est préfixée par $\mathcal{P}_i(X)$ est nécessairement un sommet du sous-arbre des appels récursifs enraciné dans l'appel de l'hypothèse de récurrence.

Maintenant, observons que lorsque nous ajoutons X_{i+1} à la solution de l'appel précédent, nous réalisons un appel récursif où la valeur de B en entrée ne contient aucun B_1 . De ce fait, le seul appel du sous-arbre des appels récursifs enraciné dans l'appel de l'hypothèse de récurrence qui a pour solution $\mathcal{P}_{i+1}(X)$ est celui décrit précédemment.

2. $d > 1$. Comme à la fin d'une itération de la boucle externe, nous retirons toutes les copies de B_1 ; nous concluons qu'après d itérations de la boucle la valeur de B_1 est B_d . Ainsi, le cas suit par le point précédent.
3. d n'est pas défini et donc $A\mathcal{P}_i(X) = A\mathcal{P}_{i+1}(X)$. Dû à l'hypothèse de récurrence, il existe un unique appel récursif de profondeur i dont la solution est X . Et pour des raisons similaires à celles du premier point, il n'existe aucun appel récursif dont la profondeur est différente de i et dont la solution est X .

Il ne reste plus qu'à montrer que l'appel de l'hypothèse de récurrence énumère X . Constatons que, comme nous retirons de B à chaque itération les compo-

santes $b_1 B_1$, la boucle externe se termine après que nous ayons retiré de B l'ensemble de ces composantes connexes. De plus, comme au cours des itérations de cette boucle, la solution donnée en entrée n'est pas modifiée, il suit qu'elle est bien énumérée à la fin de l'appel récursif. $\square$

Nous avons donc montré que $\text{EnumInfCo}(A, B, B, \emptyset, k)$ permet d'énumérer les solutions fusion-free de $AX^k \leq B$ sans doublons. Donc, pour prouver que la condition 4 du Théorème 4.26 est bien respectée, nous pouvons simplement démontrer que c'est le cas des conditions 1 et 2 du Théorème 4.29. Cela est bien le cas car la profondeur de l'arbre d'appel récursif est au plus le nombre de composantes connexes différentes de B . Et en outre, le nombre d'itérations de la boucle interne et de la boucle externe est polynomial et chacune de leurs opérations hors appel récursif est réalisable en temps polynomial sur la taille de B . De surcroît, chaque appel récursif produit une solution. Nous en concluons le théorème suivant.

Théorème 5.20. *Soit $k > 0$ un entier et soient A et B deux FDDS avec A connexe. Alors, $\text{EnumInfCo}(A, B, B, \emptyset, j)$ (Algorithme 5.17) énumère toutes les solutions fusion-free de $AX^k \leq B$ avec un délai polynomial sur la taille de B . En d'autres termes, le problème d'énumérer l'ensemble des solutions de $AX^k \leq B$ est dans la classe DelayP.*

5.1.2. Fusions de solutions fusion-free

Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS pseudo-injectif et soit B un FDDS. Dans cette partie, nous étendrons les résultats de résolution des équations de la forme $P(X) = B$ en temps polynomial en fonction de la taille de B (Section 3.2.1) pour atteindre l'énumération de l'ensemble des solutions de ces équations avec un délai polynomial en fonction de la taille de B . Notons que ce résultat implique que nous sommes capables d'énumérer l'ensemble des solutions X' de l'équation $AX'^k = AX^k$, avec délai polynomial sur AX^k , tel que X est une solution fusion-free de $AX^k \leq B$ avec A un FDDS connexe.

Pour rappel, à la fin de la Section 3.2.1 (juste avant le Théorème 3.48), nous avons introduit un algorithme qui calcule un X tel que $P(X) = B$, s'il en existe un. Il est aussi expliqué que ce X , appelé *solution canonique relaxée*, est l'unique solution qui maximise le nombre de composantes connexes. En outre, nous constatons aussi que la solution produite par l'algorithme introduit dans la Section 3.2.1 est triée selon l'ordre $\leq_P$. Finalement, relevons que cette solution est par construction fusion-free.

La procédure présentée dans cette partie opérera sur la solution canonique relaxée en fusionnant certaines composantes connexes en une nouvelle composante et en garantissant que le FDDS produit demeure une solution. Le lemme suivant fournit la condition nécessaire et suffisante pour réaliser ce type de fusion. Sa preuve est une conséquence directe du Lemme 3.47.

Lemme 5.21. *Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS pseudo-injectif et B un FDDS tel que l'équation $P(X) = B$ admet une solution. Soit $X = \sum_{i=1}^{m_X} x_i X_i$ tel que chaque X_i est connexe et tous les X_i sont différents deux à deux, une solution de $P(X) = B$ triée*

5. Sous- et sur-approximation du résultat – 5.1. Énumération des solutions de $P(X) \leq B$

selon $\leq_P$. Enfin, soit i un entier entre 1 et m_X et p la plus petite période de $\min(\mathcal{U}(X_i))$. Alors, pour tout diviseur q de

$$\frac{\ell(B - P(\mathcal{P}_{i-1}(X)))}{\text{lcm}\left(p, \text{alcm}_A(B - P(\mathcal{P}_{i-1}(X)))\right)},$$

nous avons $P(X - qX_i + C) = B$ si et seulement si $q \leq x_i$ avec C la fusion de q copies de X_i .

Soit $X_{\text{ca}} = \sum_{i=1}^{m_X} x_i X_i$ la solution canonique relaxée de $P(X) = B$, triée selon $\leq_P$ et telle que X_i est différent de X_j si $i \neq j$. Grâce au Lemme 5.21, nous proposons un algorithme qui énumère l'ensemble des solutions possibles de $P(X) = B$ obtenables via des fusions dans X_{ca} . L'idée principale est de considérer chaque composante X_i de X_{ca} et chaque diviseur q de $\ell(B')/\ell(X_i)$ et de calculer toutes les fusions de $x_i X_i$ selon q , avec B' un certain sous-multi-ensemble de B . L'algorithme prendra comme entrées :

- le polynôme pseudo-injectif P ,
- le FDDS B ,
- un FDDS X_f que nous utiliserons pour stocker le super-préfixe de la solution qui a déjà été calculé par les appels récurrents précédents,
- le FDDS X_w qui servira à stocker la partie de la solution que nous pourrions modifier dans les appels suivants,
- un entier $N \geq 0$ qui sera une borne inférieure pour les diviseurs. Cette variable nous permettra d'éviter la production de la même fusion plusieurs fois. En d'autres mots, nous l'utiliserons pour ne pas produire de doublons (ligne 8 de l'Algorithme 5.22).

Le premier appel prendra en entrée P , B , $N = 1$, $X_f = \emptyset$, et $X_w = X_{\text{ca}}$ (i.e., la solution canonique relaxée calculée dans la Section 3.2.1). Nous commençons par X_{ca} car nous pourrions produire toutes les solutions de $P(X) = B$ issues de la fusion de composantes connexes de X_{ca} sans “scinder” des composantes connexes de la solution originelle et sans annuler des fusions déjà réalisées. Dans cet algorithme, la variable *bound* (ligne 5) provient du Lemme 3.47.

La Figure 5.23 est un exemple d'une exécution de l'Algorithme 5.22. Notons qu'un appel récursif est une feuille de l'arbre des appels récurrents si et seulement si $X_w = \emptyset$.

En partant avec X_w le retour de l'algorithme décrit en fin de Section 3.2.1 et grâce au Lemme 5.21, nous sommes certains que les sorties de l'algorithme sont des solutions correctes de l'équation $P(X) = B$. Ainsi, pour conclure la correction de l'algorithme, il ne reste plus qu'à démontrer que ce dernier produit toutes les solutions. Pour ce faire, nous prouvons que l'équation $P(X) = B$ admet au plus une solution fusion-free, nommément la solution canonique relaxée, et que l'algorithme trouve toutes les solutions issues de la fusion de cette solution fusion-free.

Le premier point est une conséquence directe des Lemmes 3.46 et 3.47. Pour prouver le second, nous introduisons le concept de *profil de fusion*. Intuitivement, le profil de fusion d'une solution X de $P(X) = B$ décrit comment nous devons fusionner une

Algorithme 5.22 EnumSolRec(P, B, X_f, X_w, N)

```

1: Si  $X_w = \emptyset$  alors
2:   Énumérer  $X_f$ ,
3:   renvoyer ,
4: soit  $X_w = \sum_{i=1}^{m_x} x_i X_i$  trié selon  $\leq_P$  et tel que  $X_i$  est connexe et est non-isomorphe à
    $X_j$  si  $i \neq j$ ,
5: soit  $bound = \frac{\ell(B - P(X_f))}{\ell(X_1)}$ ,
6: Pour chaque entier  $q > N$  diviseur de  $bound$  faire
7:    $max \leftarrow \lfloor x_1 / q \rfloor$ ,
8:   Si  $max > 0$  alors
9:     soit  $C$  la fusion de  $q$  copies de  $X_1$ ,
10:    Pour chaque  $nb \in \{1, \dots, max\}$  faire
11:      Si  $nb = max$  et  $max \cdot q = x_1$  alors
12:         $d \leftarrow 1$ ,
13:        EnumSolRec( $P, B, X_f + nbC, X_w - (nb \cdot q)X_1, d$ ),
14: EnumSolRec( $P, B, X_f + x_1 X_1, X_w - x_1 X_1, 1$ ).

```

solution fusion-free de l'équation pour obtenir X .

Soit X un FDDS tel que $P(X) = B$ et soit m_X son nombre de composantes connexes. Le profil de fusion de X est le vecteur $\rho = (d_1, \dots, d_{m_X})$ calculé en utilisant la procédure suivante.

Algorithme 5.24. Étant donné un polynôme pseudo-injectif $P = \sum_{i=1}^m A_i X^i$ de graine g et deux FDDS X et B tels que X est trié selon $\leq_P$ et $P(X) = B$. Nous calculons le profil de fusion de X en :

1. initialisant i à 1 et ρ comme le vecteur de 1 de m_X éléments (avec m_X le nombre de composantes connexes de X),
2. trouvant C la première composante connexe de X (avec p la plus petite période de $\min(\mathcal{U}(C))$);
3. identifiant $B_1 = \min_{\leq_{ct}}(P(C))$,
4. changeant le i -ème élément de ρ par $\ell(C) / \text{lcm}(p, \text{alcm}_g B_1)$,
5. retirant C de X ,
6. ajoutant 1 à i ,
7. retournant à la ligne 2 si X n'est pas vide, renvoyant ρ sinon.

Exemple 5.25. Considérons l'exemple de la Figure 5.23 avec $X_{ca} = 4C_1 + 2C_4 + 2C_3$ et $X = 2C_2 + 2C_4 + C_6$. Dans ce cas, la graine du polynôme est $g = 2$. Observons que cet exemple est réalisé sur les permutations. Donc la valeur de p sera toujours égale à 1. En commençant avec $i = 1$ et $\rho = (1, 1, 1, 1, 1)$, nous avons $C = C_2$. En effet, nous avons $P(C_2) = 2C_2$ et $B_1 = C_2$. Puisqu'à ce point $\ell(B_1) / \text{lcm}(p, \text{alcm}_g B_1) = 2$, nous obtenons que $\rho = (2, 1, 1, 1, 1)$. Dans la ligne suivante (5), la valeur de X est mise à jour

5. Sous- et sur-approximation du résultat – 5.1. Énumération des solutions de $P(X) \leq B$

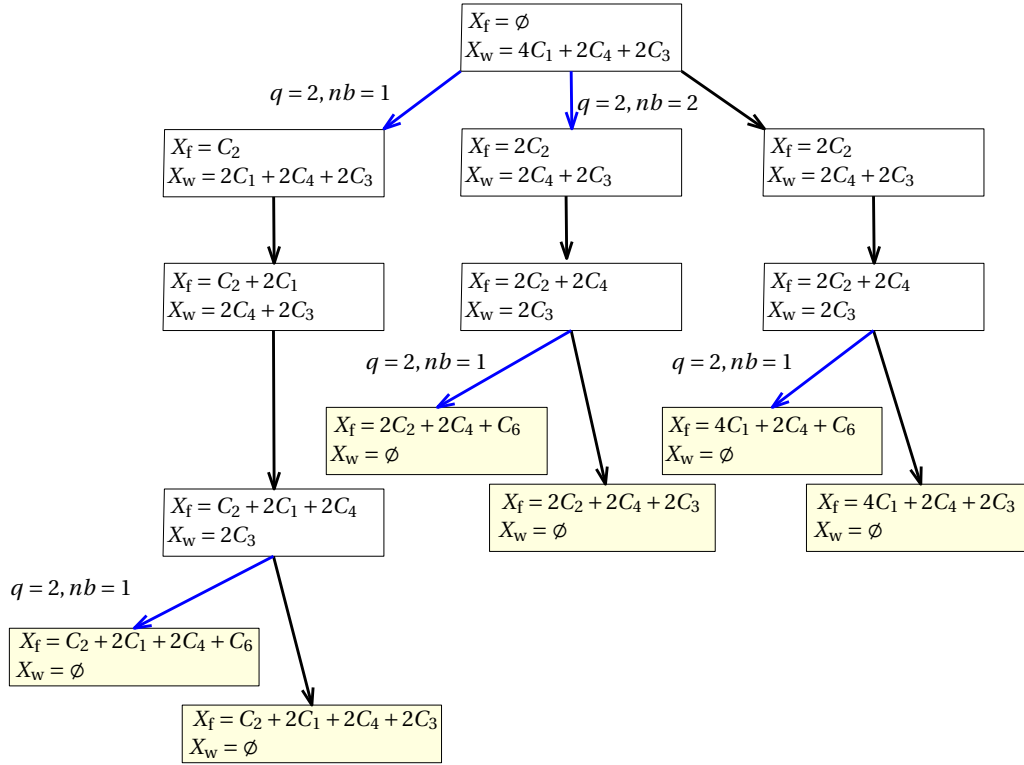


FIGURE 5.23. – L'arbre des appels récursifs pour l'équation $C_2X = 4C_2 + 4C_4 + 2C_6$. Les composantes connexes sont toujours triées selon l'ordre $\leq_p$. Si l'arc est bleu, cela signifie que nous exécutons l'appel de la ligne 13 et que l'opération de fusion est appliquée. Si l'arc est noir, alors nous exécutons l'appel de la ligne 14. Les sommets colorés sont les feuilles et sont ceux où l'algorithme énumérera une solution.

en $C_2 + 2C_4 + C_6$. Après quoi, avec $i = 2$, nous avons toujours que $C = C_2$, $B_1 = C_2$ et $\ell(B_1)/\text{lcm}(p, \text{alcm}_g B_1) = 2$. Donc $\rho = (2, 2, 1, 1, 1)$ et X est $2C_4 + C_6$. Lorsque $i = 3$, la valeur de C devient C_4 . Donc $P(C_4) = 2C_4$ et conséquemment $B_1 = C_4$. Puisque $\ell(B_1)/\text{lcm}(p, \text{alcm}_g \ell(B_1)) = 1$, la valeur de ρ reste inchangée et X devient $C_4 + C_6$. Constatons que si $i = 4$, alors nous sommes dans la même configuration que dans le cas précédent. De ce fait, ρ est inchangé et X passe à C_6 . Finalement, avec $i = 5$, nous avons $C = C_6$, $P(C_6) = 2C_6$ et $B_1 = C_6$. Cela induit que $\ell(B_1)/\text{lcm}(p, \text{alcm}_g \ell(B_1)) = 2$. Nous en déduisons que $\rho = (2, 2, 1, 1, 2)$. Maintenant, $X = \emptyset$. Donc, le profil de fusion de X est $\rho = (2, 2, 1, 1, 2)$. Remarquons que si nous fusionnons deux fois $2C_1$ en C_2 et une fois $2C_3$ en C_6 , alors nous obtenons bien X . Les valeurs 1 dans ρ signifient que les composantes correspondantes ne sont pas utilisées lors de fusions.

Comme dans la suite, nous utiliserons le profil de fusion pour montrer que l'Algorithme 5.22 calcule toutes les solutions possibles; nous devons démontrer que les profils de fusion décrivent totalement une solution. En d'autres mots, nous prouverons que l'Algorithme 5.24 est une fonction bijective entre les FDDS solutions de $P(X) = B$ et l'ensemble des profils de fusion des solutions de $P(X) = B$. Intuitivement,

chaque profil de fusion correspond à une et une seule solution.

Exemple 5.26. *Considérons $C_2X = 4C_2 + 4C_4 + 2C_6$ et $X_{ca} = 4C_1 + 2C_4 + 2C_3$ la solution canonique relaxée de cette équation. Considérons de plus le profil de fusion $\rho = (1, 1, 2, 1, 1, 2)$ d'une solution. Pour retrouver la solution correspondant au profil de fusion, nous commençons par le premier index du profil. Ce dernier est associé à 1. Donc nous fusionnons 1 copie de C_1 et nous ajoutons le résultat à la solution X . Puis nous retirons C_1 de X_{ca} . Nous obtenons ainsi $X = C_1$ et $X_{ca} = 3C_1 + 2C_4 + 2C_3$. Après quoi, nous incrémentons l'index de 1. Comme la valeur du profil est encore 1, nous répétons les mêmes opérations et obtenons $X = 2C_1$ et $X_{ca} = 2C_1 + 2C_4 + 2C_3$. L'entier associé au nouvel index est 2. Donc nous fusionnons 2 copies de C_1 et obtenons C_2 . Ainsi $X = 2C_1 + C_2$ et $X_{ca} = 2C_4 + 2C_3$. Les deux prochains entiers sont 1 et donc nous fusionnons deux fois 1 copie de C_4 . Ceci produit $X = 2C_1 + C_2 + 2C_4$ et $X_{ca} = 2C_3$. Le dernier entier du profil est 2. Donc nous fusionnons 2 copies de C_3 et nous obtenons $X = 2C_1 + C_2 + 2C_4 + C_6$ et $X_{ca} = \emptyset$. Notons que $C_2(2C_1 + C_2 + 2C_4 + C_6) = B$.*

Relevons que, puisque chaque opération de l'Algorithme 5.24 est déterministe, il décrit une fonction. Et comme par les Lemmes 3.46 et 3.47, chaque solution admet un profil de fusion; cette fonction est de plus totale. Il faut donc encore démontrer que c'est bien une bijection. Pour cela, il est suffisant de prouver son injectivité car l'ensemble des profils de fusion des solutions de $P(X) = B$ et celui des solutions de cette même équation sont finis.

Lemme 5.27. *Soient P un polynôme pseudo-injectif sur les FDDS et B un FDDS. Alors, chaque profil de fusion ρ décrit une unique solution de $P(X) = B$.*

Démonstration. Soient X et Y deux solutions de l'équation $P(X) = B$. Supposons que ces deux solutions ont le même profil de fusion. Donc elles ont le même nombre de composantes connexes, noté m_X . Supposons alors, sans perte de généralité, que $X = X_1 + \dots + X_{m_X}$ et $Y = Y_1 + \dots + Y_{m_X}$, où tous les X_i et Y_i sont des composantes connexes, et que X et Y sont triés selon l'ordre $\leq_P$. Nous montrons par récurrence sur la longueur du préfixe de X et Y qu'ils sont égaux. C'est-à-dire que nous prouvons que $\pi_i(X) = \pi_i(Y)$ pour tout entier i entre 0 et m_X .

Si nous prenons pour cas de base $i = 0$, alors la propriété est vérifiée par définition (les deux préfixes sont vides par définition). Soit i un entier tel que $m_X > i \geq 0$. Supposons que la propriété soit vraie pour i . Montrons alors que cela implique qu'elle est vraie pour $i + 1$. De l'hypothèse de récurrence, nous déduisons qu'il suffit de démontrer que $X_{i+1} = Y_{i+1}$.

Pour cela, nous prouvons que ces deux composantes connexes ont le même arbre d'unroll minimal puis la même longueur de cycle. Pour le premier point, considérons B_1 la plus petite composante connexe de $P(X) - P(\pi_i(X))$ selon $\leq_{ct}$. Étant donné que $P(X) = B = P(Y)$, nous déduisons que $P(X) - P(\pi_i(X)) = P(Y) - P(\pi_i(Y))$ de l'hypothèse de récurrence. De là, nous déduisons du Lemme 3.44 que X_{i+1} et Y_{i+1} appartiennent respectivement à $\Lambda_{\ell(B_1)}(X)$ et $\Lambda_{\ell(B_1)}(Y)$. De plus, par le Lemme 3.2 et le

5. Sous- et sur-approximation du résultat – 5.1. Énumération des solutions de $P(X) \leq B$

Théorème 2.52, il suit que $\Lambda_{\ell(B_1)}(X)$ et $\Lambda_{\ell(B_1)}(Y)$ ont le même unroll. Finalement, l'hypothèse de récurrence et le Lemme 3.45 induisent que $\min(\mathcal{U}(X_{i+1}))$ et $\min(\mathcal{U}(Y_{i+1}))$ sont égaux.

Soit p la plus petite période de cet arbre. Puisque X et Y ont le même profil de fusion, ils ont le même entier à la coordonnée $i + 1$. Donc par définition :

$$\frac{\ell(X_{i+1})}{\text{lcm}(p, \ell(B_1))} = \frac{\ell(Y_{i+1})}{\text{lcm}(p, \ell(B_1))}.$$

Nous en concluons que $\ell(X_{i+1}) = \ell(Y_{i+1})$ et que $X_{i+1} = Y_{i+1}$, signifiant que l'hérédité et le lemme suivent. $\square$

Puisque les profils de fusion des solutions $P(X) = B$ sont en bijection avec les dites solutions, nous pouvons les utiliser pour étudier l'arbre d'appel récursif de $\text{EnumSolRec}(P, B, \emptyset, X_{\text{ca}}, 1)$. Plus précisément, nous sommes en mesure de prouver que chaque profil de fusion décrit un unique chemin entre la racine et une feuille de cet arbre. Pour ceci, nous montrons en premier lieu que toute solution Y peut être générée depuis X_{ca} en lui appliquant les fusions décrites dans le profil de fusion de Y (Lemme 5.28). Puis que $\text{EnumSolRec}(P, B, \emptyset, X_{\text{ca}}, 1)$ réalisera également ces fusions (Lemme 5.29).

Lemme 5.28. Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme pseudo-injectif et soit B un FDDS. Soit X_{ca} la solution canonique relaxée de $P(X) = B$ triée selon $\leq_P$. Soit $Y = \sum_{i=1}^{m_Y} Y_i$, avec chaque Y_i connexe, une solution de l'équation $P(X) = B$, triée selon $\leq_P$, et soit un profil de fusion $\rho = (d_1, \dots, d_{m_Y})$. Alors Y_i est la fusion de d_i fois la première composante de $X_{\text{ca}} - \pi_{d_1 + \dots + d_{i-1}}(X)$.

Démonstration. Considérons $u_1, \dots, u_{m_Y}$ les entiers tels que $u_i = \sum_{j=1}^i d_j$. Supposons sans perte de généralité que $X_{\text{ca}} = \sum_{i=1}^{m_X} x_i X_i$ tel que chaque X_i est connexe et toutes les X_i sont différentes deux à deux. Pour la preuve de ce lemme, nous raisonnons par récurrence sur les coordonnées de ρ . Plus formellement, nous démontrerons que pour tous les i , l'entier u_i est tel que :

1. Y_i est la fusion de d_i copies de la première composante connexe de $X_{\text{ca}} - \pi_{u_{i-1}}(X)$,
2. $P(Y - \pi_i(Y) + \pi_{u_i}(X_{\text{ca}})) = B = P(X_{\text{ca}} - \pi_{u_i}(X_{\text{ca}}) + \pi_i(Y))$,
3. $P(Y - \pi_i(Y)) = P(X_{\text{ca}} - \pi_{u_i}(X_{\text{ca}}))$ et $P(\pi_i(Y)) = P(\pi_{u_i}(X_{\text{ca}}))$.

Si nous prenons comme cas de base $i = 0$, alors il suit directement car tout est 0 ou vide.

Soit $0 \leq i < m_Y$ un entier. Supposons que la propriété soit vraie pour i . Montrons alors qu'elle est aussi vraie pour $i + 1$. Par la condition 3 de l'hypothèse de récurrence, nous avons que $P(Y - \pi_i(Y)) = P(X_{\text{ca}} - \pi_{u_i}(X_{\text{ca}}))$. Donc le Lemme 3.47 induit qu'il existe une composante connexe C et un entier q tels que $P(Y - Y_{i+1} + qC) = P(Y)$. De là, par construction du profil de fusion, il suit que $q = d_{i+1}$. Par conséquent, nous avons que $P(Y - \pi_i(Y) - Y_{i+1} + \pi_{u_i}(X_{\text{ca}}) + d_{i+1}C) = B$ par la condition 2 de

l'hypothèse de récurrence. Et donc, Y_{i+1} est la fusion de $d_{i+1}C$. Nous savons que $X_{ca} - \pi_{u_i}(X_{ca})$ contient au moins d_{i+1} copies de C . Conséquemment, les conditions 2 et le Lemme 5.21 impliquent que $P(X_{ca} - \pi_{u_i}(X_{ca}) - d_{i+1}C + Y_{i+1}) = B$. Il suit que si nous montrons que $C = X_p$, avec X_p la première composante connexe de $X_{ca} - \pi_{u_i}(X_{ca})$, alors les conditions 1 et 2 seront vérifiées pour $i + 1$.

Remarquons que $P(C) - A_0$ et $P(X_p) - A_0$ ont tous les deux la même plus petite composante connexe par $\leq_{ct}$, notée D . En effet, si ce n'était pas le cas, nous aurions que $D_1 <_{ct} D_2$ car $X_p \leq_P C$, avec D_1 et D_2 les plus petites composantes connexes de respectivement $P(X_p) - A_0$ et $P(C) - A_0$. Et comme D_1 et D_2 sont aussi respectivement les plus petites composantes connexes, par $\leq_{ct}$, de $P(X_{ca} - \pi_{u_i}(X_{ca}))$ et de $P(Y - \pi_i(Y))$, car par définition $\mathcal{S}(P(C) - A_0) = \mathcal{S}(P(Y_{i+1}) - A_0)$, nous inférons que $P(X_{ca} - \pi_{u_i}(X_{ca}))$ est différent de $P(Y - \pi_i(Y))$, contredisant la condition 3.

Ainsi, par le Lemme 3.44 et la condition 3 de l'hypothèse de récurrence, il suit que $\ell(D) = \ell(P(X_{ca}) - P(\pi_{u_i}(X_{ca}))) = \ell(P(Y) - P(\pi_i(Y)))$. Donc, par la définition de X_{ca} et comme C est le plus relaxé possible, il suit que $\ell(X_p) = \ell(C)$ si et seulement si $\text{lcm}(p_1, \text{alcm}_g D) = \text{lcm}(p_2, \text{alcm}_g D)$, avec p_1 et p_2 les périodes minimales de leurs plus petits arbres d'unroll respectifs. De ce fait, pour conclure que les conditions 1 et 2 sont vérifiées, il nous suffit de prouver que X_p et C ont le même arbre d'unroll minimal.

Observons qu'il existe des entiers $j_1, j_2 > 0$ et des composantes connexes A et A' de respectivement A_{j_1} et A_{j_2} telles que D est une composante connexe de $AX_p^{j_1}$ et de $A'C^{j_2}$. Donc, comme l'ordre sur les arbres est compatible avec le produit, nous avons que $\min(\mathcal{U}(D)) = \min(\mathcal{U}(A) \min(\mathcal{U}(X_p))^{j_1}) = \min(\mathcal{U}(A') \min(\mathcal{U}(C))^{j_2})$. De plus, comme A, A', X_p et C ont des longueurs de cycle qui divisent $\ell(D)$, il résulte que AC^{j_1} et $A'X_p^{j_2}$ ne contiennent que des composantes connexes dont la longueur de cycle est au plus $\ell(D)$ et au moins $\ell(D)$ par la minimalité de D . Ceci induit que :

$$\min(\mathcal{U}(A)) \min(\mathcal{U}(X_p))^{j_1} \leq \min(\mathcal{U}(A)) \min(\mathcal{U}(C))^{j_1}$$

et

$$\min(\mathcal{U}(A')) \min(\mathcal{U}(C))^{j_2} \leq \min(\mathcal{U}(A')) \min(\mathcal{U}(X_p))^{j_2}.$$

Et par le Théorème 1.65 :

$$\min(\mathcal{U}(X_p))^{j_1} \leq \min(\mathcal{U}(C))^{j_1} \text{ et } \min(\mathcal{U}(C))^{j_2} \leq \min(\mathcal{U}(X_p))^{j_2}.$$

Et donc $\min(\mathcal{U}(X_p)) \leq \min(\mathcal{U}(C))$ et $\min(\mathcal{U}(C)) \leq \min(\mathcal{U}(X_p))$.

La condition 3 suit directement du choix de C et de d_{i+1} . En effet, nous avons $A_j(d_{i+1}C)^k = A_j Y_{i+1}^k$ pour tous entiers $j > 0$ et $k > 0$. $\square$

Lemme 5.29. Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme pseudo-injectif et soit B un FDDS. Si l'équation $P(X) = B$ admet une solution, alors $\text{EnumSolRec}(P, B, \emptyset, X_{ca}, 1)$ (Algorithme 5.22), avec X_{ca} la solution canonique relaxée triée selon $\leq_P$, énumère toutes les solutions de $P(X) = B$ une et une seule fois.

Démonstration. Assumons qu'il existe une solution à l'équation $P(X) = B$. Soit X_{ca} la

solution canonique relaxée de $P(X) = B$. Soit $Y = \sum_{i=1}^{m_Y} y_i Y_i$, où chaque Y_i est connexe et toutes les Y_i sont différentes deux à deux, une solution de $P(X) = B$, triée selon $\leq_P$. Soit $\rho = (d_1, \dots, d_{m_Y})$ le profil de fusion de Y . Par le Lemme 5.28, nous déduisons que nous pouvons regrouper les composantes connexes de X_{ca} en $\sum_{i=1}^{m_Y} (y_i \cdot q_i) X_i$ de sorte que $y_i Y_i$ soit obtenu en fusionnant y_i fois q_i copies de X_i . Remarquons que si Y_i et Y_{i+1} sont obtenus à partir de fusions de la même composante connexe de X_{ca} , alors $q_i < q_{i+1}$.

Nous montrons par récurrence sur la longueur du super-préfixe de Y qu'il existe un unique chemin dans l'arbre des appels récursifs de $\text{EnumSolRec}(P, B, \emptyset, X_{ca}, 1)$ tel que le sommet de profondeur i a $\mathcal{P}_i(Y)$ pour valeur de X_f et q_i pour valeur de d .

Prenons $i = 0$ comme cas de base. Étant donné que $\text{EnumSolRec}(P, B, \emptyset, X_{ca}, 1)$ est le seul appel qui admet $\emptyset$ pour valeur de X_f et que q_0 n'étant pas défini, il peut être fixé à 1 car $q_j \geq 1$ pour tous les $j > 0$, le cas suit directement.

Soit $0 \leq i < m_Y$ un entier. Supposons que la propriété soit vraie pour i . Montrons alors qu'elle est aussi vraie pour $i + 1$. Considérons l'appel décrit dans l'hypothèse de récurrence. Montrons qu'il génère bien un appel avec les bonnes valeurs de X_f et d . Deux cas sont possibles en fonction de la valeur de X_{i+1} . Soit $X_i \neq X_{i+1}$ et nous couvrons le cas par l'appel de la ligne 14. Soit $X_i = X_{i+1}$ et donc $q_i < q_{i+1}$. Donc le Lemme 5.28 induit qu'il y aura une itération de la boucle des lignes 6–13 où q sera égal à q_i et nb sera égal à x_i . Cette itération engendrera un appel récursif à la ligne 13 avec q_i comme valeur de N et $\mathcal{P}_i(Y) + y_{i+1} Y_{i+1} = \mathcal{P}_{i+1}(Y)$ comme valeur de X_f par hypothèse de récurrence.

Il reste à démontrer l'unicité de l'appel. Observons que les composantes connexes produites au cours d'appel récursif appartenant à la même branche augmentent strictement selon l'ordre $\leq_P$. En effet, si nous considérons D_1 et D_2 deux composantes connexes produites successivement dans une branche, alors nous considérons deux cas. Soit nous avons que D_1 et D_2 sont issues de la fusion d'une même composante connexe de X_{ca} , auquel cas la condition de la boucle externe induit que $\ell(D_1) < \ell(D_2)$. Ou bien ce n'est pas le cas, et comme nous considérons les composantes de X_{ca} en respectant l'ordre $\leq_P$, nous déduisons que $D_1 <_P D_2$.

De l'argument précédent, nous déduisons que tous les appels récursifs dont X_f admet $\mathcal{P}_j(Y)$ comme super-préfixe sont des descendants d'un appel ayant $X_f = \mathcal{P}_j(Y)$. Ainsi, seul l'appel décrit dans l'hypothèse de récurrence peut générer un appel dont la valeur de X_f est $\mathcal{P}_{i+1}(Y)$ et $N = q_{i+1}$. De plus, comme par construction chaque enfant de l'appel de l'hypothèse a un couple (nb, C) différent, l'hérédité et le lemme suivent. $\square$

À ce point, nous savons que $\text{EnumSolRec}(P, B, \emptyset, X_{ca}, 1)$ vérifie les conditions 1, 2 et 3 du Théorème 4.26.

Ainsi, pour montrer la dernière condition du Théorème 4.26 nous utiliserons le Théorème 4.29. Une fois cette preuve réalisée nous pourrons alors établir que l'appel $\text{EnumSolRec}(P, B, \emptyset, X_{ca}, 1)$ permet d'énumérer l'ensemble des solutions de $P(X) = B$ avec un délai polynomial sur la taille de B , où P un polynôme pseudo-injectif sur les FDDS et B un FDDS.

Pour ce faire, remarquons que la profondeur de l'arbre des appels récursifs de $\text{EnumSolRec}(P, B, \emptyset, X_{\text{ca}}, 1)$ est bornée par le nombre de composantes connexes de X_{ca} qui, lui-même, par définition, est borné par le nombre de composantes connexes de B puisque $P(X_{\text{ca}}) = B$. De ce fait, la condition 1 du Théorème 4.29 est vérifiée. De plus, comme l'ensemble des feuilles de l'arbre d'appel récursif est une solution, il suit que chaque appel de l'arbre conduit à une solution. Finalement, comme toutes les opérations hors appel récursif sont polynomiales sur la taille de B , il résulte que la condition 2 du Théorème 4.29 est vérifiée, et donc que la condition 4 du Théorème 4.26 est vérifiée. Nous en concluons le théorème suivant.

Théorème 5.30. *Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme pseudo-injectif et B un FDDS. Le problème d'énumérer l'ensemble des solutions de $P(X) = B$ appartient à la classe DelayP.*

5.1.3. Énumération de l'ensemble des solutions de $AX^k \leq B$

Grâce aux résultats des Sections 5.1.1 et 5.1.2, nous distinguons une procédure capable d'énumérer l'ensemble des solutions de $AX^k \leq B$ si A est connexe. L'idée serait d'énumérer l'ensemble des solutions fusion-free de $AX^k \leq B$ grâce à l'Algorithme 5.17. Puis, d'appliquer l'Algorithme 5.22 sur chacune d'elles pour obtenir l'ensemble des solutions fusionnées à partir des solutions fusion-free. Cette approche a cependant un problème : le nombre de solutions fusion-free pourrait être exponentiel. Ainsi, notre algorithme pourrait bien avoir un délai polynomial sur la taille de B tout en nécessitant un espace exponentiel sur la taille de B . Pour résoudre ce problème, nous pouvons simplement remplacer la dernière instruction (**Énumérer** X) de l'Algorithme 5.17 par un appel de $\text{EnumSolRec}(AX^j, AX^j, \emptyset, X, 1)$. La procédure que nous venons de décrire respecte les conditions 1 et 2 du Théorème 4.26 car c'est le cas des Algorithmes 5.17 et 5.22 (Lemmes 5.19 et 5.29) et que chaque solution est soit fusion-free soit issue de fusion. La preuve que cette procédure respecte la condition 3 du Théorème 4.26 n'est pas aussi évidente que pour les deux précédentes. En effet, même si chaque solution des Algorithmes 5.17 et 5.22 n'est produite qu'une et une seule fois, il serait possible que l'application de l'Algorithme 5.22 sur deux solutions distinctes de l'Algorithme 5.17 produise plusieurs fois le même résultat. En d'autres mots, il serait envisageable qu'une solution soit issue de la fusion de deux solutions fusion-free différentes. Le lemme suivant montre que cette situation est impossible.

Lemme 5.31. *Soit k un entier et soient A, X et B trois FDDS avec A connexe et tels que $AX^k \leq B$. Alors, il existe une unique solution X_{ca} fusion-free telle que $AX_{\text{ca}}^k = AX^k$.*

Démonstration. Supposons que $X = X_1 + \dots + X_{m_X}$, avec toutes les X_i connexes, est trié selon l'ordre $\leq_p$. Comme $AX^k = B$, le Lemme 3.46 induit que la longueur du cycle de X_i est de la forme $q_i \cdot \text{lcm}(p_i, \text{alcm}_A B_i)$ avec p_i la plus petite période de l'un des arbres d'unroll de X_i , $B_i = B - A(\pi_{i-1}(X))^k$ et q_i un diviseur de $\ell(B_i) / \text{lcm}(p_i, \text{alcm}_A B_i)$ étant par définition co-premier avec $\text{alcm}_A B_i$. Ainsi, par le Lemme 3.47, il suit qu'une simple récurrence sur les composantes connexes de X nous permet de prouver que le FDDS $Y = \sum_{i=1}^{m_X} q_i Y_i$ est tel que $AX^k = AY^k$ avec Y_i l'enroulement de l'arbre d'unroll minimal de X_i à période $\text{lcm}(p_i, \text{alcm}_A B_i)$. Donc Y est une solution canonique relaxée.

Montrons que $X_{ca} = Y$. Pour cela, supposons par l'absurde que ce n'est pas le cas. Considérons $X_{ca} = X'_1 + \dots + X'_{m_X}$ et $Y = Y'_1 + \dots + Y'_{m_Y}$ avec tous les X'_i et les Y'_i étant connexes et triés selon $\leq_p$. Comme $X_{ca} \neq Y$, il suit qu'il existe un entier i tel que $X'_i \neq Y'_i$. Considérons le plus petit de ces possibles i . Pour montrer que $X'_i = Y'_i$, nous commençons par prouver qu'ils ont le même arbre d'unroll minimal, puis nous en déduisons qu'ils ont la même longueur de cycle.

Considérons D_X et D_Y les plus petites composantes connexes selon $\leq_{ct}$ de respectivement $AX^k - A(\pi_{i-1}(X_{ca}))^k$ et $AY^k - A(\pi_{i-1}(Y))^k$. Notons que les Lemmes 3.44 et 3.45 induisent que D_X et D_Y sont les plus petites composantes connexes selon $\leq_{ct}$ de respectivement $A(\pi_i(X_{ca}))^{k-1} X_i$ et $A(\pi_i(Y))^{k-1} Y_i$. Remarquons que si $D_X <_{ct} D_Y$ alors la minimalité de D_Y dans $AY^k - A(\pi_{i-1}(Y))^k$ induit que :

$$AX^k - A(\pi_{i-1}(X_{ca}))^k \neq AY^k - A(\pi_{i-1}(Y))^k.$$

Et puisque $\pi_{i-1}(X_{ca}) = \pi_{i-1}(Y)$ par hypothèse, nous déduisons que $AX_{ca}^k \neq AY^k$, contradiction. Nous pouvons faire de même pour l'inégalité inverse et nous concluons alors que $D_X = D_Y$. Comme l'arbre d'unroll minimal de D_X et D_Y sont les arbres d'unroll minimaux de respectivement $A(\pi_i(X_{ca}))^{k-1} X'_i$ et $A(\pi_i(Y))^{k-1} Y'_i$, nous déduisons que X'_i et Y'_i ont le même arbre d'unroll minimal. Notons p sa période.

De là, puisque $\pi_{i-1}(X_{ca}) = \pi_{i-1}(Y)$ et comme par construction nous avons que les longueurs des cycles de X'_i et Y'_i sont respectivement $\text{lcm}(p, \text{alcm}_A(B - A(\pi_{i-1}(X_{ca}))^k))$ et $\text{lcm}(p, \text{alcm}_A(B - A(\pi_{i-1}(Y))^k))$, nous concluons que $X'_i = Y'_i$, contradiction. $\square$

De là, la condition 3 du Théorème 4.26 est vérifiée. De surcroît, le délai entre la production de deux solutions de notre algorithme est borné par le délai de l'Algorithme 5.22 plus celui de l'Algorithme 5.17, qui sont tous les deux polynomiaux sur la taille de B . De ce fait, la condition 4 du Théorème 4.26 est bien vérifiée, ce qui conclut la preuve du théorème suivant.

Théorème 5.32. *Soit k un entier positif et A, B deux FDDS avec A connexe. Le problème d'énumérer l'ensemble des solutions de $AX^k \leq B$ appartient à la classe DelayP.*

Cette procédure nous permet également d'énumérer l'ensemble des solutions maximales par inclusion sans passer par celui de l'ensemble des solutions. En effet, il est suffisant de tester si une solution fusion-free est maximale, et si c'est le cas, d'appliquer l'Algorithme 5.22. L'ajout de ce test ne retire pas le délai polynomial car il revient intuitivement à appliquer l'Algorithme 5.22 sur les feuilles de l'arbre d'appel récursif de l'Algorithme 5.17 (qui sont les solutions fusion-free maximales par inclusion). Et comme tester si une solution est maximale peut être réalisé en temps polynomial, nous continuons à respecter les conditions 1 et 2 du Théorème 4.29.

5.1.4. Énumération de l'ensemble des solutions de $P(X) \leq B$

Par le Lemme 5.7, la méthode présentée dans la Section 5.1.3 permet bien d'énumérer l'ensemble des solutions possibles de $P(X) \leq B$ pour n'importe quel polynôme

5. Sous- et sur-approximation du résultat – 5.1. Énumération des solutions de $P(X) \leq B$

P de FDDS. Cependant, le passage de $AX^k \leq B$ avec A connexe au cas général peut entraîner la perte de la polynomialité du délai. Cela est dû au fait qu'il peut exister deux solutions X_1 et X_2 de $AX^k \leq B$ avec X_2 issue de la fusion de quelques composantes connexes de X_1 telles que $P(X_1) \not\leq B$ mais $P(X_2) \leq B$ comme illustré dans l'Exemple suivant.

Exemple 5.33. *Considérons l'inéquation $C_2X \leq 2C_2 + C_6$. Remarquons que $2C_1$ et C_2 , la fusion de $2C_1$, en sont des solutions. Cependant, $(C_2 + C_3) \times 2C_1 \not\leq 2C_2 + C_6$ tandis que $(C_2 + C_3) \times C_2 \leq 2C_2 + C_6$.*

Cette observation induit que nous ne pouvons pas couper une branche de l'arbre des appels récursifs lorsque la valeur de X est telle que $P(X) \not\leq B$. Donc, nous pourrions avoir certains sous-arbres dont l'ensemble des feuilles ne serait pas des solutions. Ceci implique que le délai entre deux solutions n'est pas nécessairement polynomial.

Pour résoudre cette difficulté, nous mettons en lumière une propriété sur la structure de deux solutions. Soient X et Y deux solutions de l'inéquation $AX^k \leq B$ telles que $X \leq Y$. Constatons que si $P(X) \not\leq B$ alors $P(Y) \not\leq B$ (car $P(X) \leq P(Y)$). Donc, nous souhaitons avoir une méthode telle que si une solution Y de $AX^k \leq B$ est accessible depuis une solution X par quelques appels récursifs, alors $X \leq Y$.

Pour réaliser cela, il nous suffit simplement de changer l'ordre dans lequel nous appliquons nos opérations. C'est-à-dire qu'au lieu de construire les solutions fusion-free *intégralement*, nous en construisons *un morceau*, consistant en les copies d'une unique composante connexe, et nous appliquons notre algorithme de fusion sur ledit morceau. Un exemple illustrant la différence entre ces deux ordres est présenté dans la Figure 5.35. Plus concrètement, nous modifions la ligne 7 de l'Algorithme 5.17 par le calcul de tous les possibles FDDS X' issus de la fusion de qC respectant $A(X + X')^k = A(X + qC)^k$ et nous réalisons le même appel récursif en remplaçant qC par X' . De là, les conditions 1–3 du Théorème 4.26 sont vérifiées par les Lemmes 5.19 et 5.29. La condition 4 du Théorème 4.26 est, quant à elle, induite par le Lemme 5.16, car toutes les opérations hors appel récursif sont en temps polynomial en la taille de B , et la profondeur de l'arbre des appels récursifs est au plus le nombre de composantes connexes de B . Ceci signifie que l'Algorithme 5.34 prenant en entrée le polynôme $\sum_{i=0}^m A_i X^i$ pour valeur de P , le plus petit entier positif k tel que $A_k \neq 0$, une composante connexe A de A_k et le côté droit de l'inéquation pour valeur de B et B_0 permet d'énumérer l'ensemble des solutions de $P(X) \leq B$ avec un délai polynomial sur la taille de B , concluant la preuve du principal théorème de cette section.

Théorème 5.36. *Soient P un polynôme et B un FDDS. Nous pouvons énumérer l'ensemble des solutions de $P(X) \leq B$ avec un délai polynomial sur la taille de B . Donc le problème d'énumération associé appartient à la classe DelayP.*

Notons que toutes les solutions produites par les appels récursifs étant des feuilles de l'arbre des appels récursifs sont maximales par inclusion (car un appel récursif est une feuille si et seulement si nous ne pouvons pas ajouter quelques composantes connexes dans sa solution tout en restant une solution). Donc, comme

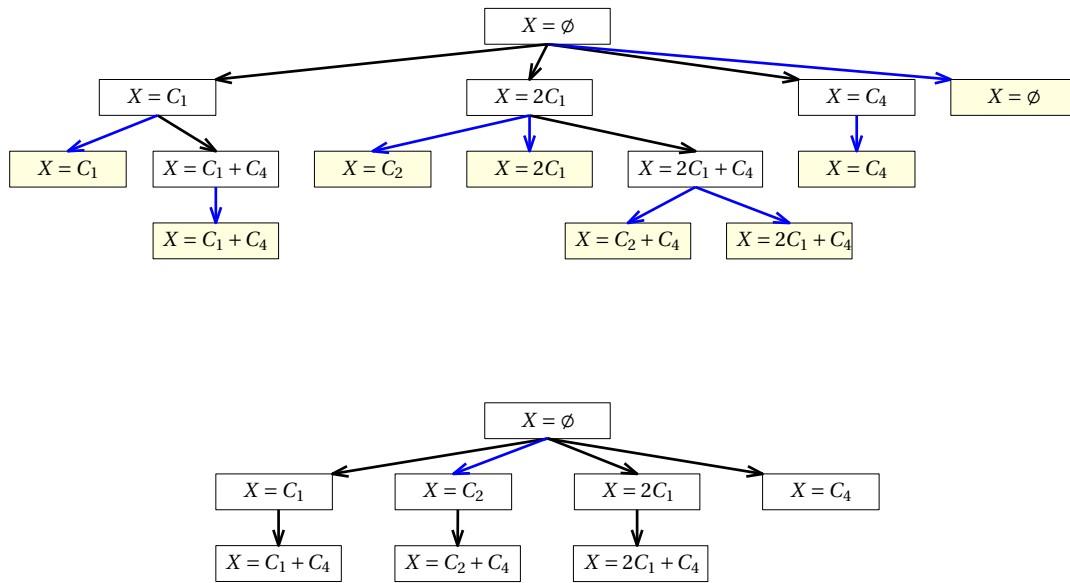


FIGURE 5.35. – Arbres des appels récursifs pour l'énumération des solutions de l'inéquation $C_2X \leq 2C_2 + 2C_4$. L'arbre du haut est le cas où nous commençons par trouver entièrement les solutions fusion-free. Celui du bas est le cas où nous construisons une partie d'une solution fusion-free puis où nous réalisons les fusions possibles sur cette partie. Les arcs bleus correspondent à l'appel de l'algorithme de fusion. Les sommets jaunes du premier arbre sont ceux où l'algorithme énumère une solution. Tous les sommets du second arbre en énumèrent une.

Algorithme 5.34 EnumInf(P, A, B, B_o, X, k)

```

1: Énumérer  $X$ ,
2: Tant que  $B \neq \emptyset$  faire
3:   soit  $B_1$  la plus petite composante connexe de  $B$  selon  $\leq_{ct}$  et  $b_1$  la multiplicité
   de  $B_1$  dans  $B$ ,
4:    $C \leftarrow \text{EnumInfCo}(k, A, X, B)$ ,
5:    $q \leftarrow 1$ ,
6:   Tant que  $A(X + qC)^k \leq B_o$  faire
7:     soit  $x_1$  le nombre de copies de  $B_1$  dans  $A(X + qC)^k - AX^k$ ,
8:     Pour chaque  $X' \in \text{EnumSolRec}(AX^k, A(X + qC)^k, X, qC, 1)$  faire
9:       Si  $P(X') \leq B$  alors
10:        soit  $B'$  le FDDS  $B_o - A(X + qC)^k$  sans composante connexe inférieure
        ou égale à  $B_1$  selon l'ordre  $\leq_{ct}$ ,
11:        EnumInf( $P, A, B', B_o, X', k$ ),
12:         $q \leftarrow q + 1$ ,
13:    $B \leftarrow B - b_1 B_1$ .

```

EnumInf($P, A, B, B_o, \emptyset, k$) vérifie les conditions 1 et 2 du Théorème 4.29, nous déduisons la proposition suivante.

Proposition 5.37. *Soient P un polynôme de FDDS et B un FDDS. Le problème d'énumérer l'ensemble des solutions maximales par inclusion de $P(X) \leq B$ est dans la classe DelayP.*

5.1.5. Généralisation au cas multivarié

Soient $n > 0$ un entier, B un FDDS et P un polynôme de FDDS de n variables. Nous étendons les résultats de la Section 5.1.4 pour montrer que nous énumérons l'ensemble des solutions possibles de l'inéquation $P(X_1, \dots, X_n) \leq B$ avec un délai polynomial sur n fois la taille de B . Cependant, nous ne ferons pas cela pour n'importe quel type de polynôme multivarié. En effet, ce type d'inéquation contient $XY \leq B$, qui admet un nombre infini de solutions (notamment $X = \emptyset$ et Y valant n'importe quelle FDDS). Ainsi, même si nous sommes en mesure d'énumérer l'ensemble des FDDS avec un délai polynomial sur leur taille (Théorème 1.38), nous ne conserverons pas ce délai dès lors que la taille des FDDS à générer sera exponentielle sur celle de B . Et de plus, notre procédure ne se terminerait jamais.

Évidemment, le cas où l'une des variables est 0 n'est pas vraiment intéressant pour le cas $XY \leq B$, et conséquemment nous pouvons juste écarter cette possibilité. Mais si nous considérons par exemple que B est connexe, alors l'ensemble des solutions non-zéro de l'inéquation $XY \leq B$ est exactement le nombre de factorisations (ou deux fois ce nombre si nous considérons différentes les solutions X_1, Y_1 et X_2, Y_2 telles que $X_1 = Y_2$ et $Y_1 = X_2$). À l'heure actuelle, nous ne savons pas décider en temps polynomial si un FDDS admet des factorisations non triviales ($XY = B$ mais

X et Y sont différents de B). Et nous n'avons pas l'ambition de répondre à cette question dans le présent manuscrit. Raison pour laquelle nous ne considérerons que des polynômes P qui admettent un monôme univarié non trivial (monôme de la forme AX^k avec A non-zéro et $k > 0$) pour chacune des n variables. En symbole, $P = \sum_{i=1}^n P_i(X_i) + P'(X_1, \dots, X_n)$ où chaque P_i est un polynôme univarié non-vide et sans terme constant et $P'(X_1, \dots, X_n)$ est un polynôme sur n variables sans condition. Nous appelons un polynôme de cette forme *polynôme pseudo-dissocié* ou simplement *polynôme dissocié* si P' est vide. Observons que nous sommes en mesure de généraliser le Lemme 5.7 pour les polynômes pseudo-dissociés comme suit.

Lemme 5.38. *Soit $n > 0$ un entier. Soit $P = \sum_{i=1}^n P_i(X_i) + P'(X_1, \dots, X_n)$ un polynôme pseudo-dissocié de n variables. Soient $X_1, \dots, X_n$ des FDDS. Si $P(X_1, \dots, X_n) \leq B$ alors $\sum_{i=1}^n P_i(X_i) \leq B$.*

Donc, pour énumérer toutes les solutions possibles de $P(X_1, \dots, X_n)$ avec P un polynôme pseudo-dissocié, nous pouvons simplement trouver toutes les solutions de $\sum_{i=1}^n P_i(X_i) \leq B$ et filtrer les solutions incorrectes. Ainsi, la difficulté restante est de démontrer comment nous réalisons ces deux opérations avec un délai polynomial sur n fois la taille de B .

La première opération, à savoir l'énumération des solutions de $\sum_{i=1}^n P_i(X_i) \leq B$, peut être réalisée en énumérant toutes les solutions de $P_1(X_1) \leq B$ (nous faisons cela avec un délai polynomial sur la taille de B par le Théorème 5.36) et en appliquant récursivement, pour chaque solution trouvée, la même opération sur $\sum_{i=2}^n P_i(X_i)$ et $B - P_1(X_1)$.

Directement, tous les vecteurs $(X_1, \dots, X_n)$ de la procédure décrite ci-dessus sont des solutions de $\sum_{i=1}^n P_i(X_i) \leq B$. Et inversement, si $(X_1, \dots, X_n)$ est une solution de $\sum_{i=1}^n P_i(X_i) \leq B$, alors $P_1(X_1) \leq B$ et $P_i(X_i) \leq B - (\sum_{j=1}^{i-1} P_j(X_j))$. Ainsi, puisque nous énumérons toutes les solutions possibles de $P_1(X) \leq B$, nous trouvons bien X_1 et, par une simple récurrence, notre algorithme trouve bien $(X_1, \dots, X_n)$. Notons que dans notre procédure, nous tenons compte de l'ordre des variables. Donc deux vecteurs de solutions sont différents même si égaux à permutation près (sauf si la permutation est l'identité). Donc le fait que notre approche ne génère pas de doublons est induit par le Théorème 5.36.

Finalement, comme l'algorithme utilisé pour énumérer toutes les solutions de $P_1(X_1) \leq B$ a un délai polynomial sur la taille de B (Théorème 5.36) et puisque le nombre d'appels récursifs est borné par n , nous concluons que la procédure énumère l'ensemble des solutions de $\sum_{i=1}^n P_i(X_i) \leq B$ avec un délai polynomial sur n fois la taille de B .

Proposition 5.39. *Soit $n > 0$ un entier. Soit $P = \sum_{i=1}^n P_i(X_i)$ un polynôme dissocié de n variables. Soit B un FDDS. Le problème d'énumérer toutes les solutions de l'inéquation $P(X_1, \dots, X_n) \leq B$ appartient à la classe DelayP.*

Maintenant, pour prouver qu'il est possible de filtrer les solutions, c'est-à-dire en prenant en compte la partie P' , tout en conservant le délai, nous introduisons le

Algorithme 5.41 EnumInfMult($P, B, B_o, \vec{X}$)

```

1: Si  $P$  ne contient pas de polynôme univarié alors
2:   Énumérer  $\vec{X}$ ,
3:   renvoyer ,
4: soit  $i$  le plus petit indice d'une variable de  $P$ ,
5: soit  $P_i$  le sous-polynôme de  $P$  contenant chaque monôme univarié sur  $X_i$ ,
6: soit  $k$  la plus petite puissance de  $P_i$ ,
7: soit  $A$  une composante connexe du coefficient associé à  $k$ ,
8: Tant que EnumInf( $P_i, A, B, B_o, \emptyset, k$ ) n'a pas terminé son exécution faire
9:    $X \leftarrow$  la solution énumérée par l'appel courant de EnumInf( $P_i, A, B, B_o, \emptyset, k$ )
10:  soit  $\vec{X}'$  le vecteur construit en remplaçant la  $i$ -ème valeur de  $\vec{X}$  par  $X$ ,
11:  Si  $P(\vec{X}') \leq B_o$  alors
12:    EnumInfMult( $P - P_i, B - P_i(X), B_o, \vec{X}'$ ),
13:  Sinon
14:    EnumInf( $P_i, A, B, B_o, \emptyset, k$ ).abort().

```

lemme suivant. Ce dernier montre que si nous considérons un vecteur $\vec{X} = (X_1, \dots, X_n)$ tel que $P(\vec{X}) \not\leq B$ avec l'un des $X_i \neq 0$, alors tout vecteur $\vec{X}'$ construit en remplaçant X_i par n'importe quelle FDDS est tel que $P(\vec{X}') \not\leq B$.

Lemme 5.40. Soit $n > 0$ un entier. Soit $P = \sum_{i=1}^n P_i(X_i) + P'(X_1, \dots, X_n)$ un polynôme pseudo-dissocié et soit $0 < m < n$ un entier. Soient $X_1, \dots, X_n$ et B des FDDS tels que $\sum_{i=1}^m P_i(X_i) \leq B$. Si $P(X_1, \dots, X_m, 0, \dots, 0) \not\leq B$ alors $P(X_1, \dots, X_m, Y_{m+1}, \dots, Y_n) \not\leq B$ pour tous les FDDS $Y_{m+1}, \dots, Y_n$.

Démonstration. Conséquence directe du fait que pour tous les FDDS $Y_{m+1}, \dots, Y_n$,

$$P(X_1, \dots, X_m, 0, \dots, 0) \leq P(X_1, \dots, X_m, Y_{m+1}, \dots, Y_n).$$

car $\leq$ est compatible avec la somme et le produit. □

Les Lemmes 5.40 et 5.38 impliquent qu'un appel de l'Algorithme 5.41 avec le polynôme comme valeur de P , les termes de droite comme valeurs de B et B_o et un vecteur de taille n rempli de 0 comme valeur de $\vec{X}$ produira l'ensemble des solutions possibles de l'inéquation $P(X_1, \dots, X_n) \leq B$.

Dans l'Algorithme 5.41, nous utilisons l'Algorithme 5.34 de telle façon à ce qu'il produise ces solutions une par une et que nous puissions utiliser la commande “.abort()” qui stoppe son appel récursif en cours. L'idée que nous utilisons est que Cette approche est inspirée de la programmation multiprocessus et l'énumération de la solution courante ainsi que la commande “.abort()” sont considérées comme ayant le même coût que le délai de l'Algorithme 5.34. Nous avons choisi de présenter cette procédure de cette manière pour rendre plus visible ces interactions avec l'Algorithme 5.34 et donner une meilleure intuition de comment nous pouvons passer du cas univarié au cas multivarié.

Théorème 5.42. *Soit $n > 0$ un entier. Soit $P = \sum_{i=1}^n P_i(X_i) + P'(X_1, \dots, X_n)$ un polynôme pseudo-dissocié de n variables. Le problème d'énumérer toutes les solutions de l'inéquation $P(X_1, \dots, X_n) \leq B$ est dans la classe DelayP.*

Démonstration. Nous montrons que $\text{EnumInfMult}(P, B, B, (0, \dots, 0))$ permet d'énumérer l'ensemble des solutions de $P(X_1, \dots, X_n) \leq B$ avec un délai polynomial sur n fois la taille de B . Par les Lemmes 5.40 et 5.38 et la Proposition 5.39, nous déduisons que nous respectons les conditions 1–3 du Théorème 4.26. Il nous faut encore prouver sa condition 4.

Tout d'abord, notons que toutes les opérations en dehors de la boucle sont réalisables en temps polynomial en la taille de B . De plus, la génération de X (instruction 9) est réalisable en temps polynomial sur la taille de B par le Théorème 5.36 car leur temps de calcul est celui du délai de l'Algorithme 5.34. Et plus généralement, toutes les opérations de la boucle 8–14, hors appel récursif, sont réalisables avec un temps polynomial sur la taille de B .

Passons maintenant au principal argument et au cœur de cette preuve : le délai entre deux $\tilde{X}'$ vérifiant le test de la ligne 11 (étant également celui décrit par le Lemme 5.40) est polynomial sur la taille de B . Relevons que le nombre de “abort()” consécutifs est borné par la profondeur de l'arbre d'appel récursif de $\text{EnumInf}(P_i, A, B, B_o, \emptyset, k)$ fois le plus grand nombre d'appels récursifs directement engendrés par un appel, c'est-à-dire le plus grand nombre d'enfants d'un sommet de l'arbre des appels récursifs de $\text{EnumInf}(P_i, A, B, B_o, \emptyset, k)$. En effet, puisque nous réalisons un parcours en profondeur de cet arbre, il suit que le pire cas correspond à avoir un unique chemin dont les solutions sont valides reliant une feuille et la racine. Pour justifier que c'est bien le pire cas, rappelons que chaque $\tilde{X}'$ vérifiant le test de la ligne 11 entraîne un appel récursif de EnumInfMult produisant une solution. Donc considérons qu'il existe un chemin dans l'arbre des appels récursifs de $\text{EnumInf}(P_i, A, B, B_o, \emptyset, k)$ dont toutes les solutions vérifient le test de la ligne 11 et qui contient au moins un sommet u ayant un autre enfant dont la solution vérifie le test. Alors, le nombre de “abort()” consécutifs (i.e., sans $\tilde{X}'$ vérifiant le test de la ligne 11) sera au plus la distance entre u et la feuille du chemin (le nombre d'appels à dépiler) fois le nombre d'enfants d'un sommet de l'arbre des appels récursifs de $\text{EnumInf}(P_i, A, B, B_o, \emptyset, k)$ (pour borner le nombre de solutions ne vérifiant pas le test) moins 1 pour l'autre enfant de u vérifiant le test.

Comme l'arbre d'appel récursif de $\text{EnumInf}(P_i, A, B, B_o, \emptyset, k)$ a une profondeur polynomiale sur la taille de B_o , et que chaque appel de cet arbre a un nombre d'enfants polynomial sur la taille de B_o , il résulte que le nombre de “abort()” consécutifs admet une borne polynomiale sur la taille de B_o .

De cette réflexion, nous déduisons que la condition 2 du Théorème 4.29 est vérifiée. Finalement, puisque n borne la profondeur de l'arbre des appels récursifs de $\text{EnumInfMult}(P, B, B, (0, \dots, 0))$, la condition 1 de ce même théorème est aussi vérifiée et donc la condition 4 du Théorème 4.26 et l'énoncé suivent. $\square$

$$P(X_1, \dots, X_n) \geq B$$

5.2. Résolution des inéquations $P(X_1, \dots, X_n) \geq B$

Soit $n > 0$ un entier. Soit P un polynôme de FDDS sur n variables. Soit B un FDDS. Dans cette section, nous analysons le problème de résoudre les équations de la forme $P(X_1, \dots, X_n) \geq B$. Remarquons que si P ne se réduit pas à un terme constant, alors l'existence d'une solution à $P(X_1, \dots, X_n) \geq B$ implique l'existence d'une infinité. En effet, comme l'ordre $\leq$ est compatible avec la somme et le produit, nous avons que $X^k \subseteq (X + Y)^k$ pour tous les FDDS X, Y non vides et tout entier $k > 0$. Il serait donc utile de pouvoir résoudre $P(X_1, \dots, X_n) \geq B$ en minimisant le nombre de composantes connexes de $P(X_1, \dots, X_n)$ qui ne sont pas dans B .

Ce problème est encore largement ouvert, et nous proposons dans cette partie une première étape vers sa résolution. Plus précisément, nous montrerons que nous pouvons trouver une solution aux équations de la forme $P(X_1, \dots, X_n) \geq B$ en temps polynomial sur la somme des tailles des coefficients de P et de celle de B . Notons bien que l'algorithme que nous introduisons fonctionnera pour n'importe quel polynôme P et sans aucune contrainte sur la solution produite. Au fil des sous-sections suivantes, nous introduirons de façon progressive un type de solution dont l'existence est assurée si l'inéquation admet une solution.

Nous commencerons par démontrer qu'il est possible de résoudre efficacement $AX \geq B$ (Section 5.2.1). Puis nous généraliserons ce résultat à n'importe quel polynôme univarié P (Section 5.2.2). Nous finirons par le cas multivarié (Section 5.2.3).

5.2.1. Le cas $AX \geq B$

Soient A et B deux FDDS. Supposons qu'il existe un FDDS X tel que $AX \geq B$. Comme dit précédemment, s'il existe une solution à l'inégalité $AX \geq B$, alors elle admet une infinité de solutions. C'est pourquoi dans cette partie et plus généralement dans toutes celles qui suivent, nous étudions seulement une famille de solutions : les *solutions couplées*.

Définition 5.43 (Solution couplée linéaire). Soient A et $B = B_1 + \dots + B_{m_B}$, avec chaque B_i connexe, deux FDDS. Nous disons que $X = X_1 + \dots + X_{m_B}$ est une solution couplée de $AX \geq B$ s'il existe une fonction $f : \{1, \dots, m_B\} \rightarrow \{1, \dots, m_B\}$ bijective telle que $AX_i \geq B_{f(i)}$ pour tout entier i entre 1 et m_B .

Exemple 5.44. Considérons l'inéquation $(C_2 + C_3)X \geq C_2 + C_6$. Alors C_2 est une solution de cette inéquation. Cependant, C_2 n'est pas une solution couplée puisque C_2 ne contient qu'une unique composante connexe tandis que $C_2 + C_6$ en contient 2. En revanche, $2C_2$ ou $C_2 + C_3$ ou encore $C_2 + C_6$ et même $C_1 + C_6$ sont des solutions couplées. En effet, nous pouvons toujours associer une copie de leur plus petit cycle avec C_2 et une copie de leur plus grand cycle avec C_6 . Notons que $3C_2$ est une solution non couplée car elle a plus de composantes connexes que $C_2 + C_6$. De même, $C_2 + C_4$ est bien une solution, mais elle est non couplée car $(C_2 + C_3)C_4$ ne produit aucune composante connexe de $C_2 + C_6$.

Nous raisonnons en deux temps. Premièrement, nous expliquons comment nous pouvons trouver une solution couplée en temps polynomial, s'il en existe une. Puis

$$P(X_1, \dots, X_n) \geq B$$

nous prouverons que s'il existe une solution, alors il existe une solution couplée. En d'autres mots, $AX \geq B$ admet une solution si et seulement si elle admet une solution couplée.

Afin de réaliser notre premier objectif, nous commençons par analyser la forme des composantes connexes d'une solution couplée. C'est le propos du lemme suivant.

Lemme 5.45. *Soient A, X et B des FDDS avec B et X connexes et soit p la plus petite période de l'arbre d'unroll minimal de X . Alors, nous avons $AX \geq B$ si et seulement si il existe C une composante connexe de $\Lambda_{\ell(B)}(A)$ et $\mathbf{a}, \mathbf{b}$ des arbres de respectivement $\mathcal{U}(C)$ et $\mathcal{U}(B)$ tels que l'arbre d'unroll minimal de X est $\mathbf{b}/\mathbf{a}$, et que $\ell(X)$ est un multiple de $\text{lcm}(p, \text{alcm}_C B)$ divisant $\ell(B)$.*

Démonstration. ($\Leftarrow$) Supposons qu'il existe C une composante connexe de $\Lambda_{\ell(B)}(A)$ et $\mathbf{a}, \mathbf{b}$ des arbres de respectivement $\mathcal{U}(C)$ et $\mathcal{U}(B)$ tels que $\min(\mathcal{U}(X)) = \mathbf{b}/\mathbf{a}$ et que $\ell(X)$ est un multiple de $\text{lcm}(p, \text{alcm}_C B) \leq \ell(B)$.

Alors CX contient une composante connexe dont l'arbre d'unroll minimal est $\mathbf{b}$. De plus, comme les longueurs des cycles de C et de X sont des diviseurs de celle du cycle de B , nous déduisons que $\text{lcm}(\ell(C), \ell(X))$ est un diviseur de $\ell(B)$. Cependant, par le choix de $\ell(X)$, nous avons que $\text{lcm}(\ell(C), \ell(X))$ est un multiple de $\text{lcm}(\ell(C), \text{alcm}_C B)$ qui, par définition, est $\ell(B)$. Ainsi, la longueur des cycles de CX est à la fois un multiple et un diviseur de $\ell(B)$, donc est égale à $\ell(B)$.

Nous en concluons que CX contient une composante connexe ayant pour arbre d'unroll minimal $\min(\mathcal{U}(B))$ et pour longueur de cycle $\ell(B)$. Ce qui induit que $CX \geq B$. De là, comme $\leq$ est compatible avec la somme, il résulte que $AX \geq CX$ et donc $AX \geq B$.

($\Rightarrow$) Supposons que $AX \geq B$. Donc, par le Lemme 3.2, il suit que $\Lambda_{\ell(B)}(A)\Lambda_{\ell(B)}(X) \geq B$. Et comme X est connexe, il résulte que $X = \Lambda_{\ell(B)}(X)$. En d'autres mots, la longueur du cycle de X est un diviseur de $\ell(B)$. De plus, nous savons qu'il existe C de $\Lambda_{\ell(B)}(A)$ tel que $CX \geq B$. La définition de alcm induit que $\ell(X)$ est un multiple de $\text{alcm}_C B$. Et $\ell(X)$ est, par définition, un multiple de p , la plus petite période de $\min(\mathcal{U}(X))$. Conséquemment, la longueur du cycle de X est un multiple de $\text{lcm}(p, \text{alcm}_C B)$ qui est aussi un diviseur de $\ell(B)$. La condition sur la longueur du cycle est vérifiée.

Pour prouver la condition sur le plus petit arbre d'unroll de X , remarquons qu'il existe des arbres $\mathbf{a}, \mathbf{x}$ et $\mathbf{b}$ de respectivement $\mathcal{U}(C)$, $\mathcal{U}(X)$ et $\mathcal{U}(B)$ tels que $\mathbf{ax} = \mathbf{b}$ puisque $CX \geq B$. De là, comme $\mathbf{x}$ a une plus petite période qui divise $\ell(B)$, il suit qu'il existe un entier $i \geq 0$ tel que $\sigma_i(\mathbf{b})$ appartient à $\mathcal{U}(B)$ et $\sigma_i(\mathbf{x}) = \min(\mathcal{U}(X))$. Ainsi, comme par définition $\sigma_i(\mathbf{b}) = \sigma_i(\mathbf{ax}) = \sigma_i(\mathbf{a})\sigma_i(\mathbf{x})$ et que $\sigma_i(\mathbf{a})$ est un arbre de $\mathcal{U}(A)$, la condition suit. $\square$

Grâce au Lemme 5.45, nous déduisons un algorithme (Algorithme 5.46) qui trouvera et énumérera l'ensemble des solutions connexes de $AX \geq B$ pour un B connexe en temps polynomial en la taille de AB . Nous montrerons aussi que l'énumération dans sa globalité sera polynomiale et non uniquement le délai entre deux solutions. Cette procédure prend en entrée un FDDS A jouant le rôle du coefficient et un FDDS connexe B qui sera le côté droit. Elle construira une liste S contenant l'ensemble des solutions que nous calculons. Elle nous servira à empêcher la production de doublons.

Algorithme 5.46 EnumCompCoSup(A, B)

```

1:  $A' \leftarrow$  le multi-ensemble des composantes connexes de  $A$  ayant une taille stricte-
   ment supérieure à  $B$ ,
2:  $A \leftarrow \Lambda_{\ell(B)}(A - A')$ ,
3:  $S \leftarrow$  une liste vide,
4: Pour chaque composante connexe  $C$  de  $A$  faire
5:    $\mathbf{a} \leftarrow \min(\mathcal{U}(C))$ ,
6:   Pour chaque  $\mathbf{b}$  de  $\mathcal{U}(B)$  faire
7:     Si  $\mathbf{x} = \mathbf{b}/\mathbf{a}$ , de période  $p$ , est défini et que  $\text{lcm}(p, \text{alcm}_C B)$  est un diviseur de
        $\ell(B)$  alors
8:       Pour chaque diviseur  $q$  de  $\text{lcm}(p, \text{alcm}_C B)$  faire
9:         Si l'enroulement  $X$  de  $\mathbf{x}$  à période  $q$  n'est pas dans  $S$  alors
10:           ajouter  $X$  à  $S$ ,
11:           Énumérer  $X$ ,
```

La correction de l'Algorithme 5.46 est une conséquence directe du Lemme 5.45. De plus, cet algorithme ne produit pas de doublons car nous n'ajoutons une composante connexe à S que si elle n'est pas déjà comprise dans S . Il nous reste encore à démontrer la polynomialité. Pour cela, relevons que l'ensemble des opérations réalisées sur les arbres d'unroll est bien polynomial via un raisonnement similaire à celui utilisé dans la preuve du Lemme 5.16. Maintenant comme le nombre d'itérations de la boucle 4–11 est égal au nombre de composantes connexes de A et le nombre d'itérations des boucles 6–11 et 8–11 sont bornés par la longueur du cycle de B , il suit que cette partie de la procédure est en temps polynomial sur la taille de AB et que le nombre d'éléments dans S est aussi polynomial sur la taille de AB . Ainsi, le temps de calcul de la dernière boucle est bien polynomial en la taille de AB .

Constatons que nous pouvons passer de l'énumération à la résolution en remplaçant l'instruction **Énumérer** X par **renvoyer** X . Ceci nous permet d'obtenir une procédure polynomiale permettant de trouver une solution couplée à l'inéquation $AX \geq B$. En effet, il suffit d'appliquer la variante de résolution de l'Algorithme 5.46 pour chaque composante connexe de B . Si à une occasion nous avons que l'Algorithme 5.46 ne renvoie pas de solution, alors cela implique, par le Lemme 5.45, qu'il existe une composante connexe B_c de B telle que $AX \not\geq B_c$ pour tous les FDDS connexes (et donc tous les FDDS) X . Donc, il n'existera pas de solution couplée. Et si, inversement, nous obtenons toujours une solution alors, en considérant $B = \sum_{i=1}^{m_B} B_i$ avec chaque B_i connexe, la somme de tous les résultats de l'Algorithme 5.46 sera $X = \sum_{i=1}^{m_B} X_i$ avec tous les X_i connexes et tels que $AX_i \geq B_i$ pour tout entier i entre 1 et m_B . Donc, $AX \geq \sum_{i=1}^{m_B} B_i = B$.

Ainsi, pour conclure que nous sommes en mesure de résoudre $AX \geq B$ en temps polynomial sur la taille de AB , il nous suffit de montrer qu'il existe une solution à l'inéquation $AX \geq B$ si et seulement si $AX \geq B$ admet une solution couplée.

Lemme 5.47. *Soient A et B deux FDDS. Alors, l'inéquation $AX \geq B$ admet une solution si et seulement si $AX \geq B$ admet une solution couplée.*

$$P(X_1, \dots, X_n) \geq B$$

Démonstration. Considérons que $B = \sum_{i=1}^{m_B} B_i$ avec chaque B_i connexe. Supposons qu'il existe une solution à l'inéquation $AX \geq B$. Nous montrons le lemme par récurrence sur la longueur du préfixe de B . Plus formellement, nous prouvons qu'il existe une solution couplée à l'inéquation $AX \geq \pi_i(B)$ et ce pour tout entier $i \geq 0$.

Si nous considérons $i = 0$ comme cas de base, alors la propriété suit directement. En effet, comme $\pi_0(B) = \emptyset$, il suit que $X = \emptyset$ est une solution et qu'elle est par définition couplée. Soit $0 \leq i < m_B$ un entier. Supposons que la propriété soit vraie pour i . Montrons alors qu'elle est vraie pour $i + 1$.

Par hypothèse, il existe une solution à l'inéquation $AX \geq B$; alors il existe une composante connexe D telle que $AD \geq B_{i+1}$. Ainsi, si nous considérons $X = X_1 + \dots + X_i$ la solution couplée de l'hypothèse de récurrence, alors $A(X + X_{i+1}) \geq \pi_i(B) + B_{i+1} = \pi_{i+1}(B)$. Il reste donc à démontrer que $X + X_{i+1}$ est bien couplée. Pour cela, supposons sans perte de généralité que $AX_j \geq B_j$ pour tout entier j entre 1 et i . Ainsi, le choix de la fonction identité $f : \{1, \dots, i+1\} \rightarrow \{1, \dots, i+1\}$ comme bijection induit que $AX_j \geq B_{f(j)}$ pour tous les j . Donc $X + X_{i+1}$ est bien une solution couplée. $\square$

Ce lemme conclut la preuve du théorème suivant.

Théorème 5.48. Soient A et B deux FDDS. La résolution de l'inéquation $AX \geq B$ est polynomiale sur la taille de AB .

Exemple 5.49. Considérons $A = C_2 + C_3$ et $B = C_2 + C_3 + C_6$. Alors la variante fonctionnelle de l'Algorithme 5.46 prenant comme entrées A et C_2 renvoie C_1 . Nous appliquons alors cette même variante à C_3 et nous obtenons de nouveau C_1 . Nous appliquons cette variante une fois de plus sur C_6 et nous obtenons C_3 . En faisant la somme des résultats, nous obtenons $2C_1 + C_3$ ce qui est bien une solution à $AX \geq B$.

5.2.2. Le cas $P(X) \geq B$

Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme de FDDS et soit B un FDDS. Pour résoudre $P(X) \geq B$, il nous faut trouver une méthode évitant que l'espace nécessaire pour le calcul de $P(X)$ soit exponentiel en fonction du degré du polynôme. En effet, puisque ce degré n'est pas borné, le simple calcul de X^k de l'inéquation $X^k \geq B$ requiert naïvement un espace exponentiel sur k . Cependant, nous pourrions n'évaluer que partiellement X^k en ne considérant plus les produits entre composantes connexes de X lorsque leur taille dépasse celle de B .

Si la méthode exposée dans le paragraphe précédent permet bien de contourner le problème de l'évaluation du polynôme, nous faisons dans cette partie un choix différent. Notre idée sera d'étendre la notion de solution couplée de la Définition 5.43 au cas des polynômes univariés, de telle sorte que $P(X) \geq B$ si et seulement si $(\sum_{i=1}^m A_i)X + A_0 \geq B$, avec X une solution couplée pour les polynômes univariés. Intuitivement, nous transformons l'inéquation $P(X) \geq B$ en mettant à 1 tous les exposants dont la valeur est supérieure ou égale à 1.

Définition 5.50 (Solution couplée univariée). Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme sans terme constant et $B = B_1 + \dots + B_{m_B}$ un FDDS avec chaque B_i connexe. Nous disons

$$P(X_1, \dots, X_n) \geq B$$

que $X = C_1 + X_1 + \dots + X_{m_B}$ est une solution couplée de $P(X) \geq B$ s'il existe une fonction $f: \{1, \dots, m_B\} \rightarrow \{1, \dots, m_B\}$ bijective telle que $P(X_i) \geq B_{f(i)}$ pour tout entier i entre 1 et m_B .

Lemme 5.51. Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme sans terme constant et $B = B_1 + \dots + B_{m_B}$ un FDDS avec chaque B_i connexe. Alors, il existe une solution de $P(X) \geq B$ si et seulement si l'équation $P(X) \geq B$ admet une solution couplée univariée si et seulement si l'équation $(\sum_{i=1}^m A_i)X \geq B$ admet une solution couplée linéaire.

Démonstration. Dans cette preuve, nous établirons que si l'équation $(\sum_{i=1}^m A_i)X \geq B$ admet une solution couplée linéaire alors l'équation $P(X) \geq B$ admet une solution couplée univariée. Puis que s'il existe une solution à $P(X) \geq B$ alors il existe une solution couplée linéaire à l'équation $(\sum_{i=1}^m A_i)X \geq B$. Ceci suffisant à prouver les équivalences par transitivité.

Supposons que l'équation $(\sum_{i=1}^m A_i)X \geq B$ admet une solution couplée linéaire, notée X . Remarquons que X est un sous-multi-ensemble de $(C_1 + X)^k$ pour tout entier $k > 0$. En effet, dans $(C_1 + X)^k$ se trouve le produit $C_1^{k-1}D = D$ et ce pour toute composante connexe D de X . Par conséquent, $(\sum_{i=1}^m A_i)X \leq P(C_1 + X)$ et donc $C_1 + X$ est bien une solution couplée de $P(X) \geq B$.

Supposons qu'il existe Y une solution de $P(X) \geq B$ ne contenant aucune composante connexe inutile, c'est-à-dire que pour toute composante connexe D de Y , nous avons $P(Y - D) \not\geq B$. Ainsi, pour chacune des B_i , il existe un coefficient A_j tel que $AY_r^j \geq B_i$ avec Y_r et A' des composantes connexes de respectivement Y et A_j . Nous en déduisons aussi qu'il existe une composante connexe X_i de Y_r^j telle que $A'X_i \geq B_i$ et que $A_j X_i \geq B_i$, sinon Y_r serait une composante inutile. De là, en construisant X la somme de toutes les X_i associées aux B_i , nous obtenons que X est une solution couplée linéaire de $(\sum_{j=1}^m A_j)X \geq B$. $\square$

Ainsi, nous déduisons du Lemme 5.51 et du Théorème 5.48 que pour démontrer le résultat suivant, il est suffisant d'expliquer comment gérer le terme constant du polynôme P .

Théorème 5.52. Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme univarié et soit B un FDDS. Alors, résoudre $P(X) \geq B$ est réalisable en temps polynomial sur $|\sum_{i=0}^m A_i||B|$.

Démonstration. Soit D le plus grand sous-multi-ensemble commun à A_0 et B . Relevons que D est constructible en temps polynomial sur $|A_0||B|$. Soit $B' = B_1 + \dots + B_{m_B}$ le sous-multi-ensemble de B tel que $B' = B - D$. Construire B' est réalisable en temps polynomial sur $|B||D|$. Par le Lemme 5.51, l'équation $P(X) - A_0 \geq B'$ admet une solution si et seulement si l'équation $(\sum_{i=1}^m A_i)X \geq B'$ admet une solution couplée linéaire. De plus, remarquons que la preuve du Lemme 5.51 explique comment nous pouvons construire en temps polynomial une solution de $P(X) - D \geq B'$ à partir d'une solution couplée linéaire à l'équation $(\sum_{i=1}^m A_i)X \geq B'$. Et comme le Théorème 5.48 montre que nous pouvons résoudre efficacement $(\sum_{i=1}^m A_i)X \geq B'$, nous concluons que pour démontrer le théorème il suffit d'établir qu'il existe une solution de $P(X) \geq B$ si et seulement s'il existe une solution de $P(X) - A_0 \geq B'$.

$$P(X_1, \dots, X_n) \geq B$$

($\Leftarrow$) Supposons que $P(X) - A_0 \geq B'$. Donc $P(X) \geq (P(X) - A_0) + D \geq B' + D = B$.

($\Rightarrow$) Supposons que $P(X) \geq B$. Donc $P(X) - D \geq B - D = B'$. Maintenant, par définition de D , toutes les composantes connexes de $A_0 - D$ ne sont pas dans B' . Donc nous avons $P(X) - A_0 \geq B'$. $\square$

5.2.3. Le cas $P(X_1, \dots, X_n) \geq B$

Soit $n > 0$ un entier, soit P un polynôme de FDDS sur n variables et soit B un FDDS. Nous montrons dans cette partie que nous sommes en mesure de résoudre $P(X_1, \dots, X_n) \geq B$ en temps polynomial. Notons que n n'a pas besoin d'être fixé pour cela. Ceci prouve que la résolution des inéquations est plus "simple" que celle des équations. En effet, pour rappel, décider si $P(X_1, \dots, X_n) = B$ possède une solution est un problème NP-complet. De ce fait, sous l'hypothèse que P soit différente de NP, il n'existe aucun algorithme polynomial pouvant résoudre $P(X_1, \dots, X_n) = B$.

Pour parvenir à notre résultat, nous allons de nouveau généraliser les solutions couplées pour que cette notion puisse couvrir le cas des équations avec un polynôme multivarié.

Définition 5.53 (Solution couplée multivariée). Soit $n > 0$ un entier. Soit P un polynôme de FDDS sur n variables et sans terme constant. Soit $\vec{X}$ un vecteur de n variables $X_1, \dots, X_n$. Nous disons que $\vec{X}$ est une solution couplée de $P(X_1, \dots, X_n) \geq B$ si chaque $X_i = C_1 + \sum_{j=1}^{m_{X_i}} X_{j,i}$, où tous les $X_{j,i}$ sont connexes, et s'il existe une fonction bijective f entre l'ensemble des couples (j, i) et $\{1, \dots, m_B\}$ telle que $P_i(X_{j,i}) \geq B_{f(j,i)}$ pour tout couple (j, i) , avec P_i le polynôme P dans lequel nous retirons syntaxiquement tous les monômes qui ne contiennent pas X_i et toutes les variables différentes de X_i . Mathématiquement $P_i(X_{j,i}) = P(\underbrace{C_1, \dots, C_1}_{i-1}, X_{j,i}, \underbrace{C_1, \dots, C_1}_{n-i}) - P(\underbrace{C_1, \dots, C_1}_{i-1}, \emptyset, \underbrace{C_1, \dots, C_1}_{n-i})$

Lemme 5.54. Soit $n > 0$ un entier. Soit P un polynôme de FDDS sur n variables et sans terme constant. Soit A la somme de tous les coefficients de P . Soit B un FDDS. Alors, il existe une solution à $P(X_1, \dots, X_n) \geq B$ si et seulement si l'équation $P(X_1, \dots, X_n) \geq B$ admet une solution couplée multivariée si et seulement si l'équation $AX \geq B$ admet une solution couplée linéaire.

Démonstration. Dans cette preuve, nous établirons que s'il existe une solution à $P(X_1, \dots, X_n) \geq B$ alors l'équation $AX \geq B$ admet une solution couplée linéaire. Puis que si l'équation $AX \geq B$ admet une solution couplée linéaire alors il existe une solution couplée multivariée à l'équation $P(X_1, \dots, X_n) \geq B$. Ceci suffisant à prouver les équivalences par transitivité.

Un argumentaire semblable à celui utilisé dans la preuve du Lemme 5.51 permet de montrer que s'il existe une solution à $P(X_1, \dots, X_n) \geq B$ alors l'équation $AX \geq B$ admet une solution couplée linéaire.

Supposons qu'il existe Y une solution couplée linéaire à $AX \geq B$. Construisons une solution couplée multivariée. Pour cela partons de $\vec{X}$ un vecteur vide de n coordonnées. Nous supposons aussi que $B = B_1 + \dots + B_{m_B}$ et $Y = Y_1 + \dots + Y_{m_B}$, avec chaque

B_i et Y_i connexes, telle que la bijection de la solution couplée est l'identité. Ainsi, nous avons que $AY_i \geq B_i$ pour tous les i . Donc, par définition de A , il existe un coefficient A_j de P tel que $A_j Y_i \geq B_i$. De là, nous pouvons ajouter Y_i à l'une des coordonnées k' de $\vec{X}$ tel que la variable $X_{k'}$ est multipliée par A_j dans P , ceci créant un couple (k, k') avec k la position de Y_i dans le FDDS de coordonnée k' de $\vec{X}$. Nous associons par la même occasion ce couple (k, k') à i . Nous pouvons effectuer cette même opération pour tous les Y_i et ainsi définir une fonction bijective f entre les couples (k, k') et $\{1, \dots, m_B\}$ qui décrit les associations entre chaque couple (k, k') et les entiers i que nous avons réalisées.

Ainsi, pour tout couple (k, k') , il existe un coefficient A_j tel que $A_j Y_{f(k, k')} \geq B_{f(k, k')}$. De là, en ajoutant C_1 à toutes les variables de $\vec{X}$, nous déduisons que $Y_{f(k, k')}$ est une composante connexe du produit des variables associé à A_j . En effet, si nous considérons des FDDS $Z_1, \dots, Z_n$ contenant chacun C_1 comme composante connexe alors tous les Z_p sont des sous-multi-ensembles de Z_p^q avec $q > 0$ un entier car issus de $Z_p \times C_1^{q-1}$. Mais tous les Z_p sont aussi des sous-multi-ensembles de $Z_1 \times \dots \times Z_n$ car ils sont multipliés par les C_1 .

À ce point, nous avons que $A_j Y_{f(k, k')} \leq P(X_1, \dots, X_n)$. Cette réflexion peut être réalisée sur toutes les $Y_{f(k, k')}$, donc par somme, il suit que $P(X_1, \dots, X_n) \geq B$. $\square$

Remarquons que l'opération d'identification (étant la bijection) que nous décrivons dans la preuve du Lemme 5.54 peut être calculée en temps polynomial sur la somme des tailles de B et de celles des coefficients de P . En effet, lors du calcul de la solution couplée linéaire, nous sommes en mesure de garder en mémoire quelle composante connexe B_i de B nous avons générée avec la composante connexe Y_i de Y . De là, pour construire la solution au cas multivarié, il nous suffit de tester les différents produits entre un coefficient A_r de P et Y_i jusqu'à ce que nous produisions un résultat qui contient B_i . Après quoi, nous pouvons ajouter Y_i à n'importe quelle variable d'un monôme non trivial et non constant de P ayant A_r comme coefficient. Ainsi, pour conclure cette partie, nous déduisons du Lemme 5.54 et du Théorème 5.48 que pour démontrer le résultat suivant, il est suffisant d'expliquer comment gérer le terme constant du polynôme P . Et pour cela, nous pouvons utiliser une approche similaire à celle utilisée dans la preuve du Théorème 5.52. Nous en concluons ainsi le théorème suivant.

Théorème 5.55. *Soit $n > 0$ un entier. Soit P un polynôme à n variables. Soit B un FDDS. Résoudre $P(X_1, \dots, X_n) \geq B$ est réalisable en temps polynomial sur la somme des tailles de B et de celles des coefficients de P .*

5.3. Conclusion et perspectives

Soit P un polynôme sur les FDDS et soit B un FDDS. Dans ce chapitre, nous avons dans un premier temps montré qu'il est possible d'énumérer l'ensemble des solutions de $P(X) \leq B$ avec un délai polynomial en la taille de B . Nous avons par la même occasion prouvé qu'il était possible d'énumérer celles qui sont maximales par inclusion.

Nous avons pu généraliser ce résultat aux inéquations où le polynôme appartient à une famille de polynômes multivariés. Dans un second temps, nous avons prouvé que nous pouvions résoudre les inéquations de la forme $P(X) \geq B$ en temps polynomial. Résultat que nous avons pu généraliser aux inéquations où le polynôme est multivarié.

Ces résultats ouvrent la voie à plusieurs pistes de recherche différentes. Tout d'abord, nous voudrions savoir si nous sommes en mesure de trouver en temps polynomial des solutions de $P(X) \leq B$ qui seraient maximum. Ceci reviendrait à montrer que nous pouvons résoudre $P(X) = B$ en temps polynomial, car s'il existe Y tel que $P(Y) = B$ alors ce Y est une solution maximum de $P(X) \leq B$. Une seconde piste serait d'étendre ce résultat d'une équation à un système d'équations. En d'autres mots, nous voudrions savoir si nous pouvons énumérer toutes les solutions du système

$$\left\{ \begin{array}{l} P_1(X) \leq B_1 \\ P_2(X) \leq B_2 \\ \vdots \\ P_n(X) \leq B_n \end{array} \right.$$

avec un délai polynomial.

Pour l'inégalité inverse, nous voudrions savoir s'il est possible d'énumérer l'ensemble des solutions couplées avec un délai polynomial. Il serait également intéressant de définir une notion de solution minimale à l'inéquation $P(X) \geq B$ et de montrer qu'il est possible de les énumérer avec délai polynomial.

Nous souhaiterions aussi étudier la complexité de la résolution des systèmes d'inéquations à une (ou un nombre fixé de) variables, mais où la partie constante peut être des deux côtés de l'inégalité, c'est-à-dire que nous pourrions avoir des inéquations de la forme $P(X) \geq B$ et $B' \geq Q(X)$. Pour ce problème, nous avons la conjecture suivante.

Conjecture 5.56. *La résolution de systèmes d'inéquations polynomiales de FDDS où la partie constante peut être des deux côtés du symbole est NP-difficile pour une variable.*

Finalement, nous désirons généraliser nos résultats à d'autres relations d'ordre sur les FDDS. Par exemple, nous pourrions définir que $A \leq B$ si et seulement si A est un sous-graphe de B . Cette relation signifiant que B contient soit des composantes connexes n'étant pas dans A , soit des états transitoires absents de A . Nous pourrions aussi la définir pour A qui serait un mineur de B . Cet ordre signifiant que nous pouvons retirer des composantes connexes de B et réaliser une séquence de transition dans B pour modéliser chaque transition de A . Ou le définir pour B qui est projetable dans A (ou que A est une projection de B), c'est-à-dire qu'il existe un homomorphisme surjectif de B vers A . Intuitivement, cet ordre permet de regrouper plusieurs sommets de B en un seul sommet de A tout en respectant la structure de B . La Figure 5.57 donne une intuition de ces trois relations.

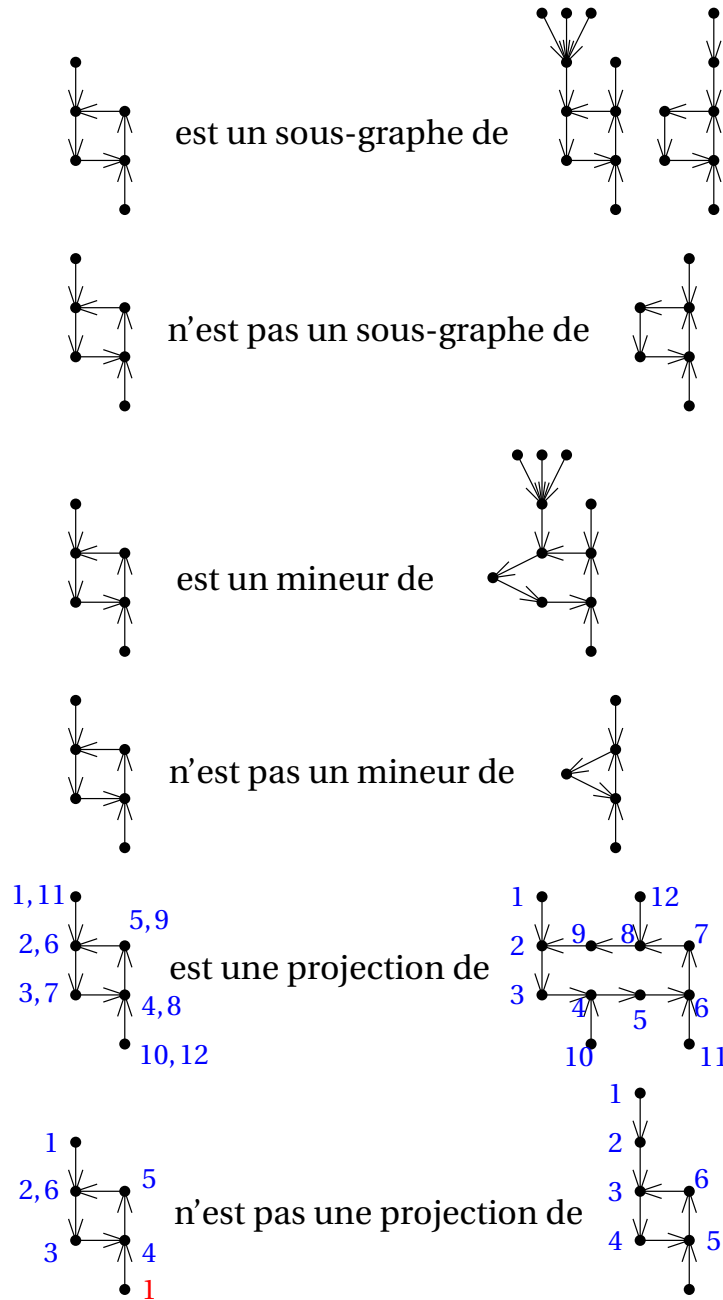


FIGURE 5.57. – Exemple et contre-exemple des relations sous-graphe, mineur et projection. Dans le 5-ème cas, les nombres en bleu décrivent l'homomorphisme surjectif. Pour justifier le 6-ème cas, remarquons que nous sommes obligés d'envoyer le sommet 1 bleu du graphe de droite soit sur le sommet 1 bleu du graphe de gauche soit sur le sommet 1 rouge. Si nous l'envoyons sur le sommet 1 bleu alors le sommet 1 rouge n'admet pas d'antécédent. Nous pouvons appliquer le même raisonnement en envoyant le sommet 1 bleu du graphe de droite sur le sommet 1 rouge du graphe de gauche et nous obtenons que le sommet 1 bleu du graphe de gauche n'a pas d'antécédent.

6. Les systèmes partiels

Sommaire

6.1. Propriétés algébriques des systèmes partiels	199
6.1.1. Premier pour les systèmes partiels	200
6.1.2. Injectivité des polynômes sur les systèmes partiels	201
6.2. Résolution de $P(X) = B$ pour les systèmes partiels	210
6.2.1. Résoudre $P(X) = B$ lorsqu'un coefficient non constant A est tel que $\mathcal{T}(A) \neq \emptyset$	211
6.2.2. Résoudre $P(X) = B$ si tous les coefficients sont des forêts finies .	213
6.3. Conclusion et perspectives	227

Si dans le chapitre précédent nous nous sommes placés dans la situation où les FDDS que nous obtenons étaient issus d'une erreur d'analyse, d'observation ou d'exécution de phénomène naturel, il serait intéressant d'étudier le cas où les erreurs sont directement induites par les observations elles-mêmes. Par exemple, nous pourrions considérer que certains états du système n'ont jamais été relevés. Ou encore que les relevés n'ont pas été suffisamment précis ou en nombre insuffisant et que certaines transitions sont manquantes. Par exemple, si nous étudions un groupe de personnes atteintes d'une maladie qui, une fois infectées, mettraient 34 étapes de temps à guérir mais que nous réalisons uniquement des relevés pour 30 étapes de temps, alors nous n'observerions jamais que les malades reviennent dans un état sain.

De ce descriptif, nous voyons que les graphes que nous considérons n'ont pas tous un degré sortant exactement égal à 1. Donc nous ne pouvons pas les modéliser dans le semi-anneau des FDDS. C'est pourquoi nous considérons un sur-semi-anneau de ce dernier, à savoir celui des *transformations partielles* ou des *systèmes partiels*.

Pour définir formellement ce qu'est un système partiel, rappelons qu'un FDDS est un couple (A, f_A) , pris à isomorphisme près, avec A un ensemble d'états et f_A une fonction totale de A dans A définissant les transitions. Un système partiel est lui aussi un couple (A, f_A) , pris à isomorphisme près, où A et f_A jouent le même rôle que dans les FDDS mais où f_A est une fonction partielle, c'est-à-dire que seul un sous-ensemble de A (pouvant être A lui-même) admet une image par f_A . Nous notons $\mathbb{P}$ l'ensemble des systèmes partiels. Cet ensemble muni de l'exécution alternative comme somme et de l'exécution synchrone pour produit est bien un semi-anneau et ce dernier admet $(\mathbb{D}, +, \times)$ comme sous-semi-anneau.

En appliquant le même raisonnement que dans la Section 1.1, nous déduisons que le semi-anneau des systèmes partiels définis ci-dessus est égal à celui des graphes

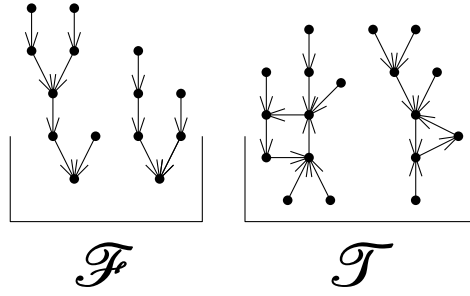


FIGURE 6.1. – Exemple d'un système partiel A avec mise en lumière de $\mathcal{F}(A)$ et $\mathcal{T}(A)$.

orientés avec degré sortant au plus 1, avec l'union disjointe comme somme et le produit direct comme multiplication.

Dans ce chapitre, nous étudierons quelques propriétés algébriques de ce nouveau semi-anneau (Section 6.1). Nous prouverons que ce semi-anneau admet exactement un élément premier (Théorème 6.6). Nous montrerons également que la caractérisation des polynômes injectifs de FDDS caractérise aussi les polynômes injectifs de système partiel (Théorème 6.25). Puis nous établirons que résoudre $P(X) = B$ avec P un polynôme de système partiel et B un système partiel est réalisable en temps polynomial si nous sommes en mesure de résoudre $P'(X) = B'$ avec P' un polynôme de FDDS et B' un FDDS (Section 6.2).

6.1. Propriétés algébriques des systèmes partiels

Soit A un système partiel. Notons que nous sommes en mesure de séparer les composantes connexes de A en deux catégories (ou sous-multi-ensembles) : celle des composantes ayant un cycle, notée $\mathcal{T}(A)$, et celle des composantes qui n'en ont pas, notée $\mathcal{F}(A)$. Cette dernière partie est une simple forêt finie. De là, nous distinguons deux sous-ensembles remarquables des systèmes partiels : ceux où $\mathcal{F}(X)$ est vide (les FDDS) et ceux où $\mathcal{T}(X)$ est vide (les forêts finies) (voir Figure 6.1 pour un exemple).

Notons que l'ensemble des forêts finies est clos par la somme et également par le produit. En effet, le produit direct de deux forêts est une forêt car chaque sommet est dérivé soit du produit de deux sommets ayant degré sortant 1 et donc à degré sortant 1, soit du produit d'un sommet avec degré sortant au plus 1 et d'un sommet avec degré sortant 0 et donc à degré sortant 0. Ainsi, l'ensemble des forêts finies, qui, pour rappel, est noté $\mathbb{F}_f$, muni de l'union disjointe comme somme et du produit direct comme multiplication, forme un *pseudo-semi-anneau*, c'est-à-dire un semi-anneau sans neutre multiplicatif. En effet, le neutre multiplicatif du produit direct demeure C_1 qui n'est pas inclus dans $\mathbb{F}_f$. Le raisonnement sur les degrés exposé ci-dessus nous permet également de déduire la propriété suivante.

Lemme 6.2. *L'ensemble $\mathbb{F}_f$ est un idéal de $(\mathbb{P}, +, \times)$, c'est-à-dire que $(\mathbb{F}_f, +)$ est un sous-monoïde de $(\mathbb{P}, +)$ et pour tout élément A de $\mathbb{P}$ et tout élément B de $\mathbb{F}_f$, nous avons $A \times B$ et $B \times A$ qui sont des éléments de $\mathbb{F}_f$.*

6. Les systèmes partiels – 6.1. Propriétés algébriques des systèmes partiels

Dans cette section, nous cherchons à caractériser les éléments premiers de $(\mathbb{P}, +, \times)$, s'ils existent, et quels sont les polynômes injectifs induits par $(\mathbb{P}, +, \times)$. Pour ce second point, nous étendrons la définition d'unroll au cas des systèmes partiels. En effet, relevons que la définition de l'unroll fournie pour les FDDS ne s'applique pas directement aux systèmes partiels puisque certaines composantes connexes n'ont pas de cycle, induisant que la fonction $\mathcal{U}(X)$ n'est pas totale, ce qui contrevient à la définition d'un morphisme.

6.1.1. Premier pour les systèmes partiels

Dans cette partie, nous montrons que le point isolé et sans boucle, plus simplement le point noté ρ , est l'unique premier de $(\mathbb{P}, +, \times)$. Pour rappel, un élément p est premier dans un semi-anneau $(M, \diamond, \star)$ s'il est non nul, non inversible et si pour tous a, b de M , s'il existe k dans M tel que $p \star k = a \star b$, alors il existe k' tel que $p \star k' = a$ ou $p \star k' = b$. Pour montrer que ρ vérifie cette propriété, nous caractériserons la forme des éléments que divise ρ , puis nous conclurons.

Lemme 6.3. *Soient A et K des systèmes partiels. Alors, $\rho \times K = A$ si et seulement si A est égal à $|K|\rho$.*

Démonstration. ($\Leftarrow$) Supposons que A soit égal à $|K|\rho$. Alors, directement ρ divise A . En effet, si nous considérons K comme $|K|$ points fixes, alors $K \times \rho = |K|\rho$ par définition.

($\Rightarrow$) Supposons que $\rho \times K = A$. Alors, nous avons que A est constitué de $|K|$ sommets. De plus, comme le degré sortant d'un sommet (u, v) d'un produit est égal au degré sortant de u fois celui de v , il résulte que tous les sommets de $\rho \times K$ ont degré sortant 0. Ceci induit que A est composé de $|K|$ copies de ρ . $\square$

Proposition 6.4. *Soient A et B des systèmes partiels. Alors, il existe un système partiel K tel que $\rho \times K = A \times B$ si et seulement si A ou B ne sont que des copies de ρ .*

Démonstration. ($\Leftarrow$) Supposons que A ou B ne soient constitués que de copies de ρ . Sans perte de généralité, supposons que $A = |A|\rho$. Alors, nous déduisons du Lemme 6.3 que $A \times B$ est constitué de $|A| \times |B|$ copies de ρ . Et du même lemme, il suit que ρ divise $A \times B$ et A .

($\Rightarrow$) Supposons qu'il existe un système partiel K tel que $\rho \times K = A \times B$. Donc, par le Lemme 6.3, il suit que $A \times B$ n'est constitué que de copies de ρ . Maintenant, comme le degré sortant d'un sommet (u, v) d'un produit est égal au degré sortant de u fois celui de v , il résulte que si A et B contiennent chacun un sommet de degré sortant 1, alors c'est aussi le cas de $A \times B$. Nous en déduisons donc que A ou B ne contient que des copies de ρ . $\square$

Corollaire 6.5. *ρ est premier dans $(\mathbb{P}, +, \times)$.*

Démonstration. Par définition, aucun élément de $(\mathbb{P}, +, \times)$ autre que le point fixe n'est inversible. En effet, si nous considérons deux systèmes partiels A et B , alors le produit

6. Les systèmes partiels – 6.1. Propriétés algébriques des systèmes partiels

$A \times B$ a $|A| \times |B|$ sommets. Donc, pour que $A \times B$ soit le point fixe, il faut que A et B ne soient constitués que d'un état. Et de plus, cet état doit avoir un arc sortant. Donc A et B sont le point fixe.

Ainsi, ρ est non nul, non inversible et, par la Proposition 6.4, il respecte la dernière condition pour être premier. $\square$

Nous voulons maintenant prouver que ρ est l'unique premier de $(\mathbb{P}, +, \times)$. Pour cela, il est suffisant de prouver que tous les systèmes partiels ayant au moins deux sommets ne sont pas premiers car le neutre multiplicatif n'est par définition pas premier. Ceci est le propos du théorème suivant.

Théorème 6.6. ρ est l'unique premier de $(\mathbb{P}, +, \times)$.

Démonstration. Soit A un système partiel avec $n \geq 2$ sommets. Nous allons montrer que A n'est pas premier. Pour cela, il nous suffit de construire trois systèmes partiels X, B, D tels que $AX = BD$ mais A ne divise ni B ni D .

Soient n_1 et n_2 deux diviseurs de n tels que $n = n_1 \cdot n_2$ et $n_1 < n_2$. Soit D le système partiel constitué de n_2 points fixes. Nous distinguons deux cas en fonction de la valeur de A .

Le premier cas est que $A \neq D$. Dans ce cas, considérons $B = n_1 \rho$. Observons que $A \times \rho = B \times D$. En outre, A ne divise ni B ni D . En effet, par construction $|A| > |B| > 0$ et comme la taille de AX est soit 0 (si $X = \emptyset$) soit supérieure ou égale à la taille de A (si $X \neq \emptyset$), il suit que A ne divise pas B . De plus, remarquons que $|A| \geq |D|$. Donc, soit $|A| > |D|$ et il suit que A ne divise pas D . Soit $|A| = |D|$. Mais dans ce cas, pour que l'équation $A \times X = D$ ait une solution, il faut que $|X| = 1$. Or, X ne peut être le point fixe, car $A \neq D$. Donc, la seule valeur possible pour X est ρ . Mais dans ce cas, $A \times X$ ne contient aucun arc et est donc différent de D . Il en résulte que A n'est pas premier.

Le second cas possible est donc que $A = D$. Alors, nous avons que $A \times \rho = \rho \times C_n$. Mais A ne divise ni B ni C_n car un produit avec un FDDS non vide ne réduit pas le nombre de composantes connexes. Ce qui signifie que A n'est pas premier. $\square$

Remarquons toutefois que ρ n'est pas irréductible. En effet, pour rappel, un élément est irréductible s'il n'est ni nul, ni inversible, ni issu du produit de deux éléments non inversibles. Or, ρ est non inversible mais est le produit de deux éléments non inversibles, en effet $\rho \times \rho = \rho$.

6.1.2. Injectivité des polynômes sur les systèmes partiels

Soit A un système partiel. Dans cette partie, nous montrerons que A est annulable dans $(\mathbb{P}, +, \times)$ si et seulement si A contient un dendron. Pour cela, rappelons que A , comme tout système partiel, peut être vu comme la somme de $\mathcal{F}(A)$ et de $\mathcal{T}(A)$. Ainsi, le produit de A avec un système partiel B est :

$$A \times B = (\mathcal{F}(A) \times \mathcal{F}(B)) + (\mathcal{F}(A) \times \mathcal{T}(B)) + (\mathcal{T}(A) \times \mathcal{F}(B)) + (\mathcal{T}(A) \times \mathcal{T}(B)).$$

Pour montrer notre affirmation, nous voudrions pouvoir décrire exactement le produit entre deux éléments de $\mathbb{F}_f$ ainsi qu'entre un élément de $\mathbb{D}$ et un élément de $\mathbb{F}_f$. De plus, comme les éléments de $\mathbb{F}_f$ sont des forêts, nous souhaiterions que cette caractérisation se fasse avec $\otimes$ au lieu de $\times$, c'est-à-dire en exprimant $\times$ en fonction de $\otimes$.

À cette fin, nous utiliserons la fonction $\mathcal{D}(\mathbf{X})$ introduite dans la Section 1.3. Pour rappel, si nous considérons une forêt $\mathbf{F}$, alors $\mathcal{D}(\mathbf{F})$ est le multi-ensemble des arbres enracinés dans les racines de $\mathbf{F}$. Nous pouvons généraliser cela en définissant que $\mathcal{D}^i(\mathbf{F})$ est le multi-ensemble des arbres de $\mathbf{F}$ enracinés dans un sommet de profondeur i . De façon équivalente, le multi-ensemble $\mathcal{D}^i(\mathbf{F})$ est celui obtenu en appliquant i fois la fonction $\mathcal{D}(\mathbf{X})$ sur $\mathbf{F}$. Nous avons alors que $\mathcal{D}^i(\mathcal{D}^j(\mathbf{F})) = \mathcal{D}^{i+j}(\mathbf{F}) = \mathcal{D}^j(\mathcal{D}^i(\mathbf{F}))$ pour tous entiers $i, j \geq 0$ et toutes les forêts $\mathbf{F}$. En outre, comme le montre le lemme suivant, la fonction $\mathcal{D}^i(\mathbf{F})$ est un morphisme de $(\mathbb{F}_f, +, \otimes)$ dans lui-même et ce pour tout entier $i \geq 0$.

Lemme 6.7. Soient $\mathbf{A}, \mathbf{B}$ deux forêts et $i \geq 0$ un entier. Alors, $\mathcal{D}^i(\mathbf{A} \otimes \mathbf{B}) = \mathcal{D}^i(\mathbf{A}) \otimes \mathcal{D}^i(\mathbf{B})$.

Démonstration. Soit $\mathbf{t}$ un arbre de $\mathbf{A} \otimes \mathbf{B}$. Alors, il existe $\mathbf{t}_\mathbf{A}$ et $\mathbf{t}_\mathbf{B}$ de respectivement $\mathbf{A}$ et $\mathbf{B}$ tels que $\mathbf{t}_\mathbf{A} \otimes \mathbf{t}_\mathbf{B} = \mathbf{t}$. Or, puisque le produit d'arbres est niveau par niveau, chaque arbre enraciné dans un sommet de profondeur i dans $\mathbf{t}$ est le produit de deux sous-arbres de respectivement $\mathbf{t}_\mathbf{A}$ et $\mathbf{t}_\mathbf{B}$ ayant pour racine un sommet de profondeur i . Il suit que $\mathcal{D}^i(\mathbf{t}_\mathbf{A}) \otimes \mathcal{D}^i(\mathbf{t}_\mathbf{B}) = \mathcal{D}^i(\mathbf{t})$. En sommant tous les arbres, le lemme suit. $\square$

Lemme 6.8. Soit $d \geq 0$ un entier et soient $\mathbf{A}, \mathbf{B}$ deux éléments de forêts finies de profondeur au plus d . Alors, $\mathbf{A} \times \mathbf{B} = \mathbf{A} \otimes \mathbf{B} + \sum_{i=1}^d (\mathcal{D}^i(\mathbf{A}) \otimes \mathbf{B} + \mathcal{D}^i(\mathbf{B}) \otimes \mathbf{A})$.

Un exemple de produit direct de forêts finies est donné dans la Figure 6.9.

Démonstration. Par la définition du produit de FDDS, nous avons que $\mathbf{A} \times \mathbf{B}$ est une forêt ayant pour ensemble de sommets $V = \{(a, b) \mid a \in V(\mathbf{A}), b \in V(\mathbf{B})\}$ et pour ensemble d'arcs $E = \{((a, b), (a', b')) \mid (a, a') \in E(\mathbf{A}), (b, b') \in E(\mathbf{B})\}$. Considérons la forêt $\mathbf{F} = \mathbf{A} \otimes \mathbf{B} + \sum_{i=1}^d \mathcal{D}^i(\mathbf{A}) \otimes \mathbf{B} + \mathcal{D}^i(\mathbf{B}) \otimes \mathbf{A}$ ayant pour ensemble de sommets V' et pour ensemble d'arcs E' . Nous montrons que $\mathbf{A} \times \mathbf{B}$ et $\mathbf{F}$ sont le même graphe (exactement le même et non simplement isomorphes).

Montrons dans un premier temps que $V = V'$. Pour cela, nous démontrons la double inclusion ($V \subseteq V'$ et $V' \subseteq V$). Observons que tous les sommets de V' sont des paires (a, b) avec a et b des sommets respectivement de $V(\mathbf{A})$ et $V(\mathbf{B})$. Donc, comme V contient l'ensemble de ces paires, il suit que $V' \subseteq V$.

Considérons $\mathbf{t}_\mathbf{A}$ et $\mathbf{t}_\mathbf{B}$ deux arbres de respectivement $\mathbf{A}$ et $\mathbf{B}$ ainsi que a et b des sommets de ces arbres de profondeur respectivement $d_\mathbf{a}$ et $d_\mathbf{b}$. Trois cas sont possibles. Soit $d_\mathbf{a} = d_\mathbf{b}$. Alors, comme $\mathbf{F}$ contient $\mathbf{A} \otimes \mathbf{B}$, il suit que V' contient le couple (a, b) . Soit enfin $d_\mathbf{a} < d_\mathbf{b}$. Alors, comme $\mathbf{F}$ contient $\mathbf{t}_\mathbf{A} \otimes \mathcal{D}^{d_\mathbf{b}-d_\mathbf{a}}(\mathbf{t}_\mathbf{B})$, il résulte que V' contient le couple (a, b) . Symétriquement, si $d_\mathbf{a} > d_\mathbf{b}$, alors V' contient le couple (a, b) . De cette réflexion, nous inférons que chaque couple possible (a, b) avec a et b des sommets de respectivement $\mathbf{A}$ et $\mathbf{B}$, est un élément de V' , induisant que $V \subseteq V'$.

Il nous reste à démontrer que $E = E'$. Pour cela, nous commençons par montrer que $\mathbf{A} \times \mathbf{B}$ et $\mathbf{F}$ ont les mêmes racines. Directement, un sommet (a, b) de $\mathbf{F}$ est une racine

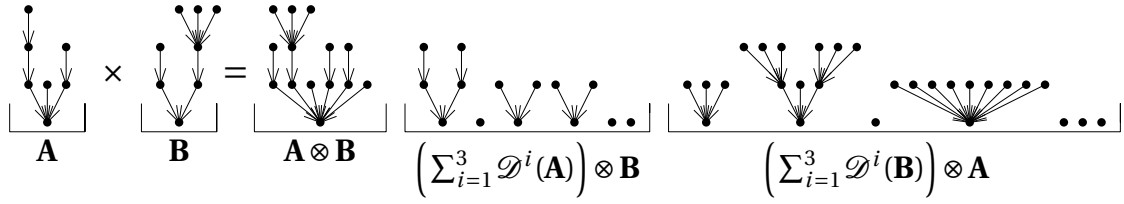


FIGURE 6.9. – Exemple de produit direct de forêt.

si et seulement si a ou b est une racine. Prouvons que c'est aussi le cas dans $A \times B$. Considérons que a ou b est une racine de $\mathbf{A}$ ou $\mathbf{B}$, alors (a, b) est une racine de $A \times B$ car a ou b n'a pas de successeur. Et inversement, si a et b ne sont pas des racines de $\mathbf{A}$ et $\mathbf{B}$, alors a et b ont un successeur dans respectivement $\mathbf{A}$ et $\mathbf{B}$, notons-les a' et b' . Donc dans $A \times B$, le sommet (a, b) a (a', b') comme successeur. Ainsi, un sommet (a, b) est une racine de $A \times B$ si et seulement si a ou b est une racine de respectivement $\mathbf{A}$ ou $\mathbf{B}$. Nous en déduisons que $\mathbf{F}$ et $A \times B$ ont les mêmes racines.

De là, comme $V = V'$, que $\mathbf{F}$ et $A \times B$ ont les mêmes racines et que tous les autres sommets de $\mathbf{F}$ et $A \times B$ ont degré sortant exactement 1, il suit que E et E' ont la même taille. De ce fait, pour conclure que $E = E'$, il suffit de prouver que $E' \subseteq E$. Soit $((a, b), (a', b'))$ un arc de E' . Trois cas sont possibles :

1. L'arc $((a, b), (a', b'))$ est un arc de $A \otimes B$, donc (a, a') est un arc de $\mathbf{A}$ et (b, b') est un arc de $\mathbf{B}$. Donc $((a, b), (a', b'))$ est un élément de E .
2. Il existe un $i > 0$ tel que $((a, b), (a', b'))$ est un arc de $A \otimes \mathcal{D}^i(\mathbf{B})$. Cela induit qu'il existe un arc (a, a') dans $\mathbf{A}$ et un arc (b, b') de $\mathcal{D}^i(\mathbf{B})$ et donc de $\mathbf{B}$. Et par conséquent, l'arc $((a, b), (a', b'))$ est un élément de E .
3. Il existe un entier $j > 0$ tel que $((a, b), (a', b'))$ est un arc de $\mathcal{D}^j(\mathbf{A}) \otimes \mathbf{B}$. Ce cas est le symétrique du précédent et donc l'arc $((a, b), (a', b'))$ est un élément de E .

□

Il paraît difficile de déduire des propriétés sur les forêts finies annulables depuis l'égalité donnée dans l'énoncé du Lemme 6.8. En effet, nous ne pouvons pas mieux factoriser cette égalité et elle ne permet pas d'établir un lien clair entre le produit par $\times$ et celui par $\otimes$. Pour pallier cela, nous introduisons le résultat suivant.

Lemme 6.10. *Soit $d \geq 0$ un entier et soient $\mathbf{A}$ et $\mathbf{B}$ deux forêts finies de profondeur au plus d . Alors, $\sum_{i=0}^d \mathcal{D}^i(A \times B) = (\sum_{i=0}^d \mathcal{D}^i(\mathbf{A})) \otimes (\sum_{j=0}^d \mathcal{D}^j(\mathbf{B}))$.*

Démonstration. Par le Lemme 6.8 et en appliquant la fonction $\mathcal{D}^i(\mathbf{X})$, nous avons que

$$\sum_{i=0}^d \mathcal{D}^i(A \times B) = \sum_{i=0}^d (\mathcal{D}^i(A \otimes B)) + \sum_{i=0}^d \sum_{j=1}^d (\mathcal{D}^i(\mathcal{D}^j(\mathbf{A}) \otimes \mathbf{B})) + \sum_{i=0}^d \sum_{j=1}^d (\mathcal{D}^i(\mathcal{D}^j(\mathbf{B}) \otimes \mathbf{A})).$$

Ainsi, par le Lemme 6.7 et en appliquant l'identité $\mathcal{D}^i(\mathcal{D}^j(\mathbf{X})) = \mathcal{D}^{i+j}(\mathbf{X})$, nous dédui-

6. Les systèmes partiels – 6.1. Propriétés algébriques des systèmes partiels

sons que

$$\sum_{i=0}^d \mathcal{D}^i(\mathbf{A} \times \mathbf{B}) = \sum_{i=0}^d (\mathcal{D}^i(\mathbf{A}) \otimes \mathcal{D}^i(\mathbf{B})) + \sum_{i=0}^d \sum_{j=1}^d (\mathcal{D}^{i+j}(\mathbf{A}) \otimes \mathcal{D}^i(\mathbf{B})) + \sum_{i=0}^d \sum_{j=1}^d (\mathcal{D}^{i+j}(\mathbf{B}) \otimes \mathcal{D}^i(\mathbf{A})).$$

Remarquons que $\sum_{j=1}^d \mathcal{D}^{i+j}(\mathbf{X}) = \sum_{j=1+i}^d \mathcal{D}^j(\mathbf{X})$. Notons de plus que si nous ajoutons et retirons $\sum_{i=0}^d \mathcal{D}^i(\mathbf{A} \otimes \mathbf{B})$, nous pouvons factoriser par $\sum_{i=0}^d \mathcal{D}^i(\mathbf{A})$ et par $\sum_{i=0}^d \mathcal{D}^i(\mathbf{B})$ puis contracter les termes et ainsi obtenir

$$\sum_{i=0}^d \mathcal{D}^i(\mathbf{A} \times \mathbf{B}) = \left(\sum_{i=0}^d \mathcal{D}^i(\mathbf{B}) \right) \otimes \left(\sum_{j=i}^d \mathcal{D}^j(\mathbf{A}) \right) + \left(\sum_{i=0}^d \mathcal{D}^i(\mathbf{A}) \right) \otimes \left(\sum_{j=i}^d \mathcal{D}^j(\mathbf{B}) \right) - \sum_{i=0}^d \mathcal{D}^i(\mathbf{A} \otimes \mathbf{B}).$$

Relevons que

$$\sum_{i=0}^d \sum_{j=0}^d (\mathcal{D}^i(\mathbf{B}) \otimes \mathcal{D}^j(\mathbf{A})) = \sum_{i=0}^d \sum_{j=i}^d (\mathcal{D}^i(\mathbf{B}) \otimes \mathcal{D}^j(\mathbf{A}) + \mathcal{D}^i(\mathbf{A}) \otimes \mathcal{D}^j(\mathbf{B}) - \mathcal{D}^i(\mathbf{B}) \otimes \mathcal{D}^i(\mathbf{A}))$$

De là, en contractant de nouveau les termes, nous concluons que

$$\sum_{i=0}^d \mathcal{D}^i(\mathbf{A} \times \mathbf{B}) = \sum_{i=0}^d \sum_{j=0}^d (\mathcal{D}^i(\mathbf{B}) \otimes \mathcal{D}^j(\mathbf{A})) = \left(\sum_{i=0}^d \mathcal{D}^i(\mathbf{B}) \right) \otimes \left(\sum_{j=0}^d \mathcal{D}^j(\mathbf{A}) \right). \quad \square$$

Remarquons que $\sum_{i=0}^d \mathcal{D}^i(\mathbf{t})$ revient à appliquer la fonction unroll définie dans la Section 1.3 sur tous les sommets de $\mathbf{t}$ si d est suffisamment grand. En nous basant sur cette observation, nous redéfinissons l'unroll d'un système pour qu'il puisse aussi couvrir le cas partiel. La Figure 6.13 donne deux exemples de ce qu'est l'unroll d'un système partiel.

Définition 6.11 (Unroll de système partiel). Soit $A = (A, f_A)$ un système partiel, alors l'unroll de A , noté $\mathcal{U}(A)$, est le graphe ayant pour sommets l'ensemble des couples (a, k) avec a un sommet de A et $k \geq 0$ un entier tel que $f_A^k(a)$ est bien défini, et ayant la totalité des arcs $((a, k), (f_A(a), k-1))$.

De façon intuitive, $\mathcal{U}(A)$ est la forêt issue de la somme de $\mathcal{U}(\mathcal{T}(A))$ plus $\mathcal{F}(A)$ plus l'ensemble des arbres de A enracinés dans les sommets de profondeur au moins 1. Pris à isomorphisme près, les unrolls de système partiel peuvent également se définir en utilisant la fonction de déraccinement.

Définition 6.12. Soit A un FDDS. Nous définissons que $\mathcal{D}^0(A) = \mathcal{U}(A)$. De plus, pour tout $i > 0$ nous définissons que $\mathcal{D}^i(A)$ est la forêt constituée des arbres ayant une racine de profondeur i .

Soit X un système partiel. Nous définissons que la profondeur de X , notée $d(X)$, est le maximum entre $d(\mathcal{F}(X))$ et $d(\mathcal{T}(X))$. Alors $\mathcal{U}(X) = \sum_{i=0}^d \mathcal{D}^i(X)$ avec d un entier supérieur ou égal à la profondeur de X .

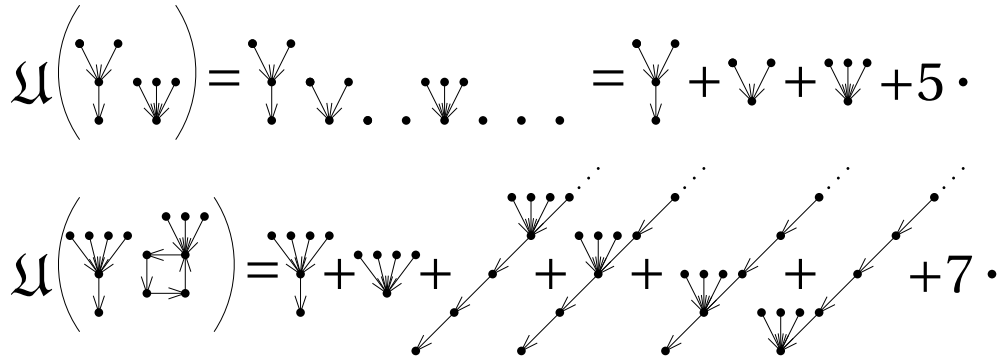


FIGURE 6.13. – Exemple d'unroll d'une forêt et d'un système partiel avec la nouvelle définition d'unroll.

Lemme 6.14. Soient A, B des forêts finies. Alors, $\mathfrak{U}(A \times B) = \mathfrak{U}(A) \otimes \mathfrak{U}(B)$

Dans la suite, nous prouverons que $\mathfrak{U}(X)$ est un morphisme entre le semi-anneau $(\mathbb{P}, +, \times)$ et $(\mathfrak{U}(\mathbb{P}), +, \otimes)$ avec $\mathfrak{U}(\mathbb{P})$ l'ensemble des unrolls obtenables depuis les éléments de $\mathbb{P}$. Mais pour le moment, nous continuons à nous concentrer sur le pseudo-semi-anneau $(\mathbb{F}_f, +, \times)$ et nous obtenons le résultat suivant.

Proposition 6.15. La fonction $\mathfrak{U}(X)$ est un isomorphisme entre $(\mathbb{F}_f, +, \times)$ et $(\mathfrak{U}(\mathbb{F}_f), +, \otimes)$, où $\mathfrak{U}(\mathbb{F}_f)$ désigne l'ensemble des unrolls obtenables depuis les éléments de $\mathbb{F}_f$.

Démonstration. Comme par définition $\mathfrak{U}(A + B) = \mathfrak{U}(A) + \mathfrak{U}(B)$ pour tous systèmes partiels A et B , il suit que le Lemme 6.14 induit que l'unroll est un morphisme entre $(\mathbb{F}_f, +, \times)$ et $(\mathfrak{U}(\mathbb{F}_f), +, \otimes)$. Il nous faut donc encore démontrer que la fonction $\mathfrak{U}(X)$ est une bijection. Puisque, par définition du codomaine de $\mathfrak{U}(X)$, l'unroll des systèmes partiels est surjectif, il nous suffit de prouver qu'il est aussi injectif.

Soient $\mathbf{A}, \mathbf{B}$ deux forêts finies telles que $\mathbf{A} \neq \mathbf{B}$. Sans perte de généralité, nous supposons que $\mathbf{A} = \mathbf{a}_1 + \dots + \mathbf{a}_{m_A}$ et $\mathbf{B} = \mathbf{b}_1 + \dots + \mathbf{b}_{m_B}$ et que ces deux forêts sont triées selon l'ordre total sur les arbres introduit dans la Section 1.3.

Comme $\mathbf{A} \neq \mathbf{B}$, il existe un entier i tel que $\mathbf{a}_i \neq \mathbf{b}_i$. Deux cas sont alors possibles. Soit le nombre d'occurrences de $\mathbf{a}_i$ est différent entre $\mathfrak{U}(\mathbf{A})$ et $\mathfrak{U}(\mathbf{B})$, signifiant que $\mathfrak{U}(\mathbf{A}) \neq \mathfrak{U}(\mathbf{B})$. Soit ils ont le même nombre d'occurrences. Donc, comme le nombre d'occurrences de $\mathbf{a}_i$ est différent entre $\mathbf{A}$ et $\mathbf{B}$, nous déduisons que le nombre d'arbres qui produisent $\mathbf{a}_i$ par l'application de la fonction $\mathcal{D}^j(\mathbf{X})$ est différent entre $\mathbf{A}$ et $\mathbf{B}$. Conséquemment, il existe deux autres arbres $\mathbf{a}_{i'}$ et $\mathbf{b}_{i'}$ étant différents et de profondeur strictement supérieure à celle de $\mathbf{a}_i$.

En appliquant ce raisonnement jusqu'à la profondeur maximale, nous déduisons qu'il existe deux arbres qui n'ont pas le même nombre d'occurrences entre $\mathfrak{U}(\mathbf{A})$ et $\mathfrak{U}(\mathbf{B})$, donc $\mathfrak{U}(\mathbf{A}) \neq \mathfrak{U}(\mathbf{B})$. Ce qui signifie que $\mathfrak{U}(X)$ est injective et donc bijective. $\square$

Comme $\mathfrak{U}(X)$ est une fonction bijective entre $\mathbb{F}_f$ et $\mathfrak{U}(\mathbb{F}_f)$, il existe une fonction réciproque qui permet de reconstruire un élément de $\mathbb{F}_f$ à partir de son unroll. Et pour pouvoir faire cela, il nous faut aussi une méthode permettant de vérifier si une forêt

finie est un unroll. En réalité, nous pouvons faire les deux simultanément. En effet, si nous considérons une forêt finie $\mathbf{A}$, nous pouvons calculer l'unroll de ses arbres les plus hauts. Puis nous pouvons ajouter ces arbres les plus hauts à une solution (ils ne peuvent pas être issus de l'unroll d'un arbre plus haut qu'eux) et vérifier que leur unroll est un sous-multi-ensemble de $\mathbf{A}$. Si c'est bien le cas, alors nous retirons cet unroll de $\mathbf{A}$ et réappliquons cette procédure sur le système créé. Dans le cas contraire, $\mathbf{A}$ n'est pas un unroll et donc nous renvoyons faux. La preuve de correction de cette procédure est directe et elle est, de plus, en temps polynomial sur la taille de la forêt testée. Nous caractérisons maintenant quels sont les $\mathbf{X}$ et $\mathbf{Y}$ possibles tels que $\mathbf{A} \times \mathbf{X} = \mathbf{A} \times \mathbf{Y}$.

Proposition 6.16. *Soient $\mathbf{A}$, $\mathbf{X}$ et $\mathbf{Y}$ trois forêts finies. Alors $\mathbf{A} \times \mathbf{X} = \mathbf{A} \times \mathbf{Y}$ si et seulement si les troncatures de $\mathbf{X}$ et $\mathbf{Y}$ à profondeur $d(\mathbf{A})$ sont égales.*

Démonstration. Nous réalisons un raisonnement par équivalence. Par la Proposition 6.15, nous avons que $\mathbf{A} \times \mathbf{X} = \mathbf{A} \times \mathbf{Y}$ si et seulement si $\mathcal{U}(\mathbf{A}) \otimes \mathcal{U}(\mathbf{X}) = \mathcal{U}(\mathbf{A}) \otimes \mathcal{U}(\mathbf{Y})$. De là, par le Lemme 2.10, il suit que $\mathcal{U}(\mathbf{A}) \otimes \mathcal{U}(\mathbf{X}) = \mathcal{U}(\mathbf{A}) \otimes \mathcal{U}(\mathbf{Y})$ si et seulement si $\mathcal{C}_{d(\mathbf{A})}(\mathcal{U}(\mathbf{X})) = \mathcal{C}_{d(\mathbf{A})}(\mathcal{U}(\mathbf{Y}))$. $\square$

De la Proposition 6.16, il existe pour toute forêt finie $\mathbf{A}$ au moins deux forêts finies distinctes $\mathbf{X}$ et $\mathbf{Y}$ telles que $\mathbf{A} \times \mathbf{X} = \mathbf{A} \times \mathbf{Y}$. En effet, pour toute forêt finie $\mathbf{A}$, nous pouvons toujours construire une forêt finie $\mathbf{X}$, composée d'un chemin $\mathbf{x}_1$ de profondeur $2d(\mathbf{A}) + 2$ et d'un chemin $\mathbf{x}_2$ de profondeur $2d(\mathbf{A})$, et une autre forêt finie $\mathbf{Y}$, constituée de deux chemins $\mathbf{y}_1, \mathbf{y}_2$ de profondeur $2d(\mathbf{A}) + 1$. Nous avons alors que $\mathcal{C}_{d(\mathbf{A})}(\mathcal{U}(\mathbf{X})) = \mathcal{C}_{d(\mathbf{A})}(\mathcal{U}(\mathbf{Y}))$. En effet, la troncature de $\mathcal{U}(\mathbf{x}_1)$ à profondeur $d(\mathbf{A})$ produit $d(\mathbf{A}) + 3$ copies du chemin de profondeur $d(\mathbf{A})$ plus une copie de chaque chemin ayant une profondeur entre $d(\mathbf{A}) - 1$ et 0. La troncature de $\mathcal{U}(\mathbf{x}_2)$ à profondeur $d(\mathbf{A})$ produit, quant à elle, $d(\mathbf{A}) + 1$ copies du chemin de profondeur $d(\mathbf{A})$ plus une copie de chaque chemin ayant une profondeur entre $d(\mathbf{A}) - 1$ et 0. Enfin, les troncatures de $\mathcal{U}(\mathbf{y}_1)$ et $\mathcal{U}(\mathbf{y}_2)$ à profondeur $d(\mathbf{A})$ produisent $d(\mathbf{A}) + 2$ copies du chemin de profondeur $d(\mathbf{A})$ plus une copie de chaque chemin ayant une profondeur entre $d(\mathbf{A}) - 1$ et 0. Ceci conclut la preuve du théorème suivant.

Théorème 6.17. *Aucune forêt finie n'est annulable dans le pseudo-semi-anneau $(\mathbb{F}_f, +, \times)$.*

Grâce au Théorème 6.17 nous savons que si un système partiel A est annulable, alors $\mathcal{T}(A)$ est non vide. Il nous faut encore montrer que $\mathcal{T}(A)$ doit nécessairement contenir un dendron pour que A soit annulable. Pour cela, comme dit précédemment, nous caractérisons le comportement de $\mathbf{A} \times B$ avec B un FDDS.

Lemme 6.18. *Soit $\mathbf{A} = (\mathbf{A}, f_{\mathbf{A}})$ une forêt finie et $B = (B, f_B)$ un FDDS. Alors $\mathbf{A} \times B$ est égal à $\mathcal{U}(B) \otimes \mathbf{A}$.*

Démonstration. Dans cette preuve $\mathcal{U}(B)$ n'est pas considéré à isomorphisme près. En d'autres mots, chaque sommet a une étiquette de la forme (b, k) avec b un sommet de B et $k \geq 0$ un entier. Soient V et E les ensembles des sommets et des arcs de $\mathbf{A} \times B$ et

soient V' et E' ceux de $\mathcal{U}(B) \otimes \mathbf{A}$. Pour montrer que $\mathbf{A} \times B$ est isomorphe à $\mathcal{U}(B) \otimes \mathbf{A}$, nous commençons par prouver que ces deux graphes ont le même nombre de sommets.

Par définition, V contient le produit cartésien des sommets de $\mathbf{A}$ avec ceux de B . Donc $|V| = |\mathbf{A}| \cdot |B|$. Il nous faut donc montrer que V' a la même taille. Considérons a un sommet de $\mathbf{A}$ de profondeur d . Par définition du produit d'arbres, nous avons que V' contient un sommet $(a, (b, k))$ pour tout sommet (b, k) de $\mathcal{U}(B)$ dont la profondeur est d . Or, comme la profondeur de (b, k) dans $\mathcal{U}(B)$ est k , nous déduisons que V' contient l'ensemble des sommets de la forme $(a, (b, d))$. De plus, remarquons que V' ne contient pas de sommet de la forme $(a, (b, k))$ si $k \neq d$. Maintenant, par définition des sommets de $\mathcal{U}(B)$, nous déduisons que tous les sommets de la forme (b, d) avec b un sommet de B appartiennent à $\mathcal{U}(B)$. Donc, un sommet $(a, (b, k))$ appartient à V' si et seulement si a et b sont respectivement des sommets de $\mathbf{A}$ et B et que k est la profondeur de a dans $\mathbf{A}$. Par conséquent, nous pouvons définir deux fonctions surjectives $f : V' \rightarrow V$ et $g : V \rightarrow V'$ telles que $f((a, (b, k))) = (a, b)$ et $g((a, b)) = (a, (b, k))$ avec k la profondeur de a dans $\mathbf{A}$. Nous déduisons donc qu'il existe une bijection entre V et V' impliquant que ces deux ensembles ont la même taille. Observons en outre que g est la fonction réciproque de f .

Montrons que g est un isomorphisme entre $\mathbf{A} \times B$ et $\mathcal{U}(B) \otimes \mathbf{A}$. Considérons un arc $((a, (b, k)), (a', (b', k-1)))$ de E' . Par définition du produit d'arbres, nous déduisons que l'arc (a, a') appartient à $\mathbf{A}$ et que l'arc $((b, k), (b', k-1))$ appartient à $\mathcal{U}(B)$. Comme la définition des unrolls des systèmes partiels implique que l'arc (b, b') appartient à B , nous inférons que l'arc $((a, b), (a', b'))$ appartient à E . Il suit que si $((a, (b, k)), (a', (b', k-1)))$ est un arc de E' alors $(g((a, (b, k))), g((a', (b', k-1))))$ est un arc de E . Par un raisonnement similaire, nous prouvons que si $((a, b), (a', b'))$ est un arc de E , alors $(f((a, b)), f((a', b')))$ est un arc de E' . Nous concluons que les deux graphes sont isomorphes. $\square$

Corollaire 6.19. *Soit $\mathbf{A}$ une forêt finie et B une permutation. Alors $\mathbf{A} \times B$ est égal à $|B|\mathbf{A}$.*

Démonstration. Par le Lemme 6.18, nous avons que $\mathbf{A} \times B$ est égal à $\mathcal{U}(B) \otimes \mathbf{A}$. Comme B est une permutation, son unroll de système partiel est constitué de $|B|$ chemins simples infinis. En d'autres termes, l'unroll de système partiel de B est égal à $|B|$ fois le neutre de $(\mathbb{F}, +, \otimes)$. Donc, il suit que $\mathcal{U}(B) \otimes \mathbf{A}$ est égal à $|B|\mathbf{A}$. $\square$

Proposition 6.20. *Soit A un système partiel. Si A est annulable, alors A contient un dendron.*

Démonstration. Nous raisonnons par contraposée. Supposons que A ne contienne pas de dendron. Alors, soit $\mathcal{T}(A)$ est vide et donc A n'est pas annulable par le Théorème 6.17. Soit $\mathcal{T}(A)$ n'est pas vide. De là, en reprenant les permutations X et Y introduites dans la Sous-section 3.1.3, construites à partir de $\mathcal{T}(A)$, par le Corollaire 6.19, nous avons que $A \times X = |X|\mathcal{T}(A) + \mathcal{T}(A) \times X$ et $A \times Y = |Y|\mathcal{T}(A) + \mathcal{T}(A) \times Y$. Mais par la preuve du Lemme 3.20, nous avons que $\mathcal{T}(A) \times X = \mathcal{T}(A) \times Y$. Donc en particulier, $|X| = |Y|$ et donc $A \times X = A \times Y$ avec $X \neq Y$. $\square$

6. Les systèmes partiels – 6.1. Propriétés algébriques des systèmes partiels

Nous montrons maintenant que si un système partiel A contient un dendron, alors il est annulable. Considérons X et Y deux systèmes partiels. Comme dit précédemment, le fait que $A \times X = A \times Y$ induit que :

$$\mathcal{T}(X) \times (\mathcal{T}(A) + \mathcal{F}(A)) + \mathcal{F}(X) \times (\mathcal{T}(A) + \mathcal{F}(A)) = \mathcal{T}(Y) \times (\mathcal{T}(A) + \mathcal{F}(A)) + \mathcal{F}(Y) \times (\mathcal{T}(A) + \mathcal{F}(A)). \quad (3)$$

De cette égalité, il suit que $\mathcal{T}(A) \times \mathcal{T}(X) = \mathcal{T}(A) \times \mathcal{T}(Y)$. Or, par le Théorème 1.71, le fait que A contient un dendron implique que $\mathcal{T}(X) = \mathcal{T}(Y)$. Nous déduisons de (3) que

$$\mathcal{F}(X) \times (\mathcal{T}(A) + \mathcal{F}(A)) = \mathcal{F}(Y) \times (\mathcal{T}(A) + \mathcal{F}(A)).$$

Pour montrer que cette égalité induit que $\mathcal{F}(X) = \mathcal{F}(Y)$, nous prouvons que les systèmes partiels qui contiennent des cycles sont annulables du point de vue des forêts finies. C'est-à-dire que si $A \times \mathbf{X} = A \times \mathbf{Y}$ avec $\mathcal{T}(A) \neq \emptyset$, alors $\mathbf{X} = \mathbf{Y}$. La preuve que nous utiliserons reposera sur l'ordre total compatible avec le produit $\otimes$ défini sur les arbres. Afin de pouvoir utiliser cet ordre simplement, nous introduisons le lemme suivant.

Lemme 6.21. *La fonction $\mathfrak{U}(X)$ est un morphisme entre $(\mathbb{P}, +, \times)$ et $(\mathfrak{U}(\mathbb{P}), +, \otimes)$.*

Démonstration. Par définition, $\mathfrak{U}(X)$ se distribue sur la somme. De plus, toujours par définition, l'unroll de système partiel du système vide est la forêt vide et celui du point fixe est le chemin simple infini. Donc l'image du neutre additif et multiplicatif de $(\mathbb{P}, +, \times)$ est respectivement le neutre additif et multiplicatif de $(\mathfrak{U}(\mathbb{P}), +, \otimes)$. Il nous reste donc à démontrer que $\mathfrak{U}(X)$ se distribue sur le produit.

Soient A et B deux systèmes partiels. Dans cette preuve, les unrolls de systèmes partiels ainsi que A et B ne sont pas pris à isomorphisme près. Nous considérons que $\mathfrak{U}(A \times B)$ a pour ensemble de sommets V et pour ensemble d'arcs E . De même, nous considérons que $\mathfrak{U}(A) \otimes \mathfrak{U}(B)$ a pour ensemble de sommets V' et ensemble d'arcs E' . Nous montrons que $\mathfrak{U}(A \times B)$ est égal à $\mathfrak{U}(A) \otimes \mathfrak{U}(B)$. Pour cela, nous définissons une fonction bijective entre V et V' et nous prouverons que cette dernière est un isomorphisme entre (V, E) et (V', E') .

Par définition, nous avons que V est l'ensemble des sommets $((a, b), k)$ avec a et b des sommets de A et B et $k \geq 0$ un entier. Considérons la fonction bijective f qui à chaque sommet $((a, b), k)$ associe $((a, k), (b, k))$. Notons $f(V)$ l'ensemble des images des sommets de V par f . Nous avons directement que f est une bijection entre V et $f(V)$. Il nous reste à démontrer que $f(V) = V'$. Pour cela, rappelons que $\mathfrak{U}(A)$ et $\mathfrak{U}(B)$ ont respectivement pour ensemble de sommets tous les couples possibles (a, k) et (b, k) avec a et b des sommets de respectivement A et B et $k \geq 0$ un entier. De plus, les sommets (a, k) et (b, k) ont profondeur k dans respectivement $\mathfrak{U}(A)$ et $\mathfrak{U}(B)$. Ceci induit que $V' = \{((a, k), (b, k)) \mid a \in A, b \in B, k \in \mathbb{N}\}$. Donc, $f(V) = V'$.

Montrons maintenant que f est un isomorphisme entre $\mathfrak{U}(A \times B)$ et $\mathfrak{U}(A) \otimes \mathfrak{U}(B)$. Considérons $e = \left(((a, b), k), ((a', b'), k-1) \right)$. Alors, par définition de l'unroll de systèmes partiels, e est un arc de E si et seulement si $((a, b), (a', b'))$ est un arc de $A \times B$ et que $k \geq 1$. Et par définition du produit direct, $((a, b), (a', b'))$ est un arc de $A \times B$ si et seulement si (a, a') est un arc de A et (b, b') est un arc de B . De nouveau,

par définition de l'unroll de système partiel, nous avons que $((a, k), (a', k - 1))$ est un arc de $\mathcal{U}(A)$ si et seulement si (a, a') est un arc de A et que $k \geq 1$. De même, $((b, k), (b', k - 1))$ est un arc de $\mathcal{U}(B)$ si et seulement si (b, b') est un arc de B et que $k \geq 1$. Donc, par définition du produit d'arbres, $((a, k), (b, k), ((a', k - 1), (b', k - 1)))$ est un arc de E' si et seulement si (a, a') est un arc de A , (b, b') est un arc de B et $k \geq 1$. Nous concluons que $((a, b), k), ((a', b'), k - 1))$ est un arc de E si et seulement si l'arc $(f((a, b), k), f((a', b'), k - 1)))$ appartient à E' . Et donc, l'assertion suit. $\square$

Corollaire 6.22. Soient A un système partiel, $\mathbf{X}$ et $\mathbf{B}$ deux forêts finies et $k > 0$ un entier. Alors $A \times \mathbf{X}^k = \mathbf{B}$ si et seulement si $\mathcal{U}(A) \otimes \mathcal{U}(\mathbf{X})^k = \mathcal{U}(B)$.

Démonstration. $(\Rightarrow)$ Si $A \times \mathbf{X}^k = \mathbf{B}$ alors le Lemme 6.21, induit que $\mathcal{U}(A) \otimes \mathcal{U}(\mathbf{X})^k = \mathcal{U}(\mathbf{B})$
 $(\Leftarrow)$ Supposons que $\mathcal{U}(A) \otimes \mathcal{U}(\mathbf{X})^k = \mathcal{U}(\mathbf{B})$. Donc, par le Lemme 6.21, nous inférons que $\mathcal{U}(A \times \mathbf{X}^k) = \mathcal{U}(\mathbf{B})$. Or, comme $\mathbf{X}^k$ est une forêt finie, nous déduisons que c'est aussi le cas de $A \times \mathbf{X}^k$. Donc par la Proposition 6.15, nous concluons que $A \times \mathbf{X}^k = \mathbf{B}$. $\square$

Lemme 6.23. Soit A un système partiel tel que $\mathcal{T}(A) \neq \emptyset$. Soient $\mathbf{X}$ et $\mathbf{Y}$ deux forêts finies. Si $A \times \mathbf{X} = A \times \mathbf{Y}$ alors $\mathbf{X} = \mathbf{Y}$.

Démonstration. Supposons que $A \times \mathbf{X} = A \times \mathbf{Y}$. Donc par le Lemme 6.21, nous obtenons que $\mathcal{U}(A) \otimes \mathcal{U}(\mathbf{X}) = \mathbf{B} = \mathcal{U}(A) \otimes \mathcal{U}(\mathbf{Y})$.

Considérons $\mathbf{t}$ le plus petit arbre, selon l'ordre compatible avec le produit, de $\mathbf{B}$ avec profondeur maximale.

Nous avons alors que $\mathbf{ax} = \mathbf{t} = \mathbf{ay}$ avec le plus petit arbre $\mathbf{a}$ de $\mathcal{U}(A)$ de profondeur au moins $d(\mathbf{t})$ et $\mathbf{x}$ et $\mathbf{y}$ les plus petits arbres de respectivement $\mathcal{U}(\mathbf{X})$ et $\mathcal{U}(\mathbf{Y})$ de profondeur $d(\mathbf{t})$. En effet, nous savons que $d(\mathbf{x}) = d(\mathbf{t}) = d(\mathbf{y})$ car $\mathcal{U}(A)$ contient des arbres infinis. De plus, le Lemme 1.57 induit que $\mathcal{C}_{d(\mathbf{a})}(\mathbf{x}) = \mathcal{C}_{d(\mathbf{a})}(\mathbf{y})$. Or, comme la profondeur de $\mathbf{a}$ est supérieure ou égale à celle de $\mathbf{x}$ et $\mathbf{y}$, $\mathcal{C}_{d(\mathbf{a})}(\mathbf{x}) = \mathbf{x}$ et $\mathcal{C}_{d(\mathbf{a})}(\mathbf{y}) = \mathbf{y}$. Il suit que $\mathbf{x} = \mathbf{y}$ et conséquemment, $\mathcal{U}(A) \otimes (\mathcal{U}(\mathbf{X}) - \mathbf{x}) = \mathcal{U}(A) \otimes (\mathcal{U}(\mathbf{Y}) - \mathbf{y})$. En appliquant récursivement ce raisonnement, nous déduisons que $\mathcal{U}(\mathbf{X}) = \mathcal{U}(\mathbf{Y})$. De la Proposition 6.15, nous concluons que $\mathbf{X} = \mathbf{Y}$. $\square$

Ainsi, si nous reprenons notre équation

$$\mathcal{F}(X) \times (\mathcal{T}(A) + \mathcal{F}(A)) = \mathcal{F}(Y) \times (\mathcal{T}(A) + \mathcal{F}(A)),$$

le Lemme 6.23 induit que $\mathcal{F}(X) = \mathcal{F}(Y)$ puisque $\mathcal{T}(A)$ contient un dendron (donc n'est pas vide). Nous en concluons que si A contient un dendron, alors $\mathcal{T}(X) = \mathcal{T}(Y)$ et $\mathcal{F}(X) = \mathcal{F}(Y)$ pour tous les systèmes partiels X et Y tels que $A \times X = A \times Y$. Ceci induit que A est annulable. En combinant ce résultat avec la Proposition 6.20, nous concluons la preuve du théorème suivant.

Théorème 6.24. Soit A un système partiel. Alors A est annulable si et seulement si A contient un dendron.

6. Les systèmes partiels – 6.2. Résolution de $P(X) = B$ pour les systèmes partiels

Nous pouvons étendre ce théorème afin de caractériser les polynômes de système partiel injectifs.

Théorème 6.25. *Soit $P = \sum_{i=0}^m A_i X^i$ un polynôme sur les systèmes partiels. Alors, P est injectif si et seulement s'il existe $i > 0$ tel que A_i contient un dendron.*

Démonstration. ($\Leftarrow$) Supposons qu'il existe $j > 0$ tel que A_j contient un dendron. Soient X et Y deux systèmes partiels tels que $P(X) = P(Y)$. Étant donné que nous avons $\mathcal{T}(P(X)) = \sum_{i=0}^m \mathcal{T}(A_i) \mathcal{T}(X)^i$, le Théorème 3.17 induit que $\mathcal{T}(X) = \mathcal{T}(Y)$. De surcroît, nous avons que $\mathcal{U}(P(X)) = \mathcal{U}(P(Y))$. Ainsi, comme $\mathcal{U}(P(X)) = \sum_{i=0}^m \mathcal{U}(A_i) \mathcal{U}(X)^i$ et puisque $\mathcal{U}(A_j)$ contient un arbre infini, la Proposition 2.51 induit que $\mathcal{U}(X) = \mathcal{U}(Y)$. Donc $\mathcal{U}(X) - \mathcal{U}(\mathcal{T}(X)) = \mathcal{U}(Y) - \mathcal{T}(Y)$ et la Proposition 6.15 nous permet de conclure que $\mathcal{F}(X) = \mathcal{F}(Y)$ et aussi que $X = Y$.

($\Rightarrow$) Supposons que A_i ne contienne pas de dendron pour tout entier $i > 0$. Alors deux cas sont possibles. Soit $\mathcal{T}(A_i) = \emptyset$ pour tout entier $i > 0$. Considérons $X = C_1 + C_1$ et $Y = C_2$. Alors $|X|^k = |Y|^k$ pour tout entier $k \geq 0$. Donc comme tous les A_i avec $i > 0$ sont des forêts finies, le Corollaire 6.19 induit que $P(X) = P(Y)$. Et donc P n'est pas injectif.

L'autre cas est que $\mathcal{T}(A_i) \neq \emptyset$ pour au moins un $i > 0$. Alors, en reprenant les permutations X et Y de la preuve du Théorème 3.25, il suit que $\mathcal{F}(P(X)) = \mathcal{F}(P(Y))$ par le Corollaire 6.19 car X et Y ne sont que des permutations de même taille et que $\mathcal{T}(P(X)) = \mathcal{T}(P(Y))$ par le Théorème 3.25. Nous en concluons que $P(X) = P(Y)$, donc que P n'est pas injectif. $\square$

6.2. Résolution de $P(X) = B$ pour les systèmes partiels

Dans cette section, nous démontrerons que résoudre les équations de la forme $P(X) = B$ dans le cas des systèmes partiels n'est pas plus difficile que de résoudre $P(X) = B$ dans le cas des FDDS. Plus précisément, nous prouverons que si nous avons un algorithme polynomial pour résoudre $P(X) = B$ avec P un polynôme sur les FDDS et B un FDDS nous en avons également un pour résoudre $P'(X) = B'$ en temps polynomial avec P' un polynôme sur les systèmes partiels construit en ajoutant des forêts finies aux coefficients de P et B' un système partiel tel que $\mathcal{T}(B') = B$.

Considérons $P = \sum_{i=0}^m A_i X^i$ un polynôme sur les systèmes partiels et soit B un système partiel. Directement, nous constatons que résoudre $P(X) = B$ est équivalent à résoudre $P(X) - A_0 = B - A_0$ si $A_0 \leq B$, sinon aucune solution n'existe. Donc, nous ne considérerons dans la suite que des polynômes sans terme constant. Nous commencerons par montrer que si au moins un des coefficients de P contient un cycle et qu'il existe une solution X à $P(X) = B$, alors nous trouvons $\mathcal{F}(X)$ en temps polynomial (Section 6.2.1). Puis nous prouverons que si tous les coefficients de P sont des forêts finies alors nous résolvons $P(X) = B$ en temps polynomial (Section 6.2.2).

6.2.1. Résoudre $P(X) = B$ lorsqu'un coefficient non constant A est tel que $\mathcal{T}(A) \neq \emptyset$

Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme sur les systèmes partiels sans terme constant dont l'un des coefficients, disons i , contient un cycle. Soit B un système partiel tel que l'équation $P(X) = B$ admet une solution. Pour montrer que nous sommes en mesure de retrouver la partie forêt finie de la solution, nous commençons par étendre le Lemme 6.23 au cas des polynômes. En d'autres mots, nous montrons que toutes les solutions de $P(X) = B$ ont la même partie forêt finie. Après quoi, nous n'aurons plus qu'à démontrer qu'il est possible de construire cette forêt en temps polynomial.

Lemme 6.26. *Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme sur les systèmes partiels sans terme constant et qui admet i tel que A_i contient un cycle. Soient X, Y et B des systèmes partiels. Si $P(X) = P(Y)$ alors $\mathcal{F}(X) = \mathcal{F}(Y)$.*

Démonstration. Supposons que $P(X) = P(Y)$. Donc, par le Lemme 6.21

$$\sum_{i=1}^m \mathfrak{U}(A_i) \otimes \mathfrak{U}(X)^i = \sum_{i=1}^m \mathfrak{U}(A_i) \otimes \mathfrak{U}(Y)^i.$$

De fait, par la Proposition 2.51, il suit que $\mathfrak{U}(X) = \mathfrak{U}(Y)$. En effet, comme l'un des $\mathfrak{U}(A_i)$ contient un arbre de profondeur infinie, la Proposition 2.51 implique que $\sum_{i=1}^m \mathfrak{U}(A_i) \otimes \mathfrak{U}(X)^i$ est injectif dans $(\mathbb{F}, +, \otimes)$. De là, par la Proposition 6.15, pour montrer que $\mathcal{F}(X) = \mathcal{F}(Y)$, il nous suffit de prouver que $\mathfrak{U}(\mathcal{F}(X)) = \mathfrak{U}(\mathcal{F}(Y))$. Ou de manière équivalente, que $\mathfrak{U}(\mathcal{T}(X)) = \mathfrak{U}(\mathcal{T}(Y))$. Notons que si $\mathcal{U}(\mathcal{T}(X)) = \mathcal{U}(\mathcal{T}(Y))$ alors $\mathfrak{U}(\mathcal{T}(X)) = \mathfrak{U}(\mathcal{T}(Y))$. En effet, nous avons que $\mathfrak{U}(\mathcal{T}(A))$ est la somme de $\mathcal{U}(\mathcal{T}(A))$ avec l'unroll de système partiel de chaque arbre fini enraciné dans les sommets prédécesseurs de la racine d'un arbre de $\mathcal{U}(\mathcal{T}(A))$. Donc pour démontrer notre assertion, il nous suffit de prouver que $\mathcal{U}(\mathcal{T}(X)) = \mathcal{U}(\mathcal{T}(Y))$.

Pour cela, relevons que par hypothèse $\sum_{i=1}^m \mathfrak{U}(A_i) \otimes \mathfrak{U}(X)^i$ et $\sum_{i=1}^m \mathfrak{U}(A_i) \otimes \mathfrak{U}(Y)^i$ ont les mêmes arbres infinis. Comme, par définition du produit, ces derniers sont issus du produit d'arbres infinis des $\mathfrak{U}(A_i)$ et $\mathfrak{U}(X)^i$ ou $\mathfrak{U}(Y)^i$, et que de plus tout arbre infini de $\mathfrak{U}(A)$ est issu de $\mathcal{U}(\mathcal{T}(A))$, nous inférons que

$$\sum_{i=1}^m \mathcal{U}(\mathcal{T}(A_i)) \otimes \mathcal{U}(\mathcal{T}(X))^i = \sum_{i=1}^m \mathcal{U}(\mathcal{T}(A_i)) \otimes \mathcal{U}(\mathcal{T}(Y))^i.$$

Ainsi, par le Théorème 2.52, nous déduisons que $\mathcal{U}(\mathcal{T}(X)) = \mathcal{U}(\mathcal{T}(Y))$. □

Pour montrer que nous sommes en mesure de trouver la partie forêt finie d'une solution de l'équation $P(X) = B$, nous prouvons que nous résolvons efficacement l'équation $\sum_{i=1}^m \mathfrak{U}(A_i) \mathfrak{U}(X)^i = \mathfrak{U}(B)$. À cette fin, nous souhaitons utiliser l'algorithme introduit dans la Sous-section 2.4 permettant de résoudre les équations sur les forêts finies en temps polynomial. Mais pour cela, il nous faut étendre la Proposition 2.57 au cas des unrolls de systèmes partiels.

6. Les systèmes partiels – 6.2. Résolution de $P(X) = B$ pour les systèmes partiels

Proposition 6.27. Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme sur les FDDS, soit B un FDDS et α le nombre d'arbres dans $\mathcal{U}(B)$. Soit $n \geq 2 \cdot \alpha^2 + d(B)$ un entier. Alors, il existe une forêt finie $\mathbf{Y}$ de profondeur n telle que $\sum_{i=0}^m \mathcal{C}_n(\mathcal{U}(A_i)) \mathbf{Y}^i = \mathcal{C}_n(\mathcal{U}(B))$ si et seulement si il existe un FDDS X tel que $\mathcal{U}(P(X)) = \mathcal{U}(B)$. De plus, le cas échéant $\mathcal{C}_n(\mathcal{U}(X)) = \mathbf{Y}$.

Démonstration. ($\Rightarrow$) Supposons qu'il existe une forêt finie $\mathbf{Y}$ de profondeur n , telle que $\sum_{i=0}^m \mathcal{C}_n(\mathcal{U}(A_i)) \mathbf{Y}^i = \mathcal{C}_n(\mathcal{U}(B))$. Soit $\mathbf{Z}$ le sous-multi-ensemble de $\mathbf{Y}$ qui ne contient que les arbres de profondeur n . Alors, nous avons que $\sum_{i=0}^m \mathcal{C}_n(\mathcal{U}(A_i)) \mathbf{Z}^i = \mathcal{C}_n(\mathcal{U}(B))$. En effet, aucun arbre fini des $\mathcal{U}(A_i)$ et $\mathcal{U}(B)$ n'a profondeur n . Donc par la Proposition 2.57, il suit qu'il existe un FDDS X tel que $\mathcal{C}_n(\mathcal{U}(X)) = \mathbf{Z}$ et donc $\mathcal{C}_n(\mathcal{U}(X)) = \mathbf{Y}$.

($\Leftarrow$) Supposons qu'il existe un FDDS X tel que $\mathcal{U}(P(X)) = \mathcal{U}(B)$. Alors, en particulier $\mathcal{U}(P(X)) = \mathcal{U}(B)$ car l'ensemble des arbres infinis de $\mathcal{U}(P(X))$ et de $\mathcal{U}(B)$ est contenu dans respectivement $\mathcal{U}(P(X))$ et $\mathcal{U}(B)$. La direction suit par la Proposition 2.57. $\square$

En nous appuyant sur la Proposition 6.27, nous déduisons un algorithme capable de trouver la partie forêt finie $\mathcal{F}(X)$ d'une solution X de $P(X) = B$, si une solution existe. Pour cela, il suffit de calculer chaque $\mathcal{C}_n(\mathcal{U}(A_i))$ et $\mathcal{C}_n(\mathcal{U}(B))$ avec un n étant supérieur ou égal à $2 \cdot |B|^2 + \max(d(\mathcal{F}(B)), d(\mathcal{T}(B)))$. Puis de résoudre l'équation sur les arbres finis ainsi obtenus. Appelons $\mathbf{X}$ la forêt obtenue. Notons que s'il existe une solution à $P(X) = B$, alors $\mathbf{X}$ est bien définie.

Après quoi, nous calculons chaque $\mathcal{C}_n(\mathcal{U}(\mathcal{T}(A_i)))$ et $\mathcal{C}_n(\mathcal{U}(\mathcal{T}(B)))$ et résolvons l'équation associée. Nommons $\mathbf{T}$ la forêt obtenue. Observons que s'il existe X tel que $P(X) = B$ alors $\sum_{i=1}^m \mathcal{T}(A_i) \mathcal{T}(X)^i = \mathcal{T}(B)$. Et conséquemment, nous avons que $\sum_{i=1}^m \mathcal{U}(\mathcal{T}(A_i)) \mathcal{U}(\mathcal{T}(X))^i = \mathcal{U}(\mathcal{T}(B))$. Et par la Proposition 6.27, cette équation a une solution si et seulement si l'équation $\sum_{i=1}^m \mathcal{C}_n(\mathcal{U}(\mathcal{T}(A_i))) \mathcal{C}_n(\mathcal{U}(\mathcal{T}(X)))^i = \mathcal{C}_n(\mathcal{U}(\mathcal{T}(B)))$ en a également une. Ceci signifie que $\mathbf{T}$ est bien définie.

Maintenant, il suffit de retirer $\mathbf{T}$ de $\mathbf{X}$ pour obtenir $\mathbf{F}$. Relevons que s'il existe bien X une solution de $P(X) = B$, alors, la Proposition 2.51 implique $\mathcal{C}_n(\mathcal{U}(X)) = \mathbf{X}$. Mais aussi que $\mathcal{C}_n(\mathcal{U}(\mathcal{T}(X))) = \mathbf{T}$, puisque n est suffisamment grand pour que tous les arbres finis de $\mathcal{U}(X)$ aient une profondeur strictement inférieure à n . Donc :

$$\mathcal{C}_n(\mathcal{U}(\mathcal{F}(X))) = \mathcal{U}(\mathcal{F}(X)) = \mathbf{X} - \mathbf{T} = \mathbf{F}.$$

Pour finir, il nous suffit de vérifier que $\mathbf{F}$ est bien l'unroll de système partiel d'une forêt finie et de reconstruire ladite forêt comme expliqué dans la Section 6.1.2. La correction suit de la Proposition 6.15.

Comme toutes les opérations décrites ci-dessus sont réalisables en temps polynomial, car le n choisi est polynomial sur la taille de B , nous concluons le résultat suivant.

Proposition 6.28. Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme sur les systèmes partiels sans terme constant et qui admet i tel que A_i contient un cycle. S'il existe X tel que $P(X) = B$, alors nous construisons $\mathcal{F}(X)$ en temps polynomial.

Pour conclure que nous sommes capables de résoudre $P(X) = B$ en temps polynomial si nous avons un algorithme efficace pour résoudre $\sum_{i=1}^m \mathcal{T}(A_i) \mathcal{T}(X)^i = \mathcal{T}(B)$,

6. Les systèmes partiels – 6.2. Résolution de $P(X) = B$ pour les systèmes partiels

nous montrons que s'il existe une solution X à $P(X) = B$, alors nous pouvons en créer une pour chaque solution de $\sum_{i=1}^m \mathcal{T}(A_i) \mathcal{T}(X)^i = \mathcal{T}(B)$.

Théorème 6.29. *Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme sur les systèmes partiels sans terme constant et qui admet i tel que A_i contient un cycle. Si nous pouvons efficacement résoudre l'équation $\sum_{i=1}^m \mathcal{T}(A_i) \mathcal{T}(X)^i = \mathcal{T}(B)$, alors nous résolvons $P(X) = B$ en temps polynomial.*

Démonstration. Nous montrons qu'il existe X une solution de $P(X) = B$ si et seulement si $P(\mathbf{X} + Y) = B$ avec $\mathbf{X}$ la forêt décrite dans les propos précédant la Proposition 6.28 et Y une solution de $\sum_{i=1}^m \mathcal{T}(A_i) \mathcal{T}(X)^i = \mathcal{T}(B)$.

($\Leftarrow$) Supposons que $P(\mathbf{X} + Y) = B$. Alors, directement, il existe X tel que $P(X) = B$.

($\Rightarrow$) Supposons qu'il existe X tel que $P(X) = B$. Alors, par le Lemme 6.26, il suit que toutes les solutions de $P(X) = B$ sont de la forme $\mathcal{F}(X) + Y$ avec Y un FDDS. De plus, comme il existe i tel que $\mathcal{T}(A_i) \neq \emptyset$, nous avons aussi que Y est une solution de $\sum_{i=1}^m \mathcal{T}(A_i) \mathcal{T}(X)^i = \mathcal{T}(B)$. Nous montrons maintenant que toutes les solutions Y de $\sum_{i=1}^m \mathcal{T}(A_i) \mathcal{T}(X)^i = \mathcal{T}(B)$ sont telles que $P(\mathcal{F}(X) + Y) = B$.

Soit $i > 0$ un entier. Nous montrons que $(\mathcal{F}(X) + Y)^i - Y^i = X^i - \mathcal{T}(X)^i$. De façon équivalente, nous démontrons que

$$\sum_{j=0}^{i-1} \binom{i}{j} \mathcal{F}(X)^j \times Y^{i-j} = \sum_{j=0}^{i-1} \binom{i}{j} \mathcal{F}(X)^j \times \mathcal{T}(X)^{i-j}.$$

Pour cela, constatons que du Lemme 6.18, il suit que $\mathcal{F}(X)^j \times Y^{i-j} = \mathcal{F}(X)^j \otimes \mathcal{U}(Y)^{i-j}$. De même, $\mathcal{F}(X)^j \times \mathcal{T}(X)^{i-j} = \mathcal{F}(X)^j \otimes \mathcal{U}(\mathcal{T}(X))^{i-j}$. De là, comme une conséquence directe du Théorème 2.52 est que $\mathcal{U}(Y) = \mathcal{U}(\mathcal{T}(X))$, nous avons que

$$\mathcal{F}(X)^j \times Y^{i-j} = \mathcal{F}(X)^j \times \mathcal{T}(X)^{i-j}.$$

Donc l'assertion suit par somme.

À ce point, nous déduisons que $\mathcal{F}(P(X)) = \mathcal{F}(P(\mathcal{F}(X) + Y))$ et $\mathcal{T}(P(X)) = \mathcal{T}(P(\mathcal{F}(X) + Y))$. Donc, $P(X) = P(\mathcal{F}(X) + Y)$. $\square$

6.2.2. Résoudre $P(X) = B$ si tous les coefficients sont des forêts finies

Soit $P = \sum_{i=1}^m A_i X^i$ un polynôme sur les forêts finies et soit B un système partiel. Dans cette sous-section, nous montrons que nous résolvons l'équation $P(X) = B$ en temps polynomial sur la somme des tailles des A_i et de B . Remarquons que s'il existe un système partiel X tel que $P(X) = B$, alors $\mathcal{T}(B) = \emptyset$. C'est pourquoi, dans la suite de cette section, nous considérons de plus que B est une forêt finie que nous noterons donc $\mathbf{B}$.

Comme tous les coefficients de P et $\mathbf{B}$ sont des forêts finies, nous voudrions utiliser l'Algorithme 2.55 permettant de résoudre en temps polynomial les équations polynomiales sur les forêts finies. Cependant, comme cet algorithme est basé sur

$$\begin{aligned}
 \mathfrak{U}\left(\begin{array}{c} \bullet \\ \diagdown \quad \diagup \\ \bullet \quad \bullet \\ \diagdown \quad \diagup \\ \bullet \quad \bullet \end{array}\right) &= \begin{array}{c} \bullet \\ \diagdown \\ \bullet \end{array} + \begin{array}{c} \bullet \\ \diagup \\ \bullet \end{array} + \begin{array}{c} \bullet \\ \diagdown \quad \diagup \\ \bullet \end{array} + 5 \cdot \\
 \mathfrak{U}\left(\begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \end{array}\right) &= \begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \end{array} + 2 \begin{array}{c} \bullet \\ \diagdown \\ \bullet \end{array} + \begin{array}{c} \bullet \\ \diagup \\ \bullet \end{array} + \begin{array}{c} \bullet \\ \diagdown \quad \diagup \\ \bullet \end{array} + \begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \end{array} + 18 \cdot \\
 \mathfrak{U}\left(\begin{array}{c} \bullet \\ \diagdown \quad \diagup \\ \bullet \quad \bullet \end{array}\right) \times \left(\begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \end{array} + \begin{array}{c} \bullet \\ \diagdown \\ \bullet \end{array} + \begin{array}{c} \bullet \\ \diagup \\ \bullet \end{array} + \begin{array}{c} \bullet \\ \diagdown \quad \diagup \\ \bullet \end{array} + \begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \end{array}\right) &= \mathfrak{U}\left(\begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \end{array}\right)
 \end{aligned}$$

FIGURE 6.31. – Exemple de division d'unroll de système partiel avec un quotient qui n'est pas la troncature d'un unroll de système partiel.

les polynômes dont l'opération de produit est $\otimes$ et non $\times$, il nous faut modifier le polynôme P . Deux choix sont possibles. Soit nous modifions P via les expressions des Lemmes 6.8 et 6.18, soit nous appliquons l'unroll de système partiel sur P . Nous décidons ici d'employer la seconde méthode. Pour argumenter de la correction de cette approche, nous proposons le résultat suivant.

Lemme 6.30. *Soit $P = \sum_{i=1}^m \mathbf{A}_i \times X^i$ un polynôme sur les forêts finies. Considérons $Q = \sum_{i=1}^m \mathfrak{U}(\mathbf{A}_i) \otimes X^i$ le polynôme sur les unrolls de systèmes partiels associé à P . Soit X un système partiel et $\mathbf{B}$ une forêt finie. Alors, $P(X) = \mathbf{B}$ si et seulement si $Q(\mathfrak{U}(X)) = \mathfrak{U}(\mathbf{B})$.*

Démonstration. $(\Rightarrow)$ Supposons que $P(X) = \mathbf{B}$. Alors, par le Lemme 6.21, il suit que $\sum_{i=1}^m \mathfrak{U}(\mathbf{A}_i) \otimes \mathfrak{U}(X)^i = \mathfrak{U}(\mathbf{B})$. Donc, $Q(\mathfrak{U}(X)) = \mathfrak{U}(\mathbf{B})$.

$(\Leftarrow)$ Supposons que $Q(\mathfrak{U}(X)) = \mathfrak{U}(\mathbf{B})$. Donc, nous pouvons partitionner $\mathfrak{U}(\mathbf{B})$ en $\mathfrak{U}(\mathbf{B})_1 + \dots + \mathfrak{U}(\mathbf{B})_m$ tel que $\mathfrak{U}(\mathbf{A}_i) \otimes \mathfrak{U}(X)^i = \mathfrak{U}(\mathbf{B})_i$ pour tous les i et $\sum_{i=1}^m \mathfrak{U}(\mathbf{B})_i = \mathfrak{U}(\mathbf{B})$. Mais puisque par le Lemme 6.21, nous avons $\mathfrak{U}(\mathbf{A}_i) \otimes \mathfrak{U}(X)^i = \mathfrak{U}(\mathbf{A}_i \times X^i)$ pour tous les i , il suit que

$$\sum_{i=1}^m \mathfrak{U}(\mathbf{A}_i \times X^i) = \mathfrak{U}\left(\sum_{i=1}^m \mathbf{A}_i \times X^i\right) = \mathfrak{U}(\mathbf{B}).$$

Maintenant comme $\mathbf{A}_i$ est une forêt finie, il suit que $\mathbf{A}_i \times X^i$ en est aussi une et ce pour tous les i . Ainsi la Proposition 6.15, induit que $\sum_{i=1}^m \mathbf{A}_i \times X^i = \mathbf{B}$. Nous en concluons que $P(X) = \mathbf{B}$. $\square$

Ainsi, en utilisant l'Algorithme 2.55, nous résolvons $Q(\mathbf{X}) = \mathbf{B}$ en temps polynomial; appelons $\mathbf{F}$ la solution trouvée. Relevons que $\mathbf{F}$ est une forêt, tronquée à $d(P)$, la plus grande profondeur des $d(\mathbf{A}_i)$. De plus, $\mathbf{F}$ n'est pas nécessairement issue de l'unroll de système partiel d'un système partiel, comme le montre la Figure 6.31.

Observons que deux cas sont possibles. Premier cas $d(B) < d(P)$. Donc, la Proposition 2.51 induit que $\mathbf{F}$ est la seule forêt telle que $Q(\mathbf{F}) = \mathbf{B}$. De ce fait, nous résolvons $P(X) = \mathbf{B}$ en reconstruisant, si cela est possible, la forêt $\mathbf{X}$ telle que $\mathfrak{U}(\mathbf{X}) = \mathbf{F}$. Ceci étant réalisable en temps polynomial sur la taille de $\mathbf{F}$, et donc sur la taille de $\mathbf{B}$,

6. Les systèmes partiels – 6.2. Résolution de $P(X) = B$ pour les systèmes partiels

comme expliqué dans la discussion précédant la Proposition 6.16. Le second cas est que $d(B) = d(P)$ et il nous faut expliquer comment nous reconstruisons, si cela est possible, un système partiel X tel que $\mathcal{C}_{d(P)}(\mathfrak{U}(X)) = \mathbf{F}$. Ceci est le propos de ce qui suit.

Soit X un système partiel et $d \geq 1$ un entier. Considérons $\mathbf{t}$ un arbre de $\mathfrak{U}(X)$. Pour inférer des propriétés qui permettraient de construire un système partiel à partir d'une forêt $\mathbf{F}$ de profondeur d , nous analysons les éléments qui constituent $\mathfrak{U}(X)$. Relevons que deux cas sont alors possibles.

Soit $\mathbf{t}$ est enraciné dans un sommet de profondeur 0 dans $\mathfrak{U}(X)$, et donc $\mathbf{t}$ est un arbre de $\mathcal{F}(X) + \mathcal{U}(\mathcal{T}(X))$, soit ce n'est pas le cas. Et alors la racine de $\mathbf{t}$ a un successeur u dans X . De nouveau deux cas sont possibles. Le sommet u peut être un sommet d'un cycle de X et donc $\mathbf{t}$ est un arbre de $\mathcal{D}(\mathcal{U}(\mathcal{T}(X)))$. Soit ce n'est pas le cas, impliquant qu'il existe un arbre fini $\mathbf{t}'$ enraciné dans u contenant tous les prédécesseurs de u dans X . Et comme par définition de l'unroll de système partiel, l'arbre $\mathbf{t}'$ appartient à $\mathfrak{U}(X)$, il suit que $\mathbf{t}$ est dans $\mathcal{D}(\mathfrak{U}(X))$. De ces deux derniers cas, nous déduisons qu'il existe un arbre $\mathbf{t}'$ de $\mathfrak{U}(X)$ tel que $\mathcal{D}(\mathcal{C}_d(\mathbf{t}'))$ contient $\mathcal{C}_{d-1}(\mathbf{t})$. De cette réflexion, nous déduisons le lemme suivant.

Lemme 6.32. *Soit $d \geq 1$ un entier et soit $\mathbf{F}$ une forêt finie de profondeur au plus d . S'il existe un système partiel X tel que $\mathcal{C}_d(\mathfrak{U}(X)) = \mathbf{F}$, alors $\mathcal{D}(\mathbf{F})$ est un sous-multi-ensemble de $\mathcal{C}_{d-1}(\mathfrak{U}(X))$.*

Nous montrons maintenant que l'implication inverse est aussi vraie. Pour cela, nous introduisons l'Algorithme 6.38 qui procède en “collant” des arbres à des composantes connexes. Pour définir formellement cette opération de *collage*, nous devons introduire quelques notions préliminaires.

Définition 6.33 (Parties hautes et basses). Les *super-feuilles* de profondeur k d'un système partiel A sont les arbres de $\mathcal{D}(A)$ ayant profondeur k . Nous notons par $F_k(A)$ le multi-ensemble des super-feuilles de A de profondeur k pris à isomorphisme près.

Le multi-ensemble des *parties hautes* d'un système partiel A est, dans notre contexte, le multi-ensemble des super-feuilles enracinées dans les sommets de A de profondeur 0, c'est-à-dire $\mathcal{D}(A)$. Le multi-ensemble des *parties basses* de A est $\mathcal{C}_{d-1}(\mathcal{F}(A))$.

La Figure 6.35 donne un exemple de super-feuilles, parties basses et hautes.

Définition 6.34 (Collage et décollage). Soit $\mathbf{t}$ un arbre et X un système partiel connexe tel que la partie basse de $\mathbf{t}$ est égale à une partie haute h de X , soit s_1 la racine de h et soit s_2 le successeur de s_1 . Nous appelons *collage* de $\mathbf{t}$ sur X par h le système partiel connexe obtenu en retirant s_1 et l'arbre qui lui est enraciné et en désignant s_2 comme le successeur de la racine de $\mathbf{t}$. Nous disons aussi que nous avons *comblé* la partie haute par la partie basse.

La Figure 6.36 donne un exemple de collage. Comme dit précédemment, l'objectif de l'Algorithme 6.38 est de partir d'une forêt $\mathbf{F}$ de profondeur d et de construire par des collages successifs un système partiel X tel que $\mathcal{C}_d(\mathfrak{U}(X)) = \mathbf{F}$. Il commence par

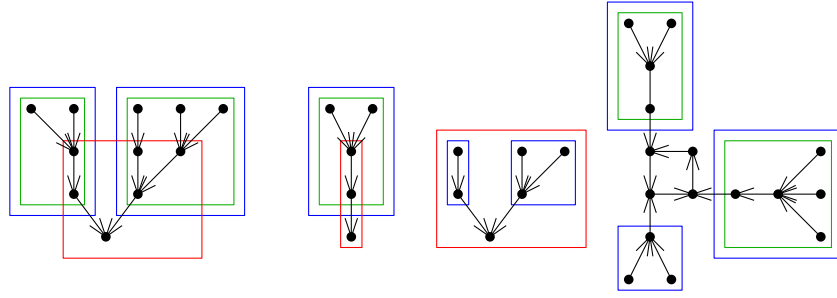


FIGURE 6.35. – Images illustrant les notions de super-feuilles, parties hautes et basses d'un système partiel. Les parties encadrées en rouge sont les parties basses, celles en bleu sont les parties hautes et celles en vert sont les super-feuilles de profondeur 2.

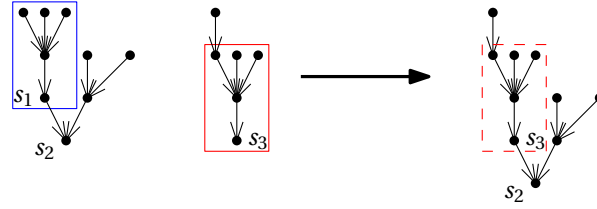


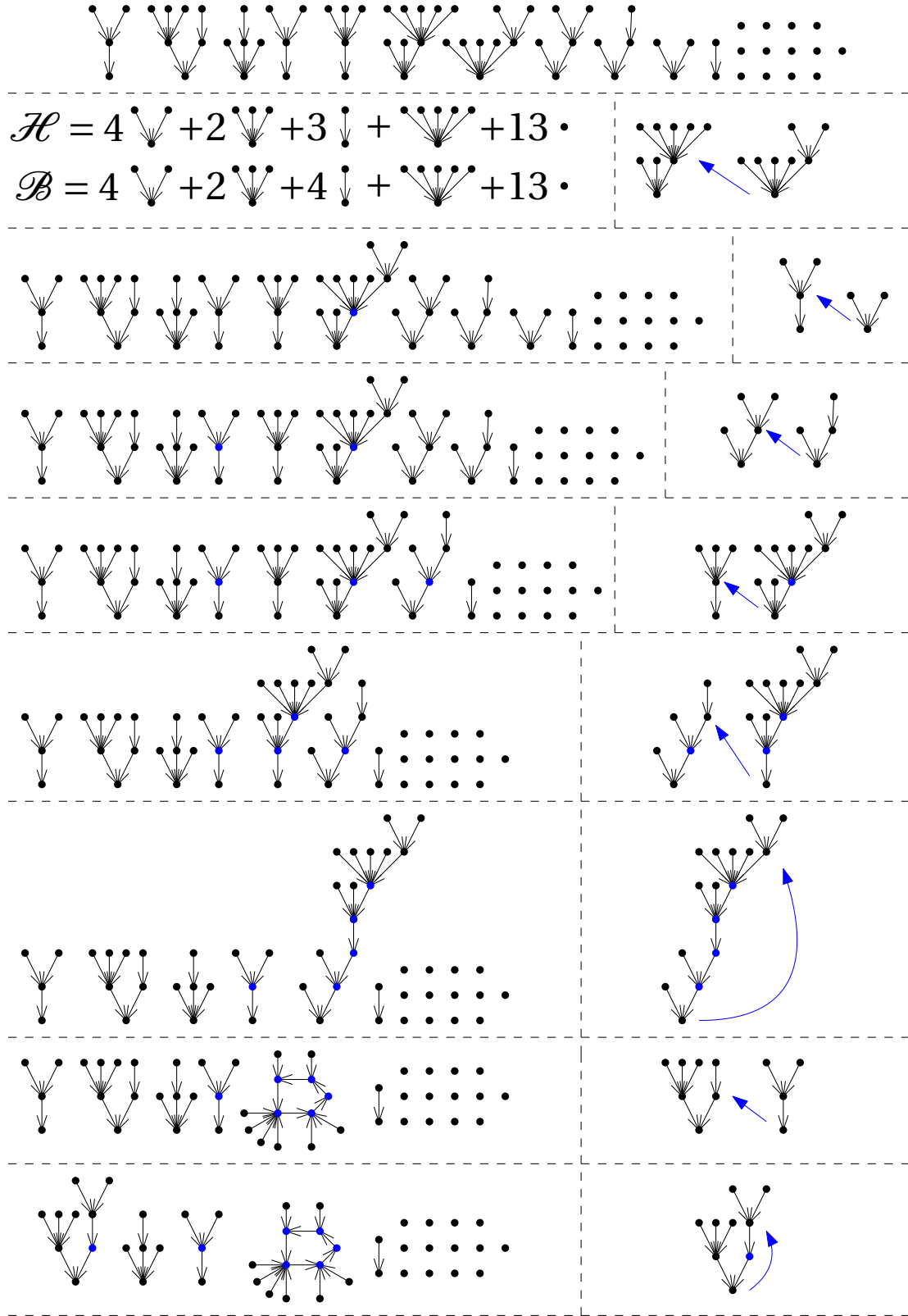
FIGURE 6.36. – Images illustrant un collage. Nous collons l'arbre dont la partie basse est encadrée en rouge sur l'arbre dont la partie haute est encadrée en bleu. Le résultat est l'arbre à droite de la flèche.

calculer deux multi-ensembles, à savoir $\mathcal{H}$ et $\mathcal{B}$, étant au départ respectivement le multi-ensemble des parties hautes de $\mathbf{F}$ (c'est-à-dire $\mathcal{D}(\mathbf{F})$) et celui des parties basses de $\mathbf{F}$ (c'est-à-dire $\mathcal{C}_{d-1}(\mathbf{F})$). Après quoi, il vérifie que $\mathcal{H}$ est un sous-multi-ensemble de $\mathcal{B}$ en accord avec le Lemme 6.32. Puis il affecte une variable X valant originellement $\mathbf{F}$. Après quoi, nous entrons dans le cœur de la procédure. Il prendra un élément h de $\mathcal{H}$ et un élément b de $\mathcal{B}$ tel que $h = b$. Puis il sélectionnera un arbre $\mathbf{t}$ de X ayant partie basse b et une composante connexe C de X ayant partie haute h . Il comblera h par b , produisant une composante connexe C' et retirera h de $\mathcal{H}$, b de $\mathcal{B}$ et $\mathbf{t}$ et C de X . Il recommencera ces étapes tant que $\mathcal{H}$ n'est pas vide. La Figure 6.37 est un exemple d'exécution de cet algorithme.

Nous montrons que l'Algorithme 6.38 est correct, c'est-à-dire qu'il renvoie un système partiel si et seulement s'il existe un système partiel X tel que $\mathcal{C}_d(\mathcal{U}(X)) = \mathbf{F}$ avec $\mathbf{F}$ la forêt finie donnée en entrée et d la profondeur de $\mathbf{F}$. Pour cela, observons qu'il est suffisant de prouver que l'algorithme termine et que le système partiel X renvoyé vérifie bien la condition attendue, c'est-à-dire que $\mathcal{C}_d(\mathcal{U}(X)) = \mathbf{F}$. Discutons premièrement de la terminaison.

Observons qu'une simple récurrence permet de montrer qu'à la i -ème itération de la boucle des lignes 5–11 de l'Algorithme 6.38, le multi-ensemble $\mathcal{H}$ contient toutes les parties hautes de la forêt $\mathbf{F}$ d'origine n'ayant pas encore été comblées.

Notons de plus qu'à la i -ème itération de cette boucle, le multi-ensemble $\mathcal{B}$ contient



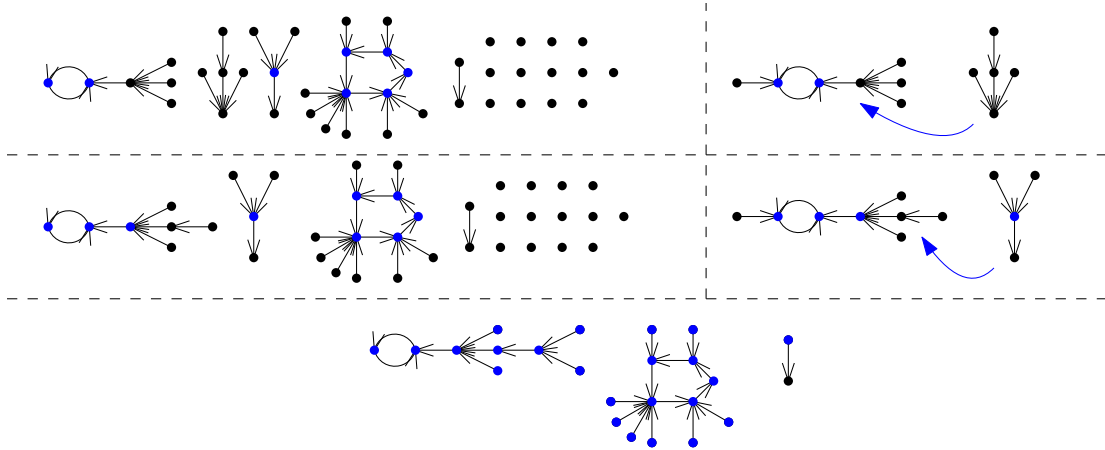


FIGURE 6.37. – Exemple d'exécution de l'Algorithme 6.38 sur la forêt donnée en haut de la page. Le résultat est le partiel en bas de page. Chaque itération de l'algorithme est séparée par une ligne en pointillés. Les sommets en bleu sont les sommets ayant été collés au cours des différentes itérations. Pour passer de l'avant-dernière ligne au résultat final, nous avons simplement collé toutes les feuilles.

uniquement la troncature des arbres de $\mathcal{F}(X)$ à profondeur $d - 1$ avec X le résultat courant puisque nous retirons toujours la partie basse que nous avons collée et que seuls les arbres ont une partie basse.

De par ces réflexions, nous déduisons qu'une preuve par récurrence permet de démontrer qu'après chaque itération de la boucle $\mathcal{H}$ est un sous-multi-ensemble de $\mathcal{B}$ et de plus $\mathcal{B} = \mathcal{C}_{d-1}(\mathcal{F}(X))$ avec X le système partiel en cours de création. En d'autres mots, à la fin de chaque itération, nous pouvons coller un arbre de $\mathcal{F}(X)$ sur chaque élément de $\mathcal{H}$.

Nous en concluons que nous sommes à même de réaliser chaque opération de la boucle de l'Algorithme 6.38 et donc ce dernier termine bien. De surcroît, étant donné que chaque opération dans la boucle est réalisable en temps polynomial sur la taille de $\mathbf{F}$ et aussi que son nombre d'itérations est également polynomial sur cette dernière, il suit que l'algorithme a un temps de calcul polynomial sur la taille de son entrée.

Ainsi, de ce qui précède pour démontrer le Théorème 6.39, il nous suffit d'établir que la condition $\mathcal{C}_d(\mathcal{U}(X)) = \mathbf{F}$ est bien respectée. Ceci est le propos du Théorème 6.40.

Théorème 6.39. *L'Algorithme 6.38 est exécutable en temps polynomial en fonction de la taille de ses entrées. Ainsi, résoudre $P(X) = \mathbf{B}$ est en temps polynomial sur la somme des tailles des $\mathbf{A}_i$ et $\mathbf{B}$ avec $P = \sum_{i=1}^m \mathbf{A}_i \times X^i$ un polynôme sur les forêts finies et $\mathbf{B}$ une forêt finie.*

Théorème 6.40. *Soit $d \geq 1$ un entier et soit $\mathbf{F}$ une forêt finie de profondeur au plus d . Alors, il existe un système partiel X tel que $\mathcal{C}_d(\mathcal{U}(X)) = \mathbf{F}$ si et seulement si $\mathcal{D}(\mathbf{F})$ est un sous-multi-ensemble de $\mathcal{C}_{d-1}(\mathbf{F})$.*

Démonstration. Soit X le système partiel résultant de l'exécution de l'Algorithme 6.38

Algorithme 6.38 EnroulementPartiel($\mathbf{F}$)

```

1: soit  $\mathcal{H} = \mathcal{D}(\mathbf{F})$  et  $\mathcal{B} = \mathcal{C}_{d-1}(\mathbf{F})$ ,
2: Si  $\mathcal{H}$  n'est pas un sous-multi-ensemble de  $\mathcal{B}$  alors
3:   renvoyer  $\perp$ ,
4:  $X \leftarrow \mathbf{F}$ ,
5: Tant que  $\mathcal{H}$  n'est pas vide faire
6:   soit  $h$  un élément de  $\mathcal{H}$  et  $C$  sa composante connexe ayant  $h$  comme partie
   haute,
7:   retirer  $h$  de  $\mathcal{H}$  et  $C$  de  $X$ ,
8:   soit  $b$  un élément de  $\mathcal{B}$  égal à  $h$  et  $\mathbf{t}$  un arbre fini l'ayant comme partie basse,
9:   retirer  $b$  de  $\mathcal{B}$  et  $\mathbf{t}$  de  $X$ ,
10:  soit  $C'$  la composante connexe issue du collage de  $\mathbf{t}$  sur  $C$  par  $h$ ,
11:  ajouter  $C'$  à  $X$ ,
12: renvoyer  $X$ .
```

sur $\mathbf{F}$. La construction de X induit que l'ensemble de ses sommets correspond à l'ensemble des racines des arbres de $\mathbf{F}$. En effet, un sommet de X est soit une racine et donc aussi une racine de $\mathbf{F}$, soit la racine d'un arbre et d'une partie basse de $\mathbf{F}$ ayant été collée sur une partie haute de $\mathbf{F}$. Par souci de brièveté, ces racines sont dites collées dans X .

Nous définissons $\mathbf{S}_i(X)$ comme la somme des arbres contenus dans $\mathcal{F}(X)$ à la i -ème itération avec les arbres de $\mathcal{U}(X)$ dont la racine a été collée dans X au cours de cette même itération ou d'une itération antérieure. Nous démontrons par récurrence sur le nombre d'itérations que $\mathcal{C}_d(\mathbf{S}_i(X)) = \mathbf{F}$.

Si aucun collage n'a été réalisé (cas $i = 0$), alors $\mathbf{S}_i(X) = \mathbf{F}$ car la valeur de $\mathcal{F}(X)$ à la 0-ème itération est $\mathbf{F}$. Soit $i \geq 0$ un entier. Supposons que la propriété soit vraie pour i . Montrons qu'elle est vraie pour $i + 1$. Soient C la composante connexe et $\mathbf{t}$ l'arbre comme définis respectivement aux lignes 7 et 8 de l'Algorithme 6.38. Soit t la racine de $\mathbf{t}$ et soit s le sommet de C dans lequel nous enracinons $\mathbf{t}$, c'est-à-dire le sommet s_2 de la Définition 6.34. Remarquons que l'ensemble des sommets de $\mathcal{C}_d(\mathbf{S}_{i+1}(X))$ ne contenant aucun sommet de la forme (s, j) avec $0 \leq j \leq d$ reste inchangé entre $\mathbf{S}_i(X)$ et $\mathbf{S}_{i+1}(X)$. En effet, la modification leur est invisible étant donné que soit le sommet s n'est pas dans leur composante connexe, soit la distance entre s et eux est strictement supérieure à d .

De plus, par définition du collage, nous avons que la partie basse de $\mathbf{t}$ est égale à la partie haute de C que nous avons retirée. Donc, l'arbre d'unroll $\mathbf{s}$ enraciné dans $(s, 0)$ et tronqué à profondeur d est isomorphe entre les deux itérations, induisant que c'est aussi le cas des arbres $\mathcal{C}_j(\mathbf{s})$ et ce pour tout entier j entre 0 et d . La propriété est donc héréditaire. $\square$

Si l'Algorithme 6.38 permet de résoudre, en temps polynomial sur la taille des entrées, les équations de la forme $\sum_{i=0}^m \mathbf{A}_i X^i = \mathbf{B}$ dans $(\mathbb{P}, +, \times)$, il ne le permet cependant pas si nous nous restreignons au pseudo-semi-anneau $(\mathbb{F}_f, +, \times)$. En effet, certaines

forêts finies peuvent être l'unroll de système partiel d'un système partiel avec des cycles sans pour autant être celles d'une forêt.

Exemple 6.41. *Considérons la forêt constituée uniquement d'un chemin de profondeur au moins 1. Alors cette forêt peut être enroulée en un point fixe. Cependant, il ne s'agit pas de l'unroll d'un arbre car l'unroll d'un arbre de profondeur au moins 1 contient au moins deux composantes connexes.*

Nous allons donc maintenant affiner notre approche pour ne chercher que des solutions étant des forêts finies.

Pour couvrir ce cas, nous construisons un graphe permettant de modéliser les différents collages possibles. En d'autres mots, étant donné une forêt F , nous construisons un graphe dont chaque sommet représente un arbre t de F et qui est composé d'un port bas, étiqueté par la partie basse de t , et de plusieurs ports hauts, étiquetés par les parties hautes de t . Voir Figure 6.43 pour un exemple. Nous ajoutons des arcs entre un port bas et un port haut si et seulement s'ils ont la même étiquette. Ce graphe est appelé *graphe de collage* de F . Remarquons que dans ce graphe, certains sommets n'ont pas de port haut. En d'autres mots, ils sont les “feuilles” de ce graphe. Notre objectif est maintenant de démontrer le théorème suivant.

Théorème 6.42. *Soit F une forêt finie de profondeur d . Et soient $\mathcal{H}$ et $\mathcal{B}$ les multi-ensembles des parties respectivement hautes et basses de F . Alors, il existe une forêt finie X telle que $\mathcal{C}_d(\mathcal{U}(X)) = F$ si et seulement si les trois conditions suivantes sont vérifiées :*

1. $\mathcal{H}$ est un sous-multi-ensemble de $\mathcal{B}$,
2. il existe $b_1, \dots, b_n$ des éléments de $\mathcal{B}$ ayant une multiplicité strictement plus grande dans $\mathcal{B}$ que dans $\mathcal{H}$,
3. pour tout sommet u du graphe de collage de F , il existe un chemin vers un sommet v ayant pour étiquette de port bas l'un des b_i .

Pour démontrer ce théorème, nous introduisons la notion de *plan de collage* d'un graphe. Un plan de collage d'un graphe de collage est un sous-graphe contenant tous les sommets du graphe de collage et dont chaque port haut a exactement un arc entrant et chaque port bas a au plus un arc sortant. Nous montrerons que chaque solution possible de l'Algorithme 6.38 décrit un plan de collage et inversement. Un exemple est donné dans la Figure 6.43.

Lemme 6.44. *Soit F une forêt de profondeur d . Alors, il existe X un système partiel tel que $\mathcal{C}_d(\mathcal{U}(X)) = F$ si et seulement si le graphe de collage de F admet un plan de collage. De plus, X est une forêt si et seulement si ce plan de collage est acyclique. Finalement, nous pouvons passer d'une représentation d'une solution à l'autre et vice versa en temps polynomial.*

Démonstration. $(\Rightarrow)$ Soit X un système partiel tel que $\mathcal{C}_d(\mathcal{U}(X)) = F$. Nous expliquons comment construire un plan de collage à partir de X . Considérons t un arbre de $\mathcal{U}(X)$ de racine $(u, 0)$. Nous créons pour ces arbres un sommet ayant une étiquette

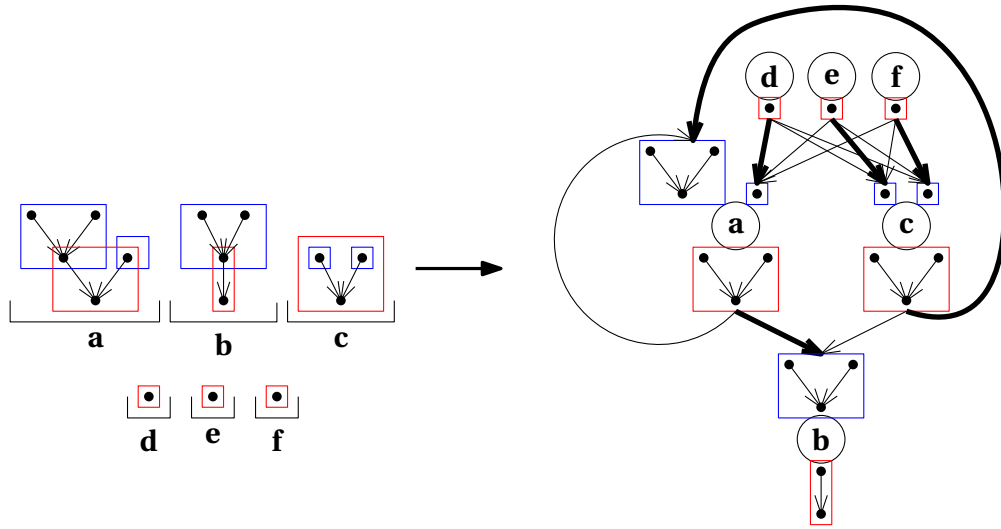


FIGURE 6.43. – Exemple de construction d'un graphe de collage. Nous partons des arbres à gauche de la flèche centrale. Chaque arbre encadré de bleu est une partie haute et ceux encadrés de rouge sont les parties basses. Chaque sommet du graphe de droite modélise l'arbre dont le nom est le centre du cercle. Les arcs en gras décrivent un plan de collage.

de port bas $\mathcal{C}_{d-1}(\mathbf{t})$, nous disons que cette étiquette a pour racine $(u, 0)$. De plus, ce sommet aura un port haut par arbre $\mathbf{t}'$, de racine $(v, 1)$ de $\mathcal{D}(\mathbf{t})$ qui aura comme étiquette $\mathcal{C}_{d-1}(\mathbf{t}')$ et cette étiquette aura pour racine $(v, 0)$. De ceci, nous déduisons que le graphe de collage de $\mathbf{F}$ admet un plan de collage.

Considérons le graphe ayant l'ensemble de sommets construit comme décrit ci-dessus à partir des arbres de $\mathcal{U}(X)$. En outre, nous ajoutons un arc entre un port bas et un port haut si et seulement si leurs étiquettes ont la même racine. De ce fait, la définition de l'unroll de système partiel induit que chaque port haut a degré entrant exactement 1 et que chaque port bas a degré sortant au plus 1. De plus, si X est une forêt, alors ce graphe est aussi acyclique. En outre, les arbres de l'étiquette des deux extrémités de chaque arc sont isomorphes. Donc, nous prenons l'étiquette de chaque port à isomorphisme près; le graphe que nous avons créé est un plan de collage.

Notons que les opérations décrites ci-dessus sont réalisables en temps polynomial et que le temps de création du graphe de collage de $\mathbf{F}$ est polynomial sur la taille de $\mathbf{F}$. En effet, le coût de création de chaque sommet du graphe de collage de $\mathbf{F}$ est polynomial. Et le nombre de ports (hauts et bas confondus) est borné par le nombre de sommets de $\mathbf{F}$. Donc, comme vérifier que deux étiquettes sont égales est réalisable en temps polynomial, ajouter tous les arcs du graphe de collage est aussi polynomial.

($\Leftarrow$) Supposons maintenant que le graphe de collage de $\mathbf{F}$ admet un plan de collage. Nous pouvons appliquer l'opération inverse de celle décrite ci-dessus pour former un arbre à partir d'un sommet d'un plan de collage. Ainsi, si nous appliquons les collages décrits dans le plan, c'est-à-dire que nous collons une partie basse d'un arbre sur une partie haute si et seulement si leurs ports associés sont reliés par un arc,

un raisonnement similaire à celui de la preuve du Théorème 6.40 implique que le système partiel X que nous créons est tel que $\mathcal{C}_d(\mathcal{U}(X)) = \mathbf{F}$. De surcroît, si ce plan est acyclique, X l'est également, car nous ne réalisons aucun collage créant un cycle. Enfin, les collages sont réalisables en temps polynomial. $\square$

En nous reposant sur le Lemme 6.44, nous déduisons que pour trouver en temps polynomial une forêt $\mathbf{X}$ telle que $\mathcal{C}_d(\mathcal{U}(\mathbf{X})) = \mathbf{F}$ avec $\mathbf{F}$ une forêt de profondeur d , il suffit de rechercher un plan de collage du graphe de collage de $\mathbf{F}$ et de réaliser les collages qu'il décrit. En d'autres mots, nous avons une réduction fonctionnelle en temps polynomial entre les deux problèmes. De plus, ce même lemme induit que pour montrer que s'il existe $\mathbf{X}$ tel que $\mathcal{C}_d(\mathcal{U}(\mathbf{X})) = \mathbf{F}$ avec $\mathbf{F}$ une forêt de profondeur d , alors les trois conditions du Théorème 6.42 sont vérifiées.

Lemme 6.45. *Soit $\mathbf{F}$ une forêt finie de profondeur d . Et soient $\mathcal{H}$ et $\mathcal{B}$ les multi-ensembles des parties respectivement hautes et basses de $\mathbf{F}$. S'il existe $\mathbf{X}$ tel que $\mathcal{C}_d(\mathcal{U}(\mathbf{X})) = \mathbf{F}$, alors les trois conditions du Théorème 6.42 sont vérifiées.*

Démonstration. Supposons qu'il existe $\mathbf{X}$ tel que $\mathcal{C}_d(\mathcal{U}(\mathbf{X})) = \mathbf{F}$. Alors, le Lemme 6.44 induit que le graphe de collage de $\mathbf{F}$ admet un plan de collage. Nous allons montrer les trois conditions dans l'ordre.

1. Par définition du plan de collage, chaque port haut a comme prédécesseur un unique port bas ayant la même étiquette et ce prédécesseur n'admet qu'un unique successeur. Ceci induit que les étiquettes des ports hauts (soit les parties hautes de $\mathbf{F}$) sont un sous-multi-ensemble de celles des ports bas (soit les parties basses de $\mathbf{F}$).
2. Aussi, comme un plan de collage est acyclique et qu'un port bas n'a qu'un port haut comme successeur, il suit que certains ports bas (donc parties basses) n'ont pas de successeur. Soit b une telle étiquette. Alors, la multiplicité de b dans le multi-ensemble des parties basses de $\mathbf{F}$ est strictement supérieure à celle dans le multi-ensemble des parties hautes de $\mathbf{F}$.
3. Du point précédent, les étiquettes des ports bas des racines des arbres du plan de collage ont une multiplicité dans le multi-ensemble des parties basses de $\mathbf{F}$ qui est strictement supérieure à celle dans le multi-ensemble des parties hautes de $\mathbf{F}$. De là, comme chaque sommet du plan de collage admet un chemin vers des racines de ce plan et que le plan de collage est un sous-graphe du graphe de collage avec le même ensemble de sommets, il suit que chaque sommet u du graphe de collage admet un chemin vers l'une des racines du plan de collage. $\square$

Soit $\mathbf{F}$ une forêt finie de profondeur d . Soient $\mathcal{H}$ et $\mathcal{B}$ les multi-ensembles des parties respectivement hautes et basses de $\mathbf{F}$. Pour montrer la direction inverse, c'est-à-dire que si les trois conditions du Théorème 6.42 sont vérifiées, alors il existe une forêt $\mathbf{X}$ telle que $\mathcal{C}_d(\mathcal{U}(X)) = \mathbf{F}$, nous donnons un algorithme qui construit un plan de collage acyclique si les trois conditions sont vérifiées.

6. Les systèmes partiels – 6.2. Résolution de $P(X) = B$ pour les systèmes partiels

Notre procédure part de G le graphe de collage de $\mathbf{F}$ et commence par tester que les trois conditions du Théorème 6.42 sont vérifiées. Si ce n'est pas le cas, elle répond qu'aucune solution n'existe. Ceci est correct par le Lemme 6.45.

Si maintenant ces trois conditions sont bien vérifiées, alors l'algorithme construit E l'ensemble des étiquettes de ports bas induit par $\mathcal{B} - \mathcal{H}$ et il applique l'Algorithme 6.38 sur $\mathbf{F}$. Comme la condition 1 est vérifiée, cet appel retourne un système partiel X tel que $\mathcal{C}_d(\mathcal{U}(X)) = \mathbf{F}$. Cependant, le système partiel X peut contenir des cycles, nous devons donc les retirer.

Pour cela, nous introduisons une sous-procédure qui nous permettra d'échanger des arcs de P , le plan de collage induit par X , afin de supprimer ses cycles. Cette dernière prend en entrée P ainsi que s un sommet d'un cycle de P et ρ un chemin simple de G reliant s à un sommet ayant une étiquette de port bas incluse dans E . Elle cherche le sommet u de $\rho \cap C$ étant le plus éloigné de s , avec C le cycle de P contenant s , ainsi que les sommets u', v et v' tels que v et v' sont les successeurs de u dans respectivement ρ et C tandis que u' est un prédécesseur de v par un port haut d'étiquette p , avec p l'étiquette du port bas de u . Elle remplace dans P les arcs de u à v' et de u' à v par des arcs de respectivement u à v et u' à v' . Notons que v peut être indéfini si u est le dernier sommet de ρ , auquel cas u' est un sommet n'ayant pas de successeur dans P et ayant une étiquette de port bas comprise dans E . De là, nous ajoutons simplement un arc de u' à v' et supprimons l'arc sortant de u .

Nous appliquons cette procédure :

1. soit depuis un nouveau sommet s' d'un cycle de P et nouveau chemin ρ' si le nombre de cycles de P a diminué par cet échange,
2. soit depuis le sommet u et le chemin ρ si nous ne sommes pas dans le cas précédent.

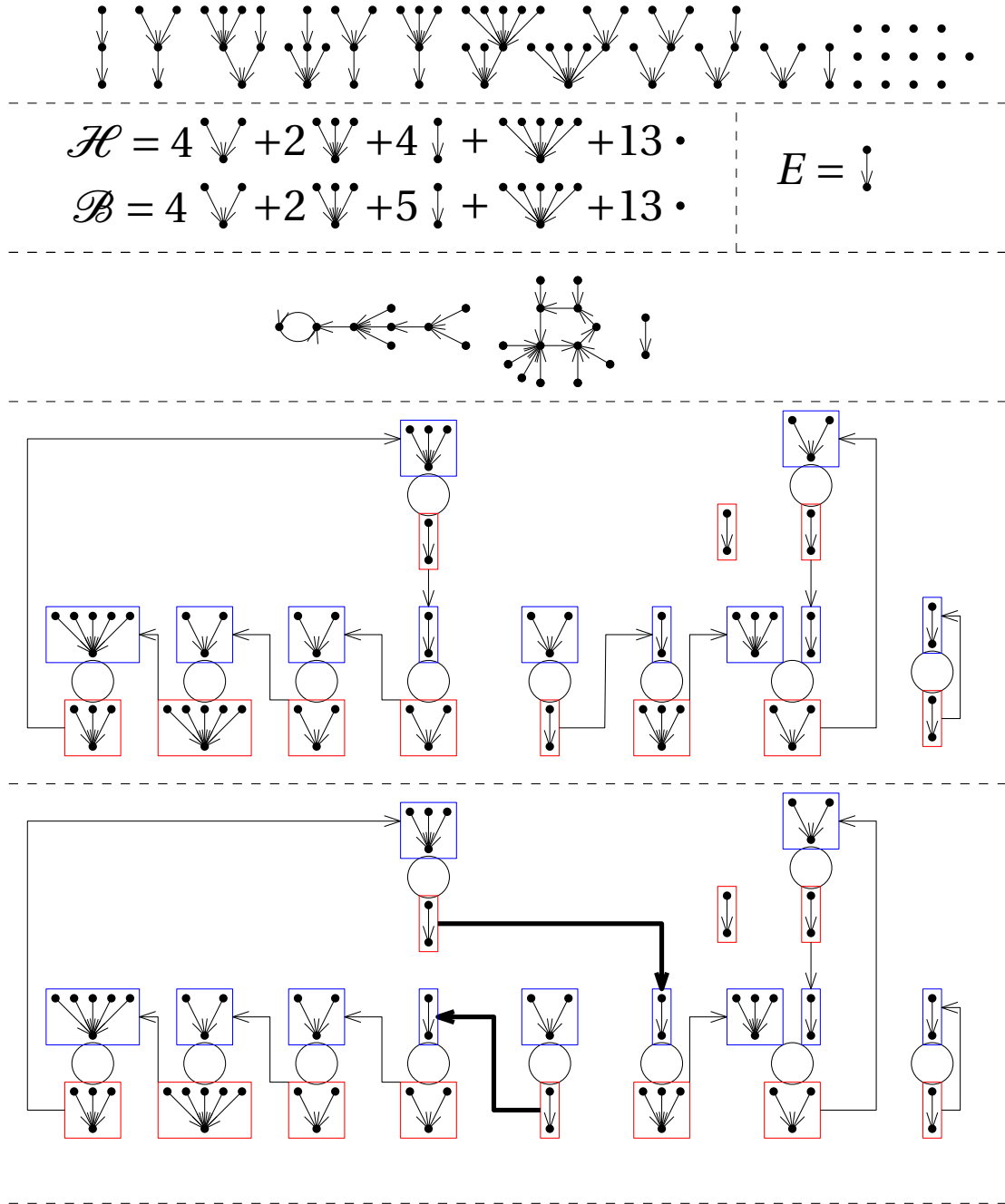
Un exemple d'exécution de cet algorithme est présenté dans la Figure 6.46.

Lemme 6.47. *Soit G le graphe de collage d'une forêt $\mathbf{F}$. Notre algorithme renvoie un plan de collage acyclique de G en temps polynomial si et seulement s'il en existe un.*

Démonstration. ($\Leftarrow$) Supposons que le graphe de collage admet un plan de collage acyclique. Donc, par le Lemme 6.44, il existe une forêt $\mathbf{X}$ telle que $\mathcal{C}_d(\mathcal{U}(\mathbf{X})) = \mathbf{F}$, avec $\mathbf{F}$ la forêt induite par le graphe de collage et d la profondeur de $\mathbf{F}$. Ainsi, le Lemme 6.45 induit que les trois conditions du Théorème 6.42 sont respectées. Montrons maintenant que l'algorithme termine.

Remarquons qu'à chaque étape de la procédure d'échange d'arc, nous ne modifions pas les degrés entrants des sommets v et v' et nous n'augmentons pas les degrés sortants des sommets u et u' . Ainsi, chaque itération de la procédure d'échange produit un plan de collage et nous pouvons bien l'appliquer récursivement comme dans l'algorithme. De là, pour démontrer la terminaison, il nous suffit de prouver qu'il y a un nombre fini d'appels à cette procédure.

Soient P un plan de collage de G , u, u', v, v' ainsi que ρ comme dans la procédure d'échange et soit P' le plan de collage produit à l'issue de la procédure d'échange. Comme l'algorithme se termine lorsque nous avons supprimé tous les cycles de P ,



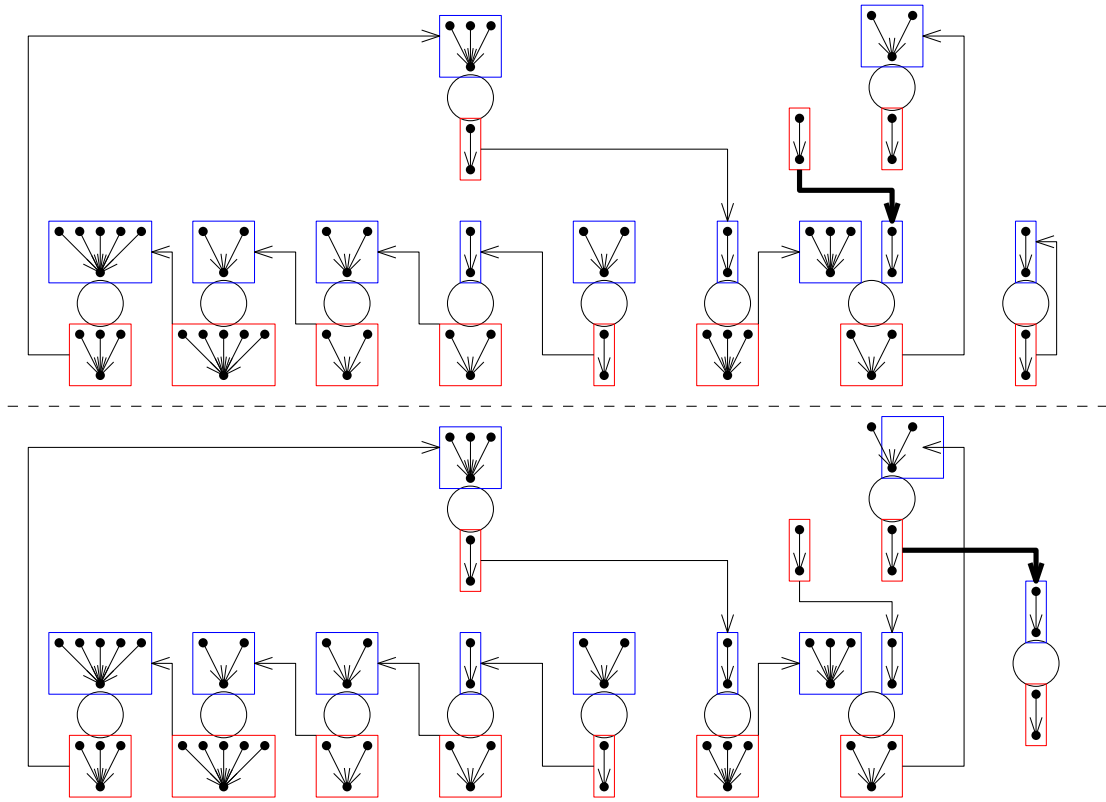


FIGURE 6.46. – Exemple d'exécution de l'algorithme permettant d'enrouler une forêt finie en une nouvelle forêt finie. Le résultat est le plan de collage en bas de page. Chaque étape de l'algorithme est séparée par une ligne en pointillés. Les arcs en gras représentent les modifications réalisées afin d'incorporer les cycles dans des arbres.

montrons en premier lieu que le nombre de cycles n'augmente pas. Pour ce faire, nous prouvons que le résultat de la procédure d'échange est une composante connexe car une composante connexe contient au plus un cycle.

Notons que si v est bien défini alors u et v ne sont pas dans le même cycle par le choix de u . Ceci signifie que chaque chemin simple vers u dans P est aussi présent dans P' , que v soit défini ou non, car nous avons modifié des arcs qui n'appartenaient à aucun de ces chemins. Ainsi, comme v' est le successeur de u dans un cycle de P , il suit que P' contient un chemin simple entre v' et u et aussi entre u' et u car P' contient un arc de u' vers v' . De là, si v n'est pas défini alors l'ensemble des sommets ayant un chemin simple vers u' dans P possède un chemin simple vers u dans P' . Ceci induit que le résultat est bien connexe. Si maintenant v est bien défini, tous les chemins simples vers v de P ne passant pas par l'arc de u' vers v sont aussi présents dans P' . Et comme dans P' il existe un chemin simple de u' vers u et donc aussi de u' vers v , tous les sommets reliés à v dans P par un chemin simple passant par l'arc de u' vers v sont aussi reliés par un chemin simple vers v dans P' . Dans ce cas aussi le résultat est connexe.

Nous prouvons maintenant que l'application successive de la procédure d'échange sur les sommets de ρ entraîne la diminution du nombre de cycles. Pour cela, relevons que trois cas sont possibles en fonction de l'existence du sommet v et de son positionnement par rapport à u .

Le premier cas est que le sommet v est bien défini et qu'il est dans la même composante connexe que u dans P . Ceci signifie qu'il existe un chemin simple de v vers u dans P et aussi dans P' comme justifié ci-dessus. Et puisque P' contient l'arc de u vers v , nous en concluons que P' contient un cycle passant par u et v . Ceci correspond au cas 2 de l'algorithme et nous pouvons appliquer ce raisonnement inductivement soit jusqu'à ce que v ne soit pas dans la même composante connexe que u , soit jusqu'à ce que u soit le dernier sommet de ρ et donc que v ne soit plus défini.

Le second cas est que v est défini et qu'il n'est pas dans la même composante connexe que u dans P . Observons que deux cas sont de nouveau possibles. Soit la composante connexe contenant v contient un cycle. Puisque, comme montré précédemment, le résultat de la procédure d'échange est connexe et comme une composante connexe contient au plus un cycle, il suit que le nombre de cycles a diminué d'un. Soit la composante connexe contenant v ne contient pas de cycle. Observons qu'il n'est pas possible de créer de cycle en supprimant des arcs. Donc, le retrait de l'arc de u' vers v n'a pas créé de cycle tandis que le retrait de celui de u vers v' en a supprimé un. Il nous faut donc montrer que l'ajout des arcs de u' vers v' et de u vers v ne crée pas de cycle. Relevons que puisque la composante connexe contenant v ne contient pas de cycle, le retrait de l'arc de u' vers v a engendré deux composantes connexes disjointes. De là, comme l'ajout d'arcs entre composantes connexes disjointes ne permet pas de créer de cycle, il suit que l'ajout des deux arcs n'en crée pas.

Le troisième et dernier cas est que v n'est pas défini. Remarquons que le multi-ensemble des étiquettes de port bas des sommets de P n'ayant pas de successeur correspond exactement à $\mathcal{B} - \mathcal{H}$. Donc, le sommet u' défini dans la procédure d'échange d'arc existe bien. De là, en utilisant le même type de raisonnement que pour le cas précédent, il suit que le nombre de cycles diminue d'un.

Nous en concluons qu'à partir d'une composante connexe cyclique D de P , l'algorithme produira un plan de collage P' contenant un cycle de moins que P après au plus $|\rho| \leq |G|$ itérations. Donc, comme après une diminution du nombre de cycles, nous appliquons le cas 1 de l'algorithme qui repart d'une nouvelle composante connexe cyclique, nous pouvons appliquer ce raisonnement par récurrence et donc l'algorithme se termine après au plus $|G|^2$ itérations et produit un plan de collage acyclique. En effet, nous devons considérer au plus $|G|$ chemins simples de taille au plus $|G|$.

($\Rightarrow$) Supposons que notre algorithme renvoie un graphe. Cela implique alors qu'il a terminé. Donc, par le point précédent, il a produit un plan de collage acyclique. $\square$

Les Lemmes 6.47, 6.44 et 6.45 nous permettent de conclure la preuve du Théorème 6.42. En effet, si les trois conditions du Théorème 6.42 sont vérifiées, alors notre algorithme renvoie un graphe qui, par le Lemme 6.47, est un plan de collage acyclique. Pour l'autre direction, s'il existe un plan de collage acyclique, les Lemmes 6.44 et 6.45

impliquent que les trois conditions du Théorème 6.42 sont vérifiées. Nous concluons donc le dernier résultat de cette thèse.

Théorème 6.48. *Soit $\mathbf{F}$ une forêt finie de profondeur d . Si cela est possible, nous construisons une forêt $\mathbf{X}$ telle que $\mathcal{C}_d(\mathcal{L}(\mathbf{X})) = \mathbf{F}$ en temps polynomial.*

6.3. Conclusion et perspectives

Dans ce chapitre, nous avons introduit un nouvel objet, à savoir les systèmes partiels qui, munis de l'union disjointe comme somme et du produit direct comme multiplication, forment un sur-semi-anneau de celui des FDDS. Nous avons montré que ce nouveau semi-anneau possède exactement un premier et que ce premier est réductible. Nous avons de plus démontré que la caractérisation des polynômes injectifs sur les FDDS s'applique également aux polynômes sur les systèmes partiels. Par la suite, nous avons prouvé que la résolution de $P(X) = B$ avec P un polynôme sur les systèmes partiels et B un système partiel se réduit en temps polynomial à la résolution de $P'(X) = B'$ avec P' un polynôme sur les FDDS et B' un FDDS. Nous avons aussi établi que nous sommes en mesure de résoudre en temps polynomial une équation de la forme $P(X) = \mathbf{B}$ avec P un polynôme sur les forêts finies et $\mathbf{B}$ une forêt finie. Plus précisément, nous avons établi que nous pouvons tout aussi bien reconstruire en temps polynomial soit un système partiel, soit une forêt finie résolvant l'équation.

Ces résultats mettent en lumière plusieurs problèmes que nous souhaiterions étudier. Tout d'abord, nous voudrions pouvoir passer de la résolution de $P(X) = \mathbf{B}$ avec P un polynôme sur les forêts finies et $\mathbf{B}$ une forêt finie à l'énumération de l'ensemble des solutions de cette équation. Et nous nous demandons si cela est réalisable avec un délai polynomial. Nous nous posons la même question, mais en ajoutant la contrainte que les solutions doivent être des forêts finies. Nous voudrions également étudier le même problème mais avec la contrainte inverse, c'est-à-dire que la solution trouvée doit être un FDDS.

Un autre aspect que nous souhaiterions étudier est le passage d'une équation à un système d'équations avec toujours une variable ainsi que le cas où le nombre de variables serait borné tout autant pour une équation que pour un système d'équations.

Conclusion

Cette thèse s'est inscrite dans l'étude algébrique et algorithmique des systèmes dynamiques finis déterministes. Elle est divisée en trois grandes parties. La première était celle où nous avons rappelé le contexte mathématique et l'état de l'art du domaine. La deuxième était la résolution des équations de la forme $P(X) = B$. Finalement, la troisième était la généralisation des résultats dans de nouveaux contextes (sur- et sous-approximation, systèmes partiels) pouvant être empiriquement interprétée comme une erreur ou un manque d'information lors de la modélisation du FDDS.

Résumé des contributions

Le premier chapitre a posé le cadre formel du manuscrit. Nous y avons défini les FDDS, établi leur relation avec les graphes fonctionnels, et introduit les opérations de somme et de produit faisant de $(\mathbb{D}, +, \times)$ un semi-anneau commutatif. Nous avons ensuite présenté l'état de l'art relatif à la résolution des équations polynomiales et introduit la notion d'unroll qui constitue l'outil central du chapitre suivant.

Le deuxième chapitre a présenté l'une des deux contributions majeures du manuscrit. Nous y avons développé un algorithme polynomial permettant, étant donné un polynôme $P = \sum_{i=0}^m A_i X^i$ sur les FDDS et un FDDS B , de résoudre l'équation sur les unrolls $\sum_{i=0}^m \mathcal{U}(A_i) \mathcal{U}(X)^i = \mathcal{U}(B)$. Ce résultat repose sur une série d'algorithmes allant de la résolution de $\mathcal{U}(A) \mathcal{U}(X) = \mathcal{U}(B)$ pour un X connexe jusqu'à la résolution de $\mathcal{U}(X)^k = \mathcal{U}(B)$. Le principal intérêt de ce résultat est que tout polynôme d'unroll est injectif. De ce fait, il montre que la difficulté de la résolution de $P(X) = B$ ne repose pas sur le fait de trouver les parties transitoires, mais plutôt sur quelle période nous devons enrouler ces parties.

Le troisième chapitre a présenté la seconde contribution importante de ce manuscrit. Il a approfondi la résolution directe de $P(X) = B$ en s'intéressant aux polynômes injectifs et pseudo-injectifs. Nous avons donné une caractérisation complète des polynômes injectifs sur $\mathbb{D}$: un polynôme P est injectif si et seulement si au moins l'un de ses coefficients non-constants contient un dendron. Au cours de la preuve de cette caractérisation, nous avons développé un algorithme polynomial pour résoudre $P(X) = B$ lorsque P est injectif, en reconstruisant les composantes connexes de X une par une grâce aux résultats du chapitre précédent en respectant l'ordre $\leq_{ct}$. Ces résultats ont ensuite été étendus à la classe plus large des polynômes pseudo-injectifs. Pour cela, nous avons introduit un nouvel ordre, à savoir $\leq_P$, nous permettant de développer un paradigme susceptible de résoudre $P(X) = B$. Ce paradigme est le principal apport de ce chapitre.

Le quatrième chapitre a porté sur les équations polynomiales sur les supports

de FDDS. Nous avons montré que décider de l'existence d'une solution est dans la classe NP et est même NP-complet si le polynôme est non-linéaire. En revanche, si le polynôme est linéaire, le problème devient polynomial. Pour montrer cela, nous avons introduit une solution canonique calculable efficacement, depuis laquelle l'ensemble des solutions peut être énuméré avec un délai polynomial. Ceci illustre une différence majeure entre les $\mathbb{D}$ et leurs supports. En effet, sous l'hypothèse que P est différent de NP, alors étant donné un polynôme linéaire P , nous ne pourrions décider en temps polynomial si $P(X_1, \dots, X_n) = B$, mais nous pourrions énumérer l'ensemble des supports possibles des valeurs des X_i avec un délai polynomial. De plus, une borne sur le nombre de composantes connexes dans les solutions a permis d'établir que la résolution des équations linéaires à côté droit constant sur les FDDS appartient à la classe FPT, par réduction à la programmation linéaire en nombres entiers à nombre fixé de variables.

Le cinquième chapitre a élargi le champ d'investigation aux inéquations polynomiales. Nous avons montré qu'il est possible d'énumérer, avec un délai polynomial en la taille de B , l'ensemble des solutions de $P(X) \leq B$. Ce résultat repose sur la notion de fusion de composantes connexes et sur la caractérisation des solutions fusion-free. Il a été généralisé à certaines familles d'inéquations multivariées. Dans le sens inverse, nous avons montré que les inéquations de la forme $P(X) \geq B$ sont résolubles en temps polynomial, résultat lui aussi généralisé au cas multivarié, mais sans contrainte sur les polynômes.

Le sixième et dernier chapitre a proposé une extension de l'ensemble des résultats précédents au sur-semi-anneau des systèmes dynamiques partiels. Nous avons énoncé des propriétés algébriques de ce nouveau semi-anneau et montré que la caractérisation des polynômes injectifs sur $\mathbb{D}$ s'étend aux systèmes partiels. La contribution principale est une réduction polynomiale de la résolution de $P(X) = B$ sur les systèmes partiels depuis celle sur les FDDS, permettant d'hériter directement de tous les algorithmes développés dans les chapitres précédents. Un récapitulatif de l'état actuel des connaissances de notre domaine est présenté dans le Tableau 6.1.

Problèmes ouverts

Plusieurs questions importantes restent ouvertes. Nous les organisons ici en deux parties, qui traversent la quasi-totalité des chapitres. Tous les algorithmes développés dans ce manuscrit résolvent des équations à une variable. Nous voudrions savoir si nous pouvons les adapter au cas multivarié ou aux systèmes d'équations.

Problèmes sur les unrolls

Pour les équations sur les unrolls, nous savons que décider si $P(X_1, \dots, X_n) = \mathcal{U}(B)$ admet une solution est dans NP, mais nous ne savons pas encore si ce problème est NP-complet ou bien polynomial. La difficulté vient du fait que nous ne connaissons aucune méthode efficace pour factoriser les unrolls et que, dans le cas où le polynôme serait dissocié, nous ne connaissons pas de technique permettant de répartir

les valeurs des unrolls que nous calculons au sein des évaluations des différentes variables.

Indépendamment de la question du passage au cas multivarié, la complexité temporelle fine de la division d'unrolls dans le cas univarié reste également à préciser. Nous ne savons pas si décider si $\mathcal{U}(A)$ divise $\mathcal{U}(B)$ est complet pour la classe P. Nous avons des raisons de croire que ce problème pourrait même appartenir à la classe LogSpace : en effet, Lindell a démontré dans [33] qu'il nous est possible de comparer deux arbres en espace logarithmique, et il est envisageable que son algorithme puisse être adapté pour comparer deux arbres avec notre ordre puis pour diviser deux arbres d'unroll en espace logarithmique. Cependant, cela nous paraît difficile, car nous ne voyons pas comment réaliser la division sans stocker l'arbre quotient. Pour la P-complétude, nous rencontrons des difficultés à trouver des problèmes P-difficiles depuis lesquels nous pourrions effectuer une réduction.

Cette question de P-difficulté se généralise naturellement à la division de deux FDDS, problème qui reste ouvert et qui semble étroitement corrélé à la division d'unrolls, car diviser deux FDDS nécessite de diviser leurs unrolls. Ce lien entre division de FDDS et division d'unrolls reste cependant encore purement intuitif. En effet, nous n'avons pour l'heure pas prouvé qu'il existe une réduction, même polynomiale, de l'un vers l'autre. Une telle réduction nécessiterait de construire, à partir de A et B , des FDDS A' et B' tels que $\mathcal{U}(A') = \mathcal{U}(A)$, $\mathcal{U}(B') = \mathcal{U}(B)$, et A' divise B' si et seulement si $\mathcal{U}(A)$ divise $\mathcal{U}(B)$. Et nous rencontrons des difficultés à construire ces A' et B' , car un arbre de $\mathcal{U}(B)$ peut être issu du produit d'arbres différents de $\mathcal{U}(A)$, mais aussi avoir une période inférieure à celle de ces diviseurs. Donc, les deux formes canoniques de FDDS que nous avons introduites ne semblent pas permettre de construire un B' correct.

Problèmes sur les équations de FDDS

Résoudre efficacement les systèmes d'équations sur les FDDS avec membre droit constant, mais où tous les polynômes seraient pseudo-injectifs permettrait d'améliorer le pipeline de [14, 46] et conduirait à un algorithme polynomial pour résoudre $P(X_1, \dots, X_n) = B$ avec P un polynôme dissocié si le nombre de composantes connexes de B était fixé. La résolution de ces systèmes n'est pas directe même si nous savons résoudre chaque équation de ce système en temps polynomial et que nous pouvons efficacement générer de nouvelles solutions à chaque équation. En effet, il nous faut parvenir à trouver efficacement l'intersection de l'ensemble des solutions possibles de chacune des équations du système.

Une autre piste serait d'étudier les équations de la forme $P(X) = Q(X)$ avec P et Q des polynômes pseudo-injectifs, voire des polynômes de FDDS quelconques. Si ce problème est décidable, car l'équation sur les tailles associée admet un nombre fini de solutions, il est aussi dans NP, car la majoration de Lagrange implique que toutes les solutions ont une taille polynomiale sur les tailles des coefficients des polynômes P et Q . Ainsi, toutes les solutions possibles ont une taille polynomiale, et vérifier qu'un FDDS est solution de $P(X) = Q(X)$ prend un temps polynomial. Nous ne

savons cependant pas si ce problème est NP-complet ou bien soluble efficacement. En effet, il hérite de l'ensemble des difficultés rencontrées pour résoudre efficacement $P(X) = B$ (car $Q(X)$ peut être constant), et nous rencontrons des difficultés à trouver un problème NP-complet depuis lequel réduire.

Dans une direction différente, nous pouvons nous demander si la résolution de $P(X) = B$ peut se réduire à celle de $Q(X) = B'$, avec Q le polynôme construit en ne conservant que les cycles des composantes connexes des coefficients de P et B' la permutation induite par B . Plus précisément, nous voudrions savoir s'il est possible de construire une solution de $P(X) = B$ à partir d'une solution de $Q(X) = B'$ en temps polynomial. La difficulté de cette approche est que $Q(X) = B'$ peut admettre une solution sans que $P(X) = B$ en admette une, et nous ne savons pas dire comment nous pourrions contourner cet obstacle.

La résolution générale de $P(X) = B$, sans hypothèse sur le polynôme P , est aussi encore ouverte. Comme il semble ardu d'y répondre, nous envisageons deux pistes intermédiaires. La première est d'identifier des paramètres permettant des résultats FPT, comme le nombre de graines de P , la plus grande de ces graines ou simplement le nombre de composantes connexes non-isomorphes de B . En effet, nous pensons pouvoir montrer que les résultats du Chapitre 4 pour $AX = B$ peuvent s'étendre si nous fixons tous ces paramètres. Le problème vient du fait que nous ne savons pas si nous pouvons résoudre efficacement l'équation sur les supports associée à P et B si ces paramètres sont fixés. La seconde piste serait de développer et de rechercher un algorithme d'approximation avec facteur constant ou même un schéma d'approximation polynomial, pour une notion adaptée d'approximation. Tout ceci pourrait nous permettre de mieux appréhender les difficultés intrinsèques de la résolution de $P(X) = B$.

Enfin, nous pouvons étudier la résolution de $P(X) = B$ où les FDDS sont codés de manière succincte, par exemple par un circuit booléen. Nous pensons pouvoir prouver que, dans ce cas, décider de l'existence d'une solution est PSPACE-difficile, car nous croyons être en mesure de démontrer que l'isomorphisme de graphe fonctionnel encodé par des circuits booléens est complet pour la classe PSPACE. Cependant, la question de l'appartenance à PSPACE reste encore pleinement ouverte.

Problèmes sur les inéquations de FDDS

Pour les inéquations avec un côté constant, nous conjecturons que la résolution de systèmes est NP-difficile même à une variable. En effet, nous pensons que nous pouvons encoder monotone-3-SAT dans ces systèmes. Pour cela, nous aurions une inéquation qui fixerait les composantes connexes possibles pouvant être dans X (une inéquation de la forme $X \leq B$) puis une inéquation de la forme $AX \geq B$ pour représenter une clause de conjonction de variables. Cette dernière forme obligerait à avoir au moins l'un des cycles dans X . Et enfin une inéquation de la forme $AX \leq B$ pour les clauses de conjonction de négations de variables. Ces inéquations empêchent d'avoir certaines composantes connexes dans X .

Pour la sous-approximation, nous voudrions également savoir s'il est possible d'énu-

mérer les solutions d'un système d'inéquations polynomiales avec un délai polynomial. Les difficultés pour résoudre ce problème sont les mêmes que celles que nous avons exposées pour la résolution des systèmes d'équations où les polynômes sont pseudo-injectifs.

Pour l'inégalité inverse, nous savons que nous ne pouvons pas énumérer l'ensemble des solutions de $P(X) \geq B$ avec un délai polynomial, car il est possible de construire des solutions de taille arbitrairement grande. C'est pourquoi nous voudrions définir une notion de minimalité et déterminer s'il est possible d'énumérer ces solutions minimales avec un délai polynomial.

Finalement, nous voudrions généraliser nos résultats à d'autres relations d'ordre sur les FDDS. Par exemple, nous pourrions définir que $A \preceq B$ si et seulement si A est un sous-graphe de B , ou encore que A est un mineur de B , ou bien que B admet un homomorphisme surjectif vers A .

Problèmes sur les systèmes partiels

Nous voudrions également pouvoir passer de la résolution efficace de $P(X) = \mathbf{B}$ avec P un polynôme sur les forêts finies et $\mathbf{B}$ une forêt finie à l'énumération de l'ensemble des solutions de cette équation avec un délai polynomial. Nous nous posons la même question en ajoutant la contrainte que les solutions doivent être des forêts finies, ou bien des FDDS. De même, nous souhaiterions étudier le passage d'une équation à un système d'équations à une variable sur les systèmes partiels, ainsi que le cas où le nombre de variables serait borné. Ces questions héritent des difficultés évoquées précédemment pour le cas des FDDS.

Problèmes d'ouverture

Enfin, nous mentionnons quatre questions qui dépassent le cadre strict de ce manuscrit. La première est la caractérisation structurelle des FDDS irréductibles. Ceci serait un pas important vers la résolution du problème de factorisation d'un FDDS, c'est-à-dire la résolution de $XY = B$, avec X, Y devant être différents de B . En effet, nous pourrions nous baser sur cette dernière pour construire un algorithme permettant de résoudre ce problème sans passer par l'énumération de tous les FDDS possibles. La deuxième est l'extension des résultats aux graphes orientés étiquetés de façon déterministe, c'est-à-dire aux graphes dont les arcs portent des étiquettes avec la contrainte qu'un sommet n'a pas deux arcs sortants de même étiquette. Ce cadre permettrait d'introduire une notion d'asynchronie.

Dans les FDDS classiques, toutes les transitions s'effectuent simultanément à chaque pas de temps, ce qui correspond à un modèle synchrone. En ajoutant des étiquettes sur les arcs, nous pouvons modéliser un système où chaque transition est conditionnée à la réalisation d'un événement. C'est-à-dire que le système ne progresse que lorsque l'événement correspondant à l'étiquette de l'arc se produit. La contrainte qu'un sommet n'ait pas deux arcs sortants de même étiquette est nécessaire pour conserver le déterminisme; ainsi, à un événement donné, l'évolution du système reste

unique. Pour ces graphes, nous utiliserions comme multiplication le produit fort au lieu du produit direct tout en conservant une structure suffisamment simple pour espérer trouver des algorithmes efficaces.

La troisième est l'extension aux graphes orientés généraux, sans contrainte sur le degré sortant. Ceci nous permettrait de modéliser des problèmes où le graphe de dynamique serait non-déterministe ou probabiliste, par exemple des chaînes de Markov à espace d'états fini. Ces graphes sont par ailleurs plus proches de certaines applications concrètes. Ce passage nécessiterait forcément de trouver un substitut au test d'isomorphisme en temps linéaire dont bénéficient les graphes fonctionnels. De plus, le produit direct de deux graphes orientés généraux peut engendrer des composantes connexes de structure bien plus complexe que dans le cas fonctionnel, ce qui rend la reconstruction d'une solution à partir de ses composantes connexes considérablement plus difficile. Ces graphes offrent également une plus grande gamme d'opérations que simplement la somme et le produit. En effet, Lovász a introduit dans [34] une opération d'exponentiation sur les graphes orientés généraux. Cette opération ne s'applique pas dans notre semi-anneau, car G^H peut ne pas être fonctionnel même si G et H le sont.

Enfin, nous voudrions étendre nos résultats au cas où les systèmes seraient infinis, requérant de passer de la théorie de la complexité à celle de la calculabilité. Pour élargir nos résultats à ce cas, il nous faudrait étudier l'impact d'avoir des arbres d'unroll non-périodiques et pouvant admettre plusieurs chemins infinis. Il nous faudrait aussi discuter de la manière d'encoder de façon finie de tels systèmes.

Problème	Conditions	complexité
$P(X_1, \dots, X_n) = Q(X_1, \dots, X_n)$	P, Q polynômes quelconques	Indécidable
$P(X_1, \dots, X_n) = B$	P multivarié sur les FDDS, B FDDS	D NPC
	P multivarié sur les permutations, B permutations	E EnumP
	P non-linéaire multivarié sur les supports, B support	D NPC
	P linéaire multivarié sur les supports, B support	E DelayP
$P(X_1, \dots, X_n) \leq B$	P multivarié pseudo-dissocié sur les FDDS, B FDDS	E DelayP
$P(X_1, \dots, X_n) \geq B$	P multivarié sur les FDDS, B FDDS	R FP
$P(X) = B$	P univarié sur les FDDS, B FDDS	D NP
	P univarié sur les permutations, B permutations	
	P univarié sur les systèmes partiels, B système partiel	
	P univarié pseudo-injectif sur les FDDS, B FDDS	E DelayP
	P univarié pseudo-injectif sur les permutations encodées en binaire, B permutation encodée en binaire	R FP
	P univarié sur les unrolls, $\mathcal{U}(B)$ unroll	
	P univarié sur les forêts finies avec $\otimes$, $\mathbf{B}$ forêt finie	
	P univarié sur les forêts finies avec $\times$, $\mathbf{B}$ une forêt finie	

(suite)

Problème	Conditions	complexité
$AX = B$	A, B FDDS	D FPT
	A, B permutations	
Système d'équations avec côté constant	multivarié sur les FDDS	D NPC
	multivarié non-linéaire sur les supports	
	multivarié linéaire sur les supports	E DelayP

TABLEAU 6.1. – Tableau récapitulatif des résultats de complexité à l'issue de cette thèse. Les résultats nouveaux sont écrits en bleu. Seuls les résultats les plus généraux sont référencés. Par exemple, il n'y a pas d'entrée pour l'équation sur les FDDS $AX^k = B$, car ce cas est couvert par les polynômes pseudo-injectifs. Dans les résultats de complexité temporelle, nous avons préfixé par D les résultats concernant les problèmes de décision, par R ceux pour la résolution et par E ceux pour l'énumération. Rappelons que si le problème de résolution est dans FP, alors le problème de décision qui lui est associé est dans P. De plus, si un problème d'énumération est dans DelayP, alors le problème de résolution associé est dans FP.

Bibliographie

- [1] OEIS Foundation Inc. (2026). *Number of mappings (or mapping patterns) from n points to themselves; number of endofunctions*, Entry [A001372](#) in *The On-Line Encyclopedia of Integer Sequences* at <https://oeis.org/A001372> (cf. p. 38).
- [2] Alfred V. AHO, John E. HOPCROFT et Jeffrey D. ULLMAN. *The Design and Analysis of Computer Algorithms*. Addison-Wesley Series in Computer Science and Information Processing. 1974, p. 84-85. URL : <https://dl.acm.org/doi/10.5555/578775> (cf. p. 55).
- [3] Gilles BERNOT, Jean-Paul COMET, Adrien RICHARD, Madalena CHAVES, Jean-Luc GOUZÉ et Frédéric DAYAN. « Modeling and Analysis of Gene Regulatory Networks : A Multidisciplinary Endeavor ». In : *Modeling in Computational Biology and Biomedicine*. Sous la dir. de SPRINGER. 2012, p. 47-80. DOI : [10.1007/978-3-642-31208-3_2](#) (cf. p. 16).
- [4] Nicolas BOURBAKI. *Algèbre : Chapitres 1 à 3*. Éléments de Mathématique. 2007. DOI : [10.1007/978-3-540-33850-5](#) (cf. p. 29).
- [5] Florian BRIDOUX, Christophe CRESPELLE, Thi Ha Duong PHAN et Adrien RICHARD. « Dividing Permutations in the Semiring of Functional Digraphs ». In : *Cellular Automata and Discrete Complex Systems*. AUTOMATA 2024. 2024, p. 95-10. DOI : [10.1007/978-3-031-65887-7_6](#) (cf. p. 46, 47).
- [6] Florian BRIDOUX, Christophe CRESPELLE, Thi Ha Duong PHAN et Adrien RICHARD. *Dividing sums of cycles in the semiring of functional digraphs*. 2025. DOI : [10.48550/arXiv.2504.11943](#). URL : <https://arxiv.org/abs/2504.11943> (cf. p. 47).
- [7] Florian BRIDOUX et Antonio E. PORRECA. *Composing behaviours in the semiring of dynamical systems*. Talk given at the International Workshop on Boolean Networks. 2020. DOI : [10.5281/zenodo.3934396](#) (cf. p. 38, 39, 64).
- [8] Johan COUTURIER. *Méthodes algébriques et algorithmiques pour la décomposition de systèmes dynamiques*. Rapport de stage, Aix-Marseille Université. 2021 (cf. p. 138).
- [9] Samir DATTA, Nutan LIMAYE, Prajakta NIMBHORKAR, Raghav KULKARNI et Meena MAHAJAN. « Planar Graph Isomorphism is in Log-Space ». In : 24th Annual IEEE Conference on Computational Complexity 2009 (2009), p. 203-214. DOI : [10.1109/CCC.2009.16](#) (cf. p. 28).

- [10] Oscar DEFRAIN, Antonio E. PORRECA et Ekaterina TIMOFEEVA. « Polynomial-delay generation of functional digraphs up to isomorphism ». In : *Discrete Applied Mathematics* 357 (2024), p. 24-33. DOI : [10.1016/j.dam.2024.05.030](https://doi.org/10.1016/j.dam.2024.05.030). URL : <https://www.sciencedirect.com/science/article/pii/S0166218X24002348> (cf. p. 43-45).
- [11] Alberto DENNUNZIO, Valentina DORIGATTI, Enrico FORMENTI, Luca MANZONI et Antonio E. PORRECA. « Polynomial equations over finite, discrete-time dynamical systems ». In : *Cellular Automata, 13th International Conference on Cellular Automata for Research and Industry, ACRI 2018*. ACRI 2018. 2018, p. 298-306. DOI : [10.1007/978-3-319-99813-8_27](https://doi.org/10.1007/978-3-319-99813-8_27) (cf. p. 16, 40, 64).
- [12] Alberto DENNUNZIO, Enrico FORMENTI, Luca MANZONI et Antonio E. PORRECA. « Complexity of the dynamics of reaction systems ». In : *Information and Computation* 267 (2019), p. 96-109. DOI : [10.1016/j.ic.2019.03.006](https://doi.org/10.1016/j.ic.2019.03.006) (cf. p. 36, 37).
- [13] Alberto DENNUNZIO, Enrico FORMENTI, Luciano MARGARA, Valentin MONTMIRAIL et Sara RIVA. « Solving Equations on Discrete Dynamical Systems ». In : *Computational Intelligence Methods for Bioinformatics and Biostatistics*. CIBB 2019. 2020, p. 119-132. DOI : [10.1007/978-3-030-63061-4_12](https://doi.org/10.1007/978-3-030-63061-4_12) (cf. p. 34, 44, 115, 127).
- [14] Alberto DENNUNZIO, Enrico FORMENTI, Luciano MARGARA et Sara RIVA. « An algorithmic pipeline for solving equations over discrete dynamical systems modelling hypothesis on real phenomena ». In : *Journal of Computational Science* 66 (2023). DOI : [10.1016/j.jocs.2022.101932](https://doi.org/10.1016/j.jocs.2022.101932). URL : <https://www.sciencedirect.com/science/article/pii/S1877750322002915> (cf. p. 46, 115, 133, 154, 230).
- [15] Alberto DENNUNZIO, Enrico FORMENTI, Luciano MARGARA et Sara RIVA. « On solving basic equations over the semiring of functional digraphs ». In : *Discrete Mathematics & Theoretical Computer Science* (2025) : *Discrete Algorithms*. DOI : [10.46298/dmtcs.14590](https://doi.org/10.46298/dmtcs.14590) (cf. p. 46, 64).
- [16] François DORÉ, Enrico FORMENTI, Antonio E. PORRECA et Sara RIVA. « Decomposition and factorisation of transients in functional graphs ». In : *Theoretical Computer Science* 999 (2024). DOI : [10.1016/j.tcs.2024.114514](https://doi.org/10.1016/j.tcs.2024.114514) (cf. p. 50, 51, 54).
- [17] Valentina DORIGATTI. « Algorithms and Complexity of the Algebraic Analysis of Finite Discrete Dynamical Systems ». Mém. de mast. 2018 (cf. p. 43).
- [18] François DORÉ, Kevin PERROT, Antonio E. PORRECA, Sara RIVA et Marius ROLLAND. « Roots in the semiring of finite deterministic dynamical systems ». In : *Cellular Automata and Discrete Complex Systems*. AUTOMATA 2024. 2024, p. 120-132. DOI : [10.1007/978-3-031-65887-7_8](https://doi.org/10.1007/978-3-031-65887-7_8) (cf. p. 69).

- [19] Andrzej EHRENFEUCHT et Grzegorz ROZENBERG. « Reaction systems ». In : *Fundamenta Informaticae* 75 (2007), p. 263-280. URL : <http://iospress.metapress.com/content/b86t11hryvwq6910/> (cf. p. 16).
- [20] Nathan J. FINE et Herbert S. WILF. « Uniqueness theorems for periodic functions ». In : *Proceedings of the American Mathematical Society* 16.1 (1965), p. 109-114. DOI : [10.2307/2034009](https://doi.org/10.2307/2034009). URL : <https://www.ams.org/journals/proc/1965-016-01/S0002-9939-1965-0174934-9/> (cf. p. 69).
- [21] Jörg FLUM et Martin GROHE. *Parameterized Complexity Theory*. Texts in Theoretical Computer Science. An EATCS Series. 2006. DOI : [10.1007/3-540-29953-X](https://doi.org/10.1007/3-540-29953-X) (cf. p. 159).
- [22] Enrico FORMENTI, Jean-Charles RÉGIN et Sara RIVA. « MDDs Boost Equation Solving on Discrete Dynamical Systems ». In : *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*. CPAIOR 2021. 2021, p. 196-213. DOI : [10.1007/978-3-030-78230-6_13](https://doi.org/10.1007/978-3-030-78230-6_13) (cf. p. 46).
- [23] Maximilien GADOULEAU et Soren RIIS. « Graph-Theoretical Constructions for Graph Entropy and Network Coding Based Communications ». In : *IEEE Transactions on Information Theory* 57.10 (2011), p. 6703-6717. DOI : <https://doi.org/10.1109/TIT.2011.2155618> (cf. p. 16).
- [24] Michael R. GAREY et David S. JOHNSON. *Computers and Intractability : A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. ISBN : 978-0-7167-1045-5 (cf. p. 38).
- [25] Carlos GERSHENSON. « Introduction to Random Boolean Networks ». In : *Workshop and Tutorial Proceedings, Ninth International Conference on the Simulation and Synthesis of Living Systems (ALife IX)*. 2004, p. 160-173 (cf. p. 16).
- [26] Eric GOLÈS et Servet MARTÍNEZ. *Neural and Automata Networks : Dynamical Behavior and Applications*. T. 58. Mathematics and Its Applications. Springer, 1990. DOI : [10.1007/978-94-009-0529-0](https://doi.org/10.1007/978-94-009-0529-0) (cf. p. 16).
- [27] Richard HAMMACK, Wilfried IMRICH et Sandi KLAVŽAR. *Handbook of Product Graphs*. 2^e éd. CRC Press, 2011. DOI : [10.1201/b10959](https://doi.org/10.1201/b10959) (cf. p. 25, 31, 40, 41).
- [28] John E HOPCROFT et Jin-Kue WONG. « Linear time algorithm for isomorphism of planar graphs (preliminary report) ». In : *Proceedings of the sixth annual ACM symposium on Theory of computing*. STOC '74. 1974, p. 172-184. DOI : [10.1145/800119.803896](https://doi.org/10.1145/800119.803896) (cf. p. 28, 75, 83).
- [29] David S. JOHNSON, Mihalis YANNAKAKIS et Christos H. PAPADIMITRIOU. « On generating all maximal independent sets ». In : *Information Processing Letters* 27.3 (1988), p. 119-123. DOI : [10.1016/0020-0190\(88\)90065-8](https://doi.org/10.1016/0020-0190(88)90065-8). URL : <https://www.sciencedirect.com/science/article/pii/0020019088900658> (cf. p. 44).

- [30] Thor JOHNSON, Neil ROBERTSON, P.D. SEYMOUR et Robin THOMAS. « Directed Tree-Width ». In : *Journal of Combinatorial Theory, Series B* 82 (2001), p. 138-154. DOI : [10.1006/jctb.2000.2031](https://doi.org/10.1006/jctb.2000.2031). URL : <https://www.sciencedirect.com/science/article/pii/S0095895600920318> (cf. p. 138).
- [31] J. KOBLER, U. SCHÖNING et J. TORAN. *The Graph Isomorphism Problem : Its Structural Complexity*. Progress in Theoretical Computer Science. Springer, 2012. URL : <https://books.google.cl/books?id=vhvnBwAAQBAJ> (cf. p. 28).
- [32] H. W. LENSTRA. « Integer Programming with a Fixed Number of Variables ». In : *Mathematics of Operations Research* 8 (1983), p. 538-548. URL : [jstor.org/stable/3689168](https://www.jstor.org/stable/3689168) (cf. p. 159).
- [33] Niels LINDEMANN. « A logspace algorithm for tree canonization (extended abstract) ». In : *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*. STOC '92. 1992, p. 574-582. DOI : [10.1145/129712.129768](https://doi.org/10.1145/129712.129768). URL : <https://acm.org> (cf. p. 55, 101, 230).
- [34] László LOVÁSZ. « Operations with structures ». In : *Acta Mathematica Hungarica* 18.3-4 (1967), p. 321-328. DOI : [10.1007/BF02020961](https://doi.org/10.1007/BF02020961). URL : <https://www.cs.elte.hu/~lovasz/scans/opstruct.pdf> (cf. p. 47, 54, 58, 233).
- [35] László Miklós LOVÁSZ. « On the cancellation law among finite relational structures ». In : *Periodica Mathematica Hungarica* (1971). DOI : [10.1007/BF02029172](https://doi.org/10.1007/BF02029172). URL : <https://link.springer.com/article/10.1007/BF02029172> (cf. p. 47, 54, 58).
- [36] John G. MARICA et Steven J. BRYANT. « Unary Algebras ». In : *Pacific Journal of Mathematics* 10.4 (1960), p. 1347-1359. DOI : [10.2140/pjm.1960.10.1347](https://doi.org/10.2140/pjm.1960.10.1347). URL : <https://msp.org/pjm/1960/10-4/pjm-v10-n4-p28-s.pdf> (cf. p. 47, 48, 51, 52, 54, 57, 58).
- [37] Yuri MATIYASEVICH. *Hilbert's 10th Problem (Foundations of Computing)*. The MIT Press, 1993, chapter 5. URL : <https://mitpress.mit.edu/9780262132954/hilberts-10th-problem/> (cf. p. 36).
- [38] M. H. MCANDREW. « On the product of directed graphs ». In : *Proceedings of the American Mathematical Society* 14.4 (1963), p. 600-606. DOI : [10.2307/2034282](https://doi.org/10.2307/2034282) (cf. p. 41, 42).
- [39] Aurélien NALDI, Denis THIEFFRY et Claudine CHAOUIYA. « Decision Diagrams for the Representation and Analysis of Logical Models of Genetic Networks ». In : *Computational Methods in Systems Biology*. 2007, p. 233-247. DOI : [10.1007/978-3-540-75140-3_16](https://doi.org/10.1007/978-3-540-75140-3_16) (cf. p. 46, 115).
- [40] Émile NAQUIN et Maximilien GADOULEAU. « Factorisation in the semiring of finite dynamical systems ». In : *Theoretical Computer Science* 998 (2024), p. 114509. DOI : [10.1016/j.tcs.2024.114509](https://doi.org/10.1016/j.tcs.2024.114509). URL : <https://www.sciencedirect.com/science/article/pii/S0304397524001245> (cf. p. 22, 47, 48, 50-52, 54, 57-60, 64, 93, 111-113).

- [41] Pacôme PERROTIN. « Simulation entre modèles de calcul naturel et modularité des réseaux d'automate ». Thèse de doct. Aix-Marseille Université, 2021. URL : <https://theses.hal.science/tel-03188307> (cf. p. 16).
- [42] Thi Ha Duong PHAN, Christophe CRESPELLE et Adrien RICHARD. *Tensor Product of Functional Graphs : Algorithms for Factorization and Polynomial Root-Finding Restricted to the Cycle Part*. 2025. DOI : [10.2139/ssrn.5316210](https://doi.org/10.2139/ssrn.5316210). URL : <https://ssrn.com/abstract=5316210> (cf. p. 47, 64).
- [43] Miroslav PLOŠČICA et Miron ZELINA. « Cancellation among finite unary algebras ». In : *Discrete Mathematics* 159.1 (1996), p. 191-198. DOI : [10.1016/0012-365X\(95\)00078-B](https://doi.org/10.1016/0012-365X(95)00078-B). URL : <https://www.sciencedirect.com/science/article/pii/0012365X9500078B> (cf. p. 59, 111).
- [44] Antonio E. PORRECA et Marius ROLLAND. « Solving “pseudo-injective” polynomial equations over finite dynamical systems ». In : *Cellular Automata and Discrete Complex Systems*. AUTOMATA 2025. 2026, p. 171-187. DOI : [10.1007/978-3-032-01570-9_12](https://doi.org/10.1007/978-3-032-01570-9_12) (cf. p. 123).
- [45] Adrien RICHARD. « There is no prime functional digraph : Seifert's proof revisited ». In : *Theoretical Computer Science* 1072 (2026), p. 115879. DOI : [10.1016/j.tcs.2026.115879](https://doi.org/10.1016/j.tcs.2026.115879). URL : <https://www.sciencedirect.com/science/article/pii/S0304397526001386> (cf. p. 43, 113).
- [46] Sara RIVA. « Factorisation de Systèmes Dynamiques Discrets ». Thèse de doct. Université Côte d'Azur et Università degli Studi di Milano-Bicocca, 2022 (cf. p. 46, 64, 115, 127, 133, 154, 230).
- [47] Ralph SEIFERT. « On Prime Binary Relational Structures ». In : *Fundamenta Mathematicae* 70 (1971), p. 187-203. DOI : [10.4064/fm-70-2-187-203](https://doi.org/10.4064/fm-70-2-187-203) (cf. p. 43, 113).
- [48] Yann STROZECKI. « Enumeration Complexity ». In : *Bulletin of EATCS* 1 (2019). URL : <http://bulletin.eatcs.org/index.php/beatcs/article/view/596> (cf. p. 44).
- [49] René THOMAS. « Boolean formalization of genetic control circuits ». In : *Journal of Theoretical Biology* 42.3 (1973), p. 563-585 (cf. p. 16).
- [50] René THOMAS et Richard D'ARI. *Biological Feedback*. CRC Press, 1990. DOI : [10.1201/9781003574842](https://doi.org/10.1201/9781003574842) (cf. p. 16).
- [51] Pravin M. VAIDYA. « An algorithm for linear programming which requires $O(((m+n)n^2 + (m+n)^{1.5}n)L)$ arithmetic operations ». In : *Mathematical Programming* 47 (1990), p. 175-201. DOI : [10.1007/BF01580859](https://doi.org/10.1007/BF01580859) (cf. p. 160).
- [52] Paul WEICHSEL. « The Kronecker product of graphs ». In : *Proceedings of the American Mathematical Society* 13.1 (1962), p. 47-52. DOI : [10.2307/2033769](https://doi.org/10.2307/2033769) (cf. p. 41).